

DS Lecture 1

Wed, 8/11 12:59PM 1:27:47

SUMMARY KEYWORDS

big, case, squared, algorithm, equal, holds, rule, questions, true, function, linear, thirds, constant, fibonacci numbers, bound, bit, fifths, growth rates, loop, definition

01:24

With your feet in the air in your head on the ground drivers drink and spin is your head will collapse and there's nothing in it annual ask yourself my mind where is my mind? Is my mind

02:17

no

02:29

I was swimming in Ruby animals were hiding behind the rock except what they told me he switched Try not to talk to me

02:54

Where is my mind? Where is my mind?

03:02

Is my mind

03:07

DS Lecture 1 Page 1 of 21 Transcribed by <https://otter.ai>

where is my mind is oh in the wall try this trick and spirit.

04:04

Your hand will collapse. Yeah, there's nothing in his head you'll ask your shoe. Where's my mind? Where is my mind? is mine. So see it sue me. Alright, let's get let's get started. Oops. All right.

05:01

Why was the homework deadline extended? Probably because, wow. Okay, first of all, for the sake of the other 300, people probably don't ask those questions. The actual reason is because for the written homework, a lot of what's being asked of you is actually today's lecture. So I wanted to give you a little bit more time to work on that. Does that make sense? All right. Cool. So, um, just a few notes. Um, and I jotted some of that down here. I did push the deadlines to the third. I actually, when we initially announced them, I said I was likely going to do that. So there's nothing shocking there. And then you have tons and tons of ta office hours. And I understand you guys have been making use of them. So that's good. I will have for this week only I have kind of a very weird set of obligations on the next few days. So for this week, only, I'm going to hold them on Friday, from two to four. And then because I have something already scheduled for four o'clock on Saturday, I can hold another set on Saturday from 230 to 330. Okay, that's just this week. Starting next week, my regular hours will begin my regular hours. And all this will be on the calendar will be on Mondays from 130 to 330. And on Wednesdays, from three to five. And then I'm planning on setting it up. So at some point on Fridays, I should have individual meeting slots that can be booked. Although to be honest, that's a little bit more for my advisees than anything else. But they'll be there. Okay. So is everybody clear on when I'm my availability is this week? And then going forward? Yes. Everybody's clear on the homework deadlines. Yes. All right. So I think we are pretty much all caught up. And good, we should get started. So last class, we were doing recursion, we did a few examples of recursion. And what I wanted to do was I wanted to briefly do something out of chapter one that involves some proofs that might occur during the course of the class. And part of the reason why I'm doing that now is that the main variety, the induction proofs, map really well to recursion. So let's do that first. And then we'll get into chapter two and bego.

And we'll make a real big dent in that today. So there are essentially three kinds of proofs that will show up over the course of the term two very briefly, and then one will show up much more frequently, every now and then you'll get super lucky and be able to prove something by counter example. So if you're being asked to prove that such and such a thing is not true, across some range, all you need to do is find one example, where it

DS Lecture 1 Page 2 of 21 Transcribed by <https://otter.ai>

doesn't hold true and you're good proof by counter example. Another slightly more complicated type of proof that will show up once later on is what's known as a proof by contradiction. And in a proof by contradiction, what ends up happening is that you assume, basically, that the opposite of what you're trying to prove is true. And then you show that it follows from that, that your initial assumptions are directly contradicted, basically showing that that can't be possible, given your initial contract, Mitchell, assumptions. But the variety that we're going to use most is what's known as an induction proof.

09:33

And then induction proof comes in. Well, induction proof takes three stages. So you have a property that you're calling to try to prove from some starting point on up. So prove something is true from say, i greater than zero or something to that effect. So the first thing you do is you show your base case. That is you show that property trivially holds true for the first value of that range. Right, that's usually just subbing a number in and just showing something holds true. The second step is our inductive hypothesis. And in this step, what we're doing is we are assuming that we are assuming that the property we're trying to hold holds true, we're gonna do strong induction here holds true from the base case up through some arbitrary K . So for trying to prove that the property holds, for i zero on up, what their inductive hypothesis will be is that we assumed that it holds true from zero to some arbitrary K . And then once we've assumed that, then we just need to prove that also holds for k plus one. So we assumed that it holds true up through K . And then we show that if that assumption holds true, then it must also follow that it holds true for k plus one. And that's all we need to do. Because we've shown our base case, we know that's true. We've shown that for some arbitrary K if that happens to be true, then it must also be true for k plus one. So if it holds for our base case, then by this it must hold for one pass the base case. And if it holds for one, pass the base case, then by this and holds one further, and so on. So a kind of accordions out. Now mathematicians tend to hate that kind of logic, but you know what I have to say to them, too bad, this is what we're doing. Okay. So let's do an example. And this example is going to be a little bit more mathematical in nature than normal, we'll be doing it with equations. Normally, when we're doing it with our data structures, a lot of it can be done via diagrams, sort of more structurally oriented. But in this case, we're going to do something that's a little bit more mathematical. And so you all should remember the Fibonacci numbers from our recursion examples last week, correct. And you remember that, we said that the zero Fibonacci number was one, the first Fibonacci number was one. And then all subsequent Fibonacci numbers greater than one is equal to some of the previous two. That was our recursive definition of Fibonacci numbers. So the property that we're going to try to prove is that the i th Fibonacci number, so $f_{sub i}$ is going to be less than five thirds to the i th power for i

DS Lecture 1 Page 3 of 21 Transcribed by <https://otter.ai>

greater than or equal to one. That's what we're going to try to prove. And we're going to try to do that using an induction proof. Now, you'll notice that $f_{sub 0}$, we know is supposed to be one, correct. But $f_{sub 0}$, this doesn't hold, right? Because if $f_{sub 0}$ is known to be one, five thirds to the zero is one that's that that doesn't hold. But notice that we're trying to prove it for i is greater than or equal to one. Somebody asked me why five thirds, that just so we're proving that's going to be the case.

14:23

that's as close as we can get. So that's just sort of an intrinsic property of this. How did we find that out? I'm not going into it. Alright. Um, so given this, we want to prove that it's true. All right. So um, Oh, actually, you know what? That brings up a point. I'm Hannah, how did you want to deal with student questions? Did you want to keep them all public? Or did you want to keep your private? Yeah,

14:56

I think just in general, if you're comfortable with us, Then publicly that's best because a lot of you guys have the same questions. But if it's something a bit more personal, yeah, feel free to pm a TA,

15:09

right? So I brought that up because that five thirds thing that I was answering with somebody who direct messaged me. So unless it First of all, don't direct message me, we have three teams all listed as co L. co hosts right now. But if you ask a question, you can ask it in the chat sort of publicly would probably be best to cut down on duplicate questions. Is that okay? folks? Good. All right. Um, okay. So let's go and try to prove this. So step one here, is we want to show that our base case holds. Now, one thing in this case, because the Fibonacci numbers actually look two steps back, I'm going to show that it holds both for is one and is two, just because we're going to be looking two steps back each time. But normally, in most cases, we only need to look one step back each time, so we only usually need to show the smallest base case. So step one, show your base case. So we know that f of one is equal to one. And so five thirds raised to the one is five thirds. Well, one is less than five thirds. So we've shown that that holds true. And then we know that f of two was the prop the sum of the previous two. So f of two is just going to be equal to two and five thirds to the second is what 25 over nine. And so in fact, 18 over nine would be two. So 25 over nine is in fact greater than two. So two is less than 25 over nine. And we've shown sort of two base cases, which means that if we're going to look back two

DS Lecture 1 Page 4 of 21 Transcribed by <https://otter.ai>

steps, later on, we're good. So the bottom line is yes, it's the smallest input you can have. Okay. All right. So that's step one. Step two, is going to be assume that this holds true. Well, actually, I'll just write it here. So assume that this holds true for let's say, for I from zero to an arbitrary K . So we're going to just assume that this holds true up through K . So now what we need to do, the only thing left for us to do is to actually prove that it also holds true. Yep, I mean, one to get that in also holds true for k plus one. Okay, so let's do that. So what does it mean to hold true for k plus one? Now, that means we have to show that F sub k plus one is less than five thirds. This down slightly to the k plus one power. All right, you have to show that that's true. Now, we know just from the definition of Fibonacci numbers, that F sub k plus one is equal to f of k plus f sub k minus one. All right. Now, k and k minus one are ones where we've already assumed that the property we're going to try to prove holds true. All right. So we are assuming that F sub k is less than five thirds to the K . And we're assuming that F sub k minus one is less than five thirds to the k minus one. so if we substitute in those two things, I'm Hannah, I can't if you can try to do it, please do. I'm sorry. So you can substitute in those values. So we can say five thirds to the K

20:01

plus five thirds to the k minus one. Now, if I add these two things up, five thirds to the K is going to be assumed to be strictly greater than this, and five thirds, the k minus one is strictly greater than that. So if the sum of these two things equals F two, f sub k plus one, then the sum of these two things that are strictly greater than their originals, this has got to be greater than f sub k plus one. But now what we need to do is we need to actually write this in terms of five thirds to the k plus one to actually go ahead and show what we're trying to show. So let's do that. So if I want to convert this term to be something times five thirds to the k plus one, if I write five thirds to the k plus one, what would I have to do to that to get it equal to five thirds to the K , anybody? I'd have to divide it by five

thirds, or put it another way, multiply it by three fifths, agreed. So this can be rewritten as three fifths times five thirds to the k plus one. We can do something similar here, we want to rewrite that as some coefficient times five thirds to the k plus one, we have to divide five thirds the k plus one by $5/3$ squared, to get back to five thirds to the k minus one. So that's like multiplying by three fifths squared, right. So now, let's consolidate these a little bit. This is still less than flat f sub k plus one. So three fifths is 9/25, and I need to get that to be kind of the same, the same denominator, so this is 9/25. Sorry, this is 9/25 fifths. And if I want to get this What do you mean, our cue? Where have I written our cue anywhere here? What notation? I'm not I'm sorry, Hannah. Maybe if you can work, one of the teams can figure that out. Thanks. All right. So if this is 9/25, this is what this is 15/25. Right. So now we have 1520 fifths times five thirds to the k plus one plus 920 fifths times five thirds to the k plus one. Let's just combine them. And that ends up being 2420 fifths times five thirds to the k plus one. Now, all of these terms were all equal. So this is still less than f of k plus one. And so we've gotten it down to f sub k plus one, the k plus wants to bonacci number is now shown to be less than 2420 fifths 2420 fifths of five thirds to the k plus one. All right. three fifths is 15/25. I'm just multiplying by five over five. Okay. So assuming our base case, sorry, assuming our inductive hypothesis, we've actually shown that F sub k plus one is less than something that's actually smaller than five thirds to the k plus one. When all we needed to show that it was less than five thirds to the k plus one. Does this show this? Yes, yes. So now we've shown that if it holds true up through k , then it must also hold true through k plus one. And then we wind back to our base cases, and that it sort of expands out. Okay. All right. So that's an example of an induction proof. Well, it's recorded, so you get that opportunity. Um, no, no, this is the only mathematical example. I'm going to do it. Absolutely the only one.

25:02

Because if I do another one, it's just going to make it worse. This is the simplest one I can come up with. The examples that we're going to do of this at a later time, are more diagram oriented, are much easier to follow. And I can't do until we've learned binary trees. So I just wanted to make sure that you've seen it that you've seen the basic structure of it, so that you're not going nuts when we do it later. And it's somewhat simpler way. Okay. All right. So that wraps up what I wanted to do from chapter one. So now we need to move into algo analysis in chapter two. And so we're talking about algorithm analysis now.

26:09

And we've already seen quite a bit of this. But we want to formalize these things a little bit.

26:22

So just to repeat, what we're trying to do here is we're trying to find a way to take our algorithms and sort of compare them to each other in a way that in a way that allows us to see which one is faster and which one is slower. And under what circumstances. Okay, I'm going to shut the video off of the person who's having a very animated phone conversation. Hold on. There we go. Sorry. She didn't notice it's all good. Any. So we'll start with a story, a story, which is about 95%. Not true, but I think it can sort of help us illustrate the situation a little bit. Let's start with the true part of this story. So the true part of this story is, um, once upon a time, we say we, once upon a time, I was a roboticist. And I had a mobile robot when I was writing algorithms for this mobile robot to go out and scan sort of the campus as a sort of the first thing I was doing. All right. And so I had an

DS Lecture 1 Page 6 of 21 Transcribed by <https://otter.ai>

algorithm that took in an initial map calculated locations to take scans from, and then the robot would go and do it. And so let's say and this is where things start to get a little bit not sure. Let's say I say my algorithm, we're going to hold on a second, let me finish what I'm talking about, before we move on to any questions. Let's say that my algorithm is, you

know, was able to do it in, say, an hour. And then there was another, there was another person that was doing sort of similar research at a university in the Netherlands, alright. And that's also true. And this is where this story ceases to have any connection to reality. But let's just keep going with it. So let's say that that fellow in the Netherlands, had an algorithm to have his mobile robot go out onto his campus, I forget the name of the university he was at. And he could scan his campus with his robot, instead of in an hour could do it in 45 minutes. So he says, Oh, my algorithm is better. Because it finished the scan faster. Now, I have a question. Is that true? Was his algorithm better in this completely fictional story? 95% fictional story? We did no. Good, we don't actually know. Um, and so several people pointed out. Well, could it have the same accuracy? Okay, that's actually a good point. Well, what would define the accuracy or to a certain extent, that would have to do with the scanning equipment and the robot itself, and how big that campus was in comparison to our campus? Better means faster, but what's faster? All right. So too many issues here. So let's say we try to subtract out some of the variables. Let's say we're running the algorithm on both on the same campus. Is that good enough? We set his robot loose in my robot loose on the same campus. It's trying to build a 3d model of the entire campus. Nothing specific. Oh, it's not good enough, what why not? What's the issue? You should test it with the same robots, okay? Raw runtimes between the OS hardware and interpreter, we have no idea what the let's not worry about even the quality of it, let's just talk about the sheer computational power. We don't even know what kind of computer they're running. We don't know what kind of hardware they're running versus mine. It just surfaces that's not really relevant. Um, so it comes down to computational power, right? We don't know what his computer and what's in mind. So let's say,

31:22

ah, comes about, maybe some campuses are better than others. For certain algorithms, it's a very good point too. So maybe we should just run the same algorithm, the two algorithms on the same system. But even that starts to become a little problematic. Because we don't know what sort of real world conditions that hits in a moment that might cause the measurements to be a little bit off, if we're physically moving through the campus, maybe on one run, a group of students walks in front of the robot and another run, it doesn't. So really, what we want to do is we want to think just about the underlying program. And we want to abstract it away from anything hardware related. And we so we give it the same inputs. And then we're essentially counting the number of operations. So

DS Lecture 1 Page 7 of 21 Transcribed by <https://otter.ai>

the number of individual operations like pluses, minuses, times divides, that sort of thing, things that take a single clock tick to do, we count the number of operations that each algorithm would need to perform, so that we get sort of an apples to apples comparison. But as folks have pointed out, that can vary on the input. So we really think about it not so much in the specific construction of the input initially, we think about it in terms of the size of the input, I don't know how many walls there are that we have to worry about scanning and things like that, um, we're not going to worry about memory usage, even for the moment, although that could be a consideration. But right now I'm thinking tirely, about runtime. So we're going to count the number of operations it takes, but we're going to count them as a function of the size of the input. And that really abstracts away as much as possible. And then we examine that function, which is a function of the size of the input, as the size of the input tends towards infinity. So this ends up being an asymptotic analysis, as the ends get very large, how does it tend? And so we can look at the relative growth rates of the functions of the run times in terms of the input size, and things that grow faster functions that grow faster means their run times, get longer very quickly than others. All right. And so it's that kind of comparison that we're ultimately going to do. But one of the things that we have all along said is we've said that we don't want to worry so

much about the way I'm trying to put this, we wouldn't want to worry so much about things like constant factors 10 times in 100 times and 1000 times n versus n squared 10 1000. And those have the same growth rates as n gets very large, they're just linear, even though they have this very different constant factor. So as n gets very, very large, those constant factors don't matter. If we have algorithms that take, say n squared plus n time, as n gets very large, the n squared becomes the dominant term. And the end doesn't really matter. So we like to be able to drop the lower order terms, which means we need some sort of mathematical formalization that allows us to drop these constant factors in lower order terms, which leads us to the formal definition of big O. So if we have a function t of n , now I should say big O, can be is something that you apply to a function, it doesn't really matter what it's a function of just something that you could apply to any function, when we talk about,

35:59

eventually, relative growth rates of these things. And so t of n could be any function of N . And we could apply Big O to it. But for the purposes of this conversation, t of n would correspond to the exact function of n , where n is the size of the input, that tells me how many operations it will take for this thing to run, which is a surrogate for the run time, a function that describes the run time t of n , also, if we were really interested, could have been a function of the input size, that tells us how much extra memory space is used. And we can do the same analysis on memory. But for the purposes of this discussion, we're going to assume t of n is the exact function that tells me how many operations are

DS Lecture 1 Page 8 of 21 Transcribed by <https://otter.ai>

performed on an input of size n . And so we could say, t of n equals big O of some other function of n , we'll say f of n . So t of n is equal to big O of f of n , if there are positive constant or I should say positive constants. See, and and not such that

37:34

t of n

37:37

is less than c times f of n when n is greater than or equal to n_0 . So for all values of n above some, sorry, you're right, it is less than or equal. Sorry, point.

38:06

So, t of n is less than or equal to c of f of n , I just gave the definition for a little while, I meant to give the definition for big O. So, t of n is less than or equal to c of f of n , when n is greater than or equal to some initial and not, so, there could be some very small ends where this relationship does not hold. But once you hit some threshold and not at that point, um, this holds. And so this constant C is sort of what allows us to drop the constant factors, because if t of n is like $35n$, I can make F of N just n and that C could be the standard in for the 35. So, if I say t of n is equal to big O of f of n , f of n provides what kind of bound on T on the actual time cost. This is an upper bound. bound is equal to n . No, I haven't said that yet. But you are going to say it. No, I was saying it's F of N . Right now, it's not n yet. Um, folks, just to reiterate what Hannah has said, I need you to ask your questions publicly. If there was a way for me to shut off private DMS, I would but I can't. You got to work with us on this. Okay. Just so that we make sure that duplicate questions are not being repeatedly asked. Alright, so we have this upper bound So if I said t of n is equal to big O of n , then n , subject to some constant C is an upper bound on the time cost above some initial threshold and not subject to some constant factor t of n will never go above n . If I say t of n is equal to big O of n squared, same thing, t of n will never go above n squared times some constant factor once we're beyond some initial and not. So big O analyses of algorithms provide an upper bound on the cost of that algorithm, subject to a constant factor and above some initial starting point. So linear search, we said was a big O of n algorithm. It's bounded by n . And so the upper bound does not mean worst case or best case, we'll talk about how to deal with worst case and best case later, but you should

not confuse them. It just means that the function will never go above it. So linear search, we know goes through each element up, this is me goes through each item once. Alright, by this definition, could I have legally said that the time cost of linear search is big O of two

DS Lecture 1 Page 9 of 21 Transcribed by <https://otter.ai>

to the n ? Could I have legally said that? I actually could have legally said that. Because that absolutely is an upper bound it, it will never go above that. better question. If I asked you what the big O cost of linear search was on your midterm, and you selected to the end, when I give you any credit for that? God no. So when I, when I ask this question that question, too, typically, I would like for you to get as close as possible as you can. All right, you're not really you're not supposed to leave any breathing room in there. All right. So get it as close as you can. Now, before I move on to the next definition associated with this, I want to look I want to address the issue of worst case, best case, an average case, alright. big O is a property that you can find for any function. So what we're going to do, this is the upper bound. Good, Miranda. Cool. All right.

43:20

Please, somebody in the in the chat, please. Well, we don't know what C is C is arbitrary. So I'll let one of the the TAs answer that in the chat, I need to move forward. So we, we need to talk about how this is something that we could do. In the worst case, in the best case, and in the average case, I could ask you to do a big O analysis, in the worst case, best case or average case. So when I asked you for the big O cost of an algorithm and don't specify, I'm really mean, what's the worst case cost? Right. So, try to construct in your mind a series of inputs that would be sort of in the worst possible way. And then give me an analysis as we go from there, but I could ask you assume the best case right. And so assume that you know, independent of size, things are ordered in the best possible circumstances, and you can still do a big O analysis in the best case. So if I tell you, m event is going to be the cost of running this algorithm on an arbitrary size input of n when that n is organized in the best possible way, you can give me a big O analysis, an upper bound in that best case. So I can ask you to do a big O analysis in the best, worst and average cases, they're sort of separate from this. So when we talked about linear search, we actually said, it's big O of n in the worst case, because in the worst case, no matter what the size, either the value isn't there, or it's the last element. So you still have to go through them all the time. Or we could have talked about linear search in the best case, which means no matter how big it is, it's always the first item. So that's big O of one, in the best case, was an N doesn't matter. Okay. So we can ask you to do an upper bound on the best case cost an upper bound on the worst case cost, and then upper bound on the average case cost. All right. So there's sort of two separate things. So that's big O. And that's really the type of analysis we will be spending the vast majority, if not essentially all of our time on. But I still want to bring up a couple of other definitions. All right. Um, so there is another one. Which says t of n . Give me a second hand. We're, we're literally writing this down. Where we say t of n is said to be equal to big omega, that was sort of the ugliest omega I've ever seen. Well, no, I could make an uglier omega, if I wanted to, but that's a pretty ugly omega, but that's supposed to be an omega. Alright. So is equal to

DS Lecture 1 Page 10 of 21 Transcribed by <https://otter.ai>

big omega of some other function, let's say g of n . If there exist positive constants. See, and and not such that t of n is greater than or equal to c times g of n , for n greater than or equal to n_0 . So with that definition, what does big omega provide for us on a function t .

48:21

So it's not the best case, you can do a big O analysis. On the best case, you can do a big omega analysis on the best case, it provides a lower bound on the function. So subject to some constant factor, and above some initial and not the function t of n will never be below this g of n that's inside of the big omega. So it provides a floor for the whole thing will never do better than that. In the particular type of case. Best case, worst case average

case that we happen to be analyzing. lower bounds are notoriously hard to prove. All right? lower bounds are. There's like a whole advanced algorithms class at the 6000 level that you can take, which is all about proving lower bounds. All right. And so the only lower bound that I What do you mean between N and N , not what you mean between n and n ? Do I inadvertently write lowercase n . So n knots which I happen to make lowercase is just the starting point for the range of ends that this is true for. Okay? So it'll never go below this. So One scenario where we actually do talk a little bit in this class about lower bounds actually comes in with general purpose sorting algorithms. Okay. So in general purpose sorting algorithms, there is actually a lower bound that you can never find a general purpose sorting algorithm that will do better than n , sorry, then $n \log n$. So for its worst case cost, okay. So that's an example of a lower bound how to prove it is sort of beyond the scope of this. But that's what lower bounds do, it tells us, we're not going to be able to get below a certain point. Now there's one more, or actually two more in the text, such that st is just me abbreviating such that. The textbook actually has a little o as well, but we're not going to worry about little Oh , the last one I want to worry about is what's known as Big theta. So t of n is equal to big theta of some function h of n

51:22

if and only if t of n

51:34

is equal to big O of h of n . And t of n is equal to big ω . Okay. So it has to be both big O of h of n and big ω of h of n meaning h of n . And it could be subject to different constants. That's fair. But it means that h of n is both a lower bound and an upper bound on the cost. And so this kind of bound is often referred to as a tight bound.

DS Lecture 1 Page 11 of 21 Transcribed by <https://otter.ai>

52:36

Sorry, both my wife and I have like these terrible head colds right now. Which is one of these things where, you know, you have a terrible head cold in the middle of a global pandemic, you're you kind of get a little paranoid. Truth is we haven't been in contact with another human being in at least a week. So Probably not, but sorry, I feel a little sick today.

53:12

Yeah, I feel like a lot of people have colds. It's Oh, thank you. I don't know everybody feel better. who happens to feel sick right now? Okay, so this big theta actually. Yeah, it really is paranoid. paranoia inducing. I mean, I'm, I am an anxious paranoid person, like, normally. So for the last nine months, it's sort of been amplified. And I've so I've gone from basically managing my anxiety very well to not Anyway, moving on, because I need to tell that to 200 Kwazii strangers moving on. Well, hopefully, it makes you feel a little better if you're in the same boat. And it's like, even for people twice your age. It's the same graph. Um, okay, so big fate is a tight bound. It provides both an upper and a lower bound. All right. And so that means, what I was saying before, you know, if I asked you for bingo, can you legitimately say linear searches to the end? Well, technically, by the definition?

54:40

Yes. But if I say it's big fate of and you apps, sorry, big theta to the end, you absolutely can't do that. That takes that away. It has to be a very, very tight bound because it also has to serve as a lower bound in that case, as well. And so one of the criticisms and I've mentioned this before That are often levelled at data structures classes is that we often use big O , when we really mean big theta. And there's That's true. But the reason why we do it is because big theta is so hard to actually prove, because it involves proving the lower bound as well. So we kind of hand wave it a bit and just use ω , for the sake of ease of showing that it's true. But a lot of these costs that we're going to talk about, even though I quote them for you, as they go, are in fact, tight bounds, they are in fact being phased about, right. So I felt like it was important that we go through those definitions a

little bit, just so you have a sense for them. Now, I will I mean, probably, but it's not always going to be possible to figure out what it is. Um, so those are the basic definitions. And I expect you to know what those definitions are. Like, I could ask you questions on them. But we will mostly be doing our analyses in terms of Big-O. Now, I want to make a quick comment in that. As I said, we're doing these kinds of analysis as an asymptotic analysis. So you can sort of think of that. If you plot in, by sort of time. If you have something that's

DS Lecture 1 Page 12 of 21 Transcribed by <https://otter.ai>

a constant algorithm, the amount of time would be sort of flat. as n increases, if you have something that is linear time, it's going to go up as a straight line, let's pretend that that line was actually straight. And you'll notice that in this particular case, the linear time algorithm, because of the way I drew it was actually a little bit a little bit faster for some very small ends before it crossed the constant time one, it's that n naught we're talking about there. And as we do our asymptotic analysis, you'll see it gets bigger, eventually, it drowns it out, you have something that's n squared, it's going to look like a parabola. You have something that's n cubed, it's going to be still sort of parabolic, but even steeper, you have something that's like two to the end. And it practically shoots up. Almost like a straight line. Go ahead and mute you for that one. Although that sounded kind of fun.

58:05

So Bo, did you have a question? When you unmuted you can unmute now and ask are you just trying to do like some fun feedback? Hello.

58:28

Yeah, I meant that one to be to the end. Okay. So yeah, that was that was an interesting, that was an interesting sound. Alright. And then we can have things that are like log of n show.

58:49

Yo, that did not improve things at all. I don't know what was going on there. Okay, and you're gonna have something that's like log event and I'm drawing that very badly, but log event keeps growing.

59:06

Briana, I don't know, man, what what what is this class?

59:16

Random comment. Okay, so um, alright. So, when we have something that's on the order of log of n , it's going to be growing much slower. And so it comes out kind of under the linear ones, alright. So this is roughly log of n . This is some constant. This is and this was

DS Lecture 1 Page 13 of 21 Transcribed by <https://otter.ai>

meant to be n squared. This was meant to be n cubed. And this was meant to be two to the N . All right. Okay. So if you're so when we talk about the relative growth rates, when I asked what grows faster than the other, we really are looking at things that that start to diverge dramatically, as n gets very large. And in fact, if you're not sure of the relative growth rates between two functions, you could always resort to using calculus, and using l'Hôpital's rule, and taking the ratio of the first derivatives of the two things and see whether it tends towards zero, in which case they're growing at the same rate. Or if it tends towards infinity or negative infinity, which means one or the other, is growing at a faster rate. I don't think you're actually going to have to resort to using l'Hôpital's rule. But hold on one second. I don't think you're going to have to resort to using l'Hôpital's rule to see the difference between them, I think you can just sort of plot them to get a sense, because you don't really need calculus for this. But when in doubt, that is the way to do it. And in fact, if you look at I guess, figure 2.1, you'll see sort of common orderings. For these kinds of things. Now somebody noted, does the base matter. And the base don't matter, although typically we're talking log base two. But remember, there's a base conversion formula, that's just a ratio of the of the bases of the logs of the basis that you

can multiply to convert bases. So converting from one base to another is simply a matter of a constant factor difference, so it doesn't matter. All right, there are a few rules, in terms of big O, that are going to be quite helpful for us. All right. And so rule one actually comes in two flavors, A and B. But the preamble to rule one is imagine you have two functions, T_1 and T_2 , one of which is known to be $O(f(n))$ and the other is known to be $O(g(n))$.

1:02:25

and T_2 to have n which is known to be $O(g(n))$.

1:02:33

All right. And so for rule one a, we have what happens if you add the two functions, T_1 and T_2 .

1:02:50

If we add them back can be said to be equal to $O(f(n) + g(n))$.

1:03:02

And you might be looking at that for a second be like whoa, of course, right? Like,

DS Lecture 1 Page 14 of 21 Transcribed by <https://otter.ai>

obviously. But what have I done here, that's a bit subtle, going from the initial definitions to this some anybody? Yeah, I've pulled those two into the big O. So it's not $O(f(n) + g(n))$, you can actually pull them in together. And one of the things that we've been talking about from the beginning is that as we do this asymptotic analysis, it's the lower order terms that get dropped, right? If you have $n^2 + n$, we still just call that $O(n^2)$, because the n^2 term dominates as n gets very big. So somewhat informally, $O(f(n) + g(n))$ is essentially $O(\max(f(n), g(n)))$. Because if $f(n)$ grows at a faster rate than $g(n)$, then the $g(n)$ just gets drowned out, and it goes away. Does everybody good with that rule of thumb? Cool. Out of curiosity, just to help a little bit here. Oh, um, so imagine $f(n)$ is n^2 , and $g(n)$ is n . So when you add them up, it's $n^2 + n$. But as n gets very large, the n^2 term dominates the n term. So inside of the Big O, we can just drop the lower order term. So $O(n^2 + n)$ would just be $O(n^2)$. Cecilia, is that good. Cool, and so I save the max of f and g . I really mean, you look at their two growth rates, pick which one of them has the faster growth rate, and that's the one you keep. Good. Now, if T_1 and T_2 correspond to the time costs of two fragments of code, this one case a in code, what scenario does that sort of map to? Me by sorting and searching, okay, more generally doing one thing, then doing something else, right, two things done sequentially, not related to each other one thing after another. Good. All right. And then the part B of this rule is let's say we have the same setup T_1 , then is equal to $O(f(n))$ and T_2 is $O(g(n))$, if we then multiply them,

1:06:20

we can also collect those $f(n)$ and $g(n)$ events inside of the big O. So T_1 times T_2 could then be said to be $O(f(n) \cdot g(n))$. Now, there's no dropping lower order terms there, right? you're multiplying them out. So if one was n and the other was n^2 , then that's going to end up being n^3 . Now, if T_1 corresponds to the cost of one code fragment, and T_2 corresponds to the cost of another code fragment, what scenario in code does this case refer to? So nested things, so if T_1 event contains a loop in it, right, then that would mean that T_2 event would be happening as one of the operations inside of the loop of T_1 event. So imagine T_1 event is a represents the cost of a piece of code that has a loop. And then inside of that loop, it invokes a method. And now it just so happens that that method invoked, its cost can be described by T_2 events. So when you nest loops together, the costs of them multiply rather than add. Are we good with that? Cool. All right, there are two more rules. The next one is actually critically important, but we've already been using it pretty constantly. So rule two, yes, that was meant to be multiple predictions.

1:08:10

Good. Now, it does not look like a plus anymore. Rule two

1:08:20

says that if $T(n)$ is a polynomial of degree K . So K , eighth degree polynomial would be a polynomial with a highest order term is N to the K . So a polynomial of degree k in terms of n might be something like $3n^k + 4n^{k-1} + 5n^{k-2}$, or something like that. Right. So if we have a polynomial of degree k , then we can say that that polynomial is actually equal to $\Theta(n^K)$. It's actually tightly bounded by just the highest order term. Now, we've been dropping the lower order terms all along, but it's actually this rule right here. That is what allows us to do that. Okay. Now, I'll note the third rule that the book uses. Just for reference, this one is the least important to us. Let's imagine that we have something that costs $\log$ to the eighth power of N . That's like $\log$ of n raised to the K eighth power, okay? $\log$ of n raised. Some constant case, power is always equal to $\Theta(n^K)$. Now, I mean that in the true sense of it is upper bounded by $\Theta(n^K)$. Now, that doesn't mean that $\log$ of n is equal to $\Theta(n^K)$. In terms of like a tight bound, it's, it's, it's not, it's it grows at a lower rate. But what we're saying is that $\log$ of n grows so slowly, that no matter how high a power k , we raise it to, like $\log$ of n raised to the 50 million power, no matter how high a power we raise it to, it will never get bigger than linear. So that's just how slowly $\log$ of n grows. That's really what that's meant to demonstrate. Okay? Alright. So those are the rules of sort of how we manipulate big O's directly. And we talked a little bit about how that actually maps to code. But it's worth bringing up as sort of more general rules associated, sort of more heuristics that we're going to be using when we actually go and analyze our code. So $\log$ of N , no matter what power we raise it to, will never get bigger than order. And that's what we were saying. So there's some heuristics that we need to talk about, um, first of all, if we have a loop, it's on it's, it's right in chapter three, it's sort of the least important one I need to move on. I don't have much time. Um, so yeah, that that one. Um, so when you see a loop,

1:12:20

and something strange happened there, sorry, when you see a loop, and we're, we're trying to figure out what its cost is, look at the bounds of the loop, look at where it starts, look at where it ends. And if you look at its contents, as long as all of the operations that are being performed in there are constant time operations, the bounds of the loop tells me what its cost is. So if n is the size of my input, and I'm going from zero to n , this loop is order n . If I have a loop written like this, over the cost of this loop be

1:13:18

Hey, bye. n^2 , C are bound here is n^2 . That's the first thing that's important in terms of dealing with loops. The second thing that's important in terms of dealing with loops is when we nest them. So if I have something like this, then inside of it

1:14:05

every operation I have here is constant. My outer loop goes from zero to n . And then for each iteration of my outer loop, my inner loop goes from zero to n . So this code fragment would analyze to what? n^2 because we can drop the lower order terms there. And so the idea here is when you nested loops you're multiplying. If I tweaked this, such that my outer loop went from zero to n times n , and my inner loop went from zero to n . What would the cost of this be? It would be n^3 because it's multiplied. Now there's a subtlety to this and this is the piece that I'm going to end on. Let's imagine I have the following loop. All right. Ok. So I have the following loop, my outer loop goes from zero to n . But my inner loop goes from i to n . And so we have to think a little bit about how this might analyze. All right. And so let's think about it in terms of what is the cost of each iteration of the outer loop. So on the first iteration of the outer loop, how many iterations

of the inner loop happen anybody. And so that contributes n to the overall cost. On the second iteration of the outer loop, how many iterations of the inner loop occur, and minus one. And on the third one, it would be n minus two, then n minus three. So the last iteration of the outer loop, the inner loop would only happen once. And so you'd have this arithmetic series, and plus n minus one plus n minus two. Does anybody remember possibly from their reading of chapter one mathematical review? Good. This adds up to n times n plus one. quantity over to side note, if you wanted to prove that you use induction, like we did earlier in the in the in the lecture. Now, n times n plus one over two, let's multiply that out. That's equal to n squared over two plus N over two. Now, if we're talking about that, in terms of Big O, we know we can drop the lower order terms, because this is a, this is a polynomial of degree two. So it's just n squared over two. And we know we can get rid of constant factors. So the one half is just a constant factor. So this is just equal to big O of what, it's just big O of n squared anyway. So that's important to note. All right, we're at a time. Um, what we're going to do on Wednesday is we're going to wrap up our discussion of our heuristics for doing algorithm analysis, which should get you all the way that you need to do to finish the written homework. That's due next Wednesday, you should be good to go with the programming homework as is. And the idea is that we'll finish up what we're going to do with chapter two on Wednesday. And then on Wednesday, we're actually going to start working on chapter three, which is abstract data types and lists, okay, so if you have a moment, you can start reading the first sections, or two sections of chapter three, or skim through them, in reality is probably best, you could

DS Lecture 1 Page 17 of 21 Transcribed by <https://otter.ai>

just look at them over to get a sense of what's there. Listen to my lecture on Wednesday, and then really read chapter three, the subsequent weekend. Sound good? Ah, I think you're actually in pretty good shape with the written Now, you might want to read the next couple of little paragraphs in the text. Daniel, what's up? Yeah.

1:18:51

Quick question regarding the definitions of TFS or oven omega are big old big omega big theta. Why do we call Big O the upper bound? If it's less than c of f of n ? I guess it would be sure to understand what is f of n g of n ? Hmm, what do they represent?

1:19:11

Those are the functions that are inside of the bego, or the big omega, where the big fail. So the function inside of Big O is an upper bound on T of N . So why is big O an upper bound if if big O of F of N means t of n is less than or equal to c times f of n ? It means c times f of n is greater than or equal to T of N .

1:19:40

That makes sense. And f of n is Okay, that makes sense. I was thinking of it as we're taking O is an operation on F of N , but it means like, f of n is our big ol Okay, that makes sense. All right. Venkat. What's up?

1:19:54

my presser is the intuition of the last example we did that if we have nested loops, that Both have an upper bound of n , that are complex time to time complexity will be n squared or is that just cool? You

1:20:09

have to be careful, you have to look at where both start. And you also have to look at the increment. Okay, thank you case, the increments were both. We're both one, somebody asked, but what if the end for the loop is constant? And I'm not going to directly answer that, because that's a homework question. But, well, we're talking about things in terms of n . And then things that are n in terms of n . Well, they're basically constant, right? Can a function get faster with greater input? Meaning can its growth rate sort of regress? I don't

DS Lecture 1 Page 18 of 21 Transcribed by <https://otter.ai>

really know how we could write something to do that.

1:21:09

Other questions? Okay.

1:21:19

I'm gonna stop the share.

1:21:29

You really need to not unmute if you can't get that that feedback issue. So can you just explain one more time why it's n plus n minus one plus n minus two? Well, that I'm pointing at something you can't see. Well, in the first loop, i goes from zero to n . So in the first iteration is zero, the inner loop on j goes from i to n . And so in the first iteration of the outer loop is zero. So j goes from zero to n , in the second iteration of the outer loop is one. So then, when the inner loop happens, j goes from one to n , because it starts at i . And then next i goes up to two. So j goes from two to n . Miranda, does that make sense? Cool.

Other questions?

1:22:35

So, um, question? Yeah, go ahead. Um, so when you said that, like the fairly spooky video, I can see a tiny linear. Sorry. It's, I'm worried about it. Just go. Please.

1:22:58

Um, so for the end, is it on for when it's the

1:23:05

log, it's always gonna be faster than the linear. And you said that log is bigger like that. I guess like, word linear is always gonna be bigger,

1:23:15

DS Lecture 1 Page 19 of 21 Transcribed by <https://otter.ai>

will always grow at a faster rate, meaning it will always be slower than long, no matter what power you raise the log event. That's what we focus

1:23:27

so that the growth rate of linear is always faster than large. Okay, thank you. Other things? Hey, thanks, Dr. Blair. Hey, without the feedback, have a good one. No, once again, I just found out did something where the microphone on my computer there's something wrong with it. So I've used my basically I've used two devices wonder, oh, you gotta make sure to like completely mute off one of them otherwise, yes.

1:24:11

But there is the that for whatever reason, I know that you can't enable intergroup chat and they actually do closed caption everything that you say so just there Okay, there's a

1:24:29

there's a person who comes in and captions and I'm not sure who does it. But I know who it is. There's a person from Disability Services who comes in but it's only

1:24:39

in your class, which is bizarre. But anyways, there are people that need it. So they really got

1:24:46

to go and there are people who need it. I mean anything since like eight o'clock this morning. Gonna go to sleep. No long day. Okay, so all right. Dr. Blair, thank you again for the call. Listen, we've had a great time. And I guess we'll have to wait until Wednesday, because I don't have any questions at this point in time. Hey, Doug, any other questions?

1:25:16

Yes. When I was writing down the rules, I missed your brief explanation about a polynomial rule, I think that was rule two. So if you have

DS Lecture 1 Page 20 of 21 Transcribed by <https://otter.ai>

1:25:30

if t of n is a polynomial of degree K , so that means the highest order term is some constant times N to the K . And then there are lower order terms, like n to the k minus one, or n to the k minus two or something like that, then that t of n is equal to big theta of n to the K . So just that that just formalizes all the other stuff drops out. Thank you. I mean, it's critically important, but it's very, very small. Sound good?

1:26:13

Can I just ask one quick question, please. Um, I was just curious about like, when you said that we might need to use the or find the average case. Is that kind of similar to like, the exact like, like data? Or

1:26:30

is that again, it's you. I could ask you for bigger Oh, in the average case, I could ask you for big omega in the average case, I could ask you for big theta in the average case, they are unrelated to each other. So when we talk about best case, worst case average case, that's sort of telling us what the circumstances are for the T event. And then the big O big omega and big theta or how we found the T of N . Okay. Okay. Okay. Thank you. Other questions? All right, folks. Okay, Anna, we have a meeting. Right. And, like 25 minutes. Yep. That's gonna be a grading meeting. So I'm going to not talk. That's all Yeah. Oh, and. And Harrison. Okay. Cool, folks. Have a good one. I will see you on Wednesday. Thank you for Thank you, professor.

DS Lecture 1 Page 21 of 21 Transcribed by <https://otter.ai>

DS Lecture 2

Wed, 8/11 1:03PM 1:26:50

SUMMARY KEYWORDS

array, add, arraylist, recursive calls, singly linked list, called, abstract data type, loop, java, size, node, location, list, item, operations, cubed, outer loop, linked list, cost, lists

05:58

All right, we should get started. Um, so let's see, um, you have all your homework set up for next week, right? Do both due on Wednesday, you have bunches of office hours, which I think the day before I bumped the deadline from Monday to Wednesday, I think we had one set office hours that had like 28 people in it. And then subsequent hours had very normal numbers. So I think that's going pretty well. As I had indicated, I'll hold a couple of hours on Friday and an hour on Saturday. And then I'll start my regular monday, wednesday hours next week. Um, do we have any questions before I move forward?

07:00

We're good. Okay.

07:08

Okay.

07:10

How to if it's okay, I think I've only made it, I gave a co host. This time, I'm gonna have a little trouble sifting through. But that actually will cut down on the direct messages a bit. Um, so let's, let's begin. Last class. We were talking about Big O analysis. And I believe that the last thing we did was we just briefly started talking about sort of in practice, how we could look at code and start to analyze the code directly and some rules of thumb for

DS Lecture 2 Page 1 of 21 Transcribed by <https://otter.ai>

that. And if I'm remembering correctly, and I didn't have a chance to review the last 10 minutes of yes of Monday's lecture, maybe you can remind me head on. I done nested for loops, and nested for loops example. I feel like I did, right. One person said no one person said yes, a lot of people are vigorously nodding. It was meant to be fairly brief. So if I

remember correctly, I was talking about well, if you have a single for loop, you look at the bounds of the loop. Right. Um, and then we said, if you have nested for loops, they're going to multiply. And then I believe the last thing that I had done, there was something along these lines. Might as well just review it quickly. That's an interesting Oh, I wrote there. Well, we'll move on and pretend that wasn't accidentally meant to be parentheses.

09:12

Actually, we don't even need the curly brackets here. And so it looks something like this, where the outer loop started at zero and went up to n . But the inner loop, which was going across day, starts at i and goes up to n . I believe that was the last thing I showed. And we recall that one of the ways to think about this was to do something like all right, let's count how many operations happen per iteration of the outer loop. So in the first iteration of the outer loop is zero. So the inner loop goes from zero to n , which means there are going to be n iterations of the inner loop throughout that first iteration of The outer loop. So that contributes a cost of n . And then when that inner loop finishes, it cycles back and then i goes up to one. And so then the inner loop happens again, starting at one going up through n . So instead of happening n times, the second time through the inner loop goes n minus one times next time through the outer loop has i equal to two. And so j starts at two now. And so that's n minus two. And then it keeps decreasing the number of iterations that the inner loop happens until eventually, it becomes n minus one. And then there's just a single iteration of the inner loop. And what we were supposed to remember from Well, from either middle school or the chapter one review section, this arithmetic sequence adds up to n times n plus one over two, which if we multiply that out, becomes n squared over two, plus N over two. And if we want to express that in terms of a big O , we can drop that lower order term and drop the constant factor and a half is a constant factor. So this is just big O of n squared. Okay, so I think that's where we left off last time. And so the remaining two little things for us to talk about in terms of how we're going to go about analyzing things, okay, we know how loops work. And we know how nested things work, sequential things just add, right. So if instead of nesting these two loops, you just have one followed by the other, it would just be n plus, well, it'd be a little bit tricky. If it started at i , when it was outside. But if they both went from zero to n , and they were one after another rather than sequential, that would just be n plus n , which would be order n . So far, so good. How much does the particular operation matter in the loop? Well, what matters is whether or not the operations in the loop are constant or not, right? constant, meaning that they whether or not they take a fixed number of operations to be performed, relative to that

DS Lecture 2 Page 2 of 21 Transcribed by <https://otter.ai>

input size. And if there are 50 operations inside the loop, and it's always 50 operations, whether n is one or n is a billion, that's still constant. But if the operation inside the loop actually operates in terms of the size of n , then it becomes multiplicative again. So for example, let's say I write a for loop less follows. And then as my one operation inside that loop, I'll just call a method called I don't know, mind method on it.

13:32

And so to analyze that, we really would need to know, what is the cost of my method, right? So you'd have to dig into that a little bit and see, if my method is just doing a constant number of operations, then this would still be order n , if my method was, in fact, something like sorting something of size n and, and doing it in a somewhat inefficient way, maybe my method would take n squared time to run, because it sort of has loops going on inside of it. So then we would fall back to that whole multiplicative thing. So if my method on n took n squared time to run, what would the overall analysis of this fragment be? They might be n cubed, right? Because the outer loop goes from zero to n . So it happens in times. And I'm saying that that method call happens to take n squared time happens every time. So n times n squared is n cubed. We're good. Thank God. Did that answer your question? So big O of a constant is typically written is big O of one. All

right. So think about it as. So let me just finish with John here. So think about it as follows. You can say big O of C, and that's fine. But really, these are polynomials, right? And so it's of a constant time, it's really c times N to the zeroth power, right? c times one. And so that end to the zero is the highest order term of the polynomial. And the C is the constant factor that we're allowed to drop out using big O. So we can safely write it like this down. Is that good? I'll take your silence as a yes. Okay, cool. Cool. All right. So vancamp, what if it's a set number instead of n? Well, that falls right back to what I said before. If there are 50 fixed operations that happen every single time, no matter what the size of n is, then it doesn't matter. It's still constant. banquettes, does that make sense? Man, ask

15:54

a follow up question. Sure. It's not the homework question. And this can get you towards that. Oh, I'm sorry. If it is. I'll try. And let's see. So if like if, if like if instead of variables that we had, like the first one was 30, and was 30. And the method inside? Well, you landed right on the homework question. Oh, okay. Nevermind, sorry.

16:22

Sorry. Sorry. I know, it's all good. But I want you to think about that in the context of what

DS Lecture 2 Page 3 of 21 Transcribed by <https://otter.ai>

we said. And it's supposed to be the size of the input, and its variable. Okay. Thank you. Okay, that then so an all should follow from that. I knew that was what was coming. All right. Cool. Um, all right. Other questions? Cool. Okay. So that's what I wanted to say about four loops. There's another control flow construct that we have to figure out what to do with. And that would be conditionals, or if statements. And so this, actually, there are a couple of ways of handling it and could be a little bit subtle. So let's say if something is true, we do s one where s one is some set of statements. Else. We do s two, where s two is some other set of statements. Okay. So s one is a set of things that you can analyze independently, maybe it's a set of loops, maybe it's a constant thing, whatever. And then s two is something that I could analyze independently. If I present you with this, what do you think you would say the big O cost of it would be? Would it be the cost of s one? Would it be the cost of s two? Would it be the product of them? Would it be the sum of them? What do you think what would make sense? So somebody said as one plus s two M. And to address that point, it was one of the options I gave, that doesn't quite work. Because this is an either or there isn't really a scenario in which we would do both. And so a lot of people are suggesting, take the bigger one of the two. And I actually rather like that, um, the reasoning there is that, when in doubt, you're supposed to sort of pick the worst case, right? When in doubt, we're doing worst case analysis. Now, I haven't said anything about what the condition here is, we don't know what the likelihood of that condition being true or false is, maybe that condition is based on something the user enters. And so we wouldn't even know until the user actually types, right. So if we have nothing else that we can possibly do, analyze the two alternatives and pick the worst of the two and use that. And that is a reasonable thing to do in most cases, when you have an if statements sitting by itself, and you can't really determine the frequency. But sometimes it can be a little more subtle than that. Sometimes this if statement might be nested inside of some other for loop. And you could actually observe that this condition might be true with a certain frequency. So you might be able to calculate how often that first one might happen, and how often that second one might happen, and then do something a little more subtle for him that. So let me give you a really, really simple example, where you have to look a little more deeply at what the code is doing. All right, so let's say our input sizes n. And we have a for loop that will have go from zero to n.

20:22

Okay.

20:26

Then we have an if statement. Let's try the following. Give me one second here. I realize

there's a bit of dead dead air here while I'm there, not narrating but whatever. All right. Okay. So for the moment, the if statement taking just by itself, if the if statement were true, what would the cost of this be? In terms of n ? Anybody? This is actually n times n times n , I threw an extra little line up there just to that would be n cubed, right? And if this V_f was not true, and we went to else, what would the cost of this B by itself? Just this would just be n , right? So, I'm not talking about the loop for a minute. It's n times n , and then I stuck another times then up there, then cubed. Christopher. Okay. Sorry, wasn't drawn that well. Okay. So, again, ignoring the outer loop. If we apply it our pick the worst cost? What would I say using that pick the worst approach? The cost of this if statement would be we'd sit? Why would you say into the fourth? I'm not I'm saying ignore the loop. Right. So you pick the larger of the two. So it would be n cubed. I do like n to the hashtag that that that was good. All right. Personally, I prefer off to Thorpe to hashtag but whatever. Alright, Anyway, moving on. So we would say that this is n cubed, right? Because the more expensive the the two options costs, n cubed. And so if we were going to take that approach, and then factor it in the for loop, using that worst case approach, what would we call this? David, now is your moment. Now it's n to the fourth, right? Because then we say that the if statement would be n cubed, but then we do the if statement. And times. But in reality, we can do better, right? Because if you observe the condition for the F statement,

24:10

we know

24:13

we know exactly how many times this case happens. How many times does this case happen?

24:24

It happens once, and only once. How many times does this case happened?

24:33

It happens n minus one times. And so because we can actually see how many times these things happen, we can add them up a little better. So we know that this one happens n minus one times. And for each of those n minus one times it takes order n time, right? So

that's essentially order n squared. Correct and minus one times Then is n squared minus n , so it's just n squared in Big O land. So we know that this condition would contribute n squared. But then we know this one happens exactly once. So it's no longer multiplicative here with the outer loop because it only happens once. So like John here said, it's really n squared plus n cubed. So if we did our slightly better analysis, where we took into account, the condition inside of the if statement, in terms of Big O, what would this be? It would just be big O of n cubed. So only default to the Choose the worst scenario, if you really have no way of knowing how frequently that condition happens. Because once we actually take into account that condition, and can really look at it here, we realize this is not an n to the fourth algorithm, it's just n to the third. Is everybody okay with that? So there's some subtlety to dealing with conditionals. And so you have to keep that in mind as you go through life. Like, well, this class I that's not life so much, but Well, okay, fine. Maybe it is. The n squared.

26:29

So this loop,

26:33

happens n times. And then the outer loop goes to this condition n minus one times. So it's n times n minus one, which is n squared minus n , which is just big O of n squared. And my we say, Why isn't it n squared? Well, we do know that this happens once. And so be n

minus one iterations contribute n^2 . But that first iteration actually costs more than every other iteration combined. So it drowns it out in terms of big O. Maya, does that make sense? Where does Oh, sorry. That's n^4 cheese, n to the fourth? No. Oh, n^3 . Rate Raven, you? Sorry, why is it n^3 instead of n to the fourth? This loop? That's n^3 . But it happens exactly once. Right? Only when n is equal to zero, which happens only once. So even though it's technically inside of the loop. We know this happens only once. Sorry, you confuse me. But yeah, there we go. We're good. Everybody good. Can one of the CIA's try? Okay, because it's n times n times n here. And it happens once. Okay. All right. So the last thing I want to say about this, and actually, I'll just leave this on the screen, and I'll just talk for a second is that you also will have to ultimately deal with recursive methods. Now there is a more formal way you can solve recursions using something called the master theorem. And that lends itself to some rather beautiful proofs. And we'll do an example of one of those later on in the semester. But when you analyze recursive functions, you can kind of intuit a little bit more simply than that, and you need only really think back to what we did when we analyzed binary search. Way back for Bob way back feels, why does? Why does it feel like how is it already 2021 and DS Lecture 2 Page 6 of 21 Transcribed by <https://otter.ai>

only Wednesday? Anyway, moving on. Way back about a week and a half ago, he probably did buy it binary search. And it was a recursive algorithm, right? You know, you look at the midpoint, and then you decide which half to recurse on. So when you look at a recursive implementation of that, and you're writing a recursive implementation of that in your homework, correct? Yes, y'all looked at your homework. Good. By the way, I did not get any confidence and still by the faces that I can see I saw one thumbs up and a whole lot of eyes kind of averting. Anyway Think about it this way. How many recursive calls do you're going to need to make before you reach that base case, right? So you keep dividing by two until you get down to an array of size one in binary search. And we worked out that you would have to divide it by two, $\log_2 N$ times before you got down to that base case. So in the worst case scenario, it took $\log_2 n$ recursive calls. And so you can almost think of that, like how many times a loop occurs. And then all you need to do is look at how much the rest of the methods sorry, the rest of the expressions within each recursive call cost, other than the recursive call itself. So if you look at a recursive method, like the one you're writing, or like the last part of the, of the analysis of algorithms, question on the homework one, which is also recursive, what you'll discover is that each recursive call takes a constant number of operations. So that's just a constant times the number of recursive calls that you have to make before you hit the base case. So you can kind of think of it that way.

31:20

questions on that? Okay. All right. So there's a little bit more. So in fact, no, it wasn't different in Fibonacci, because when you look at that tree, we kind of drew out, you would count all of the recursive calls that would ultimately need to be made. And because each call spawned two more recursive calls, we actually counted up essentially two to the n recursive calls. But the guts of each recursive call was only a constant times that. So it's really the same basic intuition for that as well. Does that make sense? Yeah, it just added up. Right, so for each call. So take Fibonacci, for example. For each call, you're making two additional recursive calls. And then all you're doing is you're doing an addition on that that edition is just a single operation. In addition to the things generating the recursion, it would be different if you have like a loop inside making recursive calls. But we don't we just at least in that case, we did. Okay. All right. So there's a little bit more going on in that chapter just in terms of examples, and I do hope you've read it. But I'm not going to spend time in class on it. And so at this point, and I did bring up that you should start reading chapter three, in the waist textbook, we are going to begin with chapter three now. Alright, so we're now past any and all material that would have been on the homework

assignment. And we are now moving forward. So chapter three, covers a lot of ground, it covers lists, stacks, and queues. And so what we'll begin with is this concept of an abstract data type,

DS Lecture 2 Page 7 of 21 Transcribed by <https://otter.ai>

33:44

also known as an ADT. And so abstract data type is, I guess, it's an abstraction of how we represent our data. It's sort of abstract away from even the language we're writing in. And it's, it's basically just the set of objects that we're supposed to have, and the set of operations that we're supposed to be able to perform on them. It doesn't say anything about how those operations are implemented, right? It's just sort of giving us a template for what things of this type should be able to do. Okay, so that's what an abstract data type would be. Now, in terms of Java, which is a little bit less abstract than just the plain old abstract data type, what would be the closest analogous to an abstract data type within Java itself? What would you think? Good interface, because an interface says nothing about implementation. It just gives us List of methods that a thing that provides that interface must have. And that's basically what an abstract data type is doing. The abstract data type gives us a little bit more information in the sense of kind of what those objects of that type are also supposed to represent. But it still doesn't speak in terms of the implementation details. So an example of an abstract data type could be a list. So an example of it might be the list abstract data type. And in fact, there exists a list interface in Java as well. But a list abstract data type is going to describe objects that are of this general form, say, a sub not a one, a two, all the way up to some a sub n minus one with a size of n . So it's a list of objects that are indexed from zero to n minus one. And the consequence of that is that there is an ordering to this list, you know, something is the first thing in the list, and then it has something next to it and something next to that, and so on. So a list is an ordered collection of objects. And then we need to talk about, okay, what basic operations would we want to be able to perform on such a list. So maybe we want to get an item at a particular interface, sorry, at a particular index, maybe we would like to set an item at a particular index, that is change the value at a particular index, maybe we'd like to add an item at the end of the list, thus increasing its size by one. But maybe we'd like to add an item

37:17

at a particular index, meaning let's say I wanted to add at location two, I would want to create a new item at location two, and then shovel the other items down so that they're not removed, and nothing's overwritten. But what was originally at location, two would then be at location three, and so on, I might want to be able to remove items. And I might want to remain be able to remove an item at a particular location. Or I might want to be able to remove an item at a particular index, meaning completely wipe that location out, and then shift all the items back. And I probably want to be able to figure out what its sizes and value, I want to be able to check to see whether or not it's empty. So these basic kinds of operations we would perform. Now the thing is, is that because this is an abstract

DS Lecture 2 Page 8 of 21 Transcribed by <https://otter.ai>

data type, I've said nothing about how we accomplish this. And I've said nothing about what their costs might be. And in fact, what we'll discover is that there could be many ways to implement a given abstract data type. And thus, there could be many different costs associated with the different methods available. Alright, so if I wanted to implement this list, abstract data type in Java, first of all, I'd probably transfer this list abstract data type into some sort of list interface. And in fact, in Java dot util, a list interface does exist. Now after we've spoken a bit more about ways we can realize lists, we'll actually start looking at what those interfaces look like within Java directly. But let's just talk about some implementation options. So we have this list interface. What's How do I implement this list? Any thoughts? How could I do this?

39:31

Hey, by. So I could use an array. So first of all, what I just want to use a sort of a naked array to provide this or what I want to sort of hide it. What would the problems be if I just said, just use an array? Where would that cause issues with these various methods that I want to do? able to have getting items is simple with an array, a location I got done. setting is fine a at location equals but the thing is arrays is a lot of people have just noted our fixed size. So you can't really add and remove items from them directly. And if you fill an array up, you're done, you can't do anything else. And so a whole bunch of people, both publicly and privately have messaged me saying, how about an array list. And in fact, that's exactly what we're going for here in Java. There exists a class in Java dot util called array list. And array list implements the list interface. And what it actually is doing is it's creating a class called ArrayList. And we'll look at a sample implementation of one of those probably towards the end of Monday's lecture into next Wednesday's lecture. But there is a private array in the array list. So you have some private array inside the ArrayList. And then it has a size. And when you instantiate an ArrayList, what is the initial size of the ArrayList? How many items are in that ArrayList? To start with? Not one, there's nothing in it, it's empty. So it starts out as zero. So somebody just said nine? Oh, and then apparently just missed the key? Actually, hold on. Excuse me, hold on. Let me just mute for one second. I muted my own speakers, not the microphone. That was bloody genius. Hold on one second.

42:42

Sorry. Alright. So initially, the ArrayList is empty. But we don't have an empty array in there, that array actually is going to be initialized to some initial size. Now, if I have the variable a, what's the initial size of a gun? How can I access the initial size of it? If A is just a plain old array, e dot life, thank you.

DS Lecture 2 Page 9 of 21 Transcribed by <https://otter.ai>

43:18

So

43:21

a dot length is going to be what it sizes. So when we initialize an ArrayList, that ArrayList will actually create an array that has some initial size, usually it's like 10. But then the size of the ArrayList itself is zero. So that tells me the index of the first location that I could actually put something in. So this array exists, but it's currently doesn't have any values sort of placed inside of its cells. So if I then do adds to my ArrayList, and I'm just adding at the end of the list each time. That's very simple, right? I it that just costs order one time, because the size tells me the location in the array, that is the sort of the first location free. So if I say add five, and that's just going to be tacking it on to the end of the ArrayList inside of the ArrayList. We just put the five here, and then increment the size to be one. And then if I add seven, well, we go to the location in the array at location size, we put the seven there and we increment size. All of that is constant time, right? Because this is an array and what we know about arrays, is that a razor To ordinate contiguous locations in memory. And the variable that references the array basically gives us the memory address of location zero. So if we want to hop to another index into that array, we're just adding an offset to the base address because they're all next to each other. So it's constant time to access locations in an array. We talked about that at the beginning of all of this. So that means adding at the end of the array is dirt cheap, right up until what point when does it become expensive again, a bite Well, when the underlying array is full, right, so the underlying private array that stored inside there has some initial capacity. And once I fill that up your stock, you have to allocate a whole new array that's big enough to accommodate the next item that you're going to add. And then copy all the items that were originally into in the array into the new one, add your one more item at the end, and then swap that array into place. So you have to reallocate the array. So adds. So we'll talk

for a moment. So somebody suggested when you don't add at the end, and you're right, that's also a problem. But we're gonna get there in a minute. Right now we're adding at the end. So when we reach the end, we have this one time, well, not one time, but we have this, this sort of big cost. What's the cost when we tack on to a full re? big O of what? It's not quite n^2 ? It's not that bad. It's just order in, right? Because you're allocating a new array and then copying the old one over so there are n copies. Now, how big should the new array be, that I allocate when I filled up the old one? And he thoughts on that? So several people are saying twice the size twice? The size is very positive, very, very common answer. And somebody said, 100, that that that might be a bit much. So twice the size is actually in squared? Well, we're gonna use up all the memory in the universe very quickly, if we do that. No, no, no, no. Um,

DS Lecture 2 Page 10 of 21 Transcribed by <https://otter.ai>

47:37

twice is actually is actually pretty good. Um, the nice thing about twice the size, is that means that it took me in ads, before, sorry, before I triggered the doubling. And then I can do any more ads before I trigger the double again. And so that kind of if you average out the cost of that one really expensive ad, when you have to double the size across all of the ads, it ends up being pretty cheap. Now, why don't you just increase it by one, because, you know, once it's filled up, it's not likely that you're just going to add one more, you're probably going to add a few more. And if you have to regrow it every time you add one more item, after that initial chunk, it's going to be way too expensive. So you double it each time. And so that gives us kind of a balance between not wasting too much space. But as it gets bigger, the 10th split, the spare space grows pretty rapidly, so that you don't have this one time cost hit that often. All right, so that's when you're adding at the end. So getting and setting is cheap, adding at the end is cheap. But now let's go back to what Hannah said, What if we're adding in the middle, right? So let's say I'm using the other version of add, where I say add an index three, the value 27. So that means I want the location three to become the value 27. But everything that was at three needs to move over by one and everything that was at four needs to move over by one. Everything from that point to the end of the list has to be shifted over by one. So adds at intermediate locations, when as I say at an arbitrary location. We don't know exactly where in the array it is. But if you add in an arbitrary location, what would the biggest cost of that ad be? It would be order in because you don't know exactly where you're going to be. All right. So if you're adding in an arbitrary location, assume the worst, it could in the worst case require n shifts. So that means adding in an intermediate location is order n . And the really the worst is if you want to add at the beginning, because then you absolutely have to shift all in over. So adding in any spot in an ArrayList, other than at the end, is automatically linear, it's expensive. Adding at the end is cheap, unless you sort of have to trigger a resize of the array. What about remove operations? If I wanted to remove an item at an arbitrary location in the array, what would the cost be, in the worst case, also order in, right? Because when you remove an item, you got to shift everything back. So array lists are fine. If you're just tacking stuff on at the end, array lists are fantastic. If you are jumping around in the array for accesses gets and sets are very cheap order one time. But if you're going to be inserting stuff in the middle, or removing stuff in the middle, then array lists are not so great. especially bad if you're going to be adding stuff at the beginning. Okay, so this is one way, we could implement the list abstract data type by leveraging arrays. But there are other ways to realize lists. Now, the first of this next category of ways of realizing lists that we'll talk about is actually not the way it's done in Java. But in general, aside from arrays, another way of representing a list is what's known as a linked list. And linked lists come in two flavors. They come in singly linked lists, and doubly linked list flavors. In Java. When you see the linked list class, and by the way, in

DS Lecture 2 Page 11 of 21 Transcribed by <https://otter.ai>

Java, there's a Java dot util

52:26

dot length list, which in fact, implements the same interface as ArrayList. So you can create an ArrayList, you can create a linked list, and they are both usable in exactly the same way. It's just the underlying representation is different. So the costs might be different, depending on what you do with them. But in Java, the linked lists are actually doubly linked lists. So we're going to start with the simpler variety, which are singly linked lists. These are not the way they're implemented in Java. So a singly linked list is built up from a bunch of helper objects called nodes. All right, so a singly linked list is a collection of nodes. And so each node is a is its own type of object, there's a class node. So here's a node. And our class node will have two fields two instance variables in it. The first instance variable will be its data. So that's where you store the payload. That's where you store the value that you actually want to store in the list at the location represented by that node, we other instance variable, a variable a singly linked list is called the next field, it's next. And that's just a reference to another node.

54:25

That's just another node object, which is the next item in the list in it.

54:32

And then the next field has a reference to the next node object in the list. And at the very end of the list, these next references next any x t. What would the next reference be at the end of the list? If two is the last node in this thing? What should that be just know where we could make it circular, but we're not going to that that's that that adds some complications that I'm not going to deal with right now. So in my version, we'll just have it referenced no at the end. So how do we gain entry into this list, while the singly linked list class will have a single node variable called Ed. And that will be a reference to the first node location zero in the singly linked list. So you have to start at the head and move forward through the whole thing. So if I want to get location zero, that's just head dot data. If I want to get Location One, that would be head dot next dot data. If I wanted to get location 57, I would have to iterate through the next field 57 times before I can extract it. So a get or a set on a singly linked list. Unless I'm doing the operation directly on the first location, what's the big O cost of getting set on this thing? It's and because we always have to iterate over to it. If I want to add an item at the end of this list, and the only reference I have to it isn't the head, what's the big O cost to add at the end? It's also on

DS Lecture 2 Page 12 of 21 Transcribed by <https://otter.ai>

our end. So so far, it doesn't really feel like I've improved anything here. Grace, what do you what reference specifically? Are you asking? So data is the field in the node, that storing the values? So my list in this case is the list of values? three, five, and two? So my linked list is this three nodes, where the data contains those values that I'm trying to store. Great. Does that make sense? Cool. By the way, when we were talking about arrays, we said location in the locations in the arrays are all sort of right next to each other right? In memory, these nodes could be anywhere, because these next fields are just like memory locations. So your link list can be scattered all throughout memory. That's why you can't jump directly to them. So it really feels like my gets in my sets. And my ads at the end are not that good. What about adding at the beginning of the list? Is that cheap? Actually, it is pretty cheap, right? Because if I want to add up me Why do I need to resign? Not resigning from anything. reassign? Yeah, you'd have to reassign. But that's okay. If I wanted to say add to at the beginning of this list, I'd create a brand new node that has to in its data field. And then I'd set the next field

58:41

equal to head.

58:47

And then I said head into my new node, just to assignments, and I've done I've spliced it in

really easy. So adding at the front of the list is dirt cheap, in comparison to doing that in an ArrayList. So that's one spot where I've actually won pretty substantially. Now you might say, Oh, that's kind of a small thing to win when everything else is so expensive. And in some sense, you're right, there are actually other benefits to this, which I'll get to in a minute. But one benefit is that you will only allocate data for locations that you need. Whereas the ArrayList will have a lot of temporary space allocated that you might not be using. Not temporary space, a lot of spare space that you may not be using. But adding and removing from the front of the list. There's actually a data structure that we'll talk about a little bit later in this chapter called a stack. And a stack is a data structure. It's like a stack of dishes or something like that, right. If you have a stack of dishes in the sink, you're going to add dishes on The top of that stack, and you're going to take dishes off of the top of that stack, and you're going to try to pull a dish out from the middle of that stack, you're going to break a lot of dishes, right. And so when you have a stack structure, you only add and remove from what we call the top. And if you make the head, the top of the stack, then this is actually extraordinarily efficient for implementing a stack, which is a rather fundamental data structure that's used all over the place. So this is actually pretty helpful. Now, let's say I wanted to add an item into one of these lists. Well, if I have a link

DS Lecture 2 Page 13 of 21 Transcribed by <https://otter.ai>

list like this, adding at the beginning was as simple as splicing a value in, if I wanted to add a value, say, after the five, well, it's expensive in the sense of, I have to iterate over to the five first. But once I have a reference to the node, where the five is splicing a value in that is very, very simple, right? I just create the new node that I want to splice in. Since I have a reference to the node that I want it in front of, we know that the next field of the new node should point to whatever the next field of T was. So we just set this thing equal to T dot next. Right. And since we want to, after that, we just set T dot next, equal to that two assignments, and I've spliced it in. So if I happen to already have a reference to the node that I want to add an item after, then it's very cheap for me to add the thing in, right? If I had the location in an ArrayList, that I wanted to add after, I'd still have to push everything over. But if I'm already there in a linked list, then it's very cheap. Same thing for remove. If I know the item before the item I want to remove it's very easy to just bypass it, right. So

1:02:18

in reality, if we're already iterating through the list, if we're already stepping through the list, and we're already at the location where we want to add the thing or remove the thing, then adds and removes become very cheap. And so there's actually another structure that we will talk about, probably not today, probably on Monday, called an iterator whose whole job it is, is to walk across these things, and keep track of where you are in the list. So if you have an iterator, and you're stepping through, and you know what node you're in, already, adding and removing is cheap. So in reality link lists are useful if you're planning on going through the list sequentially anyway. So if your algorithm is such that you know that you're going to be going through the list sequentially, and doing the operations sort of locally as you go, then linked lists are better than array lists, because in an ArrayList, even if you're going through it sequentially, you still have that cost of shifting things over which you don't hear where ArrayList shine, is when you're going to be doing sort of random accesses to arbitrary locations, rather than going through it sequentially. Now, before I move on to the next kind of link list, actually, let's let's do two more things with this. First of all, let's just very, very roughly sketch what the code for this might start to look like. So we said that these linked lists are built of nodes. So we probably want to have a class node. Now, we want these things to be generic, we want you to be able to put whatever you want into a link list. And you know that to be true because you've already used array lists in your prior classes. And you know that array lists are generics. So, the items in an ArrayList are object references, same thing here. So we will make our class

node generic. And so this is sort of like that any type we were seeing these are what are known as type variables. And so that means that when we instantiate the node class will specify a particular type here and then whenever we say See, this ti will substitute it with the actual type that we're creating the node on. The node has two instance variables, the data field, which is typed as this, this, this generic is typed as that type variable. And it has a next field, which is just a reference to another node.

1:05:35

And that's it. The linked list class. I'll call it singly linked list. Because linked lists and Java are actually w linked list are singly linked list class, which will also be generic will have a single instance variable, the head of the list. And that head will be typed as a node. And it will be called head. And that's really the only instance variable you need for this thing. That's our entry point in. Are we good with that? Question so far? All right. So Omar, go ahead.

1:06:31

I just wanted to ask, didn't know. I can't hear you. Did I mute myself? I see. anybody hear him? I can hear. Oh, man, I can't hear anything. Hold on. Let me see what's going on. What is going on? I can hear you.

1:06:55

Oh, I heard I am hearing Bo. But I'm not hearing Omer. Try again. Oh, yeah, I can't, I don't know what. Okay, I just wanted to ask, right, each node knows which node is after it does only what node is after it in a singly linked list, it doesn't know what its location is only knows where the next guy, it only knows where the next guy is. So you have to keep track of the index. As you start. You know, when you're at head, you're at location zero. So if you're moving forward, you have to keep track. Okay, thank you. Cool. Um, Wayne, go ahead.

1:07:39

The way that you just instantiated the singly linked lists with just a head data type, do we also not need a data and a next data type and a head as well? Or do we add those in? Oh, um, hold on.

1:08:03

Oh, it's a node i see. Okay to know. Yeah, no, sir. Yes, the next built into it. That's what's important. Now, I'm actually I missed something here. There should also be a size, it should keep track of the size. Okay. Daniel, go ahead.

1:08:25

of quick question. So before when you said that w link to us to exist in Java and singly was two, singly linked lists are not the ones that are implemented? Does that mean there's already an existing web link class? And that's what we had to write our own code for this one?

1:08:40

Oh, we're gonna write our own code for doubly linked lists to just so that we know what it is. But in Java, you know, how you have Java dot util dot ArrayList, that you've probably used. Yes. Yeah. All right. There's a Java dot util dot linked list. It exists in Java. And it just so happens that the implementation of that is implemented as a doubly linked list. You see,

1:09:10

why don't we write our own code like we don't code an ArrayList? Right, we just kind of fall we're going to

1:09:18

the point. So the point here, and this is actually kind of critical is that we want to see how

these things are implemented. And because there are consequences to how they're implemented in terms of how long it takes for particular operations to work. So by learning these subtleties of how array lists and linked lists work, now, if I give you code that makes use of the Java ArrayList and linked lists, you can then go Excuse me. Excuse me, you can then go back and analyze the code more accurately because Then you know what the big O costs of those operations? Are? They're no longer pure black boxes for you. Does that make sense? That makes sense. Thank you. Thanks. Cool. Wayne, you had your hand up there briefly. You're good. Okay. All right. So. So, um, I mean, because one of the methods that we're going to provide is a sides method. So it's kind of stupid that I would have to iterate all the way through the list every time I wanted to know how many items were in it. So that kind of helps us not have to make that trip through every time as well. Does that make sense? So we need that we couldn't, we could absolutely calculate the size every time. But this is just something convenient that we can just keep track of. All

DS Lecture 2 Page 16 of 21 Transcribed by <https://otter.ai>

right, so let's say we have an existing singly linked list. That's supposed to say, Hey, I made a mess of it, whatever. Alright, so let's just briefly sketch out what a get an add and remove might look like. Just so that we have a sense for how these things might work. So if I wanted to get an item, right, say I wanted to get location l, well, I have to sort of set current equal to zero. And then I can do a while loop. Well, then I could set a temp variable equal to the head. And then while temp is not equal to no temp equals temp dot next, and then increment current. And that actually gets us all the way to the end, it does not even remotely do what I wanted it to do. So let's try that again. So wild temp is not equal to No. and current is not equal to i temp equals temp dot next, and then current plus plus. So what that's going to do is it's going to keep advancing temp to reference the next item in the next item and the next item until I am on the item l that I ultimately want to return. Does that make sense? I have this little check to make sure that I don't go beyond the end of the list. And then you can just say, if Tim is not equal to No. I'll just return temp dot data. Right. So I'll start at the head and iterate over i times and then return the data that's stored there. And that's what I would need to do to get Are we good with that? If I wanted to add an item, I would do something similar to what part

1:14:20

what part is what in the wild. Now that's on the outside after the while loop is done. Okay, if I wanted to add an item after a location, I would do something similar to this to iterate over to a location. So once temp is at that location doing something similar to what I did here, if I wanted to add an item, I'd say maybe x equals new node

1:14:58

with whatever we want had to put inside of it. And then x dot next would be equal to temp dot next. And then temp dot next would be equal to x. So we create the new node,

1:15:29

we set its next field, we splice it in those three lines, once we've iterated temp over. So far, so good. Then the very last thing on singly linked lists, is if we want it to remove an item. So let's say we have our singly linked list. So if I want to remove an item from a singly linked list, I really need to know the location before the item I want to remove. Right. So if I want to remove this item, I need to iterate temp over to the one directly before it. Let's maybe give us a few more nodes after it. Alright, so I'm going to be removing the item that's after temp. All right. So if I want to remove the item that's after temp, I want temp to bypass what was after it. So that's one line of code. Once I've got the temp over there,

DS Lecture 2 Page 17 of 21 Transcribed by <https://otter.ai>

temp dot next. That's this arrow, set equal to temp dot next, dot next. And that would bypass the whole thing. Okay. Now, quick note. This node still exists in memory. But nothing references it anymore. There's no way to reach it. Because Java is a garbage collected language off orphaned memory just gets reclaimed in other languages, you

would have to explicitly do that. Just and I do not know what you're saying. What do you mean, the previous page? Help me out here. This is splicing a node and after? Let me just pick this up for one second. Does anybody have any questions on the bypassing? Okay. All right, what's up with that? We're good. All right, well, I'll leave that up. Um, what we're going to talk about next lecture is we're going to talk about doubly linked lists. And that's one where we have nodes that have both the next field and the previous field. We'll talk about what the consequences of that are. And then we'll start talking about the list interface in Java itself, and a related interface to that called collection. And in the context of that, we're going to talk about something called iterators. So that's the main thrust of Monday's lecture. And then from there, we'll go on. So what happens with the No, that was skipped it just existing and taking up memory. And so that's what we were talking about in terms of garbage collection in Java, if you have a location in memory that's been allocated, but nothing references it anymore. Java actually is able to reclaim that memory automatically. We'll talk more about that later. In another language like C, or c++, which are not garbage collected languages, you would have to explicitly free that memory or you would have what's known as a memory leak. But that's again, a conversation for next lecture. Sound good? All right. So we are done for today. Please be working on your homework. I'll have some office hours in the calendar for Friday. There are plenty of ta hours throughout the week, and I'm going to stop this Share, I can stick around for more informal questions for a little while, but otherwise, we're done for today. Hold on a second. Thanks, Blair. Take care, buddy. Thank you. Take care. Dr. Barr can ask your question. Yeah.

1:20:27

When you're removing a node, I get that it gets garbage collected, but does the size Do we have to? When we implement, we have to decrement of size? I got one. When we do the formal code for that we will absolutely decrement the size. You're absolutely right about that. Go ahead. Not chuki. Is that how you pronounce it?

1:20:51

Yeah. Uh, hi, Professor, I have a question. Where can we find the link to office hour?

DS Lecture 2 Page 18 of 21 Transcribed by <https://otter.ai>

1:20:57

So when you go to the course, works page. And you sort of scroll down a little bit? Do you see an embedded Google Calendar in the middle of that? Yeah. So on that? Let me go there real quick. I'm not going to share my screen, but I just want to make sure I'm striving it correctly. on there, you should see like, a whole bunch of office hours. I haven't added mine on there yet.

1:21:36

Right. I can see the TA hours but I cannot see your link. My link personally. No, I haven't.

Yeah. Oh, you haven't added yet. Okay.

1:21:48

I had said I said I haven't added it yet. Oh. But if you click on the individual ta hours except for Jesse's he somehow didn't do that. Right? Most of them have in their description, a zoom link, or my Friday hours when they appear there, which they do not yet. There will be a specific zoom link for that Friday, and only that Friday, when I start my regular hours on Mondays and Wednesdays that will be using a recurring link, that will be the same. Okay, got it. Thank you. Cool. Um, thanks as what's the original size of that private array in the ArrayList. That's actually kind of up to the implementer. I believe Java typically starts at 10. Our example, code will start at a 10 and double it from there. Okay. Shoe hang. Its next method in in the class note or object, it is actually an object reference. It's actually an instance variable. And so those node classes that we're talking about, this is something that you probably haven't learned in Java yet. But we'll do those node classes are actually nested inside of the linked list class. And those fields are actually usually left

public. So the list itself can directly manipulate the instance variables of the node. While you do a demonstration in the future, oh, we're gonna actually do it in Java. Okay, okay. Okay. Um, like, we're, there's a, if you go to the courseworks page, and scroll all the way down to textbook files, and then click on textbook source code. There's two files in there. One's called my ArrayList, dot java, and one's called my linked list dot java. And I'm actually going to spend a lot of time next week just walking us through those two Java files. And if you look at the my link list, you will see exactly what I'm talking about. Thank you. Cool, Dimitri.

1:24:10

Um, I was just curious. Um, so when you like add, like a, an item to the end of an ArrayList.

DS Lecture 2 Page 19 of 21 Transcribed by <https://otter.ai>

That is of the cost. And

1:24:23

that's so that's, so that only happens if you are, if you happen to reach the point where the underlying arrays capacity gets maxed out, and the underlying array has to be reallocated. So that doesn't happen that frequently.

1:24:43

Okay, so if you're so that's what I was kind of curious about because like if you have,

1:24:48

like removing and, or just adding a item in like an arbitrary spot is always going to be like a cost of, of like order an arbitrary spot. It's always in because no matter what we're gonna have to move the items out of the way, whether you have to reallocate it or not. But if you're adding at the end, you don't have to move anything out of the way. It's just a question of whether or not you have to reallocate the space.

1:25:16

Okay, that makes sense. What if there is no? Or if there is enough space, then it's because dump it's constant constant,

1:25:25

because then it just jumps to the end of the list. Right? So yeah. And then I guess it has been, what do you?

1:25:32

Yeah, how do you how would you represent like that constant is that like, is early to be around like what the actual

1:25:41

DS Lecture 2 Page 20 of 21 Transcribed by <https://otter.ai>

I mean, it's just the number of operations, it's just, you have variable in in there, right? That keeps track of how many items are filled. And so that means if the size, if the size of the ArrayList to say 20, that means the underlying array has items zero through 19. filled. So that means if you want to add something at the end of the list, you add an app location size, because that's one past the last filled location. So you just do a add size equals new value and then do size plus plus, it's just two ops. No matter what,

1:26:22

okay. Gotcha. Okay. Thank you. Yeah, cool. Anything else wasn't feeling better. Have a good weekend. Thank you. Yeah, I've just been a little congested. Um, we're good. Folks. Have a nice rest of your weekend weekend, and I'll see you all on Monday.

DS Lecture 2 Page 21 of 21 Transcribed by <https://otter.ai>

DS Lecture 3

Wed, 8/11 1:05PM 1:27:25

SUMMARY KEYWORDS

iterator, node, collection, remove, doubly linked list, list, arraylist, method, called, index, item, implement, question, add, java, singly linked list, returns, marker, link, object

01:38

Travis CI can spin

01:42

yeah

02:05

way except for the Tomi sweater I'm in shock to me. Where is my Where is my

03:25

where's my where's my

04:04

what's up

05:01

All right, we should get started. Um, I guess before. Long one second. Actually, I want to check something. Okay, that's good. All right. Okay. So before I get started one of our otas I wanted to talk to you a little bit about setting some stuff up. And Cody, do you want to

DS Lecture 3 Page 1 of 24 Transcribed by <https://otter.ai>

go ahead? Yeah, um,

05:38

I just emailed a bunch of people. If your username was not set up, like it was told in homework zero, so that your first name is your uni and your last name is your full name. I can actually Oh, okay, I guess I won't share my screen. But just make sure that if I emailed you, you go into co do theirs. Go ahead and ensure. Okay, so here what you do when you're in code, do you go to your username at the bottom, you go to your account, and then you make your first name, your uni, and your last name is actually needs to be like this. So just make sure that you have your first name to be your uni, last name, your full name. And if I emailed you, then that's you. But if not, just double check. And if you have any questions, let me know. Thank you. For anything else. backup. Good.

06:52

Okay, I think we're about ready here. Alright, so a few notes. As she said, please do make sure that you configure things properly. In terms of your naming and co do it, it helps us immensely with both helping you and with grading related things. Um, Office Hours seem to be going pretty smoothly. I had some sort of one off hours on Friday and on Saturday that were reasonably well attended my regular Monday, Wednesday, hours started today, my Monday hours sort of overlap with eBay. So I guess people see me for sort of different things than they see the TA is, but I guess that'll help. Um, was pretty busy for the first hour and pretty not busy for the second hour. I'm assuming Wednesday's office hours will be a little different. Because as you know, on Wednesday, your first homework assignment both written and programming is due correct?

08:08

Yes, good.

08:11

Remember, we provide you with 150 late hours total. They apply separately to the written in the programming. So say, You're 10 hours late with both that would use up 20 hours in your bank. So don't excessively use your late hours especially on homework one. You can use up to 48 of them, if you must on each of the written in the programming. And after the late deadline passes. One of our gas' will go through and update a non graded homework

DS Lecture 3 Page 2 of 24 Transcribed by <https://otter.ai>

assignment called late hours to reflect how many you've used. We won't actually add that

until the first round is done. So I guess please try to get your homework in by Wednesday. You have till Friday at the absolute latest with late hours. Things seem to be going pretty smoothly. In terms of questions and Piazza anything like absolutely pressing you want to bring up about the homework now.

09:28

We're good.

09:32

Okay, um, let's see. By far the most common question I've gotten involves the programming involves programming question two with the linear search and the binary search, where people get compilation errors saying type mismatch. When they try to pass their array of rectangles in in the perimeter, this search For the reason for this is because remember, the type variable, any type has to be the same for both parameters. So if you pass in a rectangle array, the second parameter has to be a rectangle. And what a lot of people are doing is they're trying to pass in an ID with the perimeter they're looking for. And that's not how it works. You have to pass in another rectangle object whose perimeter you happen to be searching for. So I figured I'd bring that up, because that is by far the most common question. Okay. So that's it for that. Homework two, will get posted. I guess on Wednesday, let's see 14 days from that as the 17th. So you'll be happy do a little bit before we have the midterm period. So that looks good. All right. Any other any other questions before I move on? Okay, so last class, we talked about abstract data types, we talked about the list abstract data type. And we talked about a few ways we could. We talked about a few ways that we could implement that, in particular, we talked about, we talked about using an array, and how we might do that with an ArrayList. And then we talked about singly linked lists. And what I want to talk about now is another construct, called a doubly linked list, which is a little bit more flexible. And then in the context of that I want to talk about something known as an iterator, which people tend to find a little tricky. And we'll revisit multiple times as we implement this stuff. And so once we've done that, then we'll talk a little bit about collections and lists in the Java API, and how that fits in with ArrayList and linked lists. And I think that's about as far as I'm ever going to get today. But if I get really aggressive, we might start looking at a code implementation of array lists. But most likely, we'll be doing that on Wednesday. Okay. All right. So our singly linked lists was an object that was constructed, basically, out of separate node objects, you had a reference to the head node, that's the items zero of the linked list. And that was it. And then if you wanted to make your way through that list, you had to follow the next

DS Lecture 3 Page 3 of 24 Transcribed by <https://otter.ai>

field of that node. Now, that presented, you know, some, some problems in terms of Big O costs for things, but improve things in other ways. But it still was constrained by the fact that you can only look forward, you can't move backwards, which gives us a whole bunch of issues, not the least of which is if we are currently on the node we want to remove, we can actually remove it without a reference to the node before it. And if you want to add stuff at the beginning of the list, that's cheap. But if you want to add or remove stuff at the end of the list, that's expensive. So a doubly linked list actually tries to address some of these things by adding only a constant factor, extra space cost. And so in a doubly linked list.

13:51

Our node objects actually consist of three fields rather than two fields. So this is still our node object. But now you've got the data in the middle. You have your next field, just like you did before, but you also had a previous field. Daniel, go ahead. I just wanted to ask, you said it's a constant, extra space cost. I might be like thinking about it wrong. But isn't it an N extra space cost? Because for each node now, I should have said constant, I should have said constant factor. That's a left the word factor off. So constant factor means like, n basically like degree one, or Well, I mean, to start, it still took n and now we're

multiplying. And so first of all, for doubly linked list. You had an extra space for the next field, correct? Yeah. All right. So now the change from the singly linked list to the doubly linked list is a constant factor in front of that extra space that you Oh, I see it's still big event. So it's the CIA and things like that. That was where I was I left the word factor off, but makes sense in that context. Yeah. All right. So we've got our data, we have a next field and the previous field. So if you're adding nodes, you can move both forwards and backwards. So the chain would end up looking something like this.

15:33

And at the very beginning of the whole thing, the previous field, you could make it circular. But we're not going to do that. The very beginning of the whole thing, the previous field of the node at the front will point to know and at the very end, the node at the end, its next field will point to know as well. All right. I'm going to pause for one second, and just point out egay. Yes, chapter one. subsection two contains the mathematics review. But it is exceedingly dense. That mathematics review. Ah, took the word right out of my mouth. So in fact, something you might take a look at, there is a website called math world, which I believe, connected with wolfram and Mathematica. Yeah, that which has, I feel like it's still a little dense, but I think it's a little better than textbook explanations of various topics. So if you want sort of a further explanation of a particular topic, and that dense Chapter One

DS Lecture 3 Page 4 of 24 Transcribed by <https://otter.ai>

Search there. Cool. Alright, sorry, I digress slightly there. Alright, I'll let the the chat deal with the rest of that. Okay. So we can now kind of move forward and move backwards on this thing. Now, the other aspect of this is that when we had our singly linked list, we had just a reference to the head node. But in the doubly linked list, the idea would we we'd have two references into the actual physical nodes structure of the list, a head node and a tail node. And in fact, to make things even more complicated, but actually kind of nice when you get down to it. In Java. Anyway, when the W linked list is implemented, what we do is we have what are known as Sentinel nodes at the beginning of the end, meaning that the first item doesn't really have any data in it at all. And the last item doesn't have any data in it. And they're not actually part of the contents of the list. So you've got an end, beginning marker, and an end marker, alright, and the begin marker and the end marker, have no data and then they point in to the list. So if we want to look at how one of these things actually is structured, what you really end up with is this, which is usually referred to by Okay, in the code, when we will, we're going to look at that probably will happen early next week, you'll see it referred to as begin marker, I'm going to just call it begin, which will be a reference to this initial sentinel node, its previous field will point to know and its next field will point to items zero of the actual list and its data field, I'm going to put an accent it will just to be empty. All right, then we have like actual data in the list. And then this is the last item. So this would be location size minus one. That will then point to a sentinel node, which has no data in it which will be the end marker. All right. And so the length or I guess I should say the size because in lists, as you know from ArrayList, we use size not length, the size of this list is three. This is index zero, this is index one, this is index two, and then the begin marker and the end marker are sort of dummy nodes outside. Alright, and we retain references to the beginning. marker in the end marker. If you when you initialize this thing at the beginning, an empty list ends up looking like this with the begin

20:26

and the end.

20:28

And so the two of really the three instance variables that you would have in a link list class would be the begin marker and the end marker. And then it would have a size variable of size zero. So when you have an empty list, you just have the begin marker and the end marker pointing to each other with nothing in between. So the neat thing about

this, the neat be, why can I begin no just be our end node, because we're not doing a circular linked list. The structure for that is just, I mean, it's useful for some things, but I feel DS Lecture 3 Page 5 of 24 Transcribed by <https://otter.ai>

like it's it overly complicated stuff for people just starting. And that's not how we're going to be using it. Um, the nice thing about this is it means adding and removing from the end or in special cases, you just splice in, and you're always guaranteed there'll be a node before you and a node after you, you don't have to worry about the nulls because the begin marker in the end marker already exists. So just like with a linked list, if I want to add something to this thing, I have to figure out where I want to add it. So if I want to add something at index one, I started begin marker dot next, that's index zero. And then I loop until I reach one, taking a temp variable, I don't know call it P and advancing it until I reach the value of index one. Right. Now, if I want to add a node here, right, I want this I want index one to be a new node, say with the value seven in it. And then the five and the two get pushed to be indexes two and three respectively, right to indices two and three, respectively. Where am I adding the node, the new node with respect to P before or after it such that the new node I'm adding will become index one. I'm adding it before. Yep. And so if I want to create a new node, well, I'll instantiate a new node object with the value I want to put in it. Now we know it's going to come before p. So I can just set its next field equal to P. And I know it's going to come after whatever used to be before p. So I can set its previous field equal to P dot previous, because I have that reference. And then I just need to make these two changes. So P dot previous, I have to set p dot previous is next field equal to the temp node. And then I have to set p dot previous is feel equal to the 10th node. And that's it. I've now spliced my 10th node in before p. If I just give you the link list where you have the end marker and the begin marker, and I say add a note at index, whatever index x, and I'm not telling you, you don't know in advance what X is, in the worst case, what's the cost for adding that?

23:53

Anyway,

24:01

there's going to be order n, right? Because you have to either start at the beginning of the list, or the end of the list and iterate over to it. Now, there was a pause there and people had to think for a moment, you know the size that's stored as a separate variable? So do I start at the beginning or the end what I'm trying to iterate over to an arbitrary location. Thoughts? Well, either, but you might compare that location to the size over two. And if it so happens, that it's greater than the size over two, you start at the end and work backwards. If it's less than size, Overture, you start at the beginning and work forwards. So that means in the absolute worst case, the number of iterations that you would have to get to move over would end up being And over to, but that's still constant, sorry, that's still

DS Lecture 3 Page 6 of 24 Transcribed by <https://otter.ai>

a constant factor. So that's still on our end to get over there. All right, I'm always talking bago. And so that means that splicing it in, if I don't give you a direct reference to the node that you're adding before, it's still ordering. But if I gave you a reference to the node that you want to add before, you already have P, what's the cost of adding before in terms of bingo? It would be constant, right? If I'm already there, I don't have to factor in the cost to iterate over to it. It's just that splice that I did. And really, there were just four links that I had to set, which I already had references to. So that's constant and independent of the size of the thing. So the cost of adding a node is linear. If you are given the index, the cost of adding a node is constant, if you already have the node object that you want to add before, good. And so we could make a similar argument if we were trying to remove a node if we were trying to remove a node. So let's redraw this.

27:05

So we have our begin marker and end marker. And we have some values. Right? So if

we're trying to remove a node, similar kind of deal, right? If I don't give you a reference to the node that you want to remove, and I just say I want to remove index one, I got to iterate over to that index. Right. And that will have in the worst case in order, and I guess, technically N over two, but order in time. But if I already have a reference to the node that I'm trying to remove, so if I already have P , removing is trivial. I take p dot previous. And I take its next field. And I set it equal to P dot next. And I take p dot next, and I take that things previous field. And I set it equal to P dot previous. And so just like we did with the link list, the singly linked list, we bypass the thing. Now, because Java is a garbage collected language, we still have this reference p to the node that I just bypassed. But if I start at the beginning or the end, I'll never actually hit that node. So it's as if it's gone. And then when this method ends, and the local variable p goes away, there won't be any more references to that node, so it'll just get garbage collected away. So that's a constant time operation, if I give you the reference to the node I want to remove. So again, it's linear. In the worst case, if I just give you the list and the entry points or the beginning, in the end, it's constant if I actually give you the node reference that I want to bypass, okay. Now, another nice thing is that if I want to add at the beginning, I just add before begin marker dot next, if I want to add at the end, I just add before the end marker, so adding, or removing honestly, at the beginning, or the end of the list is always constant time. And so one of the things that we mentioned last time was that there exists some data structure called stack, where you're adding and removing items only at the top of the stack. And that can be rather efficiently handled by a singly linked list. But similarly, there's another data structure called a queue, where you add items at one end and remove items from the other. Well link lists work wonderfully for that, because adding and removing from either end is constant time. So this actually enhances the flexibility of this underlying structure. Okay, so any questions on doubly linked lists so far? Now we're going to dive real deep

DS Lecture 3 Page 7 of 24 Transcribed by <https://otter.ai>

into this at some point next week and go through the code for it. Will this be on the exam? Um, yes, very much on the exam like this more so than some of the other things that I say will very much be on the exam. Wayne, go ahead. Hey, Professor, um, well, the exam be on the exam. Just kidding. Um, I should about a way to make that happen. So just going off the comment that you just made about stacks and queues? Is it helpful to think of stacks and queues in terms of linked lists that just have different orders in terms of how we access the first and last elements of each? Like other words? Yes, please go ahead.

31:16

I'm, I'm a little, I'm a little hesitant to say you should just think of a stack and a queue is just different types of link lists. And the reason when we do stacks and queues, like for real, which will, not next week, but the week after, what we'll very quickly discover is that link lists are not the only way we can implement them, we absolutely could implement them with just plain old arrays, and have a maximum size. And that's actually a very convenient way to implement them as well, if you know, it's not going to grow above a certain size. And so it's not always like that. However, I will say this, um, one of the things we're going to look at, we talked about a list abstract data type, right? And we said that array lists were one realization of the list. And doubly linked lists were another realization of the list. And in fact, singly linked lists, yet another and they all have their own trade offs. And so one of the ways you can think about a stack in a queue is just a way to constrain your usage of a list. A stack is like a list, you just are only adding and removing from one end. And a queue is a list not saying to the linked list, any kind of a list, where you're adding at one end and removing from the other. Now, depending on the underlying implementation of that list, that could have some consequences for the costs of the various methods for the stack. But if you're going to think of a stack in terms of any of the things we've seen so far, it's probably safest to think of it just as sort of a way to use a list. And the reason why I say that is because it's a turns out in Java stacks is an interface. And

the lists all implement the stack interface. So you actually can use ArrayList. And linked lists explicitly as a stack. That that that's, that's definitely a thing that's done in Java. That's why I wanted to phrase it more in those terms Sound good? Alright, I spent a little longer answering that question than I needed to. But I feel like it ties in really well with some of the other stuff that we're going to be doing. Okay. All right. So there's our w link list. Now, one of the things we need to talk about in the context of lists in general, but I actually think it can be useful to think about it in terms of well might actually be easier to think about it a little more in terms of the abstract. Let's say for the moment.

34:21

DS Lecture 3 Page 8 of 24 Transcribed by <https://otter.ai>

We have a list with these items in it. That's index zero, that's index one, that's index two, that's index three, that could be stored as an ArrayList, a singly linked list, a doubly linked list, whatever makes you happy. Okay. Now, in Java, and in fact, in general, there is this concept of an iterator. All right. And so an iterator is another object. It's almost like let's say a meta object. That kind of is is another object that is attached to a list and actually can be attached to things even simpler than a list things called collections. But we'll talk about collections in a few minutes. And so you can create an iterator object on a list. And that iterator objects job is to keep track of where you are in the list. All right. So an iterator kind of has a current item. So eater will be our iterator object for the purposes of this demonstration, and it will in some way, shape or form, reference what we call the current item on the list. So when we create an iterator on a particular list, the iterator starts by latching itself on to items zero of the list. So items, zero is the current item that the iterator gives us that the iterator is referencing. Now, the iterator doesn't give us access to that immediately. We haven't seen the current item yet. All right. And so an iterator has three methods associated with it. It has a method called next, a method called has next and a method called remove. So when I call the next method, and so this is actually more of an abstract data type, you would implement this differently, depending on what type of list you're latching on to, and we'll get into that in a minute. But if I call iterator our particular iterator object dot next, what it's going to do is it's going to return to us the value stored at the current location. So it's not until I invoke next, that we know that the current location has the value three, so it returns three to us. And the next method actually does two things, it returns the current item to us and advances us to the to the next item. So the current item then becomes Location One instead of location zero. The has next method simply tells me whether or not the current location is valid. So if I keep calling Next, the second time, I call next, it'll return the seven, and then the current will be the five. The third time I call next. It'll return the five and current will become the two and the fourth time I call next, it'll return the two. And then the iterator will essentially point. Are people having trouble hearing me? What's the lag the lag on the writing of the screen or the lag on my face? Like I said that rather aggressively. Sorry. So for my voice, I don't know some of you are having issues. Some of you are muted myself. I had a quick question. Dr. Blair. One second, let me finish my thought. So when I advanced past the last item, if this were a doubly linked list, the editor would then be referencing the end marker. And so has next would actually return false at that point. All right, so that's next and as next we'll talk about removing the second but first boglehead

39:12

bow.

DS Lecture 3 Page 9 of 24 Transcribed by <https://otter.ai>

39:17

Whoa. Dr. Blair, do you think it's awkward that people will have their cameras on with their faces and kind of looking at you?

39:30

I hadn't thought it was awkward until 30 seconds ago. I just clicked on something and now

I have 16 screens in people's faces looking at when unclick it man, what can I tell you? Which one is the view? standard? I have no idea. I'll figure it out. Yeah. Honestly, man, it's only awkward if somebody is like eating their sandwich in front of me. And even then, whatever I mean, I'm hungry. So I'm not allowed to eat on camera. You can I just, I'm just going on Wayne, go ahead. Um, in this example, when using a head and a tail, no that don't have any value, do we? Do they not have an index as well? Or? Do they actually have an index and marker? I guess you can sort of think about them as having indices, but they're not valid indices. When we actually go through the implementation, we can kind of think of the end marker is having index size, right? Because size minus one is the last valid location. So one past the size minus one, would it be, I guess what it would be? And I suppose you could think of the begin marker is having index minus one. But that's never even relevant. So it's probably not worth getting into that. The End marker, thinking of it as location size might not be a bad idea, just because it makes sense in the context of advancing the iterator beyond the end. Does that make sense? Yeah, it's still a little bit, because I guess, I guess, what I hear you saying is that we don't have to actually think of the head and the tail as actual nodes. They move a little, they're actual nodes. But they're, they're not actual pieces of data. They don't count have no, right, they don't count in the size of the list. Right. So a list of size zero has two nodes, and then everything offsets from that. Gotcha. Okay, cool. Thank you. Somebody else briefly had a hand up. I think their name started with an lol, but I'm forgetting. Did you have a question? Whoever you were? Are you good? He answered it. Thank you. Oh, all right. Um, okay, so we have our next and we have our hands next. And then the last thing is the Remove method. Now, the one thing about remove is remove is not removing the current item. Can anybody think of a reason why you wouldn't want to remove the current item? You'd have nothing to return except nice. Ah, cool. You have nothing to return. When you click Next. That's right. Now that gets into the beginnings of the issue. That's that's part of it, I would phrase it a little differently, but very close. Somebody else had their hand up you want to began with an A, but I didn't quite get your name.

42:52

DS Lecture 3 Page 10 of 24 Transcribed by <https://otter.ai>

I was gonna say, Is it because the next bout the like the next section, you would remove that as also you would have no pointer to the next value?

43:02

Um, well, I mean, I guess maybe not necessarily. It depends on how the list is implemented. Right? If you were using an ArrayList, they're just indices into an array, so that might not be a problem. Um, alright, so you actually, the first comment really, sort of get very close to what the way I would have phrased it the way I would phrase it, is you don't want to remove an item until you've seen it. And if you're using the iterator, you haven't seen it until it's been given to you. And it's only been given to you once current has moved past it. Right. Does that make sense? So current you haven't gotten yet. So when you call next you get what was current and then current advances. So the Remove method actually removes the item immediately before current. That is, it removes the item that was most recently returned to you. So when the iterator begins, and the iterator references the three in this case, calling remove would give you an exception because you haven't seen anything before. As soon as you call next. And the iterator now refers to seven. When you invoke remove, that will remove the three and leave current pointing to seven. And if you want to remove the very last element, you can keep going next next, next, next next next until you're referencing the item after the last element and has next is no longer returning true. At that point. When you're here, one past the end, you can still remove because that's the item that he was you most recently saw So somebody asked, What is the correct way to remove some AI objects using internal loop? You have to be exceedingly careful when using remove inside of a loop. You kind of want the loop to stop. And then once

you're outside the loop remove, otherwise it's gonna cause some problems. Do you have to be on the end node to remove the last element if you're using an iterator? Yes. Okay. So is everybody okay with the three basic methods of an iterator? Any questions on that before we move forward?

45:41

Okay. So how an iterator is implemented is going to depend a lot on the underlying list. If the underlying list is an ArrayList, the iterator can just keep track of the index that you're currently at. And it doesn't really make any difference. But here's the nice thing. If you're using a doubly linked list, what do you think the iterator should keep track of? Hey, bye. The node reference, right? If you if the iterator keeps track of the node reference of the current item, advancing to the next item is cheap. Right. If you try to access the next item using an index, it'll go to the beginning and iterate back over. But if you're already there, you already have a reference to the current node going to the next item is order one.

DS Lecture 3 Page 11 of 24 Transcribed by <https://otter.ai>

Same thing for remove. If you're using an iterator, you're already there. And so if you have a reference to that, no, you can remove it in constant time without worrying about having to iterate over to it. So even though within our web link list, you have to iterate over to a spot to remove something by index. If you're already iterating. Through it using an iterator things start to become very cheap. Now, before I move on to some stuff from the textbook, I want to show you the fact that you already have been using iterators. Since Well, probably about halfway through your intro class. All right. And so the thing about iterators is that any object whose class implements another interface, that happens to be called iterable, can have an iterator created on it. So the iterable interface just has a single method called that turns out, it's called lowercase I iterator. And that iterator method returns an iterator object on that thing you're iterating across. Does everybody remember enhanced for loops? So if I have an ArrayList, let's call it a, I'm not going to initialize it. But let's say we have an ArrayList. A, if you wanted to iterate through it, yep, something like that. Let's say the ArrayList is an ArrayList of integers. If you wanted to iterate through it, and enhanced for loop would look like the following. Actually, let's make it an ArrayList of strings just so that we don't have to deal with auto boxing and auto unboxing. So if I have an ArrayList of strings, I can use an enhanced for loop.

49:15

Like that. Right. Sorry, that's an A. So at each iteration of the loop, I takes on the next element in a and the next element in the next element. And so this would just print out all the elements in a Does that ring a bell? with enhanced for loops? Yes. Cool. All right. Turns out you can use an enhanced for loop on any structure that implements the iterable interface. Because under the hood, the way enhanced for loops are implemented is they're implemented using the iterator construct. All right. And so I can rewrite this code using an iterator. So I could create an object. Type does an iterator, this is actually an interface that all iterators need to implement. And I'll call this variable ITR. And actually, this needs a type variable on it. So iterator inner equals, and anything that is iterable has a method called iterator in it. So a, which was an ArrayList has a method called iterator in it. That returns a new object that provides the iterator interface on a. And now I can use this variable on a using those next has next, etc methods. And so what I can do is I can say while eater dot has next

51:14

string,

DS Lecture 3 Page 12 of 24 Transcribed by <https://otter.ai>

51:18

I

51:20

equals eater dot next, that will get the next item, and then advanced the iterator.

51:41

This code and this code, do exactly the same thing. In fact, the Java compiler would actually internally rewrite that as this

51:55

sound good?

51:58

Have we used this in reading from a file stream? Usually, you're used to using this to read across something that's iterable. I don't think file streams are directly iterable. Honestly.

Um,

52:13

but

52:16

but the sort of that next method does kind of come up in other contexts, but it's not quite the same thing. Any questions on using an iterator?

52:29

Okay. So

DS Lecture 3 Page 13 of 24 Transcribed by <https://otter.ai>

52:34

now that we've gone through this, I'm going to switch my I'm going to stretch segment switch my stream, I'm going to switch my share. And I'm going to show us a little bit from the textbook. I'm going to make this a little bit bigger, I think. So let's not worry about streams and buffers right now. So I want to switch back to talking a little bit I'm sorry, hold on, how does the string IE equal inner dot next, get the first element in the while loop. So just before I had the while loop, remember, I set it equal to a dot iterator that actually creates a brand new iterator on a that starts out? Sorry, that starts out on items zero. So when the iterator gets initialized, it starts out on item zero as the current item. So when I invoke next, that will return items zero the first time it's invoked, and then advanced to item one. Can we stop sharing PDFs in the chat? I just downloading files is problematic. I'm sorry about that. Um, we'll Did that answer your question?

54:16

Cool. All right. So

54:23

what are the benefits of writing in the inner way instead of enhanced for loop? In general, if you're just doing a single iterator, probably an enhanced for a loop is better to hide those details. But there are times when you actually want to have sort of multiple positions in the same list stored at the same time. And you can actually have multiple iterators clamped on at different locations going at once. Alright, moving on. So let's bring this back into Java a little bit. And let's maybe take a step up the chain from lists. In Java, there is actually a slightly more general type of structure called a collection. And so a collection, as I said, is a little more general than a list. Remember, a list is a collection of objects that has an ordering to it, there's an item zero, there's an item after that an item after that item after that, a collection is still a collection of objects, but it doesn't have any particular or required order associated with it. So you can almost think about a collection as kind of a bag that you can't really see into. And so when you add items to a collection, you're just chucking it into the bag. And when you remove items from the collection, you're rummaging around in the bag to pull it out. And you don't really know or care how it's stored inside of the bag. And there's no fixed ordering to it. Okay. Now, there's actually another construct in Java called a set. And a set differs from a collection, and that all the objects in the set must be unique. But collections don't actually require the objects inside of them to be unique. Right. So in a list, you're gonna have the same object appear multiple times a list is actually a more specific type of collection. So this collection kind of

acts like a bag that you're tossing stuff into. Now, collection is an interface that doesn't say anything about how it's implemented. It just provides a list of methods that all collections should have. All right. Now, it's generic, because you can have collections of anything, really. And it extends iterable. And what we said was that if you extend iterable, then it must be possible to create an iterator on a collection via that lowercase I iterator method, which is the required method of the iterable interface. So anything that implements collection is iterable. Meaning you can have iterators on it. Now, you might ask, what does it mean to have an iterator on something that doesn't have an order associated with it. And that's a good point. I mean, under the hood, all collections have to store things somehow, there's no sort of generic bag that you can toss memory locations into in Java, really. But when you create an iterator on a collection, all it's saying is that it will give you each item in them that exists in the collection, exactly once. So you're guaranteed that if you keep calling next on an iterator, created on a collection, until has next returns false, it will ultimately give you all of the items that were in that collection, what order they come out in isn't really specified. Just it's gonna give you all of the right so that's what a collection is. And so the methods that we have for our collection of the methods that any collection should provide are the following. There's a size method, which tells me how many items are inside the collection. Now, it's sort of you may think back to 1004 days to something that I know a lot of people find incredibly frustrating. I certainly found incredibly frustrating. When you had an array in Java, and you wanted to know how many items were in the array. What did you do? How did you get that? It was a dot length, with no with no parentheses after it, it was an instance variable of the array.

59:16

And then you probably learned about strings. And you probably learned about the char at method that allows you to access individual characters in the string. And if you wanted to know how many characters are in a string, say a string s or a, you would say s dot length, but that time, you would have parentheses after the length because it happened to be an instance very sorry, an instance method rather than a variable, which is annoying. And then just to add insult to injury, or maybe injury to insult, I'm not sure. When you learned ArrayList you're like that forget about length, call size at which point People start pulling out their hair. Now thinking back as to when my hairline started to recede, it was probably around when I learned that, I think that was coincidental, but they definitely lined up. Alright. Sorry, I'm gonna digress for 30 seconds. Do you know how I discovered I was going bald? was not by looking in the mirror. I was. Remember, so did I ever show you my robot? Okay, we'll talk about my robot much more at a later time. But when I was a PhD student, I was working on 3d modeling of large scale environments. And so basically, we had this largest robot with a laser range scanner on top. And we sent the robot out into the world to take these 3d images of structures and try to get complete scans complete 3d models

of the structures.

1:01:03

And

1:01:06

we actually collaborated with art historians. So weirdly enough, Arkham, which I took in the 90s, at some point was strangely relevant to my PhD research many years later, because I was working with art historians who really wanted to document the 3d structures of these churches, these these Romanesque churches, and it's the bourbon a region of France. And so we were just looking for test beds, to check our algorithms on to make sure we were getting complete coverage. Now, these were 802,000 year old churches, so they didn't let me drive the robot around. And then because they were afraid, I would break the center that the Millennium old thing, which, you know, I've slammed into

enough walls with that robot over my life that that was a legitimate concern. So I just took the laser scanner, and I was the robot. So I would run the algorithm, and then it would tell me where to place the scanner, and I walked the scanner over and take the scans, and test things out that way. So this was fun stuff. And I'm gonna bring this up more and more over the course of the class to sort of demonstrate some of the algorithms we're going to do. This has nothing to do with the algorithms. This has to do with how I learned I was going bald, because I was working in the field, scanning the outside of this church in the sunlight for many hours, somewhere in the middle of nowhere in France. And then as we were driving back to the it wasn't even a hotel. It was some like dilapidated Chateau, we were staying and I went to scratch an inch on my head and it's searing pain, is I realized that I had a terrible sunburn right on top of my head where I went bald. Anyway, for those of you who are still young, you've got things to look forward to. Moving on. Okay, so that was a completely random digression. But it has a lot to do with stuff we're going to talk about later, that has nothing to do with my error or lack thereof.

1:03:08

So

1:03:10

back to this, we have the size method that was annoying you because your array lists had

DS Lecture 3 Page 16 of 24 Transcribed by <https://otter.ai>

size rather than length. But it turns out, that's because list is a special case of a collection interfaces are allowed to extend other interfaces. So the list interfaces of which ArrayList. And what we'll also learn linked list implements, the list interface extends the collection interface. So absolutely everything that's in collection also must exist in list, because the list is a collection. And so because size is the method that we use to get the number of items in a collection, we have to have that in the list anyway, so that's the one we use. And so size actually makes more sense logically for a collection, because a collection doesn't really have an ordering, it's not linear. So length is kind of a misnomer for it when it comes to a pure collection. Alright, so size tells us how many items are in it returns an int, we have a Boolean isEmpty method, which just tells me true or false whether or not the size is zero, a clear method which basically just dumps out everything that's in the bag and starts you with an empty bag again. We have a contains method that collections should generally have that takes an object reference. Now you'll note that the collection interface is generic, but our type variable doesn't have a constraint on it. That the type is parables, so you don't have a compare to method or anything like that. So the contains method. There's, I mean, it's a generic class, right? The generic is defined at the class level rather than at the method level, because it gets, in this case, a generic interface. But when you instantiate a particular class that implements the interface collection, the class itself is has a generic attached to it. And that's because these are instance methods. So they all use the same type variable. So is the clear method, necessarily generic? doesn't need a type variable attached to it, but it has, it has the generic from the class that it's part of. Okay. All right. So the contains method takes a value. And because we don't have a compare to method, it just relies on dot equals, so that will go ahead and remove an item, sorry, that will go ahead and tell you whether or not an item exists in the collection, based on whether any of the items match using the dot equals method, and returns a true or false. Okay, we then have an add method. Now, because a collection is sort of this

1:06:20

opaque sack that we don't really know how things are stored in it, we're just chucking stuff in. So you don't give it an index or anything like that. You just give it a value, and it puts it in. Now, in this case, the Add method returns a Boolean, which is just a true or false whether or not the add was successful. The reason why we actually need that is because, you know, lists, you can always add something to a list, unless you just run out of memory, at which point, you've got bigger problems. But collection, there are a lot of different ways

other than lists to implement a collection. And some of them actually have more constraints on them than others. And since we don't know for sure what kind of constraints we're going to have, when we're still working in the realm of an interface, we do want to take into account the fact that sometimes ads might fail. So the Boolean just tells us whether or not the ad was successful. Now, as a side note, because lists extend,

DS Lecture 3 Page 17 of 24 Transcribed by <https://otter.ai>

because a list extends collection, anything that's a list has all of these methods. So if you have a link list or an ArrayList, there exists an add method that doesn't take an index just takes a value. And in the context of a list, when you call the collection, defined add, it's just adding an item at the end of the list. So everybody okay with that? We're good so far. All right. Similarly, we have a remove, that removes by values, so it effectively does a contains first and then removes it. Now, in this case, the Boolean kind of makes sense. It returns true if the item was removed returns false if it's not removed. But it's possible, you might have attempted to remove an item that wasn't in there in the first place, at which case, returning false makes sense, because there was nothing to remove. Now this last thing that all collections are supposed to have, this actually isn't defined directly within the collection interface. It's default. And so we didn't actually need to write it here. It's written here for just so that we don't need an extra little line. But this is something that comes from iterable collection extends iterable. So that means all the methods that are defined in the iterable interface must also be included with anything that implements collection. So this line doesn't actually need to be here. It's implied by the fact that we're extending iterable. But for demonstration purposes, we've shown it here. And this is that little lowercase i iterator method, the one you saw me use when I demonstrated using an iterator on a list. And so the job of this method is to return a new object that implements the iterator interface. The iterator interface is defined in Java dot util. So that gives the full name here just in case we didn't import Java dot util dot iterator. But it's also useful just because there are so many different things and start with the word eater here. It just sort of differentiates it very well. Whatever this lower case i iterator returns must implement the iterator interface. And that will give us an iterator object on this particular collection. And so that's what a collection is. Any questions before I move forward?

1:10:05

We're good. Now, I'll just briefly if I can find it show you what Java dot util dot iterator, the interface looks like. Here it is. This is what I sort of wrote out for you as an abstract data type earlier, the interface iterator as a has next method, which returns a Boolean iterator is generic, meaning you have to specify a type variable. And that's because the next method returns, whatever data is stored at the current item. And that's, that's going to depend on what it's a collection of, so that returns the any type and advances the iterator and then remove removes the thing you most recently saw. And so we've described all of that before. That's what it looks like in the context of the interface. Do objects in collection point to other objects? Um, they might depends on how the collection is implemented. I guess, I'm gonna guess you're asking that because of the way that I implemented doubly linked lists? Correct. So the way to think about it is that the items in a collection are actually not the nodes in the link list. But the data stored within the node. And the node itself is just a vehicle for organizing that data under the hood. So if my collection

DS Lecture 3 Page 18 of 24 Transcribed by <https://otter.ai>

happened to be a doubly linked list, which is ultimately a type of collection, the underlying data is just what's stored inside of the node, not the node itself. So in that context, I guess the answer is no. Because the nodes aren't the data to hang. Does that make sense? Cool, Omar, go ahead.

1:12:10

Hey,

1:12:10

I didn't really understand what iterator does in the collection.

1:12:13

Could you maybe go over it again? What the method? Yeah. So do you see this, this interface here? Right. And you remember, actually, maybe it's worth it for me to switch the camera for a second.

1:12:39

So remember, when I did the iterator example here, you see when I called a iterator over here, and it returned to me this variable ITR that I was able to call next and has next on this method is that iterator method that all collections must have. So if we go back to the piece in the textbook, we were looking at this line here is just stating that anything that implements collection must have that method, so we can use it exactly like we did over there. Okay, that answer your question? Yeah. Thank you very much. Okay. Anything else? You have another up? No, you're good. Okay. All right. So sorry, go ahead, Wayne. So, um, in terms of using an iterator for a link list, how would we you had said that you'd mentioned before referencing the, the next and the previous reference for each node in terms of how to remove it? Is that a way to reset what's next and what is previous, if we remove a node in terms of reset or remove a node, the previous the next fields of the surrounding nodes automatically get updated based on sort of the drawing that I did. So remember, when you remove from an iterator, you're removing the item before you so current will be a reference to not the know that you're removing. Its or you're removing the node that's directly before current. And so that remove will adjust currents previous field appropriately.

DS Lecture 3 Page 19 of 24 Transcribed by <https://otter.ai>

1:14:44

Okay, so we don't necessarily have to

1:14:48

reassign next and previous, it doesn't automatically when we run the rights, so we're going to have to write sort of remove methods in our ArrayList and linked list classes anyway, it's just part of them. And then the iterator remove will leverage those existing ones and just say, hey, use your remove on what's directly before me.

1:15:16

Okay, good.

1:15:19

Yeah, won't be a No, no, that's correct. Okay, so Wow, okay, uh, yeah, it won't be a No, no. So lists, let me see if I can find that. Here we go. Lists are an extension of collection. Right. So the list interface extends the collection interface. All lists are collections. So anything that implements the list interface has all the methods that are defined in collection, as well as iterable, because collection itself extends iterable, plus some additional ones that take into account the fact that now we're adding kind of an ordering to the whole thing. So now that we have this concept of an index, we could get a value at a particular index. So you give it an index, and it will return the value type does the generic any type, return the value at that index, the set method, which is distinct from remove, goes to a given index and changes the value at that index that's not splicing something in, that's actually overwriting an existing value. Now, this set method takes the index whose value you wish to overwrite takes the value you want to put into it. And then as a courtesy, it returns the value that used to be there. Now, since list extends collection, it has that add method that collection has, which just tax the item on at the end of the list. But now we also have an add method that will add in a given index, which is the one that we had drawn on paper for the link list a little while ago. And so that one takes both an index and a value you want to splice in at that index, pushing other items over. Now, that add is void, because in the list, we just assume we'll always be able to add stuff. And then finally, there's an

additional version of remove that says remove an item at a given index. And so that's what augments our collection. That's what makes a list sort of have more functionality than a collection. Now lists also have a list iterator associated with them, which allows you to move both backwards and forwards. But for our purposes, we will only be using the

DS Lecture 3 Page 20 of 24 Transcribed by <https://otter.ai>

collections version of iterator on our lists. Okay, all right. So we can implement our lists and the necessary iterator on our list using arrays, and that would be an ArrayList. Or we can implement it using a linked list. And that would end up being a doubly linked list.

1:18:31

Like we saw on paper. So we're at the end of today's lecture. On Monday, what we're going to do is we're actually going to walk through very thoroughly the code for an ArrayList using all of these concepts we've learned. And in fact, going through the code for that it's going to be really important, because there are a couple of concepts in Java, that you will not have learned in your previous classes that we're going to need. So we're going to learn a bit of new Java. In there particularly, we're going to learn about things called nested classes. And what are a special case of that known as inner classes. Now, the way we're going to go through the ArrayList, is we're going to be using the ArrayList implementation that the waste textbook uses. It's called my ArrayList. And you can download that from the code link that's on the front page, of course works associated with the textbook, if you want to have it in front of you, while we're going through it. Once we've gone through the ArrayList code, then we're going to go through code for the doubly linked list, which is what's called my linked list in the waste code that you can download. I don't think we're going to make it to the link lists on Monday. So we'll do the link lists next Wednesday. So that's what you should be doing. If you haven't finished reading chapter three, finish reading it. Once we've gone through ArrayList and Link lists will then do stacks, queues and some of their applications, which will then will probably into just barely introduced trees, and the beginnings of chapter four before we have our midterm. So that sort of looks ahead and for the next three weeks. Alright, that's it for today. I'm going to stop the share. And I can hang around to answer questions if you want for a little while. Otherwise, good luck with your homework. Remember, that is due on Wednesday, and I will see you on Wednesday.

1:20:39

If you're gonna go ahead, take care. Dr. Blair. Shu Han, go

1:20:45

ahead.

DS Lecture 3 Page 21 of 24 Transcribed by <https://otter.ai>

1:20:46

Yeah, I have a follow up question about like how to remove items using iterators in a in a loop. I am not sure why it won't work since I mean, since we're removing the previous object. And iterator is pointing to the next object that will that will mess up the order of the

1:21:05

though as long as you're using a while loop and has next it'll be fine. If you're trying to do it inside of like a definite for loop that goes from zero to size, or something like that, you're going to be in trouble. Because that will change the size as you go. So it's a little more subtle than what I said was you just have to be careful. That's all. I'm curious. What Why does the size matter? If I'm using the loop like, on what you use as your, your end as your sort of stopping condition for the loop? Because the size variable? Sorry, the size method will return something new each time.

1:21:51

So as long as the stopping condition is like iterator.

1:21:57

Yeah, if you're close to a while loop, we're using an enhanced, think enhanced for loops actually, specifically tell you not to use removing them. But I think if you're using a while loop with an iterator, and as long as you're paying attention to as next, that should be fine.

1:22:14

Yeah, cuz the problem I I've met before, is the index just go down by one I have to go back. I can try. But if you're not doing it by index, then it's not an issue. Yeah, but I'm using you know, that makes me confused because I think I'm using either with her but like the index using an iterator. You shouldn't be using an index. No, I mean, the it's not actually indexed. I have to go to the previous iterator, like, because the follow up is like four iterator now equals to either with or without an iterator plus TAs. ish. That's s&c pasa

DS Lecture 3 Page 22 of 24 Transcribed by <https://otter.ai>

1:22:50

that's not an iterator that's keeping track of the direct reference to the nodes in the list. Yeah, because if you're using an iterator, you don't have access to the the previous field. You don't you don't use that.

1:23:10

Like the way we act. The way I access the previous is that I read it every time I eat. I use.

1:23:17

So I have to emphasize if you're using an iterator, you don't access the previous. So you're not using an iterator in the sense that I just showed. Yeah, I should go back and look into that, like, yeah, so directly manipulating the next and previous fields on the nodes. And you got to be careful doing that. But that's distinct from an iterator. Okay, as we go.

1:23:45

So if we have an interface that extends another interface that in turn extends another interface, but then the interface that we're looking at, does it need to list out the required methods of the previous interface? Doesn't that make it kind of a headache to like, when we write our classes, do we have to go down that rabbit hole to see what every interface will have to go down that rabbit hole? Okay, there's no like centralized place to see all of the requirements.

1:24:16

The Java docs, for example, it actually there are fields in there, there are sort of headers in there that says all the things that are inherited as well, so that the Java Doc's actually do consolidate that in a place where you can see them. Now, it's interesting that you mentioned that because when we look at that my link list code, you'll notice that the my or the my ArrayList either of them, neither of them actually implement list. And the reason is because list and collection have so many other methods that I didn't list out that implementing it fully would have been ungodly pain. And so instead, they just implement iterable so We can do our iterators on them. And then we just kind of hack it and implement sort of a subset of the things just for demonstration purposes. Okay. Thank God. Yes. Professor

DS Lecture 3 Page 23 of 24 Transcribed by <https://otter.ai>

1:25:13

quick question about the Remove method in iterator.

1:25:18

I just do confused how the Remove methods works. And just for example, if the iterator is at the item number n , and then I do the Remove method, which item is done, remove item as is one and minus one. Yes, or Yeah. Okay. Thank you. Yeah. Because it wants to remove something that you've already seen. And the current item where the iterator sits, you haven't seen yet. You only have seen it once it's moved past it. Okay, so if the iterator is

add item never earned the item number, earn, it's not seen, right? You only ever see item and when you call next. And by the time you've seen it, the iterator is now an item in plus one. Oh, that makes clear. Thank you. Thank you. Cool. So you had another question or your hand just so random, like okay, anything else? Anybody else?

1:26:26

We're good. Yeah, go ahead.

1:26:28

I just wanted that like, quick question about the homework because I think you pointed out something that I might have done wrong that I didn't realize I did. Um, when you're talking about like for the second input for linear search, not like, I use it, we can't enter an integer. So, so we're searching the array for a rectangle. Okay. So in order to do that we have to instantiate or make a rectangle. Ah, okay. And then just search for that. Okay, cool. Thank you. That was it. Anything else?

1:27:08

Alright, folks, I need to like wolf down some form of dinner before my many ta meetings. So I will see you all on Wednesday. Have a good one. Thank you. Thank you. Thank you.

DS Lecture 3 Page 24 of 24 Transcribed by <https://otter.ai>

DS Lecture 4

Wed, 8/11 1:07PM 1:21:22

SUMMARY KEYWORDS

iterator, arraylist, array, method, class, index, items, capacity, size, variable, midterm, compiler, return, string, add, list, string builder, collection, instance variables, object

00:48

It's not unusual to be alone It's not unusual to have fun with anyone. But when I see you hanging with anyone it's not unusual to see me it's not unusual to go out when I see you out and about a such a crime if you sit out by anyone it's not unusual. It happens no matter what you say. You find them happy. Why can't these crazy love be more? It's not unusual to be mad. It's not unusual

02:11

to be sad with anyone. That you changed that at a time.

02:19

It's not unusual. All right.

02:44

Let's get started. Close friend. I noticed you were dancing and Chloe just looked like she was done with you. It was hilarious. Okay, um, so you guys have have your you guys have your homework due tonight? Yes. I'm good. So recall that there are late hours that can be used, they apply separately to written and programming you can use up to 48 late hours on each. You have a grand total of 150 for the semester. So I strongly advise you against burning a lot of them today. So you know, if you're an hour too late on one or the other, that's what they're there for. If you're like two days late on both of them, that's that's 96

DS Lecture 4 Page 1 of 22 Transcribed by <https://otter.ai>

late hours gone out of 150. And not to be all doom and gloom, but you're gonna need him for homework three. just just just just saying. So there's that. Um, okay, so office hours seem to be going pretty smoothly. Last semester. I think we ran into some issues with them being kind of overloaded, but I think we adapted pretty well. So hopefully that went okay. I will post homework two, which will be based on the stuff we've been doing later tonight or possibly tomorrow, I I will see. But it will be coming soon. And should be due approximately two weeks. From today. I have a little bit of leeway. It will be due such that

the late deadline will hit a couple of days before the midterm range so that you get a chance to see the solutions to homework to before the midterm hits. And you all remember that I'm giving a two day range during which you can take your midterm sitting a bell, it's the Monday and the Tuesday the week before spring break. Now, one thing to note is that that kind of straddles the lecture. And I feel like it's a bit weird for me to give a lecture in the middle of what is on your honor closed book, midterm exam that's ongoing at that point. So my plan is for the Monday lecture that would otherwise be given during that week, I won't hold a lecture at that time. And instead, what I'll do is I'll pre record some material, probably somewhat in advance, but it will be material that's not on the midterm. And then just post that pre recording, probably the weekend before, and then you can just watch it whenever you're done with the midterm. And then that Wednesday, before spring break, we will have a class does everybody good with that plan? One option, and maybe I'll do that the other possibility, I was planning on doing a review session separately from the regular class time, the week before the midterm. So it might be excessive amounts of extra material to have a lecture. Let's see how we're doing. But there will be a review session that I'll do it at different time. We'll try to schedule that somewhat in advance, but but I'm not quite there yet. And we're still about two weeks away. Okay. Um, so that's the basic plan.

06:54

Any quick questions for me before I move forward? No, nothing. Okay. My wife is frantically stuffing on envelopes for a male and behind me, are you people hearing that or no? my mic is not picking that up. Somebody is actually hearing a little bit of it. Okay. All right. If it starts to become really annoying, I'll tell her to stop. But for now, I stopped bothering me so much. But just in case, you're wondering what that fluttering in the background is. Okay. Um, alright. So what we were doing last class is we had spent some time going over on paper web link lists to sort of wrap up our conversation sort of in theory of our how our ArrayList and linked list implementations work. We talked a bit about the collection interface, and the list interface and the iterators. We talked a lot about iterators. And how those kind of were realized in Java. So what I want to spend today's lecture on, is I want to start going through some implementation details of ArrayList. And then possibly linked lists, I have a feeling I'm not going to make it through

DS Lecture 4 Page 2 of 22 Transcribed by <https://otter.ai>

ArrayList entirely, because there's some new concepts in Java, that you likely haven't seen before that we have to talk about, which may take a little bit of time. And then on Monday, today's Wednesday, right? Yes. on Monday's lecture, we'll pick up and do link lists. And that means for the rest of next week, we'll do stacks and queues. So that means the week before the midterm, we will start with our trees. So you can read a little bit ahead into chapter four. But please understand, it's going to be about a week and a half before we actually start that but some of chapter four will be on the midterm. So if you want to get some reading ahead, done, that's what you're doing. Okay. All right. So what we're looking at is code from the code package that was available. associated with the textbook. This was a file called my ArrayList dot java. So if you want to have it open in front of you, you can pull it from the site, I linked on the front of coursework. So this isn't something I'm going to post sort of separately. In the File section. We're just linking to it directly on the front. Okay. Um, and, Alright, so, what we're trying to do here is, we're trying to

09:46

sort of show what an ArrayList implementation looks like. And so instead of having an ArrayList class, which exists in Java dot util, we're writing a class here called My ArrayList. And what we want to do is we want to kind of have this simulate what it would be like to implement the list interface. However, the list interface, which also extends the collection interface, which also extends the iterable interface has lots and lots of methods in it even

beyond the basic methods we saw when we were going through examples of them last lecture. And so because of that, I'm not actually going to have my ArrayList directly implement Java dot util dot list, just because there's too much. So I'm going to implement a lot of stuff from it. But I'm not going to explicitly implement like in code with the implements keyword, the list interface, but the one thing I do want to do is I want to implement iterable. Because by inter implementing iterable, I'm tagging this thing such that we know iterators can be used on it. So for example, if I wanted to use an enhanced for loop, which we had gone over a little bit of how those work with iterators last class, if I wanted to use an enhanced for loop, I could use the enhanced for loop on the My ArrayList. But I have to implement iterable for that to happen. Okay. All right. So public class, my ArrayList. Now, my ArrayList is going to be generic. But it's a little different from the generics that you've been dealing with in the programming homework. These are not method generics, these are class wide generics. That is, we declare a type variable, in this case, any type associated with the class. So when you instantiate this class, you have to specify the actual concrete type you want in place of the type variable. So you would in this instantiate my ArrayList, exactly the same way you would instantiate ArrayList, where you have to put inside of those angle brackets, the type that you're going to be storing in that ArrayList. And we'll use any type as the variable to reference that type, whatever it

DS Lecture 4 Page 3 of 22 Transcribed by <https://otter.ai>

happens to be inside the code. Does everybody okay with that? Alright, so, before we really can get into these methods, I need to talk a little bit about the instance variables for my ArrayList. And before I even go down there, um, you should be able to tell me what my instance variables ought to be. Anybody have a suggestion? or what not the names, but what do we need? Okay, well sighs, it, we definitely need to know how many items have actually been inserted into the ArrayList. So we'll need an int that will store the size and we need one other thing, which is critically important. We need the array, right. So we need an array of any types, whatever those are, that will have some initial capacity. And then the size will start at zero, and then we'll start filling that array up with items as we add things to it. And we'll fill up those items until the array is completely filled. And then we extend the array. Okay, so that's the plan. So let's look at our instance variables. So we'll jump down. And in fact, before we even get to the instance variables, we have one of these private static final variables. So a static variable is one that is class wide. So there are different copies of it for each instance. And final means it can't be changed. This is a hard coded constant. And by convention, when we have constants, we usually use the variables for those in all caps, that sort of a thing that Java inherited from C, and other languages. You don't have to make them that way. But it's good practice. And so we'll have this int called default capacity, which will be the initial sought size of the array that every time we fill it up, we'll double. And we'll initialize it out at 10. Sound good? All right, we have an array that makes up our, our collection of items known as the items. I am not enamored with the name of that variable, but that's what the textbook gave me. So that's what I'm dealing with. All right, and you'll see that the type of that is in any type array. So it's an array of whatever we instantiate this class as, and then separately, we have an int called the size that We'll store the current number of items that are in the array. Okay, that's not the capacity, that's just the items dot length. This is the number of current number of items that are actually set. Now, the reason why it's called the size instead of size is because we actually need a method called size that that comes from, that actually comes from collection, we need a method called size that returns that to the user. So we don't want to name conflict. Okay, so that's it. Those are our instance variables and our one little constant. And now what we need to look at is how we construct this thing. So we go back up, we have our constructor, and our constructor actually invokes a method called do clear. Now, what I'll say is the following. We know from our study of the collection interface, that collections in general, including lists are supposed to have a method called clear. And what clear is supposed to do is it's supposed to dump out

whatever's in the collection, and give you a fresh one to start with. And so the way that Weiss, the author of the textbook decided to design this, and this is actually pretty good. He recognizes the fact that emptying the array out is basically the same as creating a new one. And so both the public clear method, which is part of the collection interface, which actually empties the thing out, and the constructor, which constructs it from scratch, are essentially doing the same thing. And so those two separate methods that are doing the same thing, they just both call the same do clear method, which is a private method that just handles the work. So let's take a look just to confirm for ourselves.

17:00

There is a public clear method that in turn invokes the do clear as well. And so our private method, which sort of functionally decomposes out that that functionality, do clear does two things. The first thing it does is it sets the size equal to zero. Now, before we get into ensure capacity, can I just ask if I have an existing array? Say, the array has 10 elements in it? Because it's using the default capacity and it already exists? And I filled up five of those items. Do you think that I actually, if I clear it is setting size equal to zero sufficient for clearing it? Or should I know out the items in the list? Anybody.

17:56

So somebody says, and I'm quoting here, now it's just a VAR. And that is actually kind of right. Um, if size is set to zero, it's set to zero, then all of the other methods will give you an array index out of bounds if you attempt to access something outside of that, right. So setting the size to zero is effectively clearing it. Now, separately, we have an ensure capacity method. And the ensure capacity method is to make sure that the array is big enough to contain what we want it to contain. Now, this ensure capacity method, it may not be immediately obvious why they're bothering to use it here. And frankly, I'm actually a little surprised they did it too. But it makes sense. This can be used almost anywhere. What this method does is it says, okay, we want to make sure that this array list can contain whatever I pass into it, number of items. So I could pass in, say 1000 items. And its job is to make sure that the capacity is at least 1000 items, independent of what's inside of it. So this actually can be used to resize the array as well, when it comes time to, you know, resize if we fill our capacity up. So what ensure capacity does is it first checks to make sure that well, we don't already have enough capacity if the array already exists, and the length of the array is greater than what's already here. Sorry. Yeah. is greater than or equal to what's already here. We don't need to do anything. It's already begun. I'll leave it alone. But if it's not big enough, it will allocate a new array that's big enough, and then copy the items over from the original array. That's all it does. But when it does that, it's gonna have to instantiate a new array. So we can kind of leverage that to also work when we're creating the thing from scratch, right? Because when we're creating the thing from scratch, there are no items in it yet. So it just has to allocate the initial. Alright, so let's take a look at ensure capacity directly. So it takes in a capacity. Now, you'll notice that there's this little x suppress warnings unchecked, at the front. And I'll get into in a minute what that's about. But I will just tell you, when you have a line that starts with an X sign,

DS Lecture 4 Page 5 of 22 Transcribed by <https://otter.ai>

that actually acts as a compiler directive, it tells the Java compiler something about what you're about to do. In fact, what this is going to do is it's going to tell the compiler, yeah, I'm about to do something you're not going to like and you're going to throw a warning about, just shut up. That's really what it's telling it to do. And so I'll point out the specific line that will cause the compiler to have a fit. And that lot, this line that I've highlighted, his job is to shut the compiler up, because we actually allegedly know what we're doing. Okay. All right. That sounded actually surprisingly hostile towards the compiler. I don't know what got into me there. But moving on. Um,

21:46

Alright, so first of all, if the new capacity, that's the value that's being passed in here, is less than the size. And so the size, mind you is not the current capacity of the underlying the items array, the size is the number of values that are actually stored in there. So if new capacity is less than the number of values that are actually stored in there, then we already are big enough to hold that capacity because we already have more in there than what we're asking to check for. Part of the reason why we check against the size rather than the existing capacity is because it allows us to catch the case when we need to reallocate it. All right, because remember, sorry, when we need to allocate it initially, because remember, when the constructor starts, the items is no. So if you did the items dot length, you'd get an old pointer exception. But by checking against the size, there's no issue there. So if the capacity that we need is greater than the number of items we already have in there, then it may be necessary for us to resize. So the first thing we do is we create a temporary variable called Old. And that's going to be typed as in any type array. Remember, arrays are just object references. So the items is a reference to a particular array in memory. So when I say old equals the items, I making the variable old, reference the same location in memory as the items, they now both point to the same array. So far, so good. Okay, once I've done that, I've now basically stored myself a temporary reference to the original array, which when we're doing this through the constructor was actually no, but that's okay. It just means that old would be null in that particular case. And then I allocate a new array and set the items equal to it. So this is where the items and old diverge old will remain referencing the original array or know that we had. And while the items will now reference a brand new array big enough to hold the capacity we actually want. Now, there's a little bit of weirdness here that might immediately catch your eye, you will notice that I did not create an array of any types. Now, this always gets people and I will just sort of stated as a constraint on Java. You cannot.

DS Lecture 4 Page 6 of 22 Transcribed by <https://otter.ai>

24:41

So not quite, there's no throwing here. But it's by casting not throwing. Maybe that was the word you were looking for. Yeah, by casting. So be objects. So you remember object references. It says the top level hierarchy of everything in Java. You can actually cast any reference any object to a reference to anything else. Because it's the parent of everything that I can sorry, the great grandparent, whatever, it's the top of their. So you're not allowed to instantiate an array of a generic type variable, you're just not allowed to do it. It's a restriction in Java, whatever thing that I have here has to be known what it is. And so I create it as an array of objects of size, new capacity. But the nice thing about creating an array of objects is that I can cast object references to references to anything else. And you know, the array of objects doesn't have actual objects in them yet, it just has no references to objects in them. So I can then safely cast my array of objects to an array of any types. And that's legal. And then that makes sure that the type on this side of the of the assignment operator matches what the items actually wants, which is an array of type any type. Okay? Now, this right here, though, is where the compiler pitches a bit of a fit. The problem is, is that you can cast a reference type this object to anything. But you can also have a variable type does Object Reference anything. And so what's actually stored in that array, as far as the compiler knows, might be something other than object, it might be a string, you can have a variable type does object and set it equal to a string just fine. But then when you cast that variable that was originally typed as object to something else, you could wind up with a situation where you try to cast it to something that can't be cast it to. So if object x, x is actually pointing to a string, and then you try to cast X to integer, you're going to have a big old problem. And so the compiler recognizes that you have to be sure that whatever type is passed in here, actually can safely cast whatever was originally stored in there. And so it'll throw a warning. But because we control very

carefully in our own code, what we actually put in here, so we know that will never be a problem. We're okay with that we're willing to accept that risk. So we suppress the warning. All right, so we cast our array that's initially set up as objects to type any type. And now the items is equal to a brand new empty array of length, new capacity. I then go through the old array, up to its size, meaning up to the elements that are actually filled in the old array and copy them over into the new array. So that works just fine to grow an array. But if this were being called from the constructor, what is the size when the constructor does all of this? Anybody? Nope, it's not 10. It's, it's zero. Yes. 10 is the new capacity variable, but the size is zero. So how many iterations of this loop will actually happen? If this gets invoked via the constructor, zero times. So it will never actually attempt to index into old so it's okay that old references null in that case. So this works just fine to allocate the initial array. And it also just works fine to grow arrays when you need to grow them at a later point. Any questions for me on ensure capacity? So if you want to ensure that the capacity is 10, and you have 10 elements in the array, Uh, oh.

DS Lecture 4 Page 7 of 22 Transcribed by <https://otter.ai>

29:22

You're not wrong. If you do it as equal less than or equal to, that would still be okay. But this actually grows it even when it doesn't strictly need to in that one case. So when new capacity is equal to size, it looks like it will just relocate the array. Catherine, does that make sense? There's nothing wrong with that. It just seems unnecessary. Okay, so return semi colon doesn't return anything. What's the point of having it there? That's a good question. So first of all, Ensure capacity is void, right? So it doesn't actually return any values to the users. That makes sense. So the return keyword has two functions. The first thing it does is it takes it can take a parameter, that is what the method will actually return when it ends. But the other functionality of return is it actually just ends the method. So you can have a return with with no parameter in a void method that just causes the method to end. Cool. Alright, and then Wayne asked, Do you need to reassign default capacities in a new capacity? No, you actually can't. Because default capacity is a constant. That's just what it's starting at. So we might have capacities that are larger than that later. Does that make sense?

30:55

Yes.

30:58

So is the return statement, like a break? Kinda breaks, break out of loops returns break out of methods? that that might be a good way to think about it? So somebody said, Can you explain why do clear calls and short capacity? Um,

31:18

let me think. So, a mat, so remember, do clear is being called by the constructor, right? Justin? See, when the constructor runs, the instance variables, least the object reference instance variable start out as null. So the items is a null reference, there is no array created. So from the constructor anyway, the purpose of ensure capacity is actually to initially create the array, which before didn't exist with the default capacity. Does that make sense? Yeah, so I guess a better way of saying is you absolutely have to call and share capacity. Yeah, so you absolutely have to call and share capacity for my rails, no way around it. I'm set from the constructor. From clear, it's a little less clear why that's necessary. But it doesn't really matter. Because since the end, when you call clear, the array will have already been initialized. So the capacity will be at least the default

DS Lecture 4 Page 8 of 22 Transcribed by <https://otter.ai>

capacity. So ensure capacity will essentially do nothing. The reason why, I guess they did it is just so that my ArrayList and clear, could have the same code inside of them. But that's not strictly necessary. So that okay, maybe not the most satisfying answer. But that's, that's why it was done. Alright, so we've got our setup, ensure capacity. Remember that

method, because that's going to be useful for us. As we add stuff to this as well. Let's run through some of the other stuff we have here. First of all, sizes simple. All the size method has to do is return the size instance variable that tells us how big it is. All these empty method has to do is return whether or not the size variable is zero. So it invokes the size method to do that, all good. All right. Now we get to two methods. These are do these first, because they're sort of simpler, we get two methods that actually come from list those first two size and is empty, and also the clear one that we looked at a lot earlier, all come from collection, which we're trying to sort of pretend we are extending list, which means we're pretending we're also extending. But the get method. And the set method, which are now the next simplest methods to see are actually coming from list, because they make use of the fact that there's an underlying index going on here. So let's look at our get method initially. So first of all, and you've probably encountered this with ArrayList. And I know you've encountered it with arrays is that if you attempt to access an element in an array or a list that's outside the correct bounds, you get an array index out of bounds exception, that's an exception that actually exists in Java dot Lang. So that's part of Java. And so we want ours to follow that. To follow that that pattern. So we are getting a particular index, so we check to make sure it's within range. So if the index is less than zero, or greater than or equal to size, because remember, only zero through size minus one are valid locations. If it's outside of that range, we throw a new index out of bounds exception and just to double check, you've learned how to deal with exceptions at least somewhat in your 1004 class, correct? Yes, at the very good. Um, oh my lord. Sorry, I just did something terrible. Um, I glanced at my second monitor, and stupidly glanced at my email window to see my email, which I had checked. While our music was playing at the beginning. And since the class started, I have already received Oh, my lord looks to be something like 25 new emails. That's my life. That's why I suck at email, because like, How am I supposed to deal with that? amusingly enough, 80% of those messages are to our classes. peoc support. So I see you guys are very active, as are the TA is while I'm talking to you. That's fantastic.

36:05

All right, moving on.

36:09

DS Lecture 4 Page 9 of 22 Transcribed by <https://otter.ai>

So

36:12

Oh, shut notifications, I just left the web browser window open. Oh, turning off piata emails. Um, yeah, for your own sanity, I think you should actually put piata emails on digest, that sends you a digest maybe once every eight hours or something like that, to see if you get a sense of what's there, but not get bombarded. The reason why I don't do that is the cause I need to know when you guys post stuff, so I can respond to it in under eight hours. I mean, I think we've got about a 15 minute response time, but I don't think you really want us to jump up to an eight hour response done. Anyway, um, so we were talking about exceptions. And what I will say, and I don't know how much he did of this, but you don't need to know that much about it is that exceptions are in fact objects themselves. So when an exception gets thrown, what actually happens is that an object, which is a particular type that extends either exception, or runtime exception, in this case, it's a runtime exception, meaning one, you don't need to use a try catch four gets instantiated and then thrown. So array index out of bounds exception is a class that extends runtime exception. So we just instantiate a new instance of that, and throw it. Okay. So once we've done that, and we're certain that our index isn't out of bounds, we then just extract the index from the items and return it to the user. Nice and simple. That's the beauty of an ArrayList. The underlying array, any location can be accessed in constant time, which we've just done, we've just returned. And that's it. Forget, any questions on the

get?

38:11

Good. Okay. Okay.

38:19

Set, very similar deal. The idea with set, if you recall, is that it takes an index and a new value that we are supposed to replace the value that was previously at that index. And then as a courtesy to the user, we're supposed to return the value that was originally stored there. So the first thing we do is exactly the same as what we do in the get method, that index has to be part of the valid range. Otherwise, there's nothing to set because we're supposed to be replacing an existing thing. So same bit of code that we had above just checks the range and throws an exception if it's out of bounds. We then store the original item as old. Then we in order one time index into the array and replace the item with the new item and return the old item. Are we good with that? Okay, cool. So now we

DS Lecture 4 Page 10 of 22 Transcribed by <https://otter.ai>

get to the slightly more complicated parts of this. So what we're going to do now is we're going to do add, and in fact, there are going to be two add methods, but that shouldn't come as a surprise. Why are there two admin? Yeah, add methods anybody? One at the end and one index specific good, where are they coming from? What requires us to have those two silences you don't remember, makes you hear the letters being packed more clearly, ah, collection, right, there was an add method that takes just a value and just tack it on at the end that comes from collection. And then there was a separate add method that came from list, which takes an index. And the goal when you use the list based one is to slide everything that was originally in that at that index over and everything else after it over, and then put this one in. So the size of the array should increased by one when you added a given index, which is different from setting. And so this is the collection based add, this is the one that just takes a value and Chuck's it into the collection bag. Right, except when it's a list, chucking it into the collection really just means tacking it on at the end of the list. So we really shouldn't be duplicating work. So the way that we tack it on at the end of the list is we leverage the fact that in a minute, we're going to write another add method that adds an item at a given index. So if you want to add an item, right at the end of the list, you want it to become location size, right? If the existing ArrayList has items zero through size minus one, and you're tacking something on at the end, that new items should go at location size. So what I do is I just invoke the Add method that comes from list and say, Okay, great, at location, size, add x. And that's it. Now, the collection based ad is supposed to return a Boolean true or false as to whether or not the Remove was successful, which isn't really relevant to lists. And so since we know it's going to be successful, we simply return true, because it worked should have worked. Okay, is everybody good with the collection add? Okay, next, we have another add method, which is sadly a void method because at least with lists, sorry, so why not make it a void void method owner, the reason why we don't make it a void method is because it's coming from collection. And there are a lot of other collections, other than just lists. And there exists some collections that you sometimes can't add things to despite your best efforts. And those you really need to tell the user that had failed.

42:47

Amr Does that make sense? Cool. And she'll hang why not return to add size x? Um, that's actually a good thought. The problem is just that the other add method is a void method. So there's nothing to return there. So that's, that's why we sort of No, no, no worries, it's actually a really good impulse. Like, when you're sort of acting as a proxy for another method, you should take what the other method gives you and return it back. It's a really good way to think about things when dealing with recursion, which is what your homework has tonight. But nonetheless, in this one, it doesn't quite work. Okay, so so why isn't the other ad not Boolean to maintain consistency? Um, you know,

43:29

that's

43:31

a good question. Um, I think they could have gotten away with that. It's just that in the list paradigm, the whole sort of conceit of a list is that you should be able to add an arbitrary number of items to it. Right? So in that sense, there really shouldn't be a case when it fails. So by making it void, it kind of doubles down on that.

44:01

When you say this, the collection ad deal with a full list, the truth of the matter is, there can't be a full list, because lists will happily expand until we run out of memory. Right, will just keep ensuring capacity. Right? So ensure capacity allows us to reallocate the array with the largest size if we want. And so our our list based add will just keep growing the array. And in fact, if you think about linked lists, which we did on paper, but not in code, those aren't even constrained by an array, you just just keep allocating nodes, right? So until you run out of memory, it's just not going to fail. And frankly, if you run out of memory, returning something is probably the least of your worries. At that point. Other things will have broken horribly.

44:51

Okay.

44:53

All right. So now we're on our add method. To our edge method, this is the one that comes from lists, it takes an index at a value that you want to become that index, but it's not supposed to overwrite anything. So what used to be at that index has got to get shifted over to the right. And then everything after it has to get shifted over to the right. So this add method at an arbitrary location is guaranteed to be order in, because we got to move everything over for it. This is also the moment where we need to consider whether the array needs to grow. Because this is the part where we're at, we're definitely going to make the size increase by one. So at this point, the size could potentially go beyond the

DS Lecture 4 Page 12 of 22 Transcribed by <https://otter.ai>

existing capacity of the array. And therefore we need to potentially reallocate the array, which is also an order and penalty if it happens, and copy everything over. So that's now by the way, this piece of code doesn't have the index out of bounds check in it, but probably should have, it's an omission. Keep in mind that it's probably better to err, check that, but we didn't. So here, we're going to just check the size. So if the length of the items, that is the capacity of the underlying array is equal to the size, it's the current size, before we add this new item, it means there is no more space available in the array. And so we have to grow it. Now in this code, what we're actually going to do is we're going to just about double the size of the array each time, which is a reasonable thing to do, I will tell you that ArrayList in Java actually multiply the size by one and a half, not two. And they probably have some empirical evidence to show that that in practice is a little more efficient, but we're going to double it. So someone privately asked me why plus one, so let's talk about that for a second. So you'll notice here, I insure capacity with a new, larger size. If I fill the capacity up, we already looked at the insure capacity method. So we know how it allocates the new array, and over and copies everything over. So we don't have to worry about that. The only thing we had to worry about is the parameter to ensure capacity. And in the parameter to ensure capacity, you'll notice we say we want that capacity, we the size times two, but then we do plus one. So it's actually size times two is going to give us double the amount of free space plus one additional spot for the item that we're adding, it didn't really need to do that. But it's that plus one is for the item that we're adding. Sounds good. So at this point, we are certain that the array is big enough to contain what we want. And so now we have to actually make room for the item that we

want to add. Right. So if we want to add an item and index, the item that was originally an index has to go to index plus one. But then the item that was at index plus one has to go to index plus two. And so to avoid having to like actually keep attempt when you're doing the swaps, it actually makes sense to start at the end, you start at the end, and then you copy the last element one further down into the array, then you keep moving in copying one further down into the array until you copy the item that was at the index over by one, therefore leaving a space that we can safely drop our new item in. And so the way we're doing that we have a loop that actually starts at the size, right, the size hasn't increased yet. So that's actually one past the last valid element, which is good, because we want to move into that space. So we've started the size, and we keep decrementing i until we reach index. And so for each spot, what we do is we set the items that i equal to the items that i minus one, so we go all the way one past the end, then copy the thing before it into that spot, and we move backwards by one, go further back, copy it into that spot, and so on until we've moved the value index out of the way. Once we've moved the value in index out of the way, we set the items at index equal to the new item that we want and at this point we increment the size by one

DS Lecture 4 Page 13 of 22 Transcribed by <https://otter.ai>

49:58

m

50:01

Actually, David, it will still cause problems. So it really should have had a check in there. Um, Omar says Do we lose that last value just moves to the new space, it moves to the new space, the very last value exists at location the size minus one originally correct. And so you'll notice i starts at the size. So in the first iteration of this loop, the value at the size minus one gets copied into the value at the size. So that last value isn't lost, that last value is moved over one spot. Good. Cool. Any other questions on this ad. And so for an arbitrary location, this is order ran, whether we have to reallocate the array or not, because of this loop right here.

50:59

All right.

51:02

Now, we're gonna sort of fudge it a little bit with remove, there exists a remove method that comes from collection, which removes by by value, and we're just not going to bother with that. The only remove we're going to actually bother with right now is removing by index, that's the list base remove. So in this case, again, we probably should have checked to make sure the index was valid. But the job of this remove method is to take the value that was originally at location index, store it temporarily. Get rid of it in the array, shrink the size by one, and then return what you just removed back to the user. So we temporarily store the items that location index in a variable called removed item. And then we do the reverse of what we did in the previous one, we start at the end, sorry, we start at the index the value we want to remove, and work our way to the n minus one, because the array is gonna end up one shorter than it was before. Sorry, the array list. And then we set the items that i equal to the items equal to the items at i plus one, so we're copying it back by one. So that first copy clobbers, the value that we want to remove, then we move over by one copy, move over by one copy, and so on. And then eventually, we move the value that was originally in size minus one into size minus two. And we're done. All we need to do at that point is decrement, the size variable by one and return the item we removed. Questions. Okay, so we got our ads, we got our group. So it is generally considered good practice. When you have a method that removes data from a data

DS Lecture 4 Page 14 of 22 Transcribed by <https://otter.ai>

structure to have as its return, the value that you removed, just in case the user wants to potentially use it for something else. Saying, does that make sense? You'll see that

everywhere that we clobber values. You saw it with set. Okay. So that takes care of the easy part. Now we're going to have to make our way into iterators and iterators are where things start to get a little tricky. We already looked at do clear and clear. We saw those previously. And here comes the iterator. Okay, so we'll start with this method. This method is a method that we are required to have, because we implemented the iterable interface. Remember what we learned last class was that if we implement the iterable interface, we are required to have a method called iterator. And that iterator method, its job is to return an object of some type, as long as that object's type implements the Java dot util dot iterator interface, meaning whatever it returns has to have that next has next and move method that we talked about last class. Okay? And whatever that iterator object is, it's got to be tied back to the particular instance of the my ArrayList on which we invoked the iterator method. So what this method does so saying I have a my ArrayList called a, if I say,

55:10

Actually, let me

55:14

let me just type something up here real simple. So let's say I have my ArrayList. A, let's say it's a my ArrayList of strings, a equals new my ArrayList. Then we do a dot add john. A dot add, Paul. Alright, so now we got stuff in there. Now if I want to iterate across that ArrayList, I can create an object. Type is the iterator interface. So it's an iterator on strings. Let's just call it eater.

56:04

That's, uh, I know a lot of John's man. So I always do john paul. I. Okay, so just to be clear, I'm not Catholic. I'm not even Christian. But I always don't to john paul, because the only Pope that I haven't real conscious memory of when I was a kid was john paul the second. And then I'm also Paul. And then my cousin is john. So I always start with john and Paul. Alright. So you didn't need to know that. Don't care, whatever. Moving on. I have no cousins or family members named Ringo. Sorry. I'm also He's my least favorite Beatle. Sorry. My apologies if anyone is actually related to that guy. Moving on. So we have an iterator. And so we set that equal to a dot iterator. So you remember we did something like this last class? The idea here is that this thing that a dot iterator returned is an iterator

DS Lecture 4 Page 15 of 22 Transcribed by <https://otter.ai>

on a it has to be tied to a somehow. So when I call inner dot next, and so on, it will be giving me values from a is everybody okay with that? Cool. All right. So let's go back to that code. And so Hallelujah, we have a class called ArrayList iterator that apparently does all the work. So we just renew and return a new instance of ArrayList iterator, which we know, implements iterator and gets tied to the current ArrayList. And we're good. And we're done. And that's that's it. That's the concrete class. Right? Well, sure, but we have to write it. So we're going to have to create the class ArrayList iterator in a minute, but this will actually instantiate it for us. So before we actually get into the implementation of ArrayList iterator, I want to show you an example of one of the sort of the fun things we can do. When we have a method sorry, when we have a class that's iterable. If a class is iterable, we can use an enhanced for loop on it. So you all should be familiar with this two string method. The to string method is a method that all classes inherit from object that allows a string representation of that object to be returned. It's used typically by System dot out dot print line, or things like that, to display the object. And as you all know, and probably ran into, or were rudely reminded of when you were doing your rectangle class. For the homework due today, the to string method that you inherit prints out what's effectively the memory address of the thing unless you override it and provide a better implementation of it yourself. And so what we're doing here is we're overriding the to string method that will actually go through and return a string, that is a meaningful representation of the value stored in this ArrayList. Okay, so that's our goal. I suspect you

haven't used a string builder before, if any of you use string builders before. Yeah, probably not. So you should be familiar with the fact that you can concatenate strings together just by using the plus operator right? problem with that is that when you concatenate strings together with the plus operator, every time you add two strings together, it has to allocate a new string that is of length, the combined length of the previous two strings, and copy them in. So when you repeatedly add strings together to sort of build up a larger string inside of a for loop, that gets really expensive, because each time you iterate The for loop, it has to reallocate a new temporary array for the next

1:00:35

for the next concatenation. So string builder is just a, it's a nice class that doesn't reallocate every time, it creates a buffer, and it just keeps adding your content to the end of an internal character array. And then when you're done appending things to it, you can convert it to a string, I'm not going to go into a great bit of detail about how to use a string builder. It's pretty self explanatory. Just look it up in the in the Java API if you need more information. Regardless, all we're trying to do here is we're trying to concatenate together the string based representation of all the objects in our array. And so if we want to make a list of these things, we'd have to iterate through our underlying array to display stuff. Ah, but here's the fun part. We create our string builder. And then we just iterate

DS Lecture 4 Page 16 of 22 Transcribed by <https://otter.ai>

across ourselves, using the enhanced for loop. Remind me what this is this keyword. What's this? It's not this class, but it's the current instance. All right. It's the implicit parameter to the method. It is the instance of my ArrayList that two string was invoked on. So when I switch to the other screen, and I showed how to iterate a how to create an iterator on an ArrayList, a, I can create an iterator on this the same way because this is a reference to a my ArrayList and my ArrayList, sir iterable. So I can just use my enhanced for loop, which is using an iterator on myself. And then for each item that I happen to store, I append it to my string builder. And then when I'm done, I just create a string out of my string builder. Okay, so this is just leveraging the fact that I am iterable.

1:02:39

Okay.

1:02:41

All right. So now we get to something that's completely new, which is going to be our implementation of the ArrayList iterator. Remember our iterator method instantiated ArrayList iterator. We said that ArrayList iterator must implement Java dot util dot iterator interface, but must also maintain a tie back to the underlying my ArrayList that I am iterating across. And so this is a very, very particular kind of class that I'm defining here. It's what's known as there are two names for it. It's a type of nested class. So a nested class is a class that's defined inside of another class. And so you'll notice that my private class ArrayList iterator is indented in, it's inside the body of my ArrayList. So it's nested inside of it. Now nested classes come in two varieties, static nested classes, and non static nested classes. This particular nested class, you will notice does not have a static keyword in front of it. So it is what's known as a non static nested class, there is another name for non static nested classes, there are what are known as inner classes. So this is an inner class. And the critical thing that having an inner class does is that an inner class retains a link to the instance of the outer class in which it was created. So because it's non static, you can only instantiate it from other instance methods, you can instantiate it from static methods, and so it retains that tie. So that means from within my inner class, I can access instance variables of and invoke methods on the outer class. And so that's how I actually am able to move through and access elements from the my ArrayList. I'm iterating on because of its status as an inner class, I can just directly access those variables.

1:05:16

We're good with that.

1:05:19

Cool. All right. So if you recall in our conversation of iterators when an iterator gets instantiated, hold on one second. So Daniel, you said, Does this refer to the instance of the outer class or one of the the instance of the inner class actually retains a tie to the instance of the outer class? Daniel, does that make sense? I mean, like, if we use the keyword this, would that reference the instance of the if. So, you're getting slightly ahead of me, if you use the keyword, this by itself, it's referring to the instance of the inner class. If you want to get the instance of the outer class, you actually have to do see this little thing I've highlighted here. Oh, I see. Okay, it's a messy syntax. And we were low enough on time. That's probably where we'll start next class. But there's this, which refers to the inner class, and then the name of the outer class dot this will refer to the instance of the outer class. I see. Thank you. Cool. All right, um, says we can only instantiate this class by a dot iterator, right? So we can only instantiate this class from methods within the outer class. And it turns out that in this case, the only one we do it from is the iterator method. Yes. Good. Yes. Because it's private. If it were public, there actually would be a way to instantiate it from the outside world, but we really don't want that. Okay. Cool. All right. So what you should remember is that, when we did iterators, when they're created, what we said was they latch on to items zero of the list in some way or another. In a linked list, that would be a reference to the node that contains items zero. But this isn't a linked list. This is an ArrayList. So all you need know here is the index that it is currently latched on to that is the item that is current. So whatever value is stored there is what's going to get returned to me when I invoke next. Okay. So when our iterator is instantiated, we start out latched on at current item equals zero. And then the only other instance variable we need locally in our inner class, Is this okay to remove. Remember, we're only eleven. Sorry, I said that wrong, we're only ever allowed to remove the item that is immediately before us. And we're only allowed to go one step back without taking another step forward. The reason for that is that when you're at the beginning of the list, you can't remove because of current zero, you try to remove it, we try to remove an item at location minus one, which doesn't work for us. So okay to remove is a flag indicated that we have in fact moved forward at least once, since either it's been created or the last time we removed, meaning that it is safe to invoke the Remove method. Initially, when current is zero, it ain't safe. So it just sets that to be false. All right, so we had three methods that we need to be able to have here we have a has next has next returns true. If the current item is a valid element of the list has next returns false if it's beyond the end of the list. We have next which returns the current item, and then steps to the next item. And then we have remove that if it's okay to remove will remove the item directly before current. So the first one has next is simple. If current is less than size than current is a valid element. And so it's true. You will note by the way that there is no size method in my ArrayList iterator class. So where is the size method that I'm invoking coming from? Not so much inherited but the outer class,

DS Lecture 4 Page 18 of 22 Transcribed by <https://otter.ai>

right, we said we're allowed because we're an inner class to have access to the instance methods and instance variables to the outer class. So this is actually an A method call to one of the methods With the outer class of the containing my ArrayList. Okay, so we check to see whether or not the my ArrayList were attached to its size is greater than current. If not, we still good. Our next method, well, you can't return an invalid item.

1:10:20

So if has next is false. It just throws a new Java dot util, no such element exception. And just, that's by convention, what a list iterators do. All right, or what iterators do. So if there is nothing left, it throws an exception. Otherwise, it sets okay to remove to be true, because we're about to take a step forward. And then it returns the items at current plus plus. Now you may remember, the plus plus operator increments a variable by one. But

the timing of when that increment happens is going to depend on whether or not the plus plus is before the current or after the current. Right, if the plus plus is before the variable you're trying to increment, it increments first, then evaluates the containing expression. If the plus plus is after the variable evaluates the containing expression first, then increments. So by saying the items at current plus plus, it returns the value at location, the original current, and then once that's gone, it increments current by one. So that consolidates returning the original one, and then taking the step forward into a single call. Any questions on that? All right, so somebody asked, you not need to do my ArrayList. This before using the size method, because there is no size method in the iterator class. That's right. If there's no name conflict, you don't actually need to specify the my arraylists thought this, you only need to do that if there's a potential name conflict, meaning there are multiple methods of the same name. Good. Alright, so the last thing is the Remove method. And in the Remove method, first of all, we have to make sure it's okay to remove. If it's not okay to remove meaning we're at the beginning of the list, or we already removed something and haven't moved forward yet. It's not okay to remove, which we track with this variable up here. We throw a again, this is by convention, this is what all iterators do, we throw a new illegal state exception. If it is okay to remove, well, we don't have to actually write the code to remove it from the underlying my array lists underlying list, because we already have a remove method, right? The remove method that we wrote in my ArrayList takes an index and gets it out. So my ArrayList dot, this refers to the instance of the outer class. And this time, we do need to do it because we're calling something that has a name conflict. And so we call the Remove method from the outer class. And we say remove minus minus current. Now, the reason why we say remove minus minus current is because we want to remove the item that is directly before current. So we want the decrement to happen first, before we do the Remove. But then the question arises, why didn't I want to just remove current minus one, and that's because if I'm sitting at location three, and then I, my iterator removes, that's going to remove the item at

DS Lecture 4 Page 19 of 22 Transcribed by <https://otter.ai>

location two. So if I was originally sitting at location three, when the item at location two gets removed, I kind of take a step back and I ended up at location two. So current has to move back also because the item that we were originally latched to get shifted back over by one by the other remove method. Once that's done, we just flipped okay to remove to be false, because now we've taken out an element, one step behind, and we're not allowed to move forward again, sorry, we're not allowed to remove again until we take a step forward. Okay, that actually got us all the way through the my ArrayList class. So we're good there. We are at a time So what I will say good luck on your homework that's due tonight late deadline is on Friday.

1:15:07

And on Monday's lecture, we're going to go and we're going to step through the code for doubly linked lists, where we will get a chance to see a little bit more of how iterators can be implemented in other contexts. I'll try to move through that one, maybe a little smidge faster. And so we'll probably at the very end of Monday's lecture, introduce stacks. And then we'll spend Wednesday's lecture a lot of time working on stacks and stack applications. Sound good? Alright, folks, I'll stick around for a little while if anyone has any questions. But that's it for today's class. You.

1:15:57

I professor, so I don't think I've encountered warnings before. So I just had a few questions. When we say warnings in Java on when we suppress the warning on the method earlier. We did set on the code suppress warning, and then in quotes, it was, I think, unchecked. So I was wondering what that wasn't, if it's different from, you know, declaring that something throws an unchecked exception,

1:16:26

right. So an unchecked warning is it actually means something a little different. So, um, if you try to cast an object reference to another object reference, but for some reason, don't really specify at compile time, what one of those two things is, the warning you get is an unchecked type. That's, that's very different from an exception. So we didn't, then the issue here is that that any type variable, the compiler has no clue what that's going to be, if you don't actually have any way of testing whether or not whatever object references can actually be converted to whatever we made any type into. Okay, and so that's an unchecked type cast, which is a warning that Java throws. Now in this particular case, we know it's okay. Because given the constraints on the parameters to all of our

DS Lecture 4 Page 20 of 22 Transcribed by <https://otter.ai>

methods, there's no way we would ever add anything to that array that isn't of the same type as any type way we've written the code makes that impossible. But the compiler is really only looking at it locally, it doesn't look at the global logic of the thing. So it has no way of knowing that. So it would otherwise throw a warning, unless I tell it specifically to suppress those warnings.

1:17:56

Okay, make sense? So a warning will still let you compile, it'll just say something, just watch out. Be careful. Okay,

1:18:04

well, sometimes you get strict compiler flags, that will actually force the compiler to fail even in the event of a warning. If you're doing like a big group project, like like real software development, take care, and then those kinds of stricter compilation is sometimes not a bad idea. Okay. Alright, so in the chat, somebody said during the code, you mentioned a couple times at some methods came from collection or list. How did the class we wrote just wrote, no, we need to write those. So trick, truth be told it wouldn't. I was sort of simulating extending list. But I didn't actually explicitly extend list because it would have required me to implement so many more things than I actually wanted to for the purposes of this class. So we were pretending that the class implemented, sorry, the class implemented list. And if we were going to pretend that the class implements list, then we want to be implementing some of the methods that are specified by collections and specified by list. So I was telling we're ending it was coming from Fernando is that I understand now. Thank you very much, professor. Have a good night. All right. Take care. Anything else?

1:19:27

Oh, yeah. So just wondering if you could explain briefly again, the difference between current plus plus and plus plus current.

1:19:38

So it's the moments during the execution of the underlying expression that the increment actually happens. So take a look at the chat for a second. Okay. So let's say I have then I do something like this. Because the plus plus comes before the x, the increment happens

DS Lecture 4 Page 21 of 22 Transcribed by <https://otter.ai>

before the whole expression system out, plus plus x gets executed. So if I ran that x would be six at the end of all of it, and it would print out six. Agree. Yeah. But then if I did the following,

1:20:44

and then I did.

1:20:51

The increment wouldn't happen until after the expression was evaluated. So it would print out five, and then X would become six.

1:21:01

Oh, okay. Okay. Okay. Yeah, make sense? Thank you. All right. All right, folks, anything

else? Thank you, professor. Everybody, have a good night and I will see you on Monday.
You too.

DS Lecture 4 Page 22 of 22 Transcribed by <https://otter.ai>

DS Lecture 5

Wed, 8/11 1:09PM 1:24:21

SUMMARY KEYWORDS

node, iterator, method, index, instance variables, marker, stack, size, class, remove, call, reference, object, exception, list, return, arraylist, dot, midterm, fact

05:11

Alright, let's get started. So a few notes on scheduling. Homework two written just went live, it's not in grade scope yet, but it's on courseworks. Things got posted a little later than I would have wanted. So this is how the scheduling is going to go for that. Um, because I want you to have the option of seeing the solutions to the written homework, prior to the exam, we're going to have that due next Friday, which means that you can then have your 48 hours of late hours. And then the solutions for that hopefully, solutions for homework one went up, I'd asked my ta to post it. If not, I'll make sure that's up tonight. But homework two written solutions will then go live. Right after the late deadline, I guess would be not this coming Sunday, but the following Sunday. And then your midterm period is the two days after that. So unless you were planning on taking the midterm at 12:01am, of that midterm period, and honestly, there's always somebody who does that. Which is great, because they're the one who gets to find whether or not I put any typos in the thing. But that unless you're starting in a 12:01am, during midterm period, we'll have a shot at looking at the solutions to that assignment. Homework two programming, which will also come out tonight, I will not have due until after the midterm is over. I think you need to start working on it prior to the midterm, I don't think a solution set to it is going to make much of a difference on the programming in terms of your studying for the exam. So I will make that do just before spring break, it will overlap slightly with homework three, but you will then also have significantly longer to do it than I had initially planned. Is everybody okay with that. So that way, you get some more time to work on the programming part to see you get like a good 1415 days on that at least. And that's factoring in the two days of the exam where you probably wouldn't be working on it. But the written I want you to do sooner. And so I think that that split will actually not be so bad. It might also force you to prioritize doing one over the other, rather than rushing to do both at the same time, which I think might not be such a bad thing. So that's

DS Lecture 5 Page 1 of 20 Transcribed by <https://otter.ai>

our plan. Big Questions on that. Okay. So that's the assignments. We are going to do the linked list code today, we had done ArrayList code last class correct. So we'll do the linked list code today. And that may go a little bit faster. I'm not 100% sure on that one, how quickly I'm going to be able to do that. But the plan would then be to start discussing to start discussing stacks, maybe in the very abstract sense at the end of lecture today. And then we'll really dive into stacks and stack algorithms on Wednesday's lecture. So then the following week,

09:01

which is our last oops, did I just break out Okay, our following week, which is our last week before the midterm, we will spend about mineral a third of electron cues, which following up from stacks is pretty straightforward. And then the remaining two and sorry, one and a half lectures will be on starting trees and the first section of chapter four. So we're not going to make it I think two binary trees before the midterm, but we're definitely going to make it through the first section of chapter four on like sort of general purpose trees. So if

you want to read ahead to that that would be the plan. Okay. Then later that week after the Wednesday lecture, I'll run a run a review session which will be recorded and posted shortly thereafter. If you can't make those Then I will do a pre recorded lecture continuing the discussion of trees for the following Monday. That way, you don't have to come to class while you're in the middle of the exam, period. And then I'll probably get through most of trees on that last lecture before spring break, which means that homework three, which you'll have over spring break, and maybe a week and a half following that will be mostly on that material. Okay. So I think we're in pretty good shape there. All right, so let's begin on the my link list implementation. So this is a sample implementation of link lists from the waste textbook. Again, if you wanted to download this, there's the link to the waste code on the front page, of course works, which is what I've pulled this from. So if you want it in front of you, you can grab it there now, if you haven't already. And this is meant to be a doubly linked list implementation makes use of the Sentinel nodes that we had been using when we drew and described doubly linked lists on paper. And it's meant to kind of simulate what goes on in the Java linked list class. But it's not exactly the same. And it's the same issue that came up with the ArrayList. In fact, this is going to be implementing the exact same set of things that we implemented for our my ArrayList, just using the linked list implementation. And if you recall, the issue we ran into there was the fact that the collection interface and the list interface have like way more stuff going on in them than excuse me, then we have time to do. And so what we're looking at here is a subset of all of that meant to kind of simulate it, but it isn't quite. And so the way we go about this is if you look at the code I have in front of you, you'll note that we implement iterable, which is sort of that thing that collection implements and sorry, collection extends, and then lists sort of inherits that when an extends Collection. This forces us to DS Lecture 5 Page 2 of 20 Transcribed by <https://otter.ai>

implement our iterator method, and therefore implement an iterator on this whole thing. And, yeah, that's really the only thing we're going to do here. That is explicitly specified in an interface exactly the same as last time around. We're generic, we're going to use any type as our type variable, there are no constraints on it. Because we don't need for things to be comparable for this to work. There is a slight typo in the code, which I'm going to go ahead and fix right here.

13:21

But I guess before we look at the methods, we should look at what our instance variables are. And you recall, in our previous example, we had as our instance variables, an ArrayList. And sorry, not an ArrayList, and array, and a size. But here, we're using nodes.

13:41

We don't need a default capacity, because the nice thing about using a link list is that you just allocate nodes as needed. Here, we have our instance variables. Now I'm going to highlight for one second, this whole mod count business. And what this is doing is just keeping track of how many changes are made. And we are going to basically ignore this for the duration of our conversations, it will be dealt with here and there in the code, but we're just going to ignore it. Because it's there for dealing with something called concurrency. When you have multiple threads in your program, basically, sort of two lightweight programs using the same data structures at the same time. It's to keep them from trampling on each other. And honestly, that's not something we're going to concern ourselves with. So I'm going to just ignore everything about that. So the three instance variables that we actually care about, is the fact that we do keep track of the size, it's kind of lousy to force the user to have to go through and count every item each time. And so we have our size, and we also have as our nodes, the begin marker and end marker. And what you should remember from when we diagram this out that those don't count as part of the content, those don't count part of the list there. Just before and just after the list, there are nodes, and then they point into the list. So that means if we have an empty list,

there actually exists two nodes and an empty list to begin marker in the end marker just pointing at each other. Okay. Now, the first thing we're going to need here is to figure out what the data type of these nodes are going to be. So in fact, we're going to create a node class for it. Now this is going to come back a little bit to some conversations we had in terms of array lists. Now, you may remember when we did array lists, we had declared our our iterator class their ArrayList iterator, nested inside of the ArrayList class. And we had made it a non static nested class, also known as an inner class. Now, what you may recall was sort of the most important thing about that for the iterator was the fact that it

DS Lecture 5 Page 3 of 20 Transcribed by <https://otter.ai>

retained the link back to the containing outer class instance, on which it was created. And that was critical to its operation, because the ArrayList iterator needed to be able to manipulate the underlying ArrayList. And so that was a nested class that did not have the static keyword to retain that link. But these nodes that we're going to create are going to also be nested. But a node is sort of an object that the outer class plays with. The nodes themselves don't have any function other than existing. So they don't have to have any link back to the containing outer class because they don't have any methods that would need to invoke anything in the outer class. Therefore, because they don't need that link, we don't provide it, you only provide that link if it's absolutely necessary. So this is a static nested class. So our private class node is declared to be static, it is nested inside of the outer class. But because it's static, inside of this, you don't have links back to the outer class, but we don't need that here. So this is the other type of nested class. So it is private, meaning that the only thing that can have a manipulate these things are my linked list objects, you can't access them or pass them to the outside world. That's okay. We're going to give them a constructor sorry, we're going to give them three instance variables, their data, which will be of type, any type, and two nodes, both, you know, note, any type, a previous link, and a next link, we'd sort of sketch these out for you before, this is what it looks like in practice, in practice. Now, you'll notice that those instance variables are marked as public. And this should hopefully be one of the first times you've actually seen public instance variables, because for the most part, we're not really supposed to use them. This is an exception, and a place where public instance variables are actually quite useful. There is no need for the outer class to have to invoke accessor methods and mutator methods on nodes, because they're the only class that are allowed to touch these things. And because they're the only class that are allowed to touch these things, you don't need the usual data hiding situation that you normally would have when you have private instance variables. So we make them public, so that the outer class can manipulate them. But because the entire node class is private, nothing from the outside world can get to them anyway. Now, there's a question in the chat that's actually extremely relevant to what I just said, quote, the node class also be written in a separate file in the same package. Are there any advantages to having low class being nested? And in fact, yes, you could do it that way. But the advantage here, is by constraining it to the outer class, you don't have to worry about other things. messing with your nodes, you can be sure that the integrity of the list is upheld because the only way to manipulate the list is via be instance methods of the mailing list. But then on top of that, you can keep the instance variables public. So our implementation inside the link list ends up being very, very convenient. And you'll see how we do that. Already. Does that answer? to your question.

20:02

DS Lecture 5 Page 4 of 20 Transcribed by <https://otter.ai>

Cool and had I answered as well, but I think it tied in extremely well with what I was talking about before. So in addition to having these three instance variables that each node have, for convenience sake will also have a constructor. And so our node constructor takes in the data as its first field, the previous field if you know it in advance as its second

parameter, and the next field if you happen to know it in advance as the third parameter, okay. And so that's going to be the underlying structure of our nodes. And those are our instance variables. All right, is everybody okay? With the building blocks for our class? Any other questions there? Cool. Okay. So let's go back up to our constructor. Now, sorry, what would happen if you don't want to know the next or previous nodes? I mean, if you don't know the next or previous nodes, you would just instantiate them as null, and then fill them in later when you know them. That's how you would deal with that. Okay. And in fact, we're gonna see that in about 30 seconds, maybe 40. Okay, so, um, we have our constructor.

21:22

All right, actually, I'll let Hannah and the TA answer those questions, we have our constructor. Now, you recall that we had this. We had this sort of setup in the ArrayList, as well, where the clear functionality does the same thing as the constructor. Because remember, when you clear a list, you're just dumping everything out and starting over fresh. All right. And so there's this private helper do clear now in the code that the book distributed. And actually, they messed up the private and public for clear verse to clear, but I fixed it here. And what I'm showing you, there is this private do clear, which just allocates an empty list, and sets the instance variables correctly for that new allocated list. And then there is a public clear method that will basically use that to throw out what's already in there. And then there's the constructor, which also needs to create a new one. So the real work happens in do clear, which is right here. And you'll notice that what we're doing is we're instantiating the begin marker, and we're leveraging the constructor. But remember, when the begin marker node, its data field is always No, because the begin marker and end marker don't store real data. The previous field to the begin marker is also always No, because there's we're not doing it circular. There's just nothing before it. But what should the next field have the begin marker reference when the list is empty anybody? It should reference the end marker. The problem is, is that because I've chosen to construct begin marker first, the end marker doesn't exist yet. So I can't actually set it equal to anything. And so since this is relevant to something that was asked a few minutes ago, since we don't know what its next field is going to be just yet we initialize it to null. Then we create our end marker. All right. And so when we create our end marker, again, know for the data, because while there's nothing stored in the Sentinels, the previous fields of the end marker is supposed to be the beginning marker in an empty list. This time however, because we created the begin marker already in the previous line, we can

DS Lecture 5 Page 5 of 20 Transcribed by <https://otter.ai>

directly initialize through the constructor, the begin marker that started the previous reference with begin marker. Okay, so that already exists. And then the next field of the end marker is supposed to be no, because there's nothing after that. At this point, we can set begin markers next field, because the end marker now exists. And so we set begin marker dot next equal end marker. Now this by the way, right here is an example of why we made those instance variables for the nodes public, it is so much easier to be able to just set those fields directly for the kinds of assignments we're going to do that that was really important that it's made public for this. Okay, so we've done that. And then we set the size equal to zero. And that's it. This mod count business I said we're going to ignore. Now, if this were the version of doom if this were the nuclear cold when the constructor is called begin marker and end marker would have originally Been No. And now they become something and we have an empty list. If this nuclear call happens from that clear method on an existing list, whatever used to be referenced by the begin marker is now orphaned. And whatever used to be referenced by the end marker is now orphaned. And fresh ones are created that point to each other. What happens to the old list that used to be there in the event, we were clearing it out.

25:33

It's garbage collected, because things don't reference them anymore. We're good. Okay, cool. So that is our initialisation. We have our size method. Right? For something like this bow, we really prefer a single class. That's actually why we part of why we're nesting things in there, we it just keeps things cleaner. Alright, so our size method just returns the size variable. That's all it needs to do. And R is empty method just returns true if it is empty, false if it's not. So it's just testing whether or not the size is zeros. All of that is the same with ArrayList. And now we come to the Add methods. And just like before, there are going to be two public add methods, there is the public add method inspired by the collection interface. And there's the public add method inspired by the list interface, the collection based add just takes the value that you're trying to add on to the link list to begin with. Now, because this is a list, not some opaque collection, that you're tossing something in, it does have to figure out where to put it. But as we said, with a list, if you just add a value with no index specified, you tack it on at the end. All right. And so it does this by leveraging the other version of add, that takes the site that sorry, that takes the index at which you wish to add the value, and just invokes that. Now, we also learned that the collection based add returns true or false, whether or not it was successfully added. But that's an artifact of other types of collections. And so for lists, we just always return true. Alright, so that's it for the collection based add, it leverages the existing list add. Now remember, when you added an index, the value you're adding becomes that index. So if there was something already at that index, it's going to push the rest of the things over.

DS Lecture 5 Page 6 of 20 Transcribed by <https://otter.ai>

And so the way that's going to be accomplished is it's actually going to splice in this value directly before the node that currently exists to be indexed. We want to add that. And so when we're doing the collection add when we want to add it at the end of the list, we can actually use the sentinel node to kind of keep it consistent, right. So no node, no node with valid data exists at index size, right? The valid data exists from indices zero through size minus one. But if we go just in sort of the more abstract sense, index size would be one beyond the last item of real data. In the concrete sense, what node object exists at index size. Anyway, cool. It's the end marker, because there actually does exist in node, one past the last piece of valid data. So we actually added before the end marker, which is exactly what we want, and it keeps it nice and consistent. All right. Now, let's take a look at the list inspired add. This is the one that takes the index before which we want to insert as well as the value we're drawing to insert. Now this one is where it gets a little tricky, you know, that we can't just jump to an index in

29:51

in a link list, right? You have to either start at the beginning and iterate over or start at the end and iterate back to it. So for an arbitrary location, the worst case scenario is that you'd have to iterate through half of the list, which would mean that it's order n because that one half is a constant factor that drops out. And it's still in the worst case proportional to the size of the list. But once we get the node object at that index, then splicing a node in before that is constant time. So we kind of split this thing up into two separate method calls. And we kind of fold them into each other here, we're gonna have a third add method, but this add method is going to be a private one. And this add before method actually takes a node object, as well as a value. And so when we invoke ad before, and we give it a node reference, it will create a new node out of that second parameter x, and then splice that note in before the node object that was passed in as its first parameter. And so what we want to pass in to add before is its first parameter is we need to convert the index into a node reference and then pass in that node reference. Does that make sense? And so what we need is we need a method that will actually using that index iterator over to the proper node and then return that node. And then we can work with that node instead. And so in our code, what we're going to do is we're going to have a get node method. And the get node method is the thing that in the worst case is

going to be the order and cost and the get node. So at its simplest takes in the index iterates through the list to find the node object at that index, and returns it back to the user. So we're going to take a look at how we implement that in a second. But just a quick note here, you see how this call to get node takes boundaries, in addition to the index, it takes zero and sizes, its boundaries. And I guess the reason why it does that is the following. Um, you know that if I were going to forget about ad for a minute, let's say I were going to get a value at a particular index that is returned the value stored at a

DS Lecture 5 Page 7 of 20 Transcribed by <https://otter.ai>

particular index, if I were going to get a value in a particular index, first, I would have to do a git node at that index to get the node object at that location and iterate over to it. And then I would extract the value stored there correct? What are the valid ranges for index indices into that get method? a byte zero to size No, it's not zero to size, it's actually zero to size minus one, right? Because index size doesn't contain real data. So the get node method, under normal circumstances, should only accept values between zero and size minus one as valid locations for it to work. The only time when it should allow outside of the range zero to size minus one is in the case of the ad, because in the ad, we actually need to add before and that means in that one very narrow case, we want the index at location size found which is not real data, it's just the end marker. And so the reason why get node takes three parameters, is because we want it to check whether or not your index is correct under the given circumstances. Under these circumstances, the index is correct if it's between zero and size, in every other case, it's correct if it's between zero and size minus one is everybody alright with that? What would the fourth parameter be Omar What am I missing?

34:30

Oh, so get node is not changing anything. Get nodes job is to extract the node object, the thing that calls it will use that node object to make whatever changes are needed. So in this case, no no no problem. In this case, add to before uses the result of that. In the case of that, get method The get node would, then its output would be used by the get method to actually compute the final result. All right, so let's take a look at get node. Since that's where most of the work actually happens.

35:19

You can see it's used all over the place. Now, by the way, get node is actually overloaded. There are two versions of it in our code that we're going to write. There's the version that I called an ad before, which takes the index in the bounds. But there's also a version that just takes the index. And that's the one we call an all but the most special circumstances, because that version then calls the more complicated version in the range zero to size minus one. It's this version that we want in every single circumstance, except for the Add circumstance. So this just acts as like a simplifying filter that then calls this one, so we really just need to focus on this one. So first things first, we're going to have a temporary variable p, which is going to keep track of where we currently are in the nodes as we iterate over to the index that we want. And then the second thing we're going to have to do is we're going to have to make sure the index is within range, okay. And so we have our lower and upper bounds, which were passed in from whoever called us, which will in all,

DS Lecture 5 Page 8 of 20 Transcribed by <https://otter.ai>

honestly always be either zero to size or zero to size minus one, we make sure that the index variable is within that range. And if it's not, we'll throw a new index out of bounds exception. All right. And that actually should ring a bell from last time around as well. Good. And so if we're sure that the index is in range, the next thing we have to do is we have to decide whether we're going to start iterating over from the beginning, or start iterating over from the end. Alright, and so you decide based on where the index is versus the size, so if the index is less than size over two, it means the index we want is in the first half, which means we start from the beginning. If the index is not less than size over two,

then we want to start from the end. So let's first imagine we're starting from the beginning. Now, valid indices to return are indices zero through size, there is a scenario when we want to return size. index zero is begin marker dot next correct. Index size is the end marker itself. So when starting from the beginning, since there's no way we would ever want to return the begin marker, we start p our temporary variable at begin marker dot next, then we write a loop that goes from i goes from zero while i is less than index, and then increment by one each time, basically, we're going to advance P index times until we're at the index we want. And so for each step of this loop, we set p equal to P dot next. And so when this loop is over, P will reference the node at location index, starting from the beginning. The else block is if the index is in the second half of the range. And in that case, you start p at the end marker. And the reason why there's this lack of symmetry is because the end marker is something we might want to return size is a feasible index for us to have asked for. So we start P and marker, and then we loop going backwards, we start by its size, because that's where we're starting. And we keep going while i is greater than index decrementing i by one each time, and this time, each time setting p equal to P dot proof. When it's done, P will reference the index we want. So if we pass in zero is our index p starts at begin marker, and this loop never goes so it stays in at begin marker dot next. If we pass in size as our location, then P starts at end marker never changes and we return the end marker. Alright, uh Yeah, that's a standard exception index out of bounds exception is generally what lists throw if you ask for something out of range, it's the same thing that the the Java ones returned and that's coming from the Java dot Lang. And in the worst case, Omer this would be Order and, okay, worst case, because you would have to go through half the list, whatever the size of the list is. So it's proportional to. And and so would the code still work the same way? If you did people? sighs?

40:20

No, that wouldn't work at all. Because size is an int P is not keeping track of an index. P is keeping track of a node object. Does that make sense? Krista? Cool. We can use binary search to a? No, you can't. Because remember, you can't jump to any index in the list except for the beginning or the end. So there's no way to bounce around, you either have to start at one end or the other. And what's the point of the upper and lower parameters?

DS Lecture 5 Page 9 of 20 Transcribed by <https://otter.ai>

Can you just use size and zero in the exemption check? Yeah, are you the reason for that, again, comes back to what we were discussing before. It's because most of the time zero to size isn't what you want. Most of the time, what you want is zero to size minus one. So if you're trying to get the node at location size in any context, other than this ad, you want it to throw an index out of bounds exception. Does that make sense? So if I tried to get or set or remove a node, at location size, that would always be wrong, you can't do anything other than adding inside that location size. And Dimitri, the answer to that is exactly for the same reason, because we want to be able to return the end marker itself, in certain circumstances. Would another implementation be more efficient? No, this is the best we're going to be able to do in this setup. Can't we have a unique instance variable for every node object? No, because we don't know yet. How many node objects there will be? That's the whole point of a linked list is that it's it's arbitrary. And so there needs to be a way to expand it. So you can't create individually named variables after the fact. Does that make sense? Yep. So in fact, actually, that's what an array is. An array is where you basically have a pre allocated set of individual locations that are a predictable spaces. So what you've suggested is just using an array, which, as we've discovered, is actually something we might want to do in some circumstances. Okay, cool. So that allows us to convert an index to a node, which is what we're doing in our add method. So we have our add before, which is going to take not an index, but a node. So add before we're already there. So now all we need to do is splice a new node in before whatever it gets is its first parameter. So let's take a look at that code. This is very simple. It takes in the node p that we wish to

splice a new node in before it takes a value x, which is going to be what will be in that node. And so the first thing we need to do is we need to instantiate the new node that we're trying to splice in. So we'll call that new node. So new node will have as its data x, its previous field, since it's going to be spliced in between P. And the thing that was before p, its previous field should be P dot prevx. And since p should be coming after it, its next field should be P. So this creates the node with the value x and then sends the two pointers going out from it to point to the correct things. But now we have to modify the things that are supposed to surround it to point to it.

44:30

So P dot previous sorry. So new node dot previous, that's the node that's supposed to be before new node, we have to set that things next field to point to the new node. So that takes the thing that's before it, and pointed at it. And then P, which is the thing that's supposed to be after it needs to have its previous field point to this new node. So we said P dot prevx equal new node. So these two lines here handle the inbound references. This line here handles the outbound references. And so that's for assignments two are implicitly happening within the constructor. And that's it that splices the note in. The only other thing left for us to do is increment the size by one. And as I said, before, we're

DS Lecture 5 Page 10 of 20 Transcribed by <https://otter.ai>

ignoring the mod count. Is everybody okay? With this add before? What's the big O cost of add before? Just this, what's the big O cost of it? It's order one, right? It has no bearing on the size of the list. So the order and cost of the Add methods comes from the call to get node. Once we have the node, it's constant. So remember, P is the node that new node is supposed to be put before, right? So in the new sequence of things that should go new node, then P. And so nodes are supposed to point, their next field is supposed to point to what's after it, and their previous field is supposed to point to what's before it. So if the new node is supposed to go before P, P, is previous field has to point to that new node. Cecilia, does that answer that? Cool. Anything else? Alright.

46:59

So that's actually other than the iterator, which is gonna require a bit of work. That's actually most of the hard part, we need to get method, which is supposed to give us the value stored at a given index. But that's easy, because we already wrote the get node method. So remember, if we were trying to get the value in a given index, first, we extract the node at that index. And we have that get node method, we call the simpler version, which assumes a range of zero to size minus one, because in fact, we want this to throw an exception if you give it size, because you can't get a piece of data from that. So we get the node at location index using what we already wrote. That gives me a node object. And then I extract that node objects data field, and that's the data that get returns. questions on that?

48:01

Yes, because we don't have an array. The whole point of the link list is to not use an array. Last lecture was the version where we did everything with an ArrayList, where we did everything with an array. So this is the alternative to that.

48:28

In addition to the get method, we often have a set method, the set method takes an index and a value. And instead of adding a new node in it takes the node at that index and replaces its value with this value. So it's, it's changing the value at a given location, which means that a value that used to be there is about to be removed, right, because we're overriding it. So the set method takes the index at which we want to change the new value we want at that index. And then it returns the old value as a courtesy to the user. So the first thing we do is we use our ordering get node method. To find the node object at a given index, we store that in P. Then we store the old value in a temporary variable, which

DS Lecture 5 Page 11 of 20 Transcribed by <https://otter.ai>

we're going to call old Val. And we do that by just setting old Val equal to that PII data. Now we can see safely overwrite PS data. And we do that here, we set p dot data equal to the new value. Now we've successfully changed it. The only thing left for us to do is to return the old value to the user. And that's it for set. Any questions there. We're good. Moving forward. We're going to have a remove method. Now, as you recall, we were kind of cheating here in that there exists in the collection interface, another remove, that operates by value where you tell it a value, and it's supposed to search through and remove that value. But that's a little bit overly complicated and doesn't really do anything for us in terms of demonstrating what we need to demonstrate, so we're going to omit that one. So the only remove or at least the only public remove method we're going to have in our mailing list class is the one that removes by index. Now, as was before, the first thing we're going to do is we're going to get a node that contains the the value at that index. So we need to use get node to convert ourselves to a node object. Now, the way the waste code has done this, it's actually broken, remove up into two versions, a version that takes the index and a version that takes the node object. And so the version that takes the node object will then assume we're already there, and just remove the node that's passed in. Okay. Now, that's that's a private thing. Because the outside world doesn't know about nodes, right? The outside world doesn't know about nodes. So there's no way the outside world could call, call this. And so a question I have is, why bother to factor out this remove by node location rather than index into a separate method? Any ideas why we might do that? Well, it doesn't really keep the data secure, necessarily, because this code is still being cold from the public one.

52:13

Right. But the question, I guess, is, is there ever going to be a time when we're going to do this, when we're not doing this? That's the only reason why we would break it out into a separate method. Is there ever a time we would actually want to remove by node rather than by index? No, not that. Yeah, when we're using an iterator, right? Because the whole point of an iterator is that it keeps track of where you are in the list so that you don't have to start at the beginning again. And an iterator has a remove method. Now, the iterators remove method will then call the underlying lists remove methods to actually do the work. But since the iterator is keeping track of where we'll see that the iterator will keep track of things by node rather than index because that will allow for better efficiency, the iterator will want to use this version directly. So that's why we factor it out. Okay. All right. So just quickly, the index based version uses get node to get the node and calls the node based version. Now for the node based version, all we need to do is make sure that node P is bypassed in the list. And once it's bypassed in the list, and this method returns, the P variable will ref will go away. And so that node will not be referenced by anything

DS Lecture 5 Page 12 of 20 Transcribed by <https://otter.ai>

anymore and it can get garbage collected. Now, by the way, the Remove method does a similar thing to the set method, it returns the value that used to be there as a courtesy to the user. So, the first thing it does is it bypasses P. So the thing that is after p p dot next p dot next previous field currently points to P. But we want to bypass p so P dot next previous field should now be made to point to what's before p. So P dot next dot previous is just equal to P dot previous, thus bypassing p going backwards. The thing that's before p p dot previous its next field should also bypass P. So we say P dot previous dot next will be equal to P dot next, so jumps over P and so these Q. So P dot next dot previous is a, an instance variable of P dot next, that within it contains a reference to P. But we are changing it from referencing p to what p dot previous references are, does that make sense? So it's just changing what it references, you're right, P dot next dot previous is originally p, we now need to make it something else, which is what we're doing here. Alright, now that P has been bypassed, we decrement the size by one since these things are no longer part of the list. we're ignoring the mod count business. And then for the last

moments of peace existence, we extracted data and we return it to the user. Okay, any questions on that? Oh, the last moments of peace, man, I somehow made it to that sound terrible did not anything else. later tonight, I think and do later on. I we spoke about this quite a bit at the beginning of lecture. Um, any other questions related to the Remove method? Okay. How do we know Java doesn't delete p before we return its data Oh, okay. Well, you can only garbage get so sorry. An object can only be garbage collected when there are no variables that reference it anymore.

57:10

The moment that I say P dot data, p must exist still right? Because as a variable because I'm manipulating. So as long as p still exists as a variable, it's referencing this object. So that object's reference count is not yet zero. So Java can't delete it, while the variable p exists. After it returns, then the local variables within this method go away. And so P, the variable goes away, and therefore the number of reference counts to the number underlying object decreases. And it's only after this is completed, that the underlying object can be garbage collected. Does that make sense? had our Lena could you answer Omar's question in the chat. It is the memory address of a node to hang. It turns out any variable, that reference that isn't of type a class rather than type. A primitive is just a reference every variable like that. They're all pointers. And so you're just changing what the variable points to. Did everybody hear that? The thing dropping? Hold on one second. Oh, God all the way back. I will go fish that out from under my desk at some later point. Okay, so I want to move forward. Now I need to drop not Oh, okay, funny. In fact, what I did was I dropped a bottle that bounced off my foot. Anyway, moving on. Somehow, I don't know if we want to really compare imparting knowledge on you with my dropping a bottle on my foot. Because if that's what I'm doing dropping this on you in the DS Lecture 5 Page 13 of 20 Transcribed by <https://otter.ai> sense that I just dropped that bottle. We've got some serious problems. Alright.

59:41

So now we're going to get into the iterator business. So first of all, I'm not going to go through this code because it is literally identical to the ArrayList to string method. This is simply leveraging the fact that the underlying linked list class is iterable. Because it's iterable and you can use the enhanced for loop on. And so this highlighted chunk of code is identical to the highlight to the to string method in ArrayList that we did last lecture. Okay. What we do know is that because we implement the iterable interface, we are required to have an iterator method. And that iterator method needs to return an object whose underlying class implements the Java dot util dot iterator interface. Okay. So, all we're going to do is we're going to return a new instance of linked list iterator. And so linked list iterator needs to be a concrete class that retains some tie to this particular linked list so that we can actually iterate across it. And it must implement the Java dot util dot iterator interface, meaning it must have a hasNext method. Next method, add a remove method. Okay. And so, we now need to provide an implementation of this linked list iterator class, which is what we have here. This is going to be making use of the same sort of type of construct that we use in our ArrayList iterator. It is a non static nested class aka an inner class of the linked list class. By being non static, it means that instances of this retain a link back to the specific instance of my linked list on which it was contained. Therefore, we retain a link to our my linked list. And we can use the instance variables and methods of that my linked list directly. Okay. So we need to keep track Oh sorry, I should say we have our private class linkless iterator. It implements Java dot util dot iterator. It's not static. And so we have our instance variables here. First, we have our current. Now in our ArrayList iterator, we kept track of the index that we were currently at, because in an ArrayList, you can jump directly to an index. But because in a linked list, you can't jump directly to an index. And we want to make this as efficient as possible. We make current node reference. So remember, current is supposed to latch on to the next node that we haven't yet seen.

So when it starts, it latches on to items zero, because items zero is that first item that we haven't insane. So we set this node current equal to begin marker dot next, because begin not marker dot next would be the next item that we haven't yet seen. Right? Item zero. We're going to completely ignore this mod count business so I don't care about it moving on. And just like before, we have this okay to remove business, you're only allowed to remove something you've already seen. And you're only allowed to remove one step back. So until you've actually invoked next and taken a step forward, you're not actually allowed to call remove. So okay to remove starts out as false. Good. And so now we need our has next next and remove method. So we'll start with has next has next returns true, if the current item is valid. If the current item is valid, then its data can be given to us. Now, when the current value is sorry, let me back that up one second. The End marker in the

DS Lecture 5 Page 14 of 20 Transcribed by <https://otter.ai>

beginning marker don't contain valid data. So when current lands on the end marker,

1:04:39

there's nothing more to return, because n marker has no data. So we check is current not equal to n marker because as long as it's not equal to n marker, then it has real data. The moment current is equal to n marker, then we're done has next returns. False. Now in the event, we have an empty array, sorry, an empty list, begin marker points to end marker. So if the current start said begin marker dot next, if you have an empty array, current actually starts at n marker. So out the gate, as Next, we'll just return false. Now there's something about this highlighted piece of code that should bug you, it should annoy you to your core, because it was something you were told not to do. So you can't say current less than n marker in this context, because they're not integers. They're object references, which don't necessarily have a relation to each other. So you're getting there, there's something a little more related to that. But what am I doing here that you were probably told never to do?

1:06:07

anyway? So it's, it's so close, it's not quite compare classes, but compare objects, right? You are likely told not to use the double equals and the non equals when comparing two objects. So for example, if you had two string objects, A and B, and you wanted to see whether or not they had the same contents, you were told not to use the double equal sign, you were told to use the dot equals method, right? That rang a bell? Why were you told not to use the double equals method? What happens when you use the double equals method? Right, because not primitive, but Oh, good, good. Because it compares memory locations. Using double equals or not equals on object references just tells you whether or not they are exactly the same memory location. But in this case, that's actually what we care about, right? And marker exists at a given memory location. If current points to the same memory location as the end marker. That's all we need to know. And so in fact, this is fine. Is everybody okay with that? Cool. All right. So that's the has next method. Is that a reference to a commercial that aired during the Super Bowl or something? I don't know. I, I don't watch sports. I'm sorry. I have to say this Superbowl Sunday was sort of one of the saddest ones for me, because my family had a tradition on Super Bowl Sunday, we would always go out to eat that Sunday, because the restaurants wouldn't be crowded. But you know, we couldn't do that. So we're stuck at home. I played video games instead of watching this over Well, I, I guess that made me feel slightly better. Alright. Um, so we now have our next method. Our next method, assuming the value at current is valid, returns the data stored at the current node, and then advances the current node to the next. So we're going to ignore this mod count business, if not, has next meaning that the

DS Lecture 5 Page 15 of 20 Transcribed by <https://otter.ai>

current node is not valid, which this should never really be a problem, because you should always check hasNext before calling next. But if not, has Next, we throw a no such element exception, by convention. That's what iterators do. All right. We store the data that's currently at our current location. So next item equals current dot data. We then advance

current to be equal to current dot next. So we take it a step forward. And we don't have to iterate all the way from the beginning to get there. We're just taking one step forward. we flip okay to remove to be true. Because once we take a single step forward, the thing we just returned is eligible to be deleted. And then we return the item that we extracted from the data. That's it. Questions on That.

1:10:06

Yes, that Tiffany that would also work. So remember the argument to to an IF needs to be a true or false. So that just negates what has next returns, which would be true if has next is equal to zero. Other questions? Alright, that's our next method. The last thing is our remove method, we're ignoring the mod count. First thing we do is we check to make sure it's okay to remove, we're only allowed to remove one step back. And we're only allowed to remove if we've taken a step forward. So if not okay to remove, we throw an illegal state exception. Again, that's just by convention, what iterators do. Okay. And then, if it is okay to remove, we leverage the Remove method from the outer class. And remember, we had a remove method in the outer class that takes an object reference that takes a node reference that we're going to try to remove. So recall my linked list dot this is sort of like an implicit parameter, it is a reference to the particular my linked list that this inner class was created on. And then we're invoking that link, my link lists remove method. And we're telling it to remove current dot previous, because remember, we're always removing the item that was most recently returned to us. But current is sitting on the item that hasn't yet been given to us. So the thing before current is the item we want to remove. So we remove current previous, that leverages the existing method in the outer class. Once that's done, we ignore the mod count business. And we flip okay to remove back to being false, because we're only allowed to remove one step back. Okay. Any questions on our remove method here? Okay, so that actually wraps up our discussion of web link lists. And I've got about five minutes left. So I'll just briefly introduce stacks, which are going to be another data structure that we're going to kind of use to build on the data structures that we've already had. And so let's talk about stacks a little bit, um, a stack could be thought of, like separated out from data structures could be thought of as an abstract data type initially, and then we can worry about how to implement it later. And so a stack? Well, it's literally a stack of things. If you think about physical examples of stacks in the real world. I have lots of examples that I try to use to describe what a stack is, and they all seem to fall flat. My favorite one is sort of like a bathroom cup dispenser. But then 90% of the class looks at me funny, and says, What's the bathroom cup dispenser? And then I recall, I never had

DS Lecture 5 Page 16 of 20 Transcribed by <https://otter.ai>

one either until my wife insisted we bought one. So that one never works. And then I pointed out to like the the handful of people who've handled firearms before, if they think about the magazines that they use to load the bullets into their guns, those are stacks. And that only works for a small subset of the class. Water Cooler cup dispenser. Yes, I often come to that one, which is about the same as the bathroom cup dispenser, and does not really resonate very well. No in line to a movie is the other thing we're gonna do. It's a cue. And so the one that seems to resonate most Oh, Pringles chips, um, that could work but the one that seems to resonate most is the stack of dirty dishes that sits in your kitchen. If you think about the dirty dishes in your sink, when you add a new dirty dish in there, you put it on top of the dirty dishes that are already there, because trying to jam a dish in the middle of a bunch of dishes stacked up as a good way to break something. And in those rare occasions when you go back and you actually clean those dishes, yes, I'm being slightly judgy and it makes assumptions. But yeah,

1:15:03

when you go back and actually clean those dishes, you take the dish off the top and watch that one, then the next one. So the idea of a stack is that you are adding, you are adding and removing items at one end of things. So if you have the stack, you're adding

and removing from the top of the stack. And that's it. And so there are the standard. So in fact, this is what's known as last in first out, because the thing you most recently added onto the stack is the thing that comes off first. So this is lifepo, Li fo lastin. First out. And so the two major methods that you'll see in a stack are the push method, which is where you're pushing an item onto the stack. And that's why I always use the cup dispenser or the bullet in the magazine thing, because you are physically pushing the item onto the stack to get it in place. So when you add something onto a stack, you push it on. And when you remove something from the stack, you pop it off, and then the rest come up. All right, so push adds to the top pop removes from the top. In addition to the push and pop methods, you'll typically have a size method just to keep track of how many items are on there. And he's empty method, because one wouldn't want to pop an empty stack, right. But in addition to his MTU, sometimes we'll have an ease full method. And that's going to depend a little bit on how you implement your stack. If you implement your stack, in such a way that it's you're using just a an array, you could run out of space. And if you push something on to a full stack, that's gonna be an error also, what error happens when you push something onto a full stack? Anyway, it overflows the Stack Overflow, that's where that website gets its name from. And if you pop an empty stack, that's what's known as an underflow. Okay, so the is empty and is full methods allow us we're not going to do an iterator with this allow us to, you know, make sure we don't have an underflow or an overflow. Now the very last method that our stacks will typically have is a method to look at what the top item is without actually removing it. Now there are a number of names

DS Lecture 5 Page 17 of 20 Transcribed by <https://otter.ai>

you will see for that. Sometimes people go with a rhyme and call it the top method instead of the pop method. So the top method tells you what's on the top without returning it. Sometimes we'll go for alliteration, instead of calling it top, they'll call it peak, like p e, k. So you're peeking at what's at the top. And if you're an idiot like me, you will sometimes misspelled peak, P A k. And then when I thought about it, that actually kind of works too, because it's at the peak of the whole thing. All right. So push, pop, top or peak, whatever you're going to call it and is fuller is empty, and a size method. And that makes up our stack abstract data type. And what we're going to start talking about on Wednesday, are ways in which we can actually implement that realize it in Java. And then we'll talk about some algorithms that we can do with stacks. And let me just give you a quick heads up about one thing with stacks. The real selling point of a stack is that it's great at reversing the order. Okay. All right. I will see you on Wednesday. Daniel, what's up?

1:19:15

Hi, Professor, just a question. I want to just double check something. So all of these, I guess data types already exist in Java, right? We're kind of like implementing them. So we learn ourselves what's actually in them. But they do already. Right?

1:19:30

Yeah, there exists a stack class in Java. It actually is an it's a class that extends lists, weirdly enough. But yeah, they exist. Okay.

1:19:45

Oh, I had a question about garbage collection. How does it work? Exactly? How does it determine that there is a chunk of memory which nobody points to? Um,

1:19:55

I don't actually remember exactly how Java does it but the way I like to think about it is that every time an object is instantiated, the Java Virtual Machine, you know, has to have a chart of these things, keep some metadata about that object. And the metadata about the object would include a field known as a reference count field. But what if, what if we have like two variables that reference one another, but that right, so that that's, that's a situation that is that is handled, but I do not remember the exact mechanism for it. So a

pure reference count will actually break in those circumstances, which is why it uses something a little bit more sophisticated than that. Okay, because I'm thinking that it either I know, you're getting memoryless or just go through them all. Yeah, but I'm just, I'm telling you that it, it will have another way of bookkeeping, that I just don't know what it is. Okay, thank you. Alright, Kareem, what's up?

1:21:00

Professor about the exceptions. If we are throwing the exceptions inside the method, are we assuming that we're going to, you know, apply a try catch phrase afterwards? Oh,

1:21:13

no. And remember, there are two types of exceptions. Right. There are two categories of exceptions. And so if you think back to how exceptions are created, they extend existing exception classes. And there are actually two exception classes that most of the exceptions we deal with extend from, there's exception. But there's another type called runtime exception. And runtime exceptions are what are known as unchecked exceptions. So runtime exceptions, you don't actually need to have inside of a try catch, it's only checked exceptions, things that extend from exception itself that need the try, catch. And so all of the exceptions that we're throwing here are unchecked exceptions, meaning they extend from runtime exception. And the way you should realize that is that the array index out of bounds exception is one that you yourself likely get. When you're debugging your code using arrays. It's the same exception. And you know that you don't have to encase every call to an array in a try catch. When you have a checked exception, you either have to handle it right then and there. Or you actually have to declare your method is throwing the checked exception. So if the thing compiles, and you don't see a throw keyword in the function head or sorry, the method header, you know, it's an unchecked exception.

1:22:57

Does that make sense? Yeah. Thank you. Cool. Cool. Other questions? We're good. We're good. Blair. Sorry, but last week, but I got everything in there on time, right? I assume so.

1:23:24

I mean, check with the TA is on that if you want to double check. Whenever you're going to do the the

1:23:31

word called error checking thing recheck, like you know which one I'm talking about rechecked and see the like the symbol binary palettes I get generated from the exceptions or something like that. That's the fourth or the fifth. Maybe I feel like I don't quite remember. I'm not sure exactly what we're referring to. Well, anyway, gotta go player.

1:24:02

Take care. All right. So I think if unless there are any other questions. Thank you. I'm gonna close this down. We're good. Thank you. Thank you, professor.

DS Lecture 6

Wed, 8/11 1:12PM 1:24:19

SUMMARY KEYWORDS

stack, operand, called, size, push, method, expressions, array, operator, symbol, palindrome, pop, character, mismatch, arraylist, write, adding, item, calculators, algorithm

03:46

Try this trick in your head because there's nothing Where is my Where is my mind where is

my Where is my Where is mine is mine We'll get started in just maybe two more minutes in maybe just one just waiting for a few more people.

06:39

Does anybody want me to play something that's not where is my mind for like five minutes a dealer's choice or to somebody has something they want to hear. Vampire Weekend. Maybe

07:05

the danger here is I've never heard them before, which means I'm going to critique it. The top ranked one on Spotify is campus followed by something called a punk what's safe? emphasis on safe.

07:33

Okay, fine.

07:47

My shoulders cold. I've got to leave here before I go. walk out the door. drag my feet along
DS Lecture 6 Page 1 of 19 Transcribed by <https://otter.ai>
flow. walk out the door, drag my feet along the floor and see across the campus proof system. Drag your feet along. proof.

08:47

All right, you should get started. I'm keeping a separate playlist of everything I play here. I guess I'll throw that no, that's gonna throw everything off. That doesn't count. All right, moving on. Um, so you guys should have everything you need for homework too at this point, correct. You've seen that? Yeah. Okay, good. I will send a note out to remind people of that, because the blank faces I saw was mildly concerning. Um, but you have a remember you have homework to written due next Friday. And then you have homework to programming which will be due the following Friday. So I'll give you a little extra time so that it doesn't collide with the midterm. That was the plan for that. I think we're already seeing questions about that on Piazza. Does anybody have anything they want me to address before I get started on stacks. Again, a review session. I haven't actually scheduled it yet. But it was going to be towards the end. of next week? Um, would anybody be mad at me if I did it on Friday? No, that's when your homeworks due. So here's the problem, I have to fit in yours, as well as the honors section and I also have to teach the honor section. Um, Friday is good. Let me check. So I have office hours, they have like, individual appointments till about 430 on Fridays? Um hmm. So would anyone massively object if I did it at 5pm on Friday? And then I'll record it and post it shortly thereafter? Sound good? All right. Not this Friday, the next one? Good. All right, I will try to book that. Um, I also have to, I have to reach out to disability services about that, because we need captioning potentially for that on. So let's not say that's 100% set in stone until I've coordinated with absolutely everybody who I need to coordinate with. But that's my working plan. So that would be somewhere around five o'clock on the 19th. That would be my plan. That way, you can watch it at any point over the weekend, if necessary. Okay. Anything else? No homework questions yet. All right. So, um, stacks, we were talking about stacks at the end of last class. And we talked about the stack abstract data type, in which we had the following methods that we discussed for adding elements to the top of the stack, we had a method called push for removing elements from the stack and returning them from the to the user, we had a pop method. If you just wanted to see what was on the top of the stack, we had a method called top, which was sometimes called peek. Or if you're an idiot like me, you might miss misspell like that, whatever, we have an empty method. So that we don't accidentally pop it on at pop an empty stack and underflow. And sometimes, depending on how we choose to implement our stack ADT, we have an is full method. To prevent stack overflows, if it turns out, there's a sort of a limit on how many items you can have in the stack. And we probably would also have a size method. And so those would be kind of the primary things we would expect in the stack abstract data

type. Now, now that we have that, we should talk a little bit about various ways that one could implement a stack. And so one option is to leverage a list to implement a stack. All right, so you could leverage a list to implement a stack. And the way you would do that would be that, well, a stack is last in.

13:34

First out, so that means you're adding removing and removing items at the same end of the list. So as long as you're always adding and removing the items from either the front of the list or the tail of the list, you've got yourself a stack. Now if you were going to use an ArrayList as a stack, which end would you choose to get efficient implementation.

14:10

So if we were going to use an ArrayList, I say ArrayList or linked lists now I suddenly I've always had an array, but then suddenly, somebody said something about linked lists. We're talking about ArrayList here, please help me I can't rewind. We're talking about ArrayList. So a lot of people said at the end of the list, and that's good, right? Because remember with an ArrayList if you add anywhere in the interior of the list iterator or not, you're going to have to push items over. So heatedly adding at the beginning of the list would require order and cost to add or remove at the beginning of an ArrayList. Right. adding and removing at an end is for the most part, very cheap. It's always order one unless you happen to have hit the capacity of the ArrayList, in which case it reallocates a bigger ArrayList. Sorry, a bigger underlying array. And that one time that you hit that it ends up being an order n cost. Another option, though, is to use a linked list. Right? Now somebody pointed out, do I mean singly or doubly linked lists, let's talk about both. Let's say we're gonna do it with a singly linked list. What end what I want to be adding and removing the items from the beginning, right, because you can always splice something in in front of the head reference, that's easy, but splicing thing in at the tail using the basic implementation we saw would involve iterating, all the way to the end to get it, which would be order n. Now, to be fair, there are tricks you can play by storing a reference to the last element of a singly linked list. That might speed it up a little bit. But you can bypass that entirely by just adding removing at the head. So singly linked lists actually work remarkably well for stacks. They use not too much extra space, you don't have to worry about reallocating arrays, and everything's order one. But if we wanted to use the Java implementation of a linked list, you would end up with a doubly linked list, which end what I added removed from if I were using a doubly linked list. Either doesn't matter, as long as you're adding and removing from the same end. It's good, right? Because the nice thing about a doubly linked list is that the nice thing about a doubly linked list is that what ends up happening is you can start at either end. So all of those are very simple ways we

DS Lecture 6 Page 3 of 19 Transcribed by <https://otter.ai>

can leverage our existing lists to implement a stack. But there are other ways. And so I want to do another. So let's say instead of using a list an existing type of list ones that we've already implemented, let's say we wanted to implement a stack using a fixed size array. All right, so when we initialize the stack, an instance variable that is of type, you know, an array of whatever the type of the stack is, gets created with some fixed capacity. And that's it. Alright, so inside of our stack class, if we're going to do something like that,

17:48

maybe we have an array A. Now, for the sake of my sanity here, as I'm writing these things down, let's assume that this stack is just working on It's okay. Um, so let's just assume that. So we'll have an array of intz. Let's just see how many I can fit here. These are the indices. Okay, so this particular array has length eight, with indices zero through seven. Correct. All right, so my max capacity for this array, sorry, for this stack is going to be eight items. And in terms of instance, variables, we don't need another one to keep track of the capacity is just the length of a. I'm gonna need one more variable, though, to make this

work for the stack. What other variable do you think I should have other than the array that has that intrinsic capacity? good size. So I'll need to size and initially, independent of the capacity of the stack, what should I initialize size to be? Zero because there are no items in it. And so that's going to be my basic building blocks for a stack. So now let's see how we're going to arrange stuff in here. Um, let's imagine that we're going to push a value. So let's say we're going to push seven onto this thing. Okay. Where do you think would be a good spot to put the seven? What would be a nice place to put the seven in that array? index zero. Okay, that's a good spot. There's one other answer that I think would have been acceptable potentially but zeros, the one I'm going to pick but yes, seven, the last element would have also been a Not an awful choice. But in fact, index zero ends up being a little more convenient. So we're going to put that here at index zero. And in addition to placing it there, we're going to update our size from zero to one. Good so far. All right. So now what I'm going to do is I'm going to push another item on here.

20:28

Where should I place the four? There are two potentially acceptable answers one good and one bad? Um, so Location One is sort of the good answer. Location zero is the other answer that might make sense, but has problems associated with it. If I were going to push the four on two locations, zero, what would the problem with that be? Maybe you'd have to move the seven over, right. And so what's gonna end up happening if you always want the top of your stack to be location zero, which I guess is an admirable thing to dry for, you end up having ordering costs to shove things over when you push stuff on. And it's

DS Lecture 6 Page 4 of 19 Transcribed by <https://otter.ai>

the same problem when you pop things off. If you want the top of the stack to always be zero. When you pop something off at locations error, you'd have to copy everything back. So that would result in order and operation. So I actually prefer putting it at location one.

And then we update the size. Let's push another item. Where would the eight go?

Following this pattern? location two, and we'll update the size. Good. So Wayne gets to the answer to my next question. Before I said it, instead of giving me a specific number.

Where would I always push the next item? I would always push the next item to location size, right? Because location size is always one beyond the last valid index that I filled.

Does that make sense? Yes. Cool. No, it doesn't ask a more specific question. So location size, is always going to give me the next empty spot after the last item that I pushed on, right? Because I'm growing this out, starting at location zero, then one, then two. So you'll notice the three is the next empty spot. Good. So pushing just involves going to location size. Now to make sure we don't have a Stack Overflow, we're going to need an error check, right? What would my error check be? How would I check. So you check to see whether or not size is actually equal to the length, right? Because if size is equal to the length, that means that all of my slots are filled. So if size were eight, eight is one beyond the last valid location that I could write anything down, so that would overflowing. So now that we've kind of worked out the logic of our push operation, we could actually write a rudimentary stack, sorry, a rudimentary push method for our stack. So our push method for the stack can be a void method. Because it doesn't actually return anything to us. For the sake of, as I said, my sanity, we're not doing this generic, but I guess if we were actually encoding it, see homework three, um, we'll use an integer value. So x will be the value that I'm pushing on. Before I actually do my push operation, the actual mechanics of adding it to the array and updating size can be done in a single line of code. But before I do that, I should check to make sure I don't have a Stack Overflow. So if a dot length and a incisor my instance variables is equal to size I'll go ahead and throw a new Stack Overflow, I guess it would probably be called Stack Overflow exception, but you get the idea. And now for the one line of code that I need to add my item to the stack, if I know there's space for it,

25:22

I'm setting a at position size equal to the value I'm adding in x. But I also want size to be incremented by one, correct. And so I can use my increment operator in place here. So I say a its size plus plus equals x. And if I do that, you should recall the plus plus and also the minus minus operator have slightly different operations, if you place them before or after the variable on which they're operating. If you place the plus plus before the variable, then it increments the variable first, and then evaluates the rest of the expression. If you put the plus plus after the variable, it evaluates the entire expression with the original value of the variable. And then when that's done, it increments size by

DS Lecture 6 Page 5 of 19 Transcribed by <https://otter.ai>

one. So in the current state of the stack, if I pushed an item onto it, it would set size at location three equal to that item. And then once that's done, then the plus plus would kick in, and it would increment size to four. Any questions on how this one works? We're good with that. Okay, and so now, we might want to pop an item. Now, when you pump an item, it's going to do two things, right, it's going to make it no longer part of the store three things I should say, it's going to make it no longer part of the stack. It's going to decrement size by one, right because we're removing an item. And it's going to return the value that we just removed. So if my current state of the stack is this, calling pop should return an eight. And then the array should no longer consider eight part of the stack. Now, we can actually cheat a little bit here. And we can just decrement the size by one.

28:04

Right, you can just decrement the size by one and leave the eight there.

28:12

Because if the size is too, when I push, it's going to overwrite that spot.

28:19

And if I pop, remember what it's doing, it's actually returning the item that's immediately before location size, because originally size was at three. And when I popped, it returned to me the value at location two, so I can actually leave the eight there doesn't matter. If I push another value on. It will overwrite then. And then increment size. So actually, this dirty block, you can clean it up another orbit, it's not quite cleaning it up, you just leave it there until something else overwrites it. Does everybody good with that? Okay, so let's, then I need another page for this. But let's now write roughly what that code would look like. So my pop method needs to actually return a value, right? So since this is a stack of intz, in this case, I'm going to go ahead and have my return type be a pop, start return type be an int. Pop takes no parameters, just as the implicit parameter. We want to make sure we don't underflow What is my test to make sure we don't underflow underflow means pop an empty stack. So if what is true, when would we have an underflow here? So if sighs it's actually not size less than zero? It's if size is equal to zero, right? If size is equal to zero, that means there are no valid elements in the thing. And therefore we want to throw an exception. So if size is equal to zero, we'll throw a stack underflow. All right. Now, we're going to try to make this one work other than the error checking in one line of code as well, we need to return the value that was at location size minus one, while also decrement in permanently size by one. So we can make use of our minus minus operator,

DS Lecture 6 Page 6 of 19 Transcribed by <https://otter.ai>

but just put it before the variable instead of after. So if we return a ad minus minus size, that will decrement size by one first, which is the location of the value that we want to return. But it also permanently changes size, which is also an effect that we want. So by moving the increment slash decrement operator to the front of the variable, that operation happens first, before everything else. Is everybody okay with pop. Cool. So this will return the value that was at location size minus one, which is the top of the stack. So that's what you want when you pop. Okay? Because we're accessing a at that location, and the values are stored in a. So let's write one more method, I have to compress it a little bit here. But let's write our top method just for completeness. So our talk method is

supposed to return the top of the stack, but not remove it from the stack. So the size should remain unchanged. So we still can't return. I'm sorry, what happens to that last element again? Well, it gets returned to the user, which is what we want pop to do. And then it's left in the array. But because size is decremented it's left in a spot that will never be accessed again. It'll just get overwritten later. Martha. Does that make sense?

32:46

Yeah, thank you. Cool. All right. So I'm leaving. Yeah, it would, um, garbage collection isn't actually relevant at this point. Because we're dealing with inside Anyway, you make a reasonable point, that if these were object references instead of integers, and that object reference was something that we just discarded entirely. By leaving the reference in the stack, it wouldn't necessarily get garbage collected. But on the other hand, you have to realize that the pop method is returning this item to the user, and the user might be doing something with it anyway. In which case, even if it was an object reference it, it might not actually be something we would want to garbage collect anyway, because the user might be using it. So it doesn't really matter that much. Is that okay, right. Um, John, I have no idea why you wrote if size equals zero, is there a question attached to that? Yeah, I was gonna ask about what happened with top one sizes zero, but you're probably going to get to that I am. And in fact, it's going to be exactly the same as this, because it's still an underflow if you try to look at an item that doesn't exist. Um, so I asked, so if we were to increase the size and add that value wouldn't be there anymore. Correct. As soon as you push that will overwrite the location that we had just popped here. Is that good? Excellent. All right. So as we discussed a second ago, the check for top to make sure we don't have an underflow is the same. So we still have this if size is equal to zero. we'd throw a new stack underflow. Okay. And we still want to return a value Return the top of the stack, but we don't want it to go away. So this time we'll return a at location size minus one. Because the stock, the top of the stack is at location size minus one, and that's what we've got here too. But by doing so, hold on, sorry. But by doing size minus one here, rather than minus minus size, size isn't actually changed, right? We're just temporarily calculating What's one before size and returning that value, but we don't change sighs. Good. Okay, so that's

DS Lecture 6 Page 7 of 19 Transcribed by <https://otter.ai>

what I wanted to say about stack implementations, we spoke extensively about how we might implement a stack. extensively how we might implement the stack with various types of lists, we actually sketched out code for how you might implement a stack using a plain old array. Excuse me. What I want to do now, is I want to talk about algorithms or things we could do with a stack. Okay, that's what I want to talk about right now. So what can we deal with a stack? What are things? What are use cases for a stack data structure? Any suggestions? reversing an array string? Okay. That's good. So the thing that stacks are really good at doing is reversing stuff. Right? That's what stacks are really, really good at doing.

37:01

So what else fill an array with it? What do you mean by that?

37:20

Not quite sure. Um, so terrifying. noye. Interesting. Um, so Tower of Hanoi actually does make a little bit of use of a stack, but it's kind of implicitly. So where else might a stack be useful that's related to our solution to Tower of Hanoi, you're starting to get it something that's very important. recursion, yeah. recursion actually, is essentially using a stack. So for we'll get back to reversing a string, that's actually going to be another example we'll talk about. But recursion, we really can't talk about recursion until we talk about how memory is allocated in the computer. And in the computer memory is usually broken up into two segments. All right, one segment is often called the heap memory and one segment is called the stack memory. And the heap memory is where we just sort of allocate things that could be used by anything. So if we, whenever we instantiate a new object, that's

allocating memory from the heap, and lots of different methods could have variables that reference that can be referenced from anywhere. But there's also the matter of the local variables within the method calls we make. And so each method call we make has a chunk of memory, maybe we can call it a frame of memory, that contains the local variables that are associated with that method call. So if you if your main method is invoked, a frame of memory will get created associated with that main method and all the local variables within that main method will exist in that frame. And so that frame is actually pushed on to the stack memory. So the stack memory is actually a stack of method calls. And whatever method is at the top of the stack is the one that's currently running. All right, so if you have and that's a fixed size, chunk of memory, right? So you don't have an expandable amount of memory on your computer. You just have a fixed chunk of memory. So that stack

DS Lecture 6 Page 8 of 19 Transcribed by <https://otter.ai>

40:07

is going to have some fixed size. And every time you invoke a method, like say main effect frame is pushed onto that stack with its local variables, say x, y, z, whatever. Now, as you're running your main method, right, you also want might potentially call other methods, right. And so let's think back to when we were talking about recursion. You may recall, we wrote a method called factorial, right. And so from our main method, let's say we called factorial on the value four, because we wanted to calculate four factorial. So the main method somewhere in the middle of its execution, we'll call the factorial method with a parameter that has the value foreign, and so that creates a new frame that's then pushed onto the stack. And now that's the top one, and that's the one that's running. All right. When it finishes running, the stack gets popped. And that returns back to the previous method call. But factorial was recursive. And so that invokes another call to factorial, which gets its own frame with a different value of x on it, the value three. And then that calls factorial again, with x equals two, and that calls factorial again, with x equals one. And so all the prior calls are building up on the stack, in the reverse order, waiting to be returned to so when we finally get to the base case, that returns and then it comes back to this one. And then this one calculates its result returns, and it comes back to this one, and so on, until we return back to the main method. And so that's how method calls actually stack up in memory. So recursion, when you're calling a function on itself, is actually leveraging the system stack. To do its work. So whenever you have a recursive algorithm, really, it's it's actually under the hood of stack algorithm, because it's, it's using the system stack. So they're never too cold at once. So you're thinking in Fibonacci, when we called thin on n minus one plus fib on n minus two, right? This one here would actually have to finish in its entirety, before this one even gets called. So this one would get pushed onto the stack, and then all the chain down to here would get pushed onto the stack. And then as that makes its way back up once that's finally that done, then and only then does this one get pushed onto the stack. So it actually ends up being sequential. Because this one doesn't try to invoke until this one's done. Gabriel Is that good? Cool. Reminds me of the way exceptions that are waiting to be caught work, right? Um, yeah, actually, there's an interesting point there. So remember, when your program throws an exception, like a runtime exception, and you see what method and at what line, the exception was thrown. But sometimes you see sort of a sequence of method calls, going back from the one where the error exception actually happened back up to the main. That's what's called a stack trace, because when the Prout program bombs out, it just shows you all the method calls that were still waiting for this to finish, so that you can trace how it got there. And so when you see it referred to as a stack trace, now you know why, because it is literally tracing through what was left on the stack.

DS Lecture 6 Page 9 of 19 Transcribed by <https://otter.ai>

44:47

Now, by the way, that whole stack overflow definition for when you fill up the stack, and you know that the website was called after that. So a Stack Overflow can occur when you fill up the main memory with too many function calls. That Stack Overflow, that's, that's bad. That means your computer might crash. And so that's actually I believe what that website was specifically referencing when it called itself that this kind of a Stack Overflow. Alright, so that's a really important use case of a stack, one that we've implicitly been using for a while. Now, one of the earlier suggestions of something we could do with a stack is we could use it to reverse a string. And so again, stacks are really good at reversing the order of stuff. And so everyone's familiar with the concept of a palindrome, right? Right, where it's something where if you read the characters off in one direction, it reads the same going back in the other direction. What's an example of a palindrome? Hey, by, Okay, first one, anybody ever says is race car and then Hana. Good point. You are, in fact a palindrome as well. So here we go. Hey, we got a lot of palindromes in our ta is I don't know exactly how that happened.

46:18

I went to high school with a guy who is on the wrestling team and his name was Mike Kim. So his nickname was the palindrome.

46:26

Oh, God, did everybody catch that story? Or no? Okay. Um, you went to school with nerds? Okay, sorry. Sorry, um, random interaction there. Um, yeah, lol is also a palindrome. Nice job. Um, I'm actually Okay, there we go. Now we're starting to see some of the more complicated ones. I was almost disappointed there for a minute. Okay. So let's say we let's go with the race car for a moment. And so there is a well, that's a good one. I like that one, too. geese. see God. Okay. Um, so. So it's gonna depend a little bit on whether or not you have an even or odd number of characters, right? If you have an even number of characters, then the first half of the characters should exactly match the second half of the characters just in reverse order. If you have an odd number of characters, that's true for the first half minus or the first half rounded down characters, and then you sort of pivot around the midpoint, right? Because that he kind of counts in both directions. So if you can reverse a string with a stack, you can definitely check for a palindrome with a stack, how what's what's a good algorithm for this? Using a stack, maybe just compare the reverse string equals the original. Ah, okay, so what the suggestion here is that we just reverse the whole string, and then compare it to the original. That's one way of doing it,

DS Lecture 6 Page 10 of 19 Transcribed by <https://otter.ai>

you can actually do it a little, I think it's a few, slightly fewer operations, you could push half the characters onto the stack, right? And then pop them off one at a time. And as you pop off the characters from the first half, compare them against the characters in the second half. So you really only have to add half the characters in. Okay, so that that that could be a slightly more, it's not a big difference, but a slightly more efficient way of doing it. And of course, you have to check and make sure that the thing is even if you do that, if the thing is odd, then you have to sort of make sure to omit the middle character in that process. So everybody good with palindrome checking. Okay, there are two more examples I want to use. This one is actually going to be part of your homework three. No, no, it isn't going to be part of your homework three. What was the first question on your homework to programming? here's, here's a test to see if you actually read the assignment by Did anybody read the assignment? I don't know. Okay. You've admitted it. That's the first step. symbol balance. Oh, sorry, stat class. Yeah, I got it wrong. That's the second problem is symbol balance. The first problem is a stat class, which I basically just did on paper for you. And but the second problem is a symbol balancing algorithm in a program. And so if we think about a piece of Java code,

50:34

and now I've gone and made this complicated, but whatever. You have various open and

closed symbols, like open curly brackets, and close curly brackets, open parentheses, close parentheses, open square brackets, you also have to deal with things like an open string and a closed string, or an open comment and a closed comment for block comments, things like that. Let's not worry about those just yet. Let's just worry about these. So if we have public class, my class, then we might have. And F, which takes some other value. The way we can check for balance is by using a stack algorithm. Because essentially, as we look, as we encounter close symbols, we want to look backwards through the open symbols, right? The most recent open symbol should match a closed symbol when we encounter it. Right? Right. So if you have a curly bracket here, that's going to close the closest curly bracket that opened, right, it doesn't close this one, it closes this one. So you're looking at the open symbols in reverse order. Further, you're not allowed to sort or interleave symbols. This is not acceptable. Right? They have to the closest ones have to match. And so the algorithm that you know what, let's call it, Donald, because right now, all I'm trying to do is check symbols, not whether or not the program is valid Java, can we live with that? Yeah, we can live with that. Okay, um, double, you know, what, I better make this a capital D. Because that would then that might compile if we wrote our own class called double. Moving on. Um, so the algorithm, which has absolutely nothing to do with any of the symbols in this text, other than the open or closed symbols would work as follows, we would read this file in and scan through it character by character, I think when you go ahead and implement this, what you're probably going to want to do is

DS Lecture 6 Page 11 of 19 Transcribed by <https://otter.ai>

you're probably just going to want to use a scanner on a file object, and then read it in line by line. And then for each line, go through it character by character. When you encounter an open symbol, any open symbol, push it onto the stack. So we push that one onto the stack, we keep going, we encounter another open symbol here. We push that one onto the stack. We keep going, and now we encounter a closed symbol. So whenever we encounter a closed symbol, we pop the stack. And then we look at what comes off the stack. And we compare it to the type of symbol that triggered the pop. And as long as they match. We're still good. And we keep going. So now I've taken care of that and I go here and I push this onto the stack. And I keep going and the next open symbol I get to is here, so we push the square brackets onto the stack. Right now how many items are actually on this stack? Three, there are three open symbols that are now shown. Here in the reverse order in which they were encountered. So now we come here, it's a closed symbol of sorts. So we pop the stack, we compare, they match. So we get we're good, we keep going, keep going, when we get here, that's a close symbol. So we pop the stack. that matches, keep going, we get here, pop the stack,

55:31

check, and they match. And then we get to the end of the file. At that point, we want to make sure that the stack is empty. Because if the stack isn't empty, that means there would have been a few open symbols that are still hanging out waiting to be closed for which there would be nothing to close. Does that make sense? So whenever you see an open symbol, push, whenever you see a close symbol, pop and compare to make sure they're matching types. And that's basically the whole algorithm. Now there are a number of error conditions that we might have. The first error condition, which I already just mentioned, was if the stack is not empty when the file is done, it means there are too many open symbols. Right. Another kind of error you might encounter is if you pop the stack and what comes off is of a mismatching symbol. And so that's another type of error. And the third type of error we might have is if we have a stack underflow exception. If you reach a closed character, and attempt to pop an empty stack, it means there was no open character to check against. Right? That would mean like a file that looks like this, just this would pop an empty stack.

56:55

Are we good with that?

56:59

DS Lecture 6 Page 12 of 19 Transcribed by <https://otter.ai>

Okay, so if you have the there are any symbol before you cross it out, why do you keep to check if they're the same after you pop it? Ah. So I don't know if they're the same until I pop it. Right? So you pop it off store and attempt variable, then you look and make sure it's the corresponding open symbol to the close symbol that you just encounter. That makes sense.

57:33

Oh, sorry. So um, so I was thinking like, if you don't know, if they're the same symbol, then why do you cross it out?

57:42

Because if they miss match, then the algorithms over, right? That means there's a mismatch. And that's it. I cross it out, meaning I'm simply removing it from the stack. I can't do this comparison until I've removed it from the stack. Okay. Could you compare to peak instead of popping? I mean, you could. But you're still terminating the algorithm, the moment you find the first mismatch, so whether you peak it or pop it, it doesn't actually matter. And if they do match, you absolutely do want to pop it because you want to move on to the other open symbols. Emily? Does that make sense? Yeah. Cool. All right. So those are the basic things for a simple balancing algorithm. But there are some complications, which you might have noticed in the homework. Imagine we have the following. I'm not going to write the whole code out. I'm just going to write the relevant pieces.

59:22

In Java code, would this be a mismatch error or not? What kind of error would this be, if any, or is this balanced? suggestions.

59:43

So what kind of mismatch so you're suggesting that when you encounter this, you'll pop off this and you'll get a mismatch

59:54

in actual Java does that Get interpreted as part of the Java code?

DS Lecture 6 Page 13 of 19 Transcribed by <https://otter.ai>

1:00:06

Actually, no, because we're inside of a comment. So what would end up happening? Oh, okay, you would get a mismatch, except the mismatch would be here. Because this would be ignored. These two guys would match, then you get here and it would try to match against the open parenthesis. So this would be balanced. Because what's going on inside of the comments would be ignored. So that wouldn't matter. But then this would close this. So one of the things one of the sort of the the nasty things that I asked you to deal with when you write your code, is I asked you to deal with this kind of a symbol mismatch. Okay. So I'm sorry, I asked you to deal with comments. Now, comments are tricky. Because they're involve two characters, not one. So whenever you encounter a slash, you have to look at the next character, and see if it's a star. And whenever you encounter a star, you have to look at the next character and see if it's a slash. Does that make sense? similar situation for strings. If you're inside of a string, and the delimiters, for a string are a little tricky, B, sorry, the delimiters for a string are a little tricky, because it's the same character. Um, but if you're inside a string, whatever's inside doesn't get considered. Is everybody okay with that? Any more questions on symbol balancing? Um, I do not need for you to handle escaped characters in this. And I do not need for you to handle single line comments with slash slash. Okay. So just, I think if you read through the, the, the text for the assignment, it's pretty clear on that. Is the algorithm just telling the program to stop pushing after any slash star quote, that is actually something you can do? Param,

you can, you can just stop until you encounter the close symbol for it. And so you can do that by having a separate Boolean variable that keeps track of whether or not you're inside of a comment, or a double quote, does that make sense? a shoe hang, you can't treat it as one symbol, you're only going to be able to read through one character at a time. So whenever you encounter a slash as a single character, then you're going to have to look to the next character to see whether or not it's a star. Does that make sense? Now in terms of pushing it onto the stack, you could probably just push the star onto the stack anything just to indicate that it's a comment. Okay, but yeah, it's not a single character. And that's a part of the complication. While we're talking about this, I feel the need to give some suggestions as to how to approach this problem. Because this ends up being kind of a messy piece of code. And the scope of this piece of code because there are so many things you have to take care of in it tends to throw people a little bit. And so what I would suggest that you do, what I would suggest that you do is that you first write your code to work and test it extensively. I gave you test files, but they're not exhaustive. And some of them are actually somewhat complicated. You shouldn't use the test files I give you initially, you should write simpler test files yourself. And the first thing I would do is just hold on one second, though, the first thing I would do is I would have you make it work just as normal, I would have you make it work just on curly brackets, nothing else. And then when you're sure it works on curly brackets, then make it work also for parentheses and

DS Lecture 6 Page 14 of 19 Transcribed by <https://otter.ai>

square brackets, because those are basically the same. And then test that. Once you're certain all of that works. Then and only then do you add, say strings. And once you're certain strings work, then and only then add the comments. And when I say you're certain that it works, I mean compiled and ran it a few times. The worst thing you could do is write one huge monolithic piece of code that's supposed to do everything that's laid out here in one go,

1:05:08

and then compile it. Because what's gonna happen is there gonna be a ton of errors, both compilation errors and logical errors the first time through, I don't mean that sort of as a negative thing. It's just at this stage of the game, there's always you've got monstrously lucky if it worked on the first go. And it's going to be extremely difficult to figure out what's wrong, if you wrote everything. So break it down into smaller subproblems. And make each of those subproblems work and test them before you go on to the next sub problem. That way, when you reach a point where it doesn't work, you know where to look. Because you know, what parts worked beforehand? Is everybody okay with that? feel like that's a really important thing for you guys to hear. Because I've noticed that that seems to be one of the biggest issues that that people run into. Um, I mean, is that something that was emphasized in 1004, trying to break the problem down into smaller testable pieces first, so you don't get stuck. Good. And if that didn't sink in, it's got to sink in. Now, because this particular program you're going to write is going to when it's done look like a beast of different FL statements. But if you do it in small pieces, you'll be totally fine. Okay. Side note, the third and final pro problem on the programming homework, although conceptually takes a little bit of work to figure out how to make it work, the amount of code you have to write is going to be something like 15 to 20 lines max. Whereas the second problem with the symbol balance here, this could end up being hundreds of lines of code. So there are two very different types of problems. This one is sort of a messy, make everything work together problem. And the second one is sort of figure out an interesting algorithm, and then it implements very elegantly. Okay. All right. So, with that we have one more stack, sort of application that I want to talk about. Out of curiosity, how many of you took art of engineering? with David vallencourt?

1:07:36

I see one hand raised. That is something okay. All right. I want fewer people that I would

have expected, like, more than 100 people in this room. Okay. All right. For those of you who took this with him, do you remember him being especially fond of his HP calculators? Does that ring a bell? The Hewlett Packard calculators? Yeah, okay. So, typically, when

DS Lecture 6 Page 15 of 19 Transcribed by <https://otter.ai>

you have like a four function calculator, or when you have like a graphing calculators, something like that, you write your expressions down, you type them in and you type in the first operator, sorry, first operand, then you type in the operator you want. Then you type in the second operand, Simon Diego, are you? Do you have a question? Or is that just a remnant of the previous thing?

1:08:43

Okay, do Did you have a question? Okay, just checking. No, no, I just saw hands still upside as well. So typically, when you have a calculator like that, you would, you know, first operand, then the operator, then the second operand, but in those old HP calculators that David was so fond of. And in fact, if you do finance, finance, people tend to like these calculators works a little differently. You type in the first operand, then you press enter, do something to indicate the first operand is done. Then you type in the second operand.

When you're done typing in the second operand. Then you type the operator that's supposed to apply to the previous two operands. So that would be three plus four. If you wanted to do something like this, that would be four minus one. Is everybody with me so far? Now you can do this for compound expressions. So you can say for One minus three plus. And so what's happening here? At Yep, this is RPN reverse Polish notation. What's happening here is that you're scanning through your input. And whenever you encounter an operator, you look back at the previous two operands, the most recent two operands, which should be a big ole hint, because you're reversing the order, the most recent two operands are the operands for that operator. And then it evaluates them, you end up with a three in this case, then it keeps going. It hits another operator, and it looks back at the previous two operands, which was here was a three. But now since this was already evaluated, it uses what it evaluated to as the second operands. So this ends up being three plus three. So this whole thing ends up evaluating to six. Everybody still with me?

These kinds of expressions are what are known as postfix. expressions. Because the operators come after the operands. These kinds of expressions that we are used to writing these are called infix, because the operator comes in between the operands. And as an aside, there are also versions where the operator comes first. And these would be called prefix expressions. All right, another term for these types of expressions. postfix expressions are also referred to as RPN expressions, which stands for reverse Polish notation. All right. And so those HP calculators that David loves so much are known as RPN calculators because they use reverse Polish notation. Why is it called reverse Polish notation? Well, obviously, because postfix expressions are sorry, prefix expressions are called Polish notation. So if prefix is Polish notation, postfix is reverse Polish notation. So why on earth is that called Polish notation? Because the dude who came up with it was polish, and they weren't being very creative. All right. So these are RPN expressions. Now one of the nice things about using RPN expressions or really Polish notation expressions, is

DS Lecture 6 Page 16 of 19 Transcribed by <https://otter.ai>

that order of operations no longer matters. Here in infix expressions, we actually have to concern ourselves, which operators have precedence over other operators, like multiplication and division, get handled first, then addition and subtraction and so on. In RPN, there is no concept of order of operations, the operators are applied from left to right in the order they appear in the expression. So you don't have any mix ups with respect to order of operations. So that's actually the nice thing about now, when we said that you look at the most recent two operands. When you hit an operator, let me give an example that makes that a little bit clear. Let's imagine we have the following expression.

1:13:46

That is a valid RPN expression. All right, moving through, you encounter the first you encounter the first operator. And so this applies to the most recent two operands, the two and the one. So it's actually there in reverse order, though. So the two is the left hand operand and the one is the right hand operand. So this is two minus one, which is equal to one. Then we go over here, we're at the plus you look at the previous two operands. The first one before it is actually what this part evaluated to, it's the one and the three, so this whole thing becomes three plus one. So floating point, notation is the same thing. Um, it's just a matter of how do you determine the delimiters between your your so we say tokens, right? So each one of these things, each number that we have here, and each operator that we have here is a token, and then you just have to decide whether or not a token is operator or an operand based on the types you expect. And so our variables here, this, by the way, is make sense for binary operators, operators that have two operands. I'm ignoring unary operators for the purposes of this discussion. So no unary minus sign, but you know, you can adapt it for that. Um, but if you want to have numbers that have multiple digits, we just have to indicate some kind of delimiter between tokens. And so when I have you actually code something up related to this, and that's actually going to show up on homework three, and it's slightly different context. But when you code up something like that, you just say, okay, tokens are delimited by spaces, okay. And so what we're gonna do is we're going to have a stack based algorithm for evaluating this. And that makes sense, because we're essentially reversing the order of the operands. So let's say we have an expression in RPN. And so we're going to have a stack. Now for this algorithm, our stack will only have operands on it. That's it. So it's just a stack of operands. So we're going to scan through this expression token by token. Now, the stacks job is to reverse the order of the operands. So that we're able to apply the operators to them appropriately. So whenever we encounter an operand, we push that operand onto the stack. And then we move on to the next token, also an operand, we push it onto the stack. When we get to an operator, we need to find the two most recent operands and apply the operator to them actually do the calculation. So when I'm at the minus sign, how am I going to get my two operands? What am I going to do? I'm going to pop the stack, how

DS Lecture 6 Page 17 of 19 Transcribed by <https://otter.ai>

many times Am I going to pop the stack, I'm going to pop it twice, the first time I pop, what comes off the left or right operand? The right operands. So the first one I pop becomes the right operand. Then I pop again. And that becomes so the the floating point isn't relevant if I could have called this 3.4 and 2.7. And those just would have been tokens. So let's it's really not much more than that. So the three comes off. Okay. Then we apply the operator to these two, we calculate the result.

1:18:18

Now this results is ultimately going to become an operand for another later operator, potentially. So what do you think we do with the result, we push it back onto the stack. So the algorithm is essentially this whenever you encounter an operate operand, push onto the stack, whenever you encounter an operator, pop the stack twice, apply the operator, take the result and push it back onto the stack. That's the whole that's the whole the whole thing. So we keep going. We push that onto the stack. Right now the stack has two items. We keep going. Now we get an operator. So we're going to pump the stack twice. First one that comes off as the right hand operand. Second one that comes off as the left hand operand, we apply it get a result, and we push it back onto the stack. How many items are on my stack right now? Just one, we keep scanning through our tokens and we realize now we're at the end of our list. And so when we're end of our list, we just popped the stack one more time. So you don't return the stack. You pop the stack one more time. And whatever comes off the stack We answer at a Does that make sense? Cool. All right. So that took us almost exactly to 655. We'll start up on Monday, by talking about some of the error conditions associated with this, then we'll do queues that will be a much faster

conversation. So that'll take up about half of Monday's lecture. And then we're going to start trees on Monday. And so I'm plan on reading sort of the first section of trees, up to maybe, but not including binary trees. And so what we're going to do is we're going to spend that half of Monday, and excuse me, and the, the entirety of Wednesday talking about tree algorithm trees, and stuff related to trees. And that will be the last bit of material on the midterm will be that part of trees. We'll get to binary search trees right after the exam. Okay. All right, folks. That's it for today. I will stop the share.

1:21:26

Thank you. hang out for a few minutes. But then I'm going to have to run what's up and ask you one question about the stack implementation, the size that you're referring to, if we're implementing a stack along the way of like an ArrayList, that has a size method, does the size? Is there a difference? Or can we use the size method to gain to get the size of the stack? Or does the size need to be? So? Yeah, why

DS Lecture 6 Page 18 of 19 Transcribed by <https://otter.ai>

1:21:58

not? Because the idea is that the number of sort of valid Well, you got to be a little careful with array lists, right? Because you're, you're not going to necessarily be able to use an ArrayList exactly the same way that I was using the array if you're going to actively be removing the items, right? If you're going to actively remove the items, that's a little different. But at that point, the size means the same thing, but then you don't really need it anymore. Because you're just moving from one end. So then, does it make sense to create an instance variable with the size of the array? It depends on how you're going about doing it, I would say, um, you should really think of it in terms of purely list operations, and nothing else, like just adding and removing at one end, and treat it like a list. And then you just sort of realize if I'm going to use an ArrayList using those methods, then it has the potential to be expensive. But if I use a linked list using those methods, then I won't.

1:23:09

Writer, okay. Thank you. Okay. Other questions? We're good. Alright, folks, have a good one. Enjoy the rest of your week. Oh, wait, Professor

1:23:24

Blair. I think there was someone asking if we have class on Monday. What's Monday? I think it's something like Presidents Day or something. Oh, oh, Columbia doesn't do Presidents Day. Okay. Sounds good. Does it? I don't know. I don't do a lot of days.

1:23:43

No, we know I'm in the spring semester. We don't do anything other than spring break. That's it. Okay, well, okay. This semester, we had the day off for MLK, but that's usually the day before the semester starts. Now. Nothing. We have class on Monday, for sure. All right. Sounds good. Sorry to disappoint. All right. Take care.

DS Lecture 6 Page 19 of 19 Transcribed by <https://otter.ai>

DS Lecture 7

Wed, 8/11 1:15PM 1:22:32

SUMMARY KEYWORDS

queue, tree, nodes, tail, dq, item, exam, size, array, head, operations, questions, length, add, stack, location, operand, implement, removing, list

00:02

All right. With me you chop your head there's nothing in it. So you ask yourself where is my trying to talk to me? Whichever way you have nothing. So you ask yourself where is my

Alright, let's get started somewhere to going a little bit okay. Little bit of very quickly shut their video off, you know, bit of both. Okay. Um, so recall that your written homework two is due this Friday correct. And their use up to 48 hours late that time on that if you must. And then the idea is that we will release the solutions for that right after that late deadline. So if you want you can take a peek at those solutions before you start working on the midterm, which is going to be released 12:01am New York time Monday of next week, correct.

07:03

Now homework two programming. And you know, I don't think I don't think you need to worry about too much of that right now. But, you know, you should definitely look it over and understand it. I gave you the extra weekend, right? So I didn't want to do on the exam. So you actually have until not this Friday, but next Friday, right before spring break for that. Does that sound good? All right. We have class today, we will be covering these things. We have class on Wednesday, where we will continue to discuss trees. All material on Wednesday's class will be reflected on the midterm the following week. And as you know, I announced it, we will hold I believe we said 5pm correct on Friday, an hour long review session that I believe I successfully scheduled as just another session of this zoom meeting that we are currently in. So that should be reflected in the list of zoom sessions. I put a link to it on the class on the class calendar, and it will be recorded, so it will automatically flow through. So if you don't want to spend time on a Friday afternoon

DS Lecture 7 Page 1 of 18 Transcribed by <https://otter.ai>

listening to me, you can watch it at your leisure. I do suggest watching it before the exam though, I do tell you a lot of what's on the exam during that. Okay. And by telling you what's on the exam, like, practically tell you what the questions are. Okay, that's sort of my style. So that's the plan. Any questions for me? Now I do that in lieu of a practice test, which I don't give out? Questions for me before I move on. Recommendations for studying for the exam. So I feel like the exam questions are going to be kind of closer in nature to the written questions on the homeworks probably than the programming. I'll get to that in a second. But easier, I think just because of the time constraints, and I convert some of them into a multiple choice format. Remember, we discussed this I believe, and I think I talked about it in the email I sent out but I actually split the exam into two equally length halves which you could do

10:05

Which you can do at your leisure during the 48 hour period. So you could do the first half, just after midnight on on the Monday and then wait until like 10pm. On Tuesday to do the second half, I don't care. The first half is done through courseworks quizzes. And it will be a collection of multiple choice and sort of numeric fill in the blank questions. So that's like a completely auto graded thing. Although I will give you instant feedback, I will wait till everybody's done before I released the grades on that. But that's the first half and format of the first half. Um, lots of questions about bingo on there. So remembering your big O notation will be quite helpful. And then the second half will be one possibly q programmings style questions as in, you can type up the code fragments that I'm asking for, but you're not supposed to compile and run it, as well as one or two longer form questions. Again, these are only going to be about, say about 40 minutes each these two sections, so there's a limit to how many questions there can be on each one. So that second half will involve a bit of programming. And that one's done in grade scope. Because grading that kind of thing in grade scope is much easier. And you'll be able to type up your answers or if you want, you can handwrite them, take a photograph of them and upload a PDF of your result. Okay. Um, the exam is closed book, and close notes. You can't compile anything, or anything like that really just the text editor, or whatever simple tools you use to produce what you upload are the only things allowed are the web browser, this is an honor system, because I kind of feel like proctorio is a bit of a dumpster

fire, and I really don't want to impose that on you. But on the other hand, the exam is really not designed as an open book test. So I'm gonna have to trust you guys on that. Are we okay, does that answer those questions? Christopher Margo. Good. Yes, the exam is timed, you have a window. And once you start a particular section, you got to finish it. So it's supposed to be an hour and 15 minute exams, I'm rounding it's one hour and 20, just for sat my sat like, so I don't have to do a 37 and a half minute subsection. And so the first half, you got 40 minutes from the moment you started to finish. And then the second half, you got 40 minutes from the moment you start to finish. But you have Wi Fi problems during the test, there's not a whole lot I can do try to situate yourself in a position where you have pretty good connectivity.

13:14

The list of topics will be released later on this week. But yes, I'm preparing one, at least in advance of the of the review session. But the truth of the matter is, is chapters one through three plus the first section of chapter four. And that's your list of topics right there. Um, I don't give specific practice problems, I'd say refer to the homework. And if you want to look through the book for things similar to the homework problems, that's fine. I do not know how many questions are on the first section, it varies a little bit. Usually when I do a fully hand written exam, it's like seven questions total on the test, I think I'm going to separate out three of them to the grade scope written portion. So the other sort of four questions I'll probably break down into probably about a dozen multiple choice slash fill in the blank, but I'm not exactly certain. It will include a bit of trees, whatever we get through on Wednesday, so I definitely will have spent at least an hour and a half on trees before that test. So yeah, absolutely. That'll be there. All right. Uh, somebody asked me about privately about folks who have accommodations. And so accommodations actually worked out really well like this. Um, I get a list a very detailed list from both Columbia, DDS and Barnard cards that tells me who has accommodations and what they are. And before you take the exam, I actually build in the extra time and time for breaks. So when you start your exam if you are entitled to extra time, it will be They're for you the moment you started, Does that sound good? Everybody's good with that. And usually, as I encode that into the system, I tend to shoot a note out to folks that I privately that to the folks that I encoded for just so you know that I'm aware of the situation, and I haven't missed you. Okay. So if you haven't gotten such a note from me prior to starting the exam, and you thought you were entirely titled to something, and when I say got a note from me, I mean, literally, then the hour or two before the exam period starts, that's when I encoded for real shoot me a note, and I can confirm it for you. Sound good? Other questions? Okay, um, do, I think I actually gave you a pretty good idea of what to expect for that exam, I think. So hopefully, you can get started for that. And also recall, I will not hold a live class next Monday, because I don't know when you're planning on taking your exam. So I'll leave that empty. And I will pre record a lecture or give you a pre recorded lecture that will not be on the exam that you can watch it really sure in place of that, because I don't want to lose the session. But I also don't want to eat up your time. And then we will wrap up with a lecture the following Wednesday, and then you have spring break. And after spring break, there will be a distinct shift in the starting music for the class. Okay, let's get started. Um, so the last thing that we were doing was we were talking about stack algorithms. And specifically, we were talking about the stack algorithm for evaluating a

DS Lecture 7 Page 3 of 18 Transcribed by <https://otter.ai>

postfix expression. And the one aspect of that, that I didn't get into last time is what kind of error conditions you might encounter and be able to detect when running that algorithm. So let's give an example of it. So let's say we have the following postfix expression. This one shouldn't have any errors in it. Alright, just reminding ourselves how this algorithm works. Whenever we encounter an operand, we push, whenever we counter

an operator, we pop twice ply the operator and push the result. So we push the three, we push the five, we push the seven, we have a plus. So we pop twice. First thing that comes off is the right hand operand for the plus because it reverses the order. Second thing that comes off as the left hand operand, we apply the operator five plus seven is if 12, please God be 12 is 12. We push the results. Keep going get the times good, we got a pop twice. First thing that comes off is the right hand. Second thing that comes off is the left and ply the operator, push the results.

18:30

And then when we run out of tokens, we pop the stack one more time. So when the stacks done, it should be empty. And whatever comes off, that is our answer. So that's the algorithm. Now, things that could go wrong. First of all, if I gave you the following expression, let's follow the algorithm. Now that expression to me, my eyes actually looks fine. So we push the four on goes on to the stack, were added tokens, we pop the stack stacks empty, we get four, so four evaluates to four. We're also clever, but it did work right. Now, imagine a scenario where I give you something that is slightly wrong. So let's say I give you the following. Okay, so we run through this algorithm, push the three. Move on, we get the plus we pop twice. First pop, good get the right hand operand. Second pop, we get a stack underflow and that tells me that I don't have enough operands right. If you pop an empty stack, at any point, it means you were expecting an operand and didn't get one. And so popping an empty spec stack tells you that there aren't enough operands there is another scenario though. Let's say we have the following. So we run through our algorithm, we push the three, we push the five, I'm going to stop running the arrow, we push the seven. Now we're on the plus. So we popped twice, get the aforementioned 12. When we apply, we push that result back on the stack. And now we're out of tokens. So the algorithm says, pop the stack one more time. And call that the answer. However, after you pop that stack one more time, you want to check and make sure that the stack actually is empty. Because if the stack is not empty, when the algorithm thinks it's done, and is pop the answer off, it means there were more operands, than there were operators to consume them. And so we're probably missing at least one operator if there's still stuff on the stack, when the last thing is popped off. And so those are the two major error conditions that you can detect in a postfix expression that you know, otherwise doesn't have errors, like all the tokens are valid tokens. The problem is with the expression itself. Is

DS Lecture 7 Page 4 of 18 Transcribed by <https://otter.ai>

everybody okay with that last piece of that algorithm and good with the review? Any questions on that? Which one, this one? The person who asked? Yeah, so that type of error would just be telling me that I don't have enough operators? Agreed. Cool. Okay. Anything else? All right, we would like to now move on from stacks. Get towards the end of chapter three. EU EU, eu E, cues, Q's. So Q is a FIFO, f i f o. First in. First out. And this is like you're lining up at a grocery store or something like that. Maybe lining up at a grocery store is not the imagery you really want these days. But right to cue, like in the sense of you line up in a store, the first person who shows up should be the first person that's processed. And then the person the last person who showed up should be the last person who's process. Does that make sense?

22:56

All right. Um, I think I would like to move on, I have a stupid little side story that I like to tell, which I don't think I'm going to do now. If we have a dead moment, you can remind me and ask me about it. I'm just out of curiosity, though, like, not all queueing systems are exactly like this. If you go to some of the whole foods in in like Manhattan, you know, they have like the multiple lines, which are color coded. And they tell you, which which line goes first. Yeah, those things confuse the hell out of me. But anyway, those are a little different than this. I'll tell that story another day. So this is just a basic first out queue. Now, let's imagine for a moment that we're going to try to implement this using some kind of

list. So just like a stack, there are a number of things. Well, actually, before we even do that, we should talk about what the methods are for this. Um, so if you want to add an item to a queue, the term usually is in queue. All right, that's adding an item to a queue. Um, it turns out that if you're going to use a queue in Java, and in fact queue is an interface in Java that provides these methods that we're going to talk about in this abstract data type. And in fact, linked lists and array lists implement the queue interface already. So there exists these kinds of methods already in there. But in Java, sometimes n Q is referred to as offer, like you're offering something to add to the queue. Alright, so that's adding something to the queue when you're removing something from the queue, so when we process the next item on the list, it's a dq And in terms of the Java terminology, you will sometimes refer to this method referred to as a pole operation. So pole takes an item off offer adds an item on, and Q and d q are the terms that I will be using for the remainder of this conversation. In addition to that, we might have some kind of a peek method, which will tell us who's at the head of the list without actually removing it from the queue, we'll have a size method like we always do, and we'll have an is empty. So we don't inadvertently underflow the queue. And depending on how we implement these things, we might also potentially have in is full. So this is kind of a similar setup to a stack, just we have the N q, and dq operations for adding and removing stuff. Now, just like the stack a queue as an abstract data type, or is a Java interface, there are a lot of

DS Lecture 7 Page 5 of 18 Transcribed by <https://otter.ai>

ways we could implement this thing. And, you know, there can be some consequences to the choices we make. So if I want to think about treating it as like a list, like a Java list, meaning something that's extendable, so we don't have to worry about whether it's full or empty. Um, n q would mean adding an item at one end of the list. And dq would mean removing an item from the other end of the list. So the adding and removing happened from opposite ends of the list. Now, you have to be careful if you're going to do that with an ArrayList, right. Because if you add at the front of an ArrayList, that's going to be order n and if you remove at the front of an ArrayList, that's also going to be order n right because you have to copy things over. So if you try to use an ArrayList, it's going to be a little bit expensive. Further, if you keep in queueing items back past the initial capacity, there's going to be that reallocation cost that triggers occasionally.

27:19

On the other hand, we also have linked lists. Now linked lists feel almost Taylor built for this right, particularly the link lists available in Java, because Java linked lists are doubly linked lists. And as you recall from our discussion of doubling linked lists, if you add on what if you add on either end, it's order one, right, adding on an end is cheap, it's iterating into the middle, that's expensive. So with a link list, and really matter which end is the head of the queue, and which ends of the tail of the queue, right? Because in the end, adding and removing from either side is a one or one just keep track and always add on one end, and remove from the other and you've got a queue. singly linked lists are actually a little dicier for a queue, right. Because if you have a singly linked list, at least the way we implemented them, where you just have a reference to the head, adding an item at the head is cheap. But removing an item from the tail is expensive. And so depending on which side, you end up setting the head and which side you end up setting the tail, one of your two operations would end up being order and the other would end up being order one, which isn't really ideal, because if I were using a W linked list, I could have gotten order one costs for both B and Q and the dq. Is everybody good with that? Cool. All right. So just like we did with the stack. Let's talk a little bit about how we might think about implementing a queue. If we were going to just use a fixed length array. So you've got an initially allocated array with an initial size and that's the maximum number of items you can have in the thing. Alright, so we're gonna do this simple sort of similar exercise where we might have our sort of our queue data structure. And we'll have our array that will

make it up and we'll specify our indices. Alright, so we've got here a capacity of 10 items. And initially, this queue is empty. Now we'll probably have a size that'll start out at zero. And we'll update that as we enter Q and dq items. But for this one, we're actually going to end up needing more variables to keep track of it. But let's, let's sort of talk our way through it to get to the point where we can see what those should be. So, um, for the sake of my sanity, and you're not having to stare at me right at things, I will call the method for

DS Lecture 7 Page 6 of 18 Transcribed by <https://otter.ai>

enqueueing II, and the method for D enqueueing D. All right, so let's start by n enqueueing, some kind of a values. were shot. Shall I put that in there? I think you think of sort of two valid spots. One may be slightly more convenient than the other. Where should I put that first item that I N q? index? At the which index? size? Sorry? Okay, so not size, because size isn't a valid index, but size minus one? Oh, I'm sorry, I missed it. I was thinking length, you're saying size, which would be items, zeros. I'm so sorry. You're consistent with everybody else. All right. So let's put it there. In fact, though, it's going to turn out maybe size isn't the best spot for that. But let's increase size to one. Okay. And so now, originally, that was sort of the tail of the list, right? We added the guy on at the tail. Now let's thank you another item. Where shall I put this item. So one option would be to put it also at items zero, but then that would mean shoving what was used to be at item zero over by one, which would make that potentially for larger collections would end up making things. Quarter end. So I like the idea of putting that at location one.

32:21

Right, and then we increment size two. So at this point size appears to represent the tail of the queue, right? That's the end of the line. That's where we're adding stuff. Correct. And zero corresponds to the head of the queue. That's where we should be removing things from right. Because if we're supposed to be removing them in the order in which they came in, if I dq, it's the three that should come off. Next, let's maybe add one more item. So we'll keep putting that there. Okay, so now let's dq we know that the dq oops, this is now let's dq we know that the dq should return the three. But what do we do? What do we do with the rest of this stuff? We could move them over. But if I move them over, what does that mean for my bingo cost for dq? costly, it's order n. And so I'd rather not. So instead, yeah, maybe we should keep an instance variable for the start. Maybe I'll call it head just for. And right up until the point after just after i n cubed, the five heads stayed at zero, even as

34:12

us me even as I was adding more stuff. Now for a variety of reasons that we will see shortly.

34:26

It's going to be convenient to also have a tail variable. So head will tell me where the first item is. and tail will tell me where the last item is. Okay, that's what we're going to do for now. So the suggestion that several people have said now is when I dq, that is when I take the three off.

34:54

I can increment head by one. And so somebody asks, What's the difference between 10 Alan sighs. Well, I mean, aside from being offset by one, because I'm using tails to tell me the location of the last valid element, and size tells me the total number of items, there doesn't seem like there's much difference between them right now, they're kind of tracking the same way. As they both start at the beginning. And every time I add something, they get incremented by one. So, right now, there's not a whole lot of difference, but we're going to get to a difference in a minute. Okay. So now, there's no item in locations zero. There are items in locations one and two. Let's end cue a couple of more things. So thank you a bunch of more stuff. No, I can have a second forum there. But let's not. So enqueue tu

tu a seven and a six. So the two would go here, the seven would go here, the six would go here. The size is made its way. Well, what sighs now I've dq one item. And I've added 12345 items, right. So the size is five. But the tail is also five. And you'll notice that the size and tail have actually slightly diverged already, right? That used to be offset by one because size was telling me how many items there were in the count of the item started at zero. Tail told me the location of the item. But now because I dq an item in the start of the queue was at location one, size and tail are no longer quite tracking each other the same way they used to be. Does that make sense? You could potentially calculate the size using head and tail. But it ends up being easier to keep them separate. So for my own sanity, I'm going to do that right now. Doesn't this approach take away from memory efficiency, though? At first glance, yes. And I'm actually sort of working our way over to the end, so that we have to do something to deal with that. We're going to make sure that we're not going to waste space, but it certainly looks like I'm going to waste space at the moment. Let me dq a couple of times. So I've n cubed, the one sorry, the 345 I dq the three Inq the two, seven and the six, let's dq a couple more times, the next thing that comes off is the for the next thing that comes off as the five, so I dq twice, and so now the size is no longer five, but three, the head remains five, but the tail actually starts at location, sorry, the tail remains five, because that's where the last element is. But the head now is at location three. So now you can see that the size and the tail have completely diverged from each other. All right, we've still got some more spaces here, we got four more spaces. So let's end q a few more things, let's end q 71, or 71. Let's end q 65. And let's end q one. And maybe I need one more to make this mean anything. So let's then let's end q nine. So we put 71, tack it on at the end, then 65 then one, the nine, the head stays put the tail now corresponds to location nine and the size, we actually have 1234567 items in there. Okay, so our array has seven valid items, the starting and ending points of those valid items are given to us by head and tail. We have a separate count to size just for bookkeeping

DS Lecture 7 Page 8 of 18 Transcribed by <https://otter.ai>

purposes. And then we've got these three spaces at the beginning, which are empty, right? They appear to be wasted. Because if I try to end cue another item onto this thing, well, it seemed like I kept just incrementing tail and putting it there. But but then I'm at a space. And so every time I dq it sort of felt like I was decreasing my overall capacity. So this paper is getting a little filled up. Let me redraw my array A little more cleanly. So we had a 10 item array Oops, locations 01 and two didn't have any meaningful data in them. So I won't actually draw anything in there. I didn't actually remove them the same way, I didn't actually remove them in the stack. But nothing there matters. So I won't draw them in. And then the rest of the items were filled up as follows. size was seven. ad was three, and tail was nine. So when I N q, one more item on, say I want to end q 11. The suggestion was rotate. So what do we mean by rotate? Where shall I place the 11? When I N q it onto this thing such that I don't waste any space. I don't want to move things around. Right? So we don't want to move stuff because then that would be order n. But somebody just suggested zero than zero again. And Tiffany said Can't you just overwrite those first three indices to use up that space? That's not quite head minus one. You don't want to fill up backwards, that's going to end up being a little bit harder to calculate. Because as I dq stuff, head will move in those won't work. So instead, I'd rather wrap around make it kind of circular, right? Yep. So if the tail is equal to I guess, in this case, the length minus one, or better yet, the idea is when we add an item, we don't add an application tail. Where do we actually add an app? Forget about being all the way at the end, if we were somewhere in the middle. What is the index in terms of tail where the actual next n cubed item gets written? In terms of tail? Let's not get fancy. telcos one. Yeah. So tail plus one works. So it's really when tailed plus one is equal to the length, we move back around to zero. Okay. And so let's put that there. And so tail now becomes zero. All right. But here's the thing.

42:50

We can implement this without an if statement. How could I force it to wrap around without any need for an if statement, what operator will do the job? The mod operator, it's not the size, but mod the length, the length of this thing is 10. Right? And so if I'm currently trying to place an item at anywhere from zero to nine, zero to nine mod 10, just gives me the same value back. But the moment the value I'm modding by 10 hits 10, it wraps back around zero. Okay. So if I wanted to implement an N q operation on this thing, so let's, let's try that. So an N q operation would look to be a void method, it doesn't actually, it doesn't actually return anything. Most likely, when we write this, we would probably write it as a generic. But let's for the sake of our sanity here, just use intz. For the moment. Let's say I'm trying to N q x and remind remember this array we're going to call a. So the nice thing about keeping the size as a separate variables, we don't really have to do any arithmetic between head and tail to see whether or not the thing is full or not. All we need to say is if size is equal to a dot length. We know we have an overflow. And we can just throw an

DS Lecture 7 Page 9 of 18 Transcribed by <https://otter.ai>

exception. But if size is not equal to a dot length, what we can do is we can increase tail. So we set tail equal to tail plus one. The whole thing. mod a dot length. Oops. So tail equals tail plus one, mod a dot length. And so what happens is it increments tail by one. And then mods that by the length, which does nothing to it if tail plus one is less than the length, but the moment that tail plus one becomes the length that wraps back around to zero, so that handles the rotation, sort of the circular nature for us. What if the queue is actually full and looping around would cause values to be overwritten? Would we be able to determine if it's full or loopable? With a simple if statement? Yes. That's why we keep track of size separately. Because we know they haven't crossed of sizes under a dot length. Does that make sense to the person who direct messaged me? Cool. All right. We do, however, have to increment size explicitly, there's no modeling math. All right. And then we just take a look at location tail, which we've already calculated correctly, and gotten the rotation to work with the modulus. And we set that equal to x. And that's it for our n q method using the module is to rotate us around. Are we good with that? Cool. Okay. So let's pretend a few more operations occur. And we end up with the following array. So let me write my indices first, so I don't mess up. Alright, let's do let's pretend some sequence of n Q's and D queues occur. And remember, every time we dq head keeps making its way towards the end. So let's say we end up with the following. Alright, so in this case, the size of my queue is three items. There, the head of my queue is the location nine. And the tail,

47:59

because it did wrap around will be location one. So I'd be adding here, here, here and removing from there. So when I dq this thing, it ends up being kind of a similar scenario that we had before. Once the five gets D cued. head will then wrap back around to zero as well. So remember, our dq operation was basically extract the value at location head, and then increment head by one. In this case, we extract the value at location head, then we increment head by one, but to make sure it's circular we modded by the length of the array, and so a dq operation which would return the values. So since this is an array of intz, for the moment, we'll use an integer returns right. We don't want an underflow. But that's easy to check with our size variable. So if our size is equal to zero, we underflow.

49:21

we store our old value, or the value we're D queuing in a temp variable that will be the value at a at the original location of the head. And then, well, we're removing it so we decrement size but that's just an absolute decrement and then we set head because we're incrementing head to move to the next head. So head will be equal to head plus one. But

DS Lecture 7 Page 10 of 18 Transcribed by <https://otter.ai>

just like before we move audit by a dot length. So that it wraps back around to the beginning, if necessary. Any questions? We're good with the queue based, sorry, the array

based implementation of a queue.

50:25

Oh, you're right. I missed my return. So now we need to return the old value with cash.

50:40

So, think about how modulus works. modulus is the remainder, when you integer divide one value by another. So if the length of this thing is 10, what happens when I divide one by 10? I get zero and my remainder is one. Correct? When I divide two by 10, I get zero and my remainder is two. So as long as the value you're modding by the length is smaller than the length, you just get the same value back because the mod gives me the remainder. When the value reaches the same thing as the length $10 \bmod 10$ is the remainder when you divide 10 by 10. So so 10 divided by 10 is what one with a remainder of zero. So the mod returns zero. That's where you're after, wait a second, why are we good? All right. Maybe we can do the last day.

52:06

I don't know what you mean by why we adjust the head the way we did. So let me Let's see, again, the head is here. The tail is here. We're always adding items on, hold on, we're always adding items on at the tail end removing items from the head, we don't want to decrement because if this is the head, the two than the seven, then the six than the 71 should come off in that order. The idea is we don't want to copy these items backwards in the array because that's expensive. So by simply incrementing head, we just come to the next element. We want to return each time. Christopher Did that help? Cool. Oh, good. Good. Um, all right. And then so Shuang asks when array is full, we're trying to add more elements to how should we modify the array? If the array is full? That's, that's an overflow. You don't actually want to be able to with a fixed length array to add items beyond its total capacity. So you would throw an exception? Does that make sense? Yeah, we're not going to win. So the idea here is I'm trying to describe how we would do this if we're not resizing the array. Good. Cool. All right. Um, and reassigning the values of head and Taylor order one, right? So yeah, if you look at my nq operation, nothing here is iterating. Right? everything that we're doing is just a fixed operation plus one mod eight out like whatever is so there's like a comparison, an ad, add an assignment, a mod another addition and other assignment. It's like five or six operations, which is independent of the size. So that's

DS Lecture 7 Page 11 of 18 Transcribed by <https://otter.ai>

constant, which is big O of one. And if we then look at our dq , it's exactly the same thing. It's a fixed number of operations independent of the length. Right. All right, other questions on a queue? Um, well, I mean, I guess you can sort of think of a queue is sort of an extension of a list. So in that sense, maybe, but we really don't. Usually when you use a queue The whole plan there is to just be using n , q and dQ is your way to deal with it. Um, and that ends up being kind of useful in terms of going through things in order. And so we wouldn't directly use an iterator most of the time. Is that okay? All right. So I don't have a whole lot of good examples of where we might use a queue just yet. We're going to use queues tons and tons of times, and other algorithms. And actually, when we get to some tree traversals, although looking at the time, we probably won't actually get to talk about that, until Wednesday, we'll see an example in terms of trees where we can use a queue as a data structure. Um, I mean, this isn't in the textbook, and I'm not going to test you on it. But just to give you an idea of something that you might use a queue for.

56:02

There's a problem, there's actually a mathematical problem. It's referred to as the josefus problem, which is comes from sort of a biblical story, which I'm forgetting the exact details of, but it involves a bunch of soldiers standing in a circle. And there are like, yeah, the circle very good. A bunch of soldiers standing in a circle, say there are and soldiers standing in a circle. And then they, they say, count off up to say, five. So the first person in the circle says, Okay, one, and then going around clockwise, the next person in the circle

says two, and then the next person says three, and the next person says, For the next person says, five, they're like, great, we're gonna kill the person who just said, five, good, he's dead gone out of the circle. And now we go to the next person going around clockwise, say, hey, you start counting at one again. And then when we reach five, again, we're gonna kill him. And so you keep going around. This sounds kind of bloody that Well, I mean, that's the Bible for you, at least the Old Testament. But maybe let's, let's maybe not make this so bloody. Let's make this a children's game instead. So you have a bunch of kids sitting in a circle, you have n kids sitting in a circle, and you have them count off to k . So first person says 12345, kid gets to five. You ask him to leave the circle. And then you start counting again. And whoever is left, there's going to be one person left in the circle after you keep counting around, that person wins. All right, so to draw it out, let's say we have people in a circle, they have numbers.

57:58

Okay, so you got 10 people in a circle. So n equals 10. And K , that's the thing we're going to count is five. So this person says 12345, that person leaves the circle 12345. Gone. 12345.

DS Lecture 7 Page 12 of 18 Transcribed by <https://otter.ai>

We skipped over that person, because they're gone, go to the six instead, gone 1234512345123451234. And then you loop back around 51234512345. And in that case, three wins. Okay. So it's not quite musical chairs, because it's deterministic based on n and k , not just a random stopping point. And so there's actually potentially is there could potentially be an analytical solution to this, like I give you an n k , can you tell me who wins just calculate it. But that's complicated. So instead, maybe we could just simulate it. And the way we would simulate it would be with a q , UN q , everybody in their initial ordering, so you would create a q 123456789 I really should have made this smaller. That's the head of the queue. That's the tail. And then each time you count you dq

59:51

and if it's not time to kill them, I mean time to remove them from the circle, you re n cue them on to the end. So you keep D queuing until you reach k . So you keep D queuing and then re n queuing until you reach k . And then the one who happens to be K , you just don't read and queue them onto the queue until you can just circle around in a loop, D queuing and n queuing, omitting the ones at the case time until there's only one item left on the queue. And that's the winner. So you can kind of simulate this problem using a queue. Okay? All right. So somebody says this is basically Eeny meeny miney, moe, or choosing who's in a children's game if you want the children's version. Sure. Um, alright. So that's actually it for chapter three, we are now beginning chapter four, which you should start reading if you haven't already. Trees. So trees are a slightly more complicated data structure than lists, but still quite simple. On. And if you read the preliminary sort of introductory paragraph to the chapter, you know that one of the nice things about trees is that they will kind of be they'll be critical, ultimately, in our ability to get operations for sort of searching on collections, or in this case really sets that our order log event rather than order and that sort of are ultimately going with that, when we get to a very specific type of tree, known as a binary search tree will start to have kind of average case operations, that costs order log event time instead of water and time, although worst case the order in and then from there, we'll get these trees that balance themselves, specifically something called a vl trees that will allow us to guarantee log event time operations, rather than sort of these linear order and operations that we're dealing with, with a list. So that's the where we're going with it, although I don't think we're going to get there by the time we hit the midterm, but let's define a tree for ourselves. And in fact, a tree can be defined recursively. And that's really an important thing for us, because almost all of the operations that we're going to be written on trees in this class, are going to be recursive operations. So if you love recursion, this is where things are going to start going great for you. And if you hate recursion, this is where you're gonna learn recursion really well, and

then you'll stop hating, okay. Or maybe you'll keep hating it, and then you'll hate me. And hopefully, it doesn't go there, you'll like, maybe, we'll see. So we define a tree as a

DS Lecture 7 Page 13 of 18 Transcribed by <https://otter.ai>

collection of nodes, all right. And there's one distinguishing node, which we will call the root node of the tree. All right. And then each root node can have zero or more subtrees hanging off of it. Oh, say T one, T two, T three, T for each of these triangles that I've drawn here are other trees that are hanging off of this root node. All right. And each of these trees that are hanging off of its root node consist of their own root node with potentially zero or more sub trees hanging off of them. Now, if we were just going to draw this in terms of nodes, maybe we would draw the following. A might be the root of the thing. And then it might have sub trees rooted at B C D and E. Maybe B has zero sub trees hanging off of it because I said you're gonna have zero or more sub trees hanging off of it. Maybe C

1:04:58

only has one sub tree hanging off of it. And that sub tree happens to be rooted at node f. and F has zero sub trees hanging off. Maybe D has two sub trees rooted at E, F, G, G, and H. And maybe G has sub trees, i and j, which happened to not have anything hanging off of them. Maybe he has k hanging off or something like that. So these nodes Ah, they're a special case of a network. All right, these nodes? Well, okay, so the Fibonacci numbers, when we sort of sketched out the recursive calls that were made to calculate the Fibonacci numbers, the sketch of those recursive calls kind of worked its way out into a tree like structure. Okay. All right. So nodes that have one or more children, I would call them interior nodes. So A, C, D, G, and E would be interior nodes. And nodes that have no children, I would call them leaf nodes. Right that have no sub trees? I shouldn't have said children yet. I'll get to that definition a second nodes. Interior nodes are nodes that have sub trees hanging off of them. leaf nodes are nodes that have nothing hanging off of sound good. So these are interior nodes. These are leaf nodes. Does the root node of the tree have to be an interior node? Or can it be a leaf? Which one. So you can only either be interior leaf, you can't be both. But a route is that a valid tree? I define my tree is zero or more sub trees. So this is the root of the tree, but it's also a leaf, because it happens to have no sub trees hanging off of it. So the root node could be an interior node or a leaf. Okay, we're good so far. All right. Now I had used the term child, but I probably shouldn't have yet. So let's do this. Now. Um, first of all, these arcs that I'm drawing between nodes, you'll sometimes hear them referred to as edges. Alright, these are the edges that connect the nodes. A node that has sub trees hanging off of it, those nodes that are directly connected below it, those are child nodes. So B, C, D, and E, are the children of a. So you can say that A is the parents of B, C, D, and E, a grade. B, C, D, and E are often referred to as siblings, right? So we're going with a family tree thing here. And you can have a grandparent grandchild relationship as well. A is F grandparent, f is A's grandchild. All right. I guess if we're going with that notation, a D would be F's aunt or uncle. Sure, why not? Actually, uncle is a term that exactly as I described it, here is a term that's actually

DS Lecture 7 Page 14 of 18 Transcribed by <https://otter.ai>

used in certain algorithms. F and I, what might I call them? Well, that's a bad one, F and G, what might I call them? They're not siblings, but they're on sort of at the same level.

They're cousins. Right. All right, so we can kind of go with that whole family tree hierarchy thing going on here. All right. Now,

1:09:41

excuse me, hold on a second I need a drink. You will sometimes hear the term a path from one node to another. So the path from A to F would corresponds to the sequence have nodes or sometimes we'll call them vertices, but we'll say a sequence of nodes that we have to visit to get from A to F. The length of that path, though, counts the number of edges that I have to traverse to get from A to F. So what is the length of that path from a

to s? It's in fact two, what is the length of the path from A to I?

1:10:35

What's the length of the path from D to h? What's the length of the path from H to itself? Zero, good.

1:10:49

So we have these paths. And we have these path lengths. All right. Now, there are two additional definitions that are going to be rather important for us, they're going to be the depth and the height. Let's start with the depth let me redraw a tree for us here is slightly different tree. Now, I might note that a lot of the time when we write our trees, these links are one way, we have a link from the parent to the child. But usually, there are some narrow cases where we have a link back but usually not. And so if we need to move back up the tree, we usually end up having to rely on recursion to move back up to the tree like we would make a recursive call on a child. And then when that recursive call returns, we return to the original call on the parent. So we kind of leverage recursion to move back up along the tree. So usually, we're moving down when somebody asks, Are there paths between nodes like J and J. Or say like being C. So that's what I was saying, we really don't think about going back up the tree. So we don't really think about moving back up and then down, we really think about just that path down. Alright.

1:12:38

Does that make sense? Cool. All right. So that's that. So depth, the depth is defined. So the DS Lecture 7 Page 15 of 18 Transcribed by <https://otter.ai>
depth of a node is the length of the unique path from the root to that node.

1:13:19

So because there aren't really any cycles in this, there aren't child links that link back up to things that were higher up in the tree, we're only moving down here. So we don't have links like that the path from the root to a node is unique. So the depth of the node is the length of that unique path. So what the length, so what is the depth of H. The unique path from the root, in this case, a two, h is A to C, C to E to H, it's three. So the depth of H is going to be three, the depth of E is going to be what? Two, in most cases, no child nodes do not have references back to their parents. The depth of f and the depth of G are also what they're also to just like E, the depths of B, C and D are old one, and the depth of the root a is what? Zero. And so sometimes it actually helps to think about trees in terms of levels, which sort of correspond to generations if we're going with the family tree representation here. So all the nodes that are of the same depth are considered at the same level. So level zero of the tree would be a level one of the tree would be b, c, d, e, f, g, h. Level two of the tree would be E, F, G, and level three of the tree would be h. Sound good. So that's depth. And that's an important term. But another really important term is the height of a node. So the height of a node is the length of the longest path from that node to a leaf. And remember, we can only go from parent to child here. All right, so the height of say, A, there are a number of paths to leaves, but we're asking for the height of the longest path to the leaf, the furthest number of steps that I would potentially have to go. So the height of a would be three, because the longest path to a leaf is the path to H, which is three, the height of C would be two, because the longest path from C to H, for the longest path to a leaf is from C to H, which is a height of two. What's the height of D? One, what's the height of B? It's a leaf. So a leaf has a height of zero. So b, h, F, and G all have the same height, even though they're at different levels. The height of the tree itself is the height of its root. Okay, we're good with that. All right. So it's 654. So I think I'm close enough to being out of time, that it's going to be very hard for me to start the next topic. So I'm, I'm working out of chapter 4.1. Right now, and what we're going to do on Wednesday.

1:17:41

So somebody said, Sorry, okay, let's back up. Someone said, Could you clarify what you

meant by leaves having a height of zero? So we said that the height of a node, okay, any node can have a height, not just the root, the height of a node is the length of the longest path. And remember, these paths only go down in the tree, from that node to a leaf. A leaf is sort of the end of the road. There's nothing below it. So the longest path to a

DS Lecture 7 Page 16 of 18 Transcribed by <https://otter.ai>

leaf from a leaf is just to itself. So there are no steps to get to itself. Hence, it's zero. Briana, does that make sense? We're good. Cool. All right. So what we're going to talk about next class is we're going to talk about some strategies for implementing these generalized trees, trees that can have an arbitrary number of children at each level, sorry, at each node. And then we'll start talking about some examples of where trees are useful. We'll talk about some implementations of it. And we'll talk about something known as tree traversals. and tree traversals are going to be wrapped around recursive algorithms that we're going to use to iterate through every item in the tree. Okay. All right. So that's where we're at. I'm going to stop the share. Now. I can find my mouse cursor. There it is. Alright, so I can stick around and answer any questions if you have any, but otherwise classes over.

1:19:27

Hi, Professor, I have a question. Is it okay, if I asked you about the homework or do you prefer about the lecture? Go ahead and ask. I'm not going to open your code up right now. Oh, no, that's that's not I just had a general question. For the for implementing the stacks, the two stacks with one array. I'm sorry, one of the written questions, right. Yes, go ahead. So do we have to include something that accounts for changing the, like changing the length of the array if it's overflowed or no, so the array size in this case is fixed?

1:20:15

So, yeah, you don't have to account for that. Okay, thank you so much. Other questions? Professor? Yes. Um, I asked in the chat, I was just wondering what would the height of the tree be and the picture that you drew.

1:20:36

So I don't have it in front of me anymore, I put it back on the stack, but if you have the drawing that you had in front of you, um, so I think h was the sort of the furthest leaf from the roots. And I think that was a set of step of three. So if you figure out what is the height of the root a, that would be the height of the tree is the height of the root, and that was three. Good.

1:21:03

Thank you. Other questions? Hi, professor. in this, in this implementation, you are showing

DS Lecture 7 Page 17 of 18 Transcribed by <https://otter.ai>

us how to build this queue, you use like a standard array, is did you do that just as an example, because you mentioned

1:21:26

so there could be scenarios where you want a queue with a fixed size. Like if you're simulating a grocery store, which only has a certain capacity where the line could be that that might be meaningful, and what it is you're trying to calculate or simulate? I see. So if you use the doubly linked list instead, would you still need like the head? to do any sort of rotations on that, you would just make sure you add items at one end and remove items at the other. So you could safely add items at location zero, and remove items at location length minus one without any problem, because when operating at the endpoints of a doubly linked list, you're still having to order one operations.

1:22:16

Makes sense? Thank you. Anything else? Alright, I'm going to stop the recording now but I'm not actually going to stop the There we go. Stop recording.

DS Lecture 7 Page 18 of 18 Transcribed by <https://otter.ai>

DS Lecture 8

Wed, 8/11 1:19PM 1:22:44

SUMMARY KEYWORDS

tree, node, traversal, midterm, order traversal, print, pre order, binary tree, class, left, root, children, recursive, subtree, questions, recursion, root directory, files, left subtree, directories

04:39

Alright let's get started um I get the general impression that everyone one is terrifically exhausted because of midterms and homework and everything else is that seem to be An accurate assessment of where the world is right now. Good when I feel less bad being exhausted and overwhelmed myself, alright, so I'm discussing reasons why one might be exhausted and overwhelmed. Do you have a homework due on? Friday? Correct. So this Friday is just the let me just get my thing to be, do not disturb. Okay. So this Friday, I believe we have the just the written portion correct. And you can use up to 48, late hours on that. So that's until Sunday, if you use late hours, and then at 12am, Monday, the solution set for the written is automatically set to go live in the Files section. Because that also corresponds with when the, the midterm starts to become available. And we spoke extensively about how the midterm was going to be set up last class in terms of two parts, 40 minutes each. Once you you can take them at any point during the next Monday and Tuesday, and they don't have to be taken at the same time. I'm not gonna spend very much time talking about the midterm today. We have a review session scheduled for 5pm. On Friday, correct. Right. And that's just another session of the same zoom meeting that we're using for the main class, it'll be recorded, it should feed through to the recording library immediately after Well, whatever the lag is. And prior to that, I'll send out a topics List, but it's really chapters one through the beginning of four, which is what we're working on right now. Okay. And I'll talk a lot more about that, then. For those of you who aren't able to attend, or those of you who are timeshift. In this lecture, what I will say is that, regardless of whether you attend the review session, I do suggest watching it even if it's at one and a half times speed, because I will spend a decent amount of time talking about sort of the questions the style of question I'm going to ask, okay, so you'll get a lot of hints as to what's going to be gone this Friday, next Friday, the exam will be over, I don't see any reason for a review. Next Friday. You have a programming assignment associated

DS Lecture 8 Page 1 of 20 Transcribed by <https://otter.ai>

with homework to which will not be due until next Friday. So I given you a little bit of time beyond the midterm for that. And then homework three, which will be based in part on what we've been doing. We'll go live. I'm tempted to put it out before the midterm, just because there are a few aspects of it that might be relevant. Because it's cumulative through today, the midterm, but it's not. There's still so much we have to do, I think I may just hold off. So it's not to worry you with something else. So early next week, I'll post homework three,

08:30

you'll have the spring break to work on it, you'll have next week to work on it. And in fact, I won't have it do the week immediately after spring break. We'll have to do the Monday be even beyond that. So you'll have plenty of time for homework three, which actually ends up being one of them bigger assignments. Next Monday's lecture, we will not do, I will post the pre recorded lecture in its place, which you can watch at your leisure. And then next Wednesday, we'll have a lecture prior to spring break. We shifted office hours so that we were moving office hours out from the period when the midterm happens because it's really sort of unfair for us to have office hour questions while the midterm is going so we push some people back to before the midterm and we push some people for after the

midterm to give you some extra hours to help you out with homework to I will retain my Monday office hours since people seem to come to me for other reasons. piasa will go private, not private, but in inactive during the midterm period. So we will only allow private questions, which are for logistics questions. But all that material will remain available. And then I'll reactivate it as soon as that turns over. Any questions for me? We're good. Okay, cool. So, trees, we spent last class talking about various definitions associated with trees. And we were talking about these generalized trees where a node could have one or more children. And, yeah, we talked about what the definition was the depth what the definition of was the height, what I want to talk about now a little bit is kind of how we might start implementing these things a little bit. And we'll talk a bit about what are known as tree traversals. Okay. And treat reversals are where we go through and visit every element in the tree. So is that consistent with where you remember us leaving off? Good. Alright, so sort of like we had with a linked list, a tree is really defined by its root node, right, and this was our root node here. And so if we're going to have a public class tree, really the only instance variable of note here is the root of that tree, which will be of some sort of type tree node, alright. And probably, these things will end up being generic, because we want our structures such that they can hold kind of anything. And so the real work happens in the tree node class, which in all likelihood would be a static nested class inside of tree. But for the purposes of this sketch, I'll draw it outside. And so we have our class treenode.

DS Lecture 8 Page 2 of 20 Transcribed by <https://otter.ai>

12:02

Now, our tree nodes are going to have to have some kind of some kind of payload. So you know, we have our data. But what else do they gonna have to have? What else do they have to reference? They buy that have to reference their children. How many children could this general tree have? Any amount? So how would you suggest that I represent this? I can't just have like, tree node one tree, node two, node three, that's not going to work? How would I do this? What would I use is my data structure, a list a list of tree nodes, that sounds good. So our first stab at this a little different from what it shows in the book, we'll switch to what the book does in a second would be any kind of List of tree nodes, right? Because all of the child nodes are also tree nodes. So we can make them a list. Could be an ArrayList, could be a linked list doesn't really matter. Let's, let's make it a, let's make it an ArrayList for the moment. So we could have an ArrayList. And the contents of the array lists themselves should be tree nodes.

13:42

of type T, that's a little you know what, let me rewrite this here. So we have class treenode. We have our data. We have ArrayList. And our ArrayList is of type treenode. Have type T

14:17

and we'll call that variable. Let's just call it children. And that right, there could be a way we could represent our tree sound good. Oh, this is very straightforward. Now, what does t actually T is a type variable that's T is the same thing as when we were saying any type in the textbook. Sound good?

14:54

Whatever, it's a tree of whatever you want. We'll talk about that in Okay, so your tree is a is a collection of information, whatever that information could be. So as I said, Why is the nested class static do not need to link back out to the outside class? No, we don't. Tree nodes do not manipulate the outer tree, the tree just manipulates the tree nodes. It's exactly the same logic as to why the node class in linked list was static, because it doesn't manipulate anything in the outside link lists. In fact, these trees, have we done trees before in this class really. If I I think we have, I think a linked list is just a tree. a wait list is just a tree where we allow one or zero children, and no more. So it's a degenerate tree. Like saying, degenerate. Um, it's a degenerate tree. And so when you start thinking about

linked lists that way, then you suit. Oh, man, you can tell I'm sort of losing it a bit. But I
DS Lecture 8 Page 3 of 20 Transcribed by <https://otter.ai>
really found that funny. Somebody in the chat said it's a stick. Okay, I really did find that
funny. Why is it static if we have multiple node instances? Um, so static simply refers to
the ability of those nodes to access instance variables of the outer class. Right? You can
have as many nodes as you want, none of them need to access the outside. Okay, so
static kind of refers to something a little different. Okay, then yeah. Alright, so I think we're
good there. So let's try a slightly different representation. So, the tree class itself would be
sorry, hold on.

17:24

No, I'm not sick. That was actually a piece of the espresso bean that I prior to class
starting getting stuck in my throat. I feel like I have to clarify that because several of my
colleagues are teaching their classes remotely while also dealing with COVID. And
thankfully, that's not me yet. Um, all right. So our tree is the same thing. But we could do
our tree node a little differently. do the following. Instead of having an ArrayList, or some
kind of built in Java list of tree nodes, we can just have a treenode called next sibling, and
a treenode. called first child.

18:45

So in this notation, if you have a node, it has two links. It has a link to one of its siblings,
and it has a link to its first child. So if it has many children, the node only has a link to one
of those children. But then that child, you can follow its next sibling links to make its way
over to the other siblings. So in effect, this is simulating a singly linked list. So let's see how
that actually works out. So say we have a tree that's meant to be this. Right? So we're not
going to have iterators on this. So we're going to have an A. And what we said was that
each node is now only going to have a first child in the next sibling. Does the root node
ever have any siblings? No. So my horizontal arrows are going to be the next sibling links
and my more vertical arrows will be my first child links. So the next sibling link on a will be
no. Now there could be many children, we don't really care too much about the ordering.
So a will have as its first child link might reference B. Now B, we actually add a couple
more here. B has multiple siblings, right, C and D. So B's next sibling would be C. And what
would C's next sibling be? anyway? It'd be D. Are there any more siblings? Nope. So then
these next sibling could be no. Now B doesn't have any children. So what would be his first
child reference? Good No. See has one child so it's first child, Link would reference that
that child has no children, it's a leaf. So its first child will be no. And it also has no siblings.
So it's next sibling would be no. And then D has children. So we'll have a reference to its
first child, which will have as being F, its next sibling will be G. And then G's next sibling is
no. And both f and g are leaves. So those will have as their first child's nose as well. So this

DS Lecture 8 Page 4 of 20 Transcribed by <https://otter.ai>

is what the tree is theoretically supposed to be. And then if we're going to store our nodes
using this notation, this is what it would actually structurally look like in in sort of in
memory. Any questions on that? So in some sense, you really can think of this as using a?
Yeah, no, I think if it is always going from left to right, somebody asked Is it always go to
the left. Um, so you really can think about this as using a singly linked list of children. The
first child reference of a given node is like the head of the singly linked lists of children.
And then the next sibling links kind of act like the next links as you make your way through
that singly linked lists. Sound good? All right. And you'll notice that even though E, F, and
G are on the same level as each other, they all three of those have a depth of two, they're
not part of the same list, because they're cousins, not siblings. DNF. Okay. And so this is a
rather compact way of representing this. Alright, so that gives us an idea of how we could
represent these things. Let's take a moment and discuss what might a tree be good for?

What are these things useful for where have you seen trees?

23:48

Any suggestions? particle physics? Oh, my brain isn't working well enough for particle physics. Um, hold on. So somebody said particle physics, someone said Central Park. Hahaha. Someone said state machines? Oh. So one of the things about trees, right is that lower levels don't have as their children nodes that are at upper levels, right? There are no cycles in a tree between these nodes. So a state machine actually wouldn't quite work as a tree because you could have a state that sort of backtracks, right. So state machines are a little more generalized than trees. Alright, recursion. recursion is kind of like a stack. networks, I would say are also a little bit more generalized than trees. This could be thought of as a kind of network. But in a network, you might say be able to have a link from E back to a or something like that. A decision tree sounds reasonable files so far. is actually where I was really going with that. So we'll put a pin in that for a minute. A graph traversal. Right. So that requires a lot of definitions to really unpack that, but you're not wrong. And as we get to graphs that will be pretty reasonable families. Yes, a family tree is very much like this. That's part of why we use these. We use these terminologies, history textbooks with Emperor Ayers and monarchs Sure. Um, alright, so there are lots and lots of things listed here. Oh, directories. Okay. So let's go with the file system example. I really like that. All right. And so the file system example, is actually really good. So if you think about files on a computer, in fact, the top level directory, right, and so you know that a file system on a computer contains a system system of directories, also sometimes known as folders and files. And the top level directory on a file system is actually called the root directory. So the terminology right there fits right into things. Right. And so you have inside your directories, you can have zero or more other files or directories. So in that file, slash directory structure, is a directory, an interior node or a leaf? In most cases, it's an interior node, but can it be a leaf? Sure, when can a directory be a leaf? It's empty, right.

DS Lecture 8 Page 5 of 20 Transcribed by <https://otter.ai>

So if we have an empty directory, that's a leaf node, there are no children. If you have a file, it's got to be a leaf because you can't nest anything inside of it. And so we can represent our file structures in that way. So what I'm going to do now is I'm actually going to bring us over to an example from the textbook and work from that for a little bit, rather than handwrite. This. Um, so let me shift the share a little bit.

27:40

Okay, by the way, in the chapter four, here's an example of a tree a little more complicated example than what I was drawing on paper. And if we scroll down a little bit, that's what that example would look like using that first child. Next sibling examples. This is sort of the slightly more complicated version of what I had drawn. Alright, so here is an example of a directory structure. Now I'm going to put a little caveat on this is that I don't, I don't like this example, because he puts the home directories of the users in a directory called slash usr, which is exactly how Unix systems don't organize their files. Last user is actually where Program Files are supposed to go, but whatever. So in a Linux or Unix file system, or you typically think of the root directory is sort of the slash, right? Or in the Windows machine, a backslash and then slash usr would be a subdirectory of the root directory. But for this purposes, we're going to start at slash user. And that's going to be the root of our tree structure of files. Now, the things that have asterisks on them are marked as directories. And the things that don't have asterisks on them are going to be files. So within our root directory user, we have three children. So three this case subdirectories, Mark, Alex, and Bill. All right. And inside of Mark, they have two sub directories, book and course, as well as a file called junk. And then it goes down from there. Alex. I don't know what to say about Alex. Alex seems to just like jump. And then oh, well, I didn't mean you Alex. And then we've got bill bill, who doesn't like to do very much work, because their work directory is empty, but does have a course directory, which has their course. CLP 3212. And of course, there can't be two falls, I assume this is supposed to be fall and spring, which are subdirectories, which would then have files underneath them.

So this is kind of what directory structure looks like. And then you can use a tree to represent them. Now, one of the things we might want to do with a directory structure is we might want to list sort of all the files and directories. Starting at a given root directory, let's get sort of a map of what's in there. And in fact, certainly in the command prompt on Windows, and I think there's such a command in Unix and Linux, there's or at least with bash, as well, there's a command called tree that you can call, which will actually print out a tree like representation of all of the directories and files below that point. And here, I'll show you an example of what an output of that tree might look like. So this job, this thing's job is to go through and print out all of the nodes in that directory structure all of the nodes in that tree. And so when you want to go through and visit every node in a tree, that's what's known as a tree traversal. All right, so when you traverse a tree, you visit

DS Lecture 8 Page 6 of 20 Transcribed by <https://otter.ai>

every node in a tree, what should the best cost be for a tree traversal

31:34

order n , where n is the number of nodes in the tree because you're supposed to visit every node exactly once. Now what it means to visit a node depends on what you're trying to accomplish. In this case, the tree traversal. Visiting a node simply means printing it out. And so let's see how this thing printed it out. First thing it did is it printed out the root directory of it. And then it tabbed in once and printed out its first child, that it tabbed in twice and printed out its first child's first child, then it tabbed in three times and printed out all the things inside that book subdirectory. Then it came back to the original tab level of book and printed out the next child of mark which was course tabbed in printed out everything underneath that. And then finally, once we've exhausted everything under mark, we go back one level on the tab and print out Alex tab in and print out Alex's junk. And then we move on to build what to the number of tabs correspond to in the tree

32:54

corresponds to the depth, right, so every level of the tree is tagged in the same number of things. So each tab corresponds to a step down in the tree. So the root directory has a guess like a new generation root directory has a depth of zero, so there are no tabs in front of it. Everything on the first level of the tree, all of the users children, Mark, Alex and Bill are tagged in once all of the sub directories or, or files inside of Mark are tagged in twice, same for Alex, same for Bill, and so on. Now, you'll notice here, in addition to the fact that the number of tabs in is specified by the depth, since we're sort of printing these things out sequentially, we have to think a little bit about the order in which we're printing them out. And so what it looks like when I talk about traversing a graph, I talked about visiting an item. And so if something is printed out earlier, it means it was visited earlier. So it looks like when we are traversing on a given tree, the sorry, a given node, we print the node itself out first. And then we go through and what we're actually doing is we're recursively printing out each of its children sequentially. So this could actually be thought of as a recursive method to do our traversal. The base case being we're at a leaf in which case we just print that node out. But since in this traversal, we're printing the node itself first, if I invoke my recursive tree traversal on a node that is an interior node, first we print that node itself, then we recursively traverse on each of its children sequentially. Okay. And so this particular kind of traversal Where we print the node out first. And then recurse, through its children, is what I would call a pre order traversal. pre order in the sense, we print the node first before recursing on its children. Now there is another variant of this printout that you can get me scroll down and show it to you. And it looks a bit like this.

DS Lecture 8 Page 7 of 20 Transcribed by <https://otter.ai>

Where it looks like what's happening is it's actually printing out the deepest things in a given subtree. First, and then kind of working its way back to its parent. And then eventually back to mark, then all the stuff printed out is what's underneath Alex, then all the stuff that's printed out as what's underneath bill, and then at the very end, it prints out

the root directory. This is still a recursive traversal. But it's what's known as a post order traversal. And in a post order traversal, what you do is you first invoke the recursive traversal on the children. And then once those are done, then you visit or in this case, print out the route, which is why the route is printed out last. And in fact, the same thing is true for each subdirectory. We print out everything underneath Mark first before mark is printed out. So this ends up being what's known as a post order traversal. Does that make sense? That's sort of, don't worry about it. It's it's not relevant to this. It's that's just, that's actually supposed to be the number of bytes stored in the files. And it's very specific to an example that the textbook is doing. So Celia, hold on one sec. Oh, I got it. What's the other kind of traversal called pre order versus post order? in pre order, you print out the route first, then go through the children recursively, in post order you recurse on the children first, then you print out the root. Good. Okay.

37:20

So now that we've discussed that, we should take a look and see how we might in this case, it's going to be in pseudocode, how we might actually go about implementing such a pre order traversal. And so what we're going to have here is we're going to have a method called list all I'm starting with the public one over here. And these are actually meant to be methods on the tree nodes themselves the way the textbook has written it. So imagine if I have the root of the tree, I would say like root dot list all would go and print out everything from that route on down. Now one of the things we're going to find with our recursive methods that we're going to do on trees, and pretty much everything we're going to do with trees is going to be recursive. The thing we're going to find with this is that we're always going to have this public driver method, private recursive method. And we had that same format, when we were doing the recursive binary search back on homework one. The idea is that the recursion needs to book keep something as it goes through and prints this stuff out. And in fact, the thing it needs to book keep is what depth are we currently at. Because when you invoke list on a given node, it doesn't necessarily know what's going on above that node. It just sort of assumes that's the beginning of the show. But that may not be true. So our actual recursive list all method will take as a parameter, the depth, and the depth will just tell me how many times do I have to tab in before I print that note out. So the public list all gets invoked. And then that in turn, invokes the private list all passing in the initial depth. And so since the assumption is that this gets invoked on the route, the route will have a depth of zero. So now, we're in the private version, which takes in the decks. And so the first thing we do since this is meant to

DS Lecture 8 Page 8 of 20 Transcribed by <https://otter.ai>

be a pre order traversal is we print the node, we print the name of the thing. That's what I mean by visit that node. Now the print name is just this is sort of pseudocode. So we haven't actually written the print name method, but the print name methods job is just to print out the day At that note, we'll print out the name of the file or directory that we're currently at. And it takes as a parameter, the depth, and all it's going to do is just going to tab that many times in, then print it out. Okay? So that's visiting. Now, once we've visited it, then we trigger the recursion, right. And so there's got to be a base case, right. And so if the node is not a directory, then we're done. There's nothing else to be done, we simply return. If the node is a directory, that means it could have children. So then we have a for loop. And this is really just iterating, across the linked list of children. And so that could be as simple as iterating across an ArrayList, or following the next sibling links depending on which representation of the tree we choose to use. And so for each file C, in this directory, which means for each child C, of the current node, we invoke list all on C, but with the parameter of depth plus one, because all of its children should be tagged in one more than what the root of this subtree was tagged in. And that's it. That's the entirety of a preorder traversal on this, and this will print out the thing. If I wanted to convert this to a postorder traversal, where it prints the name of the node last after already recursing on its

children, all I have to do is take this line, this line one and move it to after line for just data ordering, make the recursions happen first, push the print down to the end, it's not inside the loop print just happens at the end. And that's it. I have magically turned my pre order traversal into a post order traversal. Any questions on this? All right.

42:18

Now I mentioned pre orders, reversals and post order traversals. I want to do a quick side note, let me switch my share. There's another kind of traversal. That's a little different. Why is the list all method on the bottom there, if we were only calling the top method, remember, it's a public driver private recursive method, that bottom list all would get called exactly once on the route, and then that would trigger the recursion. So imagine we have the roof. what we'd do is we'd actually call roof dot list all. And that would be the public version. And then that in turn, invokes the recursive private ones. Is that okay? All right. So let's say we have these general purpose trees. So if I were to do a pre order traversal of this, where I want to print the nodes out in pre order, it would print out a, then it would print B. And then it would go through B's children, then E and F, then it would print C and it would go through C's children, then it would print D and then it would print ages children. So that would be a pre order traversal. Is everybody good with that? If I were going to do a post order traversal. Well, you recurse on the children first before printing a value. So first I recurse on B and then I go through B's children. So it would actually hit leaves E and F first. Then B that would finish come back Go to the next child have to go through that things children then it would print g then C then H and I

DS Lecture 8 Page 9 of 20 Transcribed by <https://otter.ai>

44:35

then D and finally a. So far, so good.

44:44

So there's also this concept of a level order traversal. And in a level order traversal, what we're going to do is we're going to print out everything at depth zero first, then everything at depth one, then everything at depth two. So a level order traversal of this would be like a, then B, C, D, then E, F, G. h, i, Does everybody see how that kind of differs from preorder and postorder? Where you're going through level by level rather than recursing? On the children? Okay, this one, you actually can't implement or don't want to implement recursively? Does anybody have any idea how you might implement a level order traversal? So actually, no pre and post order traversals are stacks? Because remember, they're using recursion? And yes, we're saying the alphabet. So when they the pre and post order traversals are using recursion and recursion is by definition a stack, right? So these two are stack algorithms Do we agree? And so Ryan actually got it because this is taking a slightly different approach. This is actually a queue. Right? So you can imagine, okay, and queue the route. And then when you dq the route, and keywords, children, and then print it out. And so every time you dq UNQ, all of that nodes children, so you guaranteed that they're going to get tacked on after all the other nodes at the level. So when you use these are, these are stacks. This is a cube. I actually bring that up. Because on homework three, I'm going to have you do a level order traversal. So I want you to know that this could be a cube. Okay. So for now, though, we're going to focus on pre and post order traversals. Good. Alright. So that's a general tree, a general purpose choice. As we saw, there are more constrained trees, and sort of the most constrained tree is really just the link list. But in sort of an intermediate scenario, there is a more specific type of tree known as a binary tree. All right. And so in a binary tree, each node can have 01 or two children. Right? So in a binary tree, where constraint is 01, or two children for any given node, right. So in that case, if we have a binary tree node, so I'll call it A bt node.

48:40

A binary tree node would have its data or elements or whatever you want to call it. And then it would have to binary tree node children. But because there are only two, we can

actually have more of an ordering associated with them. So this is the left child. This is the right child. So there's this concept of left or right child. So we'd have a binary tree node left and a binary tree node. Right. And that's what our structure for a binary tree would

DS Lecture 8 Page 10 of 20 Transcribed by <https://otter.ai>

look like. Now, because in this case, there is this concept of left and right, we can actually kind of come up with another kind of traversal because the root node can almost be thought of as being in between the left and the right. So if the root node has sort of its left to sub tree that would correspond to everything to its left. And if it has its right sub tree, that would correspond to everything to its right. So a preorder traversal would be print out the root first, then recurse on the left, then recurse on the right, a post order traversal would be recurse on the left recurse on the right, and then visit the node itself. But now we can have another variety that's known as an inorder traversal. And in an inorder traversal, couldn't really easily have an inorder traversal, when we were talking about Gen trees, which could have an arbitrary number of children. Give me one second, or we could have an arbitrary number of children. And so in that case, you would print the left subtree first, then you would print the root node, then you would print the right subtree. So that would be in order. Now Orly makes a good point, D is the left child of B, because I drew it to its left. So B has only one child, its left child and its right child would be no. But that also then brings up the point of f, where f also only has one child, but in this case, F's left child is no. And its right child is G. So just when you have one child in a binary tree, there's a distinction between whether or not that child is a left child, or a right child. Orally. Is that good? Cool. All right. Any questions so far? Okay, so let's, let's ram home, these traversals a little bit. I think they're important. And we're going to do something potentially fun with holding on to the show what's going on? But I thought I heard sound. Was that coming from you? Oh, man, oh, man. What's up?

52:26

I just had a quick question. In the inorder, traversal. You'd count them from who's the most left? like you'd start from the, from the leaf, like from the child to the parent to the root? Or would you just wait, say this again? Because right, you said that in the inorder traversal. You start from the most left to the most right, and you go to the most right? So an

52:48

inorder traversal only makes sense in the context of a binary tree. So first, you go left, then you go root, then you go right.

53:00

But you start from the left most like the chat like if from the left, there's

DS Lecture 8 Page 11 of 20 Transcribed by <https://otter.ai>

53:04

no left most there is the left and that's it. Okay, thank you, you're thinking, you need to think about it a little bit more recursively, we recursively invoke the inorder traversal on the left child, period. That's it. Now, it so happens if there's stuff below that, then it would go to the left of that next, which I think is what you're trying to get at. But that's probably the wrong way to think about. So let me draw something out here. Um, okay, so let's draw the following tree. Okay. So let's start with a pre order. traversal a lot of people are asking me remind me what's free, remind me what's post? I'm going to not answer those directly. We're just going to do them here. Okay. So let's start by doing a pre order traversal on this. So on a pre order traversal we are printing the root first. And then we are recursively invoking pre order traversal on the left subtree and then recursively invoking pre order traversal on the right subtree Okay, so what prints first? Anybody? Da then we recursively do pre order traversal on the left subtree so if I'm doing a pre order traversal on this, what prints first? So pre, we print the root of that first. And then we go through its children. So b first, then we go left, then we go right, so B, D, E, that finishes the left sub tree, then we do a pre order traversal on the right subtree. So pre order means we visit the root first, then

we go left, which is F, then we go right, which is nothing. Good. Now, for the post order traversal in a post order traversal, we recurse left first, then we recurse, right, and then at the very end, we hit the root. So, we start up here, first, we recurse left, so then we're invoking post order traversal on this. So, when I invoke postorder traversal on this, we have to go to its children first. So what's the first thing that prints out here?

56:10

It's the D, then we go to its right child, it's the E.

56:18

Then now that we're done with its children, it's a post order traversal. So we do the beat. Now we're done with the left subtree of the original route. But before we can print out the original routes, we have to do the right subtree. So then we do a post order traversal. On this part, we go through the children first, which is F. There's no right child, so there's nothing to print there. Then we print the root of that sub tree. And now that we finished both the left and the right sub trees of this thing, then we print Finally, the root of the whole tree. Are we good with that? Okay, now, V in order traversal in the inorder traversal, we recurse left first, then we print the root, then we recurse, right? So, if I'm doing an inorder traversal. Here, I then first recursive Lee in order traverse, I'd be in an inorder traversal. We go left than root then right? So if I'm doing an inorder traversal, here, where

DS Lecture 8 Page 12 of 20 Transcribed by <https://otter.ai>

do I go next? I go to D, so the D prints out, comes back up here, but we're printing in order. So the be the root of that subtree comes next. Then we go right? It's the E. So now we've successfully done an inorder traversal of the left subtree. The left subtree is done. We're in order. So now we visit the root here. So the A prints out. Then we do an inorder traversal of the right subtree. And for the right sub tree, okay, fine, we go left first. Then we go to the root. Then we go right, but there's nothing. And so then I'm done. I finished my in order to reverse pre order post order in order. Good. Okay. Now this lends itself to why does in order start with D, you go left first, then root then right. And when I say you go left, it means you completely print using a call to inorder traversal. The left subtree. So when I'm printing out this subtree in order, the D is the first thing that would come out. Good venca. Alright, Jesse for CF, do you decide if f is left or right? So I'm stating when I draw the tree that it's left? So if I'm drawing it to the left, I mean it's left if I'm drawing it to the right. I mean, it's right. Does he does that make sense? So you can have a node with no left child, but with the right child or vice versa. So it's you just specify that's probably the wrong way to think about it. Oh, okay. No, that's the right way to think about it, I guess. So. If he had a child would that be printed before after In the inorder, traversal, depends on which child it is right? If it were a left child, the left child would be printed before the if we're talking in order. If it were a right child, it would be printed after the E because you would just recursively apply the same in order traversal, that you did it every other level to the subtree. rooted at that. Okay, Alex. Cool. All right. So I will want to sketch out a little bit in code what these traversals look like. But before I do that, I actually want to I want to try something kind of interesting. So let's say let's take the preorder traversal of this tree, which was A, B, D, E, C, F, and the inorder traversal of this tree, which was d BEAF. c. Let's take that.

1:01:14

And let's see if we can use these two existing traversals to reconstruct the tree that they came from. All right. So let me ask you a question. If I just gave you this, without knowing what the original tree was, I just said that the pre order traversal. And that's the inorder traversal. Can you immediately tell me what the root of the tree is? What is it? It's the a and why is it the A?

1:01:57

It's the first in the pre order traversal. We know that if I give you a pre order traversal the absolute first thing that gets printed out is the root of all trade. Now imagine instead of

giving you a pre order traversal, I gave you the post order traversal. Where would the Yep, if I had given you a post order traversal, the route would absolutely have been the last.

DS Lecture 8 Page 13 of 20 Transcribed by <https://otter.ai>

Okay, so now comes the fun part, if I have a pre order and an order, and I hate to say this, but I actually do find this fun. If I have the pre order in the an order. And I know then that A is the root, you can actually figure out what nodes are to the left of a and what nodes are to the right of a by locating the route in the inorder traversal. We know that anything in an inorder traversal that gets printed out beforehand is before the route should be to its left. And anything in a pre in an inorder traversal. That's printed out after the route should be to its right. So we know DB n e is the inorder traversal of A's left subtree. And we know that FC is the inorder traversal of A's right subtree. And now we can play the same game recursively located locate the DB and E in the preorder traversal. And now we know this is the preorder traversal of the left sub tree. And this is the inorder traversal of the left sub tree. So apply the same rule, we know that if that's the pre order traversal the root of it's got to be B and then we locate that in its inorder traversal. And we know D has to be to its left. And he has to be to its right. And now we've reconstructed the left subtree. Now let's reconstruct the right subtree. This is the pre order traversal of the right subtree. This is the inorder traversal of the right subtree. Well, if the pre order of traversal of the right subtree is this what's the root of the right subtree? It's C, right? This is the pre order traversal. The root of the sub tree comes first. Then we locate that C in the inorder traversal. And whatever is to the left of it is on its left side F and there's nothing to its right and we've reconstructed the tree. So you can do this if I give you a pre order traversal Oh, because we know in an inorder traversal the Yeah, I know that. That's right. But we also know from the pre order traversal, that the C has to be the root. Right? Because the root always comes first Omer Does that make sense? in pre. So just looking at the pre order person, person, the pre order person, the pre order sequence, we know that C must be the root. And so then when you look at the in order, you know that what's to its left has to be on its left hand side. Right? Because the F shows up to its left, and we know C is the root of it. Good. Alright, how many orders are necessary to construct a unique tree? Well, that's an excellent question. Um, you absolutely can do this with a pre order and an inorder traversal. Because they just gave you an algorithm to do it, you should be able to do this no problem with a post order traversal. And in order traversal, because it's the same algorithm, you just look at the last element for the route rather than the first. Whether you could do this uniquely with a pre and post order. traversal is a question that I'm asking you to actually prove on homework three. So I'm going to I'm absolutely leaving that as an exercise to the to the user. Okay. Any questions on this? Alright, so I think it would be helpful. If we sketched out what a preorder traversal might look like in code. All right. So let's say our pre order traversal is going to, is going to just print it. You know what, I'm actually going to write it out as code first. Um, let's not talk about Markov chains. So our pre order traversal,

1:07:10

DS Lecture 8 Page 14 of 20 Transcribed by <https://otter.ai>

we'll take in the root of the tree that we are attempting to traverse. So that will be a binary tree node. We've called T. Because it's a pre order traversal. Well, we need a base case, right? And so if t is No, there's nothing to print, right? So we just return. So if t is no, we just added the function, we don't do anything. Because it's a pre order traversal though, if t is not, no first thing we do is print the root node out itself. So we'll do print tes data, T's elements.

1:08:07

That could be elements or data, whatever we call it. Okay. And so that could actually just be a system out print line. Then we recurse on the left subtree. So we do pre on T dot left.

1:08:26

And then we recurse on the right subtree. We do t pre on T dot right. And that's the entire code for pre order traversal. If t is null return, then we visit the element itself, then we recurse left, then we recurse. Right? That's the whole code. If I wanted to make this a postorder traversal, I would simply move the print to the end. And if I wanted to make this an inorder traversal, where would I move the print? Just in between the two pre calls? And that's it. A Zoia did that actually maybe help in terms of repeating the process? I felt like writing it out and code actually kinda explains it really well. Excellent. Okay. So are we good with our traversals? So what is this big O of what? Oh of n squared. No, this is actually just big O of n. Because if you think about it, we will only ever visit every node on it exactly. Once. Okay. All right. So I got about five minutes left. So I'm going to give us um, you know what? Uh, yeah, I'm going to give us this is just dangerous because it's just throwing more information at you that you're going to need for the midterm. I'm going to call that a day. All right. So that's it. That's all we're going to do for the midterm. So all of these traversal things, trees up through basic binary trees, and traversals is fair game for the midterm. Alright. And we're going to review up to this point, this is basically section 4.2, through maybe the first quarter of Section 4.2. There were some definitions of binary trees, there's like this, the definition of a full, complete and perfect binary tree that you might see in the text. That's what I'm going to start with. That'll be in the pre recorded lecture for after the midterm. Okay. All right. Alright, folks, let me stop the share. And I guess I'll stick around, answer any questions. But yeah, that's, that's it for up to the midterm. And I will see you guys on Friday for the review. Daniel, what's up? Hi, Professor,

1:11:37

a quick question. So regarding that static keyword for nested classes, it's accomplishing

DS Lecture 8 Page 15 of 20 Transcribed by <https://otter.ai>

something different from what we learned in 2004. Right, it's like from what I understand it's making it such that the inner class cannot reference the outer class, or so.

1:11:59

Sure, that's one way to think about it, the inner class cannot reference the outer class, the truth of the matter is, is that it actually does kind of mean the same thing. Because in AI, an element of a class that is declared as static, is what's known as class wide. There's only one version of it for all for the whole world. So whenever you instantiate that static class, that's fine. But that's the only thing it can be. When it's non static, it means it's tied to a particular instance, of what it's been declared inside of. So when you instantiate a non static nested class, you're actually instantiating, something that is inextricably tied to a particular instance of the outer class. And that's kind of the same thing. As static, when we talk about static for a method and a variable, it's just the consequences of it for a nested class is that if it's static, it doesn't have that tie to the instance variables of the outer class.

1:13:20

But I just want to reconcile one thing, so to say, so let's see the link to us. We have obviously more than one. Right?

1:13:27

But wait a minute, please don't interrupt. Hold on. I don't know. I thought wise was still talking. I'm sorry. I'm keep going.

1:13:37

So you know, linked to us, for example, we have multiple nodes, right? But we still make the node class, the inner node class static. So what how does it allow us to do that?

1:13:51

Because the inner node classes have no tie to the outer class, from methods inside of those nodes, you have no way of referencing the outer class, the outer class can touch whatever it wants, inside of the static. The static nested class, but those nodes, there's no

there are no methods inside the nodes. So they have no way of connecting, there's no chance of them connecting back to the outer class.

1:14:25

Right. Okay. All right. I guess that makes sense. Thank you.

1:14:29

Okay. All right. Your next. Me? Sorry. I was just wondering, like, kind of what what what, what our motivation is for, like, using trees, like I understand, like, why ArrayList and stuff makes sense and like, application wise, but like, is there use cases besides?

1:14:48

That's the entire next unit. I just don't have until one of the modifications is that one of the motivations is representing directories which are, by their nature tree like structures. And so that's actually a very good way of doing it. The next example we're going to do is using them to represent mathematical expressions where no leaf nodes would correspond to operands. And interior nodes would correspond to binary operators. And then from there, we're going to impose some constraints on sort of what can go to the left and what can go to the right. And so we'll compose some constraints such that everything on the left has got to be smaller than the root. And everything on the right has got to be larger than the roots. And so we end up with something known as a binary search tree. Which, if you can then constrain the height of the tree and have the tree be of height log event, and we're going to talk about that a lot in the upcoming lectures. Then our search operations become log event, and it becomes a lot like binary search. So there are lots of ways we can use this sound good? Yeah, yeah. Thank you.

1:16:05

Okay, you're next. Hi, Professor, I have a quick question. Maybe I'm overthinking this, but I was trying to practice like the, like the big O analysis. So like with the last one, I thought it would be n^2 because I thought it'd be similar to Fibonacci with two recursive calls. Is that just like under the hood? It's different? or something? Oh, it is

1:16:25

it? Is it? Okay. So think about what you're counting. Right? So when we said Fibonacci

DS Lecture 8 Page 17 of 20 Transcribed by <https://otter.ai>

ended up being two to the N , not n^2 , but two to the n . Oh, right. You were actually counting the total number of calls in that that tree of function calls was two to the N . But n was actually the height of that tree, not the total number of calls, the total number of calls was two to the N , N was the height. And so in fact, when doing a tree traversal, say on a binary tree, that's perfect, like, as every level is ever completely filled in. If n is the number of nodes, that's different from the height, and what we'll discover is that the relationship between the height and the number of nodes is the same. So if you have n nodes, which is what we had in this case, then its height would be log base two of N . So if its height is h , then the number of nodes would be on the order of two to the H , exactly two to the H plus one minus one. So the relationship between the number of nodes and then the height goes by the log base two. And it's so it is kind of like that. But since the number of nodes is already the part that was sort of to the age, it's just on the order of the number of nodes. Does that make sense? So we actually, were actually trying to figure out how many nodes there were given only the height, it was going the other way,

1:18:17

okay. And we had a height of n and Fibonacci Is that right? Here? We have a height of log something. Basically, log

1:18:25

then is our height being the amount of nodes Yes. So some you were wrong in that connection is just that what we were talking about in terms of our annual was a little

different.

1:18:38

Okay, cool. Thank you so much. I appreciate all right. Bow. I think you were next What's up? Yes, this Weiss textbook. I'll be it. It's very interesting. Um, should we rely more on course the in the original non international or Weiss for the midterm? Sorry, say that again? Should we rely more on course theme, the original non international version? or white? What's core steam? core steam? Yeah, there's a good book. The one that I got from Graham after I don't even know what that is. So, okay. So as far as so on, no, no, of course the news anymore. Of course the the domestic version edition five or six? horsemen. horsemen. Yeah, that's what I'm, oh,

DS Lecture 8 Page 18 of 20 Transcribed by <https://otter.ai>

1:19:46

that's the Java textbook. That's a that's the one that just talks about. Just how to use Java specifically. That's

1:19:55

sort of the feather paper. I love them equally. But But what what the midterm Weiss

1:20:04

textbook is what's important for the midterm? I knew it was gonna be Weiss. Yeah. Because the horstmann textbook does not really cover this material. The horseman textbook is sort of a, what's the word? I'm looking for a it's sort of like a reference text for like the Java's specific components. Whereas the exam is more based on the theory, which is all coming out of Weiss. As one w formation, I got to get into Weiss a little bit more than Yeah, very much. Yep. All right. Gotta go. Go take it easy. All right. Other questions? Hi, professor.

1:20:54

So just thinking about the recursion calls here and the rules that we talked about for recursion, we talked about making progress towards a base case. Like, intuitively, this almost seems like we're sort of starting at a base case, like the root and then working out from that, what is really the base case we're progressing towards.

1:21:18

So you are progressing. You're so the base case is you're progressing towards No. Right? you're progressing towards an empty subtree. So when the when you have an empty subtree, you stop. So you're recursing on the children and you stop when you reach a no. So when something's children is no, you stop and return back.

1:21:44

Okay, makes sense. So it says, essentially, we stopped at a leaf, right? Whenever we find it, and then go back. Yeah. Thanks. So

DS Lecture 8 Page 19 of 20 Transcribed by <https://otter.ai>

1:21:54

Aiden, when you said can you talk more about how the height of the tree is the log of age? Um, sir, that. So that's that's only if the tree is basically if it's perfect if it's completely filled in. And so that requires a few definitions that we didn't get to. And that's actually going to be a major thrust of the lectures next week. So I don't really think we need to beat on that right now. But it's coming very soon. Does that make sense? Yep. Thank you, professor. Awesome. All right, folks. I think I'm going to head out. I will see you at the review.

1:22:36

Thanks.

DS Lecture 8 Page 20 of 20 Transcribed by <https://otter.ai>

DS Lecture 9

Wed, 8/11 1:21PM 1:01:45

SUMMARY KEYWORDS

exam, questions, familiar, problem, arraylist, recursion, code, tree, binary trees, answers, generics, written, homework, array, part, class, midterm, stack, quizzes, textbook

02:26

Ma Ma Ma Okay, we will get started in just one moment. Let me just confirm that we can all see my web browser and specifically the midterm topics file. Yes. Yes. Cool. All right. So I need to get one more thing open and ready to go. And you all are still just seeing the midterm topics, right? I have to ask this just because I am about to open up. I'm about to open up a copy of the midterm. And I don't want you to see it. So almost there.

07:30

Okay.

07:32

All right. So, um, we are going to do a quick run through of the topics that we had. And I'm going to then sort of bring up a few things that I might ask you to do on the exam. Okay. And then I'll give you the opportunity to ask some questions. And I think that'll be it. For this, I've set aside an hour, I really don't think we need anywhere near that much time for this. But we've got, we've got some time. So I guess before that, I should just remind us of what the remind us of what the exam is going to be formatted as I will have split it into two parts. The first part will be a multiple choice slash fill in the blank problem exam that will be done through courseworks quizzes, and you'll have 40 minutes to do it. So the idea is that whenever you're ready to start it, you go to courseworks, click on the quizzes tab. And then when you're ready to start, it'll start showing you the instructions and the questions. And you'll be able to answer by choosing from multiple choice answers, drop down menu answers and filling in numeric answers. That's it. And so that's sort of there's

DS Lecture 9 Page 1 of 15 Transcribed by <https://otter.ai>

no real partial credit on any of that it's just right or wrong. We'll talk a little bit more about what kinds of problems will show up there in that section, but that's 40 minutes. And then the other half will be some written oriented problems. One will probably involve some diagrams, and one will likely be a coding problem. And there might be a second sort of shorter. What's the word I'm looking for a shorter code fragment problem. Now for that problem, that part of the exam they intend for that to also be 40 minutes. So the total of the two is 120 minutes, our typical lecture time. Actually, I should turn my video on tonight, our typical lecture time is somebody privately messaged me that my camera was off, just as I said that nicely done. Um, so that will be done in grade scope. So there will be an assignment in grade scope, I guess will be called midterm part two. And that one will be timed in grade scope. So when you open grade scope, it'll give you a PDF through grade scope. And the reason why I do it that way is so that I can control when the PDF is released to you. And then you leave that window open because it's got the timer going. And then you can type your answers up or handwrite your answers on paper and take a picture of it. The bottom line is you need to consolidate your answers into a single PDF file, and then upload it into grade scope and label the problems in much the same way that you did your written homeworks. Okay has to be a single PDF file though. Now I recognize that grade scope can be a little touchy. And it will require a little bit of time to get it uploaded. So I will actually give you instead of 40 minutes, I will give you 15 minutes, but those last 10 minutes are fully intended to be for consolidating the stuff and uploading it, though I'm gonna only leave it to you to budget your time. But if you email me, like two minutes after it's over saying, Oh, I missed here it is I'm like, Well, what about those 10 minutes I gave you is going to be my response. Okay. So please make sure that you budget your time and watch the timer. Okay. Um, you can take the two parts at the same

time or at different times, I don't care, I would actually suggest doing the multiple choice part first, but it's really up to you.

12:14

You have to do the exam, yet both parts have to be finished before 11:59pm New York time on Tuesday. And so the two together are, I guess an hour and 30 minutes, it's an hour and a half, including that 10 minutes. And you'll probably need a little bit of time in between. So the bare minimum amount of time that I think you could safely set aside if you want to do it in one setting is probably two hours just in case. So whatever you do, make sure you at least start the exam by 10pm, New York time on Tuesday, otherwise, you're not going to have enough time to finish it. So everybody clear on that. Alright, other than that, it's up to you, when you do it. piasa will go private, you'll be able to see everything that's there. But posts will only be private, they'll be for logistics questions, primarily, my ta is will monitor things as much as they can, I will monitor things as much as I can. We've had this conversation before, I'm not awake 24 hours a day, I don't know

DS Lecture 9 Page 2 of 15 Transcribed by <https://otter.ai>

when you're going to take the exam. So I can't guarantee I'm going to be able to respond to things instantaneously. If you're really worried about my being around, when you're taking the exam, the main thing I can tell you is that I'm a night owl and I usually sleep in the mornings. So it's very typical for me to be going to bed at around 3am or 4am. And waking up at like 11am. So if you're going to break up bright and early at 8am in New York, time to take the exam, there's a good chance that we'll be right in the middle of my nighttime sleeping, and I will not see a single message you send me until I wake up later in the day. I'm just telling you this. So he can schedule around that if you want. I don't think you should need to be asking too many questions during the exam. And we're going to be probably not all that forthcoming. It's mostly if something breaks. We'll try to fix it. Okay. Sounds good. It is closed book. It is closed notes, no calculators, no computers beyond the computer, you're you know, doing the exam on but you're not really supposed to use anything other than the web browser or the text editor or whatever you're doing the exam and you're not supposed to be compiling the code or anything like that. You're not allowed to look up anything. For your own sake and my sake, I'm not going to use proctorio just because that's kind of a security nightmare in all sorts of ways, which means this is an honor code. thing, so I really need to trust you guys. So please take that seriously. Okay? You know, if it becomes a point where I can't trust people on that, then we're going to have to resort back to something like proctorio, which is really, I just don't want to go there for a whole lot of reasons. Okay, so that's how it's gonna work. Um, somebody said, I do my written homework on my iPad. And I use that to write for the test, too. Oh, yeah, for sure. You can use a, you can use whatever device you want to generate the PDF, as long as you're not getting sort of outside help from the device. So that's fine. One thing I will note though, there have been some reported issues with the safari web browser, and the courseworks quizzes, particularly images on the courseworks quizzes. So I would recommend not using Safari for the courseworks quizzes portion of the exam. Which means I would recommend not using an iPad for the courseworks quizzes version, because all web browsers on an iPad are using the same rendering engine. But for the grade scope part, I don't care. And if you want to write up your answers on the iPad, that's fine. Sound good? All right. Any questions on logistics? Let's take a pause and do that. We're good? Okay, excellent. So that's how the exam is gonna, oh, how many pages of work about on the written part? I don't really know. It'll probably be sort of three written questions.

16:41

And so I would guess I would say I want each one at least on their own page. One of them might require two pages. But that's about it. Um, PDF scanners can be finicky. So the other option is to simply type your answers in. In a you can type it in Word, just save it as a PDF.

That's fine. There's one problem that could involve the diagram, but you can you can kind

DS Lecture 9 Page 3 of 15 Transcribed by <https://otter.ai>

of do that in text. And that's fine. Okay. Good. All right. Um, I figured there'd be a way to do the quiz on the iPad courseworks app. Mark, I gotta tell you, I don't know if they fix this bug. But when I gave an exam last May. Next year, it was June, to about 1000 high school students via courseworks quizzes, and I embedded a lot of images in that. Um, I think everyone using an iPad, it just didn't work. Even if the courseworks app seeing the images. Yeah, I was, it was Oh, boy. Was that fun? Um, but. So I don't really foresee there being much in the way of imagery on this. Um, I think there may be one, but I'm just giving you that warning, just because I've had that problem in the past. Okay. So fair warning on that. Okay, cool. So in terms of readings, oh, sorry, one more code should also be written straight into the PDF. Yes, it's probably easiest to just type it and then save as PDF. That's totally fine. So terms of reading, we did chapter one, although chapter one was kind of a messy chapter. Focus on I could ask you a little bit about an induction proof since we did an example of that, but I was thinking I might save that for the final. But you should be familiar with the proofs, you should be familiar with the basic math that sort of went into helping us with our big O analysis. And you should be familiar with, you know, generics to the extent that we use them in class. Um, so chapter one is a little more on the skim side of things. Specifically, well, we'll talk more about the contents of the document in a second. But chapter two, you are pretty much responsible for in its entirety. That was the big O analysis chapter. Chapter Three, you are, again, pretty much responsible for in its entirety, that was lists, Link lists, array lists, stacks, queues, that sort of stuff. And then chapter four, you're responsible for Section 4.1. And sort of the first part of Section 4.2. Remember, we did general trees, we did sort of the recursive tree traversals on a generalized tree, and then we did binary trees. And we were sort of doing traversals on binary trees. And we had kind of sketched out what those would look like. Does everybody remember that? That was the sort of the material from last class. There were some examples in Section 4.2. That dude does a thing called expression trees, which we haven't really done yet. That will be for next week. So you don't have to read through the expression trees part. But certainly, those in order, preorder and postorder traversals of binary trees are absolutely fair game on the exam. So that's the material. Now, outside of the textbook material, I think there were two things that I may have brought up that weren't directly in the textbook, there was the Tower of Hanoi recursion, the code for witches in the you know, there are a couple of lectures that I posted code for, because they didn't come from the book. So you should have some familiarity with that. And then

20:51

hold on one second. And then I guess when I was talking about queues, I may have briefly mentioned like, the josefus problem was sort of that circular way, you could use a queue like to go around in a circle, but that's not super relevant. That was just an example of

DS Lecture 9 Page 4 of 15 Transcribed by <https://otter.ai>

how one might use a queue. And I think other than that, everything else was in the textbook. Okay. Somebody asked privately, if internet is interrupted during the test, can I change and reopen the test and another device to continue? My understanding is that it does sort of save your answers as you select them and type them in. So if you close your browser down and reopen it, it should restore them where they were. Which means it should be fine. If you open them in another device. I'm not getting super nitpicky about making sure you keep your browser open all the time. So that should be fine. word of warning, and there isn't really much I can do about this. The timer starts when you start the exam. And if you leave the exam page, for whatever reason, that does not stop the timer. Time just keeps marching on. Okay, so the person who direct messaged me, does that answer your question? Cool. Okay, so is everybody good with the readings? Alright, um, okay, so chapter one, basic proofs by induction proofs by counter example, those are

things that could potentially come up in some form. Induction was the thing we spent the most time the rules for recursion, right. And so recursion was discussed in there, we've been talking about recursion a lot in the context of trees, and there will be, I don't know, if I'm going to actually have you implement some kind of a recursion on the trees. Although there was definitely, if you remember that list all code that we went over, there was definitely some examples of recursion. And we also sketched out what the preorder post order and in order traversals would look like in code as well. So you've definitely should be familiar with that, and be prepared to deal with it. Um, I mentioned here, that Tower of Hanoi was an example we did you should review that. Alright, generics, you should be familiar with generics to the extent that we use them. Will I be asking you to generate sort of the generic headers for code from scratch on the exam? No, will I use them without really telling you what they are? Yes, because you've been seeing them repeatedly. So you should be able to deal with that. You should be familiar with nested classes and the inner classes. This wasn't really chapter one, but this was sort of Java material that we did, they came up in the context of the nodes, for our linked lists, and will eventually also be used for our nodes for our trees. But also, they came up the inner classes in particular, that is the non static nested classes came up in the context of iterators and iterators, will be reflected in some form on the exam. Okay. In section in chapter two, basically, you're responsible for the whole thing, really, it's up through 2.4 point two that's important. Beyond that, or examples that I didn't really do in class. They don't necessarily hurt as extra practice, but it's not strictly necessary. You should be familiar with the definitions of big O, big theta and big omega, okay, um, you know, upper bound, lower bound, tight bound. You really need to be familiar with big O. Most of the stuff will be done in terms of big O, but I could ask a short question about big theta or big omega. You should also be familiar with worst case, best case, average case, and be able to analyze things that we've seen in those cases.

DS Lecture 9 Page 5 of 15 Transcribed by <https://otter.ai>

25:05

And as also as well, you should be able to analyze things that are novel. And you should be familiar with the relative speeds of those runtimes. Okay. And so let me tell you a lot about how I'm going to ask these questions. We saw a bunch of algorithms, we knew the big O costs for a lot of these algorithms. And we saw how they varied a bit in best case and worst case scenarios, there will be quite a number of multiple choice questions, specifying algorithms and methods on those algorithms, under certain circumstances where you will have to choose what the big O cost is, for them, there will be at least a half a dozen, multiple choice questions like that, in addition, I will provide you with at least a couple of, again, multiple choice where I'll give you a bunch of different snippets of code, very similar to the kinds of bits of code you saw in homework one, excuse me, where you'll be asked to analyze those pieces of code and give a big O cost, possibly in the best case, possibly, in the worst case, somewhat less likely in the average case. So that's going to be a non trivial chunk of the multiple choice questions that you'll see on that part one of the exam. Okay. So by the way, let's see, the theme here is I'm trying to tell you what a non trivial number of the questions on the exam are. All right. And so there, there's a bunch of them right there. in the chat, somebody said, should we be familiar with different exceptions? For examples, we need to throw an exception? Do you need to know the exact syntax? I mean, you need to know what the syntax is for throwing an exception. In general, like you need to know, the, you know, the throw, and throws keywords, those are things that I assumed that you knew how to do. Do you have to remember the exact exception classes? Like, you know, there was like, illegal state exception and things like that? No, but I mean, it couldn't hurt to remember array index out of bounds exception. And you know, things like Stack Overflow and stack underflow might come up. Does that answer your question on?

27:30

Cool. All right.

27:34

Chapter Three, this chapter, you're responsible for the entire thing, you should know what the list abstract data type is, and the typical operations you would see in them. You should be familiar with various ways on how we might implement these lists, you should be familiar with array lists, you should be familiar with ArrayList implementation, what we do when the ArrayList capacity fills up, you should be familiar with the runtime for these inserts, removes and getting and setting elements in the array. And you frankly, should be

DS Lecture 9 Page 6 of 15 Transcribed by <https://otter.ai>

able to implement those methods in Java, if I asked you to on paper, or sorry, in code. All right, linkless, you should be familiar with singly linked lists versus doubly linked lists, you should be familiar with the various runtimes associated with operations on those, you should be familiar with how we use Sentinel nodes on doubly linked lists. And you should be familiar with iterators. And how they are implemented. I am going to give you at least one coding problem that will be directly related to link lists implementations. So you're going to have to code something associated with that, what? Well, you'll find out on Monday or Tuesday, but it's going to be something alright. And that's going to be a big chunk of the second part of the exam. Okay. Implementation Yes. Means writing it. Right. Will I ask you to write an entire class? No, I'll probably have you fill in a couple of methods. Okay. So everybody cool, clear on that. Okay, you should be familiar with the stack, abstract data type and its various operations, and the various ways we can implement it, how we implement it, and then array, how we implement it with a list and so on. And you should be familiar with a variety of applications. Symbol balancing, that's something you should have started working on for the homework two programming that's due next week. Reversing lists is really what stacks are good at, which is related to palindromes And importantly, we used it as a way of evaluating postfix expressions, right. And so you should be familiar with all of that. You should be familiar with the queue abstract data type, its operations and implementations. And remember for stacks and queues, the goal here is to get it so that when we do our operations on them, we want them to be operating in order one time. So you should make sure you're able to do that. This is a typo. It's actually you're not responsible. Through 4.3. It's really through 4.2, I believe. But the topics are correct. 4.6 was a section on tree traversals, which might not be a bad idea to read as well. You should be familiar with generalized trees, you should be familiar with that first child, next sibling representation, I could ask you to try to write a little bit of recursion on the first child. Next sibling, I'm not exactly sure how I'm going to ask that question. But it's, there's going to be something related to that on the the sort of the second grade scope portion of the exam. And so that's related to traversing generalized trees, you should remember all the terminology for trees, things like Parent Child leaf versus interior node, I will use those terms extensively. And I want to find them on the exam. So you need to know that they are, you should remember height and depth, be able to identify the height and depth of various things. And you should be familiar with binary trees and their representation that is, in a binary tree node, you have your data, and you have a left and a right link. And you should be familiar with those tree traversals pre ordered order in order and post order traversals of those binary trees. And you should, if you didn't see it yet, you should go back and watch Wednesday's lecture and go through some of those examples I was doing towards the end of the lecture on tree traversals. Because there definitely going to be multiple choice questions at the very least related to that. And I suspect that the bit of code that I sketched out there will be helpful for the piece of code that I may have you deal with on the generalized trees as well. Okay.

DS Lecture 9 Page 7 of 15 Transcribed by <https://otter.ai>

32:31

That is, I think, relatively exhaustive. Before you start asking me questions. I have a question for you. As you saw me doing this, I need you to tell me if there was something you expected to see that you didn't. So that's not to remind me of things to put on the exam, the exam already is well, is mostly I still have to sort of format it. I just want to make sure I didn't forget to tell you something. So what did you expect to see that I didn't bring up? Anything.

33:14

I kind of expected to see implementing one data structure with two others in some way, like, a lot of the homework too, has been sort of centered around thinking about how to implement one data structure that we've learned something about with a stack or as to sex as a cue. So those are definitely ways that I could formulate my questions. And so understanding how the data structures work, and then being able to use those data structures in other contexts is definitely important. I'm going to, for the most part, try to stick to contexts that we've already seen in some way. That doesn't mean that it won't show up. So the really the key example to that is like, we've talked about how to say implement a queue using a link list. And so that kind of relationship, and possibly in a slight twist of that relationship, that maybe isn't exactly the way we did it before. That's very likely to be how I framed these things. But I think that was kind of implied in the sense of there won't be sort of really novel data structures, but I could definitely ask you to sort of build something interesting out of the ones you've learned, just like I was doing on the homeworks. So that's definitely possible. interfaces. Yes. In terms of Java, you should definitely be familiar with interfaces, mostly in the context of comparable so if I asked you something on say binary search, which I guess I should have mentioned in the context of recursion, you should definitely be able to deal with comparable if they show up. Does that make sense? A million?

35:15

Do you expect us to? I feel like we should know this, but like, like, understand all of like the ins and outs of the collections? classes?

35:25

Oh, that's actually a good point. Um, you should definitely be familiar with the distinction between a collection and a list. Like, what can't you do with a collection that you can do with a list? Right, like, you're not really doing things by index and the collection? The main part, though, where that's going to come up? Is that the iterators that we were talking about, were really the iterators that were available from collection. And so it's mostly going to manifest itself in terms of dealing with an iterator. Okay. Got it. Thank you. Cool. Anything else you didn't hear? Okay, so I've told you what quite a number of the questions are going to be? Um, we've gone over the format. I've told you what topics are fair game? Do you have anything you'd like me to clarify any other questions? I'm going to stop the share at this point. We're just sort of in sort of freeform question answer now. Because I feel like I've, I've given you a pretty good sense of what this exam is. Aside from the textbook, how would you recommend studying, I would go back to the written problems and practice them. I would definitely sort of make an exhaust. I do not give out practice tests or practice problems. But you can you can sort of look in the textbook at other problems. Actually, Sally, that study guide, I was viewing from courseworks, it's actually there. No, no, no, I hadn't announced it, I'm just FYI, I was actually viewing it on the way that you can view it too, you can go ahead and double check and make sure I'm right. Um, so I also wanted to say it's probably worthwhile to sort of go back through your notes, enumerate the various algorithms we did, and then jot down what the big O costs were invest in worst cases for them, and just try to absorb that. And, you know, maybe memorizing, it doesn't hurt, I feel like you should be able to figure it out from scratch based on what we discussed, but having that kind of available to you, right then in there.

And you're, as you're thinking about things is going to be helpful. Um, and so, yeah, you should definitely do that. Um, how many questions can we expect for each part? Sorry, hard for me to say probably be at least a dozen multiple choice questions, since those shouldn't take very much time to answer on the written part, I would figure it max three, with one sort of slightly longer programming question, and maybe one shorter one. And maybe one thing that involves a diagram? Will there be any questions on searches? linear and binary search? It's certainly a possibility. Okay. How should we expect to see questions related to the proofs on the exam, I haven't really decided yet, because the real proof that I very much want you to do is going to be related in a lot of ways to some of the stuff we'll be doing next week. So I may save the full on induction proofs for the final. So I wouldn't worry about it too much. Just remember the basic structure that you show that the base case is true. Assume your inductive hypothesis, and then prove that assuming the hypothesis, it also holds true for one step further on. So remembering the process, I think would be most useful in terms of anything I might ask. Tiffany, go ahead.

39:30

Yeah, I was just gonna ask, you briefly went over like the Xbox was actually part of the homework, the written homework, the like how to allocate memory when you're doing like using a stack. So is that something that we should like kind of go over will be asked to

DS Lecture 9 Page 9 of 15 Transcribed by <https://otter.ai>

review or know the efficiency of the memory?

39:51

Or the so we're talking more Oh, this is sort of like questions for B, I guess you were referring to on the So bego in terms of space usage is usually not a major thrust. But I wouldn't spend too much time on it. Beyond just sort of being aware of that, for the purposes of that that homework problem is probably good enough studying, as far as this is concerned, but do remember that I could ask potentially, what the space costs are for doing something. And so when we talk about the space costs, we're mean amount of extra space beyond the space that we're already using to store the data. So an algorithm that involves constant extra space is where like, say you have a big array of like, n size, if you have a few variables, but it's always the same number of variables. And they're not themselves expandable data structures, but they're just like individual values. If you have a handful of them that's fixed, and doesn't change how many of them there are, regardless of how long the array is, that's going to be constant time. But if you need to make a copy of the array, or the list or whatever, that's order n extra space, because the amount of extra space you're using is proportional to the original input. Now, we've not seen anything like this, and it's not going to come up on the exam. But you could also imagine a scenario where, say you have an array of length n , and then you need to make n copies of that array. That would be an n squared, extra space. Okay, so it doesn't hurt to keep that in mind. Really, in the context of that question for B, on the homework, which you'll need to have finished prior to the exam anyway, is probably the most that it would need to be. Okay. Well, thank you. So somebody asked the window of time. Yeah. So um, it should automatically open at midnight on Monday. Um, you may have to draw a diagram of a tree of some sort on one of the written problems, I haven't decided yet. But that's definitely a possibility. My feeling is that you can kind of do that just by spacing over and typing, and then going to the next line and spacing over and typing. So I don't think that would necessarily require you to like handwrite part of it and type part of it. Gabriel, does that answer your question? Yes. Cool. So I'm going to have the exam, try to open it midnight, we also are going to try to have the solutions to homework to become available or how much you've written become available just at that same moment, because that's one of the late deadline closes. So you might want to take a second and not start the exam instantly, after it becomes available. But yeah, that's the plan. Other questions? Right, so related to the the question I was answering in the chat, um, those are actually

important definitions. Remember, a node in a tree is an interior node, if it has one or more children, a node in the tree is a leaf, if it has zero children, and then related to that, remember, the root of the tree could be an interior node, or it could be a leaf, because a single node just by itself is a valid tree. Okay. All right. You guys aren't asking a whole lot of questions. So that's encouraging.

DS Lecture 9 Page 10 of 15 Transcribed by <https://otter.ai>

44:12

We're good.

44:15

All right. Well, I'll hang around for a little while longer in case anything comes up. JOHN, that code absolutely is available through courseworks. I just haven't been posting it. Because it comes from the textbook, front page of coursework, scroll down to the link for the code associated with the with the textbook, and you can download all of it and that's exactly what I've been going through.

44:45

And you might consider downloading that for subsequent lectures because then you can have an open wolum talking. Okay. Anything else? Okay, well, um, I feel like I got another 20 minutes Here I'm adding stuff to say people are being not overly talkative probably because they're exhausted and I get it. So I will bug you with music in the chat and just sit here listening to that music until somebody asks the question. That's my plan. Somewhat unrelated. Can we use ArrayList? Some problem for in the homework written?

45:41

So for part A, of homework for written have question for written? I guess you could, but the input is the input. And Part B, of homework for written says that you're not allowed to use any extra space. So if you copy the input in over to a new ArrayList, you're already violating that by using order and extra space dominate. Does that make sense? Or am I misinterpreting your question? Yeah, that's fine. There is no, there's no constraints on what you can use to make that work, you can totally use a link list, you can totally use an ArrayList, you can totally use a stack, whatever, whatever makes you happy. There are a lot of ways to do that. Um, hold on a second. It would be better to ask questions, not as direct messages to me, but publicly. But let's see. Do you support what do you support canonizing call in linked list sticks.

47:09

Help you? You said in a lecture the other day, and someone made a joke about linkless

DS Lecture 9 Page 11 of 15 Transcribed by <https://otter.ai>

being basically sticks because they're just like, Oh, it's

47:19

just like a branch on a tree. Okay, sorry, I was so confused. But now that actually makes a lot of sense. And No, I don't. Sorry. I thought it was funny, but no. Um, okay. Could we go over the syntax for generics again. Um, alright, so let me say the following. Uh, you recall, there were two ways of using generics in. In our class, there was the method generics, which is where we define the type variable for the specific method. And that's when we put the name of the type variable in the angle brackets right in the beginning of the signature, just before the return type of it. Now, that's what you do, when you're invoking those methods, sort of independently of the class, they're part of you usually do that on static methods, like we were doing in the early part of the class. Now, they weren't static on homework one, just because of the way I set up the tester. But you usually do that just on static methods. The other variety was when you excuse me, when you have a generic type associated with the class itself, so you have it right after the class name, so that when the class is instantiated, you specify the type. And then that type variable gets replaced with that type throughout. And so what you're going to see on the exam is really

going to match exactly what you see on the class wide types, variables, such that you saw on the mailing list and the my ArrayList. And so that's really what you should be looking at. And that's kind of the simplest representation, um, by Professor Sorry to bother you, but do you mind showing how you would do a plus b times c plot What? Oh, I'm, I'm very specific. Not going to do that now. Because that's actually from the part of the chapter four that we're going to do next week and isn't reflected on the midterm. So that that actually is very specifically not something we need to do on the midterm. Does that make sense? Okay, Yeah, I just don't want to confuse things. I want a little more context to that before we have that conversation. There'll be a pre recorded lecture in lieu of lecture on Monday that will cover some of that. And then we'll do some of that on Wednesday before spring break. Um, cool. Uh, the homework to written we can't use iterators. And we can't use any link list methods like get index are not related to the midterm. Oh, good. Okay, I got that. All right. Other questions?

50:37

Well, we still got about 15 minutes left, will we be able to go back to old questions? Okay. So in fact, I'm on the courseworks, multiple choice, you will be presented with all of the questions at once. So you can move around through them as you see fit. You don't just answer one at a time and submit them. Miranda does that? Does that work for you? Yeah, so yeah, you're just gonna get all of them at once. Can you please explain the end and

DS Lecture 9 Page 12 of 15 Transcribed by <https://otter.ai>

start marker in a doubly linked list? Right. So the begin marker and the end marker in a doubly linked list are Sentinel nodes, they're not part of the list. Right? They are sort of meant to be entry points to the list. So as you know, when we have a doubly linked list, we want to have a reference to the beginning of the list and a reference to the end of the list. But that actually gets a little tricky. If say, the list is empty. So what we do is we create an empty node that acts as the begin marker, and an empty node that acts as the end marker, and then just point them at each other. And that's an empty list. So items zero of the list, like if you actually want an index zero of the list, that would be begin marker dot next. Alright, and item, list length minus one would be n marker dot previous. So they're just just outside of the guts of the list pointing in. Emily does that kind of give you a sense of what their purpose was. So if you're going to insert an item at the front of the list, you would take the begin marker, and then splice that item in after it. Good. Cool. So, um, I'm forgetting the, what did the problem say the data was originally in the singly linked list, didn't even say

53:05

just says write an iterative algorithm in Java, like code printing a singly linked list in reverse. So

53:18

you can kind of gloss over it. This is pseudocode. If you actually create an ArrayList, without specifying the concrete type, in place of the type variable, it will default to making it an object, which you can actually put anything into, you just have to cast it back. When you take it out. And that I feel like it's something you can safely gloss over in the pseudo Kodi aspect of this. Right. And Alternatively, if you assume the singly linked list was just made up events, then you would create an ArrayList of integer and use the wrapper class for okay. So that that that that can be gotten around for the purposes of this.

54:02

Other questions?

54:13

Okay. Do we necessarily have to use ArrayList for free or is just an array? Okay. Which one was problem three? We were talking about an ArrayList. In terms of problem for a I

DS Lecture 9 Page 13 of 15 Transcribed by <https://otter.ai>

think, was problem three, the the two stacks one array? Yeah, you actually very

specifically shouldn't use an array, not an ArrayList. It's supposed to be fixed length.
Good.

54:44

Right, we were talking about a different problem. That problem for the two stacks in one array, we absolutely mean an array, not an ArrayList. If I did I misread the earlier question. No, I don't think so. I think yeah, I think we're good. Okay. Problem three, does it matter what the array length Flash content type is, um, you should initialize the array to some fixed length. I don't really care what, um, since it's Java like pseudo code, I don't really care about the type for simplicity's sake, you can make it an int. Um, I think some of my ta said, Yeah, make it any type, and make it generic. And you could do that. But I think I said I would be fine if you just made it. So you can simplify that and just like, call it 10 size and call it an int. Dominic, is that okay? Cool. So Dimitri asks, What is the difference with this syntax, again, angle brackets, T and angle brackets, any type. So the thing that you put inside the angle brackets is what's known as a type variable. It's a placeholder for the actual type. And the term type variable emphasis on variable. You can name it whatever you want. So calling it T or calling it any type is just a matter of personal preference. Does that make sense to me? Cool. For problem three, do we need to return the item for the pop method? Or is it its prop? So pop should always return the item it's removing? That absolutely should happen? Does that make sense? Cool. Okay, well, I'll stick around for another five, six minutes. Um, and I will bother with you with music since no one's asking. Oh, wait, someone asked. Good. Good. You just don't want to hear that music. Problem. One, just checking position four would mean like index four, right? Yes. Yes. The the position corresponds to the index. Good. does pop usually return the index of the remove that it always returns the actual item from the outside users perspective? index isn't really a thing with a stack. Does that make sense? Okay.

57:58

You mean an enhanced for loop? And in fact? Yes. Because an enhanced for loop is implicitly using an iterator. So I believe that does fulfill the the parameters of the problem. Would you say the textbook reading is fairly comprehensive for the test. Besides the Extra examples you mentioned, the textbook goes far beyond what's on the test, you really want to cross reference your class notes with the textbook, because what I mentioned in class is what I consider important and what I test. So if the textbook is going off on a tangent that I didn't speak about, in class, you really don't need to focus on it. And I don't think I did very much in the class. Um, that that wasn't in the textbook other than those
DS Lecture 9 Page 14 of 15 Transcribed by <https://otter.ai>
that those two little examples I brought up.

59:07

So that to that person who sent me a direct message, um, so when I'm talking to the whole group, it's really, really hard for me to read those sort of that wall of text unless it's sort of a group thing. So let's maybe put a pin in that one for a minute. Okay, well, I will turn the music back on until somebody else asks a related question. Live ratings Yes. When. And I'll never lose. And I choose to serve is mine. Because

1:00:55

I think it's a good example of recursion, but I'm not going to ask you very detailed questions on Tower of Hanoi directly in the exam. Does that make sense? Cool. Okay, folks, so I think on that note, I'm going to call it a review session. are

1:01:28

are we are we good? All right. And yes, it was news. Take it easy. Thank you. Yep.

DS Lecture 9 Page 15 of 15 Transcribed by <https://otter.ai>

DS Lecture 10

Wed, 8/11 1:24PM 1:15:04

SUMMARY KEYWORDS

binary tree, tree, operand, expression, node, binary search trees, midterm, height, operator, complete binary tree, traversal, homework, order traversal, called, postfix, left, question, evaluate, holds, class

00:35

Stop

01:20

where's my where's my, my

01:27

my

02:08

where's my

02:11

where's my

02:15

my

DS Lecture 10 Page 1 of 18 Transcribed by <https://otter.ai>

03:03

Where's mine?

03:06

Where's my

03:10

my

04:27

Alright, let's get started a few stored I'm looking for a few notes um

04:40

I don't have this one public I think the only thing public I have is from like a year ago. Um, so uh first of all, hopefully you all sat the midterm because the window for that is closed so There's that. So, part one of the midterm, which was the, the courseworks part that I mean, that's already graded, right? I mean that that auto graded itself. Um, so as not to like monstrously freak people out, I'm going to release both parts at the same time, rather than give you half and then make you wait for the other half, because that just seems sort of mean. Which means it probably is better if I don't start telling you statistics of the first half, either. So I'm just not going to do that. I'm meeting with the TA tonight to get started on the grading of the the part two of the midterm. Well, you feel that way, I don't know, the private messages, the average that bad. No, the average was fine. The average was the same as it is every year. It's it's it, everyone did fine. But the point is that we're going to release it all at once. So the plan is that we'll grade that second part this evening. Uh, I mean, there are more than 300 of you. Wow, this like, little less than half of you that come live, and then the rest, purportedly watch it later. Probably at double speed right before the exam, but whatever. Um, and then the one person who's watching it, like five hours from like, now is like, hey, anyway. So we will hopefully get that done. That's the word I'm looking for. We will hopefully get that done before Friday, or by Friday, but you know, there are always little snags and always takes a little while. So like, don't bug me about it. Because it's not going to speed anything up and you didn't miss anything. The hope is that we get it done by Friday, at which point I will just release all of it. Okay, as a nice little, you know, going away present for those spring break or whatever it is. And all join Ted

Cruz in Cancun or whatever it is you're doing. I'm sorry, that was it was a poor joke. But anyway. Okay. So that's one thing. The other thing, and this is this is a thing. You do have a programming homework due this Friday, right? Yeah. And I think some of you probably put that off because of the midterm, and are now very quickly realizing holy crap. That's a homework. Um, yeah, it is. I will say at the very least, you don't have the right thing to worry about, because we've decoupled them so that that's helpful. But yeah, we there's some work you guys have to do before, before you're free.

08:13

So what I'm afraid of, and I'm just gonna, there's not much I can do to mitigate it at this point. But what I'm afraid of is that office hours that remain are going to start getting really crowded. I was a little surprised that my own office hours, I mean, had a steady stream of people, but it wasn't crowded, crowded. But then I realized y'all went to eBay's office hours, and there were like, 40 people there. So, uh, yeah, office hours are going to be crowded over the next few days. I'm sorry, I did have some of my ta to shift their hours away from Monday and Tuesday into the next couple of days to try to take some of the load off. But you know, it is what it is. We'll be monitoring piasa I do have some slots open on Friday if you need. But that's due on Friday, late deadline, I guess it's Sunday. And that's it. Homework three will come out. I'm not in a huge rush at this point to get it out. Just because I know people are still frantically working on to my feeling is that homework three will probably end up being due somewhere in the vicinity of March 15. That's what I'm thinking is the most likely scenario given me a little bit of padding for spring break and whatnot, which is good because that I think I can have homework for do around April one. And then homework five do around April 15, which I believe is roughly the last day of classes. Right? Think I have that right? Um, so I think we're in ok shape there. So homework three. I will push out over the next day or two but my main priority right now is to get going on the grading for the midterm just to get that out of the way. Okay? So that's where we are with that. Homework three is sort of the most probably the most difficult of the homework. Honestly, with that in mind, although my ta is do not have to hold office hours over spring break, because that would just be mean, my intention, and I don't think it's on the calendar, but I'll put them back. I will hold my office hours pretty much as normal over spring break. Okay, so I'll be making myself is there a curve in this class? That's well times question. I will make myself available over spring break. Okay. One little word of warning on my office hours over spring break. I have, it is considered spring break, but I told my ta to update their calendars to update the calendar with what they're planning on doing. They probably haven't done it yet, because they also have midterms. So I'm going to reiterate tonight that the TA is need to update their office hours on the calendars tonight. So if you still see them there tomorrow, the expectation is that they'll

be held. I'm not sure yet exactly what they're going to do. They are not required to though. Okay, I'm not requiring it. Alright, so the one little word of warning about my own office hours over next week, I am, because I'm probably going to start doing some hybrid and in person teaching over the summer, I'm on the hunt to get a vaccine shot. So when I get one on going, even if it's during my office hours, so if I have to reschedule that would be the only reason why. But I will update folks. Okay. All right. So that's the plan. Any questions for me? Oh, is there a curve in this class? Can I just say no and move on? No, of course there is. I don't do a, I don't do a what's the word I'm looking for. I don't do like a standard bell curve. I, I try to center my grades at a B plus. So the mean, does end up being a B plus sort of a mid to low B plus is how I grade. But then where I draw the lines between letter grades tends to be I look at the histogram of the grades, and I look for gaps. So I like to see a spot where like very few people get a score. And I draw my line there, just so that there's actually sort of a differentiation between the grades. Right. So I

don't like just like an arbitrary line is like your point one on one side, you got to be plus point one on the other, you get an A minus. I don't curve down. If everybody does like fantastically, I'll just default to like standard American High School grading practices. But the likelihood of that is not super high. I mean,

13:14

I could. Alright, so my expectations is that the midterm averages usually fall in the mid to high 70s. somewhere around there, sometimes a little higher. The average on the multiple choice section was a smidge higher than that. But usually the average on the written section is going to be a smidge lower than that. So we're kind of there, which means that in all likelihood, like an 80% or so in this class is where the B plus will be. Which means we're like way way off from like high school grading, which would be like an 89 for a B plus we're not there. Okay, one other note about grading I guess this is relevant, since a lot of you have the option of past failing this class. Now, because of the various rule changes. With the exception of Barnard I don't know what the hell happened there. It's it's dumb, but whatever. And you can kind of quote me on that. I don't mind. I didn't think it was a good, good choice. But maybe don't. I'm sure they had their reasons. But whatever. But the way I say that I guarantee that I'll give you a C minus or better as long as you have a 50% or more overall average, okay. So, if you have a 50% or better overall average in the class, you will get a C minus or better. And the logic on that is that a C minus is what counts as a pass. So if your pass failing this class, you try Trying to get over a 50% average. Now, if the averages in the class are significantly lower, that I feel like lower than a 50% might warrant a C minus or better, I reserve the right to go lower than that. But I'm not going to like drag it up, you're guaranteed to C minus or better? Do you have at least a 50%. That's the only sort of hard line I draw on migrating just to give people who are sort of on the, on the cusp. And there's not too many people in that category. But you

DS Lecture 10 Page 4 of 18 Transcribed by <https://otter.ai>

know, there's some, just so that they have something specific they can target for. If you if you're asking me, what do I need to target to get an A plus, you'll get a slight chuckle from me, and then I'll move on. But actually, maybe I will make a comment about that. I'm really, really generous, sort of in the mid to low end, I am really stingy at the very high end. I do not give a lot of a pluses. I do give a pluses, but I don't give a lot of them. I usually try not to give more than a few percent of the class, a pluses. Okay. All right. So that's my spiel on grading everybody. Okay, with Well, okay, Everybody understand that? Okay. Now, in terms of a makeup lecture for Monday, the decision was, I'm going to see how far I get today. And then I will post an additional lecture following on from where I go today. And then you can watch that at your leisure over break. I didn't want to pile everything else on. So that has not been posted yet, because I don't yet know what it's going to be. Alright. Um, so that's the plan there. Okay. Other questions? I think I've addressed everything that was asked privately and publicly so far. We're good. Oh, one other thing that I feel like I got so many emails about this. I feel like I'll address it in class. Question eight, on the multiple choice, which was that there's a makeup lecture for the Monday that we skipped mark. That hasn't been posted. Okay, good. Question eight on the multiple choice section, which said, remove it removes every other element. Everybody was really weirded out. But in the fact that I was incrementing it by one, not by two. But the thing you have to realize is that when you remove at a given index in any type of list array, or linked list, the indices of everything ahead of it, shift over by one. So everything shifts back by one, and then I increments by one. So those two things together actually do shift it forward to rather than one. Okay. So that wasn't a typo. Nonetheless, that should not have affected your analysis, even if it was removing every element rather than every other element. I don't even remember if it was a linked list or an ArrayList. But the answer was not really affected by that. Okay. All right.

18:06

Cool. So that's where we are. So in our last lecture, we were doing tree traversals. And I think we were doing some little reconstructions of trees, from pre order and in order traversals of trees, which showed up on your exam pretty prominently. So that was meant kind of as a gift by my doing that as well, like very explicitly in class. Um, so we're going to keep going with our binary trees. Now, where I'm aiming on this is a topic called expression trees, which is a good application of binary trees. And that's actually going to be one of the major one of the two major programming problems on homework three will be expression trees. And then from there, we're going to start binary search trees. Most likely our support and supplemental our lecture that will be pre recorded will be a binary search tree lecture. I don't think I'm going to quite get there. So before we get into expression trees, though, I want to talk about some definitions of types of trees or types of binary trees. So the first variety I want to talk about our full binary trees, okay, so a full

DS Lecture 10 Page 5 of 18 Transcribed by <https://otter.ai>

binary tree is a binary tree in which every node has zero or two children. So every node is either a leaf, or has every child that could possibly have. So an example of a full binary tree might be something like this. every node is either a leaf, meaning it has zero children, or it has two children. Is that clear? Now, forgetting about full for a moment, remind me again, what's the worst case? Big O height of a binary tree of n nodes? It's order n , right? Because you're gonna have something that degrades to a link list. It's a big O of n , it's actually probably n minus one, right? Because the height is the count of links. Now, for a full binary tree. What would the worst case be? height of a full binary tree be? nearby? So a lot of people are saying log in. And, you know, I get why but no. So an over two is sort of the most promising one I've seen, right? Because notice what I'm doing here, this example that I gave you happened to be sort of kind of the worst case it could be in the hype, right? So for every level, right, there are only two children except for the root level, right? One is a leaf, and one is a node with two children. So one is a leaf and one is full. I, I don't know. Alright, so one is a leaf and one is full. And so that means with the exception of that top level root node, every level has two children, sorry, has two nodes. So that means the height of this thing would be half the number of nodes, which as at least one person noted, is big O of n still, right, because that one half drops out. So that's a full binary tree. Now, there is another variety known as a complete binary tree. Now a complete binary tree has a little bit of subtlety to it. A complete binary tree, at every level of that complete binary tree, there is the maximum number of nodes possible with the possible exception of the very bottom level of that tree. And the very bottom level of that tree has to be filled in from left to right. And so in the second to last row, the second to last level of this thing, the left most node on the second to last level, right, left node, right node would get filled in, then you move over and just remain complete, then the left node of the one next to it would have to be filled in, we're filling in from left to right. So given that this little squash guy here is a left node, do we understand that this is a complete binary tree? Yes. Now I have a question. Is a complete binary tree necessarily full? No. And in fact, the example I gave you is an example of a complete binary tree that is not full. Because that last row is filled in from left to right, you might have at the very end of that level, a node which has a left child but then doesn't have a sibling, in which case its parent would not be a full node. Okay. Question. If I gave you this tree and then I added this node over here, is this now still complete or not? It is no longer complete. Okay. So that is a complete binary tree. Right It is only the last row that has to be filled in from left to right. The remaining levels have to be completely filled in there can be no gaps in the remaining levels. All right. Now, this is nice because this gives us a really tight bound on the height of this thing, all right, because the height is defined is sort of that last level, which everything else is totally filled in. But there's one more other than complete, which will refer to as perfect. And a perfect binary tree is one that has every possible node for its height, meaning that last level has to be filled in entirely. So for a given height, this is actually the maximum number of nodes that a binary tree could have. It's as tightly packed as possible. is a is a perfect binary tree

complete? Yes,

25:52

it absolutely is. Its last level is filled in from left to right. And it happens to be totally filled in, which isn't necessary for it to be complete. But it's definitely complete is a perfect binary tree also full?

26:12

Yep, absolutely. So a perfect binary tree is both complete and full. Whereas a complete binary tree is not necessarily full. And so now I want to talk a little bit about the number of nodes in a perfect binary tree as it relates to its height. And this is where we start getting the very useful relationship. And so if I have a perfect binary tree that is a binary tree with the maximum number of nodes possible in it. So a perfect binary tree. So you can't have a, you absolutely can have a full binary tree and a compliant complete binary tree that's not perfect. These are examples of full and complete binary trees that are not perfect. Okay. So a perfect binary tree of height h has two to the H plus one quantity minus one nodes. And we would like to prove that. All right. So we'd like to prove that a perfect binary tree of height h has two to the H plus one minus one nodes. And so to prove this, we're going to use induction. All right. And so if I hadn't made it to this, this point in the lecture last Wednesday, this would have been on your midterm, right quite make it so I had to put something else on instead. But I want to prove this by induction. And so our induction proof is going to take three steps as normal. Our first step is to show that this holds for the base case. So for our base case,

28:37

imagine we have a perfect binary tree of height zero. So a perfect binary tree of height zero. Looks like that, right? It's just the root node, and the height of that tree is zero, because well, the furthest leaf is itself. So there's one node in that. So now all we have to do is show that our our property Hell's so two to the zero plus one minus one is equal to two to one, so that's two minus one is equal to one. So a binary tree with the maximum number of nodes possible of heights zero has one node and so our property holds in the base case. We now assume that the property holds for all perfect binary trees with high with hates zero Through and I have no idea why I wrote that through in script. But that's just what my hand decided to do zero through some arbitrary K . So we're going to assume that this property holds true for perfect binary trees of heights, zero through K . So our last step is now show that if this is true, must also hold for four H equals k plus one. So

DS Lecture 10 Page 7 of 18 Transcribed by <https://otter.ai>

basically, what we need to show is that a tree of height k plus one, assuming this is true, that it must follow that a tree of height k plus one must have two to the k plus one plus one, right? We're subbing in k plus one for H , so we have to show that it has to the k plus one plus one quantity minus one node. So we have to show that that's true. Okay. Now for this, we're going to do sort of a, a very, very loose form of, I guess, structural induction. So I'm not going to be quite so formalized about it, what we're going to do is we're going to using a diagram, construct a perfect binary tree of height k plus one, and then show the little bit of math that shows that in fact, it still does hold. So to do this, we'll start we'll start by changing the page Come on page. We'll start by, yeah, there we go. Let's say we have a perfect binary tree of height K . All right.

32:10

So I'm representing that perfect binary tree of heights k by this triangle, based on the inductive hypothesis, where we assume this held true through K , how many nodes are in this perfect triangle? Maybe remind me, we should know it. $3k$, not two, so two to the k plus one. minus one, basically, we just take the formula we had that we were trying to prove and subbing k for H because we assumed that would be true. So we know there are that many nodes and and now what I need to do is I need to construct another perfect.

No, not technically x to the k plus one minus one. And now what we need to do is a wing. Sorry, we need to construct a tree of height k plus one. And so we'll do that by creating a new root node and hanging this existing triangle off of it. Do we all agree that that is a binary tree of height k plus one. This triangle had high K . We added another level to it. So now it's high k plus one. But is this perfect? Right? It's not perfect. It's actually completely imbalanced. But we can make it perfect, right? How would we make it perfect without changing the height? So we have the same triangle on the right. Right. So now we've got two triangles, both of high K , but they're next to each other. So that doesn't increase the height at all. So this is still high k plus one. So now we have built out of this, which we knew how many nodes were in this based on our inductive hypothesis, we've now built a perfect binary tree of height k plus one. And now all that's left for us to do is to count how many nodes in based on our assumption, we know that this triangle has two to the k plus one minus one nodes, but so does this one. So we have to add them together. So in fact, taken together, that's two times two to the k plus one minus one is that how many nodes are in that tree? Or if I missed something, what else is there plus one more, right, because we've also got the root in addition to those two triangles. And so now we can just multiply this out. Two times two to the k plus one is two to the k plus one plus one. Two times minus one is minus two. And then we've got this plus one hanging out on the outside. And so these two combine, and we end up with two to the k plus one plus one, still minus one, which was exactly what we were trying to show. So now we know that if it holds up through height of k , then it follows that it holds for height of k plus one. And that proves

DS Lecture 10 Page 8 of 18 Transcribed by <https://otter.ai>

the proves the relationship. Okay. So if the height tells me that there are two to the H plus one minus one nodes, then if two to the H plus one minus one. Now, Bo, now we can have x , if two to the H plus one minus one equals x , we can now solve for h in terms of x by taking the log of both sides, right? Yes, so we take the log base two of both sides. And so when we do that, then we end up with H is going to be on the order of log of x , or I guess we should probably have called it n , right? Because if there are n nodes, that means the height is on the order of the log of it a grade. So now we know through this, that if we have a perfect binary tree of n nodes, then its height is going to be on the order of log of n . And in fact, if we have a complete binary tree, we actually still know that because the difference in height at any one spot is not more than one.

37:12

And so this actually tells us very useful information that if we have this perfect binary tree, you have a bound on its height. So if we have operations that move down one level per step, we now can constrain our costs to the log event if we wanted to. Okay, which is nice. And that'll come in handy when we do our binary search trees in a minute. Now, this is actually somewhat more restricting than it needs to be, there is a more broad definition of a so called Balanced Binary search. So we'll talk about that extensively the week after spring break. But a Balanced Binary Search Tree is going to be a binary search tree, where the differences in height at every level differ by no more than one. And if you have a Balanced Binary Search Tree, it's still gonna be a lot deeper than a perfect one. But its height is still constrained to order log of n , it's actually two times log of n . But so balanced is not necessarily complete or full. All right, because just says that the heights differ by no more than one. And that still gives us the log of n costs. All right. So there's some definitions for us, there's our I guess, our second induction proof and on your written for homework three, you'll have to do another one. Also, the first problem on the written portion of your final exam will be an induction proof to a I can just tell you that definitively right now, so you might jot that down. Um, but now what I want to do is I want to do expression trees. This is going to be an example

39:08

of an application of binary trees. Okay. So our expression trees are going to be a way for

us to represent mathematical expressions. Alright, so something like three plus five or minus four, or something like that, okay. We want to be able to represent our mathematical expressions in a tree form. And so each node of our expression tree is going to be either an operator or an operand. And the idea is that if you have an operator its left child will be Left hand operand and its right child will be its right hand operand. Now

DS Lecture 10 Page 9 of 18 Transcribed by <https://otter.ai>

for the purposes of this conversation, we're going to assume that our binary that our operators are only binary operators, that is, our operators take exactly two operands. The other possibility for a node, if it's not an operator is that that node is an operand, an actual, like an actual number. And for nodes that are operands, that would then be a leaf. Okay. So if we just had three plus five, and that was our expression, the expression tree that might represent three plus five would look like this as your root node, you would have an operator node, and then its left child would be the operand and three, and its right child would be the operand five. And as somebody noted in the comments, if I did an inorder traversal, of this expression tree, I would end up with this expression we saw here, agreed. Okay, let's show a slightly more complicated version of this. Now we have a slight issue here, because we got to add three and five First, take the result of that and subtract four from a grade. So an expression tree that might work for that could potentially look like this, we might route it at the minus, but the left hand operand of the minus is going to be the result of evaluating that sub expression. So the sub tree that hangs off, in this case, to the left of our operator must evaluate to what we want as its left hand operand. So it doesn't actually have to be an operand here, it has to be a sub tree that will ultimately evaluate to its left hand operand. So what I would then do is I would just take this whole sub tree, that amounts to three plus five, and hang it right there.

42:27

And this entire sub tree would correspond to the left hand side of that, and the right hand side is the four. Are we good? Question. For an expression tree. Which of those which of those definitions I did in the first segment of this class is guaranteed to apply to an expression tree? It's guaranteed to be full. Right? Because a full binary tree is one where every node is either a leaf or as two children. And that pretty much corresponds to my definition here. My nodes were either operators or operands. And because my operators are only binary operators, my operators must have two children. And my operands must be leaves. It's definitely doesn't have to be perfect leaving this is an example of one that is not perfect. This one happened to be complete, right? But it didn't need to be complete. Okay, there's no output, so to speak yet, this is just representing the expression. For now. We'll figure out how to evaluate it in a few minutes. But this represents the expression. Are we good with this? All right. Now, there are a few things that we might like to be able to do with this. Alright. And I'm going to sketch out some code for how we might do some of this. And you should take note of these sketches, because this is going to sort of be very helpful for how you do your homework three, programming problem one. So first of all, we should talk a little bit about how we might represent these things, right. And so you know, we might have as our public class, expression tree. By the way, these expression trees very much do not need to be generic. We're very constrained in what we're storing. Were just operators and operands. Right. And what is the only instance variable we need for a tree

DS Lecture 10 Page 10 of 18 Transcribed by <https://otter.ai>

node Yes, but which node? The root node, right. So we'll have a separate class called expression node.

45:20

And that will be the type for my root instance variable for my expression tree. Now the expression node class, you could probably make a static nested class inside of expression tree, following the same pattern we followed for our link lists.

45:40

And so we might have our class probably make it static, our static class, we'll call it expression node.

45:58

And our expression node is going to have to be able to take two forms, right? It's going to have to either be an operator or an operand. Now there are a lot of ways of kind of getting that to work. If it's an operator, I would expect an operator that least the way we're doing and if you want to have plus minus times and divide, if it were an operator, I would imagine my operator would be a character. If it were an operand. Let's assume for the moment that No, there's nothing generic here. Because we're either operators or operands. We're nothing else. So it doesn't need to be that flexible. Um, so our operands let's, for the moment, let's constrain our operands to be intz. And so what we'll actually do is we'll have two data variables, one that gets filled in if we're an operator, and one that gets filled in if we're an operand. So we'll have char, operator, and int, operand. And if it's an operator, that variables filled in, if it's an operand, this variable is filled in. And then we just need left and right children, right. So we need additional expression nodes. Left and expression node, right. That's really all we need. Once I have a built tree, how can I tell whether or not a node is an operator or an operand? So I know which which field to look at? If I yeah, if leaf. So if both the left and right children are no, it's a leaf, and therefore the operand field is the one that matters. If the left and right children are not no, then it's an operate that it's an operator. And this field is the one I care about. If the left child is no and the right child once again, if the left child is no and the right child is not. Well, then there's a mistake because this was supposed to be a full tree.

48:30

DS Lecture 10 Page 11 of 18 Transcribed by <https://otter.ai>

Oh, what's up? Hey, which coffee shop do you go to? You go to Starbucks or Dunkin donut because Why do you have so much energy right now? Seriously?

48:40

Okay, so I'm gonna go ahead on this I'm I'm seriously asking you this question. Because, quite frankly, I'm getting really sick and tired of dunkin donuts. Okay, I'm gonna go ahead and lower everybody's hand I think. And I will note, I don't drink coffee at home. It's it's this it's too much of this I coke. Okay. All right. Um, and for the record, I hate Dunkin Donuts coffee. I'm happier drinking Diet Coke, though. It's much healthier. Not really. Either. It's the sugar it's some chemical that's doing something else for you. But whatever. I need the caffeine. That's my one addiction. Moving on. Um, so Are we clear on our structure? Now, imagine I wanted to write a method of expression tree that evaluates itself. I want to evaluate an expression tree. What would I do? Now I'm self conscious about drinking the Yeah, whatever. More caffeine. I was about to make Comment because I said that's my my addiction. And then I was like, some story about a Columbia faculty member and addiction. And I think I need to not go there right now. So I'm going to just shut up. Moving on. Yeah, definitely need to shut up right now I'll stop. Alright. So perform and. And in order traversal? Not quite. You're very close. But if I want to subtract two things, I have to know what they are first, right? I need to know, right? So it's actually going to be a post order traversal. Because I have to know exactly what the left subtree evaluates to first, then I need to know exactly what the right subtree evaluates to. And then when that's done, then I need to apply my operators. So only once I've evaluated the left and the right, can I apply the operator. And so it actually can be done recursively as a post order traversal. If I have the root of the tree, I can then evaluate the tree using such a traversal. So let's sketch out what that code might look like. Okay, so we're gonna probably have to follow. So right now I'm inside of expression tree. All right, so I'm writing methods that are inside of expression tree. And what we're going to do is, we're going to imagine a public Val method. Now, since this is an expression, tree events will assume integer operations, so integer division and things like that. So it'll return an int. Okay. And we'll call it a valve.

But remember, this is a member, a public member of expression tree, which means that the person who invokes this has absolutely no knowledge of the nodes structure of the whole thing. And if we're going to do a recursive traversal on it, we really need a version of this, that takes the nodes. So we're going to need a private one that this thing invokes. So the public of Val will return the results of invoking a private Val, on the root of the tree. And then the real work is going to happen in a private one. Which is going to take in an expression node, which I'm going to call T. Or, okay, that's a curly bracket, not a face, but whatever, what I'm going to call t, which is going to Yeah, these are methods inside of expression trees, somebody has to private question. So these are methods inside of expression tree. We're all good with that. That's worth emphasizing again, I think. So that's

DS Lecture 10 Page 12 of 18 Transcribed by <https://otter.ai>

going to be the root of the subtree that we're currently evaluating. Now, this is initially invoked on root. So we said this is going to be a postorder traversal. So base case will assume a non empty tree.

53:35

So if t is a leaf, so if T dot left

53:43

is not equal to No. Oh, sorry, that was wrong. If t dot left is equal to No, and T dot right, is equal to No.

53:58

Then we're a leaf. And so what do I do if I'm a leaf? So I have to return something What do I return?

54:14

Non there it's fine to be a leaf. That sounded like some sort of self affirmation, it's fine if you're a leaf sorry, you return an offer okay, because you are an operand at this point. So you need to just return your value. So if you are a leaf, you evaluate to what whenever the operand value so return T dot operand. And just to ram that home. That is a perfectly valid expression tree of a single node with just an operand in it because remember, the root node can be a leaf just fine. And that would just be an expression that evaluates to three. So that's okay. So if you're a leaf, return your operand, if you're not a leaf, then we go into the post order traversal aspect of this, we evaluate the left subtree, then we evaluate the right subtree, then we apply the operator. So, int I Val are some temporary variable that will store the result of evaluating the left subtree would be equal to the result of recursively, calling a Val on T dot left. Okay, then we need to evaluate the right subtree. So, our Val will be the results of evaluating T dot right. And then once that's done, I mean, most likely, we're going to have to write another helper function, which I'm not going to write here. But we can imagine that we write another helper method called apply. And apply will take in as an as a char, the operator that you actually want done, and the two operands you want to apply to. And so you would, when you implement apply, you probably just have a switch case on the operator, like if op equals equals plus, you know, Then do one thing else if people's equals minus, then do something else. Um, so somebody

DS Lecture 10 Page 13 of 18 Transcribed by <https://otter.ai>

asked me a really good private question, I'm gonna put a pin in it for a second. And let me just finish this thought. And I'll get to you in a second because it's a very good point. So now the last thing we need to do is we need to apply so we're going to return the results of calling this not shown, but you can easily figure out apply method on T dot operator, because whatever the operator is, you're applying, and you're applying it to L Val. And our valve that will do the calculation and return it back to the user. So is everybody okay with the structure of this? traversal? Okay, so this is a standard postorder traversal, it will visit every node exactly once, right. And this recursion will allow that to happen. So this will be order n, where n is the number of nodes in the tree. Now, the question that I was asked privately said, isn't that bad, because it's two recursive calls. And which is a really nice

note, because it recognizes the fact that we had that compound interest rule, which resulted in exponential cost when we did something like that. So let's actually think back to that Fibonacci example. And just so that we understand what n was in that example, and was which Fibonacci number we wanted. And then when I drew out the chart of calls, because it branched, right, we said it was going to be two to the n number of calls. Well, if you think about it, that end was the height of that tree. Because at the very bottom of that tree of recursive Fibonacci calls, you finally got down to your base case. So in that Fibonacci example, my n was the height of the tree. And we just proved that if your height of the tree, if you have a height of tree H , then the number of nodes is two to the H . So if my height was in, of course, the number of nodes would be two to the N .

59:00

In this case, n is already the number of nodes.

59:07

So it's actually not a problem, because in that case that we saw previously, that was just saying it was actually just causing us to realize that the number of nodes was very large. In this case, it's just going to visit however many nodes there are person who asked private question, Is that good? Cool. All right. And then so there was a follow up, it is not redoing what another recursion is doing. And so that's actually a valid point. Also, um, especially in this case, because we're recursing on things that we haven't already calculated, we wouldn't have had a choice. Anyway, what was particularly bad in the Fibonacci cases, we were reiterating work that we didn't need to do again. In this case, we have no choice but doing it. Okay. All right, any questions on this? Good. So this was a post order traversal. Now, let's go back in Look at the tree for a minute. So I'm just going to go ahead and draw sort of an arbitrary tree. Um,

DS Lecture 10 Page 14 of 18 Transcribed by <https://otter.ai>

1:00:15

could why would CA's answer the question? It's a good question. But I want to move forward a little bit. So I'll let the teachers answer that. Um, okay, so let's draw a tree. Alright, does this look okay? Just a valid expression tree? I think so. Now, we saw in code now already how we might evaluate such a tree. But

1:01:16

but maybe there's some other stuff you might want to do. This isn't exactly, easily human readable. In the earlier examples, what I was doing was I was actually showing you sort of a regular infix expression that I then showed a tree that it represented. So you may remember the concept of a postfix expression or RPN, from the stack conversations and from your midterm. Yes. So you remember postfix expressions where you had the operands, followed by the operator. And you may also remember that there existed a thing called prefix expressions where you have the operator first, followed by the operands. And then what we're typically used to seeing are what are known as infix expressions where you have the operator in between its operands. So you can actually generate prefix expressions, postfix expressions and infix expressions by performing pre order post order, and in order to reversals of the tree. So if I wanted a postfix expression from this tree, I would do a post order traversal. Okay. So let's do that. A postorder traversal means recurse, left first, then recurse, right, then visit and in this case I'm trying to print. So when I say visit a node, I mean, print the node out. All right, I assume my ta is are getting the questions in there. It's just taking a while to type up. Yeah. All right, just want to make sure. So we recurse left first. And now I'm very explicitly not writing this out for you, you're going to need to figure out how to do this yourself. I already wrote one for you. This is actually part of your homework, but we recurse left. So now we're doing a post order traversal on the left subtree which means we have to recurse left again, and then recurs left again. So the first thing that prints out will be the four then the three then the times then the two then the minus i don't know what seven. Okay, then the minus. Now

we're done with this. Now we recurse here, so then get the eight, then the two, then the divide. And then finally the plus. And that's my post order traversal. And that gives me the postfix expression. The grade Wait, I don't understand. Oh, geez. Better. Where's the four before is a magical thing floating around in my brain that nobody else can see. Are we okay? Okay. So that's our right. That's our postfix expression. Let's do a prefix expression. So a prefix expression is going to be a preorder traversal

1:05:03

DS Lecture 10 Page 15 of 18 Transcribed by <https://otter.ai>

And in pre order traversals, we print the root first, then recurse, left, then recurse, right, so we print the plus, then we go left, then we print the minus, then we go left print the times, then we're at the leaves. And we're here, we're done with the left sub tree. Now we do the right, print the root first. And we have a prefix expression. I see no fours in that one, I think we're in better shape. Now, in order expression. Now, I believe I happen to have written this in such a way that my infix expression will work. But that was a matter of luck. Let's try it out. So this is going to be an inorder traversal. And I'm going to put a big ol asterisk next to that. Because remember, when we do an inorder, sorry, an infix expression, order of operations start to matter. We don't have to worry about order of operations here, right? Because it's very clear, based on the structure of the tree, what gets evaluated first, followed by what there's, there's no ambiguity, the same way postfix expressions and prefix expressions do not have any ambiguity, the operators are done from left to right. Okay. But infix expressions, it's a little trickier. But let's see what happens if I just do a straight inorder traversal on this. Okay, so an inorder traversal says recurse, right, then visit the root, then recurse left, then visit the root, then recurse, right, so left root, right. So first, we go left. Alright, we're here, then we go left again. And so that gives me the seven times three that's done here, then we do the minus,

1:07:07

then the two. Now we're done with the left hand side of the whole thing. So then we do the plus. And then we do the right hand side.

1:07:19

Okay, so order of operations is such that we would want to do the multiplications first, and the divisions first, then the additions and subtractions. Now, we got a little bit lucky here, in that this worked out, I could have ordered this in another way where my infix would not have worked properly. Let's try to concoct one where it will break.

1:07:46

So let's do let's start with a minus as the root What am I doing, Paul? Why are you being such Sorry? 30 seconds, my brain just want flew off into a corner.

1:08:18

Hold on, let me let me do this without you looking. Here. Look at my keyboard for a

DS Lecture 10 Page 16 of 18 Transcribed by <https://otter.ai>

minute. Okay. All right. I think I'm happy with that one. Okay, so what does this mean? It's supposed to do three minus four first. Wait, am I happy with that one? Yeah, it's supposed to do three minus four first and then take the result and multiply it by seven. Do we agree? So this expression tree, why did my thing just beep? What just happened? Something strange just happened. Sorry, I don't know. Whatever. I'll find out later. Um, thank you. Good call. Alright. So we want three minus four to happen first, then multiply the result by seven. So you expect this to evaluate to minus seven. But let's do an inorder traversal. If I did an inorder traversal on this by itself, the three would print then the minus than the four than the times than the seven. And if you applied standard order of operations to this, this would not evaluate to what you want it. This is not equivalent to that expression tree because Cause the Times has a higher order of precedence. So it multiply four by seven first before doing the subtraction. Do we agree? So how do I fix this?

1:10:21

How would I print out a valid? Yeah, I need parentheses. So every time I recurse on a sub tree, I should print an open parentheses first, then do the sub tree, then print a close parenthesis. Does that make sense? Now, if you're not careful, if you print open, then recurse, then print close, you could end up with something awful that looks like this. And yet, that would still be correct. Agree. So when you do an inorder traversal of this, the net result is that you need to make sure you print out parentheses so that the ordering of the actual calculations matches what's in the expression tree. Okay, so I've got like three minutes left. And so I can't quite do justice to what I need to do next. So let's discuss next steps. The next thing we need to do with expression trees, we need to actually learn how to build them. And so the way this is going to work is that, what we're going to do is we're going to have our constructor for this take in a postfix expression, an RPN expression. And then we're going to run an algorithm. And it's actually going to be pretty much the same algorithm that we use to evaluate postfix expression. So a stack based algorithm, a stack of expression nodes, that will construct this tree for me. And so that will be the constructor for our whole thing. And that'll wrap this up. So I'm going to pre pre I'm going to record another session for you, I may actually piece it together from prior semesters because it's the same material. But I'm going to put together a video for you in the next couple of days, not gonna be instant, it's probably going to be a little later this week, because I want to get the grading done first, which will do two things. And it's important that you watch this at some point over the break, okay? get your homework done, then watch this, it's going to do it's going to go over the stack based algorithm to build an expression tree from a postfix expression. And you're going to need that to do programming homework one. And then I'm going to introduce binary search trees. Okay, in that video. And so it is really important that you watch that, that's going to be critically important. And we're going to pick up in the lecture on Monday after the break with binary search trees,

DS Lecture 10 Page 17 of 18 Transcribed by <https://otter.ai>

assuming you already know what they are based on what you watched. Okay. And in fact that Monday, we're going to do code for binary search trees, which will be coming from the Weiss textbook. So readings, read up through the portion of chapter four entitled avielle trees. Okay. That's all you need to read for chapter four. So we're going to wrap up binary search trees on the Monday we come back, we're going to do avielle trees on the Wednesday we come back, we'll finish avielle trees the following week, and then we're gonna start hash tables. Okay. So that's the plan. Keep an eye out for homework. Homework for keep an eye out for homework three. We'll try to get your midterms back to you by Friday. And, yep, you've got the rest of homework to programming to get done. I'll stick around for a few minutes. But then I got another meeting to go to so I'm going to stop the share. Have a good break. Thank you. Thank you. Thank you. Have a good night. Dr. Blair. Take care. Everybody was in a rush to get out today. Everyone loves to see you. Take care. All right. Um, I will join you guys in that other meeting shortly. Take care. Okay, fine, right

DS Lecture 10 Page 18 of 18 Transcribed by <https://otter.ai>

DS Lecture 11

Wed, 8/11 1:28PM 1:19:04

SUMMARY KEYWORDS

binary search tree, tree, node, left, midterm, binary tree, comparable, remove, tammy, insert, check, unique, root, question, worst case, regrade, bank account, person, good, case

00:18

Hey, Dr. Blair I'd like to take you now on wings of song as it were, and try and help you

forget, perhaps for a while, your drab wretched lives. Here's a song all about springtime in general, and in particular about one of the many delightful pastimes. The coming of spring affords us all.

01:21

Spring is here. Suffering is here. Life is Skittles and my best beer. I think the loneliest time of the year is the spring. I do, don't you? course you do. But there's one thing that makes spring complete for me, and makes every Sunday a treat for me. All the world seems in tune on a spring afternoon when we're poisoning pigeons in the park. Every Sunday, you will see my sweetheart and me as we pies and the pigeons and when they see us coming the birdies all try and hide. But they still go for peanuts when code headwinds I unhide. The sun shining bright everything seems all right when we're poisoning pigeons in the bottom.

02:27

We've gained notoriety and cause much anxiety in the Audubon Society with our games. They call it impiety and a lack of propriety employ a variety of unpleasant names, but it's not against any religion to watch Oh, this falls off. So now you're free. Why don't you come with me and we'll pies and the pigeons in the box. And maybe we'll do in a squirrel or two while we're boys and he

DS Lecture 11 Page 1 of 21 Transcribed by <https://otter.ai>

03:08

will murder them all. I'm in laughter and merriment, except for the few we take home to experiment. My pulse will be quick and and with each drop of stringing a pigeon, it just takes a smidgen to five.

03:30

I felt like we needed something you know, for spring break and all that. I guess I kind of enjoyed myself there because you know I have that usual square of about eight of you with about that actually had your cameras on when we started and the look of horror on one of your faces. Made it all worth it. How are we doing? Break went okay. I see no response which that's too short. Okay. Ah, all right. So, um yeah, it was a it was a tiring break. I spent most of it dealing with sort of family type issues. On the plus side I did manage to get my entire my entire family their first vaccine shot so that that that was a that was a win. But other than that, not a whole lot happened. So we need to, we need to talk about a few things. Um, you guys have your homework three. Yes. And some of you have been working on that. I know because I started Seeing Piazza posts, and had some folks in office hours, we did hold some office hours over break, which were not that heavily populated, but actually had more people than I would have expected. So I think that is a good sign. Um, number three will be due on March 18. So, late next week, I guess, what 10 days from now. There was one. So I sort of had a little bit of a thought about how to deal with the extra lecture, and I realized that introducing binary search trees in a pre recorded thing was probably a mistake. So this is what I've done, I posted and I sent an email out to this to this effect five minutes ago. Um, what I did was I posted you sort of a sub 15 minute lecture, which kind of goes through the stack based algorithm for expression tree construction. It's basically the same algorithm we did for evaluating postfix expressions, and it was part of your reading. But that'll help you in terms of the constructor piece of your, the constructor piece from the, what's the word I'm looking for, for the homework three, programming problem one, okay. We'll introduce binary search trees today. And the way to kind of buy some time back is, I will, instead of going through, hold on one sec, instead of going through all of the code for binary search trees, I'll have that as a follow on to this in a separate lecture, just the code review. Okay. Now, in terms of readings, in chapter four, you should be reading through avielle trees, you don't have to read any further than avielle trees. Okay. Um, we'll probably get to avielle trees. With that little supplemental lecture that'll give you with the code. We'll get avielle trees probably on

Wednesday. And so after ACL trees are done in chapter four, you should start reading chapter five. And in chapter five on hash tables, you should be going through the section on quadratic probing. Okay, is everybody clear on the readings? Everybody clear on the lecture schedules. Super. Um, now midterm.

DS Lecture 11 Page 2 of 21 Transcribed by <https://otter.ai>

07:34

So all of the midterm is finally graded, there was a little bit of delay on part of some of part two, I wasn't able to click release on midterm Part Two just yet, because there are a couple of things I need to review this evening. But I'm going to click release on that within grade scope, not courseworks grade scope for midterm part two, hopefully before midnight tonight, okay. But in the interest of giving you guys some feedback, since I know you'd like to know where you stand, I went ahead and I released the scores on midterm part one, just a little bit earlier today. Now, a couple of comments on that, before folks panic. It's probably too late because within I don't know how to say this. Um, let's say while I was typing the email explaining what I just did, I got two emails. And there was a Piazza post while I was still typing, and it was something. So just so that we're absolutely clear on what you saw, what you saw was a score out of 50 on midterm part one. Okay. So if you saw a 50 on there and said to yourself as some sort of a swear word like, Oh, God, I didn't realize I did that badly. No, you did perfectly fine. So far, so good. But that's only half the battle, right? That's an uncurbed score. That's a raw score. I don't really post curved scores. I don't really curve that way. Because in the end, I assign grades based on your overall averages. Um, I believe the average on that which I think I had mentioned, already was something along the lines of, I guess, was it.

09:34

Oh, now I'm forgetting what I said. Does it listed in there? I should probably just check so I don't say the wrong thing. Um,

09:49

so it's 40.6 According to courseworks, that sounds pretty much spot on. So the average out of 50 was 40.6. So just a smidge over 80% It looks like the average on Part two is a bit lower than that. But, you know, high 70s, very low 80s is pretty much bang on what I was expecting for this midterm. So I'm, I'm really, really, really happy with how this has come out. Okay. Now, I'm, in addition to releasing the scores, I also reconfigured the quiz. So that now when you click on the quiz, you should see what your answers were. And you should see what the correct answers should have been. All right, that that should be viewable within the quiz itself. Okay. Now, one thing I will note on midterm part one, there shouldn't be any regrade requests, because those were multiple choice questions. And there, there were right answers, and they were not right answers. So that's your score. Um, for the second part, that's greater than grade scope, much like the written homeworks are concerned. And for that one, I will allow regrade requests, in case we misread something,

DS Lecture 11 Page 3 of 21 Transcribed by <https://otter.ai>

but please manage one's expectations on that, because we were pretty consistent about how we applied rubrics. And I'm probably not going to be particularly flexible on that. But if, you know, we completely misread something, we will accept regrade requests on Part Two through grade scope for at most two weeks. Okay. And that that that Val mentioned that when I send out the note, when I've actually released that part in great scope, the midterm part two will not be the scores will not be moved into courseworks until the regrade. period ends so that we could just move it over once without having to worry about updating in multiple places. Is everybody good with that? And just again, in case you didn't remember the conversation I had about this previously, I sort of set my averages to be a low to mid B plus. So like, if you have 40. On the first half, you have about a B plus on the first half of the exam. Is that? Does that sort of give you a calibration point? Yes. If you have below 25% on the first half, that's a little bit of a

problem. But let's see how you did on the second half. If the total is less than 50%. Between the two, I would suggest coming by office hours and having a chat at some point just so that we can see how to pick things up. If you had over 50% you pass the exam, no problem. And that's really how you should calibrate where the line is for an A plus, don't ask me that question. Okay. I don't know. I suffice it to say it's very, very high. Okay. All right. Anything you want to ask about before I move on?

13:17

Yeah, it's, it's cumulative. That's, that's the way I operate on this. Okay. Um, anything else? We're good. Okay. I'll say what that is when I send that out. Presumably, it'll be two weeks from today. Alright, nothing beyond that. Okay.

13:47

All right.

13:50

binary search trees. Let's get started. Um, I really can't tell everybody's very quiet. Most people have their cameras off. And it just feels like everyone is incredibly tired right now. Which seems like the exact opposite of what you would expect after like a week off. All right, let's do this. So we saw binary trees. And we saw some applications of this. Three assignments due right after spring. Okay, you can't blame me for that. I I didn't throw anything at you. And basically, nobody touched homeworks. Okay, so let's do the binary search trees. Okay, so, what we're gonna do now is, we're going to have our binary search trees. Sorry, we're gonna have our binary trees be a vehicle for storing data in a more

DS Lecture 11 Page 4 of 21 Transcribed by <https://otter.ai>

structured order, okay. And in particular, our binary search trees are going to represent what in Java we would refer to as a set. If we talked about sets before in this section, I'm forgetting. Now. All right, so a set. All right, is sort of like a collection, only there is the constraint that every item in your set is unique. Does that make sense? So all of the items we're going to be putting in the binary search trees are going to be unique. And that's going to allow us to implement a few different types of things. In Java, I don't think you guys would have seen this before. But Has anybody encountered a map in Java before? Like a tree map or a hash map? A few of you might have seen it, but maybe not. But I suspect a lot of you are. Yeah, so Okay, I said yes, to a person who messaged me privately. So a person suggested like a dictionary. And in fact, that was what I was going to say, um, for many of you, you've probably have had some experience with Python. And in Python, you've probably encountered a construct known as a dictionary. And a dictionary in Python is basically where you look something up, and it gives you the corresponding information back. All right. And so that's really what a map is, in Java, there are two components to a map, there's a key and a value. And the keys are six. That is, they are unique. So every key into a map is unique. And so if you search a map on say, Paul, that would have to be unique. And then that would give you back some value, some data associated with Paul. Now that value that data doesn't necessarily have to be unique. It's just the keys are unique. So it's like a way of looking up records. And so our binary search trees is a nice way of organizing our keys, because it's a good way of storing things that are unique. And so since this is the first time we've discussed sets, I get the feeling I probably haven't told you a story about Tammy Orman, is that correct? Nobody has heard the story about Tammy Orman from me. Okay, so we're going to push that to the end, because I want to do content first. But yeah, Bo, I think has heard that one before. But so I'll tell you a story that has to do with sets that will call the Tammy Orban story, in the last, say, five minutes of class. So if you want to skip that you can go ahead and leave. But we're not going to do that just yet. So our binary search trees are going to be storing unique values. Now for the purposes of this conversation, I think what we're going to do is we're going to just have them be integers. But the main thing is that to work with a binary search tree, they have to be orderable. Right? They have to be sortable, you have to be

able to ask the question is something less than greater than or equal to something else?
And so when we implement this, what does that sound a lot like a buy?

18:41

Sounds like comparable, right? So when we do our binary search trees, we really want our data to be comparable. But again, for these diagrams, we're just gonna use integers, which are easily comparable. Alright, so in our binary search trees, each node will have a piece of data, which for the moment, we're going to talk about as integers. But in the more general sense will be some kind of comparable. And now there's going to be the binary

DS Lecture 11 Page 5 of 21 Transcribed by <https://otter.ai>

search tree condition that we're going to attach to this. And the binary search tree condition is that at every single node in a binary search trees, all the nodes to the left of a given node must be smaller than that node. So everything in the left subtree of the five has to be smaller than the five. And all of the nodes that are to the right, of a node have to be larger. So everything in the right subtree has to be greater than five. That makes sense. And we don't have to worry about what happens if they're equal because we said we're a set. So these are the keys. These are the unique things, we don't have to worry about them being equal. So would that be a valid binary search tree? Looks good? How about that? Still looks good about that? Nope, because everything to the left of the five is no longer smaller than the five. Now notice, we said that, um, that this condition has to hold true for every node. So the one in the seven are okay, right, because they have no children. And in fact, the two doesn't violate any conditions either, right? Because everything to the right of the two is larger, and everything to the left of the two is smaller. It's the five, that's got the problem, because of something lower down here. So this can't be a seven. But it could be a four just fine. I agree. And, you know, maybe we'll put, say, a seven here.

21:30

So this looks like a valid binary search tree. Excellent. Okay, now, in our binary search trees, we have a number of operations that we want to be able to perform. And so one thing we want to be able to do is we'd like to be able to do a contains check. Like, we want to be able to ask is a value in the binary search tree or not? Okay. Now, this is a binary tree, the representation and code of this binary tree will be very similar to the representation of other binary trees, we actually sketched out what that representation would look like for expression trees, right, where the binary search tree itself would have a single instance variable, which would be a binary node, which would be the root of the tree. And then the root node of the tree would then have other nodes hanging off of it. So the same kind of structure we've been seeing all along. So the only thing we have access to when we do a contains check on a binary search tree is the root node. But that's actually good enough. So let's say I'm going to do a contains on, say, a six. So what we can do is we look at the root, and we see whether or not the root is the value we're looking for. If it is good, we're done best case scenario, order one. If it's not, we actually have the ability to decide where to go, right, because we can compare the six to the value that's stored at the root and see which side of the room it would need to be in, if it's there at all. So if the six is in this tree at all, which way would I have to go?

23:35

DS Lecture 11 Page 6 of 21 Transcribed by <https://otter.ai>

I love how you're all pointing in your screens. And some of you have your screens mirrored and some don't. So I just see people pointing at each other. And it makes no sense. So we have to say it out loud. You want to go to the right, okay? Because the six is larger. And so this actually lends itself to a recursive structure, you start by doing a contains check for the five on the root. And if it's larger than the roots, you recursively do a contains check on root dot right, and then you're starting at the subtree here. So now we go here, and this is our sort of, intermediate route. And we check to see if the six is there, and it's not. And so

we compare the six against that and we see that the six is smaller than that. So we recurse on the left subtree. And the root of the left subtree of this node actually contains the CICS. And so we found it and we can sort of propagate or return true, back up to the top. Sound good. All right. Now let's say I do a contains on something else. Let's say I do a contains on a three. Alright, now a three turns out isn't in here, but let's see how this contains check would go. We start at the root the three is not equal to the root, we compare and we realize it's smaller, so we recurse left, so we make another call to contains on the left subtree here. Okay, the root of that is at two, we compare the three to the two. Okay, that's not it. But it's larger than that. So then we can recurse to the right. And then we get to a node that has a four, the three is smaller than that. And then we recurse to the left. If this was the node we were currently at, if that was our current T, what is T dot left here? No. And since this is a recursion, that would be a good base case, right? If you're trying to search for a value on a subtree rooted at No, you should return false. So basically, what you're doing and it contains check is searching for the value. And one of two things is going to happen. You're either going to find it, or you're going to land on a no link, that should have been it. If it were there. Sound good? What would the big O cost? In the worst case B, for a contains check on a binary search tree? So a whole bunch of people just said the log of n with one person at the top saying n. So n is the number of items in the tree? Can you maybe give me the cost in terms of something other than the number of items in the tree? I'll tell you which one of you the logins are the ends is correct, for the worst case in a minute, but could you maybe give it to me. So it's not quite the depth. It's the height, and it's not the log of the height, it's really just the height, right? So if you guys recall, the height of a tree, right is the height of the root node, and the height of the root node is the length of the path to the furthest leaf. What is the furthest leaf that I would have? For me? That's the height. And in fact, in worst case, I would have to sew at each step, right, I move down a level in the tree. So I'm moving down a level in the tree and each step. So the worst case I could have would be to find out that I'm going to the furthest leaf. That's the longest I could possibly go. Now in this particular tree that I've drawn, this particular binary search tree is in fact, perfect. And if you had a perfect binary search tree, then the height is the log of the number of items. So if you have a perfectly balanced binary search tree, the height is the log of the number of items. But it doesn't have to be let me draw you another perfectly valid binary search tree.

DS Lecture 11 Page 7 of 21 Transcribed by <https://otter.ai>

28:23

Is there anything wrong with this? Has it violated any parts of the binary search tree condition? It is most definitely not. But I've kind of degenerated into a link list by building the worst possible one I can think of. So a binary search tree is not necessarily perfect. It's not necessarily balanced at all. In fact, in the worst case, you could have a valid binary search tree, that's essentially just the link list. And so in this case, yeah, the cost of doing contains check on this is the height of the tree in the worst case, but in this case, the height of the tree is actually in. So if I asked you for the worst case, cost of a contains check on a binary search tree, you know what it's still order in. Now, a couple of notes. First of all, it doesn't actually have to be exactly perfect for us to get kind of a log of in height for a tree for a binary tree. In fact, we have a definition of a quote unquote, balanced binary tree that says that as long as at every node, the height of the left and right subtrees don't differ by more than one. You can still say that the height is bounded by $\log n$. Now, in fact, it's actually two times $\log n$. If you do that. But that's a constant factor and drops away. So if I gave you a Balanced Binary Search Tree based on that definition, which we're going to get into a lot more detail on hopefully on Wednesday, then you would have something that's order $\log n$ for your contains check in the worst case. Now, the textbook will point out. We're not going to do big fate on this. But in fact, we actually kind of can. The textbook will point out that on average, a binary search tree

is on the order of $\log$ of n . And so that means that if you just sort of insert your items into a binary search tree randomly, on average, you can kind of expect the height to be on the order of $\log$ of n . But that's average case, not worst case. If we wanted the worst case cost to be bounded by $\log$ of n , we would need some way of ensuring that the tree is always balanced. And this kind of binary search tree does not do that for us. The next topic, we see the AVL trees actually will ensure that for us, okay. All right. So that's our contains. But there are other things that we might want to do. Like, for example, we might want to find the minimum item in a binary search trees. How would I find the minimum item in a binary search tree? Anybody? Yeah, just keep going left, until you can't go left anymore. Right. I shouldn't have said right there. Correct. So keep going to the left until you can't go left anymore. And that's guaranteed to be the smallest item in the tree. And so let me give you another example of kind of a worst case binary search tree.

32:14

Alright, that's equally bad. Is that one? Well, when you start at the root, you can't go left anymore. So that's got to be the smallest. Does everybody good with that? So that's our find me? How do I find the maximum? Just keep going right? Until you can't go right anymore. And that's the maximum. Good. Okay, next thing we need to be able to do. We need to be able to insert items into a binary search tree. And a lot of these operations are

DS Lecture 11 Page 8 of 21 Transcribed by <https://otter.ai>

always going to start with a contains check, right? So first of all, we're not supposed to have duplicates in a binary search tree, right? So you have to make sure that the value isn't in there first anyway. But if you think about it, what where do we land? In a contains check if the value we're searching for isn't in the tree? What do we land that? We land at a no. So if we're actually trying to insert the note that we land that is, in fact, what we want to replace, with the new value that we're trying to insert, it's a perfect spot to slot it in. So whenever we insert, our new node will always start its life out as a leaf. Because what we'll do is we'll search the tree for the value. If we find it, we stop and do nothing, because we don't want duplicates. And if we don't find it will land at a no. And that no, we're going to replace with the value we're inserting. So if we have an initially, and we have a tree that has some stuff in it already.

34:20

And I would say want to insert maybe a two. The first thing I would do is I would search for two.

34:32

Hold on one sec. Sorry, my camera was doing something weird there. The first thing I would do is I would search for two. So I started the three the two is smaller, so I go left. I'm now at the one the two is larger, so I go right there's a no that's where the two goes. And that's it. Okay. Now. So let's say we start with an initially empty binary search tree, the order in which I do the inserts will kind of determine what the tree looks like, because we're not going to adjust the tree later. So if I insert items, say,

35:33

let's go through and see how that would go. The tree initially is empty. So it's no, I insert the three, the root is no, so I'm just going to replace that root with the three. I insert the seven, go to the root, go, right? Because it's larger. There's a no, there's my seven. Now I go to the to start at the root, smaller go left, we got to know. Now we're at the four. Well, we start at the root, it's larger we go, right? It's smaller, we go left, now we have the know. And then we have the one. It's smaller, it's smaller, we have the No, so the one goes there. What's the worst way that I could possibly have inserted these things into a binary search tree? By inserted them in order, right? Now, first of all, we can't always decide what order they're going to go in, right? Um, we could find ourselves in a situation where the user is just feeding us data in the order that the user encounters the data. And therefore, well, you know what? That's it. We just got to go with what we're given. But if it so happens,

that the data is given to us in sorted order, that's pretty much as bad as it can go.

37:06

Okay, so that kind of worst case structure is going to depend on the order in which things are thrown in. Is everybody okay with how the inserts work on this? In this case, it looks like a linked list, but each node still has no references, right? Because each node has a left and right pointer. It's just happens on one of them. Only one of them is filled. Sound good? Right. Any other questions on inserts? All right, we've done a contains check. We've done a find min, we've done a fine Max, we've done an insert What am I avoiding here? What else do we need to do? We got to do a remove. Okay, you know what? I'm going to avoid remove for one more moment. Let me ask you a question. Let's say I wanted to print this tree out. And let's say I printed it out using an inorder traversal. What would I get? Maybe it would be sorted. Right? So if you did an inorder traversal of this tree, you'd end up with the values in the tree in sorted order. Is everybody good with that? Alright, so we've avoided remove for long enough, we must now remove. So just like with insert, we start with a contains, right? Because if the value isn't there, there's nothing to remove. So let's create this following tree.

39:25

I kind of backed myself into a corner here. Alright, fine. That's an integer. All right, so we start with the following tree. So a remove is always going to start with a contains check just to make sure it's there. So when we remove a node, there are basically I want to say three cases that we have to worry about. Now, the simplest case to deal with is let's say I want to remove a node that has tends to be a leaf. Right? What happens if I try to remove a leaf?

40:14

You just replace it with a no. Right? So you have to, you know, it returns back to its parent. And whatever the link was that pointed to it, you replace it with a no. So if we're removing a leaf, it's simple. Just replace it with a No. What's the second simplest case?

40:40

Okay, not there is, I would argue is the simple as simple as case. But let's, what's the second simplest guess? If it's there? remove a node with one child, right? Because if you

DS Lecture 11 Page 10 of 21 Transcribed by <https://otter.ai>

have only one child, what can you be replaced with? The child, right? everything below you still has to sort of fit within the greater structure of the tree, right. So if I'm removing the one, any of the children or grandchildren, or any of the descendants of the one still have to be smaller than the two. So anything that I choose from down here will be correct in terms of the two. So if I only have one child, I just pull that child up. And that solves the problem. And in fact, let's imagine that that one child has children of its own. That's a minus two. So let's say I did a remove on a one, I can still pull the zero up along with everything below the zero and it would be fine. Is everybody good with that? Good. So the simplest cases are if you have no children, you just replace yourself with a no, if you have one child, you replace yourself with that one child. The harder case is if you have two children. All right, so let's say we decided we want to remove v eight. So let's say Our target is the eight, and we decide we want to remove the eight. There are actually two nodes below the eight that I could realistically replace the eight with which two nodes Could I realistically replace the eight with without damaging anything? Anyway, so the nine would work, I can put the nine here, the nine is still larger than the four. The Nine is by definition smaller than the 10. And also, by definition larger than the sevens Oh, we're good. So the nine would work. Also, the seven would work, because the seven is smaller than everything on this side. So in fact, you have two choices. If you have two children,

43:31

you can either choose the largest node on the left subtree. So the largest note on the left

subtree, by definition is larger than everything else on the left subtree. So it can become their parent. But because it's on the left subtree is by definition smaller than everything on the right subtree. So the largest node on the left subtree can then be removed and can replace the node we're trying to remove. Alternatively, we can take the smallest node on the right subtree. Because if we take the smallest node on the right subtree that is by definition larger than everything on the left. And because it's the smallest node on the right will work because it is smaller than everything else on the right. Would this always work when there are two node choices on the left side make right child parent on the right side make left child parent No, it wouldn't, because there could be a lot more stuff down below. Does that make sense? examiner? So, for example, um, let's see.

45:28

If imagine there was like, point five over here, and we decided to go with the right subtree, we would actually have to use the minus one. So we'd have to go one lower if we were going to remove the zero. Does that make sense? Okay, good. All right. Now, it doesn't matter which one you do. In general, you should just always be consistent. All right. Um, I believe the textbooks code just always takes the largest node on the right. Sorry, the smallest node on the right. So it usually goes with this one. Right. Any questions on the Remove?

46:07

We're good with this. Okay.

46:12

Now, what we're going to do now is we're going to talk about and all. So what about removing the root? It's the same rule, right? If the root has no children, the whole tree becomes No, if the root has only one child, that becomes the root. And it's the same thing, largest thing on the left or smallest thing on the right. If I decided I wanted to remove the for the largest thing on the left is the three and that would work just fine. And the smallest thing on the right is the seven and that would work just fine. So it actually isn't a special case, which is a really nice thing about this is actually why a lot of this stuff works recursively you know, you just do the thing on the root. And then if it's not happening at the root, you just do the exact same thing on the subtree and all is good. Okay. All right. Other questions on remove? Okay, so there's another thing we could do. Hold on a second. Sure.

47:30

So this isn't something that we usually do with binary trees. But I like to teach it here as sort of your first exposure to it, because we're going to see a similar concept when we do hash tables in about in about two lectures. And this is something known as lazy deletion. And so lazy deletion is another alternative that one could do if you want to remove something, which is basically modify the nodes structure. So that in addition to storing the value, and the links to the left and right subtrees, you can also have like a flag, deleted, true or false, or valid, true or false. Let's go with valid, true or false. So if valid is true, then the node is considered real, it's considered to actually be there. If valid is false, then the node is considered as deleted, meaning you could actually leave the nodes there, just mark them as deleted. And that still retains the structure of the tree. You would not want that to be static, you would actually want that to be an instance variable of the node, because each node would have that value associated with it. So if we were going to use lazy deletion, let's say we have a tree like this. So if I wanted to say, right, so if a node is not valid, and checking contains would have to return false. Let's say I wanted to delete the nine. In a lazy deletion, I would search for the nine. And when I find it, I would simply

DS Lecture 11 Page 12 of 21 Transcribed by <https://otter.ai>

flip the tag so it would no longer be valid. So I'll put an X there to indicate that it's no longer valid. And so now we have to modify our contains check a little bit, right. So if we

were going to do a contains check and we're going to check for a nine, we'd search for the nine, we'd find the nine but then before we return true or false, we have to check whether or not The Nine is valid. So now that we found it, if it's invalid, we're still returning false. And you don't have to go any further because it's only going to appear in the tree once, whether it is valid or not. Now, if I'm going to do an insert for a value, say I insert nine, before it was marked as deleted, insert on nine would do nothing, right? Because it would just, it would find that the nine is already there and just go. But in this case, when we have the lazy deletion, if we try to insert nine, first we search for it, we find it and then we check whether it is valid or not. If it's valid, then there is still nothing to do. But if it's invalid, what would I do? Just flip it back to being valid, right? Flip the false back to true, and we're done. So you just turn it back on. And so this sounds really nice. Our inserts and our contains really haven't changed that much beyond having to check whether or not the node is valid or not before actually doing our thing. Um, but our removes are simple, because you don't have to restructure the tree when we remove something we just mark it. But sincerely his comment is valid. So let's slow down on the questions. Let me just work through. First of all, Are there additional time costs, so the size of the tree doesn't decrease even when you remove items, right? So you still have to pass through them. So the order and cost of the tree, our operations is and including everything that's been easily deleted. So if you delete everything, that cost of having them in there is still there. So that's not optimal.

51:57

And Omar says, but don't you need to restructure the eight and the 10? when changing to false? In fact, no, that's the beauty of it. Because you end up with a situation where the eight and the 10 are still correctly located with respect to the nine. And when you do a search operation, you ignore the valid or invalid. So if I'm searching for the eight, I started the route eight is larger than the seven, I then land on a nine, I don't care that it's invalid. I just say, Well, this is a nine. That's not what I'm looking for. What I'm looking for is less than the nine. So I move left. And I just scroll right over it over here. Does that make sense? Cool. So I says Why can't you just change the sevens reference to not reference to nine to be no? Well, wouldn't that then just lop off the eight and the 10? Because we have to pass through the nine to get to the eight and the 10. Does that make sense? I think that's what the follow up also noted. Right? But then we're right back to the original kind of remove that we were doing before. So this is meant as an alternative to that. Okay, cool. All right. So so far, everything looks beautiful. There were a couple of methods that we discussed in the previous parts of this lecture on binary search trees that I haven't really talked about yet in terms of the lazy deletion, what were they? What were the other methods? Ah, the DS Lecture 11 Page 13 of 21 Transcribed by <https://otter.ai>

find min in the find max. And that's where we get into a little bit of trouble. So find min was nice, because we just kept going left until we can't go left anymore. And we're done. Fine. Max was keep going right until we can't go right anymore. And then we're done. Imagine I deleted the one. Now when I do find, min, it's gonna keep going left till it can't go left anymore. But then we realize that the minimum we landed on the minimum we landed on was no. Right? So now we actually have to say that four is the minimum. So we have to backtrack. That's not so terrible. But the antagonist strikes again and deletes the four. So now, not only does backtracking not solve the problem, we actually have to go right to get the minimum value in here. The antagonist strikes again. So we go left all the way out. Okay, that's deleted. Great. So we'll come back up here. Wait, you're deleted too. Now I have to check to the right. Oh, that's all deleted. Now. What do I do? Okay, now it's the root of all thing. Super, you could have told me that to start with thanks very much lazy deletion. Okay, but Now it's got the answer. Let's make it even worse. And I could keep playing this game until you got this big old tree but the biggest item in the thing is the only one left and that's what the smallest as to return.

55:16

So,

55:18

find min and similarly find max go from being a simple just keep going left or keep going right to suddenly it becomes a tree traversal. And a tricky one that because you have to decide whether or not you need to traverse more or not. Did I leave that in in the homework to do the lazy deletion for Fineman forgetting if I removed that I think it's one of the written questions in there. So enjoy. You have to make the recursion work for that. I will tell you it does end up being a traversal. So that that that should help some. So somebody said store every new min every step down if it's invalid, skip if not change it. I didn't consider if all the sides on the left were canceled. Yeah, I can. Yeah, I can counter that at every step. So it becomes a little more complicated. Does everybody see why lazy deletion can cause some problems here? Yeah. Okay. So at this point, I believe we've actually gone through all of the major components of a binary search tree. So we're really in quite good shape. So since we still have a good 15 minutes left, oh, no, we're not. We're not ending. No. And we're not even going to do the Tammy Orman's story yet. No, not bad either. I think it would be best if we start going over the code now, we really should push forward. So give me 30 seconds to set up for that. And we'll take a look at the code for this. Now, I felt like I sounded fairly excited when I said that, and I have a feeling most of you were decidedly not. So give me one more second, let me stop this here.

DS Lecture 11 Page 14 of 21 Transcribed by <https://otter.ai>

57:31

Terminal open.

58:02

Alright, so let me share my screen again. And you should all be seeing my terminal now. Correct. Good. So what we're looking at here is sorry. Okay, guys, what we're looking at here is from the Weiss code, you can pull this code down from the textbook code, and we're looking specifically at a file called binary search tree. All right. And so this is an implementation of that set, the binary search tree set that we had before. Alright, so um, somebody is like, super psyched for green terminal. I don't, I don't know how to read that. Alright, so we have our binary search tree, we're going to have our insert, we're going to have our remove our contains find min, find max. And we'll have an is empty make ft and that print tree, which will actually just be an inorder traversal. That'll give us the values back out in sorted form. Now the first thing we have to look at here, and this is actually probably something that might make your eyes cross initially is our generic. Okay, so what we have here is our public class binary search tree. It is generic, but like we did with our binary search way back at the beginning, we need to constrain our type variable to be only things that descend from comparable, okay, so whatever this thing is, inside the angle brackets we say extends comparable because it's just a type, but in practice needs to implement comparable. Now here's where things get a little messier. What we've been used to seeing is that you want to be implementing comparable on yourself, right? Because you want to be comparable against yourself. And so you would have expected here to see any type extends comparable against any type, because you want it to be comparable against itself. But instead, you see this nasty little question mark, super any type. Now, let me just briefly describe what's going on here. What this is saying is that your any type needs to be comparable to anything that is a superclass of itself, not just itself. But anything that's a superclass of itself. And that actually is kind of important. Let me give you an example where that might matter. So when you did inheritance back in 2004, a professor cannon, did he use bank accounts as the example for inheritance? Right. So what you had done was you had created a class called bank account. And then you created subclasses of bank account, called savings account and checking account, which had sort of the basic functionality of a bank account that it inherited, but added

additional functionality associated with the particular type of account that you had. Now, imagine that we implemented that we had our bank account class implement comparable, okay, so let's say bank account implements comparable against bank account, which is a perfectly reasonable thing, to compare one bank account against another, it's like, well, one has more money in it than the other. That's a very straightforward thing. And it's pretty much common to bank accounts is their balance.

DS Lecture 11 Page 15 of 21 Transcribed by <https://otter.ai>

Now imagine that you say checking account extends bank account. And you just said that, when you say checking account extends bank account, it inherits the fact that bank account is comparable against itself. So your checking account is actually not comparable against checking account. your checking account is comparable against what? It's comparable against bank account. Right? So in that example, if bank accounts were comparable against itself, checking account and savings account, are both comparable against bank account, which means they can all compare against each other.

1:02:57

But if I had made my binary search tree, be any type extends comparable against just any type, it would require them to be comparable against to the exact same type. So I wouldn't be able to insert checking accounts can examine the question mark. Second, hold on. So savings accounts and checking accounts would not actually work with this, if I didn't have this part. Because savings accounts and checking accounts don't implement comparable against themselves, they implement comparable against their superclass. So by using this notation, those savings accounts and checking accounts could interchangeably be inserted into this. Is everybody okay with that? Bo, go ahead now

1:03:48

are using the question mark and set him to compare to

1:03:52

know the question mark is a wild card, it matches whatever class was, is actually as comparable against, and so whatever class any type is comparable against, it's saying whatever that is, whatever for

1:04:07

that question mark can make reference to any class. Yes, it's that implements the comparable or just any class

1:04:18

standard class at all, as in this case, as long as it's a superclass of any type. Do I have to

DS Lecture 11 Page 16 of 21 Transcribed by <https://otter.ai>

use that notation? In this? Yes, you absolutely have to use this notation. No choice. All right. All right. Um, so Alexander says, Why don't we do this all the time, then? You know what? Every time we're going to use comparables in our data structures from this moment forward, we will. So have we used inheritance before today. I don't remember I've not actually needed to use inheritance before in this class, but I assume that you've seen it, because it started an intro. I've only, okay. Alright, so that's what's going on here? Um, Hannah, could you ask that question of the TA and let them re type it? Alright, so that's what we have here. And now, let's just take a very quick look at what the binary search tree class itself has, in terms of instance variables. The question mark is what's known as a wildcard. So we actually only have a single instance variable in this thing. And it is the root of the tree. Its type is binary node. Okay, now binary node, just like we did with linked lists, is actually a private static class of the binary search tree. Alright, so the binary node is a private static class of the binary search tree that has the element that's the data, which will be typed as any type, and then the left and right subtrees, which are just references to additional binary nodes, you'll notice that none of these are marked as private. So that means the binary search tree class will be able to access and manipulate

those fields directly, which I think is a very useful thing. Okay, we have a constructor just in case, a one our constructor just takes in the data and assumes that otherwise, it's a leaf, and a three, our constructor, which takes in the data as well as the left and right subtrees that you want. Okay. Why with a constructor, there is no public or private indication. Um, so you can leave the public and private off. And that sort of becomes a default visibility, which is very similar to public. So it doesn't really need to be specified as public explicitly because nothing outside of binary search tree could even instantiate these things. person who asked me that directly. Are you good with that?

1:07:41

All right. Okay, so um, I think I'm gonna stop in terms of going through this class a little more. And so now that we have basic structures, I'll post a supplemental lecture that actually goes through the code, and then kind of pick up from there and start talking about balanced binary search trees and avielle trees on Wednesday. Sound good? Alright, so we got five minutes left. Um, I was going to tell you a very quick story about sets. And I guess I called it the Tammy Orman story. So we're at a point where this is not relevant. Beyond just an amusing use case of sets and unique keys. So if you want to head out, you can feel free. So what I'm going to tell you here is this, were part of the story of my wife and my honey. So we were married about 10 years ago. And we, we went to Seattle of all places, you might think, Why on earth, would we go to Seattle? And the answer is, well, a we'd never been there. And B, we were actually taking a cruise that left from there to go up to Alaska. But we were still in Seattle for a week. So we rented a car to kind of drive around a bit seedy area. So we get to the airport, from the plane, and we go and we rent

DS Lecture 11 Page 17 of 21 Transcribed by <https://otter.ai>

the car. But Blair

1:09:27

will need a time reference here was this in the last 10 years or 10 years

1:09:31

ago? Alright. So I go there, and this was Alamo rentacar. I'm going to tell you the name of it because I'm still mad at them. And so I rented this car from Alamo rental car and they go up to the front and they give me this contract. I say okay, I looked at the contract in the very, very top. It said Tammy Orman, and I looked at the woman who was handing me the contract and I assume They just at the very top, they printed the name of the person who I was working with. midway down, I did see my name because I had given them my credit card. And at the bottom, I saw my wife's name, which, you know, made sense because she was going to be an authorized driver. So I take this contract. And then I go to the place where I'm supposed to pick up the car. And so I show the contract to the guy there, it looks at it, scratches his head, looks at me, looks at it looks at me, gives it back to me, gives me the key to the car, I get in the car, I start driving the car. Now, I'm talking to a bunch of teenagers, I have no idea if any of you've ever actually rented a car, I have no idea how many of you have ever driven a car. But whatever. If you ever happen to rent a car in an airport, if any of you ever happened to get a chance to fly again, if you rent a car at an airport, as you're driving, to get out of the airport, there's like this big security gate where they check everything to make sure you were supposed to be leaving with the car. So I give my driver's license and the contract to the guy at this big security gate, making sure that everything is on the up and up, he looks at the driver's license, looks at the contract looks at me. And then does repeat of that circle, looking at all these three things, two or three times shrugs, hands it all back to me opens the gate and we go on our way. At this point, now we have this car. And we're driving around the Seattle area at one point, we actually decided we were going to pop up to Vancouver. So we actually took it out of the country. I had it in Canada for a little while. And we come back from Canada and we're driving around. And as we were driving around, there was actually what I would describe as probably one of the more romantic moments. My wife and I shared on this

vacation when a pickup truck decided it wanted to be in the same physical space as we did and tried to merge into me, barely, just barely missing us. So we nearly get killed by this pickup truck. And I react the way that I normally react. And I you know, gave this person the finger. And the truly romantic part is that so did my wife. And we hadn't planned it at all, it was just this sort of simultaneous giving this person in the in the truck the fingers. It was it was it was a lovely moment. Anyway, so we nearly get killed by a

DS Lecture 11 Page 18 of 21 Transcribed by <https://otter.ai>

truck. We take the car out of the country. And finally this part of the trip is over. And we need to get the car back so that we can get on our cruise. So go back to the airport, and I drive in through the security gate and the guy comes up to check the car and I give him the contract and he looks at me, looks at my wife and then walks over to my wife and said you're Tammy, right. And she's like, No, I'm not Tammy. And so he comes back to me. He looks at me somewhat incredulously and says, You're not Tammy, are you? I say no, no, I'm not Tammy. And he sighs and he says and I will never forget what he said. He said not again.

1:13:29

So this has happened before. So he takes me back in when we talked to the manager. And it turns out, of course, that Tammy Orman at the top was not the name of the person who was running the kiosk, the Tammy Orman the name at the top was the person to whom they had actually rented the car. Now, let's come back to first of all, I went through something like four layers of security who physically looked at this mistake. And just let me go. So that was an interesting thing. But now let's tie this back into computer science Challis. And we were talking about keys, and how keys need to be unique when you have a map. And so if you have a database system, the expectation is that you would use a unique identifier, a unique key into the system to pull up the record for the person in your system. That's the value. And then the key is this unique identifier. Now, just as a fun little side note, I'm at Columbia, what do you think we used to use is that unique identifier that key like cu ID or your uni? federal sounds nice. But I've been here for a very long time. And 24 years ago, as Matt pointed out, he was your social security number. And so if you came to Columbia in the 90s, they would often ask you to write your social security number at the time. top of your paper before handing it in. Not kidding. People used to be asked to put their social security number so we could ID you in the comments at the top of their code. I was a TA, I had, oh, I think most people had a tin at that point. Anyway, I'll go with card number for card number ID. How about that? Well, that would be smart. But we were using social security numbers. So actually, I have a lot of people's social security numbers, I have probably hundreds of students social security numbers buried in my email somewhere from when I was a TA in the 90s. Sorry, side note, back to the unique key at Alamo rentacar. If you happen to have a driver's license, and you look at the front of the driver's license, there is an ID number on it that looks a lot like a social security number on it. And so they use your driver's license ID number as the unique key. Except there's a problem. Unlike social security numbers, driver's ID numbers are not unique. They are unique within the state, or the province, the Canadian driver's license and us driver's license follow the same pattern. There. It's a sort of a unified back end system. And so the unique identifier is the combination of the driver's license ID and the state. And so I had a New York license with a driver's ID. And so she dutifully typed the driver's ID in I had never

DS Lecture 11 Page 19 of 21 Transcribed by <https://otter.ai>

been to Alamo before. And so when she typed it in without the state, the system happily popped up whatever record happened to have that ID number, which happened to be Miss Tammy Orman from Alberta, Canada. And she just kept pressing Enter and rented the car to her. So you got to make sure that when you are going to have a key into a system, it is unique. And so otherwise, you're going to run into some stupid problems like this, like

1:17:05

a car thief or something like that. Like, did you get in trouble? Did they like pull out like, you know, ar 15 assault rifles? And were they like, Alright, step out of the vehicle, because she was like, you know, Bonnie and Clyde type person? Or is it just,

1:17:19

you just have the same unique ideas, or I just had the same unique ideas, or I mean, I did look her up on Facebook, she seems to be a fairly religious, Christian Canada, at which point I decided I am not going to message her in any way shape, or form or tell her that this happened. But no. I mean, technically, I wasn't entitled to be driving the car. I don't know what would have happened. If we had actually gotten to an accident. It's terrible

1:17:48

parking tickets that maybe you didn't tell Tammy about? No, I

1:17:52

paid for all my parking. They had I known I wouldn't have but I did. Alright. So there's my little story. It ties in Well, I think with the unique keys, and actually in a particularly nasty element of it is that in Java, if you use a map, like a tree map, which uses these kind of binary search trees in the back, if you insert an item under a key that already has an item in it, it'll just overwrite it. So if they were using the map, and they were trying to insert me into the system at Alamo rentacar, they would have removed Tammy and overrode her with me. So you have to be careful about these things. All right, folks, I've talked too long and I have a TA meeting to go to have a good one.

1:18:34

DS Lecture 11 Page 20 of 21 Transcribed by <https://otter.ai>

And I tell Emily that we that we spoke and that I was supposed to speak with her after midterms after the break, please. Sure thing. Thank you. Alright folks, take

1:18:45

it easy. Thank you. Thank you.

DS Lecture 11 Page 21 of 21 Transcribed by <https://otter.ai>

DS Lecture 12

Wed, 8/11 1:31PM 1:23:03

SUMMARY KEYWORDS

binary search tree, height, tree, node, rotation, root, left, insert, left subtree, imbalance, valid, balance, subtree, abl, good, triangle, call, condition, question, draw

03:53

come from

03:59

to breathe

08:53

Alright let's get started. Um So I ran into a small issue in that as I was going through the seven or eight midterm part twos that required attention that occurs to me I have to regrade seven or eight midterm part twos not because they were graded incorrectly but because they were submission issues. And so what I'm probably going to do is I'm going to just send an email to the eight or so people that I need to fix their grades on so that I can stop stalling and just release everybody else's grades. And so that should get done this evening at the very least everybody the vast majority of people's grades. Um, so that's that. How is our homework going? That's due towards the end of next week, correct? Yes, any questions on that?

10:04

Yes, no. I don't know. I always think the coding will take longer.

DS Lecture 12 Page 1 of 19 Transcribed by <https://otter.ai>

10:22

I do think the comment that one person made about civil balance being worse than all of homework three has some truth to it. I think simple balance is probably has the most nitpicky aspects to it, though, honestly, I think the expression tree one, even though I did in lecture give you a lot of pieces to put stuff together does involve integrating, I think, somewhat more complicated concepts than the simple balance. So yeah, I think the programming would likely take a bit more time. The induction proofs on the written can sometimes take a little bit of time to wrap your head around, particularly, there's one that's a summation of things related to the levels of the leaves, that people tend to find to be tricky. Now, I've had a conversation already with probably about a half dozen students about that problem, including last week's office hours, and this week's office hours, which led me to making a comment about that problem that I made almost universally to everyone who asked me about it. And it occurs to me that I should probably make that comment to the class as a whole. The what you need to do for that problem. First, before you make any attempt to trying to do an induction proof on the on the property is to draw a tree and calculate the summation for that tree. Then take a leaf away, and recalculate the summation. Then add 123 more leaves to that tree, each time recalculating the summation. And I think once you actually go through, draw it out and do the calculations on a variety of trees as you add and remove leaves, that's really going to ram home what it's actually doing the summation. And that will help you articulate the sort of multiple cases that you'll need to address in your induction step. Further, the second part of that problem is asking you When is it not an inequality, but when is the summation equal to exactly one. And as you draw it out the way I described, I'm not asking you to prove that part. As you draw it out the way I'm describing here, you will very quickly see when it adds up to one. So I think the first thing you need to do is forget about the proof aspect of it and just draw it out so you understand what the property actually is. And everything should follow from that sound good? All right. Any other questions? Okay. So last class, we did binary search trees, we went through all of the various methods that you should have on a binary search tree and kind of describe them and how you would implement them sort of in an abstract sense. There'll be a supplemental video, I think it's actually processing now, that will have me actually going through the code for that you can watch that at your leisure. But today, what I said we were going to do is we're going to talk about getting a binary search tree that automatically balances itself. What we had said,

14:19

was that with our plain old binary search trees, if I don't make any comment about whether or not the tree is balanced, and there's no reason why it has to be and in fact, we

DS Lecture 12 Page 2 of 19 Transcribed by <https://otter.ai>

drew an example of it. When we added items in order to the binary search tree where it was about as far from balanced as you could possibly imagine. That in the worst case, those operations on the binary search trees are actually order n operations. In the average case, banned up being log event but not the worst case. avielle Though that gets us a worst case cost of log event, and we need to talk about how. So first of all, when I say that we have an avielle tree, I'm avielle, by the way stands for allisonville. ski and Landis. It's just the names of the two people, I think the first one's a hyphenated name, who came up with the concept of an N of these by balancing binary search trees, you don't really have to remember that. But an avielle tree has two properties to it. The first property is that it is a binary search tree. So any avielle tree is a binary search tree. So every single node in an abl tree must adhere to the binary search tree condition, everything to the left

of a node must be smaller, everything to the right of a node must be larger. Okay. So that's property one and an AVL tree is a binary search tree. So the thing like it contains check on an AVL tree is going to work the same way as it contains check on a binary search tree. But then there's a second condition that must hold true at every node in an AVL tree. It is sometimes referred to as the AVL condition, but it's probably better to call it the balance condition. Now, I'm not going to prove this formally, but you can look in the book for the formal proof. But the balance condition says that at every node,

16:50

the height

16:56

of that node's left subtree can differ from the height of the right subtree by no more than one. Alright, so the heights of the two subtrees of a node, any node can differ by no more than one. So that's not exactly a perfect binary tree, there are going to be some little imbalances. And in each, each node, they can differ by more than one. And in practice that in practice, if you proved it, what you would actually show is that in the worst case, the height of that tree would actually not be $\log n$, but it would be essentially two times the $\log$ of n . But in terms of Big O, that's still $O(\log n)$, it's only a constant factor off. So this somewhat looser balance condition than the tree actually being perfect or complete, still gives us an order $\log$ of n in height. And so as long as we enforce this condition at all times, we are guaranteed that all of those operations that are on the order of the height of the binary search tree, even in the worst case, will not be any worse than $O(\log n)$. Is everybody okay with that. Now, we don't take an unbalanced binary search tree and then fix it so that it becomes a balanced one. The idea is that we actually always maintain the balance condition. So whenever we insert or remove an item from an

DS Lecture 12 Page 3 of 19 Transcribed by <https://otter.ai>

AVL tree, we have to ensure that the balance condition is always upheld when it's over. And so that means whenever we start an insert, we know that the original tree that we're inserting on is already a valid AVL tree. And then that single insert could potentially inject a small imbalance into it, which we will fix by one or more rotations as the recursion closes back up towards the root. But those rotations are going to be constant time operations that maintain the balance so it doesn't change the cost. Okay. And so that's going to be the thrust of today's lecture is learning how we're going to do those rotations. So that we can maintain that balance. Now, before we get into that, though, I think we need to just make sure we fully understand what we mean by the AVL condition by the balance condition. So let me draw some sample trees and let's talk about them. Okay. So let's start with the following. Okay, that is a valid binary search tree, correct a single node? That's, that's a fine binary search tree. And it's also a valid AVL. Because remember, what we're doing is we're comparing the heights of the left subtree and the right subtree. What are the left and right subtrees? In this particular example? There no. What is the height of this node? The five? It's zero. So the height of the left subtree is not zero, right? Because the left subtree is just no. So we're going to need kind of a stand in for the height of a no tree. If the height of a single node is zero, what would be a good standing height for a no tree? Yeah, negative one will work for us here. So the left subtree is no, so we're just saying that the height of the left subtree is negative one. And the right subtree is null. So the height of the right subtree is also negative one. They don't differ at all. So we have a valid AVL. So everybody good with that.

21:42

Alright, let's add another node to this. Now remember, the AVL condition, this balance condition, just like the BST condition, must hold true at every node. So let's say that now we have the following binary search tree, sorry, AVL tree, this is definitely still a valid AVL tree because the three is smaller, and I put it to the left. Good. And so we got to make sure that all of our nodes are valid AVL. So it actually makes sense to work from

the bottom up. Because remember, we're inserting things at the bottom right? inserts end up happening, it leaves each time

22:26

because it's one less than a single node. Right? So if I just had a single node like this,

22:35

that has a height of zero. So if there's one fewer levels than this, then that would sort of

DS Lecture 12 Page 4 of 19 Transcribed by <https://otter.ai>

make sense to be one less level. Is that good? Cool. All right. So now, looking at this bottom level three, what's the height of the three's left subtree? by this one, because it's not in the height of three, right? subtree? thing? So they're equal? So the three is good. Now let's go to the five. What is the left subtree of the five? First of all, so that we're clear on that? It's the three just the three. So what is the height of that left? subtree? height is zero, correct? Because remember, the height of a node is or a height of a tree is you take the root of that tree, and then you find the length of the path to the furthest leaf. The root of this subtree is the three. And so it is the furthest leaf from it anyway. So the length of that path is zero. Remember, we're asking for the height of the subtree. The height of the whole tree is definitely one. But I'm asking for the tree rooted at the three. So that's as a height of zero. So this is a height of zero. the right subtree of the five is a no so that has a height of negative one. And so the heights of the two sub trees do differ, which visually makes sense. One has a node there one doesn't. And they differ by exactly one. Is that okay in my avielle condition. Yeah, so we're good. So that's still a valid avielle. So the avielle condition constrains the height of the tree, so that the height of the tree can never be greater than on the order of log of n. So when I do inserts and contains checks, I can guarantee that the big O costs of those are never ever gonna be worse than big O of log of n? Whereas in our unconstrained binary search tree, the worst case, those operations could be order n. Miranda, we're good, cool. All right. So don't think of it in terms of Big O and big theta. Think of it in terms of average case analysis versus worst case analysis. On the worst case analysis are different in an AVL in a BST. Good. Alright. So we're good. That's a valid API. Let's Let's add another node to it, but I'll actually draw it separately.

25:46

So this is still a valid binary search tree. Now to check to make sure we're a valid avielle, we want to work our way back up from the point of insert and check our heights. So this node, it's left and right sub trees are both No. So it's left subtree is a height of minus one and it's right subtree has a height of minus one. So that is a valid avielle node. Moving back up through as the recursion returns, the three's left sub tree is just the one that has a height of zero, the three's, right subtree as a height of negative one. And so the difference in the heights are just one. So the three is a valid avielle. But now we get to the five, what is the height of the left subtree of the five? It's one, because it's the tree rooted at the three and there is actually a single step to get relief. So the height there is one and the height of the right sub tree of the five is negative one. So we have an imbalance at the five because the heights of the two sub trees differ by two now. Are we good? So that's not a valid avielle. Right? being balanced doesn't necessarily have to happen at the root. Let me draw another example.

DS Lecture 12 Page 5 of 19 Transcribed by <https://otter.ai>

27:33

All right, so

27:41

Okay, so let's say that is our tree. All right. So let's check. This is definitely a valid BST. But let's check our, our heights of the left and right subtrees at every node. So this one is negative one and negative one. So this node is fine. This node is zero and negative one. So that node is totally fine, right? Now the root node, what's the height of the left subtree of the root node? To correct, you're starting at the tree rooted at the three for the left

subtree. So it's not three, but you take two steps from the root of the left subtree. So the height of the left side is two. And what's the height of the right side? Maybe it's one, because we're rooted at the seven. So the left and the right sub trees differ by only one. So the root of the entire tree actually is a valid AVL node. Now let's check the other side, negative one and negative one. This one's zero and negative one. But now here, it's a three, we might have a slight problem. The height of the left subtree is one, but the height of the right subtree is negative one. So the imbalance which makes this an invalid AVL actually happens not necessarily at the root of the whole tree. And so you can have imbalances at different levels. So you can imagine. I'm not sure what the 513 is supposed to mean. You could imagine that this one weren't here. If this one wasn't here, this tree without the one would actually be a valid AVL. Yeah. Oh, I see what you mean. So it'd be 5321, I think would be the worst case there. Sorry. Anyway, um, so if the one wasn't here, the left and right subtrees would have the same height, and this would be a valid AVL tree. So then if I did an insert of the one, this is where it would go initially, right? Because you start at the root, and you just end up here. When we insert that one, you have to check for the imbalances as you work your way back, and as you can see, I can inject an imbalance actually at a slightly higher spot at slightly lower spot than the root. Is everybody okay, with the heights? And what the potential for imbalances? Yes. Questions. Hey, so what would happen, instead of making the heights of the sub trees differ by one we made the sizes, number of nodes of the sub trees differ by one with a height of a tree still be order log of n? No, because imagine that. So if I want to build a tree with n nodes in it, depending on the order in which they're in, I could make it a linked list, that would have a height of n, or I could make it perfectly balanced. And that would have a height of log n. So you could imagine a scenario where the left subtree and the right subtree could have exactly the same number of nodes. But one could have a height of n and the other could have a height of log of n. So it would definitely not be balanced, either. Does that make sense? Cool. All right, moving on down. Okay, why would the height of the root of the subtrees be two in one, would it be three and two. So remember, the height is going from the root of the subtree? down to could we slow down with the comments for a

DS Lecture 12 Page 6 of 19 Transcribed by <https://otter.ai>

second so that I can finish? And then we can pick up in a second. Um, so it's going from the root of the subtree, down to the furthest leaf, but it's the length of the path, and the path length counts, just the edges you would traverse. So if I'm starting here, getting to the furthest leaf, I would traverse just two edges.

32:32

So to the person who asked me that privately, does that make sense? Excellent. Good. All right. The next question was, could you play again with the head of the tree? Oh, this next question was exactly the same. Sorry, two people asked the same question didn't realize good, we're in good shape. Um, okay. And 11 to the 531. search tree is AVL. I know some, I don't know what you mean by the 531. search tree? Oh, you mean this one? Oh, sorry. I got it. And so if you were to add an 11 to this one, then it would actually cease to be a problem. Because once you add the 11 in here, then the heights of the two sub trees differ by only one, right? That would change the situation. Good. Okay. All right. So this next person said to be a valid AVL tree, should every node in the tree be a valid AVL node? Yes. every node must be a valid AVL. Okay, I think I've caught up on the backlog of questions. Anything else? All right. Now, the next thing we need to talk about is how do we keep track of these heights. And so in a regular binary tree, which we've discussed already multiple times. And in fact, when you look at the code for the binary search tree, which you saw on your text, and there'll be some supplemental discussions on, you'll note that the nodes for a binary search tree will just have your data, a link to the left subtree and a link to the right subtree. And that would be it. Right? So if that was all you had, how do I quickly query the height of a sub tree, you wouldn't be able

to you'd actually have to foot calculate the height. And calculating the height actually would involve something of a tree traversal to find the deepest leaf. And so that's not optimal for this to really work with Well, we need to be able to, in constant time, query the height of a given node. And the only way we're ever going to be able to cleanly do that is if we augment our code. So that if every node Yes, in every node has in addition to its data, a link to its left subtree, and a link to its right subtree, it also keeps track of its height. So when you insert a node, it's a leaf. And so you know, the height is zero. But then as you insert more nodes, and the heights of other nodes change, as the recursion closes back up towards the root, you can reassign the heights as you go and keep them updated at no additional real big O costs. So as long as you keep track of the height, set every node and update them, whenever those heights change, you can trivially query the height of a node, just by looking at one of its instance variables. Is everybody okay with that? So somebody is saying no, but you're gonna need to be a little more specific as to what's wrong with it.

DS Lecture 12 Page 7 of 19 Transcribed by <https://otter.ai>

36:18

So can you say that one more time, so you're storing the height and each node,

36:24

right, so there would be an extra little variable in each one. So like, I would store a little zero here, I would store a little one here, and so on. And so if I were going to say insert something into this tree, right, you would recursively insert left, then recursively, insert left again, if it were, say, a one.

36:46

And now you update the root nodes height, and then

36:49

as the recursion closes back up, you can very quickly recalculate? Oh, this guy's height has now increased by one. And so you can update

36:59

it. I understand. No, thank you. Good. Cool. Other quick, Kareem, go ahead.

37:07

So professor, I was thinking of, you know, if we have an avielle tree by the, you know, by inserting another element, like we have 11 elements, and we add a 12th. To kind of counter that we would rebalance it by changing, you know, by making one of the the node that is income intention, let's say to, I think to the right or to the left, and then recalculate everything. But when we add the variable, the heights, wouldn't that need to change as well?

37:45

So the, you're correct, it does when you do the rotations. We haven't shown the rotations yet. But what I'll say about the rotations is that the rotations will always involve at most three nodes. All right, no matter wherever they happen, they involve three nodes. And so

DS Lecture 12 Page 8 of 19 Transcribed by <https://otter.ai>

those three nodes because their orders within the trees will shift, those three nodes will need to have their heights change. But we can trivially calculate that because we'll know whether they've moved up by one or moved down by one. Everything below them, those things heights don't change if the stuff above them pulled them somewhere else. Because nothing got pushed below them. So it's only those three nodes whose heights will change. So at the moment that we do those rotations? Yes, we absolutely do you have to update the heights of those three nodes as we go. Does that make sense? Thank you. Yes, thank you. So when asked, and that's because we're calculating the height and constant time every time we insert or remove a node, right. So we're always updating this every time we insert or remove. So we don't actually have to look at the tree globally, we just have to

track back up through whatever part of the tree we went through in the first place. And since that's all happening recursively as those recursive calls return, that will bring us back up to the appropriate level.

39:08

But isn't it like not n but isn't it like $\log n$ recursions? Why would it be

39:14

sure it is but it that's not extra cost? You already made those $\log$ of n recursive calls, and they all have to finish their work no matter what. And so it's a constant factor, extra cost to each of those calls. It's not adding new calls. So maybe for each of those $\log$ of n calls, instead of performing seven fixed operations per we're performing eight fixed operations per that doesn't affect the big O cost. It affects the it affects the factor in front of them. Does that make sense? perfect sense. Thank you. Cool. Um, so Fang is asking Can I He's Deaf instead of height to check the balance condition, no, you really need to use height, because you want to be looking down, not up. Okay? All right. So let's start looking at the actual rotations to fix these things. So the first way I'm going to do this is I'm just going to do it in terms of very simple avielle trees that involve just three nodes, the three nodes that will have to change, all right, and the rotations that we're going to do involving just those three nodes break down into four distinct cases, with four distinct types of rotations that we need to do. But when I say distinct, I maybe I'm not being fair, there are really two distinct cases and the other two are just mirrors of the original two. So now once we've seen that just on simple three nodes, then what we'll do is we'll expand it out to more general avielle trees of arbitrary size. Okay, so let's start with the simple examples. And let's call this case, case one. So the case one of the types of imbalances you can inject looks like the following. Let's say we start with this tree. We start with an avielle tree that looks like this. Okay, everybody, okay, that this is a valid avielle tree based on everything

DS Lecture 12 Page 9 of 19 Transcribed by <https://otter.ai>

else we've discussed today. Yes. Excellent. Okay. So what we're going to do now is we're going to insert something into this, that will cause an imbalance. And what we're going to insert is we're going to insert something that is smaller than both of these existing nodes, specifically, we're going to insert a one. All right, so if we're going to insert a one, we start here, the one is smaller than the three, so we go left, and the one is smaller than the two, so we go left again. So that's where the one gets inserted, based on a standard binary search tree insert. Now as those recursive calls to do the inserts close back up towards the root. To finish up, we said, we're going to augment each of these nodes with their heights so that we can check the heights of the sub trees as we go, the one has less than right sub trees of height minus one, totally fine, the two has a left subtree height of zero, right subtree height of minus one, totally fine, the three has a left subtree height of one and a right subtree height of minus one. So you have an imbalance here. Now this kind of imbalance occurs the case one, when the imbalance had you move to the left, and then to the left, again, from the point of imbalance. So maybe the node we were inserting was actually somewhere deeper over here, I don't care, All I care about is that if I have an imbalance here, and the first step of the insert went left once, and the second step of the insert went left again, I know I have a case, one imbalance. And that case, one imbalance involves these three nodes. So if you have three nodes, and you want to arrange them into a perfectly balanced binary search tree, which of those three nodes should always be the route

44:00

in general, the midpoint, right, the middle node should have been the root because that means the one that's smaller than it will be to its left than the one that's larger will be to its right, which means it's this first node to the left, that we actually want to become the root. And so for this, we perform what's known as a single rotation with our left child, which will rotate the two up to become the root of the tree. Okay. Now, what I would say

there is, the way I like to think about this is I like to think about, sort of think about the nodes being affected by gravity. But in our world, gravity follows the the avielle sorry, the binary search tree condition. So when something flops down, it should flop down to the right if it's larger and flop down to the left, if it's smaller, so that's what we're gonna assume. So the way I think about this is I'm grasping the two and pulling it up, and then shaking it so that things flop down into the right position. So the three has to come down to its right. So that single rotation pulls the two up. The one, gravity doesn't change it, because it's already flopped to the right already on the right side, then the three flops down to the right, because it's larger. And with that single rotation, we have now balanced this trade. Now, this works when we go to the left and to the left again. Okay, so that's case one. case two, starts out a little bit differently. In case two, let's say the root of this thing is a three. And let's say initially to its left, we have a one. Alright, so again, this is

DS Lecture 12 Page 10 of 19 Transcribed by <https://otter.ai>

a valid avielle. And now what we're going to do is we're going to insert a two into it. All right, so something that's smaller than the root, but larger than the left child, so we inserted two, so the two will first go to the left, and then we'll go to the rights. And so our two lands here. Now, in this case, we know still three nodes, the middle one should ultimately become the root of this thing, right? So we still want the two to become the root, what we're no longer going to be able to accomplish that with a single rotation, right? If we rotate the one with the three, that isn't going to help anything. And if we rotate the two with the one, that's still not going to balance it, so we need to pull the two up twice, wants to sort of get it in place of the one and then a second time to get it in place of the three. So this kind of case occurs when from the point of imbalance. First we went left and then we went right. Okay. So in that case, we do what's known as a double rotation.

47:37

So that's going to comprise of two rotations, the first rotation is we're going to pull the two up one level. All right, so we'll call this rock one for rotation one. In that example, the three is currently on change, because we're doing a rotation further down, the two gets pulled up, we shake it, the one flops down based on our gravity based on the BST condition, so it has to flop to the left. So we end up with the following. which incidentally, just converted case two back into case one, right, and we know how to do that. Another rotation to pull the two up a second time. So this is the second rotation brought to all right. And so the two comes up, the one gets pulled with it and the three flops down. And we've now balanced it, this time requiring two rotations instead of one. Everybody okay with that. So, the remaining two cases, cases three and four are actually just mirrors of these cases. So one sort of sounds a little funny to say it out loud. But one way of thinking about this, of this case, too, is like we're going one direction and going in another direction. So that's sort of like the phrase zigzagging, right, so zigzag. So I think of Zig is going to the left and zag is going to the right. These are actually technical terms in some data structures. So this would be the zig zig and this would be the zig zag. And then we need zag zig and zag zag. That would be our other two. So I got a message from somebody saying Would you please repeat the first rotation of the double rotation? Ultimately we want? Yes, I think so. I mean, it is very consistent, right? We assign a definition to zig and zag and just stick with it. So sorry. Excuse me. Don't worry. I'm not sick. I was just choking on my own exciting Isn't that stupid joke? So sorry about that? No worries. I'll my stupid joke not you'll see mostly I just laugh at myself. Right. All right. So um, the second? Sorry, the question was repeat the first rotation? You mean, what brought the two up? person who had emailed me? Yeah. Okay. So the idea is that the middle of these two, no of these three nodes ultimately has to become a route. And the thing is, the rotations will only pull a node up one level at a time. So first, we pulled the two up to take

DS Lecture 12 Page 11 of 19 Transcribed by <https://otter.ai>

the place of the one. So imagine one is the root temporarily, and then we're doing a rotate with its right child. So the two gets pulled up to be in the place of the one as I put it here to the left of the three. And then we have to figure out which side the one falls to. And because the one is smaller, it has to fall to the left of the two. Does that answer your question? Cool. And then there was another one, sorry for the repeat. Why do we make the middle one there? Oh, cool. I actually answered both questions at once. So other private question. We're good with that now. Just make sure I actually answered that. Cool. All right. So that's case one, and two, let's just very quickly showcases three and four. So case three, which I guess we were calling these Zags Zig, where we go to the right, and then back to the left would be a situation that would look sort of like this, we'd start with, say, a one is the root with the three is its right, that's perfectly fine, then we insert a two, which means we'd go right then left, working our way back up, we find the imbalances at the one. And this is the mirror of the case two, we still want this node, the middle of the three to end up being the root. But we're just reversing the sides that things are on. So the first rotation, again, this is a double sorry, the first rotation brings the two up, the one is untouched for the moment, the two comes up, and then the three falls down and it has to fall to the right because it's larger. And then the two has to come up a second time to make it the root. So that'll be our second rotation. So the two comes up, drags the three up with it, and then the one will have to fall down to the left of the two because it's smaller. Okay. And then. So that's the Zagazig. And then the final one, case four, is if we go to the right then the right again. So that would start out with a situation that looks like this. Then we would add a three, go right? Go right again, we actually already kind of saw that case over here is the second rotation of case three. So that's a single rotation to bring the two up. So the two comes up, one falls down, three came along. And we're bounce case 123 and four. Any questions on the simple cases? We're good.

54:09

All right. These are for three nodes. Yeah,

54:12

yeah, these are just for three nodes. And so now we got to worry about a scenario where there are many more than three nodes. And so we're gonna have to figure out how do these rotations manifest themselves and far more complex trees. So that is our next task. Now, for the purposes of that, now that we see the symmetry in the cases, I'm just going to show you case one for a single rotation, and case two for a double rotation. And you should be able to be okay with case three and four is just mirrors of those cases. So we're going to go back and do case one again. But what I want to do now is I want to construct

DS Lecture 12 Page 12 of 19 Transcribed by <https://otter.ai>

a much more generalized tree And so before we do that, I have to talk about the symbols that I'm going to use here. So nodes will still be circles. But I am no longer going to put the actual values that the nodes store within them on the paper. Instead, I'm going to label them with like a variable. So I might call the root node k_2 , for example, that doesn't mean the data stored in it is k_2 , it means it can be referred to as the variable k_2 . So I'm not going to put the values in there, but we're going to assume that everything that I draw upholds the binary search tree condition. So if I put k_1 to the left of k_2 , that means k_1 is smaller than k_2 , whatever it is, right. And so the side of a node, that something is on should tell you whether it's larger or smaller, because I'm just telling you, these will be valid binary search trees. Now the other thing I'm going to do is I'm going to actually use representations of entire sub trees. And this should ring a little bit of a bell because I use this representation when I was showing you some induction proofs. week or two, actually, probably the week before spring break. Um, we all have little triangles, little triangles will represent avielle trees. So these represent avielle trees of some arbitrary size. I'm not telling you how many nodes are in there, just that whatever nodes are in there are valid avielle trees, that's all I'm telling you. Okay. And so we'll have little triangles. And we

will have big triangles. And the difference is that little triangles and big triangles, big triangles will have a height one greater than the little triangle. All right, that's it. We don't know what those heights are just big triangles have one more on their height than little triangles. Does everybody okay with the symbols? Good. All right. So let's draw an example. That will become case one. And so we'll start with the root node of this tree, which we'll call K_2 . To the left of K_2 , we'll have another node called K_1 . And because we're just saying everything is, I'm just telling you everything is valid BST, whatever value is in K_1 has got to be smaller than K_2 . Now, to the left of K_1 will be an entire valid AVL tree, which will be a little triangle, which I will label A so little triangle A is a valid AVL tree. And because of its position in this drawing, must be every element today must be smaller than K_1 . And every element in A must in fact, also be smaller than K_2 . Do we agree

58:33

to the right of K_1 will be another little triangle, which we'll call B . Okay. So A and B are AVL trees of arbitrary height, except that's the same height between A and B . So K_1 's left subtree and K_1 's right subtree have the same height. Because of where I've placed being the tree. Every item in B has to be by definition larger than K_1 , but smaller than K_2 to a grave. And then to the right of K_2 , I'll have yet another little triangle that all labels see. And so everything in C must be larger than K_2 , and therefore also larger than everything over here. At this point, we still have a valid AVL tree because K_2 's left subtree and K_2 's right subtree different heights by exactly one, that presence of K_1 adds the additional level. Does everybody agree that that's a valid AVL tree? Good. Okay.

DS Lecture 12 Page 13 of 19 Transcribed by <https://otter.ai>

Now let's break it. All right. We're going to break it by performing an insert into this AVL tree of a value that is smaller than K_2 and smaller than K_1 . Okay, so I'm inserting a value smaller than K_1 . So I start at the root for this insert. And since I know it's smaller than K_1 , it's got to be smaller than K_2 . So we go left, it said, it's smaller than K_1 , so we go left again. Now, I don't know anything about the structure of A , I just know that it's a valid AVL tree. So now I am inserting a value into the AVL tree, and I'm doing it recursively. So what we're going to do is we're going to rely on the design rule, and just assume that it worked. All right, that whatever happens after I insert this value into A will result in another valid AVL tree. Okay, now, when I insert this value into A , one of two things is going to happen to A , since A satisfies the AVL condition, either its height will not change, because this new value I'm inserting will sort of fill in a gap that might be in one of the levels that's differs by more than one. So one possibility is that the height of A doesn't change. And if the height of A doesn't change, as the recursion returns back up towards the root, nothing would change, and we wouldn't have to worry about a rotation. The other possibility is that maybe A is all filled up. And so the addition of this node could potentially increase the height of A by exactly one. That's the only other option. So let's assume the worst, let's assume this insert forces a two increase in height by one, we okay with that. And so by doing that, little A suddenly becomes big triangle A , because I've increased the type by one. And so we're saying that these triangles are valid AVL tree. So we're going to assume that as this returns back here, we just know that's a valid AVL tree. Now, that will return back to K_1 . And we have to check to see whether or not there's an imbalance. Prior to this insert the left and right subtrees of K_1 different in height by zero. Since we've only increased the height of A by one now the left and right subtrees different height by exactly one. Is K_1 still a valid AVL node. Yeah, we're good. So now we return. Sorry, what's that from the Okay, now we return back to K_2 . And if we look at K_2 , the height of its left subtree. And the height of its right subtree originally differed by only one. But now because the furthest leaf has gone down one further, because A is increasing height by one. This, the difference in height between the two sides is two. And so we have an imbalance here.

1:03:18

And so the only question is, what were the first two steps that I performed from the point of imbalance. And so from the point of imbalance, I went to the left, and then I went to the left again, I don't care what happened below that point. I just care about the first two steps from the point of imbalance. So I went to the left and then the left again. And what does that mean? That means it's case one zigzag, and it's a single rotation. That should pull k one up to be the new root. Do we agree? Okay, so let's draw that and just see how it might look. So when I pull k one up, k one will be the new root. And so k two is going to have to fall down to be somewhere below, which means this link here goes away, right,

DS Lecture 12 Page 14 of 19 Transcribed by <https://otter.ai>

that link is broken. Do we agree? Because k two's left subtree will no longer be k one. Now what we saw in the three person case is that the K to the original root needs to drop down and become the right subtree of K one. Okay. But for it to become the right subtree of K one. Little B has to be orphaned for a minute. Alright, so we're going to orphan little B. Okay? So we break that link, and then k two will become the right subtree of K one. And so what that will look like k one comes up,

1:05:17

it drags big a with it, k two gets pushed down, little c gets pushed down with it. And then little B

1:05:34

is hanging out here, orphaned, except I very explicitly broke k two's left link, right. So in all cases, this left link will be empty. Now let's think about little B, by definition, little B has to be smaller than K to all of its nodes, because it lived on the left subtree of K two. And by definition, because it was the right subtree of K one, it's got to be bigger than K one. So little B, once K one comes up, it's got to be to the right of K one, and has to be to the left of K two, and serendipity. That's the only space that's free. So b just moves over and becomes the left child of K two. And now the height of the left subtree and the height of the right subtree are exactly equal of the new root. And we haven't completely balanced. Any questions on that. We're good. I really enjoyed this. I think this is actually quite nice. The rest of you probably don't agree, but I think it's pretty the original A, B and

1:07:00

C, you said were the same size or your Yeah, the original A, B and C, I

1:07:06

said we're all of the same height. Okay. And then when I because I'm saying all things that I call little triangles have the same height. And all things that I call big triangles have a height of one greater than a little triangle. Okay, so I turned a from a little triangle into a big triangle, and that created the imbalance, and then the rotation fixed it. Okay.

1:07:31

DS Lecture 12 Page 15 of 19 Transcribed by <https://otter.ai>

Is there like a super big triangle four plus two or no?

1:07:36

You won't get that? Oh, because it has to be a valid aviel at the start. And so the insert is only going to screw it up by one at most. Understood, that makes sense? Yep. Okay, so let me take a look at the questions. Why did you need to break the root of k one? And k two, if they stay connected after you rotate? Well, remember, k one is now the root of this tree. Right? We're pulling k one up. So k one becomes the root of the tree. And k two isn't the root anymore? Tiffany? Does that make sense? Cool. And then a private one asked, shouldn't k one and K two be swapped in the second diagram? Actually, no. The know that I labeled k one here. The whole point of this is to make it the root. Are we good? Private question person? Yes, no. Cool. All right. So let's actually sketch this out a little bit as if this were a piece of code. All right. And so let's call the piece of code, single, rotate, left. And

what it's going to do is it's going to return a new node object. Sorry, not a new node object, it will return the new root node of the tree after performing a single rotation on the left, which is what I did here. And the input parameter to this will be the root of the tree. Alright, so I'm passing in K two. And I'm saying I want to do a single rotate with K two's left child. Does that make sense? Yes. All right. So let's see. What does that mean. So k one, A, B, and C don't actually exist as variables, yet. What is k one in terms of K two, if you have a regular binary node with data left and right, whatever we can call it a vl, load binary node, let's just call it binary node for now. So k one is actually k two dot left, agreed. So if I want a variable, actually just so that's actually Java, let's call it k two, if I want a variable called k one, which will ultimately be the root, and what I want this method to return, I'll declare a variable called k one.

1:10:46

And I will set it equal to k two dot left. So now k one actually references the same node that I called it here. Okay? We're good. Okay. So the first thing I want to do is I want to walk be off of K one and make it the left child of K two, right? Because that'll break the link from k to to k one. And we'll move B over B is k one dot, right? Correct. So all I need to do now is set K to dot left equal to k one dot, right. So that moves B over to K two, and breaks this link. Now that I've stored be somewhere else, I can reset k one dot, right? That's k ones, right child to what it should be, which is k two. So I said k one dot rights equal to k two. And then all that's left for me to do is return the new root of the tree. And the new root of the tree is k one.

DS Lecture 12 Page 16 of 19 Transcribed by <https://otter.ai>

1:12:31

And that is the entire code that I would need to write to do a single rotation with the left child. Three operations sorry, well, four operations, including the return, and it's done, no matter how big the tree is. Are we good with that? Okay, so now the last thing we're going to do is we're going to diagram out case two. So we'll start out with the same setup. Except this time, maybe just to keep things consistent with how the tree how this is drawn. So that code and as is supposed to return the new root of the tree after the rotation is done. So it gets the old route passed in and returns the new route. That's how we've defined it. k one is the new route. Are we good with that? Cool. All right, so now we've got case two. And so I'm going to label my root node k three. And that's just to keep things consistent with how this is written in the textbook. Okay? Just a note. k threes, left child will be k two. Alright, k two is smaller than K three. And just just like before, we'll have little triangle a little triangle B. And little triangle. See. Now we're gonna break it, but we're gonna break it.

1:14:21

That wasn't me.

1:14:24

Whose alarm just went off? Oh, no, that was me. I have a meeting at seven. Okay, nevermind. Oh. Oops.

1:14:39

Leave it to me to pack things together. Okay, so, now what I'm going to do is I, oh, that meeting at seven is actually relevant for you guys. Okay, so I'm going to set this up, and then we're going to pick up from this point At the beginning of next class, okay, after that, we're going to start hash tables. So you should start reading chapter five. But let me just set this up. What I'm going to do now is I'm going to insert a value that's smaller than K three, but larger than K two, because I want to zigzag. So smaller than K three goes left, larger than K two goes, right. So now we're recursively inserting into B. And in the worst case, scenario, B goes from being little triangle to being big triangle. And so just like before, this now introduces an imbalance because the left subtree will now different height and the right sub trade by more than one by different by two, it induces an

imbalance here, but this time, it's a zigzag. Now, to do this, we need to do a, a double rotation. And that involves three nodes, it involves k 3k, two, but it also involves the root node of B. Because in this case, it's the root node of B, that needs to become the root of this whole thing. So I need to rewrite the tree. All right, so this is just a rewrite, this is going to be the same thing, just with a slightly different notation, we're still rooted at K 3k. Two is still to its left. That's a K two little C's over here, little a is over here. But now we need to factor the roots of B out. So we'll call the roots of b, k one. And then k one will have two sub trees, which I'll call B one and B two. All right. And so these three things k one with its sub trees, B one and B two. That's big triangle B, just broken down so that we have its root with the two sub trees, which are little triangles of the same height, ANC. binomial trees, just a very different setup. We'll talk about that later. Okay. Is everybody clear on this? And so now it's the three nodes k 3k, two and K one that will be involved in the rotation. And you know what? Yeah, we'll just finish up with this. But what's going to happen is that the K one is going to get rotated up. And then it will get rotated up a second time so that it becomes the root of the tree. And so we'll wrap up with that, do a few more examples at the beginning of Monday's lecture, and that should be it. Okay. All right. Now, why am I needing in? Now, six minutes is relevant for you. So how many of you remember, in 2004, last semester, there was like a little secondary class called Java labs that was being run by some of my ta is, I don't know how many of you use it, but it was something that was available. Um,

1:18:42

and so what Java labs had been, was it was a supplemental thing that we did, where, excuse me, where we did additional problems in a group setting. And we had tbas, who would like individually work with groups to work through a problem during the lab. It was pass fail, it was sort of supplementary work. There was no extra homework, you just came for the lab, where the problem would be described, and then he would work on the problem live in a small group setting with some direct ta help it was to give some more hands on practice for folks that maybe were a little bit less confident in their programming. Okay. So one of the things we've been talking about, and I actually have a grant funding part of this, which is why I'm able to hire people to do it. One of the things we've been talking about is possibly creating a similar kind of labs for data structures. Right. And so I have a number of my ta is sort of working on a labs Now unfortunately, we wouldn't be able to do a full one. This semester, because we're already halfway through. Um, but we were interested, we've been developing them. And we wanted to test one or two of them. Okay. And so at some point, it's not going to be tomorrow, but we might do it this weekend. We would like to run one of our labs is sort of a pilot. All right. And so if you have I not going to be able to this is just a test. We're really hoping to potentially do this thing. Starting next semester, we might even try it over the summer. Not sure yet. The summers. The summer is weird, you know. But we're trying. I'd like a few volunteers to come and try our sample. Lab, this potentially this weekend. And so if you're interested in

DS Lecture 12 Page 18 of 19 Transcribed by <https://otter.ai>

trying it out, send me an email. Okay. I'm, I will, hopefully be able to pick maybe 15 or so 1520 people, if possible, because I have enough staff for that. I don't know yet. Let's see who's interested. And then we'll try to schedule a time. Sound good. And so we just wanted to try it out. So if anybody is interested in trying it out this week, this is not for a grade. This is not for credit. This is just seeing if it helps seeing if it works, because we've been slowly developing it. So just shoot me a note if you're interested. Sound good? All right. Yeah, this summer is bizarre. I, I have an intro, I'm teaching 1004. And I'm teaching data structures over the summer. And my god. I'm like, usually my summer classes are 50 to 70 people. I slammed into my cap of 120 already. And I've got waitlist building. So it's a little bit horrifying what the summer looks like. But anyway, we're increasing the caps to

400. I'll take all comers over the summer, but let me know if you're interested in trying this out. Otherwise, we'll pick up with the last little bit of AVL trees on Monday. Keep an eye out for a supplemental lecture going through the binary tree code. And we'll start hash tables next week. Keep working on homework three. And I don't really think I can stick around for all that long just because I have another meeting but I can stick around for like two three minutes. Okay. Have a good one. Thank you, professor. Take care.

DS Lecture 12 Page 19 of 19 Transcribed by <https://otter.ai>

DS Lecture 13

Wed, 8/11 1:35PM 1:20:25

SUMMARY KEYWORDS

binary search tree, homework, insert, question, left, node, tree, subtree, root, method, equal, instance variables, element, return, midterm, code, write, dot, imbalance, rotation

03:05

This neither of us want to see Daniel and then driving and then saying he can turn back down on his luck became a sad sad robot. robot. Robot robot robot robot sad sad, sad, sad, sad, sad, sad, sad robot here a sad, sad, sad, sad robot. Alright, let's get started. This is a

07:26

sort of an interesting thing where no one except for I guess two people that have actually bothered with a video today. Ah, there we go people woke up. I mean, you don't have to I don't really mind either way. Oh wow, okay, this is different from before I had to say something right. Okay, so several things to note, um you know, I was getting a lot of messages via the private chat, I'm not going to be able to monitor those right now just because I have to focus on this. So sorry about that. Um, but you know, there are other people in the chat so they can handle those. So, a few votes. Let's see, homework two grades were released few minutes back. Yes. I also very, very quietly released the midterm part two grades, very quietly, oh, no, several people just suddenly looked away on their screens. The reason why I did it very, very quietly is because there were still I think, four people or five peoples that I have to fix something on. And grade scope just wouldn't let me release part of them. So I just released all of them. And I'll send a big grand announcement once I've had a chance to tweak those last few that I need to tweak. So if you sent me a note about like missing a piece of an exam during the exam, do not panic. I am still getting to those. Okay, is everybody clear on that point. And we also spoke about the averages on this already, I believe, but just so that everybody understands what they're looking at.

DS Lecture 13 Page 1 of 21 Transcribed by <https://otter.ai>

09:31

The average was 38 out of 50, which was roughly the same a little bit lower than the average on the first part, which would mean that we'd be looking at an overall average on the exam of about 80 81%, I believe. So if you had overall, about 80%. On this exam, you have a low B plus just on that exam. Alright, remember, I don't just assign letter grades to different pieces. As I look at the overall average, and that's only 20% of your grade. So even if that wasn't that grade wasn't exactly what you would have liked, even if it's quite low, it's still easily recoverable. And remember, the final looks very much like the midterm in structure, and to a certain extent in content. So you should be able to start studying as soon as possible for that just on the midterm. And you're in good shape for the final. Okay. Um, so while there are two, okay, so I'm going to, I'm going to save this at the risk of generating myself a lot more email, but I think it's fair. There was only one problem on

midterm part one that actually I think, can realistically be regretted if and so that you should listen, um, the multiple choice ones, the answers that we have or the correct ones. So there's nothing to regrade there. But do you remember there was one problem that, had you type in the value that was at the bottom of the stack? Does that ring a bell? Yes. The answer to that one was seven, I believe, because remember, you're popping the items off, and then applying the operator and pushing them back. So I believe there was a six then a one minute plus, and that that plus pop the six and the one and push to seven, and back, then I think a four got pushed on top of that. So the seven was at the bottom. But where are you can check if you typed anything other than a seven in that box? Like if you typed the answer is seven, where you typed seven is at the bottom of the stack. If you typed anything other than seven, the auto grader graded it wrong. Okay, so if you typed a seven in there, but then embellished it with other texts, take a look. You can see. Shoot me a note? Because I can regrade that one because you got the correct answer. It's just the extra text threw off the auto grader. Does that make sense? Everybody? Now for midterm part two, I'll leave the grades open for a couple of weeks. And you can put regrade requests in the way you do on the homeworks for the midterm part two. Okay. Um, and the TA is and I will look at those sound good. Right, any questions on the exam? And bear in mind, there are a few that I still need to update that's not going to materially impact the average though it might go from 38.23 up to 39. But the average is pretty sickly. Right. Okay. So that's where we are with that you have your homework, basically, you have all your grades back at this point. So I think we're good there. Homework three, is due on I think we said Thursday, right. So there's that we need to talk about that a little bit. And so, Brianna, I'll get to your question is saying there was a private one before that. There's a homework for and there's a homework five. Um, a homework five is a little bit shorter than the previous homeworks. I think we've, we've hit the peak on I think somewhere between homework two, and homework three was sort of the peak, some people find homework to harder. I think homework three is harder. But the amount of code that you have to generate for homework three is a lot less intricate than the code you have to generate for homework two. But this is the peak of homework four, and homework five will.

DS Lecture 13 Page 2 of 21 Transcribed by <https://otter.ai>

13:53

Actually one sec. Homework foreign homework five will actually get somewhat easier. And homework five is a little bit on the short side specifically, because that one ends up being due around reading week. And I'm not going to make it do much into reading week because you don't really have much of a reading week this semester. Do you now. Oh, I made a mistake. I should call it reading week. It is the reading period or the study days. It's actually never been a week. I it annoyed me. So one day in a fit of procrastination. I went back and I checked my records. And it has never been called reading week in the last 26 years. It's always been reading period, and it's always been three days plus the weekend. You still have the three days. There's no weekend this time. And so I'm going to I'm sure that makes you feel much better. So I'm going to I'm going to try to have that do as close to the last day of classes as possible. I'm now So let's see. That's what the homework is. I think we pretty much have exactly four weeks, so we're good. When homework grades are dropped, are they dropped from the same pair of assignments? Yes. So when I think of a homework raid, I think of it as homework one, homework two, homework three homework for homework five, as the pair written programming, I am not going to pick and choose the best for writtens. And the best for programming. I can't, because the weighting between the written and the programming are slightly different on each assignment. Okay, so it's the pair. And the solutions to part two, I have not pushed, but they're written I will push them within a day. Okay. So that should be that. Um, but but so somebody asks, Professor Blair, so just to clarify, we won't have a homework six, new, we do not have time for homework six, but I will still drop the lowest pair out of what we

do have. So I didn't want to take that away from me. Okay.

16:09

Any other questions?

16:12

Cool. All right. So, um, does anyone have any questions on the upcoming homework three?

DS Lecture 13 Page 3 of 21 Transcribed by <https://otter.ai>

16:22

We're good. Really? Okay.

16:29

So I have two comments on it. A comment number one, if you are just starting the expression tree problem, okay. Um, you should go back. And I actually went through actually went through my lectures to find the exact moment, I believe it is the February 24 lecture, somewhere in the vicinity of the 45 minute mark, where I actually write down on paper, a lecture 14, I think that was February 24, I actually write down on paper, the starter code for that, like, I show you how to write the eval method and how it's one way of setting up your expression nodes. There are a number of ways, but I show you one way. And so my suggestion is, if you don't know where to start on that one, go there. Okay. Now, the other thing is that for the binary search tree stuff, I was originally going to give you a pre recorded lecture that goes through some of the existing binary search tree methods that I was asking you to look at and code, I'm actually not going to do that. I gave you a pre recorded lecture that's already there, it actually just got posted of the abl tree code. So what I'm going to do today is I'm going to wrap up my discussion of AVLs, you can look at the code for the AVLs after this lecture, whatever, you don't need it for the homework, but a little later on. And I'm going to go through the binary search tree code now because I feel like a lot of you need to see that and ask questions on that. Because that seems to be driving a number of the questions I'm seeing on the homework. Sound good. On if I have time, we'll start hash tables at the end of lecture today. But we'll probably do hash tables really starting on Wednesday. And so let me get a sense here. How many of you have started the expression trees? Click Yes to kick the checkbox or the plus don't? Okay.

18:55

Yeah, right. So a little less than half of you say they've started about a 10th of you say they have not, and the other 40% are too afraid and or not listening to me. Um, how many of you have started the binary tree? The binary search tree part, the better BST Okay. All right. Thanks. So, um, how about this? Your homework three written stays due on Thursday. All right, the I'm not changing that. Okay. Um, I'd say you should still consider your homework three programming due on Thursday if at all possible, and I gotta post homework for on Thursday with the deadline for homework four determined based on it coming out on Thursday, because we don't really have enough time to do anything else. But with all that said, I will then overlap homework three with homework four and give you

DS Lecture 13 Page 4 of 21 Transcribed by <https://otter.ai>

the extra weekend on homework three programming. Is that acceptable? Good. All right. And I think the two people who who said, Hey, guys started now it takes forever are probably very mad at me right now. So, the least they're laughing. At least they're publicly laughing privately, they want to kill me. Actually, probably privately, more than a few of you want to kill me. But that's that. That's okay. That's, that's my life. Anyway, so when will it be due? Then I said Monday night is that I give you extra weekend to be due on Monday. Okay. All right. So it'll be due on Monday. So I don't want to do on Friday, that ends up causing other problems. Everything's due on Friday, so you can have the weekend. So I'll have to do at 11:59pm on Monday. Um, so that's where we are. But that doesn't change anything in terms of homework for homework four will go live Wednesday or Thursday of

this week, and B to two weeks from there. Homework three written still got to be done by Thursday. I need my ta is to start on that. Okay. We're good. Anything else we need to discuss?

21:40

Okay, cool.

21:43

All right. So, um, abl trees, we actually kind of did abls theoretically just fine. Um, I believe the last thing we had done was we had sort of set up the, the, the, to rotation for the case to case at the end of last lecture, but we didn't completely follow through with it. So let's wrap that up now. And we'll do a couple of examples. And then we'll do the binary search tree code, like straight out of the Java file. Okay, so what we had

22:24

was a root node. This marker is on its last legs, but we're gonna go with it.

22:34

So I'm just going to answer this question publicly, you can still use the 48 hours of late hours, if you have them left on homework three programming beyond that Monday, just be aware that you're eating into your time on homework four. And so I'm not giving you another extension on homework four. But that you can still use those hours. Good. Okay. Okay, so we had case three.

DS Lecture 13 Page 5 of 21 Transcribed by <https://otter.ai>

23:07

We had k two over there. We had a little triangle on the other side, which I believe we called C.

23:16

To the left of K two, we had little a. And then we had originally had little B over there. And but then what we did was we inserted something that was larger, we inserted something that was larger than K two, but smaller than K three, that made the little B that was over here become big B. And then what I did was I broke it in half. And so I factored out its root node. So that's k one over here. And then I sort of split Big B into two little B's so little B one. And little bit too. That's what the insert did. And this was originally the triangle Big B. And so what this does, is it gives us an imbalance. That's a zigzag right, first we went left, then we went right. Okay. So this was a two rotation, a double rotation. So rock one brings k one up one level. And then the second rock will bring k one up to become the root because k one is the mid median of the three nodes involved in the insert because remember, you detect the imbalance. And then you just look the first two steps down, even if those three nodes don't include the node that you inserted. Those are the three nodes involved. So first things first, we're going to rotate the K one up. So that means the K three in this initial phase and little C are unaffected by this PE one comes up But when k one comes up, all right, it's going to bring little B two with it. Now, when k one comes up, k two has to go to the left of K one, because k two is smaller than K one, which means we got to break the link on k one dot left to be one. So we're going to orphan that temporarily. And we're going to put k two here. k two got pushed down along with a little a. But remember, k twos, right child also got broken, because k two no longer has k one as its right child. So k two has an empty space for something that's larger than it but smaller than K one, B one is smaller than K one, but larger than K two, because this is a valid DSD. So B, one can just nicely go here. Alright, now as you know, this does not fix our problem. But this part here really is just another big triangle. So effectively, this is transformed us into the case one. And now we got to do our second rotation to get k one up to be the root. So we'll call that rock two. We'll write it down here. So now K, one comes up and becomes the root. That's what our double rotation method will return. As k one comes up, k three gets shoved down, right. So k three, no longer will have k one as its left child, in K

three, we'll come down to the right of K one, which means we got to break this link also. So b two temporarily gets orphaned. So k three goes down. Little c got pushed with it. And then the issue here is little B two is larger than K one, so it has to go to the right of it. But it's smaller than K three, so it can get slot in right in this little empty space here. K, one drags k two up with it. And k two takes little a and little B one up for the ride. And now we end up with a well balanced, perfectly balanced tree. And now you can sort of see how all

DS Lecture 13 Page 6 of 21 Transcribed by <https://otter.ai>

the other stuff comes along gets dragged along with it. We okay with this? Alright, so really, I had shown you a sketched out what the code might look like for a single rotation. And so really, for the double rotation, you're just calling the single rotation code twice, once one level down, and then once at the root level. Now, the supplemental lecture, which is already in the video library, goes through this in detail. Okay, in the actual code. Any questions on this second rotation?

28:20

Excellent. Okay. Now, what I would like to do is I'd like to do a few

28:30

inserts into an initially MTA vl tree, this should be useful for one of your written problems. Um, so let me just bring up one thing.

28:55

So I'm going to assume that we have an initially empty a vl tree, and then we're going to start inserting items into it. And so I'm going to write the items down, we're going to insert I'm not going to write them in advance, I'm going to come up with them as we go, because I want to demonstrate very specific things. Okay. So let's insert. Let's insert a five right now in an initially empty avielle. That just becomes the root node. perfectly balanced, no problem. Let's insert a three. We go left. But the three here. We still don't have an imbalance because the left and right subtrees of the five differ by only one. Okay.

29:44

All right, now,

29:46

let's break something. So let's insert

29:52

maybe a one.

DS Lecture 13 Page 7 of 21 Transcribed by <https://otter.ai>

29:57

Alright, so I insert the one that So we go left, we go left again. The one goes here, and we work our way back up to the root checking for imbalances, no imbalance here. No imbalance here. But we do have an imbalance there. Which cases that

30:19

they buy the first case, right case one. So, because we went left and we went left again,

30:32

that's the single rotation where the three comes up, five goes down. So now our tree will look like that. And we are balanced again. Okay. So let's do another insert. Um, boy, I, I chose poorly, as usual. So we're going to have some fractions in our integers here, just because, but for the moment, let's insert a two. Okay, so to first we go left, compare, here we go, right? work our way back up. negative one, negative one, all good. Negative one, zero or good? one, zero. All good. No imbalances. Okay, so let's, let's do another one. Uh

31:30

huh. How about we insert of six? So the three, okay, six goes, right? Six goes, right again.

31:44

Any imbalances here? Nope. Okay, now, you will see where I messed up, we're going to

insert a 1.5. Okay. So the 1.5 is less than the three. So we go left, the 1.5 is greater than the one. So we go right, and then the 1.5 is less than the two. Now we're going to work our way back up towards the root looking for an imbalance. There's no imbalance here, zero and sorry, negative one and negative one. There's no imbalance here, zero and negative one. But there is an imbalance here, we've got negative one on this side and one on this side. So what you'll notice is that the imbalance actually occurs at an interior node, that's not the root of the whole tree. And in fact, if you took one more step up, is there actually an imbalance of the three? Right now? If I know the height of the left subtree is two, the height of the right subtree is one. So the imbalances really just here. Okay. So the three nodes involved in the imbalance are these three, the rest

DS Lecture 13 Page 8 of 21 Transcribed by <https://otter.ai>

33:19

are uninvolved. And so which case is this? This is the Zagazig case three, so it's still a double rotation, so we got to do a rock one, because the 1.5 really wants to be the root of this subtree because it's the midpoint. So first, we're going to move the 1.5 up. So in that first rotation, the one stays put, the 1.5 comes up, and the two gets pushed down. We're still not balanced. But now we do one more rotation to Wow, that was wrong. We do one more rotation to get the 1.5 to be the root of that subtree. So we'll take that around, call that rock to the three, the five and the six are still unaffected by all this. And now the 1.5 comes up drags the two with it. And the one gets pushed down. And now this subtree is balanced correctly. Okay, then the recursion returns back up one level, and this top node is even better, right? Because the left sub trees height is only one and the right sub trees height is one. So the balance condition is pretty much spot on at the root. Is everybody good with that? Okay, and so the last thing I wanted to show you with respect to these inserts sonza discard pile. Um, let me switch my share for a second. So you're seeing my web browser here. Hold on, please tell me, you, you're just seeing the avielle tree visualization, right? This is very dangerous I've got, I've got my inbox over here. And I don't want to inadvertently click on that and see the hell that is my inbox. Yesterday afternoon, I had 4005 messages sitting in my inbox 4005 you know how many I have in there now. 18 137, I did a little bit of a purge. It actually felt very good, though a little bit alarming, but I still had stuff in there from 2018. So what I'm going to post here in the chat is actually the URL for this. So this as at the University of San Francisco, there is a very nice set of tree visualizer. So you can actually check your work on this stuff. And so the only problem with this is that it doesn't really show the double rotations the way I show them. But overall, it's still quite good. So let's actually try the example that I did just let's use 50, and 30 and 10, just so that we don't have to deal with the half. So there's my 550. Here's my 30. Here's the 10. And you can see a trigger the single rotation and it'll visualize it very nicely. Then we inserted a 20. Then we inserted a 60. And then we inserted a well, was 1.5, but we'll call it 15. Which will trigger the double rotation adjustment but at this level.

37:39

And you'll see it actually just shows it as a single step, which you could do but it's easier to think about it as a double rotation. All right, everybody get with this. Right. So you might find this useful as you're testing things. Okay. So you know how on the homework actually have you do a sequence of abl inserts on paper? member? Yeah. Oh, those I think their bookkeeping the heights a little differently than the way I book keep the heights. Ignore them. You want to book keep them the way I bookkeeping them. Okay, Tiffany. Cool. Um, so what you, you might notice that you can actually check your work against this. And that's okay, on the homework, but not on the on the final. You're not allowed to do that.

DS Lecture 13 Page 9 of 21 Transcribed by <https://otter.ai>

But please. Some people actually just submit screenshots from this as their submission. Don't do that, then I know you didn't even try. Okay. I wouldn't give you credit, if I see that. Okay. We'll ignore the fact that the homework solutions that will probably have

screenshots from this because, well, shortcuts must be taken on some points. And I'm still letting you check it, just try to do it by hand first, and it's important that you do because I'm going to ask you to do this on Final, I guarantee it. Okay. Now usually the way I do it on the final is I sort of don't have you draw it, I just have you tell me what number is where, but this is something you need to be able to do. Alright, so let me stop the share on that. And let's switch now.

39:43

Okay, so that's all I'm gonna say about avielle trees. Alright. What I want you to do though, is I want you to watch the video that I posted, and it's already there, that goes through the code for avielle trees that will build off of this code that you should have already read through, but I'd like to talk through it a little bit myself, which is for a regular non Balanced Binary Search Tree, and then the abl free will modify this code to make it balanced. Okay, so here is the waste code for the binary search tree class. And you will note that we have a number of methods, here we have the insert the Remove, we'll have a contains check, we'll do a find min and a find max. But these are assuming regular deletion, not lazy deletion. So that doesn't really help you much in terms of that written problem. As simple as empty, make empty, and then there's this print tree method, which is just an inorder traversal. And as we already discussed, if you do an inorder traversal, on a binary search tree, the elements come out in ascending order. Okay. Alright, so the first thing we're going to look at, and this should be very familiar, because we talked about it, in the context of other binary trees are the instance variables for this class, or I should say, instance variable for this class, this class only has a route. All right. And you'll notice, by the way, that the whole thing is generic within any type, but there are no constraints on it. Sorry, here, but up here, there is a constraint on it. And I think I may have actually shown you this at one point in an earlier lecture. All this means is that any type must extend must implement comparable on itself, or one of its super classes. Okay. So effectively, it just means it has to be comparable against itself. Alright, so our route is of type binary node, which we create as a static nested class, because nobody other than binary trees did have that this is similar to the way I told you to create your root nodes for your expression tree, except in this case, we're generic. Whereas an expression tree don't need to be. Now there are three instance variables for the binary nodes, there's the element. And then there are two additional binary node references, one to the left and one to the right. And then there's a set of constructors that just initialize those with whatever values you want. And you'll notice, we don't have any visibility modifiers on this.

DS Lecture 13 Page 10 of 21 Transcribed by <https://otter.ai>

That's just because, well, we give it the default level, which means that anything that can access this class can access these methods. And since it's private, to the binary tree, only the binary tree can access these instance variables directly, which is good enough for us, because we don't want to be calling accessor methods for that. so we can directly manipulate those. Alright. Any questions so far? On the instance variables? For the binary notes? We're good. Daniel, go ahead.

43:15

Is there any way to access these instance variables from another class? Not the one in this file?

43:20

very specifically? No. Um, they are intentionally hidden. Got it? Alright. Thank you. Because this is private. Okay, this

43:30

sorry. Does any other file know about the existence of this class at all? Not really. No. That's why we make it private. Got it? All right. Thank you. Wayne, what's up?

43:45

Hey, Professor, just a note on the overloaded constructors here a general question. Since we're able to access in all of our methods, you know, we're able to assign or not I couldn't say all but in some of our methods, we're assigning left and right. Children to parent nodes. Because we have two constructors here and one constructor automatically assigns left and right to know and the other one actually takes in a left and a right. Does the class is it? I guess my question is, is the class aware of a right a left and a right child because they are instance variables? In the binary node class? Yes. Okay. Yeah, maybe I need to be loud. I sorry. Okay. Yeah. happy to help. Um, they're,

44:43

they're available as instance variables as why it knows of their existence. Um, the constructor is taking in the initial values are just out of convenience. The truth of the matter is in a binary search tree, when you insert a new node, it always is a leaf initially.

DS Lecture 13 Page 11 of 21 Transcribed by <https://otter.ai>

Right, it always is. And so when we initially construct all of our binary nodes, we're going to want the left's and rights to be null. So this is gratuitous having both of them, but it could come in handy in other types of trees where maybe that's not always true. Okay. Cool. Other questions on this? All right. So those are our, our building blocks. Now we'll actually look at the methods. And I think it's helps to start with a contains check. Because everything. So this be so an overloaded constructor? No, no worries. An overloaded constructor means that there are multiple versions of the constructor that are differentiated by their signatures, right? And so when you have multiple constructors like that one constructor can call another, just by using the this keyword directly, and then passing in the parameters for the overload with one turn. Does that make sense? Cool. Awesome. All right, when did you start a question? Or is your hands is still floating? Okay. Um, so let's start with our contains method. Now, the thing will note about all of these methods is they come in these public private pairs. And you've seen that already with your expression trees, and everything else. Because the outside world doesn't know about nodes doesn't care about nodes. But any recursive method that we would invoke on the tree does care about nodes. All right. And so we're always gonna have two versions of the methods, one that takes either no parameters, or just a one parameter with a value that we're either trying to insert, or check to see if it exists. And then a second version that takes both the value that we're either looking for or trying to manipulate, as well as the root of the subtree that we're trying to perform this operation on. That's the private one, because that takes in a binary node. And, again, those are private. So are contains method, the public one takes in a variable x, which is a value that I'm looking to see whether or not it's in the tree at all. And then it returns a Boolean, true or false. But in fact, there's also a prop, that one, which takes in both the variable that you're looking for, and the root of the subtree, that you are currently searching for in. And so initially, you're currently searching for it in the subtree rooted at the root of the whole tree. So all we do is we call contains on AIX and the root of the entire tree. We take what it returns, and we return that as our own. Is everybody okay with that initial setup? Yes. Good. So now let's look at the private contains method. The private contains method, as we said, takes the value we're looking for and the root of the subtree, in which we are currently searching. We'll call that T because it's not always the root of the whole tree. And so the base case is that T, the root of the subtree that we're currently looking for it in is no, now we know that if we land on a no link is we're traversing through the tree, that no link is where that value would have had to have been. It's not there, it's not in the tree. And so we return false. Okay. Next,

48:51

if the root of the tree of the sub tree is not false, then we actually have to compare x the value we're looking for to the value currently stored in the root of the subtree. So we have

DS Lecture 13 Page 12 of 21 Transcribed by <https://otter.ai>

this temp variable compare results. So we don't just keep calling the Compare to method. You recall that any type was constrained to make sure that it implemented comparable on itself for one of its super classes. So we call `x dot compare to T dot element`. If compare result ends up being a negative number, it means `x` is smaller than `T element`. If it comes a positive number `x` is larger, and if it's zero, it means `x` is equal to `T element` and we found it okay. And so now we go into our condition with our recursive calls if compare result is less than zero, if it's anywhere, it's in the left subtree. So we return the results of calling `contains on X` and now `T dot left` because we want to look in the left subtree. Else if compare result is greater than zero, we return `contains on x` and `t dot right` because if it's a positive number `x` is larger, and if it exists anywhere, it's to the right. And then the final unconditional else would only be reached if compare results zero. If the compare results zero, it means `x` and `t elements` are equal, it means we found it. And so we can just safely return true. Sound good? By the way, you may recall that these binary search trees and ABS, whatever are sets, every value is unique. So we don't actually have to worry that it would appear somewhere else in the tree, once we found it at a location. That is the only location where it would be. Alright, any questions on `contains`? Hey, let's knock out some of the simple things first. So `contains` is easy. No, no, let's not. Let's just jump right in. Um, so the next piece is the insert method. The insert method, just like before, as a public and private version, the public version just takes the value I'm trying to insert the private version. And this is where things get a little bit tricky. All right. Sorry. Oh, someone said what if the value is not there? It would return false. Oh, students have answered that very well. So the issue is that if it's not there, so when you're asking what if the value is not there, you mean that for the `contains` correct? Yeah. Okay, so if you go back to the `contains` code, and you should be able to download this. If you go back to the `contains` code, you'll notice that at the beginning, if `t` were no, it does return false. That means if the root of the subtree that you think it's in is no, then it can't be theirs. So that's when it returns false. Are we good? Thank you. Yes. All right. Cool. All right. So now to the tricky thing about insert the private insert method. Now, first of all, trees are sets. So if you try to insert something that's already there, it can do anything. But here's the thing. The private insert method takes the `axe` that you're trying to insert, it takes the root of the subtree, that you are currently trying to insert it into which in the beginning, the root of the subtree we're trying to insert it into is root. So far, very similar to `contains`. But here's where it changes the root of the tree that you're inserting it into, could very well change. Right? When does the root of the tree change, when you insert `x` into it? There's one time when it changes in a regular binary search tree. Right if it's the first insert, if root was no. So the private insert method actually returns the new root of the subtree that you inserted into into just in case that route was supposed to change

DS Lecture 13 Page 13 of 21 Transcribed by <https://otter.ai>

53:25

the color, the original color actually has to take that new route and make sure it's set correctly. So we set `route` equal to the result of inserting `x` into root. And so there are two possibilities that could happen. Either root was initially No, in which case, this insert method will return the brand new node that contains `x` and that becomes the root and this assignment makes that happen. Or the root doesn't change. The root doesn't change when the value `x` gets inserted at some lower level. Or if the value `x` is already somewhere in the tree. But if the root doesn't change, we still need insert to return something. And we still have to make sure this assignment happens. So insert will just re return `route`. If the `route` didn't change, so you're guaranteed that whatever the private insert returns is the correct route, whether it's the same value it was originally or a brand new node. So you can safely set the root variable equal to that without worrying about breaking the trade. Is everybody okay with that? This is an incredibly important point, because we're actually going to exploit that at the lower levels as well when we do our recursive inserts lower

down. So any questions on that point? Excellent. So now let's take a look at the private insert. As we said, takes in x takes in the root of the subtree. We're trying to insert it into returns the new root of that subtree. If that subtree changed. Alright, so the base case, if t is no, if t is no, that means the root of the subtree is no, that means x is going to get placed in place of that no. And so if t is no, we create a new binary node that contains the value x, as well as no links, because when you insert something, it's inserted as a leaf. And then we return that new node. The parent color will set the parent equal to this new node, thus changing it from No. Is everybody okay with the base case? Okay, now, if t is not No, we begin our process. Oh, sorry, Daniel, go ahead.

56:08

This isn't 100% related to the algorithm, I just forgot when we're instantiating, the binary node, why don't we actually put the type between the angle brackets Why do we leave it empty?

56:19

Um, so Java infers it. Like from I think Java seven on it infers it from the type of so think of an assignment, right? When you set x equal to a new buy, it will not x is a bad example, when you set like root equal to a new binary node, right? The variable root in that case will have a type associated when you declare it. And that type will include something inside of the angle brackets. So I'm setting something equal to a new binary node. But this is going to have to match the type of the thing on the left hand side of the expression. So it can infer it from that.

57:01

I see. So if we were to stick any type in there, it wouldn't be wrong, right? Java would still know,

57:05

but it's redundant. Got it. But you might actually get a warning saying deprecated, but it will work. Okay, got it. Thank you. All right. Good. Okay. So we do our compare to see which side x should be inserted in. And so if compare result is less than zero, it means x needs to get inserted into the left subtree. So we just utilize insert, insert x into T dot left, and that will return the new roots of the left subtree after the insert finishes. So we have to make sure that we set T dot left equal to that after that has happened. So that's exploiting the same property that we exploited in the public driver method. Questions. Okay, else, if compare results is greater than zero, we set T dot right equal to the new route after we insert x into T dot, right. That last else is if x and t elements are equal. And because this is a set, that means we don't do anything, because T is unchanged. So we left this else in here, as a matter of symmetry with the other contains method, but we just put an empty statement in there, so it doesn't do anything. So after all of the aspects of T have been modified, if necessary, oops, um, then we need to return key, right? Because either it returns a new node, if t were no, or it returns the original route back, so the assignment in the parent color doesn't break. And that's it for my insert. Any questions on that?

59:02

Okay. So now, um, we go to the more complicated one, the Remove method.

59:18

So just like with the others, we have a public driver method just takes the value we're trying to remove. And just like with the insert, the private remove, takes the value we're trying to remove, takes the root of the subtree, we're trying to remove it from and then returns the new root of that tree after the Remove has happened. In this case, the Remove changes the root if the value that we're trying to remove is at the root. If the value we're trying to remove is at a lower level, it just returns the original Ryback. Okay. So now let's look at the private remove So just like the other two methods, the starts with something

that looks a lot like it contains, okay, we start with a contains. And then once we find a value that has it, then we remove it. Now there is the possibility that the value was not in the tree, which would mean that in our process of trying to find the value we're trying to remove, we might end up at a node. So if the root of the subtree that we're trying to remove x from is null, it means x isn't in the thing. So we just return t right back up to the parent color, and we move on with our lives. Okay. But if t is in there, we have to continue our search. And so we do the Compare result business like we had done before. Get that out of the way. If compare result is less than zero, we set t left equal to remove x from t left elsif compare result is greater than zero, we set t right equal to remove x from t write nothing different from the insert. Here's where it gets interesting. These two conditions that we have here are both conditions that occur when T element is equal to x. So if we hit either of those two cases, it means T is the element we're trying to remove. And the reason why there are two cases here, in reality, there are actually three, possibly four cases depending on how you count is because remember, what happens when you remove an item. If you remove an item that has two children, you are going to replace it with either the largest element to its left, or the smallest element to its right. Does that ring a bell? Good. And so that's that first case. So else if t left is not no, and T right is not? No, it means we have two children. In this case, we are assuming that we're going to remove the larger the smallest element from the right subtree. And so we haven't gone through it yet, we will towards the end of the lecture. But there's a find min method that takes the root of a subtree and then finds the smallest node in that subtree. And then we find the min, we extract the element from that node that was the smallest node and T dot right. And we replace the current node with that minimum node from the right subtree. So we're actually not creating a new node, we're just overwriting T's data with the appropriate value from the right subtree. So far, so good. But problem with that is that the original copy of that item still exists in the right subtree. So then we just called remove again, to get rid of that value from the right subtree. And that'll take care of it for us. Okay. Do we have any questions on how to deal with the case of two children?

1:03:33

We're good.

1:03:35

Okay. The other cases are if you have one child, or zero children, if you have one child, whether it's a left child or a right child, you just replace yourself with that one child, you just pull it up. And if you have zero children, what do you replace yourself with? They by No, no? Good. So this final ELS case actually handles all of those cases in one go. All right,

it just changes t to be equal to something and this, I realized that we probably hadn't seen this when I saw a question about this on Piazza. Like what on earth is this question mark colon business? It's something known as the ternary. operator. Did you learn the question mark colon in 1004? Have I ever talked about it in this class? Oh, bummer. It's fun. Let's take a look at it in a separate file for a second. All this test. I'm not even gonna write much Java code. I'm just gonna write a fragment. Alright. So let's say we want to Okay, so question mark Cole. It's a ternary operator. Because it takes three operands, the first operand comes before the question mark. The thing that comes before the question mark is a condition. This is really like an if else statement baked into one line. So the thing that we put before it has to be a Boolean, right? It has to evaluate to a true or false. Okay, so let's say we have a variable x, which we set equal to one. Let's say we have another variable y, and set that equal to five. Let's say we have another variable z. So this says if x is equal to one. So if that's true, then the second operand is what this whole thing evaluates to. And the second operand comes between the question mark and the colon. So if I wrote it like this, if x is equal to one, then this whole expression will evaluate to y

minus one. Okay? And if x equals one is false, then it will evaluate to the third operand. All right, which would be what comes after the question mark. So if x is equal to one, the whole thing evaluates to y minus one. If x is not equal to one, the whole thing will evaluate to y plus one. I said evaluate to which means you really want to set something equal to this. All right. So what's gonna happen here is that z will get set equal to whatever this whole thing evaluates to. And this whole thing will evaluate to four if x is one, and this whole thing will evaluate to six, if x is not one. So after all, this is done. What is z equal to?

1:07:15

Four? Is everybody good with that? And if I change x to something else, z will be equal to one, six. Any questions on this? So somebody asked, Is there a difference between using this in a switch case or an if else, it is a very compact way of doing it, and you can sort of include it directly in an assignment, rather than saying if x is equal to one, then set z equal to y minus one else, set z equal to y plus one. So it's a very compact way of doing this in one line, you can absolutely replicate this with an if else actually a switch case, I guess you probably could, depending on whether or not the condition was an equality check, or an inequality. Because with a switch case, you can't do any qualities with a switch case. Does that make sense? All right. So let's go back to our original code.

1:08:35

Now there's actually

DS Lecture 13 Page 17 of 21 Transcribed by <https://otter.ai>

1:08:39

so this case hits true. If there's one child or zero children, it encompasses both of them. So the test condition is if T dot left is not equal to one, we set t equal to this whole thing if T dot left is not equal to one. Omar, let me actually finish this first, and then we'll go back to the other one. Um, if T dot left is not equal to one, wow, I just said that completely wrong. If t dot left is not equal to No, that means there is one child and that one child is the left child. So we set t equal to the left child. If t dot left is equal to No, it means there's one child and that child is the right child or it means there's no children. But if you if you know that the child is he is the right child then you want t to be equal to T dot right? If you know that, there are no children, then you want t to be equal to no but T dot right is no if there are no children. So this will set it to the correct thing in all cases. Are we okay with that? So any questions on this specific piece? And then I'll move back to the question about the previous. Alright. Good. So now going back to the previous one. Ah,

1:10:19

sorry, somebody, what do you mean, when would it be hit? I'm sorry, I didn't quite understand that question.

1:10:30

Oh, so

1:10:34

these last two cases are only reached if T dot element is equal to x, meaning you found it. So you reach this point to even test this only when T element is equal to x. And so if we reach this point t element is equal to x. And then this highlighted piece here is true if both t left and T right or not, no. So you would hit this final else in the event that x is equal to T dot element. And either t left, or T right? Or both of them are No. Does that answer your question? Cool. All right, backing up to omers question. Um, can you explain again, why you were removed twice in the two children situation? You'll notice when I find the minimum elements in the right subtree, I'm not actually setting t equal to a new node. I'm modifying the contents of t. Do you see that? And so because I'm modifying the contents of t, I'm the original element that had that the original node that had that element is still stuck somewhere lower down. So we got to get rid of it. All right. Now it so happens that the one that you remove lower down is guaranteed to be this case. So it'll only be one

more step to get rid of that other one. Okay, Wayne, go ahead.

DS Lecture 13 Page 18 of 21 Transcribed by <https://otter.ai>

1:12:16

Thanks, Professor in this example, when you're reassigning T dot element to be the new element, the smallest element on the right side. The old the old assignment, like what element originally was, that? is gone. Oh, it's overwritten? Yeah. But is it in this instance? Is

1:12:36

it garbage collected at all? Or is it Joe? So okay, so t elements used to be a reference to the thing you were trying to remove? t element will become a reference to this other thing. And so if there existed other variables in the ether that referenced, what t element used to reference, it would stick around. If not, it would get garbage collected away. But we don't know or care about that from the perspective of just this method.

1:13:06

Good. Okay. Yeah. Cool. Thanks. All right. Karen, was that a question? Or statement? Um,

1:13:15

okay, cool. So we're good.

1:13:19

Okay. All right. So that was removed. And I think that puts us in a pretty good position here, the only thing really left to look at is the fine men in the fine max. So let's take a look at fine men initially. So just like with the others, we have a public one, which doesn't take anything. And then we have a private one that takes the root of the tree, or the root of the sub tree that we're trying to find the minimum of. So first of all, if the tree is empty, which just means if t is null, sorry, if root is null, then we just throw an underflow exception because there's no min to find. But if it's not, we call the recursive find min. On the route. We extract the node that's found to contain the minimum we extract its element. And then we return that element. Because the public one doesn't return a node. The public one returns the data, the private one. One we actually used as a helper over in our remove method, the private one.

1:14:40

DS Lecture 13 Page 19 of 21 Transcribed by <https://otter.ai>

Hold on one second.

1:14:45

The private one returns the node that contains the minimum and so it takes the root of the subtree that we're going to try to find them in a min and it returns the node that takes it now in this case, whenever turns the node that contains the minimum, we actually do consider the possibility that we're trying to do this on an empty tree. We could still have that case, if we were calling it from over here without the help of the public driver method. So if t is no, meaning we're trying to find the minimum of an empty tree, we just return null, you probably utilize something like that to do your lazy deleted version of this. Else, if t left is equal to No, we return T, because it's the leftmost element. Otherwise, we recurse on T dot left, because remember, fine men is supposed to just keep going left until you can't go left anymore. And that's the minimum. Anybody have any questions on that? Okay, now, one little side note about this. I've been sitting too long, I just crapped my leg up, that was not good. I really need to get up and walk around more good advice for the zoom world.

1:16:06

So

1:16:09

there is this concept of a tail recursion. And a tail recursion is when you have a recursive method that has as its recursive call the very last line. And it's just a recursive call to one step closer to the base case, no other computation at all. And so this is an example of a tail recursion, because you're not even multiplying it by anything, you're just returning the

result of calling the same function on one step closer. And so when you have such a tail recursion, it's very, very trivial to rewrite these things as a while loop. And so just as a way of demonstrating that, we could have written the fine max method, at the same way, just substituting T dot left for T dot right. But just to show that you can trivially redo find Max has a while loop. It doesn't do it recursively. If t is not equal to No, while T dot write is not no t equals t dot write, and it accomplishes exactly the same thing. Any questions on that? Awesome. Okay, so I think we've managed to go through the binary search tree code. And so that should help. You should view B abl tree code material. That's a modification of this, to see how AV ELLs are done. That video is already available in the video library. Your homework three written remains due on Thursday. Um, your homework four will be posted Thursday or Friday. But homework three programming, I will update this A little while later after my ta meeting. Homework three programming, you're actually

DS Lecture 13 Page 20 of 21 Transcribed by <https://otter.ai>

now allowed to submit as late as Monday. And you can even use late hours beyond that if you must. Okay. All right, folks, I'll send an announcement out to reflect the fact that the midterm was grades were posted in a little bit. I do want to finish cleaning up the few ones that need to be cleaned up before I send that announcement. But you can look if you want. So benefits of coming to class live right? You get to see it sooner. All right. I'll stick around for a few minutes to answer questions. Oh, there was one. so we can write Fineman with the same while loop is shown and find max. Yes. just substitute t left for t right. Okay. All right. So let me stop the share. Daniel, go ahead.

1:18:59

Hi, professor. I have a quick question regarding the written so I was thinking about it, I guess trying to generalize the cases. And I can't seem to narrow down to one. Is there a real outage due to that? Never seen it? I'm sorry. I don't. I'm not sure exactly what I'm sorry about that. Sorry. It's like the number of full nodes plus one is the number of weaves that proof. Yeah. Can we have two inductive steps for that? Because I've never you

1:19:27

there are multiple cases that you would need to deal with, right? Because there are two ways there are multiple ways of adding another full node.

1:19:36

Okay. And so it would be one inductive step or with two cases we still would break it down. Okay, great. Thank you. Thanks. Bye bye.

1:19:45

Take it easy. Anything else? In the private message, sure, just stick around. I'll stop the recording after I'm done with everybody else, anybody else? Alright folks, if you have nothing else unless you're the person that I was just talking to, could you step out of the room. Thank you, professor. Take care.

DS Lecture 13 Page 21 of 21 Transcribed by <https://otter.ai>

DS Lecture 14

Wed, 8/11 1:39PM 1:20:36

SUMMARY KEYWORDS

hash function, hash, collisions, hash table, hash code, password, table, hash value, calculate, insert, stored, size, add, lambda, character, key, original, integer, file, array

02:39

Some song Frank Sinatra why to

04:42

161

05:35

wrapped around a little bit there. Alright, let's get started. So

05:49

you guys have homework due tomorrow the written part correct. And we hit said and I believe I had announced that should probably turn my video on. Um, maybe there we go. Um, I had said that the programming we had extended the deadline to Monday correct? All right, good. So did anybody have any questions? On the homework briefly now? Okay.

06:32

Okay.

DS Lecture 14 Page 1 of 21 Transcribed by <https://otter.ai>

06:34

All right. So

06:39

last class, we had kind of wrapped up our discussion of avielle trees. And I then went back, and I did some examples of the code for binary search trees. And then I posted a video that went through the version of that code for the avielle trees is everybody. Everybody saw that correct? Okay, so I think we're pretty much up to date on everything, I do need still Oh, you solution set for the midterm, which should be coming within a day or so. And also, your homework set for will be coming very soon. Okay. So within a day or so, as well. And so what we're doing right now is we are working on chapter five, the hash table chapter. And so let's start by talking about hashing in general, and what is a hash or a hash function. So a hash is when you apply a mathematical function to some input of arbitrary size, and then it spits out an output that's calculated from that input of a fixed length. And so for example, in Java, actually, every method has Sorry, excuse me, every object has a method called hash code. In it, the object class actually contains a method called hash code. And when you print out an object that doesn't already have a two string, you're actually seeing the output of the hash code, which for an object ends up just being some play on the memory address, right. Um, and so baked into these objects is a mathematical function that calculates a fixed length output, that is our hash. And it is in Java anyway, it's an int. So it boils that data down into a 32 bit value. Now, there are a variety of uses of such hash functions and the various hash functions have kind of different properties that we would consider valuable. One property that we would consider very valuable is the likelihood of having so called collisions. And collisions are when you have multiple pieces of data, that when you apply the hash function to them, you get the same hash value the same hash code out of them. All right, so if two things hash to the same value, that's known as a collision. Now, of course, there are always going to be collisions, or in most cases, there are going to be collisions, because the idea is that you're taking something that could be quite big in terms of the amount of data in it and ran ramming it down into a fixed length output. So data is going to have to be lost in most of these cases. So if you have a 32 bit hash value, there are only two of the 32nd possible numbers that could come out of that. And so you've got something very big, which might have a lot more than to the 32nd possibilities. Obviously, there are going to be collisions, but you don't want them to be that frequent. And you don't want things that are very similar to each other, to clash to collide. All right, and we'll talk for a minute in a minute about why. Now, for certain kinds of

DS Lecture 14 Page 2 of 21 Transcribed by <https://otter.ai>

10:39

applications of these hash functions, having a hash function that is not reversible, tends to be very useful. What does it mean for a hash function to be reversible? Well, it means that if I gave you the hash code, you can easily recalculate what the original data was. So that would be a reversible hash function, if you could easily do that. So one that's not

reversible is one where it's very hard, or bordering on impossible to calculate what the original data was that resulted in that hash. Now, admittedly, if you're having a huge amount of data, and you're boiling it down to a single end, you're losing so much data there that the likelihood of being able to reverse it is pretty low. All right, because data is gone. So now that I've sort of described what a hash function is, what are some use cases that folks might think would be potentially good? For a hash function? What might we use these things for? So encryption, or some type of cryptography, um, so unique identifiers? Yes, that's actually very much how we're going to use them now. Fast lookups are actually quite related to unique identifiers. I'm going to pretend I didn't see blockchain. I'm sorry. Compress compress files. Yeah, so use them as keys into some sort of structure that's related to things like fast lookups, or unique identifiers. Okay. Um, so let's dive into the crypto aspect of a little bit. Oh, somebody next to cryptography suggested passwords. And so this is an example that is related to cryptography where passwords are actually very tightly related to hash code. So let's talk about that example, for a minute, just so we understand how these things can be useful. Now, for passwords. What's important to us here is to use a hash function that is very hard to reverse, very hard to calculate what the original data was for the hash. So let me give you a description of a scenario. So imagine you're writing a website, alright, and you're writing code for the back end of the website, and folks need to log in to that website. Now, if you didn't know better, and a lot of people seem to not know better, what they would do is that when somebody sets up their username and their password, maybe on their server, they'd open up a file or put in a database somewhere. pairs of data username and password and just store the password the password very good Paul, store the password in plain text on the server, the server is you know, it's a supposedly secure and so then when a user comes in logs in, um, you know, they'll they type their password, they type their username, they type their password, the site looks at the password looks at what the password should be just straight compares them. And if they match, good, they let you in. What is the problem with storing the passwords in such a way on this site?

14:22

Anybody doubles? What do you mean by doubles? I'm curious because it actually Okay, so if two people have the same password, that's actually not such a disaster. Because remember, you type in your username and your password so it's going to match the password stored for a particular username. Um, Alright, so let's see, if the site is hacked, everyone is exposed, someone can hack into database theory with info easily hacked, blah, blah, blah. Good. All right. So if they're stored in plain text, um, it's not so much that they're easily getting And it's not necessarily that it's much easier to hack. So one thing that you can say is, all right, this is stored privately on the server. And if somehow I'm, an attacker breaks in, and is able, yeah, there we go. If somehow Apache attacker is able to break in and access the password file, they're kind of already in. So at first glance, it doesn't seem it really that much worse. But the real problem here is that you probably use your email addresses the username, right? And so you put, and I know PSB 15. If that's if you're me, I hope you're not me. Because that would be strange. You put PSB 15 is your email address, and then they probably figure that's also likely your login for columbia.edu. Or if you your email addresses, people are@gmail.com or something like that. Don't ever email that address, I'll never respond to it even less than when I respond to with psba columbia.edu. Anyway. So there's a pretty good chance you reuse your password on this site. And on whatever email providers, there's a pretty good chance. And by you feel attacked, does that mean that's exactly what you did? And now you're, you're worried about it? Yep, there we go. or, even worse, maybe you use that email addresses, you log into your bank account, and oh, yeah, use the same password. Now, you're really screwed, right? Because now they can log in and try to access your bank account and possibly

succeed. You know, people tell you to never use the same passwords across multiple sites. And especially not to use the same password is like something important on something like random for exactly these kinds of reasons. And they're right, you shouldn't, but a lot of people do, and you really don't want to facilitate them getting hacked any more than already. So the way that this can be mitigated to an extent if you use a hash function, preferably one that is not reversible one where you can't trivially calculate the original data. So the way you would use a hash function in this circumstances at the site would not store the password ever, the site doesn't know what your password is, the site will initially take the password that you set up, apply the hash function to it, get the hash value out, and then save the hash value, not the password. So the only things stored is the hash value. So if it happens to be the type of hash function that's not reversible, if somebody breaks in and steals the password file, they can't actually convert that back to the original passwords. But then how do you log in? Well, you give your real password when you log in the site stores it temporarily, and calculates the hash again, uses the same function that it uses normally, and then it compares the hashes. And now we don't have to worry about the password file being stolen so much. This, by the way, explains why when you forget your password on a site, it asks you the reset process resets the password, it doesn't tell you what the password is. Because if the site was set up correctly, it didn't actually save your password. Now, if a site emails you back your password after you go recover password, you know, they're storing it in an unsafe manner, and never use that password again, anywhere. Okay. All right. Can anybody think of some potential

DS Lecture 14 Page 4 of 21 Transcribed by <https://otter.ai>

drawbacks to actually storing the hash, rather than the password itself?

18:58

any potential issues that might crop up? I will tell you truthfully, they're not that severe. But space constraints? Well, actually, hash functions typically smoosh things down, right? So if you can use a very long password, but the hash might be short, that's actually not such a terrible thing. Like for certain things, I actually use phrases like long phrases, with sometimes with various things substituted in. Alright, so now we go. Right? So using the wrong password that has the same hash value we'll get in. So now we're starting to get into collision. So the likelihood of that is quite small if it's a good hash thing. But here's the trick. There could be a scenario where multiple things hash to the same value collisions, like we were talking before. And so if we have collisions, right, then theoretically, there could be things other than your password that would work. But so what, right it's extraordinarily unlikely that somebody would guess the collision, but not your original password. And so really the only place I could see there being a problem is if two people entered a password, different passwords that hash to the same thing, maybe another user could accidentally log into your account. And even that's very unlikely if you use a good hash function that has a nice spread and doesn't have collisions very close to each other. Does that make sense? Alright, so there's other things you can use hashes for that are sort of in the security vein of things. And so another interesting thing that crops up is, so you may have seen this, maybe not, I don't know. But I don't know you go to a website, like I you see this a lot. If you're using like open source software, then you download a zip file that contains open source software. And usually next to the download link, you'll see like hash codes, or signatures, sometimes you'll see it referred to as signature. And what the signature is, it is the result of applying a hash function to the file. So remember, you're supposed to be able to take arbitrarily large things. And that will boil it down to a fixed length hash code. So you could have a huge file many gigabyte gigabytes long, but then apply a hash function to it and you get like something that's like 1024 bits, like a kilobyte. Now, obviously, you're never going to go back to the original file from the signature. But the signature has some useful things to it, it's a way of validating that you actually got the file you thought you were getting. So if somebody is distributing a program, this big

program, they could then sort of make a well, so calculate a signature on it, and then make that signature known widely. Right. So you probably wouldn't use them for usernames, because you kind of that one's not the secret, right? That That part's supposed to be known. So anyway, um, you might, you know, apply this hash function to this huge file, it comes down to this little thing, and this little thing will be the signature that's widely known everywhere. So you can easily look up what the signature should be. So then when you download the file, you can just apply the well known hash function yourself to the file and get the signature back. And the hope is, the signatures will match, that's a good

DS Lecture 14 Page 5 of 21 Transcribed by <https://otter.ai>

indicator that there were no modifications to the file. Now, in these cases, the idea good hash function for that are things where stuff, obviously, they're going to be tons of collisions in the world, right? But you want it such that things that are not that different from each other, have wildly different signatures, right, and you can design hash functions that would do that. And that's useful in these cases, because the real scenario that you want to worry about is alright, imagine you're going to some perfectly legitimate website, and you're downloading some perfectly legitimate copy of say, Photoshop, I don't know, something like that perfectly legitimate, sort of like the perfectly legitimate PDF of the textbook that you all have, right? Anyway. Um, so you, you have this perfectly legitimate copy of the file that you download it from some random website.

23:36

And in fact, it's not perfectly legitimate. It works exactly like the Photoshop that you were expecting you install it, you get Photoshop, it works. But somebody made a very slight modification to it, that records every keystroke you type and steals your bank account info. The next time you log into your bank, but works just like Photoshop. But if the signature is very well known, for this to really work, they would have to make its tiny modifications to the program to make it malicious. But then at the same time, well, I mean, if it's worth it, just give your bank account information to Adobe. So like, I did that, and it perks every month. Anyway, um, and then I charged it to Colombia, and I feel a little better. So I'm sorry. If you can make a very small change to that file, do something malicious, like steal keystrokes, but then otherwise, have it work exactly the same. As long as the signature that gets calculated from that little change is wildly different from the original signature. It's very easy to validate that somebody did something wrong. But if you could calculate it took something that would collide with the original file, even though there's only a tiny little change to it, then that opens it up. To a lot of bad things happening. So everybody good with that. Alright, so those are some use cases for hash functions that we see. But now let's, let's go to things a little closer to how we're going to use them. And so we're going to use hash functions to facilitate our inserts into a data structure known as a hash table. Okay, now, I should note

25:31

that hash. Okay, what happened here? Hash tables

25:37

are our sets, right? We all remember sets. And I think a few of you stuck around for my

DS Lecture 14 Page 6 of 21 Transcribed by <https://otter.ai>

little story about sets and unique keys, right? sets can't have duplicates in them. So a hash table, you're applying the hash function to a key. So every key in the hash table is unique. So there are no duplicates in the hash table. And so the way this ends up working, is you apply a hash function to the data you're trying to insert. And that boils down to a number, like an integer, which is what that hashCode method that all objects have. So you boil it down to an integer. And then you can use that integer as an index into an array. And so if you are very lucky, and your hash function has no collisions at all, every value, you attempt to insert into this thing, hash function, excuse me, gives it a unique location in the table, you suddenly have a very efficient way of storing data, the table is an array.

27:02

Right? And so if the hash function gives you the exact location where a value should be, I can insert something in effectively constant time. And by constant time, I mean, it's completely independent of the number of items in the hash table, it's just whatever the cost is of applying the hash function to the piece of data, that gives you the index. And that's it. And then you just jump to that location, write the value down, and you're done in constant time. And again, if you have no collisions, no, this is actually stored as an array. All right, so you would actually instantiate an array. Um, and then if you want to look something up in the table, to see if it's there, and possibly get some additional information stored in the object, if you just have the key, then what you can do is you can hash, the key that you're looking for, that will give you a unique location in the table. And then either it's there, or it's not, it's there, you retrieve it done constant time, it's not there, you know, it's not there. And so, in a perfect world, where there are no collisions in your hash functions, and inserting things into the table, we now have a data structure that allows for inserts and retrievals in constant time, instead of like the login time that we've been dealing with, with our avielle trees and our binary search trees that are balanced, which is really neat. And you have to think, Well, why don't we use hash tables for nearly everything? And well, I mean, there are scenarios where, like stored in disk or something like that, where you might not want to do that loss scenarios actually might not do that. But the truth of the matter is a lot of the time that is what we do, we do use hash tables. I wonder how on earth is Google able to if you give it a search term to like instantly pop up all the locations that happen to have that key term in it? Right, almost instantly. After it's indexed, like massive chunks of the web. It's not searching through that linearly. In fact, its database is so huge that it's not using the login algorithm either, because that still would be very long. No, it's using some kind of hashing in the background, because that tends to be much faster. Now, of course, it's not that straightforward, right? I said that the hash code returns an int, which has 32 bits of data, so two to the 32nd possible locations that could be filled in. All right, you're not going to create an array with 236 32nd locations. If each location in the array is an integer, that's four gigabytes. You're not going to do that if you're just

DS Lecture 14 Page 7 of 21 Transcribed by <https://otter.ai>

storing like 10 pieces of data. And so what you're actually going to have to do is you're going to apply the hash code and get an int. Now how would I squash that down into an array of a certain size? By my table size, the length of my thing here is seven. And so I apply my hash function, I get an integer value, how do I convert that into something that can safely index me into an array of length seven, some odd, right? You just mottet by the table size. Now, the moment we do that, like even if you happen to have a perfect hash function, one that has no collisions in terms of converting the inputs to a hash code, you're still going to have collisions, the moment you mod it by a small table size. So much of the conversation that we're going to have about hash tables is really going to focus in on how do we deal with these collisions in a way that allows us to preserve some semblance of order one time, this boiler spoiler is that we're not going to be able to preserve order one time in the worst case, but we might be able to preserve order one time in the average case if we're careful. Okay. All right. So let's talk a little bit before we get into a lot of details about, like actual strategies for dealing with hash tables and hash table collisions. Let's talk a little bit about good to hash functions. And first of all, a good hash function may differ a little bit, depending on what your use cases, like if you're using it for a password or something like that, you definitely want it not to be easily reversible. But for a hash table, I don't care if it's reversible, right? The only thing I care about, is to get something that has as few collisions as possible. Does that make sense? And so in our setup, we're going to have a function called hash code. Maybe we'll just for our sanity, we'll call it hash. That we want to return an int. Okay, that's the idea. We want to return it so a 32 bit. And there are different kinds of data that we could pass in and different

strategies that we can perform. To get an integer hash code out of a piece of data, let's say, I want to write a hash function for the integer class. So the integer class is storing and it can you give me

32:39

a perfect hash function for an int, one that will give you an absolutely unique hash code? For any throw at it. You just return the ID, right? Because in that case, the hash code is supposed to be an int, we don't care if it's reversible or not. And so each integer is unique. Just return it. So a perfect hash code for an int. If the answer is x, hash of x would just be equal to x. Right? That's perfect. Now, of course, once we have to mod it by the table size, then collisions will start to arise. But just in terms of the code itself, you can't do better than that, if you all you're worried about is collisions. Good. Alright, so now what we should look at are some examples of hash codes that we might want for, say something a little more complicated than an end. So let's look at hash codes for a string. So I'm going to actually show you a smidgen of the textbook from a completely legitimate PDF file.

DS Lecture 14 Page 8 of 21 Transcribed by <https://otter.ai>

33:52

Hold on. Just so we're clear, and I think I've said this before.

34:00

I must own a dozen copies of this textbook, like owned and paid for, there are at least two copies of them in my parents house that date back from when I was an undergraduate and there are like 10 copies of it sitting or seven copies of it's sitting in my office. Weirdly enough, there are no copies of it sitting in my home because of this PDF file, but I do own many. Alright, so let's switch this.

34:34

Uh, no PDF file to speak of not the one you're looking at or anything. Um, okay, so is everybody seeing this? Alright, so first of all, let's see what type page this is. Um, I'm not sure this page 173 if anyone's keeping track. So first of all, I should note that The textbook has kind of, I'd say, merge together two functions that I would have really preferred, they kept separate. And so let me just clarify exactly what it is, when you calculate the hash value for something in my mind, that's before you mod it by the length of the table, you're trying to insert it. But you'll notice that this hash function both takes the key that you are trying to insert into the hash table, and the table size, which is the length of that hash table. Now when I think of a hash value calculation, and this is in fact how the hash code works, I think of computing an int. And then once I have that int, i mod it by the table size, in this code, that hash Val variable, that's what hash code would return what that ultimately is. And then at the very end, you see me mod, that hash value by the table size. So is everybody clear, there are two separate steps. And what I want to think about right now is the part that goes into calculating the hash Val before we do the modules. So here's an example of calculating a hash code for a string. It's not a good one. And we will talk about why in a second. But we'll start the value of the hash code at zero. And then what we'll do is we'll have a loop that will go through the string input the key, one character at a time. And then each step, we extract an individual character using the Chara method, and we add it to the hash Val. And as you should remember, if you do an arithmetic operation with a character, it just treats it like an end. And so it's just adding the Unicode really ASCII because that we're using that subset of it. It's just adding the ASCII codes of the characters and the key together. Why is there some problems with this? We're trying to avoid collisions. That's really my goal right now. And so what's problematic about this? So many keys can add to the same, yes, that actually is a very big problem. Can you be a little more specific? The lower and upper case aren't actually a bad

DS Lecture 14 Page 9 of 21 Transcribed by <https://otter.ai>

thing that gives me a little bit of differentiation between strings. And so actually, well palindromes. So palindromes are an individual phrase or an individual key. Right? And so

what we're really talking about here is the relationship between the result that would happen if you had several with two different keys I get what you're where you're coming from on that. Um, alright, so two strings with the same characters, but different order the input is reordered, right? anagrams asked the problem, right? So if you try to hash dog, and you try to hash God, you end up with the same hash code. So anything that that that's an anagram would end up colliding. In the lessons, a lot of words are anagrams of other words, there are lots of potential collisions. So this isn't very good. And one of the ways to think about why this isn't very good, is because there's no attempt to encode positional information as we go through the string. If there's an A at position zero or an A at position 20. They contribute the same way. So let's look at another hash code also bad but for a completely different reason, that attempts to take some positional information into account. Alright, so palindromes are just a subclass of anagrams while palindromes are. Right? So a palindrome is a single word, the first half of the palindrome should be an anagram of the second half of the palindrome. That's sort of the way I would think about it. But yeah, I absolutely get where you're coming from on that. Um, Alright, so let's scroll down a little bit. That was the simple hash function. So here's one, we don't see the whole thing. Let's make this a little bigger. Or maybe you do see the whole thing and I just don't on my screen.

39:28

It's funny, I bought a 27 inch monitor, and I still managed to fill every space and things overlap. And yet, I was doing just fine on a laptop screen. Just once you get more you just fill up the space and then you want more. And you end up with my office, which has like three monitors arrayed out in front of me. Really, I'm very good at spending Colombia's money when they have any right now they don't seem to have it. Anyway. Um, so Christopher says so God, dog and og and yes, they would all have the same hash value for the original thing. All right, now, let's look at this one. So this one again, it kind of merges the hashing and modding by the table size into a single thing. And so here, it takes the key of char at zero, and then adds it to 27 times the key HR one, which then adds it to 729 times the key HR two. Now, somebody in the chat already caught what's wrong with it, but let's put a pin in that for a second. And just see what this is trying to do. What it's trying to do is it's pretending for a moment that the inputs are just A through Z all have the same case. In that case, you would imagine there are 26 possibilities for each character. So then it's just multiplying the first one by one, the second one by 27. So which is fully offsets it from the original 26 possibilities, and then multiplies the second the sorry, the third one by 27 squared. So it offsets it by all the possibilities there. So you've encoded positional information into each character considered, and that's your hash Val, and then

DS Lecture 14 Page 10 of 21 Transcribed by <https://otter.ai>

you modify the table size. But as Daniel points out in the chat, we're just doing that to the first three characters. So if I tried to hash the word pre, and then I tried to hash the word prefix, I would end up with the exact same hash code, because any words that have the same prefix would hash to the same thing, the first three characters, were okay with that, Why that? That's still too simple. Good. All right, so let's move on to something that's actually quite close to how hashing is usually done in Java for strings. So let's move on down to this next one. Let's make this a little bigger. Hold on. Alright, so here we go. This one was labeled in the textbook, a good to hash function. So let's take a look and see what a good hash function looks like. So in this good hash function, right, hash Val is what my sort of hash code is. And so I started at zero. And just like in the first one, I'm going to loop through each character of the key, so I don't have this prefix issue that I had in the middle example. And then so at each point, I take hash Val. And then I multiply the original value, the previous value, I should say, of hash Val, I multiply the previous value by 37. And so prime numbers are actually good for these kinds of things. Because you don't have so much in the way of common factors. So they're less likely to be things that that

sort of overlap. We'll see more examples of that, or it comes up, there's actually Well, I'll talk about that later. So we're gonna multiply hash Val by some sort of prime number, then we add the next key to it. Then we loop around to the next one, and then multiply that whole thing by 37. And add the next character the key to come around, multiply it by another 37, then add the key to it. So basically, the first one's multiplied by one, the second character is multiplied by the second step, it multiplies the previous ones by 37. The next step, it multiplies the previous ones, essentially by 37, squared, and so on. Does that make sense? So you're spreading it out. And then you mod the hatch, the bow that that thing that you get from doing this computation by the table size, but that actually input in injects an interesting little problem into our hash code. Yeah, it's basically combining the strengths of the two previous ones, where that sort of increasing power of 37 as each character, you add in codes, a little more positional information each time.

44:17

You will note here that we do something if hash value is less than zero. But here's the thing. How on earth is the hash Val ever potentially going to be less than zero? Like, ASCII codes are not negative? Right? overflow? Yeah. 37 times 37? I don't remember what what's 37 Square? Anybody was gonna say Is anybody in front of a computer? Sorry. Why am I waiting? Who cares? What 37 times 37 is, if you Okay, it's 1369. Great. What's 37 to the fifth By I, I know someone can calculate it. That looks pretty big. Okay? I don't even want you to try to calculate it. But imagine now 37 to the 10th. Okay? This is just getting monstrously huge, right. And you wouldn't need a very big key to reach those heights, because 10 characters isn't all that long for for a word. I mean, the average word length is, I think the average word length is five or six in a novel. The average word length for the

DS Lecture 14 Page 11 of 21 Transcribed by <https://otter.ai>

dictionary is 4.7. I'm sorry, I'm going to go off on a separate little side rant for 20 seconds. Do you know why I know the average word length in the dictionary is 4.7. I was doing, I run a high school program, or I help run a high school program. And we have an application that was due a couple days ago. And we asked them for an essay. And in the instructions, we say your essay shouldn't be longer than 250 words. But because of some various constraints on the system that I was doing, that we were using, I should say, we could only constrain it by the number of characters. So I gave them 1525 characters. And we got this really indignant email from an applicant right before the deadline, just so you know, 1500 25 characters is only 237 words, not 250. And they were very mad at me because they couldn't edit out those 13 words from their essay, which then led me to calculate a few things, write them back in extraordinarily polite email explaining what the average character length was, and then told them to edit their damn essay and moved on.

46:52

Alright, so

46:55

moving on.

46:57

So the problem here is now we have Okay, so when calculating 4.8 times 10 to the 15th. Well, thing is the max int is two to the 16th minus one, which is less than that. So eventually, very quickly, actually, you'll reach the largest integer representable. And so if I have the largest integer representable, by a two's complement, 32 bit int, and I add one to it, it wraps around, right, you think back to 1004 in two's complement, because a one followed by all zeros is the smallest number representable, the largest magnitude negative number. So once you add up over a certain amount, the integer wraps around to the negative side, and then adds up, adds up, adds up, adds up, and then wraps back around to the negative side again. And so if you're adding these huge numbers together using 32 bit arithmetic, it'll just keep spinning around the numbers, and you might land on a negative side, you might land on a positive side, there's nothing wrong with that, like we

said, a hash code is 32 bits in the set this situation. So there are only two to the 32nd possibilities, and we want to use all of them. And negative numbers are part of them. The problem is, is once we have to smash that together into a hash table, indices are only non negative. And so what we're going to have to do here is we're going to have to somehow convert the hash value if it's negative to a positive number that's within range. And so

DS Lecture 14 Page 12 of 21 Transcribed by <https://otter.ai>

instead of taking the absolute value, we do something slightly different just because it's cheaper to calculate. If the hash value is less than zero, then we just add the table size to it. Because remember, the maximum we would get after modding by the table size would be table size minus one. So if you're a negative number, and you add table size to it, you're guaranteed to get a value that's at most table size minus one. And that's what we return. Any questions on this hash function? We're good. Okay, now, um, I'm gonna do one more little side note before we get into hash tables proper. In Java, all right, you will find a number of classes and I'm about to write on something that you can't see. So let me stop the share. Or let me switch the share. So in Java, you will encounter a number of classes that make use of hash tables under the hood. I really need to replace this marker. Somebody can just unmute and laugh at me.

49:57

That was strange.

50:00

Did you have a question or no? No, I accidentally hit the button. Sorry. Okay, no worries.

50:11

That fed into my fears that everybody sits there just sort of hiding behind there off camera laughing at me all day, I realized I don't have to worry about that. What I really have to worry about is people sit there wondering when this doodle shut up and keep, like scrolling through Instagram or something. Anyway, moving on, you'll see the following three. So I said use a hash table under the hood, not hash hash table. So there are three classes that I think are worth bringing up. a hash table is sort of outside of the Java collections, the Java util hierarchy, really, and it just works pretty much the way I described, the key is the value and that's it, you're inserting unique values into the hash table, there's an insert, there's a remove whatever. Okay? The more commonly used one now, and this is actually the one that I'm going to have you use in your homework for is something called hash set. And so in Java, we talked quite a bit about the collection interface, if you recall. And in addition to the collection interface in Java util, there's also a set interface. And the set interface is like a collection, it's just every object has to be unique. And the entire object in the set is the is the key. So each object you've put into the hash set is unique. And I'll let you look at the Java API for exactly how to use it. It's very straightforward. And it's generic. And you can have it be a hash set of anything. So if you want a new hash set of integer, you can say something like, hash set

DS Lecture 14 Page 13 of 21 Transcribed by <https://otter.ai>

52:07

of integers, integer. H equals new asset.

52:21

All right. Now, maps, though, are a little different. Had we talked about maps at all, when we were talking about binary search trees, I'm forgetting. Okay, so a map is another type of utility structure in Java. Now, I know not all of you have had this. But I know a number of you are familiar with Python. So if you happen to be familiar with Python, you may remember there was a structure in Python called a dictionary, right? And so when a dictionary, what you do is you have pairs, right, you have a key, and then you use that key to look up the value associated with it. And that's what a map is. A map uses what are known as key value pairs. Where, sorry, what which bracket? Here? Oh, no, that's just an open and closed bracket, followed by an open and close parentheses, just what Mark said.

It's just the constructor for `HashMap`. Same way we do `LinkedList`. Leon, and we're good. Alright, so a `HashMap` actually has key value pairs in a hash set. The key is the value, every value is unique. In a `HashMap`, you have a unique key, but then you have a value associated with that key. And the values aren't necessarily unique. So you can imagine a scenario where maybe you use a hash map to represent, say, a grade book, right. And so the key is the unique identifier of the students. The keys are all unique, every student is unique. I may be overstating that a little bit, but every sight, every student is unique. But a lot of people got A's a lot of people got B's, fewer people got C's, but some but more than one, right? And so you look up using that unique key, and that gives you the value associated with that particular key back. Okay, so that's the idea there. So if you use it as a lookup, and then over a `HashMap`, there's the data type of the key and the data type of value and they don't have to be the same. So if you want to create a `HashMap`

54:49

you actually can you actually have to provide two generic replacements.

55:01

Let's `HashMap`s take to generics. That's the data type of the key. That's the data type of the value. All right? And how just comment, could you use a `HashMap` for usernames and passwords? Yes, the unique piece, the username, whatever data type, that is, that would

DS Lecture 14 Page 14 of 21 Transcribed by <https://otter.ai>

be the first type. and the value, which would be the password, and two people could have the same password, they can't have the same username, but they can have the same password. That would be the second one. So everybody clear on that on how what that is. So you're mapping from a key to a value, it's really hash set that we're going to use, okay? Because we're just going to be interested in an existence. So the existence of a value, not a mapping. And so joy, in fact, a hash set and a hash table pretty much work the same way. They're doing the same thing. It's just that set is an interface that exists in `java.util`. And lots of different things implement sets, you know that binary search trees are a set. And so in Java, for example, there's also what's known as a tree set, right, which also implements the same methods as the hash set, it just under the hood uses a binary search tree instead of a hash table to represent it. So this is using a hash table to provide a very specific abstract data type known as a set. a hash table is just sort of the structures sort of separated out from that interface, but it's doing the same thing joy, does that make sense? Cool. So that's how we'll encounter hash tables in Java. But what we really need to do is we need to have time, okay, oftentimes, what we really need to do is we need to talk about the various ways of actually implementing these things under the hoods and want to implement our hash tables. And as we've said several times, and we have a perfect hash function, that would be great, except the moment we deal with a finite length, table size, and the moment we mod by that table size, the likelihood of getting collisions becomes very high. So really, what we have to deal with here is what happens if two things happen to hash to the same location. And there are a number of ways we can deal with it, the first collision resolution strategy. So these are the collision resolution strategies. The first one we're going to use is something known as separate chaining. Okay, I spell separate, right?

58:06

Help? Yes, no?

58:10

Okay, good. Um, so the first sorry, you guys hold on for one second, plus,

58:27

I lost the captions or other people seeing captions. Oh, there they are. Sorry, that was strange. So I'll make sure everything is working correctly. Alright, so um, separate chaining. And so what we're gonna do, I'll get into what that is in a second. But let's, let's let's, let's give some examples that lead us up to our first collision. So first of all, for the sake of

simplicity, let's say that we are inserting as our values into our hash table. Let's say we're inserting integers. And we know a perfect hash function for integers, right? The hash of x equals x . So that means if I have a hash table of length, table size, if I want to insert something into it, it take the integer and want to insert, I apply the hash function, which gives me the same integer back, and then i mod it by the table size, and that tells me the location. So for this example, what we'll do, because modding by things in my head is not that hard, but I have a large audience here. So I'd like to do this easily. So let's mod by 10. So let's create a table size of 10 elements. So here's my array, so my table is just an array.

59:48

So if we're going to have a 10 element table, we're modeling by 10. The values we get are zero through nine, which are the correct indices. So let's try out this table. That was wrong. There we go. Okay, so let's do an insert on this thing. Let's insert, insert 75. So we apply the hash function to 75. Because we're using integers, we're going to use this perfect one. So 75, the hash of 75 is just 75. And then we mod it by the table size here, which is 10. So what's the location that I would want to insert this, I put it at location five. So I'll drop that into the array there. Now, we can insert some other stuff. If I insert 16, what location with that one want to go into six. Let's maybe insert one more. Before we mess things up, let's insert 41. And that'll go in location one. So far, so good. But now to mess things up a little bit. Let's insert 25. So if I insert 25, and I go through the same process that I did before I apply the hash function, I get 25 back by modded by the table size, and I tried to put it in location five. But wait a minute, there's already something there. So what we do in separate chaining is we actually, we don't store the value directly in the slot of the array, this ends up not being an array of integers. Instead, what we do is we make each element in the array, a linked list, and we can call it a Java linked list. So we can assume that it's a doubly linked list, but it really doesn't. Hold on one second.

1:02:03

You see what matters.

1:02:07

Somebody rang our doorbell. That's very strange. So why for 16? we inserted six, not 16. So there is no 16. Right? Um, the array size is only up to nine says of 10. And so we have to take the hash value get and squash it down into the size of the table. Does anybody there? Apparently a package. Ah, okay, sorry. Oh, sorry. I got thrown off there. Hold on one second. Just sorry, hold on one second.

1:02:56

Sorry about that. Um, I can explain what happened there after the class is over. So we can have each location in here. So Lily, does that. Explain what you were saying? Yes. Good. Okay. Um, so each location here is going to be a linked list. So now I'm going to insert these values into the link list stored there. So each location in the table just has all the values that happened to have hash to that slot. So I'll tack 25 on either at the beginning or the end of the list. Since I'm using a doubly linked list, it doesn't really matter. And so what is the worst case bego cost of my inserts. A Bye. quarter one. It kind of is based on the way I described it, but in fact I left something off that makes it have to be ordered. And what is the worst case length of a link list in any given slot in the table. The worst case is in like if I have a 10 element array, and I answered 7585 95 105 and so on and everything ends at a five they all land in the same spot the way I have this thing set up. Now, Hunter I think was probably going by the fact that I made a big deal of it being a doubly linked list and saying you could add a spy I'm gonna go ahead and mute everybody. Alright. Um

1:04:45

so uh

1:04:49

how that was, that was interesting was that we had a ghost Come on. Quietly whispering who Okay. Oh, nevermind. Okay. Okay, so a doubly linked list I could have added at either end in order one time. But why isn't? Is it the simple? Is my just tacking it on at the end of the beginning? anyway? That I just forced me to everybody. So you can't unmute. Yeah, I'm allowing people on mute themselves. What Why? Oh, okay. So, let's figure out why first, and then I will make a comment on Dune. Alright, why is it still ordering? There's something important going on here that we need to realize?

1:06:02

Anybody?

1:06:08

If we all just given up for the day? quest. I love it. All right. So we were insert? Yeah, so was I, we were inserting into this thing. And we said, we're adding the element to a doubly linked list. And normally, if I added an element, either at the beginning or the end, it

DS Lecture 14 Page 17 of 21 Transcribed by <https://otter.ai>

would cost me no more than order one, even if all the items were on that list, and it was length of n . Okay. Why? Why do I still have to go through all? There's nothing on this page that answers a cue. Yeah, we have checked for repetitions, right? Remember a hash table as a set. And you're not supposed to insert duplicates. So even though I could add the value to the list cheaply, I don't want to insert duplicates into the list. So when I do an insert, doesn't matter if I'm putting it in an order or not, I still have to go through looking for the item to make sure it isn't already there. So my inserts is worst case. order one, because she's order and because we have to go through every element to make sure it isn't already there contains checks.

1:07:45

So let's say I do a contains on 65, we go through the same process, we're going to apply the hash function, we'll get 65 back, we mod it by the table size, which will give me five and then I have to iterate through the list looking for the 65 whether it's there or not, it's still gonna cost me order. And time. In the worst case, if I got unlucky, and every item happened to have landed in the same bin, I mean, I couldn't put it in an order. And that might make for a slight improvement, you could put a BST in there. And it might make for a slight improvement. You know, I'm not going to really talk about how to do a BST in there because as it turns out, I'm going to leave that part as an exercise to you possibly on a final exam or something. Okay, so let's stick to link lists for now. Okay, then you can think about maybe how things might change. But assuming these are just linked lists, these are going to be order and in the worst case, which is a far cry from the constant time that I advertised when I started. So now, what we need to do is we need to talk about another property of a of a hash table, which is known as the load factor. So the load factor, also known as λ , lowercase λ , is the ratio of the number of items inserted over the table size. So ratio of the number of items inserted over the table size. So now I have a question. Is there a maximum value for λ if you're using a separate chaining hash table? Anyway. Well, one sounds nice. While you figure there are 10 slots in the table, we put 10 items in. But the problem is all right enough with Orange Julius, the problem is that how many items Could I theoretically put in a given bin? If each bin is a linked list, maybe arbitrarily large, right? So I can put as many as I want. So λ could get real big. Okay. Now, here's the trick. Another way of thinking about λ , in the case of separate chaining, is that λ is effectively the average number of elements for bid per event, right? Even if all the elements are in one bid, on average, you know, it's the number of elements in that one bin divided by the number of bins, that case most of them have zero and one as n , right? But the λ is the average number of elements per bin. So if I constrain myself, such that I keep λ , less than or equal to one, what is the average number of elements prevent?

1:11:45

One. So that means since at any point, if I perform an insert, or remove a contains check, I have to go through all the items in the bin. if λ is less than or equal to one, then on average, I would have to go through one item in a given bin on average, some will be more, some will be less, but on average, it's one. So in a separate chaining hash table, if I keep λ , meaning I keep the ratio of the number of items I insert to be less than or equal to one, then my average case cost ends up being order one. So the average case, sorry,

1:12:39

is order one. As long as we do that, well, if we have 10 bins, and I say there are 10 items in the thing, on average. Right? There's one item per thing. The average case for all items, because all items have to go through each location sorted through each position in the linked list. Does that makes sense? So contains you go to a location go through good. Alright, so that means as long so yes, worst case, still absolutely ordering. We're not getting away with that away from that. But our average case, as long as we maintain λ less than or equal to one is constant. So that means hash tables that you separate chaining, as long as you keep the λ at a reasonable level, aren't going to like instantly give you the result for all of them. But on average over many operations, it will still be much, much faster than say a binary search tree. Does that make sense? Alright, so now the last thing we need to address is how do we keep the load factor? How do we keep λ less than or equal to one? Well, if the table size is 10, and I add 10 elements, I'm still good. And I could add an 11th element to what should I do when I add that 11th element? Yeah, yes, it's the tables. I resize the table. And so the operation for resizing the table is what's known as in rehash. Because what you do is you allocate the new table, that's roughly double the size of the original I save roughly for reasons I'll get into in a second, but you roughly double the size of the table. And then you go back through the original items that were in the table, and then you re mod their hash values by the new table size, and that will spread them out further over the new now enlarged table. And because you've doubled the table size λ now half's good So the table size will keep growing as you insert things, and generally speaking, you will end up having to rehash every table size number of inserts. Is there a trade off between increasing λ , increasing the Lam, increasing λ and increasing the table size? I'm really one is our trade off point, things degrade very quickly once it goes over one for reasons that I'm not going to go into. Hold on one second. So somebody asked me, Isn't it dangerous to have the hash and the value be the same? So to the person who asked that privately? The answer is really, it depends. On the use case, if you are trying to use your hash code, for the purposes of storing a password, it's a terrible idea. But if you're trying to

use it for inserting it into a table in a unique location, it doesn't matter. It won't cause any problems. Okay? All right. So, um, we talked about rehashing the very last point that I'll make is that I use 10 is my table size, because it's easy to mod. The truth is you want to have a table size. And we'll we'll talk a little more about this another time, you want to have a table size that minimizes the likelihood of things landing on the same spots that really spreads things out as much as possible. And cause they're, they aren't factorable. Really good table sizes are prime numbers. And so it's not required for separate chaining, to use a prime table size, but it's a good idea. I just didn't feel like modeling by 11 in my head. It's not gonna stop me from having you mod by 11, and you had on your exam, but I didn't feel like doing it right now. Okay. And so you probably will go into that later. But you want to use prime numbers, when you double the table size. You don't want to double it exactly, because then it's not prime. So you double it, and then you go up to the next nearest prime. And what prime numbers will do just generally, is it will keep things spread

out as well as possible. Okay. All right, folks, I think that's it. Um, keep working on your homework, we will continue talking about hash tables on Monday, you should read up through the probing hash tables, and maybe a little bit about rehashing we've got section 5.5. And then the last thing I would say is we're going to start the next topic priority queues towards the end of next week. So if you'd like to start reading ahead, you should start reading chapter six through section 6.3. Have a good one. Thank you. Thank you.

Thank you.

1:18:11

So Cecilia, you're reading chapter five up through the section that's titled rehashing. I don't remember the exact number. Okay. I would have much preferred you're watching dude.

1:18:29

Do you mind if I ask you a quick question about like, a rehash? Like so when you're modding? Do you mod like, so you might like a four and then you mod it by 11? Or do you remod the original number by the new size.

1:18:44

So the approach is you take the original number, you apply the hash function to it, whatever it is, and then you mod it by the size of the table, whatever it is at the moment, and that tells you what location to put it in. So if you have the value, say, I don't know, a 23. Right, and you try to insert it, first, you would apply the hash function to it. And that would put, you know, let's say the initial table size is 11. Right? So you mod, so you apply the hash function to 23. You get 23 back, and then you mod 23 by 11. So 11 goes into 23 twice with one remainder. So the 23 would go into Location One, a great loss. Okay, now we insert a whole bunch of other stuff. And eventually the load factor gets too high. So we rehash. And so when we rehash, we're going to create a new table that well, it was originally 11. So we double it, so that's 22. But then we want to go to the next highest prime, which is actually 23. Makes sense? Yeah. Okay, so then we go back through all the numbers and we apply the hash function again and modded by the new table size and put it at that location. So I go through the old table and I find that 23 I inserted before we apply the hash function, which gives me back 23. And then I mod that by the new table size, which is also 23, which means I would get zero. So instead of being in location one where it was in the original table, it would go in location zero and the new rehash table.

1:20:28

Okay, great. Thank you so much. Okay. Thanks. Bye. Take care.

DS Lecture 14 Page 21 of 21 Transcribed by <https://otter.ai>

DS Lecture 15

Wed, 8/11 1:42PM 1:19:24

SUMMARY KEYWORDS

probing, insert, size, hash function, table, linear probing, location, hash, equal, rehash, lambda, prime, items, chaining, duplicates, squared, linked list, question, quadratic, call

01:25

Oh you shouldn't be so excited. I read he was delighted. So we went in Belsize Park. Oh, you shouldn't have kissed me. Dude started. But then I found out that you're serious. You lied about your status. You lied about your life. You never mentioned your three children and the fact you have a wife

02:07

while I was telling you to go really badly, because you're very nearly had pity on my heart I

might never have found out you lied about just you lied about your life. You forgot you have three children you but God is not in my nature.

02:42

Sometimes

02:56

the object

02:58

is bound to turn out to be a cereal box had been mentioned my kids. I've got way too many. But I am friends single. Cuz he lied. He lied about his life. He forgot he had three children.

03:26

Got it. All the Way Home call my friend. cried down. I played some tragic music and I lay down today. But later I

03:44

tested off the seats. But I'm not a piece of meat. So I'll be the one down in time. Rachel now you've been going still it's England.

04:51

Okay, we should get started now. Maybe I'll even turn my video on. Hold on. There we go. I am the only person with my video on which is kind of funny, but no, that's changed slightly. All good. Like you don't have to feel obligated I just I just noticed. Um Okay, uh let's see, where are we here? All right, so you have a programming homework that is now due tonight. Correct? Since I already pushed that once. Huh? Oh, that's why the cameras off. You're trying to wrap up laundry. I'm like, I think you've typed that in the wrong window, sir. But nevermind. So I can't extend that any further. I'm weird were readying for for release, it will be not like three in the sense that I expect that the difficulty levels will now be sort of tailing off. I think three was sort of the pinnacle of the amount of time these things can take. I think now it sort of levels off. Now, homework five, will probably be about half the size of the remaining homeworks. And you'll have less time to do it, because I really want this not to bleed out into the finals. But I think we're good there. I did eventually managed to catch up, I think with everything but a couple of random private posts on Piazza. So we're, we're good there. mercifully. And you can use office hours on office hours, what am I saying you can use late hours on this. So if necessary, you can submit it as late as Wednesday, but be aware homework for you should really be starting to work on almost as soon as I post it tonight. Okay. Any questions on anything before I move forward? Problems anything we need to discuss? Okay.

07:24

So, last class, we were discussing hash tables. And we were discussing how do we deal with collisions in hash tables, right. And the way we decided to deal with that was to use separate chaining in our hash tables initially, that is, each location in the table was actually a linked list. And then we could add as many nodes as we wanted to that linked list. So that meant there was no cap really to how many items we could add to the whole hash table. So our load factor in recall the load factor, λ was the ratio between was the ratio between the total number of items you've added which in the separate chaining hash table was unlimited. And the total number of slots in the table, which was really just a count of the number of, of the number of linked lists in the table. As a result, we needed to do something to kind of keep them from getting too big. And what we noted was that, under those circumstances, the load factor amounts to the average number of items per cell, which means that if we can keep that load factor under one, our average case, cost is still order one. But there are other ways of dealing with collisions. And one of the ways of

dealing with collisions is I've heard this referred to as sort of open addressing schemes, which essentially means that each location in the table is now constrained, such that it can only hold one item. So no binary search trees, no link list, nothing hanging off, there is one item per location. And because there's only one item per location, now when we collide, we're going to have to do something other than just add it to a link list. Shop. How are we going to deal with that? Well, we're going to deal with it with is something known as the probing schemes. And so for probing, what ends up happening is the first place you try to do the insert will be what you expect from all of the other hashing we've done. You apply the hash function to the data that you're trying to insert since We were inserting integers, our hash function would just give me the integer back, and then you mod it by the table size, you go to that location, you put it there if that space is free, but then what to do if that space isn't free? Well, one thing we could do is think, well, if the value if you have a 10 element hash table, and the value you're trying to insert is 55, with put that 55, and location five, and now if the next location that we tried, sorry, next value, we try to insert as 15. Um, well, you try to put that at location five, two, but if five is filled, okay, then we'll try it at six. And if six is filled, then we'll try it at seven, and so on. And so that's a sequence of probes. Now, one of the ways we can refer to those probes is the H_0 probe, followed by the h_1 probe, followed by the h_2 probe, and so on, up through let's call it, the $H_{sub\ i}$ probe. Alright, so up to an arbitrary amount. And so this is the sequence of locations in the table, that we would try until we find one that's free for our insert. Okay. And so now the question becomes, how do we calculate this sequence of probes now, we already know that we want the H naught probe, the first thing we try to be, where would just you would put it if you just modded the result of the hash function by the table size. And now for the example I just gave you, then the next thing we would do is we try one over and two over and three over and so on. That's a specific type of probing called linear probing. But more generally, we can express our probing function as follows. We calculate the $H_{sub\ i}$ location by doing the following, it's equal to the original hash function

12:12

DS Lecture 15 Page 3 of 19 Transcribed by <https://otter.ai>

of x . All right, and so that's, you know, in our case of integers hash of x is just equal to x . But you know, if it's not an integer, that could be something else, our original hash function of x now, plus some arbitrary, not arbitrary plus some function, f of i . That's my probing function. And there could be many things. So depending on how we decided to do our probing, that's going to tell us how we define F . And then what we do is we take that whole thing, and we mod it by the size of the table. So for table size, I'm just going to refer to it as T going forward. Now, what you might note is that when I sort of talked through that linear probing where we try it at five, then six, then seven, then eight, we try it at nine, it still can't go in and it's a 10 out in table, you try it at 10, you'd actually want to wrap back around zero. So you want those probing steps to be modded by the table size as well. And so you can actually factor the mod table size out. So you take the original hash function, add the probe, and then only mod it by the table size once. And that is our general probing function. Now how we apply this will depend on what we decide for our F . So one type of probing is linear probing, where if the location you want to put something in is full, you just try one item over and wrapped around as necessary. So in linear probing, that's simply the definition of F such that f of ϕ is just equal to i . So for the H naught probe, we add zero to the original hash function, which is the behavior we want. We want the H naught one just to be the original hash function on the table size. For h one, we're adding one to that location, and so on. Okay. So is everybody okay with the general probing equation that we have here? And what we're going to use is our probing function for linear probing, are we good with that? Okay, so now that we have that, let's start using it. To do an example of insertions using linear probing. And then we're going to have to talk a little bit not just about insertions, but we're also going to have to talk about

contains checks and we're also going to have to talk about removes, and there are going to be some complications. So right now we're doing linear probing. It is always good to have your table size be prime. Um, for separate chaining, I said the same thing. But in the case of separate chaining, I said it wasn't necessarily required. And in linear probing, it's the same thing. It should be prime, but it doesn't have to be. So for my own sanity, I'm going to make it not prime, I'm just going to make it a table size of 10 for the moment. So let's create a table of size 10. So that we're going to have indices from zero to nine.

15:46

Okay. Draw this table. Okay. All right. So that's our table size. And sorry, it's just this is sort of when I would have dinner otherwise, I'm starving. My wife's eating, and I can smell dinner. But no, I have to keep going. So forgive me if I start eating a sandwich somewhere in the middle of my probing. So moving on. Actually, that would be awful if you hear me chewing over this microphone, my god. Okay. So let's start by doing some inserts into this. Now, again, we remember that for integers anyway, which is what we're dealing with here, the best hash function in this circumstance is just going to be hash of x equal to x . Okay.

DS Lecture 15 Page 4 of 19 Transcribed by <https://otter.ai>

So if I'm going to insert 55, well, hash of x is 55. And the first location that I try is the H naught location. So f of zero is just zero. So I add zero to 55. Get 55 back, and then mod that by the table size, which in this case is 10. I get location five and I put it in, it's empty, we're good. Then, let's do an insert of say 15. Same deal. 15 plus f of zero is just 15. Because f of zero is zero, 15. mod table size is five. Oh, we have a collision. So now I goes to one. So f of i is one because f of i equals i . So now I do 15 is my hash function, plus one is 16. mod, the table size is six. And then I'll go ahead and insert that there. Okay, let's try inserting 76. What location is that going to try for the H naught location? Gonna try location six. So we have a collision that got triggered by some of the earlier inserts, and somewhat insultingly earlier inserts of things that did not originally hash to that location, so it kind of got its spot stolen out from under it. So where do we try next? Anyway, we're gonna try location seven, and that's clear. So we can put 76 there. Um, all right, let's insert. So let's just show how this can get a little bit messy. So let's insert 25. And we're going to hit here. So the H naught collides, then we're going to hit here, so the h_1 collides. So then we're going to hit here. So the h_2 collides, and now we got to go to h_3 . All right. And so now let's insert 99. mercifully, that one doesn't hit. But this actually uses a slight problem. Now, you'll notice that there was kind of a cluster growing here, right? Every time I hit something in this cluster, it kind of grows the cluster because you're just taking a single step at a time. So now, let's insert five

19:54

and all of a sudden we're in real trouble because first we're going to try it five then six then Seven, the eight, the nine, and then that's not going to work. So now I'm going as 01234. So now I have to do the H sub five. So that's five plus five is 10. mod the table size. So where do I put this five? What location, it wraps back around zero on the modules. So this actually kind of illustrates a problem with linear probing, right? The problem with linear probing is that you get what is sometimes referred to as primary clustering. And in primary clustering, what you're essentially saying is that every time you insert something that has a collision, it kind of grows, this little cluster of nodes that it had a step through, before it gets another one. But the next, open space and fines will be attached to that previous cluster by the very nature of this being linear in the probing, so every time you hit a cluster, it grows in size by one. And every time it grows in size by one, there's an even better chance that you hit a cluster the next time. So there is some some explanation in the textbook that I'm not going to go through here, that says that also on average, the number of probes that you would have to take is order one, the logic behind that is a little bit less intuitive than the logic for separate chaining, but it could be still order one, but there is a new constraint. First of all, as long as you assume there's no primary clustering,

meaning you have a good hash function that spreads everything out very well. So

DS Lecture 15 Page 5 of 19 Transcribed by <https://otter.ai>

assuming no primary clustering, which is frankly, a terrible assumption. But if you assume no plot primary clustering, and you keep λ , that's the load factor, less than or equal to point five, then and only in those two circumstances, you can't really control this one. The average cost is order one. So in reality, because there's no way we're going to be able to guarantee this linear probing is not a good choice. And we're not going to use it very much as more than just demonstrating the whole concept. Okay. Now for λ . In separate chaining, we actually wanted λ less than or equal to one. But now one is actually a hard limit to the value for λ , right? Because there's only one item per space. So now the numerator can only be at most the table size, so the largest λ could ever be is one in reality. In reality, we want to keep it below a half. Now, this is actually becomes a requirement for items to even be guaranteed to be inserted in the next type of probing we're going to look at, but for now just understand that less than a half is what we need, just to keep the time complexity down in the average case. Now, just like we had with separate chaining, in the event that the λ gets too big, then we're going to have to do a rehash. And the rehash is where we double the size. And again, because we're usually trying to go for prime, we double the size, and then kind of round it up to the next time next prime. And then we go back through the initial values in the table in the order in which they're in the table, not the order in which they're inserted. And we hash them into the new table using the same functions we did before just with the new table size. And that should spread them out more. Is everybody good so far? Now, then, I think so. I'm not actually sure about that. 100 sounds like it could be anyway. So let's see.

24:41

I really want to call it llama now. So if λ is the number of items over table size and why does λ will point five non sure items will be inserted. Okay. So are it does, it doesn't matter for linear probing, right? Because it's As long as there are spaces, you'll find it because you're taking a single step at a time. So you're right, what I was saying was that λ over one half causes a problem for a different probing function than the one we're seeing here. And so the next probing function, I'm not there yet, but I'll just preview it, the next probing function we're going to do is for what's known as quadratic probing. And in quadratic, probing, f of i is equal to i squared, rather than i . So that means h_0 is the same as it was before h_1 even is the same as before, because one squared is just one, but then h_2 jumps by four spaces, right, and then each h_3 will jump by nine spaces, and so on. And so the jumps get larger and larger, which gets rid of the clustering issue. The problem is, it can get stuck in this cycle, where it keeps cycling through locations that are already filled. Without, even though there might still be spaces available. And there are several things that have to be true to guarantee that it won't get stuck in the cycle. And one of those is that the load factor be under a half. But that's for a conversation towards the end of this lecture. Okay. All right. So we've looked at the inserts

DS Lecture 15 Page 6 of 19 Transcribed by <https://otter.ai>

and the inserts is clear, you try the h_0 , then the h_1 and so on until you find an empty space. And once you find an empty space, you put it there. But now the question is, how do we do it contains check? Well, it contains check is basically the same operation that you would expect to be performing for an insertion, right, you're going to keep trying each of the h serves. Starting at the h_0 location, until you either land on a location where the value has been found, or be when do you know you can stop probing? They might?

27:20

Well, all the way through once, that's sort of guaranteeing that if it's not there, we're going to have an order in. And so we need some way to stop first. Um, now when the hash function wraps around, that's not good enough, because we might start at the middle

point. In this case, that's true, but it's not true. In all cases, um, it's not so much when you're out of a primary cluster is when you're out of a cluster. But how do you know when you're out of a cluster? Well, as several people said, and Kelly managed to shout when it comes to an empty spots, right, so when it comes to an empty spot, then we're going to stop. All right, the same way we decide to stop when we're going to do an insert, right, because when we're going to do an insert, when we come to an empty spot, that's when we know we can write it down. And so we're going to use the same thing for our contains check. Now that's going to cause some problems for us down the road. We'll see that in a second. But let's just try this. So contains five first here, then here, then here, then here, then here, then wraps around zero, finds it done returns true. We can do another contains. Let's say we do a contains check on seven. So the H naught location for seven ply the hash function to seven he gets seven back, add zero to it, modify the table size we try here first. All right, it's filled. So then we go to the h1 location, try here, it's filled h2 location, try here, it's filled h3 location, try here it's filled h4 location, it is empty. And now we know that if we were going to have inserted seven, we've checked all the possibilities up to this point. So if somebody else was trying to insert a seven, we would have put it here. So we know now that we're at an empty spot, we can safely return No, God, sorry, false. This, however, brings up a complication for the last method that I haven't yet discussed. So we haven't yet discussed the Remove method. And this is where things start to get messy. Let's try performing a remove on the 25. Okay, so let's say that's what we're trying to do try to perform the Remove on the 20 Five. And when we perform the Remove on the 25, your first thought should be really is perfectly reasonable first thought your first thought should be to simply null out that location and make it an empty space. Right? Because if you know that out and made it an empty space, no, then that gives you more space to insert other things. But there's a problem. If I know that out the 25, meaning I searched for it got here and just empty that space and made location eight be empty, then my contains check for the seven would have broken, right? Because what would happen, the seven

DS Lecture 15 Page 7 of 19 Transcribed by <https://otter.ai>

would try here at the seven spot, find the 76, then try the next spot, which would be the eight. But if I removed 25, from that spot entirely, it would find an empty spot assume that the seven would have gone there if it had been in the thing, and would return false, which, frankly, is fine, because seven isn't in there. But let's pretend I said the right thing and said we were searching for five. If I searched for five, we would eventually make it to that location, return false, even though five was still there. Is everybody seeing what the problem is with just knowing the spot out? So anybody needed clarification on that? Yes, no? Okay. Great. So I can't just knock it out. What do I do.

31:44

And a variable was removed, and somebody said, I feel like we're moving towards lazy deletion, we are very much moving towards lazy deletion. Right? Even a placeholder variable would be effectively the same, because we're sticking something in there to indicate that the spot is filled. Now, this is a good thought. And this is how we're going to do it. A lot of people also sometimes suggest, well, why don't we reorder all the items later in the array. And my answer to that is, that's very complicated, because they don't all necessarily hatch to the same spot initially. And that would end up costing order. And in the worst case, I could construct something where that wouldn't work. So what we're going to do for our remove is lazy deletion. So there will be some metadata associated with each location in here. So each location in this hash table will end up being a sort of an object that contains both the data and the Boolean as Omar suggested, and the Boolean would be is valid. And so all the items as we insert them would be marked as true for is valid. And so when I remove 25, what I actually do is I first search for it, and when I find it, I just set is valid to false. So the 25 is still there. Um, but okay, that 25 is still there. But now there'll be a little mark in there indicating that this value is no longer valid, and

therefore we're not considering it. So now by doing that, if I did a contains check on the five, it wouldn't look at whether or not things are valid, right, it just looks for an empty space before stopping. And so the five would go here, then here, then here, then here, this box still has thing in it, valid or not. So with no to keep moving, until eventually it gets here, and we're good. We find it and we're true. Now after I've removed 25, there is a question of what to do when I do a contains check on 25. And just like before, when you switch to lazy deletion, you're going to have to modify the other methods to deal with that lazy deletion. So for a contains check on 25 it would start here, go here, go here, go here, and it would discover that the 25 is in that location, so that would be fine. And then what it would do would be like alright, I found it. But before I say that I found it. Let me check to see if it were valid. And it says Oh, it's not valid. So I just go ahead and return false. Now, I want to remind you that hash tables are meant to be sets, values can only appear in the hash table once and so this is the only spot where the 25 could be. So if I were going to re

DS Lecture 15 Page 8 of 19 Transcribed by <https://otter.ai>

insert 25 after deleting it Laser Lee? Well, I would discover that the 25 is already there. But then I would discover that it was marked as deleted. So what can I do?

35:14

If I were trying to re insert it, just flip it to be true again, and I'm done. That works if I'm reinserting, the 25. Let's say I didn't do that. Now, let's say the next thing that I do is I do it contain no, let's say the next thing that I do is I do an insert on a, how about 47. So we do an insert on 47 bH naught probe would be here. And we'd see that that was filled. So now I do my first step. And I get here. So I discover that 47 um, could potentially go here. What? Sorry, I said that wrong. I said, discover that 47 could potentially go here. But wait, this location is filled? Do I overwrite it? Because it's not valid to reuse the space? Or do I just move on and insert it at the next truly empty space? Which one do I do?

36:35

So over so overwriting it is an extraordinarily tempting thing to do, because you're wasting space by having this extra thing here. But it actually causes a problem. In this case, I could, but not all cases. Yeah, it's not so much that it ruins the order, that would still be alright. Here's where the problem is. Imagine I didn't insert 47. Imagine I tried to insert five. If I try to insert five, I would first try here, then here, then here, then here. Now if I decided to overwrite the five in this spot, why is that a problem? Right? So somebody pointed out in the private comments, repeated elements. If you overwrite a lazily deleted item with a newly inserted item, you can only safely do that if it's the same item that was deleted. If it's a different item, you have to just treat it as a full spot and keep moving until the next empty location. Otherwise, you run the risk of having duplicates. Is everybody okay with that? Any questions so far?

38:11

contains, so somebody asked how contains works. So contains follows the probing sequence, you hash, the value you're looking for using the hash function, you go to that location, and then you check if that location is filled, empty or empty. If the location is empty, then you say that it's not found. If the location is filled, you check to see if you're equal to the thing that's in there. And if you are equal to the thing that's in there, then you check the Boolean flag to see whether or not it's valid. And if it is, you return true. If it's not, it returns false. If something is in there, that's not the thing you're looking for. Then you go to the next probe location and repeat the process up Is that good? Excellent. Okay, so then the next question was, would it be so bad to run a contains check before

DS Lecture 15 Page 9 of 19 Transcribed by <https://otter.ai>

overriding? Um, hold on a second.

39:30

Sorry, I heard some very strange banging right outside my, well, I wonder if it's my neighbor banging in anger on his back porch, because I can tell when he's out there

because he cooks out there. So I smell the steak cooking, which right now, since I'm hungry is not the best thing in the world. But I also sit in the same spot giving these lectures and this is EPV eighth time I've given this lecture in the last year, and I am sure he's getting very sick. about hearing the same lectures over and over again while he cooks his steak.

40:08

Okay, so let's get back to. That's what I was saying I think his kids in college, why not?

40:16

Alright, so let's get back to what Mark was saying. So wouldn't be so bad to check before overriding bingo costs would not be any different, right? It would just be a constant factor difference. And that is a reasonable point. My feeling, though, is that it ends up being excessive, because the contains check costs, we'll have to go all the way through the entire sequence to find the one that it is you don't really win anything by short circuiting that when you overwrite because you still have to go through all of them to that point. But then the other thing you would say as well, doesn't that still mean there are more free spots. And you're right, that's true. But we're not going to be permanently keeping these lazy Lee deleted items, the next thing is going to be the time when we can get rid of them. And so since getting rid of them is a thing and is baked into our process, it then becomes probably not worth it to go through that extra cost, because they're going to go away anyway. Does that make sense? You can just nod. your camera's on. Good. Okay. All right. So the next question was, is it badge just insert to the end, instead of inserting it over the same deleted element? So yes, so no, it's sorry. That's what we're telling you to do. Unless it's the exact same item. That's the only time you can directly override it. Tiffany? Does that make sense?

41:48

Yeah, but so if it's if we are using a duplicate, should we always add it to over override it?

DS Lecture 15 Page 10 of 19 Transcribed by <https://otter.ai>

Or

41:54

you can safely do that? Because there's there's no danger of breaking the sequence. Okay, got it. Thanks. Okay. All right. All right. So where are we okay, with lazy deletion and how it affects the other contains and insert methods? Yes. All right. So the one thing that we just discussed here, based on Mark's question, was the fact that well, it might actually go away at some point. So in all of this process, there's going to be a moment when we're going to be able to eject the lazy deleted items. What is that moment? The rehash? Yes, so the thing we said is that we need to ensure that the load factor remains below a half. Now, I should also note that the load factor in this case with the lazy deletion includes all occupied cells, whether they are valid or not. So the load factor includes the laser Li deleted items. So as soon as the number of occupied cells, those with valid and invalid items reaches a half of the total table size, we didn't need to rehash and recall the rehash procedure is we dump roughly double the size of the array. And then we go back through the array starting at location zero and just iterate through it in the original array, and re insert each item into the new array using the new table size. And as we're iterating through that, it's easy enough to just skip the items in the original array that were marked as invalid. So after the rehash, only the actual valid locations exist. So we have a cleanup moment after the rehash. Does everybody good with that? Any questions before I move on to the next problem? Would it be a new hash function? So no, we're saying that our hash function is just this. And then the table size gets baked in when we mod it by the table size after we add the probing function. So that final calculation is affected by the table size, but that underlying hash function doesn't need to change. Does that make sense? Okay, cool. All right. So that's linear probing. So we mentioned briefly quadratic probing. And so quadratic probing tries to get rid of the issues associated with those

primary clusters by having items jump around more as we go through the probe sequence. So just as a reminder of our probing function, H_i is equal to the hash of x plus f of i , whole thing, mod the table size. And so for quadratic probing, f of i , instead of being equal to i is now equal to i^2 . And we are still okay in the sense that the H naught location is just the original hash of x mod table size, because zero squared is just zero, so we're not shifting it at all. And even the first probe doesn't go very far, because one squared is just one. It's only when we get up to two probes that we really start jumping around. Okay, so they actually used up a little more space on that paper than I intended. So we will continue with our quadratic.

45:55

DS Lecture 15 Page 11 of 19 Transcribed by <https://otter.ai>

And let's do an example. So we haven't gone over rehashing in detail yet. Well, yes and no. rehashing amounts to just reinserting all the items into a new size table. I don't feel the need to actually draw that out. Um, but when we actually dive into the code, we'll see what those look like in code. And that'll probably be the deepest dive, we go. Does that is that okay? All right, cool. So my plan for today, by the way, is to get us through quadratic probing, I will briefly mentioned very briefly something called double hashing. And then we'll look at the first piece of code associated with separate chaining. I really want to get us into priority queues on Wednesday. So it's possible that I might give you a side like drawn quadratic probing that you can watch on your own. Let's just see how far we get. Okay. Anyway, regardless, we're going to start something called priority queues. On Wednesdays, you might want to start reading chapter six if you have a moment. But you'll all have after you finished submitting your homework tonight, right? Sorry. Um, so quadratic probing. Now, with quadratic probing, because we're jumping around, there's actually a requirement that λ must be less than or equal to a half. And table size should be prime. It turns out that if the table size is not prime, and λ is less than a half, it becomes possible, though not very likely, but possible, that you won't be able to find the spot to correctly insert a value. Even if there are free spaces in the table, which would be a problem. As long as we keep the table size prime and load factor are half or less, you are guaranteed to find a free spot. And I will show you a proof for that just because I really enjoy that proof not because I would expect you to be able to duplicate it. But we'll get there in a few minutes. Now I make this whole rigmarole about the fact that the table size should be prime. And for the purposes of this example, I'm going to use a table size of 10. Because I just don't feel like modding by 11 with an audience. Okay. On your final exam, when you have to insert things into a hash table, as one of the questions on the part one of your final you will have to mod by 11. But you won't have an audience. Okay. So let's draw our table. Grace. Sorry, I shouldn't have said that. It was a private message. My apologies. A private question asked is the load factor include easily deleted elements? And the answer is yes. The load factor includes spaces that are filled that we would have to jump over as we're doing our inserts. Is that okay? Yeah, sorry about that. Um, all right. So I agree. It's better to keep those preset. Okay. Fine with me. All right, moving on. So let's go ahead and draw out this size 10 table, which is not going to but it'll allow us to demonstrate.

49:37

Actually a problem I run into a little bit on Piazza, because we can all see who posts what on Piazza, but most people post privately so I've tried to train myself to just refer to the office up and not use names ever on Piazza because I just never noticed whether or not it was meant to be public or not. Um, Alright, so that's going to be our thing here. So let's do a couple of inserts. So let's start. Okay, so let's start with insert. Well, we'll start with 55.

DS Lecture 15 Page 12 of 19 Transcribed by <https://otter.ai>

Again. Now for quadratic probing, you know, the first place we try the H naught is always the same as normal, because we're adding i^2 , but i zero so it's just 55 plus zero, my

table size. And we'll go ahead and put the 55 here. And so now let's do another insert. Let's insert, say 26. Again, no probes play six goes here. All good. Let's insert 66. Alright, so in this case, the H naught Pro plus zero. Okay, fine. So $66 \bmod \text{table size}$ is six, we have a collision. So now we go to the H one probe. So it's $66 \text{ plus } i \text{ squared}$. Where is one? That's 66 plus one . So that $67 \bmod \text{table size}$ is seven. Not a big change, because that first probe is still a single step. Somebody asked, Why was I 00. So remember, I refer to my probes as the H sub i 's. And the first probe you test is where i equals zero, first, you try equals zero, then i equals one, then i equals two. That's the sequencing. So for my initial test, I said I will zero and go. Is that good? Cool. I think that is there were I see a y at the beginning of that statement. I'll go with that as a as Alright, so um, now, let's try inserting 15. Okay, so we're at the H sub zero Pro, because it's the first one we do. And so that adds zero squared to the $15 \bmod \text{table size}$, try here it fails, then we add one squared to it, which is just makes it $16 \bmod \text{table size}$ of six. So we hit here still have a collide, then we're adding two squared to it. So it's 15 plus now four , so that's $19 \bmod \text{the table sizes}$, location nine. So the 15 ends up here. So it jumps a little further for that h sub two probe. And now we get a little bit of gap spacing in here. Does that makes sense? So the quadratic probing formula was this. This was our general probing formula. And then for f of h , we just said f of i equals i squared. Okay. All right, so let's do one more, just to get a sense of how this jumps around a little bit more. Um, okay, so let's insert 95. Alright, so 95. That sir. So 95 is going to be? Well, that's the H naught, right, because you add zero squared to it, my table size, that's the h_1 because you add one squared to it mod the table size, then we're going to add two squared. So 95 plus four is 99. We'll try there, that doesn't work. And so now we're at the age three. And so that's going to be $95 \text{ plus three squared}$. So that's 95 plus nine , and 95 plus nine , my brain is almost there. That's 104. Right? So $104 \bmod$, the table size is four. And so it actually, in this case, it actually does grow the cluster a little bit because it jumped all the way around and landed right here. Okay.

54:35

And so the next one, if there had been a collision here, it would have added instead of nine to it, it would have added 16 to it. So you'll see it's sort of spinning around a lot. And so that's where we can run into these issues where you might get stuck in a cycle, even though there happened to be spaces free. And that's why we have to ensure that this is true, because that guarantees we don't get stuck. In a cycle, I'm going to show you why that's the case in a second. But first of all, is everybody okay? With a quadratic probing sequence? We're good with this. Now, all the other rules for inserts, contains and removes the use of lazy deletion, the rehashing, and so on, it's all the same, the only thing that's

DS Lecture 15 Page 13 of 19 Transcribed by <https://otter.ai>

different is the probing function itself, that we use in the context of all of those. Good. Alright, so I'm going to try to sketch out for you a quick proof, why we want to keep λ less than or equal to a half, and the table size prime. So the way we're going to do this is, we're going to simplify, what we're looking for is that if you have a sequence of probes, right, h_0 , h_1 , h_2 , and so on, up to some h_k , right? The idea should be that we guarantee that we'll find the spot free, as long as all of these probing locations are unique, right? As long as all of these probing locations are unique, and there's still space, in the array at all, will be able to find a spot. Do we at least agree on that? Right? So as long as h_0 ? Yeah, so as long as h_0 and h_5 , aren't both like location 10? We're good, right? The moment h_0 and h_5 , end up being the same thing? As long as there are there aren't duplicates. In your sequence of probes, you're going to find a place free,

56:52

anonymous questioners that answer your question. Good. Alright. And so what we're going to do here is we're going to do a proof by contradiction. And this is actually one we haven't seen before. We've, you saw a counter example, I think, on the homework, that

was the written part that was already do. And you've definitely been doing inductions. But a proof by contradiction is that basically, we assume the opposite of our initial conditions are true. And then we show that somehow that will contradict our base assumptions. Therefore, it must the opposite must be true. Alright, so what we're actually going to say is assume you have two probes, i and j . So that would correspond to H sub i , in each sub j . So h sub i and H sub j are just two unique probes along that sequence at location that R i and j . And so because they're unique locations in the sequence, I'm saying that i is not equal to j . So there are two different ages. Right. And so there are two different ages, we're also going to assume that i and j are both greater than or equal to zero, right, because it doesn't make any sense to have a negative probe in the sequence. And i and j are also going to be less than the table size over two less than I would say less than or equal to the table size over two now why they're going to be less than or equal to the table size over two, because we said that the load factor has to be less than or equal to a half. If the load factor is less than a half, then you at worst, should need to go through table size over two unique locations to find a free spot. Because if you had to go through more than table size over two locations to find a free spot, even though only table size over two locations were filled, that would mean that there were duplicates in the sequence. So this is associated with the fact that the load factor is being kept under a half. Alright, so those are our initial parameters. Okay. And so the last thing is that table size is prime. Those are our initial conditions. Okay.

DS Lecture 15 Page 14 of 19 Transcribed by <https://otter.ai>

59:46

And so what we're trying to prove, is that given these initial conditions, H sub i should never be equal to age subject. All right, that's what we're trying to approve what we're going to do this by counter example.

1:00:17

So we assume, for the moment that H sub i and each day are equal to each other, which is what's supposed to never be able to happen. So h sub i is calculated as the hash of x plus i squared, whole thing mod the table size. Do we agree? We're just applying the hashing function. Good. So hash of x . How would I do that? contradiction?

1:00:57

Thank you, I made a noise there, when I noticed that we will move on and ignore that noise. Um, and so hash of x plus i squared madhva. Table sizes what H sub i is. And so if we're assuming the opposite, we're gonna assume that that's equal to hash of x plus j squared. Also mod the table size. Now you'll notice that I'm leaving the mod table size off. And so what I'm going to do is I'm going to just say, mod T sort of off to the side, that just means that both sides are mod the table size. Now I don't know whether or not you guys have ever actually encountered. Whether or not you guys have actually encountered like modular arithmetic more formally. But suffice it to say basic algebra works, even when you're modding both sides by the table size, okay? We're good with that. Alright, so now what we're going to do is we're going to try to sort of simplify this out into a way where we can get our contradiction, contradiction knocked out. Let's, let's go ahead and keep that muted, shall we? Yeah, thanks. Alright. So first of all, hash of x is the same on both sides. So I can subtract that off and it goes away. Do we agree? So really, this could be rewritten as i squared equals to j squared. That's the same thing. Both sides mod table size. Now, let's collect terms. So let's subtract j squared from both sides. So now we have i squared minus j squared would have to be equal to zero for this to be true, whole thing off the table sighs. All right, let's factor this out. So i squared minus j squared can be factored into i minus j times i plus j .

1:03:11

And now what we're going to see is that this can't be true based on the initial conditions, so how could i minus j times i plus j be equal to zero? And then remember, mod the table

size for that to happen? Well, one possibility is that $i - j$ would have to be equal to

DS Lecture 15 Page 15 of 19 Transcribed by <https://otter.ai>

zero. Okay. Is it possible for $i - j$ to be equal to zero? Given the initial conditions? No, i and j are by definition different. So this by itself can't be zero. Now the question is, can $i + j$ be zero. Now, we have to say, well, $i + j$ can't be zero in non modular arithmetic, because they're not negative numbers. So there's no way that's going to be zero. But remember, both sides are mod the table size. And for both sides mod the table size, that just means $i + j$, if $i + j$ added up to the table size, that would be zero mod table size, do we agree? So $i + j$ can add up to a table size, because the load factor is most to half. That means the largest j would be half the table size or size over two. And so the only way the two of those things could add up to table size is if both i and j are equal to the table size over two. But i is not equal to j . So the largest that they could add up to is table size minus one. shuhai. Yes, that's exactly right. Okay, so now this can't be zero mod the table size and this can't be zero mod the table size Omer. Go ahead.

1:05:17

We're doing proof by contradiction. And we want to start with the opposite. We said that i can't be equal to j . So we

1:05:26

were trying to prove that $h[i]$ is not equal to $h[j]$. We know that i and j are not equal to each other. But $h[i]$ is equal to this formula here. And $h[j]$ is equal to this formula here. So we're trying to show that these two things are not equal to each other, which is different than saying that these two things are equal to each other. Okay, good. Awesome. All right, so we assume the opposite of what we're trying to prove, which is what this line is. Now we've shown that that can't possibly be true. Because this term can't be zero. And this term can't be zero, both mod table size, there's one more possibility it's possible, or it could be possible that the product of these two things is zero. Now, for the product of those two things to be zero, well, your initial thought is that each term should have to be zero. But that's not true. Because we're mod the table size, if the product of those two terms multiply out to be the table size, then mod the table size, it's also zero. And this is where the prime number comes in. If table size is prime, then you can't factorize it right, that's the definition of being prime, a prime is not the product of two numbers. So the product of these things will not actually be equal to the table size either, because the table size is prime. So because of each of these things, because i and j are not the same probe, because the load factor is a half and therefore, neither i nor j would ever be over half the table size. And because the table size is prime, this cannot happen. And therefore we've proved it by contradiction. Okay. All right. I never going to ask you to repeat this proof, though, you

DS Lecture 15 Page 16 of 19 Transcribed by <https://otter.ai>

might find seeing this useful for say, I don't know, an algorithms class or discrete math, but I really liked this proof. But anyway, that's why, yes, you will not be tested on this proof. But I really wanted to spend a little time on it. Because I don't talk about proofs enough in this class. Or maybe you think I talk about proofs too much. But I don't talk about proofs enough in this class. So it makes me feel better to show you a fun one. Alright. So when we rehash? How can we make the new table size remains prime, what you'll see is that we'll have a primality checker in there. It's not the most efficient thing in the world, but it just sort of sequentially increases until and then checks for Prime until it finds one that's prime. So what we'll do is we'll take the original size, multiply it by two, and then keep increasing it until we hit a prime number. And we'll call a method that will check to see that it's prime. Does that make sense? Cool. All right. So now I'm going to switch my share over to the terminal. And what I would like to do is I would like to start showing a bit of code for our hash tables. And ultimately, with this class, I would really like to show you both an implementation of separate chaining, and an implementation of

quadratic probing, and it helps oops, I just dropped a bunch of stuff. And I'll get that later. It helps that the textbook has such things. And I'm going to show you code that Excuse me, I'm going to show you code that comes from the textbook so you can pull it from that code pack. And so we're going to start with an implementation of separate chaining hashing, okay. And so our primary methods, the ones we were talking about, were an insert method, a remove method and a contains method. And then for this one will also happen to have an effect The method, okay.

1:10:03

So we'll go ahead and look at how these things are set up, I guess the first thing we really need to look at Oh, first of all, the class is generic, we should be able to have a hash table of anything. But we end up not having any particular, any particular constraint on the any type. Alright, um, you don't need them to be comparable to each other. The only thing you need is a hash function. But as we discussed at the beginning of all this, anything that descends from object, so basically, anything has a hash code method baked into it, right? Even object has one. And what we know is that hash code returns a 32 bit integer. And so we're going to use that as our hash function. All right. So if I, if I am trying to insert x, a x dot hash code, that's the hash of x. And then from there, you can modify the table size, or in the later piece of code, do our probing from there. So we're going to have a hash function built in that we're assuming we're going to use. Now, for our instance variables.

1:11:22

We really only need Well, two instance variables and one constant for this. There's a DS Lecture 15 Page 17 of 19 Transcribed by <https://otter.ai> private static final, which is a constant, which is just going to be the default table size, which is what we will create as our array size if the user didn't specify one when they construct this object. And in that case, we'll default the table size to 101, which is a prime number. Okay. In addition, we're going to need our hash table itself. Now what you should recall is that the hash table is not an a table of any types, because in separate chaining, each location has a linked list of items. So in fact, we're going to call our table the lists, because each item in the table is a linked list. And so well, any list really. So we will make this an array of lists of any type. And then current size will tell me how many items I have currently inserted into the thing. And as you know, there's really no limit on that, because you can add an arbitrarily large number into each list. If you wanted to calculate lambda, the load factor that's just current size divided by the list start length. So the list start length is the table size. So far, so good, everybody. Okay, so now that we have that, we will note that this constructor is overloaded. There is a constructor that takes in no parameters, which in turn invokes the constructor that does take a parameter with the default table size, the constructor that takes a parameter it takes as a parameter, the initial size that you request. Now, even though it's not strictly required, the way it is for quadratic probing, it is still better that the table size be prime, so we're going to enforce that. So what we're going to do is we're going to instantiate a new array of linked lists. So the lists, that's our instance variables equal to a new linked list array. And the size of that array won't be size, but will be the next prime larger than s the size that was bad, sorry, the next prime larger than the size. Now I'm not going to go through the next prime code. It's just a basic primality checker like what you'd probably did in 1004, where you just go from, you know, to up to the square root of the value and see if any of those evenly divided and if not, and call it prime. And so what it does is it just starts at size checks if it's prime, and if not, keeps adding, well, it's not quite one because you know, that even numbers aren't prime, but keeps increasing it to the next odd number up until it reaches a one that triggers is prime. So we're guaranteed a prime number. Everybody good with that? And then, because the initial lists are all No, we instantiate each list in the lists to be a new empty linked list. And now we have a fresh table. So everybody good with that. Cool. So let's start instead of starting with an insert. Let's start with a contains check. So our Boolean Contains is going

to take in the value x that we're looking for, it's returning true if it's found false, if not. Now, the first thing we do is we have to apply the hash function to x . And so that attempts to do that via a local method called my hash. Let's take a look at my hash. My hash takes the value x , the first thing it does is it mods it sorry, geez, the first thing it does is it calls that hash code method. So we're leveraging the existing hash code baked into the object, that gives me back a 32 bit int, which could be a positive or negative number, we mod that hash value by the length of the list that's modding it by the table size. And because hash Val could have been negative, just from this line, if hash value is less than zero after the modding, we add the table size back on to it to get it into the correct range. And you already see us make that move when we were looking through some hash functions. Last

DS Lecture 15 Page 18 of 19 Transcribed by <https://otter.ai>

Lecture. Does everybody okay with this?

1:16:14

Alright, let's go back to our contains. We'll get there eventually, I was there. So now what we do is we yx. So my hash figures out what the index into the array that is that contains the link list that we want to use. So we extract that list just by indexing into the array. And we store that in a temp variable called which list. So which list is a reference to the linked list that contains x , if it is there at all. And then we just return the results of calling contains on which list and you must recall that link lists or any list have a contains methods themselves. So this just searches the link list for x and returns true if it's there false. If it's not, then that handles might contains any questions on this. All right, I'm gonna run over by two minutes, because I just would like to finish and we're very close. The insert method is very similar. Remember, we're not allowed to have duplicates. So we find the list that it should be in first, then we check to make sure it's not in there already. If it's not in there already, we add it to the list. Then we check to see if we need to rehash. If we do we call the rehash function, we'll do that one next class. And then finally, we have our remove method, which works in exactly the same way. We figure out which list it would be in, we see if it's actually there. And if it is actually there. We remove it from that particular list, and decrement the size of the whole thing by one. Okay, I should note that after we added it, we also incremented the size by one. All right, that's it. At the beginning of Wednesday's lecture, we'll talk through rehash, we'll take a quick look at quadratic probing. And then we're going to start priority queues. All right. Why will your homework, I'll stick around for a few minutes, but I have a TA meeting pretty soon. So I will see you on Wednesday. Take care. Thank you. As the homeworks been posted. Now, I will later this evening. We're actually we're making a few edits to the the interface for to sort of basically every time we do these homeworks we try to look at where everything went wrong last semester and try to make them a little better. And we're still trying to fix that one. That's marked, right. So you probably it's in your best interest that we put it up. Yeah. All right. by Professor. Take care. Thank you. We're good. Alright folks, have a good night.

DS Lecture 15 Page 19 of 19 Transcribed by <https://otter.ai>

DS Lecture 16

Wed, 8/11 1:45PM 1:16:25

SUMMARY KEYWORDS

priority queue, array, heap, insert, location, method, binary search tree, priority, item, min, rehash, hash, delete, hash table, remove, dealt, occupied, child, active, binary

03:42

All right, we should get started. Let's see, I don't think there's much in the way of announcements here. Other than you saw that homework for written in programming came out Yes. Okay. So for written I gave you a little bit of a short deadline on so that

one's due next Friday. You have most I think of the material you need for that. And the rest of it we will be wrapping up today and maybe a bit will bleed into Monday but it's it's it's a pretty good thing where you can kind of go through this stuff as we go. The programming for homework for you will have until the following Tuesday, so you have a full two weeks for that. Um, I think homework for programming is easier overall than the previous ones frankly. So I don't expect it to be quite quite as big an impact or over the hump with homework three or we will be for those Viewers going to be submitting it tonight. But given that we're down about 16 people from where we normally are for the live lectures, my guess is those folks probably are still working on it for this evening. So that's where we are with that, there will remain a homework five for which you'll have a little bit over a week to do. And in fact, you'll probably have close to two weeks, because I'm going to overlap it with four. But work five is I want to tell you, probably about probably about half as long as the other homeworks to account for the somewhat less time that you'll have to put into it completely by itself. So that'll end up being do. Um, hold on, let me just shut something up here. Good. That will end up being due last day of classes, because I don't want anything to bleed into reading day, or whatever it is. So, you know, that's where we're at. Any questions for me before I continue with the material? No, we're good. Okay. So um, last time, I think we only hang we said we would do is. So it's slightly amused, somebody logged in to ask me a question before lecture and then very quickly logged out, but didn't realize they'd remain in my chat window, and I could see it grayed out. Oh, my. I think you guys probably don't find that as funny as I do. That's probably horrifying. Anyway, so last time we left off, we were about to start rehashing. Yes, we had gone

DS Lecture 16 Page 1 of 19 Transcribed by <https://otter.ai>

through the separate chaining hash tables. And the only thing we hadn't seen was the rehash operation, which we're going to go ahead and take a look at, then we're going to do a quick pass through an implementation of quadratic probing. And then we're going to start talking about priority queues and binary heaps. And so I would like to make a decent amount of progress on binary heaps today. Okay. All right. So here we go. We have our read hash method, which was invoked from the insert method, when it was time to grow the size of the array. So if you attempted to insert something, and you added it in into the link list, but the overall current size went over the length of the the array of lists, then we had to double the size, and then go the next prime up and copy it over. And so that's all this does. rehash saves a reference to the original lists, which you recall is our hash table. So now old lists points to that. We re instantiate a new array for the list to size is the next prime over two times its original length. Okay. And then what we do is we iterate through each of the locations in the lists, okay. And we initialize that to a new empty linked list. So all the lists exists, and they're all empty.

08:40

The current size of the table is zero. Now, so what we have is an empty hash table. And so all of our insert methods that we already wrote, will work just fine. on an empty hash table. So we now iterate through each of the lists in the old link list. So we go through each bin. And then for each bin, we go through every item in that bin, and we insert those items. And that insert, we're just calling our regular insert method that we wrote above, which will work just fine on an empty list. And we don't have to worry about rehash getting triggered in those inserts, because we've already grown it big enough so we know it can accommodate. And so the net result of the whole thing is the net result of the whole thing, did you just get lunch delivered? Okay, just check. I saw that I'm very I'm very lucky. Nicely done. Anyway. The whole for once I actually ate before lecture, so I don't mind either way. So that's it. That's the whole rehash and we're done. So it's actually quite simple in this case. And remember, with separate chaining, there's no lazy deletion, there's no need for in separate chaining, so there's nothing to throw out. So that's really all we needed to see in terms of in terms of our separate chaining hash table. Does anybody have any

questions on that before I move on to our quadratic probing hash table? We're good. Okay, let's do it. So moving on into the quadratic probing hash table. This is again, code from the waste textbook. And all the methods that we had previously, we have now we have an insert, remove, contains and make empty. Now before we can go into the constructor, we have to look at what our instance variables look like for our quadratic probing hash table. And this one is going to look a little bit different. Um, first of all, we are going to start with a default table size of 101. Because we do want it prime but this time, and based on the proof, we saw last time, we know that we need the table size to be prime. So our default table size is still prime, we have our table which this time, we

DS Lecture 16 Page 2 of 19 Transcribed by <https://otter.ai>

somewhat more easily call array. And you'll notice by the way, it is not an array of any types. Even though we have one item per we make it an array of this separate class, which is actually declared as a static nested class called hash entry. And the only reason why we do that is because we need the metadata of whether or not something is active or not to facilitate the lazy deletion that we're going to need for our removes. And so our hash entries, you'll notice has the elements baked into it, but then also has a Boolean is active. And when you instantiate the hash entry, by default, it starts out as true, it's active. Alright, so that's our hash table itself. The capacity of the hash table is defined by hash table dot length. So that's our tapes, our array dot length. So that's our table size. But you'll notice this time, we have two variables instead of one, to correspond to the items that are filled in it. The first one the size, that one's telling us how many active elements are in there, from the perspective of the user, how many elements that haven't been deleted, or in the thing. But now that's different from the number of occupied cells, right, because the number of occupied cells also includes those that are lazy deleted. So the number of occupied cells is how many of those cells are actually filled with something active or not. Now when we calculate our lambda, our load factor, right, when we calculate that we have to, well, our denominator is going to be the array dot length, because that's table size, but which one of these occupied or the size is my numerator? Anyway, so we're gonna want occupied, we're gonna want occupied, because what ends up happening is that

13:40

because the slots are taken up, by those lazy, the deleted items, we have to move through them. So they're part of the sequence of probes. And so they matter in terms of finding a free spot. Everybody good with that? Excellent. Okay, so that's our initial setup. Now, let's go back up to our constructor. We have a default constructor that triggers the one that actually kind of does the work with the default table size. Then we have the slightly more complicated one, which takes in the size that you want. But then we functionally decompose this down into an allocate array and do clear. So let's take a quick look at allocate array. And you'll notice that we actually reuse allocate array later in our rehash methods. So that's why we've functionally decomposed it. All that does is it sets the array equal to a new array of hash entries that are the next prime greater than or equal to the size specified. So it's just this one little line to allocate the new array and then the do clear method Which we also use for make empty, goes through sets the number of occupied locations to be zero, and nulls out all of the items. Okay, so that initializes everything nicely. So now our constructor is done, it's time for us to perform. Some of our inserts, removes and contains, let's start with the contains. Okay. And so I mean, you see, there's the size and the capacity that we can get just from our instance variable. But here's where we're going to do this. So the first thing we have to do with our contains method, which

DS Lecture 16 Page 3 of 19 Transcribed by <https://otter.ai>

returns true or false, depending on whether or not x is in here, is we have to find the location where x either is or x should be. And that's actually what the find position method does. All right. I really wish she hadn't abbreviated this POS. because, frankly, I

think of something else whenever I see that abbreviation, but our find position. This is how I'm going to phrase it method takes in a value. And then what it's doing is it's actually going through the sequence of probes that are needed on x. So it'll hash dex check H naught location. Oh, seriously, you need me to say this out loud. POS is a pretty standard thing. Yes, thank you, John. I don't know maybe it's because I'm old. That's how I abbreviate that, all right. I feel like I've managed to not like directly curse on the camera. Yet, maybe once I don't quite remember, I'm trying not to do more. It's just it's not, it's not good. Anyway, so

17:11

thought so. So um, this fine position, we'll go through the sequence of probes, the H naught, the h1, the h2, and so on, until either it finds a location that contains the value x in it doesn't check to see whether it's active or not just that location either contains the value x in it, or that location is not. So fine position will either tell me where it is or where it should be. So it ends up being a very useful function, because we can use it in the context of insert, and in the context of remove, it does all the probing for us. So you can really think of it as the probing function. So before I move on, and actually work us through that probing function, if we assume for the moment that it actually works, and it returns the position where it either is or it's supposed to be, then we check whether or not that position is active. Okay? And so is active is done, because if the position is no, you can't check these active fields directly in the hash entry, or you get a null pointer exception. And so is active uses that uses. Actually, the logical end here returns a Boolean. And the first thing it checks is whether or not the current position is not no, if the current position is no, then it's clearly not active and the value is in there. So if the current position is not no, that first, that first piece evaluates to false, the whole thing evaluates to false. And that's it. If this location is filled, that means x is there. So then it checks to see whether or not x is actually active or has been legally deleted. Okay? So is active returns true if current position actually is filled with something which has to be x because of this, and it's marked as not deleted? And that's it, that tells me contains true or false. So we do need to go through the fine position, but assuming that works correctly, does anybody have any questions on this method? We're good. Okay, cool. So let's go through the probing. Now, I will preface this by saying that the probing is done in a slightly different way, just mathematically the way it's calculated than the way we did it on paper. I actually stepped through a little bit of that in one of the questions on Piazza. So if you don't follow what I'm saying here, I've actually written it down there. So you can go through and look there I we DS Lecture 16 Page 4 of 19 Transcribed by <https://otter.ai>

actually see if I can find the exact spot where I Did it? Oh, wow, there are seven five unanswered questions. I'm gonna have to go back to Piazza later tonight. But let's see if we find it. I'll find it. Okay, if anyone's curious. It's number 838. On Piazza, it's called the fine POS method. And so if you want to see me go through that math, I didn't prove anything there. But it's related to this conversation we're about to have. So let's go to the fine physician method. So the,

20:40

the find position method. So first of all, current position is going to keep track of the current probe. And so you're going to be starting at the H naught probe. And what you know is that the current probe really should be the result of calling the hash function on the data x, and then just modding that by the table size. That's the starting position. Now, I don't really feel like going into the my hash method, because it is identical to the My hash method that we did in the separate chaining on Monday. So we're just calling the method we already did you know what that does, it calls hash code mods by the table size, and then just checks to make sure it's not negative. If it is it moves it over. So that's where it would be for the H naught probe. But then what we need to do is we need to check it. And if that's not it, we have to go to the h1 probe and the h2 probe and so on,

which is what this offset variable is for. Now, this is where the mathematics goes a little differently the way I wrote the probing function, my probing function, which was i^2 is the offset from the original location that my hash returns. But in this calculation, instead of it being the offset from the original location, it's the offset between the current position and the next position. So remember that if say, the first probe lands on location zero, then the second probe lands on location one, the third probe lands on location four, the fourth probe lands on location not I said that slightly off, but it's 0149 and 16. Right, that's what you would expect, this offset is the offset between the two of them. So if I'm currently at zero, I have to move one space from that, to get to location one, if I'm currently on one, I have to move two spaces to get to location. Sorry, I have to move three spaces to get to location four. If I'm currently on four, I have to move five spaces to go get to location non, it's this, how much to go between them. Now, this can be proven using induction, and I'm not going to do that. But the quadratic can also be done by keeping track of your offset. And for each additional one, you do, the offset starts at one for the first jump, and then you just keep adding two to it. So the first offsets one, the second offsets three, the third offset is five, the fourth offset is seven, and so on. So by adding two to the offsets to the subsequent locations, you end up with the same sequence of probes that if you did that, that i^2 offset from the base location, and I wrote that out a little bit in that Piazza post date 38. So with that in mind, we loop and go through our probes. So while the array at current position is not equal to no meaning while it's not empty, and the array at cars, position element equals x, and then that negated. So while it's not no, and it's not equal to

DS Lecture 16 Page 5 of 19 Transcribed by <https://otter.ai>

x, we have to keep probing, the moment we either land on a spot where it is no meaning x should go there, or where x actually is we stop. And that current position is where we want to be. So if it's not at the current position, we increment the current position by the current offset, and we increment the offset by two. All right, and then this just wraps us around back to the beginning. If we end up all the way at the end of the array, okay. That's And so once this loop stops, we have the current position of the location where either x is or the first note, any questions on our probing function, our fine position. Okay. All right. So between this and the rest of the stuff in contains, we know how contains works. Now let's take a look at our insert method.

25:37

So, first step is always the same. All right, we find the position, we go through all of the probes using the find position. And that's it. The next thing we do is we look to see whether or not x is actually there. And so if we found a position where x either is or there's an empty space, if that location is active, that means it's equal to x and it's not marked as deleted, then we don't need to insert anything, because this is a set, and it's already there. And so we just returned false to indicate that we weren't able to insert something. But we didn't need to because it was already there. All right, if the current location is not active, there are a couple of scenarios. The first scenario is that B array at that current position is no. If the array at that current position is no, that means we're occupying a new spot, right? That means the total number of occupied cells and that includes both leisurely deleted and active items, increments by one. But if that location is not no, and we reach this point, it means that x is in the cell, but x is inactive, which means what we're about to do is we're about to overwrite an easily deleted item with something else that's equal to it, which is okay. But if we're about to overwrite a lazy Li deleted item, we're not changing the number of occupied cells, right, we're getting rid of all easily deleted item, and replacing it with an active item. So occupied does not get updated in that case. So this just decides whether or not we need to update occupied. Then once we do that, we set the current position equal to a new hash entry with x and true. All right, it's just easier to overwrite it even if we're flipping it from false to true, because you can just do it with that one line. But then, in either case, the number of actual active elements has increased by

one that always is increased by one. And so we increment that. And then the very last stage is we have to check and make sure that we haven't reached a point where we need to rehash. And remember, if we go above a load factor of a half, we must rehash. And so if the occupied cells is greater than array dot length divided by two, well, that means load factor is over a half. And that means we're going to have to rehash and so this is the trigger for the rehash function. Before I go into that rehash function. Does anybody have any questions about the insert? We're good. Omer. Go ahead.

DS Lecture 16 Page 6 of 19 Transcribed by <https://otter.ai>

28:47

This isn't really about the insert. But right, when you check if occupied is bigger than array dot length over two. Why don't you make a variable that's called lambda, and just check if checked by lambda instead of doing divided by two?

29:02

because lambda changes whenever occupied changes, right? So sometimes it's easier to just kind of do the comparison in one go, rather than redo the calculation of lambda and then compare against lambda. It's the same amount of work and just does it in one line. Does that make sense? Yeah, right, because right is every time we change occupied, we're going to have to recalculate lambda. Anything else? Okay. All right. So let's take a look at our rehash which is conveniently right here. And so this is going to feel rather similar to what we did before with just a little bit of an extra tweak, to deal with the laser Lee deleted items. So the first thing we do is we store our reference to hash table in a temp reference called Old array, we now allocate the actual array to be twice the length of the original one. And if you remember, allocate array actually does the next prime business, we reset occupied to zero, we reset the size to be zero. And then we have to copy the hash table over. Now this is just one loop because it's not an array of linked lists, it's just an array and there's only one value in each. Alright, so for hash entry entry in the old array, so we iterate through the old array. And if that entry is not know, and that entry is active, we then use our insert method to insert that value into the now newly cleared out and larger hash table. If an entry is no, we don't need to insert it. And if an entry is not active, if that's false, meaning it's been easily deleted, we can also skip it. Okay? All right. So that's our rehash. And now really, the last thing we need to look at is the Remove method. So just like before, we're going to start by finding our position, which will either tell me where x is, or if that location is null. And then we just have two possibilities. If that location is active, is active means x is there, and it's true. we flip its active field from true to false. We decrement the size by one because we removed inactive element. And then we return true to indicate that our remove has been successful. The else condition hits when is active returns false, which means that either the location was no or the location was already deleted. And in that case, we return false because there is nothing to remove. And that's really it. That's our quadratic probing. Any questions on these implementations? Yes, one? Go ahead. Um, thank you for just for the Remove method, is it considered bet Not, not best practice to actually, like, return the item that was removed? For tables? There are two lines of thought in this. And it kind of depends on the structure that we're actually working with. The first line of thought is that, yeah, if you're removing it, it's usually considered a good idea to return back what you removed. However, in this case, the data, the key and the data are the same. And so we're asking it to remove x. And so whatever we're going to

DS Lecture 16 Page 7 of 19 Transcribed by <https://otter.ai>

remove is going to be x. So it probably isn't necessary to return it back, since we clearly already knew what we were trying to get rid of. Does that make sense? The second thing is, when you're dealing with structures, where adds and removes might fail, for whatever reason, like a hash table might not find a free element. If you didn't set things up properly, or, you know, maybe it's already not there, then returning true or false to indicate that it failed can be helpful. Okay. Yeah. Whereas in something that you just said about the key

and the value are the same in this particular instance. So basically, it's like a hash set, right? That if we were using a map where that wasn't using a map that then I might consider returning the value know? All right, Other questions? We're good.

34:06

Okay, we are done with hash tables. And now we begin priority queues. So let me switch my share this right. So priority queues. So in a priority queue, it's like a queue. Except the condition that we use to determine that we're going to remove something is a little different. So in a wrap, dealer queue, what you're used to is, all right, we add an item. And when we remove items, they basically come off in the order in which we added the right. So first in, first out in a priority queue, there is a priority, so to speak to the ordering, in which they're going to come out, okay. And so, in a regular queue, that priority is whoever showed up first as the highest priority and therefore needs to come out first. But in a priority queue, each item is assigned some sort of a priority level and importance level. And the importance levels are not unique. So you can have a bunch of things have the same importance level. All right. And so when you remove from a priority queue, what comes off ends up being the most important thing, or if there are many things that are tied for being the most important thing, one of those has to come off. So just as a quick thought, exercise, um, what? What are some examples of priority queues in the real world? Forget about computers for a moment. But what are some examples of priority queues in the real world? All your homework assignments? Yeah, I'm not sure about that. Okay, let's slow down for a sec. So let's see, we had printers. And so printers definitely have a print queue. Welfare peanut butter. What? Nevermind. Um, so please, let's just go and what we have here to start with. So we've got printers, the person's thinking about print views, every print queue that I've dealt with has been first come first serve.

37:05

So more like a traditional queue, but I suppose certain print jobs to be considered have a higher priority than others. Um, vaccines? Well, there's a, there's a topic, right? So you've got these priority groups, the only thing that I would say is maybe a little, a little difference about the way at least in New York, I don't know how they're doing it where you

DS Lecture 16 Page 8 of 19 Transcribed by <https://otter.ai>

are, at least in New York, the way they're doing the vaccines is that there are priority groups who are allowed to come, but they don't wait for all the people in a priority group to be served before opening up the next priority group. So in a priority queue proper, they're coming off this queue in the order of priority. That's not quite what's happening here. Because not everybody is in the queue initially, they can choose to add themselves. And so it's really they only add themselves to the queue, if there have a sufficient level of priority, but that's not really the order in which they come off. Right. So for example, I think I fall into priority queue priority level one B, as far as the vaccines are concerned, but that's not priority level one a, but there are definitely people who should have been in one eight who just chose not to get vaccinated, and they'll get vaccinated after me. So it's not quite the same. And so plain boarding, assuming everybody comes when their boarding group is called, that one's a little closer, right? First, they call all the people who, you know, have disabilities then they call the people who have little children, and they call the people who paid a lot of money than they call the people who paid a little bit less money than they call everybody else supposedly in the order in which it makes sense to fill them in but not really. Um, wait lists, um, pens on how they're implemented. Some of them are first come first serve. airport terminals similar to plain boarding. Who to fire like last one hired. Ah, yeah, I guess depends on what the contract says. Suppose. Halloween candy. How does that work? How is that a priority queue? I'm curious. Best candy go. Oh, okay. You take all the best candy. Okay,

39:45

okay, fine. Um, easy. Same concept as welfare peanut butter mostly. Oh. Okay. And they

run out of regular peanut butter. You have to get the welfare of peanut butter. And then we do is you mix it with something else to make a jelly like substance so taste Okay, okay, okay, I,

40:07

I Okay, um, I, I don't like thinking about peanut butter and jelly. Personally, I think mixing peanut butter with jelly is sort of the worst thing that ever anyone ever came up with and is disgusting and ruins perfectly good peanut butter. I was saying that to make no no, I don't like

40:29

jelly, it's up, I'm saying peanut butter, more more buttery,

DS Lecture 16 Page 9 of 19 Transcribed by <https://otter.ai>

40:34

peanuts. Alright enough with the peanut butter. Let's move on. Okay, so we finally get to, I think, probably the most sort of the most. Alright, it's an example that I think does it pretty much perfectly, which is how you handle people in an ER, right in an emergency room. And so triage folks with injuries, I think is a good is a good way to explain that. So let's look at it this way. If people are dealt with in order that they show up in emergency room, people will die, right? Because if I walk in with a paper cut, you know, which for me is an absolute tragedy, right. And then I'm sitting there and I'm waiting. And there are people behind me also, apparently, there's an epidemic of paper cuts that day. And then, you know, an hour later a guy shows up with a heart attack, they're going to deal with the guy with the heart attack first, before the 50 people with paper cuts, because for the 50 people with paper cuts, if they don't deal with them, they'll live the whole deal with the guy with a heart attack, they will not live. And so that's sort of the idea that that person immediately jumps to the front of the queue, because it's of higher priority. Okay, so an example in a computer system that involves priority queues, and that's actually going to feed us very well feed us. Oh, no, we're back to the we're back to the we're back to the peanut butter. Somebody asked, What if 50 people have heart attacks? Well, there's the thing. Um, things that have equal priority just are going to get dealt with in, you know, in arbitrary order. And that actually brings up an interesting thought there is a concept of a stable data structure versus an unstable data structure. And in an unstable data structure, the order in which they came in is lost. So if 50 heart attacks show up, they'll just get dealt with in an arbitrary order. Right, because you know, there are only so many people you can deal with at a time, no matter how serious it is. But in a stable system, then things will get dealt with in the order they showed up. And if they are otherwise equal priority, you know, and so it's going to depend a little bit on whether or not the queue is considered stable or not. And so the ones we're going to implement are not necessarily stable. But maybe for a situation like an ER, maybe they should be. Okay. So worth it to talk about that concept of stability. Now, the next thing we need to discuss is, so processes in a computer kind of vibe for processor time, right? They're sitting there waiting to be run, you have many programs running at once I have, you know, my camera program. second camera program, zooms, running, I've got a web browser open, I've got Spotify open, I've got my slack open, I've got a lot of things open on my computer right now. And you know, I look at all my monitors, I have several because you know, I need to see your faces in one of them. I look at all my monitors. And it looks like they're all running at once. But they can be all running at once. Because while I've got one processor, or Well, actually, I think I've got four. But we're maybe it's a core, I don't remember the point. Bottom line is I have fewer processors than there are processes running. So they have to split the time up. And so processes that are considered higher priority, sort of jump the line and get dealt with first. So these are things that might be needed for the computer to just still run. Like, you

DS Lecture 16 Page 10 of 19 Transcribed by <https://otter.ai>

know, basic computations, basic network connectivity, whatever. And then less important

stuff like the slack window that isn't even viewable by me right now. It's still running in the background, but it is less important that it gets updated in a timely manner. Now you're probably not going to see this if you try it yourself just yet. So don't worry about I'll just describe it. But there's actually a program you can run on a Unix like or Linux system, called top or P, you can run top from a terminal. Now the version of top that comes on a Mac actually doesn't show what I want to show. But if you ran top, from the terminal in

45:20

in Kodi Oh, you might see it, it'll show you a list of all the processes that are currently running. And then it'll have a n field there that's listed as n i. And you'll see a bunch of things, some of them have zeros in them, some of them have negative numbers, some of them have positive numbers in it. And that ni stands for a property of these processes, called their nice number. And the nice number in a Unix or Unix like system is a measure of their priority. And so it actually works a little bit backwards from the way you'd expect, the lower the nice number, the higher the priority. So if you have a very negative number, you consider yourself very high priority. And the way I kind of remember it is they call it nice, because if you're less nice, you're not going to let people go ahead of you, you're just gonna go. Now if you're more nice, and you have a higher nice number, then your guy, whatever Slack, you go right ahead, even if you think you're important, whatever. So the higher the nice number, the lower the prop the the lower the priority. And so when we implement our priority queues, we're actually going to go with that paradigm, that the things that have lower values come off first. And so the way we're going to talk about our two main methods for priority queues is we're going to have an insert. This Sharpie, I think, is at the end of its life, we're going to have an insert. And our remove method is going to be a delete min meaning that we remove the minimum of priority item each time because we're going sort of by nice numbers. And so this is sort of a min ordered priority queue. We insert things and then we delete, Min, delete, Min, delete min. Okay? Now it is possible to create a max ordered priority queue, where we delete max each time and that in the end, that's just gonna amount to flipping some less than signs to greater than signs. But for now we're dealing with min ordered. Is everybody good with that? Now another name for a priority queue is a heap. All right. And so you'll sometimes hear people refer to mean heaps versus max heaps. So min heaps are going to be heaps that are min ordered, and Max heaps are going to be ones that are max ordered. Um, is there a reason why we choose Delete min over delete max that ends up being the more commonly useful way of doing things. In fact, you'll very quickly realize in homework for one of the problems I gave you there will actually, despite my asking you for the N largest items in a ever growing set of things, it's actually a min heap that will allow you to solve the problem. So it's more

DS Lecture 16 Page 11 of 19 Transcribed by <https://otter.ai>

frequent that min heaps are useful when we find Does that make sense? Alright, so now we have to talk about how we would implement such a min heap. Now. Can anybody think of a really, really trivially simple implementation of a min heap? Like the simplest thing, you can come up with? No complex structure? How might we do it? So data structures specifically? So sorting, it could be related to it? For sure. But what what structure would we put the values that we're inserting into this min heap, be in

49:29

like a linked list or an array? binary search tree might make things a little bit faster. But the reality is, that's not even that's a little more complicated than what I had in mind. So let's go with the simplest thing. Let's just call it an array. All right. Um, yeah, let's even simpler let's go with a linked list. And so with a linked list, if we want there are two ways we could do this. You could say whenever I insert, just add the item to the header of the table The list right because if you have a doubly linked list, which was what Java's linked list is adding to either the head or the tail would cost order one. So all of our inserts into this

linked list based priority queue would cost constant time. But then our removes our delete means from this simple link list priority queue, what order would that removes the I removes would be order in because we'd have to do a find min, where we sequence through everything to be sure we have the minimum. So our inserts are order one, our delete mins are order n. But we could flip it around. And so as your head suggested sorting, and so you can kind of fit sorting into this if you're using a link list. So you could say when I insert, because it's a link list, I can splice a value in wherever. So I could keep my link list sorted. So every time I insert something, I could go through the link list until I find the spot where it belongs. But that would make my inserts cost order and right, because it would potentially I might have to go all the way to the end for the insert to work. But if I keep the list sorted, then my delete man is trivial because I just pluck the first item off, and I'm done. And so if you just use something simple, like a link list, you're going to have one of those two methods B order in and the other B order one. And just depending on how you choose to implement it. It could be either, all right. But now what we want to do is we want to try to get the costs of these inserts and delete mins down, we want to get it such that we want to get it such that, you know, maybe they both have the same cost, but they're both log event. And so the binary search tree idea wasn't such a bad one, we are going to use a tree based implementation for this, and one where we're going to keep the tree very well balanced. But we're actually going to use something a little different than a binary search tree. We're going to implement this using a tree based structure known as a binary heap. All right, and so this binary heap is going to make it so that our inserts and our delete mins are at most log event by making sure that these are at worst login operations, which means making sure that this binary heap is a binary tree by making sure this binary tree is very well balanced. And the order of its height is log events.

DS Lecture 16 Page 12 of 19 Transcribed by <https://otter.ai>

Now a binary heap has two properties that are going to have to be maintained at all times. The first

53:11

better

53:14

I did I go for like 10 minutes without anybody else noticing that. We all just staring at that wondering if I'd ever noticed. Okay, Molly, you're my ta

53:28

was actually looking at my phone. Why didn't you just not? Yes. The other people in the class who were probably watching it and

53:48

also Professor you lag there. I was a little bit so we were kind of like

53:55

you lags the video. Oh, okay, for a good little bit. So was did everybody actually ultimately hear what I was trying to say? Okay, that you heard. All right, fine. Okay. So our binary heap, not head heap. Okay, at least I wrote heap up here. So this wasn't as quite as bad. So our binary heap which is a version of a heap, an implementation of a heap has to satisfy two properties. The first property is that it's going to be a complete

54:44

binary tree. Alright, and so does everybody remember complete binary tree is it's one where all the levels of the binary tree are completely filled up with the possible exception of the last level. Which is filled up from left to right. And this is actually where that filling it up from left to right is going to become really useful for us, because it's going to allow us to do a kind of a neat trick such that we can represent this binary heap not with the usual links that you're probably used to, from the binary search trees we've been doing. But

DS Lecture 16 Page 13 of 19 Transcribed by <https://otter.ai>

instead, using we can use an array. So we're going to use an array to implement this binary tree, we're going to keep this binary tree complete at all times. And we know that that's going to have a height on the order of log event.

55:39

Another condition is that every node must maintain what's known as the heap ordering. And the heap order is the following. In a min heap.

56:10

every node, let's see, there are two ways I can describe it, let's say every node

56:22

must be less than or equal to its children. Now, when I say less than or equal to when I'm comparing nodes, I'm comparing them by their priorities. So every node must have a lower priority, or at least equal priority, because priorities are not unique to its two children, or one child, if it's all the way down at the bottom, right. So this doesn't say anything about the relationship between the siblings, you could have a bigger one on the left or a smaller one on the left, it does not matter. But it does say that a relationship between a parent and a child so if every node is less than or equal to its children, what node has the minimum value in the root. So now we know the root is the smallest item. So deleting the man is still requires some work because you have to fill that root in. But if you just want to know what the minimum is go to the root. That's it, you're done. So it has to be a complete tree, and it has to uphold this heap ordering. Now if you want it Maxi Porter, then every node must be greater than or equal to its children. And then the root would be the largest. But for now, we're doing everything using min heaps. So let's draw some min heaps, or Let's see whether or not certain things are valid min heaps. Now what I'm going to do is I'm going to assume just for the sake of these drawings, that each item is an integer and the integer value is its priority. So we can have something that looks like this. Is that a valid min heap? Sure, it's complete, because all rows that exist are completely filled from left to right on the bottom row. It's mean he bordered this thing has no children. So it's definitely less than or equal to them.

58:37

Is that a valid min heap? Why not? Why wouldn't it be? Okay, well, is this complete?

DS Lecture 16 Page 14 of 19 Transcribed by <https://otter.ai>

Remember, there's a distinction between perfect and complete. The very last row has to be filled in from left to right. This is the last row and I filled the left child. Does that sound okay? Cool. So it's definitely complete. And then the 12 is greater than the one so the one is smaller. So it's also min he bordered. So yes, this is a min heap. about that. Is that a min heap? Looks like it's good, still complete tree. The relationship between the parents and the children is still good. relationship between siblings is irrelevant. Is that a valid min heap? So a lot of people are shaking their head No. Does it satisfy the min heap ordering at all nodes? Forget about the completeness for the moment does it satisfy the min heap ordering it does But it's not a complete tree. So but it's a situation where this still isn't valid, but

1:00:16

that a valid min heap think we're good. Alright, so is everybody okay? At least recognizing he bordered things? Yes. Questions. Okay. So now what we're going to do is we're going to talk about how we might represent this thing. What's the question?

1:00:49

This is a very strange thing, right? So when you go, why don't we put the big number on the left side now, or we put the small number on website.

1:01:01

So it didn't matter. I chose sort of randomly which way to go.

1:01:08

And that's what I was trying to figure out. I was like, Why all of a sudden, do we do this? And I looked through the textbooks very, very closely. And I couldn't find any structures. why we did it that way. It doesn't.

1:01:21

So here's the reason, it doesn't have to be the relative order between the left and right children is irrelevant. The only thing that matters is the relationship between a child and a parent. And so I chose to do it sort of the exact opposite way of what you used to seeing with a binary search tree, just to ram home the point that it that aspect of it doesn't

DS Lecture 16 Page 15 of 19 Transcribed by <https://otter.ai>

matter anymore. Could the 12 has been over here and the five over here? Yes, that still would have been completely valid. But at that point,

1:01:58

yes. Because as long as the children are bigger,

1:02:02

right, and so the ordering between the siblings is irrelevant. And this was just meant to drive at home, because you're used to ordering it the other way. Does that make sense? Of course 100%. Cool. All right. Other questions? Okay. So, um, what I want to do now, we look at this, and we've drawn it, and it looks nice as a tree. But we should talk a little bit about how we're going to actually represent this in code. Now, one thing that's going to be a little bit different in our algorithms on the binary heap than what we're used to, from the binary search trees and the binary trees we've been dealing with, is that, we're going to need to be able to both move up and down this tree. All right, we're not going to be able to just live with having references to the left and right children, we need to be able to go from a child back to its parent. And we need be able to do that not just in the context of recursion. In fact, most of our methods for this, all of our methods on this will not be recursive, we're going to be able to doing it all iteratively. And so there's a really neat way of representing these trees using an array. And so I'm going to draw an array that's going to be a little bigger than we need to store all the items, I have 123456 items, I'm going to draw like a 10 item array. So let me say we got 01256789. So that's going to be our array. Let's divvy this up. Okay. Now, what we're going to do is we're going to actually, in our binary heap implementation with arrays, we're not going to use location zero for part of the tree. All right, so location zero is not real data. We're going to use it a little later on as sort of a temporary space when we actually implement this thing, but it's not going to be part of the tree. And now what we're going to do is we're going to fill in the array in the order of a level order traversal of the tree. Alright, so item one, the root goes in Location One, then the root's children go in locations two and three. And then that next level will fill in also from left to right, we'll fill in 2113 and then six. All right, and then the rest of the space hasn't yet been filled in so we can use that as scratch space. For later on, so that's the six, whatever. Now, here's where this being a complete binary tree really, really wins for us, there are no gaps in the tree. And when we go filling in from left to right, that guarantees that even on the very bottom row, there are no gaps, we just fill it in sequentially. And this level order traversal will have no breaks and every location from one to the size of the binary heap, because there are six items in the heap. So it fills locations one through six is completely filled in. If I want to add one more item to the binary heap,

DS Lecture 16 Page 16 of 19 Transcribed by <https://otter.ai>

we know in the array, it goes one past the location of the last item filled in, that will correspond to this. And if there's one more beyond that, that would wrap around to the next level over here, that would go here. So it becomes very easy to find the next spot that has to be filled in, just by keeping track of the size tells me where the last item is. And then one past that is where the next one goes. Is everybody good with that? It's really nice. But now comes the best part, you can trivially calculate the children of a node and the parent of the node from the index of a node.

1:06:36

So if you have a node at index i , its left child is that index two times i turns out, so the root is an index one, so its left child would be at that index times two, which would be location two, which works out the item at location to its left child will be at two times two will be at location four. Perfect. item five here, which is at location three, its left child will be at location six. It's if you want the left child of say item 21. That's an index for the left child of item 21 would be at location eight. And we know it doesn't exist because it's greater than the size of the array of the field and part of the array. So you can always calculate them left child by multiplying it by two. And that, by the way, kind of shows us why we started this thing at one not zero, right? Because if we started at 0×2 would just be zero and it wouldn't work. Are we good.

1:07:58

All right now the right child. I left the tee off of left I'm doing great here, the right child?

1:08:10

Well, you'll notice because this is a level order traversal when we were writing this stuff in the right child is always directly next to its corresponding left child. Right. And so as Daniel points out, it's just two times i plus one, and that will always give me the left child. So if i as location two, it's sorry, right child, if i is it location to its left child is four. And so it's right child is it four plus one, which is five, which is exactly what we see here. So we can calculate the left and right child trivially. But now comes even better part, we can trivially calculate the parent, the parent, assuming is an integer, right, because these are, these are indices, so they have to be integers. If we do i divided by two, that's going to be the parent. Now let's think about this for a minute. This is integer division, which means it will truncate the result of the division. So for the left child, i divided by two make sense. So if you are a left child, you're two times your parents. So divided by two you get your parent back. If you're a right child, then you divide it by two, you get your parent plus point five

DS Lecture 16 Page 17 of 19 Transcribed by <https://otter.ai>

from this extra one, but that point five gets dropped in the integer in the integer division, so this still works even for right children. So now we have a way of calculating left child right child and parent and so we can easily move up and down in this tree cheaply. Are we good? Okay. So what we're going to do now I've got about three minutes left. So I'll start doing this is I want to talk about how we can insert something into one of these things. So let's draw binary heap real quick. That everybody okay, that that's a valid buy. Oh, Daniel, go ahead.

1:10:39

Oh, this is just a quick clarification. The reason that we care if the binary heap is full is so that this works, right, like the two eyes care that it's full, we care that it's complete. Right? Right. The reason we care that it's complete is because of this

1:10:52

partially, but also, we care that the height is constrained to be log event. I see the completeness also gives us right, that makes sense. All right, thank you. So this is a slightly tighter form of being balanced, that also gives us the ability to do this. Okay. Now, let's do an insert. For this, I'm going to just show what it looks like on the tree, but it's also happening in the array just using the calculations. So let's insert two. Now you all know that when I insert two, we're ultimately going to want that to be the root of the tree, right? Because that's going to be the smallest item. Now, there are two things that must always hold true in a buy in by a binary heap mistake complete, and you must hold them in keep ordering. And so what we're going to do is we're going to do an initial insert of the to add a location that maintains the completeness of the trade. So we just plop it in over here. In fact, they won't even put it in, I'll make a little hole for it. And the two is trying to go into that hole. And so in the array, it's trivial, you just go to the size plus one, and that's where this location is. So you drop that down there, you got this little hole here. And now you

have a complete tree still with one more item in it. But your mean heap ordering may have failed. And so now we do what's known as a percolate up.

1:12:39

So we're going to move this whole up towards the root until it's at a position where the two can safely be put in it. So we calculate this whole's parent, which we can do in our array representation. And we check to see if two is smaller than what's here. If two is equal to what's there or greater than what's there, we can just put two here and be done. But if two is smaller, two has to become the parent it gets percolated up, and the 10 gets

DS Lecture 16 Page 18 of 19 Transcribed by <https://otter.ai>

shoved down into the hole. So in this case, 10 goes down. And now the hole is here, we look at the parent again, and see whether or not we need to keep moving or we're done while the two is less than the parent. So the three comes down, and the whole goes up. Now we're at location one, calculate the parent and Oh, the parent zero, that means we're already the roots. And so if we percolate all the way up to the root, then we know the value can go there because there's nothing above it. And we write it in and we're done. So the big O cost of this is such adding it to the very end of the array is cheap. So keeping the whole thing complete is cheap. And but then the percolate ups, you're gonna do one swap per level. And as a result, it's log of n because this is a complete trade.

Alright, I think we're at a time. So we have completed our insert. And on Monday, we're going to do our delete min, we're going to talk about another operation called a build heap. And then we're going to go ahead, and we're going to talk about for a few minutes anyway about some applications of heaps, and then I'm going to move on to sorting. So you should start reading chapter seven sorting. Now the one thing that we haven't done is gone through the code for the heaps. I'll see I'll either try to go through the code on Monday. If we have time, or I'll post it as an extra lecture. I'll try to do it in lecture, but we're going to finish up heaps and we're going to try to start sorting next week. The reason is I want to kind of To finish sorting next week, so I have a couple of weeks to talk about graph algorithms. But I think we've got time. Do you percolate up on all inserts? Or rather than Yes. Because even if I were inserting something that didn't make it all the way up to the root, if I were inserting, say, five, I still would have swapped it with the 10. But then it would have stopped here, leaving the three here. And it still would have been the same percolate up operation just stopping halfway up. Does that make sense? All right, folks. Yeah, this is, uh, this is a lot easier. Okay, I'm gonna stop the share. I can stick around. Take care.

1:15:48

And then I will head out. Thank you. Take care. Thank you. Thank you. Anybody have any other questions? Going once, going twice. Alright, folks, have a good one. I'm going to close this down.

DS Lecture 16 Page 19 of 19 Transcribed by <https://otter.ai>

DS Lecture 17

Wed, 8/11 1:49PM 1:20:48

SUMMARY KEYWORDS

array, percolate, heap, operation, min, swap, sorting, hole, item, child, node, equal, smaller, binary, complete binary tree, delete, midpoint, root, algorithms, insert

02:25

But you're still

02:41

touching, john. It's hard to do but you're still the same. There's something inside you It's

hard to explain. Starting one second

05:10

Okay, so let's get started here. Um, so a few things a, you have a written homework that we said was due on Friday correct. That remains, and then you have the corresponding programming homework that is due after the weekend. So I think we are pretty much in ok shape for that. Um, we have a little bit more material to cover to wrap up the last little bit of what you're going to be doing on this written assignment. But we're mostly there, we're working through section 6.3 on heaps. Today, I'm going to try to wrap heaps up today and go through some of the code for it. And then on Wednesday's lecture, we're going to do sorting algorithms, and we're going to try to get through as much of that as we can on Wednesday. We'll wrap sorting up next Monday, and we will begin a graphs then. So you're reading chapter seven, through the section on quicksort, you can skip the section on shells sort. And then you need to start reading up in chapter nine on graphs, we're gonna skip chapter eight, it'll pop up very briefly, for one of the graph algorithms, and I'll give you what you need from that during the lecture. Any questions for me before I move on? We're good. Cool. Okay. Um, so what we did last time was our binary heaps. And we did our insert methods on our binary heaps. And now we have the other end of DS Lecture 17 Page 1 of 20 Transcribed by <https://otter.ai> things, we need the Delete min operation, because those were our two primary operations on are binary heaps. And so what I've drawn for you here should be a valid binary heap, min heap ordered, do we all agree on that? I don't see any mistakes. But since I wrote it on the fly, if you guys see any mistakes, let me know. And then over here, we have the array based representation of it, noting that we don't put any data in the zero location. So if we have a heap of size seven elements, we're actually in the array representation of this graph, filling up locations one through seven. All right, and so everything is set up there. Now, if we want the basic find min operation, that's sort of like a peak operation, you don't want to remove it, but just return it, we know that the min is the root node, if your min e bordered. And that amounts to just returning the array at location one, and you're complete. But the Delete min operation involves both returning that minimum value and actually removing it from the overall heap from the overall array. So what we do? Well, if you remember, in terms of the insert, the way we approached it was we first we tried to maintain the completeness of the tree. That's, it's easier if we try to remain tane, the completeness of the tree off the bat. And then we adjust the remaining complete tree, such that the min heap order is then preserved. And so we're going to do the same thing here. So the first thing we're going to do is we're going to take that minimum value out, and we're going to just store it in a temp variable. Because Well, we're going to return that to the user, right. So we'll just store that until we're done. Now we have a hole in the tree, and we got to fill this hole with something. Now what I would initially like to fill this hole with because you can't have gaps in the tree that kind of breaks the whole thing. I want to fill this hole with something initially that won't necessarily maintain the heap ordering but will maintain the completeness of the tree. What node from below Could I remove entirely and placed in that hole while maintaining the completeness of the tree? In fact, the 41 right, the thing that would be at the very last item in the array, the bottom right most note, so that's what we're gonna do. We're gonna just take that out, because we're decreasing the size of the heat by one. And now we're going to be trying to put that 41 get rid of this fine men business into that hole. And you'll notice that now the heat takes up locations one through six, because there's one fewer item in it, because we're bleeding. And so the goal is to then manipulate that hole, such that I can safely put 41 into it without reaching the mean IE bordering. Okay, now, when we did an insert, we created the hole at the bottom right, and then percolated the value up until it was at the right location. When we create the whole of the route, we actually have to percolate the hole down, right, because we've got something that's potentially too big. So we got to push

that hole further down until it's at a place where the main heap ordering is maintained. So this is a percolate down operation. All right. Now, when we percolate down, it is a little bit more complicated than the percolate up. The percolate up, we would just calculate, excuse me, we would just calculate the parents of the hole and compare against that. Here, we have to be looking at the children, right. So when I potentially move the hole down, which child do I need to consider? A buddy,

DS Lecture 17 Page 2 of 20 Transcribed by <https://otter.ai>

11:39

what child could I realistically swap with if I swapped with any of them? Okay, after looking at both, but only one of them is going to be swappable. And we have two people suggesting that the smaller child is the only one we really need to consider. And I like that. The reason is that if we move the hole down and pull a child up, that child that we pull up has to be valid as both the parent for the thing that we're trying to put in that hole, but also valid as a parent for the other's original sibling, right? You can't pull the 21 up legally, because the 21 cannot be the parent of the seven, the parent has to be less than or equal to the children. And so the easiest way to do that is look at the two children. While there's only one child like there is over here, you don't have to do this, there's only one child, it's the left child, and you're done. But if you take the smaller of the two children, that's the only one I can ever realistically percolate with. So I compare the children first pick the smaller one. And then that leads me to the seven in this case. And so then I compare the value that I'm trying to put in the hole and see whether or not the hole needs to move down, does it?

13:10

Yes, right, you want the smaller thing up the 40 ones big, so the seven comes up and the hole moves down. And now we try to put the 41 into this new location for the hole. And just like excuse me, just like we did with the previous percolate down, we look at the smaller of the two children, because that's the only one that could realistically be pulled up, which in this case is the nine. And then you see whether or not the 41 is in fact, less than that or equal to that or if it's greater, it's greater, so the nine comes up and the hole moves down. Now once the hole reaches a leaf, there's no further percolating down to go. And so we can put the 41 in there. And that's now our I'll rewrite it seven 921 2041 and 35. That is our new binary heap after the Delete min. Now in terms of this, you would just change the locations in the array. So this is the whole, the smaller the two children is the seven. So the seven goes here, the whole moves there. It's smaller child is the nine that ends up coming up. And now that's a leaf because its left child would be at location 10, which is outside of the range of the 41 goes here. And that's how it would be stored in memory. And that's what it would represent. Does everybody good with that percolate down, then all we need to do is return that temp five, and we're good to go. Let's do one more. If I delete min. I'm just going to do this on the tree this time. Yeah, it's easier to think of the hole as being empty as it moves, and then keep the 41 on the outside. That's so that you don't have to swap values so that you just slide the one that's the smaller child up without worrying about what you're overriding, and then just write the 41. And in the end, that's sort of easier. Does that make sense? Why skip the fifth index? Where did we skip the fifth index? That's where I wrote the 41. Does that make sense? Sorry, if it wasn't clear, this is the 41 is

DS Lecture 17 Page 3 of 20 Transcribed by <https://otter.ai>

here. Okay. What do we do know heaps are not set. So you can have duplicates. So the rule is that the parent must be less than or equal to its child during. So if it's equal to its smaller child, you just stop. There's no need to swap it down. Does that make sense? Yes, no? Oh, well, in that case, it doesn't really matter which child you would potentially promote, right, because either would be valid as the parent of the other. So it's kind of dealer's choice. Um, usually what we do is we, we start with the left child, and then only updated to the right child if we have to. So the way I would implement it most likely would

swap with the left child if they're equal, but either one is valid. And so another way to think about it is that if you look at these numbers, the only thing that's prescribed in terms of their relationships is the relationship between parent and child, the sibling relationships don't matter. So there could be many valid binary heaps for this collection of numbers, right. And so it doesn't necessarily matter if what comes out is a little different, as long as it's valid. Good. Alright, so let's do one more delete min here. So we'll take the seven out in the hole, we'll go there, we'll try to fill it with the 35. To keep it complete. The smaller of the two children is the nine, the 35 is larger than the nine. So we bring the nine up, the whole moves down. The smaller of the two children is the 20. that's smaller than 35. So we bring the 20 up. Now, the whole is here. And the 35 is now at a leaf so it goes there. So to rewrite, we have nine at the root, which is the smallest. So that's good, 20 21 35 and 41. And it proceeds from there. Do we have any questions on our percolate downs?

18:05

We're good. Okay. So next thing we're going to do is we're going to talk about what to do if we have a set of unordered.

18:27

Next thing we're going to do is try to pull the two pieces of paper apart. Okay, I've succeeded in that. Barely. So the next thing we're going to do now that I've succeeded in pulling the two pieces of paper apart, is we're going to see let's say we have a completely unordered set of values. So let me just write a bunch of numbers down. I don't actually know if I'm, what I'm doing is good, but Okay, so we have here 12345 numbers, okay. Now, let's say we wanted to put those into min heap ordering. There are two ways we could do that, right? You could have instantiate a new min heap that's initially empty. And then you can one at a time insert each of these items into this new min heap, right. And when all is said and done, we would have a new min heap with these things in a min heap ordering. What would the cost be? For me to just repeatedly insert these items into an initially empty min heap assuming there and of these right.

DS Lecture 17 Page 4 of 20 Transcribed by <https://otter.ai>

19:55

So probably $n \log n$ in right? So we know that are doing Men in our inserts are both login operations individually, because they move at most one step per level of the tree, which is guaranteed to be balanced because it's complete. So each insert is a log of an operation. Now, admittedly, the first one is very cheap. And the second one is a little more expensive than the third one's a little more expensive. But it's a similar sequence to what we saw with linear algorithms. So you're doing an order log in inserts, so it would be $n \log n$. So that's not necessarily the cheapest way to do this. There is another operation for constructing a heap out of an initially completely unordered array of values, called the builds heap operation. Now, the builds heap operation says, well, let's say that each of that this original array actually represents a complete binary tree. Right? If you have an array that's filled in with no gaps, you can treat that as a complete binary tree. And so let's draw the complete binary tree that this array would correspond to. Because the three is the first one, the three would be the root. And then this is a level order traversal, we'd have a two and a 35, and a seven, and a 21. All right, oh, let's make the slightly longer, let's give another value, let's say 16. And so that would be the complete binary tree associated with this array of numbers. Now, clearly, this is not a valid heap. But maybe we can make it a valid heap. So the first thing we're going to do is we're going to go to the midpoint of this array. Now, in this case, the midpoint is going to round down, right, because it's an even number. So we'll use this as the midpoint of the array. And interestingly enough, the midpoint of the array actually corresponds to sort of the bottom right most interior node, do we agree? So it's sort of the bottom right most interior node. And so if I were to work my way backwards from the midpoint down towards the first element, I'd sort of be going from right to left along the lowest level of interior nodes, and then from right to left on the

second lowest level, and so on until I get back to the root node. Now, because this is the lowest level of interior nodes, we know that the things that are below these nodes have to be leaves, because it's the furthest down you can go without being a leaf. So what you can then do is a sequence of percolate downs, we start with doing a percolate down on that midpoint. And if I do a percolate down on that midpoint, what I do I look at my two children, or in this case, one child, compare against the smaller the two, and then swap them if need be. So with the 35 gets swapped with the 16, based on the percolate down operation we did before, yes. So the 35 would land here. And the 16 would come up here. And now I have a question from this point down. Is that a valid binary heap? It sure is, because we were guaranteed from where we started, that there would only be one level below it, which and those would be leaves. And so that means this is a perfect design, not perfect. This is a binary heap now starting at this node. And then we work backwards. We go here in the array, which corresponds moving backwards from the rightmost second right most now this node here. Well, it has children that are leaves, right? So we know that it's children are correct binary heaps, because they're just individual nodes. And so now I

DS Lecture 17 Page 5 of 20 Transcribed by <https://otter.ai>

percolate down again, do I need to swap the two at all? Why

24:50

know the parent needs to be smaller than the children and the smaller the children, the seven and the two is smaller than the seven. So this by itself is now a valid binary heap. Agreed. So now let's move backwards again. Well, now we're going back up one level. But we know that we already percolated down all of the things below it, because we did the lowest level of interior nodes first. So everything below this point is a valid binary heap. So now let me percolate this one down. It's two children. The smaller the two is the of the two children is the two Do I need to swap? to percolate it down? Yes, the three is larger. So the three comes down and the two comes up. Now I keep percolating down, right, I keep moving down until I'm either correctly situated or relief, smaller, the two children is seven, do I need to swap it all? Good. And then I move backwards. And I realized, Oh, I'm done. I'm at the root. And because at every point, everything below where you were doing, the percolate down was already guaranteed to be a valid binary heap. By the time you do it at the root, the whole tree is a valid binary heap, you can see it right there. So this is what's known as the builds heap operation. You start at the midpoint in the array percolate down, and then just keep working backwards, percolating down until you've gone through that top half of the of the nodes. How would you reach the last interior node? Which one do you mean? You mean, the bottom right most wonder the route. When starting out, you know how to access it's the midpoint of the array, half the so in a complete binary tree, half the nodes are leaves, and the other half are interior nodes. So by going to the midpoint, you know that. Cool. This is the build heap operation. Now, it turns out that unlike what we were describing originally, this is actually linear. This is actually an $O(n)$ order and operation. Now, I'm not going to prove it. But I'm going to give you a little bit of intuition on as to why. Okay, at first glance, you might be saying, wait, no, you're doing and over to percolate downs. And you made it clear to us, apparently, you right now, you made it clear to us that a percolate down was order $\log$ of n in the worst case, right. And so if you're doing N over two $\log$ of n percolate downs, wouldn't that just be N over two $\log$ n , which is big O of $n \log n$. And at first glance, that is a reasonable analysis. But there is another sort of a little more insightful thing that one can notice about this is that the maximum number of swaps, that I will ever perform in this build heap operation is the sum of the heights of the nodes, right, because all the leaves, which have heights of zero, we don't do any percolate downs for the leaves. all the nodes that are one level above the leaves all have a height of one. And for each percolate down, we did on those the worst case number of swaps we could do is one, all the nodes one level above that the worst case number of swaps is to it tight. So in fact, the worst case number of swaps is really the

sum of the heights. Now here's where I'm not going to prove it. It can be shown. And the proof for this is in the textbook in I believe chapter 6.3. It can be shown that the sum of the heights of a complete binary tree is $O(n)$.
DS Lecture 17 Page 6 of 20 Transcribed by <https://otter.ai>

heights of a complete binary tree is $O(n)$. So there is a telescoping proof, which is a little bit nasty. But if you feel like going through, it'd be my guest in 6.3, that proves that this ends up being on the order event.

29:39

And so that extra little bit of insight that actually gets us a bit beyond just the sort of the crude Oh, each of these is a login operation allows us to get that order in cost. So in that second heat problem that I asked you on the homework where it says using the linear algorithm It's actually talking about this one, go to the midpoint perfectly laid down and work your way back up to the root, percolating down as you go. Any questions on that? We're good. Okay. So I'll make a couple of comments about some other operations that might see in a heap implementation. The Java heaps don't actually have these operations. But there are a few others that are mentioned in the book, which can sometimes come in handy. And so one of them is what's known as a decrease key operation. And that's where you jump to a particular location in the binary heap. And you say, hey, that locations keys value will be decreased, you'll make it smaller. So if the key value the value you're comparing on a node, somewhere buried in the tree gets smaller, which way in the tree? Would it have to move potentially to maintain the heap ordering? have to move up, right? Because the smaller the key, the further up in the tree it is. So a decrease key operation amounts to find the node that whose key you want to decrease, lower its key and then percolate up. And then there's the alternative, maybe you want to increase something's key, which is where you want to make its key value larger, which direction would you have to percolate in if it's, you'd have to percolate down. And so you would find it in the tree and then percolate down, if you want it to decrease the key, sorry, increase the key. Any questions on that? All right. So at this point, we may come back to the paper in a little while. But at this point, what I want to do is I want to actually go through some of the code for this stuff. So let me switch my chair.

32:10

And what you should be seeing on your screen now should be my terminal, where you should be seeing the contents of a file called binary heap, dot java. And the binary heap dot java, this is one of the waste code files that you can pull from that code pack that you should all have access to. And this implements sort of the basic insert, delete men find men, which has just returned the root really, and also has an is empty and to make empty. Now, this is using the array based implementation that we were doing on paper. And it's a growable array. So whenever you fill up the initial space, it'll just double the size of the array to allow you to put more stuff in. So there's no limit in size, just that you have to copy

DS Lecture 17 Page 7 of 20 Transcribed by <https://otter.ai>

the array over if it gets above the current capacity. Okay. And so as we go through this, the first thing we need to look at. So interestingly enough, there is a built heat method. But what you'll see here, these are the public operations, the public methods, in this implementation, the build heap actually ends up being private. Because what we'll do is we'll actually have a constructor that takes in the unsorted array, and that constructor will use the private build heap to build up the array initially. So that's sort of subsumed by the constructor during Does that make sense? All right. So these are the public methods. So the first thing we need to look at is we need to look at our instance variables. So let's look at our instance variables. Oh, actually, the first thing we should look at is that our class binary heap is generic. Our type variable, any type is said to extend comparable on itself for any of its super classes. This is exactly the same thing that we had done when we did our binary search trees and our AV ELLs on the comparable is what we use to order things based on their keys. So we assume that the key the priority that we're ordering on the

ordering is just taken care of by the comparative method. Does that make sense? Excuse me. All right. So now on to the instance variables. So here is our section actually isn't technically an instance variable, but we have a private static final called default capacity, that will be the initial capacity of the binary heap, which will grow as needed. So if we set an initial capacity to 10, that means our binary heap should be able to hold at least 10 items, what should the size of the array B, if we want the initial capacity to be 10? Good, it should be 11. The 10 items plus the zero space that will not have real data will actually will technically use that as sort of a temp variable. Okay. We have our current size, which is distinct from capacity, that's how many items are currently in the thing. And then we have our array of any types, which is actually what's going to store the heap. So those are our instance variables. Now, let's go up to our constructors, and we actually end up having three of them. The first No, our constructor just assumes you want to create an initially empty binary heap of the default capacity. So that will in turn invoke the overloaded constructor that takes in an initial capacity, which we see here. This one takes in the initial capacity, it indicates the current size, because you start out empty in this case, it knits the current size to zero, and then instantiates, a new array of note here we're making it an array of comparables, you can create an array of any sort of valid type. So you can create an array of things that can be set equal to anything that implements comparable, it does this instead of object just because when you cast it to the any type, it will complain if you try to cast it from plain old object because these things actually must implement comparables. So that makes sure that the types match nicely. So we instantiate a new array of comparables of capacity plus one, because we need that extra space for the zero node.

37:11

We cast that comparable array to any type. And we set our array instance variable equal

DS Lecture 17 Page 8 of 20 Transcribed by <https://otter.ai>

to that. And now we've got a nice initialized array. And we're ready to go. The third constructor, which I'm going to put off until the end, actually takes in an initial array of items, and then that goes through and does the builds heap operation on it. So we'll come back and look at that, after we've made it all the way through all the other methods and have done building. Alright. So that instantiates our array. And now, let's start with our inserts. So this is going back to last week's or last Wednesday's lecture. And on last Wednesday's lecture, we saw Well, one passed the last element one pass the current size is where you want to put the thing to maintain the completeness, and then you percolate up. Now, of course, we want this array to be growable. So we make sure that the current size is not already equal to array dot length minus one. If the current size is equal to the array dot length minus one, well, that means we've, we've ever add a space. And so we enlarge the array to something that's twice the original sides plus an extra one for the empty space. Now, the enlarge array I won't go through, it's just the plain old instantiate a new array, copy the old one over, it's the same method that you would have seen in your ArrayList implementations way back at the beginning of all of this. Alright, so once we reach this point, we're going to go ahead and assume that our array is big enough to hold us. And so we create the hole in which we want to insert the value x, and we create the hole at plus plus current size. Right, one past, the current size is the location of the new element. But while we're at it, we might as well also increment the current size variable, because while we're inserting something, so the size will change the plus plus happening before the current size variable, as you know, just make sure that the increments happens first before the assignment. So now we have the index of the hole. And we need to percolate the hole up until you can safely put x into it. Now that means that we're going to keep comparing against the parent of the hole until x is a well until x is greater than or equal to its parent, as long as x is greater than or equal to its parent. That means I don't have to percolate and then I'm done. If x is never greater than or equal to its parent,

where do I stop the percolate ups? Anybody?

40:16

Not at home, but at the root. Good. Now, remind me what know the route is that what index? One? Excellent. Now, if you tried to calculate the parents location of index one, what would you get? Just from the basic calculation, not know, not the value what index it would be index zero, right, because one divided by two with integer division gives me zero. Now that zero location wasn't supposed to hold real data, but what we can do, and you can see it right here. The for loop that does the percolate up has an initialization phase, the assignment that happens at the beginning, before we start going through the iterations. And so what we do here is we're actually going to set array at location zero, that's the node that doesn't have any real data that we're just using a temp as x. Now, here's the fun part, you're going to keep percolating up until x is greater than or equal to

DS Lecture 17 Page 9 of 20 Transcribed by <https://otter.ai>

its parent. When you reach the root, if you just blindly calculate the parent, the parent is at location zero. And if x is already there, our basic check is x greater than or equal to its parent will come out as true, right? Because x will be equal to itself. So by putting x temporarily at location zero, it means we don't need an extra check. To check to see if we're at the root, it means the basic check to see if we're greater than or equal to our parent will cause it to stop automatically just by comparing against the zero value. Does everybody okay with why I've done this, and by I've done this, I mean, why once is done this? Yeah. Okay, so that's our initialization to sort of Oops, that's our initialization to sort of allow us to stop when we reach the root. Now, here is our condition to keep percolating up, we do x compared to array at Whole over two. hole over two is the index of the parent. So array and hole over two is the value of the parent. So if that whole compared to is less than zero, it means x is smaller than its parent, which means they have to swap. And so that's what we do, we set array at Whole equal to array at Whole over two, we take the value at the parent and we put it into the whole and then the increment is whole divide equal by two, which is whole equals hole divided by two. So we slide the parent down into the original hole, and we make the hole the parent and then we compare again until we end up not being less than our parents, which will either happen when we're in the right spot, or when we're at the root because of this initialisation. So once this loop ends, that means hole is at the spot where I can play sex. So I said array at hole equal to x, and my insert is done. Do we have any questions on this insert operation?

44:01

were good. Okay. What do we mean by the root return? Right so so the root is always at index one. Right and so we'll put x at the hole which may or may not be at the root. Oh, cool. So we're good. Cool. All right. Anything else?

44:39

Okay, so that's our insert and I just jumped the screen Sorry about that. So now let's look at the Delete min but before we did the Delete, man, we might as well so do the Fineman.

44:52

So the Fineman is easy if the thing is empty, which just what did I just do? Hold on.

45:07

DS Lecture 17 Page 10 of 20 Transcribed by <https://otter.ai>

So let's look at is empty is empty just returns true if current size is equal to zero, right that's what you would expect. So in our Fineman method if the thing is empty we throw an underflow because there's no min

45:32

we simply return the ray at one. So remember the Fineman operation which is clearly an order one operation. The find min operation is sorry. It says my internet connection is unstable. Did I did you guys lose me there at all? Can you hear me now?

45:54

Okay. A bit. That's that's worrisome. Oh, it's okay. Ever. We're sure it's okay. Now. Does anybody need me to explain the fine men again? Are we good?

46:12

I sideboard for a second? No, exactly. Okay. All right. So fine men, if it's empty throw an underflow if not return array at location one because that's where the smallest item always exists. That's order one. Because the Fineman doesn't actually have to remove anything. Okay, Christopher, we're good. Cool. So now the Delete. So just like with the Fineman, we check to make sure we're not empty. If we are empty, we throw an underflow exception exception and be done with it. If we're not empty, while just like what I did before, on paper, I store the minimum value when we use Fineman for that in a temp variable, which will note that once we're done readjusting the tree, that's what we return because we want to return the minimum. And then I take the item at array at current size minus minus. So I take the current size. So if there are 10 items in it before the Delete man, I go to location 10. I extract the value at location 10. And I put that value into array at location one, that's basically temporarily putting it in that hole at location one that happened when I extracted the minimum. And then I do the minus minus afterwards. So that's current size minus minus. So it takes the original current size extracts the value and decrement. Because Yeah, the array is now one smaller. And then I perform the percolate down operation, assuming the hole is at location one. And that the value currently in the hole is what we're trying to insert into it. Now, you will note that in being certain method, which had to percolate up, I didn't font or waist didn't functionally decompose. Excuse me, the percolate. Excuse me, the percolate up into a separate method. And the reason for that is because it's only used there, nowhere else. But the percolate down that we have here is used when we do our builds heap operation later. So it makes sense to store the percolate down code in a separate method because it gets used many times. So

DS Lecture 17 Page 11 of 20 Transcribed by <https://otter.ai>

assuming the percolate down works. When that rearranges the tree and then we return the minimum. So before we go through the percolate down, is everybody okay with how this delete min works, assuming percolate downwards. We're good with that. Alright, so now we need to look at the percolate down operation. So that's an a helper method that's private. So, here we go. The percolate down method takes the index of the hole that we want to percolate down. The reason why it takes that is because if you were calling the build keep operation, the percolate downs can start sort of midway through, I'm going to need to ask you to hold on for a second I need to cough very badly and have a drink Hold on one sec.

50:04

Sorry, I find the coughing Yes, I agree. But that's not what I have in front of me. The coughing I find mildly alarming because I had had to spend something like six hours in the emergency room on Friday. Which means I ended up like in contact with lots and lots of people, which I did not like. And now I'm coughing, which makes me uncomfortable. Anyway, moving on. I'm fine. Really, the ER was fine. It was. Let's just say that the second dose of the Pfizer vaccine kind of hit me in the head a lot harder than I expected. And I had a bit of a problem. But then I felt fine. 12 hours later. Yeah, it was a weird weekend. So Oh, also, my apologies to the folks whose meetings I had to cancel on Friday.

51:00

Yeah, that's what happened. I was I was I was at the hospital. Um, alright. So

51:12

that's our channel now eligible for it in New York State starting like next week, or something? Yeah. Okay. All right. Yes. I think most of the people in the room are like 19. But anyway. Okay.

51:34

All right. So percolate down. Sorry, that was that was a little mean. Sorry about that. made you feel any better, I'm still like, at least a decade older. At least. I'm sorry. Okay. So let's get back into the percolate down now that I've cycled back around to coughing again. Alright, so this is the whole and temporarily, the value that we're trying to insert in the hole is actually sitting there. So we're going to need a variable called child. So we're

DS Lecture 17 Page 12 of 20 Transcribed by <https://otter.ai>

just going to declare that for a moment, that's going to be the index of their smaller child. And then we take a temp variable, and we use that to store the value that's in the hole right now, that's the thing we're trying to percolate down, we put that in the 10th. And now we have to actually do the work for the percolate down. So what's going to happen is that we don't need to initialize anything. So you can leave the initialisation field of the for loop blank, if you don't need to initialize anything. And then we're going to check to see if whole times two is less than or equal to current size. If hold times two is less than or equal to current size, what does that mean about the location of all right?

53:03

So it means specifically, it's not a leaf, it means it's an interior node, right? If hold times two is the index of the left child. So if the left child's index is less than or equal to the current size, that means that the left child exists, so hole must be an interior node. Now to the person who says it's the far right interior, you'd be correct if it were the equal to the current size, but this is asking for less than or equal. So it includes that in many others. Are we good? Cool. All right. So as long as we're not a leaf, we keep going. And so with the percolate down, there's supposed to be two stopping conditions, the first stopping condition is that we are actually smaller or equal to our smaller child, which case no swaps are needed, but that we're going to manually break out of, and so the condition in this one just ends if we hit a leaf. Alright, so if we're not a leaf, the first thing we have to do and I'm going to highlight the code that does that. The first thing we have to do is calculate the index of the smaller of my tubes of wall of holes to children. So I start by setting child equal to hold times two, I start by setting child equal to the left child and it's good to start it at the left child, because in the case you happen to be the bottom right most interior node, you only have a left child, all right. So you start child at the left child and then you check to see if child is not equal to current size. If child is equal to current size, it means there is Only a left child. And that's the smallest, nothing to update. But if child is not equal to the current size, that means it has a sibling. So then we extract the value add child plus one, which is going to be its right child of whole. So array A child plus one, compared to array a child, if that's less than zero, it means that array A child plus one is smaller, it means the right child is smaller. So child started out as the left child, but if the right child is smaller, we increment child by one, making it the right child. So when these three lines of code are done, child is equal to the index of the smaller of the two children. Does everybody agree? Good. And so now that we have the smaller of the two children, we compare, if that child is smaller, excuse me, then the parent, then we slide the child up into the hole. And then we set the hole equal to the child, so we swap them, and we keep going. If that smaller of the two children is not smaller than the value of the hole, it means we're done. And that case, we break out of the loop. And whichever ends the loop, either that we're at a leaf, or that the relationship with the smaller child is correct.

DS Lecture 17 Page 13 of 20 Transcribed by <https://otter.ai>

We know that hole is where the temp value goes. And we set array at Whole equal to the temp value. Do we have any questions on that? We're good. Exit. Excellent. Okay. So that is our percolate down. Now take it in conjunction with the Delete min we had before we now have a fully working delete min. Now, you'll notice that below delete min is this private builds heap method, we're going to use that in that third constructor that we had skipped earlier. And if you recall from the paper, we saw a little piece of paper we saw a

little while ago. For the build here we go to the midpoint of the array. So we set it equal to current size of a two, we know that that's going to be the bottom right most interior node. And then we percolate that down using the method we just went through. And then decrement i moving back along that bottom row, then along the row above it from right to left until we get to the root. And that's all the builds heap is once we have the percolate down written. Any questions on that? Wayne, go ahead.

58:02

So does that mean that from the midpoint in the array, everything after the midpoint is going to be? leaf? a leaf? Yes. Okay. So cool. There's another proof, which I don't remember if I had it on the if I don't remember if it was in the homework, or maybe something I can put on the exam? I don't know. I'll think about it. Um, that said that. That said that the the number of leaf nodes in a perfect binary tree is equal to two to the n , where n is the searches a to H , where h is the height. And then you may remember what we proved was that the total number of nodes is two to the H plus one minus one. So in that perfect tree, literally half the nodes are leaf nodes. And so this is exploiting that. Okay. And private question. You're sure we're good? Okay, cool. All right. So that is the builds heap operation code wise. Once we have that percolate down, it's really nice and compact. And so that'll bring us back up to Alonzo. Second. That'll bring us back up to our constructors. And so the final constructor, the one we skipped over, was the one that took an array of items that we wanted to initially populate the heap with. Now, first of all, this is assuming that the array of items that I give you starts at 00 indexed, but for our binary heap, we actually want it to be one index. And so the first thing we do is we allocate our array, it doesn't the current size plus two times 11, tense, whatever, we allocate the array so that it's big enough to hold the items plus a little bit of padding plus that extra zero. And then we copy the items over into the new array, but shifting it over by one, so it starts at one instead of zero. And then once that's done, we just run the build keep operation, and that constructs leave. Any questions on that? We're good. Okay, so that's the implementation of the heap. And, wow, it's only 640. Super, we're gonna start talking about sorting algorithms today. And so the first way we're going to sort is actually we're going to stay in the heap chapter for a moment. And we're going to look at an application of a binary heap. And the application of the binary heap is going to be let's sort the thing. Now, what you should have realized by now, is that excuse me that keep ordered is not

1:02:08

Oh, wait. Yeah, keep doing return min. So we already built heap.

1:02:19

But once I built the heap, delete min, and that'll give me the smallest item, which I can now put at the beginning of a new output array. And then I delete min again, and put it just after that first item in the output array. I keep deleting min, each delete min costing me $\log$ of n time, until I've deleted all of them. And they plop off in sorted order. So I'm performing and delete means that's $n \log$ of n operations. So the total cost for using a heap to sort something is $n \log n$ plus n . What's that plus n for anybody. It's for that builds heap operation, which is a prerequisite for all of this. So this is what's known as heap sort. And heap sort costs in for the builds heap, plus $n \log$ of n . For the repeated delete mins, that ultimately give me the sorted list. Now that n gets drowned out by the $N \log N$ portion of this. So it ends up being big O of $n \log n$. Now, the one problem with the heap sort that I described to you is that it is not what's known as in place. An in place sorting

algorithm is a sorting algorithm that doesn't need more than constant extra space. It's a sorting algorithm that uses the existing array for the sorted items. No, you have to pull the heap apart to get the sorted items out. Um, but what I did here was I destroyed the heap and then I copied the items into a new array, which means that uses order n extra space. And so you can actually implement heap sort a little smarter than the way I just described, you can actually implement it in place, but you can't use a min heap for that. You actually have to use a max heap. Alright, so if you want an in place heapsort you have to use a max heap. So a max heap just slips the condition. So you have an insert method and a delete max method. And so in the max heap ordering, the parent must, instead of being less than or equal to its children, the parent must be greater than or equal to its children. Okay? So let's imagine we have a max heap.

1:05:30

Okay, so that's Maxi bordered. In fact, if we look at the related tree to this array, it would look like this 35 17 21, three, and four. So that's Maxi bordered. So this is my array.

DS Lecture 17 Page 15 of 20 Transcribed by <https://otter.ai>

1:05:51

Ignore the zero element for a moment, you can use an offset and not have to worry about that. When I delete min, sorry, I don't delete min here, when I delete Max, what item comes off maybe the 35. So 35 will come off. A hole will be created here, I pick the bottom right one to fill the hole right, so I'll try to fill this with a four. And this item now goes away. This space remains empty. So now I need to percolate my four down. And so instead of comparing against the smaller the two children, I now want to compare against the larger the two children. So I would swap the four with the 21. And then I would be Maxi bordered. Okay, and so my array would end up looking like this. But notice, I have extra space right at the end. What can I put in that extra space right at the end? Anyway, input the 35 here. It's not part of the heap. But it's the largest element. So it's now in the correct spot. And so when I delete max again, the 21 comes off. Right? I put the three here, we're trying to put the three here, it percolates down. So the 17 lands here, the three lands here, now this space is empty. The 35 is there. Now I can put the 21 into the space that I just vacated. And you'll notice that I'm filling the array up with the maximum elements in the space that I'm vacating as I go. And so by using a max heap instead of a min heap, I can actually do my heap sort in place. Thank you still $n \log n$ in time with that plus and at the beginning to do the bility. Just this time a maxi. Any questions on heapsort? We're good. All right. So now let's just quickly introduce sorting. More formally now we're really in chapter chapter seven now. Now heapsort is a little bit of an outlier. Because it's a non recursive algorithm that attains a $\log$ of $n \log n$ costs. But we can sort of split our sorting algorithms into comparison based algorithms. And those are ones where we're comparing every item to every other item. And so our comparison based algorithms for general purpose sorting, by the way, these are general purpose sorting algorithms, they work on anything that can be ordered. Alright, so we don't know anything other than we can order them. And so the comparison based ones compare every item against every other item in the worst case. So these are going to be worst case, order n^2 . But there are also divide and conquer algorithms. For example, merge sorts, which take the approach of splitting things in half, and then fusing things back together. And the divide and conquer algorithms actually tend to give me an $N \log N$ costs. Because again, if you're dividing Everything in half, many times you get sort of that $\log$ base two of N factor in. Now, in terms of sorting, general purpose sorting, we actually learned that that general purpose sorting, we're not going to prove it, but you can read about it in the text is big $\Omega n \log n$, you're never going to do better than $n \log n$ for general purpose sorting, that's as good as it gets.

1:10:24

DS Lecture 17 Page 16 of 20 Transcribed by <https://otter.ai>

The comparison based algorithms though, because you're comparing every item against

every other item, in the worst case, are never getting any better than n^2 , which means you can always do better than them with like a heapsort, or divide and conquer algorithm. But the thing to note about these is that they tend to have a fairly big constant factor in front of them, or require extra space or something like that. And so if you have a very small end, because the comparison based algorithms don't have a particularly big constant factor associated with them, for very small ends, these can sometimes be better. So we still use them in certain limited cases. Now, there are a few comparison based algorithms, I'll name three of them, but we'll only talk about two of them. There is selection sort. Which I'm sort of assuming that a lot of you saw in your 1004 class, there is the best of them, because it does a little better. In the best case, there is Insertion Sort. Which I also know you do in 1004, it sort of comes towards the end of the first lecture slides and pseudocode I think, that usually scares the ever living crap out of people when they see that I know this, because when I teach tenos for I actually use Adams slides for that lecture. I think the only difference between how I present it now he presents it, as I say, Hi, this is gonna scare the crap out of you don't worry about it. But anyway, um, the last one is bubblesort. I hate to say this, but I don't like bubblesort. I don't know why I don't like bubble sort selection sort. It's objectively worse, but I don't like bubble sorts, you know what we're not going to do bubblesort. We're going to talk about selection sort and Insertion Sort. And, in fact, the textbook doesn't even talk about selection sorts, the textbook starts out with Insertion Sort. So we're going to start with selection sort. Okay. And in fact, this is quite possibly one of the few things that I'm going to do all semester that isn't actually directly in the textbook. Alright, so selection sort, is an algorithm that actually will start to look rather similar to heap sort, except it isn't using a binary heap, and therefore its cost ends up being terrible. What we do with selection sort, is we select the item from the list of items that should be first, then we select the item that should be second, then we select the item that should be third, and so on, we're finding men repeatedly, until we found them in n times, and have filled up our sorted array. Is everyone alright with that, just keep finding men until you have a sorted array, that selection sort.

1:13:46

So in terms of I'm not going to code it. But in terms of just a diagram, let's just take a look at an unsorted array.

1:14:00

And so what we could do is we can sort of think of the array as being partitioned into a sorted portion and an unsorted portion. Initially, the whole thing is unsorted. And we go

DS Lecture 17 Page 17 of 20 Transcribed by <https://otter.ai>

through and we find the smallest item in the unsorted portion. Now, we don't want to use this extra array. So what we do after we find that smallest item, we swap it with the item at the front, so the one goes here, and the five goes here, and there's no harm in swapping it because that five was unsorted. So it doesn't matter where you push it. And so after that operation, the array will look like this. You don't want to push the items over because that requires copying them over. You just want to swap it with one. So the array will look like this except now that's the sort of portion. And that's the unsorted portion. And we just keep finding min

1:15:07

swapping with that first item. So now our array looks like this. Find the min as the forest swapping into place. Find the minutes the five, swap it into place, I'll just write it over here.

1:15:33

Then we've got one item left. And we confirm that it's the minimum of itself and move on.

1:15:40

Now, in the best case scenario, in the worst case scenario, you have to go through all n items to find the minimum the first time plus all n minus one items the second time, plus n minus two and so on plus one at the end. And you know that that adds up to n times n

plus one over two, which is just equal to big O of n square. Now, that's the worst case. But what about the best case? In the best case? Well, you get a sorted array, but you know what? Finding mechanisms any cheaper if you already have the minimum because you don't know it. So the best case is still order n squared. Because you know what, you still have to go through the all the items each time you find min. So best case, worst case, average case, all cases, it's n squared. And so with our other one Insertion Sort, what we really want is to find a situation where Yeah, in the best case, in the average case, it's always going to be n squared, sorry, in the worst case, in the average case, it's always going to be n squared. But we'd like it to do better in the best case. And that's what insertion sort is going to give us. So we'll pick up with insertion sort on Wednesday. And from there, we'll do merge sort, and the divide and conquer. Okay. All right, folks. Have a good one. I will see you on Wednesday. Hi, professor. Quick question. Yeah. So for given list of unordered items, and we use first we use insert to put them into a heap, but then on another ability to play. Okay, so if we use the two different methods, is it possible that the two resulting trees look different? Yeah, yeah. totally reasonable. Because as I said, if I give you a set of numbers, and I said, given me those is a min heap. Because there's no relationship between the left and right siblings. It doesn't matter. And so there are many valid heaps that could come out of this, the same numbers. And so if there are different ways to construct them, that could result in different trees. As long as all those trees are still fine. still valid min heaps, that's fine.

1:18:27

Okay. We're good. Yeah. Other questions? This is regarding the homework question. Would it be a smart idea to store like, I guess, the order that you follow, like left to left to right, right. To get to that node, this is for

1:18:47

the implicit position? Yeah. Yeah. Would it be something like that is kind of what I had in mind.

1:18:54

Okay, sounds good. Thank you. Other questions? Yeah. Sorry. Go ahead. Sorry, quick follow up. With this space constraint, make it not dual. It can be stored in an array or something. I don't mind. Okay. Sounds good. Thank you. All right. There was somebody else trying to ask a question. Go ahead. Yeah,

1:19:15

I wanted to ask if you explain heapsort again, I got lost somewhere in the middle.

1:19:22

So remember, with heapsort, you could repeatedly delete men, right? And so if you repeatedly delete men, you get the smallest item than the second smallest than the third smallest and so on. Does that make sense? Yeah, you fill that into an array. Now what I had done in the second half was say instead of using a min heap, use a max heap. So that means each time you delete Max, and so instead of putting each delete max the beginning of the array you put it at the end. Okay? And so the nice thing about that, is that because the Maxi is sort of shrinking towards the front as you delete Max, it frees up space at the end. So you don't need an extra right.

DS Lecture 17 Page 19 of 20 Transcribed by <https://otter.ai>

1:20:08

Awesome. Thank you so much. Cool. Anything else? Thank you professor. Take care. Yvonne, did you have anything else sir? Sorry, it has you highlighted I guess because you're the last person to speak. Okay, I will see you all on Wednesday.

DS Lecture 17 Page 20 of 20 Transcribed by <https://otter.ai>

DS Lecture 18

Wed, 8/11 1:53PM 1:24:19

SUMMARY KEYWORDS

array, merge, quicksort, sorted, item, algorithm, divide, case, pivot, equal, constant factor, swap, partition, write, question, sorting, insertion sort, cost, left, split

01:47

Just got to come to don't say nothing. Get your hands on sex. That's the place

04:03

Come on.

04:06

Shut your mouth.

04:12

Don't say nothing

06:06

Okay, let's maybe get started. Um Hold on one second. I just want to Sarah captioner here. Oh, see anybody? Yeah, sorry. It's it's Rebecca Koster. I just want to make sure I just didn't see. Okay. You should be all set now.

06:42

Okay.

DS Lecture 18 Page 1 of 23 Transcribed by <https://otter.ai>

06:45

Okay, um, so we should get started. You can tell the semester is, is coming to a close as more people time shift the lecture. We're going to play a little game with that not today, but sometime late next week, we're going to have a little fun with that. run some experiments on the class. Um, anyway. So we have more tears than usual here. Did any of you have anything specific? You wanted to share? You all just answered? Excellent. Okay. All right. So you guys had a homework due on Friday? Correct. Do we have any pressing questions about the homework due on Friday?

07:41

We're good.

07:46

All right. Um, that's encouraging. Everybody seems to be either really on top of their game or really, really tired and wants me to shut up about this and move on. haven't decided which you will also have a programming assignment that I do believe we said was due on. Hold on one second, there's a slight technical issue. Give me one second.

08:32

Sorry about that, I think we resolved our little technical issue. Excellent. Um, so we are good. So we have a programming assignment due on Tuesday. For those of you who started it, you may have noticed that it is distinctly shorter than homeworks, two and three, it might not look like that on the surface. But once you actually go ahead and start coding up the second problem, you will discover that you can solve that problem in I mean, if you comment it, maybe 3540 lines of code, if you're really expressive in your comments. So it's more of just figure the algorithm out. And once you do, it's all good. The other one is a little bit more in the vein of the things we were doing last time, but overall, I think it should be an easier experience. We will have a homework five, which I said will be shorter. Homework five will be well, the written part of homework five will be based on some of the work we've been doing. And some of the work that's upcoming the programming problem will be based on work that I'm hoping to get to by the end of next week. So you can either read ahead or you have about a week to do that programming

problem, but it is a single problem. Okay. And it is. It does involve a graphical user

DS Lecture 18 Page 2 of 23 Transcribed by <https://otter.ai>

interface of sorts. But not one that you have to write, we will be providing you with a big chunk of code that you will be able to just sort of drop in the part that I need you to implement. But that will be based on material from chapter nine that we'll be doing next week. So that's our schedule, everything seems to be going pretty well. We'll talk more about the final exam, I guess later on, just remember that you have basically the last three days of the final exam period to do it. I the specific dates are on courseworks. So it's sort of straddles when the registrar set our exam period for and it's the same structure as the midterm was, so you'll have an enormous amount of time to get to the exam. It's timed once you started. Okay. All right. So last time, we were talking about sorting, and we had gotten as far as discussing selection sort, which we sort of, I like to call it our sort of trash sorting algorithm, right? Because it is a comparison based algorithm. It is order n^2 in the worst case, but it's also order n^2 . In the best case, because it's based on find min, and find min takes order n time each time you do it no matter what. So what we want to do first before shifting to the divide and conquer algorithms that will hopefully get us back to $n \log n$ in time, sort of like the hat the heapsort did, we want to talk about another comparison based one that is a little better called Insertion Sort. And in fact, insertion sort is good enough that for very small values of n , because it has a much smaller constant factor than some of the $N \log N$ algorithms do. So for very small values of n , sometimes insertion sort is our choice. So it is actually used. Now the nice thing about insertion sort is that while in the worst case, and in the average case, it's still an n^2 algorithm, we're not getting away from that, in the best case, it can actually do a little bit better. And sort of locally in sorted portions of the array, it can do a little better. And so before we get into that, let's discuss a related algorithm. Imagine I gave you an array, and I simply wanted you to tell me true or false, whether or not the array was sorted. Can you do that in linear time? And if so, can anyone suggest how they by? Daniel, go ahead. You could go through any two consecutive variables and check that the second is greater than the first. And if at any point, it's not you return false. Okay, good. So the suggestion here is that you are looking at the elements pairwise. So you start at the beginning, look at items zero and compare it to item one, if it's sorted, item zero should be less than item one, then you go to, I don't think anyone of you can actually see the pen.

13:27

ascending order we're talking about here, you can just reverse it for descending. So then you look at item one, and well, yes, less than or equal to because we can deal with duplicates. Good point. And so you will get item one and then compare that to item two. And as long as item one is less than or equal to item two, we're still good. And you keep doing that up through item n minus one, right up to the second to last element, looking at the next one. And so with this pairwise comparison, if you make it all the way to the end, and you find that that's always true, you can return true order and time. And so what we

DS Lecture 18 Page 3 of 23 Transcribed by <https://otter.ai>

would like, we don't really want to have to do that check first, and then going into the sorting algorithm, because that's just extraneous work. What we want is a sorting algorithm that will essentially devolve into that verifying order and algorithm in the event we happen to give it something sorted. Now, you could also imagine starting at the end and working your way backwards, making sure the last element is greater than or equal to the item just before it and work backwards. Again, we're always sorting here in ascending order. Then if you want it in descending order, just flip the comparison operators from less than or greater than or vice versa. Okay. Alright, so insertion sort the way we're going to do that. I'm going to go ahead and write down some arbitrary values.

15:01

That was probably a little overly ambitious. So I probably won't softball thing. But the way

we're going to look at this, I'm going to start out in a way that looks sort of similar to the way I had written selection sort. And we're going to partition this array into an unsorted portion and a sorted portion. Now, the way we determine which part is, which is well essentially just keep a single index, which will always refer to the first item, a sorted portion and an unsorted portion, I did that backwards. You guys always jump on correcting me you didn't, you didn't correct me on that one, I got to it first. Come on, I'm alright. You know, I rely on you guys correcting. So we'll keep the first item in the unsorted portion will keep that as an index. And that's the item we're currently trying to deal with. And so instead of selecting the next item that belongs in the sorted portion, we are inserting that current item, that current boundary item into the correct location in the sorted portion of the array. Now, initially, the sorted portion of the array is empty. So that first item, this is sort of a redundant step, and we end up usually skipping it in the actual code. But what we end up doing is just rewriting it. And now, the array itself hasn't really changed. But now the seven is construed as sorted. And the unsorted portion starts with the eight. And a seven is sorted with respect to itself, right? If you just have a one item array, and it contains a seven in it, you would say that sorted, correct. All right. And so now we keep going by inserting the next item into the sorted portion. And the way this insertion works is we start at the index where it currently lives and we look to our left. If the item to our left is less than or equal to the current item, then we know we are correctly situated with respect to that item. And we therefore don't need to change anything about its order. If the item directly to our left is larger than us, then to get them pairwise sorted, we'd have to swap them and then keep looking to our left until we're at the correct position, or we're at the beginning of the array. And so in this case, there's nothing to swap because the eight is already larger than the seven. So that's just a single comparison. And now the array looks like this, not really much has changed here. Other than the current location of the breakpoint between the sorted item in the sorted items and the unsorted items we insert again, we look to our left, but this time, they're incorrectly arranged, we've got an

DS Lecture 18 Page 4 of 23 Transcribed by <https://otter.ai>

item that is smaller than the thing to its left, so we got to swap them. So the three comes here and the eight goes here. And then we have to look to our left again, and do that check again. And in fact, the three is still smaller. So the three in the seven swap, the seven comes here, and the three comes here, we look to our left again. And we realize that the three is at the beginning of the array, so we can stop. So I'll squeeze this in at the bottom of the page, the array will now look like this 378 there's the break between the sorted portion which is here, and the unsorted portion, which is the nine to one and six. And you keep doing that until you've inserted all of them. Alright, and so now that we have that I'm not going to go through the rest of this because it's just gonna take up unnecessary time. I want to ask, what would be the worst case scenario, what input array when you want to sort things in ascending order would result in the largest number of comparisons and ultimately swaps that needs to be performed? Right. So if we have it sorted in reverse order in descending order. So to show that example, let's give another array in descending order. It starts out you can sort of skip the first step because that one doesn't really count. And so now this is sorted, this is unsorted. To insert that item requires one swap once we got that done,

20:01

To insert, the next item requires two swaps. And the next item requires three swaps. And the next item requires four swaps all the way up to the last one, which could require n minus one swaps. And really n comparisons, because there's one extra comparison, when it reaches the beginning of the list to check to make sure it's actually at the beginning of the list. So that last four would take the full n comparisons to actually make it into place, this one would take n minus one, and so on. And so we end up with basically, usually, we end up with basically the same series that we ended up with, when we were analyzing the

worst case of selection sort. And that's just big O of n square. And that remains the case even in the average case. But now let's talk about the best case scenario. And that's where Insertion Sort does better than selection. So work, the best case scenario would be if I give you a pre sorted array.

21:17

And so if you think about it, there's a single check to make sure that it's in the right spot. But because it's already sorted, that check succeeds without doing any further swaps and checks, then it moves to the next one, does a single check, realizes it's good, moves to the next one, and so on, which really ends up being the exact same scenario that we were discussing, when I asked you to come up with an algorithm to verify that something sorted, it degrades into that. And so in the best case scenario, this is actually linear. Right.

DS Lecture 18 Page 5 of 23 Transcribed by <https://otter.ai>

And you know, they don't have to be you can have negative numbers. These are general purpose sorting algorithms. And what that means is they work as long as the items are orderable. Alright, as long as you can, basically, as long as the items can implement compared to meaning you can say whether or not one item is less than greater than or equal to another item, you can always sort them. So this would work with negative numbers as well. Are we good there? So why is one necessarily in the right spot? Well, that's because in this case, it's sort of irrelevant that it's one, all that's relevant is that it's the smallest item is in the first spot. And the second smallest item is in the second spot, and so on. Does that make sense? Cool. All right. Do we have any questions on Insertion Sort? Now, I feel like you guys have all seen implementations of this in the previous class, like previous like course that you took. And so we're actually going to use Insertion Sort as a tool to allow us to build up one of our divide and conquer algorithms, one that we'll probably won't do until Monday, we'll we'll see if we can get there today. So when I get to the code for that algorithm, it's actually gonna be an algorithm called quicksort. Now, I'll briefly show you Insertion Sort, but for now, I think we're in a good spot. All right. So that's what I wanted to talk about in terms of the comparison based algorithms. What I want to do now is I want to talk about these divide and conquer algorithms. And when we divide and conquer, sort of like binary search was divide and conquer, you split it in half each time, you usually end up when you split things in half each time with a cost that involves some sort of log of n factor in it. And so the first of these algorithms that we're going to look at is an algorithm called merge sort. Now, let me explain to you the way we're going to go over merge sort, the first thing I'm going to do is I'm actually going to write down pseudocode for the merge sort algorithm. And we'll, we'll sort of talk through it and try to understand what it's doing as we write that down. Then once we have that down, we'll actually look at a visualization of merge sort in practice. Okay, and that will hopefully get everybody clear on how it's working. And once we've done that, then I'm going to do a proof of why I liked how somebody was nodding along right up until I said proof and then they just got a face. That was fun. And that person knows exactly who I'm talking about to that's very clear. It wasn't just you, I can only see like 12 faces at a time. Anyway, so we'll do a proof which you're not going to be responsible for. But I feel like it's a it's actually a really, it's a really beautiful proof, and I liked doing it. And so I like it, we're gonna do it. Moving on. Um, so let's start with some pseudocode for merge sort. Now merge sort is going to be a recursive algorithm. In general, when we do these divide and conquer algorithms, they're recursive. And so our merge sort algorithm. Now, for the sake of, for the sake of my own sanity here, I'm going to demo this using arrays of intz. Just so I can use comparison operators, when you actually implement this, you would use comparative. Alright, so we'll say it's taking in an array of intz.

26:09

DS Lecture 18 Page 6 of 23 Transcribed by <https://otter.ai>

A, called a. And we'll say that it returns the sorted into Ray. All right. And so the first thing

we need, since this is going to be a recursion is we need some kind of base case, some a that we could just immediately say with no work just in constant time a is sorted. And so when can I just return when do I know I'm done with no additional work? anyway? Well, you have a one element array, or in fact, if you're trying to sort an empty array, and then that case, a is already sorted. So if a dot length is less than or equal to one, then we just return a. And we are done. Now we're going to do our recursive calls plural. And we're also going to get the moment where this is going to be very clearly pseudocode. And so I'm going to have to use so in the actual encoding of this, we're going to use a single temporary array, I'm going to use multiple temporary arrays, right now, I'm just because it's easier to write the pseudocode. But in reality, we just be passing around a single temporary array, that's exactly the same length of a. And you'll actually see how that comes into play when we visualize this. So for now, though, I'm going to create a temporary array called a one. So we have int array A one. And what I'm going to do is I'm going to set a one equal to the results of recursively, calling merge sort on, and here's where it becomes pseudocode on the first half

28:12

of a.

28:15

Now, just so that we're clear what I mean by that, let's give an example. If say this is a

28:31

Okay, so we got a six item array 381279, then the first half of a would literally correspond to the first half of the elements of A, in this case 381. All right, now by the design rule, we're supposed to write our computation in terms of the recursive call as if the recursive call worked. So what will a one B once merge sort on the first half of a returns? What will a one actually contain? A by so not 381? It will contain them sorted, right? Because merge sorts job is to return the inputs sorted. And so it'll sort that part of the array. So assuming it worked, which is what the design rule saying, a one should end up being 138. Are we good? specifically looking at the folks who said no clue. Alright, and then we're gonna do the same thing on the second half, and we'll have a second temporary array for that reality, we only need one and just have to be careful about what parts of the array rewrite it down in. But for this one, we'll just say we have int array a two and that's going to be recursively calling merge sort on the second half, Okay. All right. So a two will end up like

DS Lecture 18 Page 7 of 23 Transcribed by <https://otter.ai>

an ass, I actually made the second half already sorted. So it'll just return that back, because it's already sorted. Are we doing agree? Good. So a two will be 279. And so now what we have are two independently sorted arrays of the first half and the second half, right? And we would like to fold them together into a single sorted array. Now, you can't just say, Okay, we'll put a one first and then a two. Because if you look at it, the items in a one and a two actually should be interleaved. But the question then is can we come up with and this is where it gets its name, a merge algorithm, that will merge two independently sorted arrays into a single sorted array. And can we do that in linear time, that is linear in the total length of the final merged array. And if we could do that, we'll factor that code out into a separate method that we'll just call merge. That will take a one and a two will merge them into a single sorted array, and then that's what we return. And that completes the pseudocode for this, but we haven't yet done the merge. So I'm not going to code merge for you. I'm going to kind of leave that as an exercise. But I do want to just sort of draw out how we'll do it. So sorry, there was a question that we say, Ah, you can be an on spaces of the array started in zero? Sure. So it just so happened, that the length of the array that I gave you here was an even length. But that's that, you know, that's just because that's how I drew it. If it were an odd number of items, you'd sort of cheat to one side, and you'd have slightly different length arrays. Does that answer your question? Or get at what you were Okay, cool. What's the reason for splitting into two

halves? Well, that's what gets us the log of n costs. So if we can, I'm going to prove that at the very end. But intuitively, how many times would we have to split an array of length and in half, before we reach the base case? This is very similar to the discussion we had about binary search. How many times do we have to split it in half, before we reach the base case of something that's length is less than or equal to one, right, the log base two of N . So we're going to do log base two of N splits throughout the recursion. And for each of those log base, two events splits, we're going to do an order and merge. And so that intuitively, explains why the total cost would be $n \log n$.

33:20

Does that work for you? Cool. All right, let's talk a little bit about the merge. So we want to do this in linear time. So let's say we have the following two arrays that we wish to merge. And so a one we said would be 138. That was a somewhat overly stylized date, sorry. And a two would be two, seven, and nine. And I want to dump that down into a single sorted array in linear time where it's linear in the total length of the tourists. Where would I start any thoughts on how I might do this?

34:07

DS Lecture 18 Page 8 of 23 Transcribed by <https://otter.ai>

While I wait for somebody to get the courage to say something, I'm going to cue something else up. Anybody? You can go one index at a time. Ah, hold on one second. I don't want this to start playing. Yeah. Sorry. Um, yeah, AI knows what's coming. Alright. So create a new array that the size of both and compare the two first items, that's what I'm putting together from this. I like that. So imagine, oh, we were gonna get there. I never go without that. That's my favorite thing. Um,

34:56

we'll have two

34:59

indices. Which will say like i and j , i and j will both start at zero. So the anonymous comment, sorry, the private comment. Yes, but let's not say that publicly. And I don't think it's Swedish. Anyway, moving on. i and j both start at zero. All right, so you're comparing initially, a one zero and a two at zero. And those two locations are the two smallest values, right? The smallest one in each. And so that means that the smaller of those two must be the first item in the output, right? So we compare them, pick the one that's smaller and write it down. In this case, it's the one pointed to by H_i .

35:52

All right.

35:57

And then we increment i by one. And so i and j , still point two respectively, the smallest values in each array that remain, we compare them, the smaller of the two must be next, which in this case is J . And then we increment J . Compare, pick the smaller, and then increment. Compare, pick the smaller, and then increment. Compare, pick the smaller, and then increment. But now we run into a slight problem, in that we're adding items in a one. So once we run out of items in one array, or the other, what do we do

36:52

just dump the rest of the other one. Because they're all they all have to be larger, which in this case is just one. That's it, we're done. We have a fully sorted array. And if you want to

DS Lecture 18 Page 9 of 23 Transcribed by <https://otter.ai>

talk about costs, for every item that I wrote down here, I did a single comparison, a single assignment and a single increment. And so that's a fixed number of operations per item I wrote down. So this merge by itself is big O of n , which, when dropped in here, give me the overall and login costs that we will prove a little bit later on in this lecture. Now, um, you might be having a little bit of trouble, sort of seeing how this works. Let me say something

quickly about that. And then we'll, we'll look at this visualization. Um, what imagine you start with an array of length two, all right. And so you split that in half, and then you recursively call merge sort on each half. Now, those will hit the base case, and just return those two values back to you as separate arrays. But the merge will take care of the sorting of those two items, right? The merge is good enough to sort two items. At that point, you have a sorted array of length two. So we are we were sure it works on a sorted array of links to so I'm almost doing this, like you might do an induction, right. And then now imagine a scenario where you have an array of links for Well, you're splitting that in half into two sub arrays of length two, you know, those can be sorted because we just showed that. And then once you get two sorted sub arrays of length two back, the merge will fold them together, and it all expands out from there. Okay. All right. So with that in mind,

38:55

let me switch the share. So we're going to watch ourselves a little video here.

39:13

And let's begin. So this is going to be really useful in showing how we use a temporary array. And here is one way it'll get there. Alright, that's our array that we're sorting. And you'll notice there's the B array at the bottom where

39:40

we do our first split. And then we split again. Now we're going to split again and we get down to that case.

39:55

Now we do the merge and you'll notice that what used to take more rights than it comes to rest. And now that part, how we do with and half, this one doesn't have to do anything because it hits the base case, perhaps you have an odd number, these two split in half and

DS Lecture 18 Page 10 of 23 Transcribed by <https://otter.ai>

you compare the first item of each and write the smaller down, then we write the larger one. Now this happened this half are sorted. So now we have to merge this and this is the smallest of each pair. Write that down, you'll notice Maurice on beaming right, the next one and then write down what's left. So now we've got a sorted array of length two and a sorted array of length three. And we will take this point that I feel like we need to speed things up a bit. So we do the merge. Write down the rest. Now we finished the entire first all this point, we're going to kind of repeating the same thing we did before. All right.

41:44

I could go faster than two times.

41:51

Alright, now the original are gone. We're at the final merge.

42:09

Hold on.

42:15

Now we're gonna have to move back a little bit. So let's see. Ah, did three check with one? That's a good question. Let's go back. All right.

42:37

No, I didn't go back far enough.

42:42

So it's actually kind of fun to watch it going backwards. Okay. They're, they're happy now. Now we're splitting again. One in five check. The one writes that five writes the rest of them. They're done. This works. Merging against each other. Now 789. All right. So this is where you're saying there's a problem, right? Let's see if they messed it up. All right, I'm going to slow this back down a little bit. So one in three checks. Isn't that what they were supposed to do? We had one in five is one array, three, seven, and nine is the other array.

And the one and three are the first items of each. Why do they do that? Oh, so? Um, that's a good question. So notice that we split the portion of the array that goes from five to nine and a half, right? And then we recursively merge sort of these two halves, right now what we're trying to do is we're trying to merge the half that corresponds to a five, a six with the half that corresponds to three, seven and nine. So why are we so far good. And so in that merge, that I NJ, that I had when I was drawing the merge out the eye points to the guy with the one on him, and the J points to the guy with the three on him. And so the values that i and j have to compare against each other, and the smaller one gets written into the output, which is the biro.

44:32

So that's, that's embarrassing. So now the five and the seven Jews and Arabic

44:44

And now the rest of this, write themselves down because they were they're part of the same array. Yes, that's exactly right. So when you split these things in half, it then splits in half again. splits in half again, and then all of those sub arrays have to merge back up towards the top, which means many little merges will happen. Does that make sense? Awesome. Kareem, are we are we good as well? Okay, so um, yeah, use the corresponding locations and be that way, you don't need a separate a one a two, you just use the portion of B that corresponds to the portions you're trying to merge. So it just helps with having to recreate arrays many, many times, you just have a single, temporary that passes through all of the recursion. And so that means the extra space cost is only order and the way I wrote it, it's actually much worse. So this is the much better way to do it. Are we good? Christy? Somebody mentioned that Adam cannon uses this what is what did he use it for? I've never seen him use this in his class.

46:05

Okay.

46:07

All right. Anyway, let's not worry about it. So in case anybody's curious, and this always makes me roll my eyes a little bit whenever I have to say this out loud. I'm the channel on YouTube for this is called algorithmics. Yeah, it's a little painful. They seem to do different

types of European folk dancing for the various sorts. So you can definitely find ones for insertion sort and selection sort, which is what some of you may have seen. They also do one for quicksort. But unfortunately, I can't use that one. Because there are a lot of different ways of doing quicksort. And the way they've chosen to dance, it doesn't line up with my way of doing quicksort. And I do not think my ta is would appreciate me trying to choreograph my version of quicksort. To that just anyway, that's probably a bad idea. Um, and by the way, the person who thought it was Swedish, it's actually Romanian. I Oh, okay. All right. Any questions on how Merge Sort actually works? We're good with this. Alright, cool. So let me switch back to the camera. Alright. So what I would like to do now is I would like to do a little bit of a proof for how we actually get that in login court costs a little more formally. And this type of proof is a type of proof one does for solving recurrence relations, recursion relationships. And it's an application of an algorithm for doing this that comes from something known as the master theorem, which we'll learn more about possibly a little bit in discrete math. But I think usually you learn that in CS theory. Alright, so I save this because it's not going to be immediately apparent, what motivated me what motivates me to take certain steps in this proof, but you'll see it all come together very nicely at the end. So the first thing we're going to do is we're going to use the following notation, we're going to say that t of n is the cost, the time cost of running merge sort, on an input array of size n , the actual time, whatever it is, okay. And now we're going to build up an expression that calculates what that t of n is based on the

code we wrote. And so those Merge Sort calls that I made those recursive Merge Sort calls, that's just calling merge sort on an input array of exactly the size N over two. So that means each time I call Merge Sort recursively, in my code for this, I get a cost of T of N over two. So far, so good. How many times do I call merge sort? recursively? remain, remember, or remind me?

49:24

Not quite.

49:29

within a single run of the algorithm recursively? No, so not the total number of times, how many times do I call merge sort? On an array of size N over two? How about that? twice, right? In the original code, I had Merge Sort called twice. So the cost that that's going to contribute to my total cost is two times whatever the cost is to run. And over to is everybody okay with that, when the very last thing we did was we merged the results together, and the merge was linear. So that just costs n . So my recurrence relation says that t of n is going to be equal to two times T of N over two, plus n . And we got that

DS Lecture 18 Page 13 of 23 Transcribed by <https://otter.ai>

entirely just from looking at our pseudocode. Now one other thing that will come in handy is that we had a base case. So we know that T of one is just equal to one. Okay? Now, again, do not worry about why I'm choosing to make this maneuver. That's what's coming from the master theorem, we'll just see what the consequences of it are. So what I'm going to do now is I'm going to divide both sides of this equation by n . Okay, so I'm going to divide both sides of the equation by n . So on the left hand side, I'm going to get T of N divided by N . All right, and on the right hand side, I'm going to get two times T of N over two divided by n . But I can actually move that to down two times something is the same as dividing that thing by a half, right. So I could rewrite this as T of N over two divided by N over two, it's just convenient to bring that down, because it's going to give us some symmetry later that we'll be able to explain. And then the last bit here is plus n , which now is divided by n , so it's plus one. All right, so we have t of n over n is equal to T of N over two divided by N over two plus one. Now, to solve this, we need to know what T of N over two is, right? So I'll just sketch it over here, T of N over two would be equal to two times t of n over four, right? Because you're going to divide that by two again, when you make those recursive calls. And the merge instead of costing n will cost N over two. Agreed. So everybody good with that. That's the expression for T of N over two. Cool, now let's divide both sides, instead of by n , let's divide both sides by N over two. And so that means for my second equation, I'll get T of N over two divided by N over two is going to be equal to two times t of n over four divided by N over two. But I'm going to bring that two down again, for symmetry sake. And I'm going to end up with T of N over four divided by n over four. And then that last term is N over two. Because it's T of N over two now, instead of instead of t of n , so I'm always dividing by what's inside the parentheses on the left hand side, why I chose to do that will become apparent once I do it a few more times. Are we okay? Or, alright, yeah. So when we divide by n over two, that last term also becomes one, because N over two divided by N over two is just one. And we can keep doing this. So you'll see I now have this nice symmetry here. So now if I wanted to work out what t of n over four is, I actually want to divide both sides of that by n over four. So now I'll have T of N over four divided by n over four is going to be equal to t of n over eight, divided by n over eight, also plus one. So I can keep doing this.

54:18

Now, if we assume for the moment, just it makes this a little bit easier in the analysis. And you know, if, if this ended up not being true, it's just rounding up or rounding down in the end. But let's assume for the moment that n is a power of two. So if n is a power of two, eventually we'll keep dividing n by two until we get a scenario that looks like this. m t of eight over eight equals T of four over four plus one and then T of four over four e TF to

over two plus one. And then finally, we'll get down to T of one over one plus one. And we don't have to go any further than that because T of one was the base case. So we knew what that was already. Right. And so if we want to count how many of these equations we've written, it's the log base two event, right? There are log base two of N different equations here that I've written. And now comes the trick, we're going to just add them all up. Okay, we're going to add up everything on the left hand side and everything on the right hand side, just add it all up. And now the symmetry really comes into play, because we notice that we're adding T of N over two divided by N over two on the left, but we're also adding it on the right. So they cancel out when we add them up. And that cancellation goes through the whole thing, you got t of n over four over n over four here, and here, you've got t of n over eight divided by n over eight here, and the next term would have that on the left hand side. And so they'll keep canceling the previous term would be t of n over 16 over 16 equals T of eight over eight. So the T of eight over eight, that would be here, we cancel here, you can see that pattern continuing. And after all the cancellations are done. After we add all this stuff up, we're left with only three things really, we're left with the t of n over n . On the left hand side, we're left with the T of one over one on the right hand side. And we're left with one more thing, what is the other category of things, we're left with a whole bunch of ones. But we know exactly how many of them there are, there are log base two of N ones. So all these ones add up to the log base two of N . So I think I'm going to need to go to another piece of paper for this. But what this ends up adding up into his t of n over n , well is equal to T of one over one plus the log base two of N . Now further T of one over one. We know that's just one, right? Because T of one was one. And so now the only thing left for me to do, I was supposed to be solving for t of n . So I want to isolate t of n on the left hand side. So then I just multiply both sides by n . And that will do it. So t of n will equal to one times n is just n plus n times the log base two of N . And we've solved for t of n . Now this should look actually really familiar. Because this was the same exact expression we ended up with when we did heapsort. Right? The builds the cost order n and then we had $n \log n$ costs for the Delete mins. So it's equivalent. But now if we wrote this in big O , $n \log n$ term drops out because this is an asymptotic analysis, it gets drowned out by the larger order term. The base of the log doesn't matter because that's really just a constant factor. So that's just equal to big O of $n \log n$. And we've proven it.

59:02

Everybody okay with that. Now, I'm not going to ask you to sort of do this or this style of proof on an exam or a homework, but I think it's important for you to see it, because it is a really powerful approach to solving recurrence relations. Right. Any questions on merge sort? Okay. So the next thing I want to do continuing in the conversation about

DS Lecture 18 Page 15 of 23 Transcribed by <https://otter.ai>

59:35

about, hold on, let me

59:41

get this out of the way. Continuing in the conversation about divide and conquer based algorithms for solving for sorting, comes another one called quicksort. Now quicksort there are two things about quicksort that are really sort of surprising. The first is that the worst case cost of quicksort. And I feel the need to get this out of the way at the beginning. Because when I asked you for something that isn't worst case cost of $n \log n$, people immediately jump to quicksort. And then they get the question wrong. The worst case cost of quicksort is not $n \log n$, it is actually an order n squared algorithm in the worst case, which might make you wonder why are we bothering with it? If I just showed you over the last few lectures to perfectly reasonable order $n \log n$ algorithms? Well, the answer is the following. The average case, actually, is order $n \log n$. So on average, it's in $\log n$. But

occasionally, you will hit the worst case. But that worst case scenario is actually very uncommon. And there are some things we can do as we implement this, that will make it even less likely to hit the worst case of the worst case is very uncommon. On average, we get an $n \log n$ in costs. And so why do we often choose to take the risk, I'll be in a very low risk of getting a very expensive cost for quicksort over the others? And the answer is because quicksort. First of all, unlike the Merge Sort I just showed you can easily be done in place, no need for a temporary array and copying back and forth. And second of all, it has just generally a much lower constant factor, you can really tightly optimize the inner loop of this thing. And so it has a lower constant factor. So on average, quicksort tends to be the best choice. And in fact, when you invoke short algorithms that are built into a lot of things, oftentimes quicksort is the algorithm that's happening. Now there is a brief conversation to be had about when do we choose to use quicksort versus one of the ones that's guaranteed. And $\log n$ in the worst case, even though overall, it's going to be slower because of its larger constant factor. And the answer to that comes from, excuse me comes from how important is having a time bound really for your application. So let me give you an example. Somewhat contrived, but you'll get the point. Imagine you're you're writing code for a video game that simulates flying a plane. And for one of the operations in simulating flying this plane, you have to sort a big thing. And so your choice is going to probably be quicksort. Because so what if the framerate drops very briefly, because it's slowed down dramatically, because it got the really unlikely worst case scenario cares. Most of the time, it's going to be really snappy with quicksort. That one out of 100 times and slow things down. So what to gain. Now imagine you're writing the same code, but not for the simulator, but for the actual airliner, like the plane that people get on. In that case, if maybe the sorting algorithm goes too slow, maybe the auto pilot fails, and the

DS Lecture 18 Page 16 of 23 Transcribed by <https://otter.ai>

plane crashes and everybody dies. But the other 99 times, very fast, everything works beautifully. That goes from being perfectly acceptable to suddenly and I hope anyway, for you perfectly unacceptable. And that in those scenarios, where you know, it's mission critical life for something that has a lot of value is on the line, you don't want to risk, the unlikely event of a slowed down thing, because that could be catastrophic. And so instead, you take the penalty of having it being overall slower across all cases, but in all cases, at least fast enough that you don't die. Okay? And so those are the kinds of trade offs you have to make. Alright, so let's talk a little bit about about quicksort. And how quicksort works. So quicksort what you're doing is you're choosing one of the items as a so called pivot. Now the way we choose that pivot, it's only a subject for discussion on Monday. But the way we choose that pivot

1:05:00

has to be constant time. Otherwise it's going to throw everything off. So we choose a pivot. So let's write down, sort of a sample array.

1:05:17

Alright, so let's say, for whatever reason we choose five as our pivot.

1:05:28

The way quicksort works is it partitions the array around the pivot, it pushes all the items that are smaller than the pivot to the pivots left. And all the items that are larger than the pivot to the pivots, right? The items to its left are not themselves sorted, the only thing we're guaranteed is that they're all smaller than the pivot. And so we partition, Part partion, part, partition. And we might end up with something that looks a bit like this, you have five in the middle. And then you've got say, three, two, and one on the left, and seven, six, and eight on the right there on the correct sides, but they're not themselves sorted. And so then what do we do we call quicksort.

1:06:39

recursively on each of those two sides. And so if we assume that that quicksort works, then

once those return, you'll end up with a sorted array that we agree. So it's almost like it's flipped from the way we did Merge Sort with merge start, we recurse on the halves first, and then we did a linear merge in quicksort, our partition algorithm is going to have to be

DS Lecture 18 Page 17 of 23 Transcribed by <https://otter.ai>

order n . So we do a linear partition first, and then we recurse on the two halves. Does everybody understand that part? We're good. Now, in terms of the Big O cost, you would end up with exactly the same analysis, as we just did on merge sort, you know, T of n will be equal to n for the partition, plus two times T of N over two for the two recursive calls on quicksort. And you get that same analysis and the same $n \log n$ cost if and only if the two halves after the partition have the same length. Right? If you partition it evenly into two roughly the same half sizes, then it really is T of N over two plus T of N over two. But if you mess up, and you choose a partition that partitions it very badly, what do I mean by a bad partition? A bad partition might be if you chose the smallest item as your pivot, or the largest item is your pivot. So you end up splitting up the array into an array of length one with just the pivot and an array of length n minus one with the rest of the items. At that point, your recurrence relation is no longer two times T of N over two plus n . It's now T of one plus T of n minus one plus n , you are effectively doing selection sorts. So quicksort, if you happen to always choose the smallest or the largest item as your pivot becomes selection sort becomes terrible, and becomes order n and square. And it turns out, we have no way to guarantee that it will do that. And that's why the worst case is actually order n square. It turns out and you can read a bit more about in the textbook, I'm not going to dive into that in class, that on average, you can expect it'll roughly break it in half. So over time, you really are getting an $\log n$ cost. The question is, how do we pick the pivot? What would the perfect pivot to guarantee us an $\log n$ time? So the median item, right? If you found the median, and use that as your pivot, you're gone, you're done. The thing is finding the median is basically the same operation as sorting. So you can't just choose the median in constant time, and it really has to be constant time for the analysis to work. Now, if the array, we're sort of arbitrarily arranged, it was sorted in a random order. Any item is likely to be the median is any other item, right? So if I were giving you a truly randomly shuffled array, is there any reason why I can't just pick the first item is the pivot every time? Yeah, if it's truly random, you can just pick the first item every time, or the last item every time or the middle item every time it shouldn't matter. But it's life truly random. I don't know. That's the don't answer that, um, please don't answer that. Got into a slight reference to religious stuff in my lecture yesterday, and I desperately never like to have that happen again.

1:11:01

How about this?

1:11:05

Is the data that you're being given ever truly random? Right, I'm giving a like a bunch of

DS Lecture 18 Page 18 of 23 Transcribed by <https://otter.ai>

students in my class on a roster is that going to be random? is a pretty good, let's not go there, please. Well, if it's a if it's if it's a course roster, it's not if God wills, it's the registrar wills it. Right. And the registrar is probably going to sort the data before it gives it to me. Just call the register it man, his name's Barry and I just booked in like two hours ago. Anyway.

1:11:50

The point is, if I give you data that you're likely to encounter in the real world, there is a slightly better chance that it will be partially sorted just by the nature of how you know, humans set things up than it being completely random. So that means even though it's still pretty unlikely, it means even though it's still pretty unlikely that the first item would be the smallest item, it is more likely that it would be the first item in real life than if it were truly random. And so we have to be careful about always picking the first item, we're

always picking the last item, because just in real world circumstances, that's slightly more likely to be the worst possible outcome than it would have been if it were truly random. And so the approach that we take to avoid those real world pitfalls, but still choose our pivot in choose our pivot in how do I split this in constant time, what we do is something known as the median of three. So our pivot selection strategy

1:13:16

is the median of three, which is what we do is we look at the first item of the array, we look at the last item of the array, and we look at the middle item of the array like the physical midpoints, not the median. And we just take the median of those three items. And it's always those three items, whether the array has three items in it, or the array has 3 billion items in it. So that median of three operation is constant with respect to the size of the array. If the array is already sorted, that's going to give us the perfect pivot. Right? If the array is actually random, it's not going to matter so much. But that avoids those cases where you get a pre sorted array, and it doesn't really add any cost to us or any appreciable cost to our overall algorithm. So let's go back to the original array that we were looking at, which was 537216, and eight. And the three items that we would look at in the median of three would be the five, the two and the eight. And calculating the median of that were which one would we pick? In fact, the five would be the one we'd bet. Alright, so that's how we choose our pivot by taking the median of three. What we need to do now is we need to actually do our linear partition of the array. So the first thing we do is we push the pivot out of the way, we don't want it in the part of the array, we're partitioning, we'll swap it back into place when we're done. Okay. And so what we'll do is we'll move it all the way to the right, just to get it out of the way. And so we'll swap, what was it the

DS Lecture 18 Page 19 of 23 Transcribed by <https://otter.ai>

right, so now what the array will look like after we push the pivot out of the way, the eight will end up here, because we swapped it, the three, the seven, the two, the one, the six, and then the pivot is over here. And so now this part of the array is the part where partitioning, and what we're going to do for our partition, and I'm going to do this quickly. Now, just to show you a quick example. And we'll do it a little more slowly at the beginning of next lecture, is we start at the end points, and work our way towards each other. The one that starts at the beginning, is supposed to be pointing to things that are smaller than the pivot, right, because things that are smaller than the pivot are supposed to be to its left. So you keep moving it in until it's pointing to something that's on the wrong side of the pivot, which in this case, would be something larger than the pivot. And so we don't actually move it in because it's starting out on something that's on the wrong side of the pivot. And we do the mirror operation on Jay Jay is starting from the right hand side. So for something to be on the correct side of the pivot, when we're looking at it from the perspective of J, it should be something that is larger than the pivot. So we're going to keep moving j in until we get to something that is not on the right side of the pivot something that is smaller than the pivot. So j moves over by one, and now it's on the one. And the one is on the incorrect side of the pivot. So when i and j have moved in sufficiently, so they're pointing to two things that are on the wrong side of the pivot, and i and j themselves have not yet crossed, what operation would make this right. Make this correct.

1:17:12

You just swap them.

1:17:14

So the six, not the six, the one comes here. And the eight comes here. And now i and j point to things that are on the correct side of the pivot. And then you start marching i and j and again. So now I will move over here, that's still on the correct side of the pivot, because three is smaller than the pivot. And then I will eventually land here at the seven, which is on the wrong side of the pivot. J will do the same thing j moves in the two is on

the wrong side of the pivot with respect to J because now we're looking for things that are larger than the pivot. i and j have still not crossed, they're pointing to things that are on the wrong side. So we swap again. And now i and j continue their march. So I lands here and j lands here. This point we're not going to swap them because they've crossed once they've crossed we know we've correctly partitioned. And the only thing left for us to do is to put the pivot back into place. And you must swap the pivot with A_i. Because A_i is now pointing to something that's larger than the pivot, and the pivot is on the right hand side.

DS Lecture 18 Page 20 of 23 Transcribed by <https://otter.ai>

So I swapped the pivot with I. And we're done. Let me rewrite this, we've got 132586 and seven. And now my array is correctly partitioned around the five, and then I can recursively quicksort the left recursively quicksort the right and that will sort the whole array. Alright, we are out of time. Keep your homework is due on Friday and then again on Tuesday. And we will continue this and then we were going to begin graphs probably late on Monday. So you should start reading chapter nine. So someone says by implementing these types of sorts on different data structures we enhanced diminish their run times. If a particular algorithm holds on one second, if a particular algorithm is a uses sorting, then the choice of sorting algorithm will impact its cost. So if you're using sorting as a building block for one of your other items, rhythms then it would impact its cost. And then somebody said, Can you repeat what Jay is. So i and j are indices that start at the end points, I starts at the beginning, J starts at the end, and they work towards each other swapping to get things on the correct side until I n j cross. That's it. J is just the thing that starts at the end of the array. Does that make sense?

1:20:29

So Kristin, the issue here is that on average, it's also $n \log n$. But remember, when we talk about big O, we are dropping constant factors. So it's $50 n \log n$ or a million $n \log n$, we'd still call it $n \log n$ in the issue is that quicksort has a much smaller constant factor in front of that $n \log n$ than merge sort. So when you do get the average case, in quicksort, it actually ends up being much faster than Merge Sort just not enough to change the big O cost. That makes sense. Cool. Andrea, I thought we posted that gal didn't Harrison post that. Like after our meeting?

1:21:22

He said he would I can check in with him, you double

1:21:24

check on that. It should be there. Um, if it's not, it will be very soon. Okay. Tiffany, what's up?

1:21:36

I was just a bit confused about when you're doing the quicksort. So like, we the pivot is actually at the end of the list. But we're

DS Lecture 18 Page 21 of 23 Transcribed by <https://otter.ai>

1:21:44

you move it to the end of the list temporarily while you're doing this swapping operation. It just gets in the way if you haven't in the middle while you're doing this,

1:21:53

okay. But you so you're kind of like hypothetically, imagine, like comparing if it were in the middle still,

1:21:59

right? You're comparing the values of i and j against that value, which you've moved off to the side, but you know exactly where it is. And once i and j crosses, then you can bring it back. Okay, that makes sense. Thank you. Yeah. Somebody asked, Is this all in place? And the answer is yes. You're just doing this in one array. The only thing you need, are those two extra variables i and j. Y'all, did he also post the problems?

1:22:30

I don't think there's a file of it. But in the midterm solutions, the question is, like the screenshot of the question, and then the solution.

1:22:39

Perfect. So that should do the job. Yeah. Could you send an announcement to that effect? While we're thinking about this? Yeah.

1:22:46

Thanks. All right.

1:22:50

Any other questions?

DS Lecture 18 Page 22 of 23 Transcribed by <https://otter.ai>

1:22:54

Yeah, I have a question. Yeah. is order of $n \log n$ better than old n ? order $n \log n$ is slower than order and slower in terms of growth, or

1:23:10

Yeah, growth. So order $n \log n$ algorithms take more time. That means the function that plots the time as n grows, actually, the function itself increases at a faster rate. But the larger the function is, the slower the algorithm is running, because that just means it takes more time.

1:23:36

So if we had an $N \log N$ and N , we would prefer the the linear one.

1:23:45

Yes, always. Okay. Okay. Thank you. You're not going to find the linear one in the general purpose sorting case. But if you did that, you would always want to use that instead. Okay, thank you. So $n \log n$ sort of falls in between n and n squared.

1:24:08

Okay. All right, folks, unless there's anything else I think we're gonna call it a day

1:24:14

and take care of us.

DS Lecture 18 Page 23 of 23 Transcribed by <https://otter.ai>

DS Lecture 19

Wed, 8/11 1:55PM 1:18:41

SUMMARY KEYWORDS

vertices, graph, quicksort, directed graph, array, edge, good, partition, median, pivot, unweighted, sorting, loop, called, questions, insertion sort, directed, algorithm, left, people

01:00

There's so many songs about rainbows and what's on the rainbow nation but only illusion and rainbows have nothing to

01:28

be final there boy moraine dreamer who said that every wish would be heard and answered when wish on somebody so far so amazing that keeps our star gazing and what we might we'll find the rain but creamer me we know that it's probably by bird balling boys might say I did too many times to ignore it it's something that I'm supposed to be some day we'll find the rain Samer me

04:03

Alright folks let's get started so few things you should you should have your homework

three's back regrade requests as always go through the TA is through grade scope for the written and directly to the TA so graded as per the message sent out for the programming. So, you know if you were to come to me today and ask about a regrade my answer which several people did my answer is you got to go through the tiers first. So make sure you do that. Um homework four Written, I guess was due last Friday. And we'll have solutions to that out shortly. Homework Well, after homework for programming is out is done homework for programming is due tomorrow, correct. All right. Homework five will go out tonight. Single programming problems, slightly smaller number of written problems we do the last day of classes. So it's a significantly shorter assignment. It's the way that the same as the others. I'm the plan. I'm going to juggle the order in which I do things a

DS Lecture 19 Page 1 of 20 Transcribed by <https://otter.ai>

little bit. The plan is today, I'm going to finish up quicksort. And I'm going to introduce graphs. Normally, after I introduce graphs, in a few days, I'll talk to my TA this evening, I thought they were already out, honestly. Hey, could you check and see what the deal was with homework three solutions. Um, by the way, the biggest issue with homework three, when people did their proofs, you really need to be clear. Show your base case, state your inductive hypothesis explicitly. If you didn't do that, right in the middle, you were losing points. And that's not really up for discussion, and then prove it from there. That seemed to be by far the biggest issue was that the structure was all wrong. So anyway, so we're gonna do quicksort. Today, we're gonna start graphs, and then I will shift the order around a little bit. And we'll do single source shortest paths, and Dijkstra is algorithm on Wednesday. I know a lot of you've probably already seen Dijkstra's algorithm, but I do it a slightly different way. And that will get you ready to go for the programming problems. So you have the weekend to work on that because it starts there's, and then we'll wrap up the remaining the remaining graph topics the following week. One possibility is that I may post a supplemental lecture for one of the graph topics just so that we get everything in. I'll see about that, after Thursday's lecture, and see where we are, does anybody have any questions before I move forward?

07:26

Okay, let's get started. Um, so quicksort what we did last time as we sort of discussed how quicksort works, right? And we discussed the linear partition algorithm. Was everybody okay with that? Any questions on that? Okay. You want to know what's funny? I'm looking at the eight people. 910 Okay, fine. 910 people, including myself who have their camera on right now. Every single one of you, myself included, just looks dead. And I don't know why. But it just seems to me universal. Alright, more people are turning their cameras on to display their deadness.

08:17

Okay. Okay.

08:25

Sorry, just, I was looking at and it just struck me everybody was just like, okay, so is it really? Yeah. Okay. You have to take finals in two weeks, I have to write two finals for two weeks from now. I'm okay. So the textbook actually uses a little bit of a, how should I put this a little bit of a hack to kind of improve not a big O cost but just do a slight improvement on how the quicksort algorithm runs. So as I do my next quicksort pass, I'm

DS Lecture 19 Page 2 of 20 Transcribed by <https://otter.ai>

going to show you that I'm going to show you the tech the way the code does it. And so we'll do that out on paper, and then we'll switch and we'll actually go through the implementation of it. So let's start with an arbitrary array.

09:38

Okay, and what we know is that to pick our pivot point, we choose the median of three. Now, I managed to sort of set this up in a way that the median of three is kind of the worst that could possibly be right because even though I'm choosing three elements, the

median of those three elements is like the second smallest item anyway. But whatever my point is just, you can have a problem like that, where even with a median of three, you get something close to the worst case. Now, when I calculate the median of three, and we choose the two, in the version that I wrote out initially, which is also the version of the textbook does Initially, I move this over all the way to the right to get it out of the way, and then I do the partition on this part. Is everybody good with that? Yes. All right. Cool. Now, that's, that does a little more work than it really needs to. And the question is why? Well, if I'm doing the median of three, I've effectively sorted those three items, right? I know that the one is smaller than what I picked is the median, I know that the three is larger. So what the algorithm actually does is it rearranges those items that we do the median of three on such that the one that's larger, and the one that's smaller, are not so much pre sorted, but are pre partitioned. So as I choose my median of three, I can rearrange this array to look like this. And now this is the only part that I actually need to partition correct. These guys are already on the correct size of the pivot. Do we agree? Yes. And so we proceed on this interior portion of the array. And we'll leave those guys where they are, because we don't need to consider them. Now the first thing we do is we move the pivot out of the way. And usually that means move it all the way to the right. But in this case, I'm not moving it all the way to the right, because I'm only considering this interior portion. So I actually move the pivot to the right minus one. Because the one and the three aren't part of this party anymore. Okay. And so now this situation looks like this 187452. And then the three, this is the piece that I'm partitioning. That's my pivot. I starts here. J starts there. Good. And so the cost of this is the same, really, but it's slightly better in terms of constant factor, because I've already got those things on, on the outsides taken care of. So I move on until it's on the wrong side of the pivot, which this one is, and oh, oh, because remember, you're supposed to push the pivot all the way to the end of the portion, you're partitioning. And you don't want to slide items over because that's expensive. So you just swap it with the last item of the section that you're partitioning. Does that make sense? Cool. All right. So the it doesn't move. And the J actually is going to keep moving until it is on the wrong side of the pivot means something smaller than the two, which means it's going to move all the way over here. Because Well, you know, that's there was nothing to swap. And so now you know, you restore the pivot, by swapping with whatever I points to. And now the array is correctly partitioned, even though it's a little bit prominent.

14:03

And so then we recursively quicksort, the left side, and we recursively quicksort the right side. Now, one thing I will note about the recursive quick sorting, and I may have mentioned this last time, but if we're using the median of three, if I have an A sub array of length three, just the calculation of the median of three itself, sorts those three items, right, so you're not even really quick sorting them. And same thing goes for two or one. So the way we actually encode this is our base case isn't, when we get down to a sub array of size one we stop, it's actually when we get down to a sub array of size three, we stop, because it's size three, there's no need to quicksort anymore. Now, what do you do if you get down to a sub array that so small that you no longer need to recursively quicksort? Well, one thing I mentioned last time, That insertion sort is sort of the sort of choice if you have a very small array. So if the array is below a certain threshold length, and you can adjust that, but three is sort of the minimum, um, then you're going to insertion sort it instead of recursively. Quick sorting sound good. Um,

15:28

so the person who privately messaged me, can you actually change your name to something useful other than a mess of my own name? I don't know who I'm talking to. But, uh, just to check my understanding unless the first last and median elements are

duplicates of each other. Working off the medium three means we'll avoid the worst case of n^2 , right? Yeah, for the most part. Part. That's true. But as you can see, I still picked something that's pretty close to the worst case. So if you assume that it's mostly random, then it doesn't really matter. But it avoids the absolute worst case. Okay. direct message person, we're good. All right. And then next person, why did you choose to slow down for a sec? Why did you choose three is the meeting in the beginning and not for since four is the median of the list, you can't choose the median of the list because finding the median of the list costs order and login time. That's the same as sorting, you take the first element, the last element and the middle element of the array itself, and take the median of those three. That's the best we can do. And it actually isn't three that I chose is the median. It's two. Does that make sense? Cool. All right. And then the follow up was just something similar to that. Then how do you know when to swap the pivot and I again, so yeah, that happens when i n j cross. So when i n j cross, that means we're done with this partition, and we restore the pivot by swapping specifically with i . Okay. Any other questions? All right. So what we'll do now is we will switch and take a look at the code for DS Lecture 19 Page 4 of 20 Transcribed by <https://otter.ai>

this. Alright, so let me switch the share. Alright, so what we're looking at is the source dot java. Alright. And the source dot java is a class that just has static methods in it, this is not something we're ever going to instantiate. And because we're never going to instantiate it, this is from the waste code. All of the methods end up being method generics, so you don't need to instantiate the class to get the generic type. So this might ring some bells from back on homework one. So all of these methods to be sortable. The Met, the arrays that you're passing in have to be comparable. So we make sure that the any type, the type of what you pass in, extends comparable meaning and implements comparable on itself for one of its superclasses. And here is the basic loops for Insertion Sort. Okay, we don't need to go through that you should have seen that before. But we've done this on paper, so you know what this is doing effectively. Now going through this, we didn't really do shell sort, so you don't have to worry about it. Here is you see a percolate down. And you can actually see heaps are actually very compact if you have a purple eight down. So this is using a max heap percolate down, and does it from there. And then we have an implementation of merge sort. And you can see this one uses a temporary array, just like we showed in our lecture on merge sort. And then finally, we get to quicksort, which is where we are now. So let's take a look at quicksort. First of all, because quicksort is a recursive algorithm. We don't want to have to have sort of copies of the array moved around, we want to do everything in place. So the actual recursive quicksort will take in as additional parameters, the indices of the left and right sides of the portion of the array that we're trying to quicksort Does that make sense? And so the primary quicksort, the one that just takes in the array that we're trying to quicksort in turn invokes a private recursive quicksort that takes both the array that you're sorting and the portion of the array that you're currently trying to sort.

20:08

Now we have this private static Final Cut off here, the purpose of the cut off is exactly what we were talking about a few minutes ago, when we said, when the size of the sub arrays gets down to something small, like a length of three, then we're going to Insertion Sort instead of recursively quicksort. And so you can adjust this, but whenever the length reaches this cut off, that's when it comes time to call Insertion Sort, instead of continuing our recursion. Now we have some helper methods here. Since quicksort, involves swapping items, we'll have this nice little helper method that takes an array A , and then takes two indices and simply swaps the values have those two indices. So it allows us to swap two locations in an array without actually having to have three lines of code every time. So that's useful. And then we separately have the median of three. Now the meaning of three method is going to do several things, the first thing it's going to do is going to look at the

first item, the left item, the right item and the center item, and figure out which one's the meeting. But in the process of that, it's going to sort all three of them. So after these initial

DS Lecture 19 Page 5 of 20 Transcribed by <https://otter.ai>

if statements are done, and I'll let you work through them on your own, but just briefly, we calculate the centroid of the thing, the center point, and then those three comparisons, along with the corresponding swap references, shift them around, so that the middle valued one of those three is at location center. The smallest is that location left, and the largest is that location, right? So it pre partitions the biggest and the smallest, and sticks the median of the three in the middle element. That's the first thing it does. The second. Oh, thank you. The second thing it does, is while you recall that we usually want to shift the, we want to shift the pivot over to the right. And so we do that by swapping references with the center to the right minus one. And the reason why it's the right minus one rather than the right is because the right minus one is just inside of those two pre partition locations, locations, left and right, we're already partition they're not part of the party anymore. So right minus one is actually the right most location. So that's where we shoved the pivot. And then as a courtesy, we return the value of the pivot. So in the main recursive quicksort, when I call median of three, it does the median of three pre partitions, the largest and the smallest, shows the midpoint over to where it belongs over at the right minus one, and returns the value of that pivot. So it does all three of those things. Anybody have any questions on that point? Before I move forward?

23:27

We're good. All right.

23:35

Now comes the actual recursive quicksort. It's a private method, because we're invoking it from the public one. And it takes in the endpoints left and right in the array. And the first thing it does is it checks the base case. Now in this case, the base case, it adds the cut off that length of three to the left, and checks to see if that's less than or equal to the right. So if that's true, that means the arrays length is larger than three, and it goes into the main quicksort routine. But if that's false, it means the length that's the space between left and right is so small, that we just Insertion Sort. And so in that else case, which is really our base case, we recursively invoke the insertion sort method on a and the portion that we're trying to insertion sort of left and right, everybody good with the base case. Alright. So if we're not at the base case, we start by invoking that median of three method, it pre partitions things, shoves the pivot into location, right minus one, and returns the value of the pivot which we store in the pivot variable. And then we begin the partitioning. So remember, we partition first, then recursively quicksort. So here we go. I, what you'll notice is that we're actually sorting Already iMj one out from the first location that we actually want to consider. So location left is not part of the thing, we're partitioning. So we started

DS Lecture 19 Page 6 of 20 Transcribed by <https://otter.ai>

just outside, on location left. And we started just outside of the right, which is actually not right, it's right minus one. And so the moment you increment i by one, and the moment you decrement, J by one, they will land immediately on the first and last elements of the portion being partitioned. And if you look at the loops below, you'll notice that the increment decrement happens before the comparisons. So that's why we're actually starting things slightly outside. Now, this for loop is, has anybody ever seen this format before? With the two semicolons. Alright. Um, so if you leave all those fields blank in a for loop, it's just an infinite loop. It's treated as the same as saying wild true. Yeah. And so you'll remember it From now on, I will tell you one of my colleagues, a fellow by the name of John Kendor, who absolutely hates infinite loops, and absolutely hates break statements and continuous statements, because he feels like they're basically just go twos. And he is one of those people who thinks go twos are bad. Personally, I think sometimes you just kind of need him. And that's the way it's gonna be. But he doesn't like him. So he calls this

kind of loop, a vampire loop. Because the semi colons look like fangs. So now you'll never forget it. Alright, so. So we have our vampire loop. And now that's going to just keep looping around and looping around until we break out of it manually. So essentially, that's going to just keep advancing i and j towards each other swapping as needed. Until i and j cross when i and j cross, that's when it manually breaks out. Okay, so if i and j haven't crossed yet, remember, the first thing we have to do is advanced i until it's on the wrong side of the pivot. And the wrong side of the pivot means that it's pointing to something that's larger than the pivot. So we take a look at plus plus i write so that increments i first, so we never look at the right side at the leftmost element, which weren't supposed to, it immediately goes one after and then compares a at that newly incremented location to the pivot. And if that comparison results in zero, starting something that's less than zero, it means that a at that location is smaller than the pivot. And so we keep incrementing it so we just have an empty block. And it loops around and does it again, this is actually kind of a very compact way of doing this, because all the work actually happens in the condition of the while loop. So when this finally stops, it means a it whatever i had stopped on is greater than or equal to the pivot. It's okay if it's equal to the pivot, it's sort of, it doesn't matter if you have multiple items that have the same value as the pivot, they can get shoved to either side, it doesn't matter. As soon as the recursive sorts happen, they'll get pushed into the right spot.

28:37

So now i points the right thing. And then we do the same thing with j , we decrement j , first we take it as step in, and then we compare it to the pivot. And as long as we're larger than the pivot, we keep decrement thing until we're either equal to or less than the pivot, meaning we're on the wrong side. At that point, we stopped. Now i and j after both, while loops finish are on the wrong side of the pivot. If i and j have not crossed, that means we

DS Lecture 19 Page 7 of 20 Transcribed by <https://otter.ai>

swap the values at i and j using our helper function. If they have crossed or have landed on the same thing, it means we're done. And we break. At that point, we restore the pivot by moving the pivot which existed at right minus one into the location of $A[i]$ and shoving the value of $A[i]$ over to the right. And then we recursively quicksort, the left side and the right side of the pivot. And that's the entire method. Any questions on quicksort?

29:45

Good. All right.

29:56

So the elements that go from So somebody asks, Why does it say sort larger and sort of smaller in the comments. And so this one is quick sorting from A to i minus one, sorry, from left to i minus one, i is the pivot. So from left to i minus one is the portion of the array that contains items that are smaller than the pivot, not sorted, just all smaller than the pivot. And all the items that are to the right of the pivot, which is i plus one, two, right or larger, so, it's just describing what's going on. But other questions? Okay. So that is the last of what I wanted to talk about in terms of general purpose sorting. You know, when I say general purpose sorting, I mean sorting algorithms that work on things that can be ordered, but we know no further information about the fat than then they can be ordered. All right. And general purpose sorting, as we had said before, is big $\omega n \log n$ can't do better than $n \log n$. A. Hold on one second. Somebody asked, Why does i started left plus one? Where do we see left plus one? I don't see that line of code anywhere. Help me out? Oh, you mean up here? Why it doesn't start at left plus one. Okay. And you could also ask me, why doesn't j start it at? Right minus two instead of right minus one. And the reason is, because notice that in these comparisons, the plus plus and the minus minus are before the variable. That means the increment happens before this comparison. So it starts at left, but then immediately gets incremented to left plus one before it ever compares anything. And the mirror case on the right, does that answer your question? Yes, no? Cool.

All right. Sorry. Were there any other questions on this? Okay, so it's worth mentioning an example of something that is not general purpose sorting something where we know more information about the situation than just that the values can be ordered. And as we said, the $N \log N$ lower bound applies to general purpose sorting. So maybe for these cases where we have more information, we could actually do better. So let me give you the following scenario. Imagine you have an array that could be arbitrarily long could have millions of elements in it. But I tell you that every location in that array has one of two values in zero or one. How would you output a sorted version of that array? in something that's faster than $n \log n$ in time? Anybody? Anybody? Whoo, we jumped to the clever

DS Lecture 19 Page 8 of 20 Transcribed by <https://otter.ai>

answer. Nice. So let's start with the correct but slightly less clever answer. No, no offense, it's a good answer. Count the number of zeros and then print those out first, and then print out the rest as ones. That's exactly right. And in fact, that's a very useful way to phrase it, because this type of sorting is actually called counting sort. But in this case, there's actually a nice little trick, which came out first, was that a way of counting all not the zeros, but all the ones is to simply add them all up. Because the sum of the array of zeros and ones is simply a count of the number of ones, then you take the length, subtract the number of ones, and now you automatically have the number of zeros. So then you print out the zeros, and then print out the ones. And we can do that in linear time. And so that's actually an example of something called counting sort. So yeah, that really does. Hmm, I guess you would need another variable, because as you pass through and add them up, you could also have a counter if you must, right?

34:47

No, yeah, you could. I'm saying if you don't know the length, you can calculate the length while you're adding them up. So that doesn't really change the big O cost. Okay. Alright, so that's, that's linear time. And that's pretty cool. And in fact, you can kind of make that work for any scenario where you have a finite number of possibilities for the values. If you know that your array only contains ones, twos, threes, fours and fives, you keep five counters. And then as you iterate through, you just uptick, the appropriate counter. And that's it. All right. And so that that actually works in a linear fashion. Now, there's a hybrid approach that makes use of a form of counting sort, to do general purpose sorting, but then it ends up still costing $n \log n$ or n^2 in the end. And that's called bucket sort. And what you can do with bucket sort is you can basically break the values up into ranges, like say, you know that the values that are in your array go between zero and 100. But maybe it's continuous values. So you can't say that it's discrete values. But then you could break it up into buckets, like zero through 9, through 19, and so on. And then you can linearly go through the numbers, and toss each number into the appropriate bucket. But you're still then going to have to go back through and sort each bucket. And that's where you lose the linear nature of it. Now, one of the reasons why I mentioned bucket sort is because sometimes it can be helpful to think about these sorting algorithms in terms of how you would physically sort a large stack of things. How many of you have had the pleasure of having to sort a stack of 350 papers before? Anybody? One person is raising their hand, you know, who's had to do that? Me repeatedly, every time there's an exam. You know, somebody said, that's like a whole tree. I want Google to see how many reams of paper you could get from a tree to calculate how many trees I've printed out over the last 15 years, the number was decidedly more than one tree, which was a little bit disturbing. Anyway, moving on. So the way I physically sort stuff, is I break the alphabet up into ranges. Because I sort things by last name, first and then first name. And then I just

DS Lecture 19 Page 9 of 20 Transcribed by <https://otter.ai>

thumb through all the papers, tossing the papers in the appropriate range of the alphabet. Then I pick up each bin, and I manually sort the bin. So is it still $n \log n$? Yes, or n^2 , really, because the way I manually sort each bin, because you really want to

think about how I physically sort things I, I look at I take the first sheet of paper, and I jam it under my neck. Then I look at the second sheet of paper, and I decide does it go before after the piece of paper under my neck and jam it in the appropriate spot. And I keep doing that taking a piece of paper and figuring out where under my neck it goes. What sorting algorithm is that? It's not next sort. What is it? It's just Insertion Sort. So when I physically sort stuff, I do a hybrid of buckets sorting Insertion Sort, because I tend to find that physically easiest for me to do. My ta is once thought they'd be clever. They said, You know what, let's try quick sorting this, you know, physical exams on a big conference table. And I said, Okay, I'll get a drink. And I'll be back in an hour. And yeah, humans attempting to physically quicksort things doesn't work too well. So some sorting algorithms work very well for a computer, and some work not so well for the humans. Anyway, does anybody have any last questions on sorting? Or can I move on to graphs? We're good. Alright, so let me switch the share. You should now be reading in chapter nine of the textbook and kind of skipped over chapter eight. I may do a little bit from chapter eight at some point. But I'll you don't really need to read it. I'll. I'll go through that part when we need it.

39:35

So what we're talking about now, are graphs. All right, and we have a whole bunch of definitions for graphs that we need to go through. And then we sort of need to talk about how these graphs are going to be represented. Before we can really get in to algorithms associated with graphs. And so what graphs are are a collection Have vertices and edges. So this might be a graph. And then the edges Connect pairs. A vertices. All right. And so in set notation, a graph is the set of vertices V . And the set of edges E . And each edge is a pair of vertices from V . So, for example, $v_1 v_3$ might be an example of an edge. So you have this set of vertices and you have this set of edges. Now, it's worth noting early on that when we talk about our costs for graph algorithms, we're not going to really talk about n anymore. And the reason why we're not really going to talk about n anymore, is because there are actually two values that matter in terms of our algorithms, it's could be in terms of the number of vertices, but it could also be in terms of the number of edges, and the number of vertices and the number of edges aren't necessarily directly related to them. Well, we'll talk more about the relationship between the number of vertices and edges a bit further on in our definitions. But what's worth noting is that this is set notation. So v doesn't stand for the number of vertices, V stands for the collection of vertices. So if you want the number of vertices, it's actually the magnitude of the set. So if you want the number of vertices, you would write it like this using the absolute value symbol, because

DS Lecture 19 Page 10 of 20 Transcribed by <https://otter.ai>

that's the magnitude of that set. And likewise, the number of edges

42:25

is magnitude $|E|$. So when I write my bego costs for these things, we're going to talk about them in terms of the magnitude of V and the magnitude of E . So far, so good. So this graph that we have here is what's known as an undirected graph.

42:51

And then an undirected graph, the order of the vertices in the edge doesn't matter. If you can go from v_1 to v_3 , you can go back from v_3 to v_1 . But there's also what's known as a directed graph, sometimes called a digraph. And an a directed graph. edges have direction associated with them. All right, so these little arrow heads give me a direction. When you have a directed graph, you can go from v_1 to v_2 in this case, but you can't go back from v_2 to v_1 . So when we write our edges, the order matters. In a directed graph, this would be the starting vertex, and that would be the destination vertex, meaning we can get from v_1 to v_3 that is v_3 is adjacent to v_1 . But we can't get from v_3 back to v_1 . All right, is everybody okay with a directed graph? Could we simulate an undirected graph with a directed graph? Yes or no? Yes, how? I, I like the the gesture. I've got several

gestures going. One person went like this one person went like this. They're okay. I'm alright. I'll let other people look at the video. But the idea is that if you want to simulate an undirected edge in a directed graph, you would have to have two edges, one going from v_1 to v_2 and then another edge going from v_2 back to v_1 . Does that make sense? All right. Now, in addition to directed versus undirected, you also can have weighted versus unweighted graphs. And in any weighted graph, edges have some kind of cost associated with them, depending on what the graph represents. In an unweighted graph, and ages and ages in edge, they all essentially cost the same. Does that make sense? All right. Now, another concept that we have with graphs could be paths within a graph. And a path within a graph, you have a starting vertex and an ending vertex. And the path is listed as the sequence of vertices that you visit as you travel across edges in your graph. All right. Now, the length of the path is a count of the number of edges you traverse. So this path from v_1 to v_3 has a length of what

46:10

know there are three vertices in there, but there are only two edges to traverse. Does that make sense? So the edge v_1 to v_2 and the edge v_2 to v_3 . So the path length counts edges traversed, whereas the path itself shows the vertices visited. Good. Alright.

DS Lecture 19 Page 11 of 20 Transcribed by <https://otter.ai>

somebody commented, it looks like the Markov chain. Alright, well, well, you could have a path that goes $v_1 v_2 v_3 v_1$. But that's a different path, because you're visiting a different set of vertices. Does that make sense? Definitely. Yeah, so that's absolutely fine. Um, so a simple path is a path where every vertex along that path is unique. With interestingly enough, possibly the exception of the last vertex, meaning you can close the cycle, so to speak, and come back to the beginning. But and it's still considered a simple path, but only if none of the interior vertices are repeats. A cycle is a path where the starting vertex of that path happens to be equal to one of the later vertices on the path. So I guess you could call a simple cycle, one that only repeats at the very end going back to the beginning. But a cycle is something basically which comes back on to it's the starting point. A graph itself is known as *sicherlich*. If it contains any path, that is a cycle. So a cyclic graph is a graph that contains a path or paths that are cycles. So interestingly enough, there is a fairly common variety of graph referred to as a DAG, which is a directed acyclic graph. That's a very common scenario directed acyclic graph. Alright, so we have our definitions here. There's some more but before we move on, I should ask if we use graphs before in this class, yes or no? Yes. What are we trees? Yes, trees are a great example of a graph, right? And the way we implemented most of our trees, they were directed graphs, because you could go from the parent to the child, but not back. Right. And the other thing about trees that was really important was that they were a tree, like, you didn't cycle back to something further up in the tree, they only went down, right? So a tree was an example of an acyclic graph, a directed acyclic graph, actually, that we use, and then our suggested lists as well. And you're right. a linked list would be a great example of a very, very heavily constrained graph, but absolutely would be a graph. In fact, a doubly linked list could essentially be thought of as trying to simulate an undirected graph, right? Because you can move in both directions, right? Although you would still need two links. Now, by the way, you will sometimes see scenarios like this, although you'll never see them again. In this class. You have a vertex that starts and ends. Sorry, a vertex that has an edge that starts and ends on the same node. That's actually not called a cycle so much as it's called a loop. And the reason why I draw attention to this is don't call cycles, loops. loops very specifically mean an edge that just returns back to the same vertex. Are you in? Oh, are you in fundamentals of computing? phonies, as the kids call it? Yeah, any of you?

50:40

Have you seen these kinds of loops? Yeah, state machines, right. So a state machine, could

be a way of describing a system, I like to think of an example of a state machine is like the way you build a vending machine, right? You know, the vending machine is in the state of waiting for money, and then you put money in, and then it loops back to the state of

DS Lecture 19 Page 12 of 20 Transcribed by <https://otter.ai>

waiting for more money, because no vending machine on the face of the planet will give you anything for a single quarter anymore. All right. So state machines are examples of things you can represent with this. Now. Since we're talking about things we can represent when people suggested Markov chains, in statistics, what are some things we could represent with graphs? trips? What do we mean by trips? That kind of like that, but be a little more specific? networks like computer networks, trips and stops along the way, acceleration and torque? Hmm. Okay. So the traveling salesman, I guess, is sort of an example of an algorithm you can apply to certain kinds of graphs. Oh, social networks. That's a good one. Alright. So let's not do the physics examples. So much I thought those don't quite work as well, in terms of the in terms of sort of the visualization. I want not saying that it's wrong. But let's stick to trips and social networks for a moment. Let's go with social networks to start with. All right. So a social network? What's a social network? Facebook, right? There is a relationship on Facebook, that car that is called friends, right? You can be friends with someone on Facebook, that establishes a pairwise connection between two profiles on Facebook. So you can think of the profiles as the vertices, right? And then the friend relationship is an edge between two vertices. So if you think of the Facebook, the Facebook, it hasn't been called the Facebook since I was like 25, or something like that. If you think of Facebook as a graph with vertices is profiles and friends, relationships is edges. Is it an undirected or directed graph?

53:28

Thunder acted? Right? If someone's a friend of me, then I'm a friend of them. is all on Facebook? Well, well, that's the thing. That's not even an edge yet, right? Because it doesn't create the friend relationship until both sides have clicked Okay, on right. So is there a directed graph, component to Facebook? Are you all too young to even use Facebook? Probably. Alright, so following, right, this is concept of following where you follow something, but they don't necessarily have to follow you back. That's a directed edge. And so now let's go to maybe slightly more modern social networks. I don't even know what counts is modern, because I think it's a decade old already. And people are migrating off of it already. And Facebook owns that anyway. But how about Instagram, right? Instead Instagram, you don't? It's just following, right? Either you follow someone or they follow you back, going back a little further Twitter, same deal, right. So those are directed graphs. Sound good. And so some of the graph algorithms that we can perform might be able to tell us information about relationships between groups of people in a social network, which can be kind of useful if you're Well, the term is networking, right? I guess LinkedIn maybe is a good example of that, like third degree, second degree connections, if you're trying to find somebody that you might know who works at a company that you want to work at, those algorithms can be useful for finding those kinds of relationships. They can also be useful for marketers or governments to mine

DS Lecture 19 Page 13 of 20 Transcribed by <https://otter.ai>

information out of them. So that's one thing. Another example, people said trips. So I would rephrase that as maps. Maps could be a good example of something that could be represented as a graph, specifically, if you want to get directions from a map. Yeah, like, like a street map, right? You want to get know how to get from Colombia to like, some sort of, why am I thinking from Colombia to a bagel shop?

55:51

Have all the random things? Haven't had a good bagel in a while? Anyway, um,

56:01

so how would you represent a street map as a graph? What would the vertices and what

would the edges be? anyway? streets and crossroads. So the edges would be roads, and crossroads, I guess I would phrase it as intersections, but intersections or crossroads, I guess, would be the vertices. Okay. So would that be a directed or undirected graph? You want it to say let's say we're doing a map in New York City or Manhattan? Would that be a directed or undirected undirected? Well, one way roads are directed depends on whether or not you're on a bicycle. Hmm. But sorry, that we let's not go down that route. Short answer, it really does seem like it should be directed right, or some sort of mix, because you've got one way roads, and you've got two way roads. So at that point, maybe it makes more sense to represent your maps is a directed graph. So you can have edges that only go in one direction for one way roads. And if you have two way roads, you'd have an edge that goes in one direction and an edge that goes in the other. So you can almost think of the road going in one direction is one separate road and the road going in another direction is another separate road. And the best way to visualize that this is dangerous. How many of you are first year's again? I asked that question. Because this is like, how many of you have ever actually been on campus and have any clue about the I'm talking about? Alright. Come fall. We hope you'll know what I'm talking about. But Broadway, the street that's right in, in front of Columbia, right? If you ever get a chance to visit Brian, I hope soon or Yeah. Well, what's Broadway, it's a street. All right. It's actually weirdly enough, it's the same street that I live on. Even though I live several cities north of the city, it's a long Street. Anyway, um, Broadway is a Boulevard by Columbia. That is you have northbound traffic on one side, then a big divider, and then southbound traffic on the other side. So even though it's one street, it's actually physically broken up into two separate streets. So that's a good way of thinking of sort of simulating a directed graph. Sorry, an undirected graph of the directed graph. And now what about time, though? There's a good question. What about weighted versus unweighted for your street graphs? Would they be weighted graphs or unweighted graphs? If you were trying to write

DS Lecture 19 Page 14 of 20 Transcribed by <https://otter.ai>

something that was able to give you directions from one intersection to another waited? How would you wait those edges? Oh, there are lots of choices. somebody suggested a speed limit. somebody suggested straight distance, somebody suggested tolls, somebody suggested traffic, you can weight your edges in lots of different ways. And they all might make sense. Does that make sense? All right. Somebody said wouldn't a subway also be something like that a subway network, a subway graph would be a very good example. Another example might be, you could think of cities or airports as vertices and flights as edges. Right. So that's another way of thinking about waiting when you choose the flights to get you from here to wherever you want to go. Maybe you think about straight distance, but sometimes you think about the actual cost of the flights. And even if it sends you through buffalo on your way to Los Angeles, you're probably going to go through buffalo if it's cheaper to fly there for Because who would want to go to Buffalo? Any of you from Buffalo? I hope not. Anyway, moving on. I didn't have the best experience flying the Buffalo. So questions about graphs and what they can represent anything? Are we good? Alright, so let's do a couple more definitions. And then let's talk about how we're going to represent these things.

1:00:34

So for undirected graphs, we have this concept called a connected to graph. So a connected graph is an undirected graph, where there exists a path from every vertex to every other vertex. So if I start on any vertex, I could find a path to get myself to any other vertex doesn't have to be a one edge path, but I can get from any vertex to any other vertex. So this would be connected. And then that would be as distinct from disconnected where you might have vertices. Some that you can get between and some you can't. So there's nothing saying that you don't have a graph that that's that's connected, that that's not connected, you can have a graph that has vertices that aren't incorporated in with

edges, and that's fine. Everybody okay with that? All right. So that's the term for undirected graphs. But for directed graphs,

1:02:13

Alexa, stop. Sorry, I triggered Alexa. Um, so for directed graphs, would that necessarily fit? Well here? Let me add another edge. Would that necessarily fit my definition of connected? That I used up here for unweighted graphs? Not really, right? Because if I started at v_2 , I can't get anywhere. Does that make sense? But it feels weird to call this disconnected, because everything seems sort of physically incorporated into the graph. So we actually don't use the term connected by itself. When talking about directed graphs, we call this a weakly connected graph.

DS Lecture 19 Page 15 of 20 Transcribed by <https://otter.ai>

1:03:26

So a weakly connected graph is a graph that would have been connected if we ignored the directions of the edges. But because of the directions of the edges, you can't actually get from every vertex to every other vertex. Is everybody okay? With weakly connected and then related to that is strongly connected. And then a strongly connected graph. While in that one you're really can get from every vertex to every other vertex, even taking into account the edges. Okay? And these directed graphs could in fact still be disconnected.

1:04:31

Alright, right. So I think that about wraps it up for our definitions. So let's do one more example before moving on to representation.

1:04:54

Oh, perfect. Someone just left. Excellent. We just dropped from 9091 to 90 this is such perfect timing for this. Okay, good. How might we describe this graph? learn a whole bunch of definitions, how might we, it's disconnected? What else is there are more things? No, no, it's good. Well, I know you're still here, but someone left, it's directed. You'll see why in a minute. Um, what else? It's not really weak. It's not weakly connected, because there are elements of it that are completely disconnected. You can't get from these vertices to these. So it's disconnected. It's directed. It's unweighted. Yes.

1:05:46

That's pretty good. That's it? That's the only way you can describe this.

1:05:57

Did I not put the edges up too much? It's a cyclic, it's a smiley face. Right?

1:06:05

So

DS Lecture 19 Page 16 of 20 Transcribed by <https://otter.ai>

1:06:09

I don't know if it's gonna be the first question on part two of your exam, or the last question on the part two of your exam. But I am going to give you this graph on the exam. And I am going to say, describe this graph, and it's gonna be worth two points out of like, 150. And there are two correct answers you could give one correct answer is you could actually say it's the directed on weighted disconnected graph. Or you could say it's a smiley face, your choice, you'll get full credit for either. I'm gonna stick to the smiley face for a minute, and then I'll go back to the previous. Alright, your choice, you can say either, but I have to say, if anybody asks me about this ever again, I will deny this conversation ever happened. mentioned it on Piazza, I might as well delete your posts. Okay. No one discusses this. This is between us who are actually here. And the people who actually watch this lecture and nobody else. Are we clear? Alright, this is very important to me, because this is a test. This is an important test, it breaks the class up into three categories. It breaks the class up into the category of people who don't show up and don't watch the

lectures, the people who do show up and write smiley face, and then the third category, and you'll know what if you're you that third category, who cannot bring themselves to answer a question on an exam, smiley face, just can't do it. So instead, they just don't believe me that I'll give you credit. And we'll write all the things that it is, and then at the end and say, and also smiley face. So there are the three types of students, the ones who paid attention, the ones who didn't pay attention, and the Paranoid ones who just can't bring themselves to go along with a joke. And so it's very important to me to do this test, so don't ruin it. Okay. Moving on. You never saw this. It's gone. Yvonne, you wanted to see this sheet?

1:08:32

I don't know what previous paper we're talking about. Seriously? Was this the sheet that we were talking about? Yes. Cool. Did you have a question on this? time? We're good. Okay, so the next thing we need to talk about is how we're going to represent these graphs. And there are going to be two ways that we're going to represent our graphs. The first way is what's known as an adjacency matrix. And,

1:09:11

here, let me move this off. And let me draw this out for you.

1:09:33

DS Lecture 19 Page 17 of 20 Transcribed by <https://otter.ai>

So the adjacency matrix is the first way, we're going to represent our graphs. It's actually not the way we're going to do it that frequently, but it's important to understand. So let's say we have a graph that looks like this.

1:10:04

All right, so we have an unweighted directed graph. And so in an adjacency matrix, we're going to create a matrix of entries. And we're going to label them along both the rows and the columns with the vertices, so we've got vertices v_1 , through v_4 . And that is going to give us a grid. And since this is an unweighted graph, um, what we can do is if an edge goes between two vertices, so let's say this is the start. And this is the end. If there's an edge that starts here and ends here, we'll put a true, if there isn't such an edge, we'll put a false so there is an edge that starts at V_1 and lands at v_2 . So that's true, there's an edge that starts at V_1 and lands at V_3 . So that's true, there's no loop for v_1 , and there's no edge from v_1 to V_4 . From v_2 , there is no loop. But there is an edge that goes back from v_2 to v_1 . v_2 can also go to V_4 . But not v_3 , V_3 can't go anywhere. So that's false as across the board, and V_4 can only go to V_3 .

1:11:47

And so that is our adjacency matrix. So we can look up a starting vertex and an ending vertex and see whether or not an edge appears. Does that make sense? Now, if this graph were weighted, what could we change? To represent a weighted graph. So change the trues to one in the F to zero off? So let's say the weights don't necessarily have to be intz. Right? The weights could be any double precision floating point possibly. So we would just have a grid of doubles, rather than a grid of true false as a grid of Booleans. And then you would have the weight of the edge there instead of true or false. But given that, what would you do if an edge didn't exist? What would the weight be if an edge didn't exist? So zero or negative were the first two suggestions. So first of all, you could have edge weights of zero, versus edge weights that are positive an edge weight of zero just means there's no cost to traverse an edge. So it depends on what your vertices what your edge weights stand for. Remember, in my example, I said you could potentially Think of your vertices as airports and your edges as flights between the airports, you could think of the one of the options for the weight would be the cost of the flight. So if there's a free flight, for whatever reason, it doesn't cost you anything to go from New York to Buffalo, because man, getting people to go to Buffalo is hard. That would have an edge weight of zero, and that would be perfectly legitimate. negative numbers, similar deal. Imagine it's so

hard to get people to go to Buffalo that you actually have to pay people to go to Buffalo. All right, so then those edge weights would be negative, because you would get money to go there. So negative edge weights or zeros, unless you're a bills fan, okay, fine, but you always watch it on TV. Look, man, I had to fly to Buffalo once for a wedding. And honestly, the wedding wasn't that terrible. The problem was it was I had flown to France and back then to North Carolina, and back and then to Florida and back. And then I had a flight of Buffalo all within the span of three weeks. So I was really tired of flying. Oh, and by the way, that was like only the second span that I'd ever flown in before. So I had flown once to Chicago, and then I had to do all those flights. And the last one I did was buffalo. And that was the first flight I ever took that had turbulence. So I have always not like buffalo it also did not help that right after the turbulence. The guy sitting next to me tried to convert me to his religion. So it was just not a great flight. Okay, we won't get To the details of that. So suffice it to say, you'd have to pay me to go to Buffalo so that edge weight would need to be negative. So if there is no edge between the two infinity, then starts to become a good choice. Because if you remember, double precision floating points actually do have a representation for infinity. If you have a double, and you say you divide 5.3 by zero, it will actually come out to be infinity. And it will represent itself as infinity. Okay. I'm forgetting in this class that we have a conversation about my robots existential crisis.

DS Lecture 19 Page 18 of 20 Transcribed by <https://otter.ai>

1:15:42

Does that ring a bell? No. Okay, um, did it have to do with my hair? No, my robot doesn't care what happened to my hair?

1:16:01

I'll tell you the story of the robots existential crisis another day, we don't have time for this. Um, oh, I was sorry, for another day. All right, whatever. We don't have time for this. Um, so infinity would mean there's nothing there. Now, the problem with this is that if there are v vertices, no matter how many edges there are, there's good. Well say their magnitude v vertices, no matter how many edges there are, the amount of space you need to represent this is magnitude v squared, right? That's the space cost. No matter how many variables there's like one edge in the whole graph. So this ends up being quite inefficient space wise, unless you have what's known as a dense graph. In a dense graph, there are on the order of v squared vertices, like basically every vertex connects to every other vertex. So if you have a dense graph, this is fine. But if you have a very small number of vertices, you have what's known as a sparse graph. And this is not so good for sparse graphs. All right. So on Wednesday, we're going to talk about sparse graphs, and what's known as adjacency lists, and then we're going to talk about single source shortest paths, algorithms, and Dykstra. Alright, folks, good luck with your assignment. A person who messaged me privately No, no, I will not. That was the point of that conversation. Um, alright, folks have a good one.

DS Lecture 19 Page 19 of 20 Transcribed by <https://otter.ai>

1:17:55

So a dense graph is a graph where the number of edges is on the order of magnitude v squared, meaning every vertex tends to have a connection to every other vertex or close to it. That's what I would call a dense graph. Margot, does that make sense? Cool. Alright, I'm going to stop the share. Any other questions before I head out?

1:18:32

Thank you. All right, folks. In that case, I will see you guys later.

DS Lecture 19 Page 20 of 20 Transcribed by <https://otter.ai>

DS Lecture 20

Wed, 8/11 1:58PM 1:22:49

SUMMARY KEYWORDS

vertex, shortest path, adjacency list, graph, unweighted, algorithm, edges, distance, magnitude, v4, cost, path, adjacency matrix, queue, sparse graph, v3, chart, adjacent, dykstra, starting

06:25

All right, folks, we'll be getting started in just one second. Just getting the camera lined up here.

06:54

Okay, almost. All right. So we should get started. You had a homework due last night correct. Which you could submit potentially late as late as tomorrow night I think is the late deadline if necessary. Homework five written, which I think it should be useful for the final exam is live, you have a little bit over a week. Um, the programming problem, as I said is tiny in comparison to what it normally is, then remember, I do drop the lowest assignment, and I will allow you to use late days on homework five. In fact, I'm Yeah, I think that's what we're gonna say. So, we're gonna be covering the material that you'll need for homework five programming today. Um, and, yeah, so we started. So because before I go into that same I have any questions they want me to address before we get started.

08:23

Good. All right. I'm just trying to get up in my side window here.

08:35

There you are. Alright, so last class, we were talking about graphs. And we were talking

DS Lecture 20 Page 1 of 19 Transcribed by <https://otter.ai>

about various definitions associated with graphs. And we were talking about

08:57

you're in a bakery. I see. And we were talking about. We were talking about how we recognize that the Hungarian bakery Dr. Blair, I'm sorry, it's just

09:15

so we're going to finish talking about how we represent graphs. I'm going to skip around in chapter nine a little bit. And what I'm going to spend time on today is specifically single source, unweighted shortest paths, and then we're going to talk about weighted shortest paths, and specifically Dykstra his algorithm. That's the plan for today. I'm skipping over a topic called sorry, as a topic called topological sorting, because I want you to get Dykstra his algorithm, which I know some of you have seen in the context of Professor canons 1004 class, but I want to get to that first. I have a pre recorded lecture. You're on topological sorting. And I'm probably going to release that, instead of doing it in class. And then next week, we'll wrap up the portions of chapter nine that we're going to cover, and that will be it. Um, so last class, we were talking about adjacency matrices, right, the adjacency matrix where we had a grid of vertices, and then we would simply mark true or false, or the weight itself at the entry for a particular edge. The problem with an adjacency matrix is that we said the space cost for it was the magnitude of v squared, no matter what, because you have a grid, V by V , to represent it. And so that means even if you had very few edges, in the graph, like you have a lot of vertices and like one edge somewhere in the middle, you're still taking magnitude v squared, extra space. And so that actually brings up one final definition we may have for graphs, specifically, called a dense graph, versus a sparse graph. So a sparse graph is one that has relatively few edges with respect to the number of vertices. So in a sparse graph, the number of edges, which we can refer to as magnitude E , can actually be written as big O of magnitude V , it's upper bounded by the number of vertices. So you have either just one edge per vertex to like a link list, or even fewer, that would be a sparse graph. And so in a sparse graph, adjacency matrices are a very inefficient way of representing. But there's another type of graph called a dense graph. Now, we mentioned last time around that there is this concept of a complete

graph. And a complete graph is one where every vertex has an edge to every other vertex, right. And so in a complete graph, the number of edges you could have would actually be magnitude v squared. You couldn't have any more than that, because then you would just have duplicates. And so in a dense graph, we say that magnitude E is big O of magnitude v squared, its upper bounded by that maximum number of edges we could have. And it's relatively close to that. So I'm not really giving you an exact dividing line between the

DS Lecture 20 Page 2 of 19 Transcribed by <https://otter.ai>

two. But I'm saying that we call it a sparse graph, if you have, you know, kind of roughly the same number of edges as you have vertices. And we'll call it a dense graph. When you have many, many more edges, then you have vertices approaching the maximum. Yeah, so I'm saying big O of v squared. So if we're not going to include loops, like what you're describing Andre, um, that would just be subtracting off, I guess something that's on the order of magnitude v . So it would still be V squared in vago land. Do we agree? Under Yeah, cool. Miranda, remember that verdict? Sorry that graphs are the sets of vertices and edges. So these are the vertices.

14:03

And these are the edges. So magnitude E is the number of edges in the graph. So we're trying to use these to define potential relationships between the number of vertices and the number of edges. brand. Is that good? Awesome. Okay. So is everybody clear on these definitions? And so if we have an adjacency matrix, that's fine. For a dense graph, you'd need that much data stored anyway. But if you have a sparse graph, it's kind of accessed. And so that leaves us to the other way. And in fact, the most the way we're going to represent our graphs for the most part, which are adjacency.

14:49

lists,

14:52

edited adjacency lists. So let's say we have this vertex, this graph here. Let's say it's a directed graph. So v_1 and v_2 have edges going back and forth between them, maybe v_1 , and v_2 , also go to v_3 , maybe v_3 goes to v_4 , maybe before it goes nowhere. Okay? Now, the way we're going to generally represent graphs is we're going to have a collection of these vertices somewhere. Actually, it's really a set because each vertex is unique. But most likely, we just store it in a linked list of vertices, or an array of vertices. So we're going to have an array of vertex objects, v_1 , v_2 , v_3 , and v_4 . So these are the vertex objects. And you're going to have to have an object for each of them. All right. Now, the instance variables of these vertices is going to depend on what we're going to be doing and what we're going to be storing at these vertices. But if we're going to use an adjacency list representation, what we'll do is we will have a list of vertices that are adjacent to a given vertex stored within it. So there's going to be a link list within each vertex object that will have the list of vertices to which you can reach from the one. So what vertices would be on the adjacency list for v_1 ? Anybody? So v_2 and v_3 . So

DS Lecture 20 Page 3 of 19 Transcribed by <https://otter.ai>

that's going to be my adjacency list. That's my vertex. What's on the adjacency, list for v_2 ,

16:55

maybe,

16:59

we're gonna get to for v_2 , we want in v_3 , v_3 , I can only get to v_4 . And what is on v_4 is adjacency list. Nothing, the adjacency list is empty.

17:21

So

17:24

let's figure out what the big O cost is. For the space constraint of storing our graph with an

adjacency list? Well, we're going to need one vertex object for each vertex. So that's going to cost magnitude v right there. But then we're also going to need the space to store the link lists. Can anybody tell me how much space the link lists take up in total across all vertices?

18:05

Oh, but you do?

18:08

magnitude l , right? If you add up the lengths of all these adjacency lists, there is one entry across all the adjacency lists for each edge that is ultimately in the graph. Agreed. So if you sum up the lengths of the lists, that adds up to the number of vertices, sorry, the number of edges in the graph. And that's actually a very important feature. And we're going to come back to that when we analysis, analysis, analyze, wow, that was good when I was just talking to somebody in office hours, where we realized that they posted something on PX at two in the morning, and I replied at like three in the morning. So you can tell I'm doing here. So where we analyze the cost of our algorithms, the sums of the

DS Lecture 20 Page 4 of 19 Transcribed by <https://otter.ai>

links of these adjacency lists will crop up quite a bit, and they add up to magnets, you need the number of edges, which means that this is the space cost for an adjacency list. Now, the question is, is that any better? Is that any better than? what's the word I'm looking for then an adjacency matrix? Well, it depends on whether or not the graph is sparse or dense. Remember, the sparseness or denseness of the graph was going to tell us what E is in relation to V . So if the graph is sparse, magnitude l can just be rewritten as magnitude v . Right? It's just linear in the number of rows. So if it's sparse, the space cost of this adjacency list is just big O of magnitude V , because the E can convert to a V , and then you're just adding them, right. But if the graph is dense, magnitude e becomes magnitude v squared, which sort of overwhelms the magnitude v cost of just storing the vertices. So if it's dense, it's big O of magnitude v squared. So as a result, using an adjacency list has the same space cost as using an adjacency matrix if the graph is dense, but if the graph is sparse, then it's much better. It's just linear in the number of vertices. So an adjacency list tends to be the best option. Because what ends up happening is that it's no worse than an adjacency matrix, in some cases, and it's significantly better than the rest. So this would be the choice. And that's how we're going to be representing our graphs, as we go through the pseudocode for these graph algorithms, so everybody good with that? Okay, so what we want to talk about now is this concept known as the single source shortest paths, and that's cool algorithms. And so what we're going to be doing here, ultimately, what we're trying to do is find the shortest path from one vertex to another vertex, while traversing through the edges of a graph. And that actually should resonate pretty well with you. We talked about some of the use cases for graphs. And you know, one of the big ones was to represent a map, right, where the vertices were intersections, and the edges were the streets. And so if you want to find the shortest way to get from one intersection to another intersection, having an algorithm that can find the shortest path in a graph, that's how you're representing your math solves that problem for you. Okay. And so the first version of this algorithm we're going to do is we're going to do the unweighted version.

22:49

And in the unweighted version, we're gonna say every edge is the same as every other edge. So when we say what's the shortest path between two vertices, we were really asking what is the shortest number of edges we can traverse to get between them. That's the unweighted case. And it turns out, that's a bit simpler to deal with. So we'll start with that. Now, I should emphasize paths, plural. The reason for that is we pick a starting point. But it turns out that calculating the shortest path from a starting point to another vertex in the graph, is in terms of bego. Worst case, the same cost is calculating the shortest path from that single starting point to each vertex in the graph. So if I have a graph with v_1 , v_2 ,

v3, and v4, and I'm starting at v1, I'm actually going to calculate the shortest path from v1 to v2, and the shortest path from v1 to v3 and the shortest path from v1 to v4. And that's

DS Lecture 20 Page 5 of 19 Transcribed by <https://otter.ai>

not any more expensive, then all I need to do is choose what my end point is, and then I can reconstruct the path back. Why is that? The reason is, imagine the shortest path from v1 to v3 involves going through v2. So I go v1, v2, v3. Let's say that's the shortest path to get to v3. The first part of that path that gets from v1 to v2, that has to be the shortest path to v2. If it weren't the shortest path to v2, then I would have found a shorter path to v3. Anyway, that would have taken the shorter path to v2 first, then go to v3. Remember, these are unweighted. And so that means that the shortest path from one vertex to another could potentially go through all vertices. And if they go through all vertices, you have the shortest path to all of them. Alright, so let's draw a graph. And let's think about how we could go about calculating this. So give me one second to draw graph.

25:31

Need some edges. So I'm going to use directed edges here. But again, this is going to be unweighted. You Excuse me. Alright. Now, we'll pick a starting point, let's say I choose v three as my starting point. So what I'm going to do is I'm going to try to work out the shortest number of edges that it would take me to get from v3 to each of the remaining vertices in the graph. Now, because this is unweighted, we can actually do something that amounts to, I guess you could call it a breadth first search, or a breadth first traversal of the graph. What do we mean by breadth first, let's just for a moment, think about, imagine you have a tree. And imagine the tree has a root. And then it has a bunch of nodes that are children of that root, then a bunch of nodes that are children of that, and so on.

Alright. So if you're searching for a particular value, one way to do it, is by starting at the root, then look at all the things that are one step away from the root. And then if you find it, you have the shortest number of steps to get to that. Then if you don't find it on that first level, then you look at all the things that are two steps away from that root. If you find it, and you found it a two step length, and so on, going through one step out from the root each time breadth first. This is, you know, as distinguished as distinct from depth first, where you go all the way down first, and then work your way back up. Now, this is a graph, not a tree. So you have to just make sure that you don't get stuck in cycles. So you look one step out, then you look two steps out, but you can ignore things that you've already visited. Right. And since these edges aren't weighted, the moment you take one step out from the starting point, we know those are the shortest paths to those because they take one step to get to get to them. And you're never going to find anything faster. Because you did all the things that are one step out first, and the next thing you're going to do or two steps out. So the way I'll notate this on the graph, initially anyway, is we'll start by just annotating what the length of the shortest path is, to each of these. Now we can initialize the length of the shortest path to the starting vertex is zero, because we know we don't have to take any edges to get to that. Then we look through that starting vertices, adjacency list. And those are all the things that are one step out from it. And so v six and v

DS Lecture 20 Page 6 of 19 Transcribed by <https://otter.ai>

one are one step out from it. So we can mark those as having path lengths of one. And we're never going to find any faster way to get to them from v3. So we're done with them. Then we go through all the nodes or the vertices that are one step away, and we go one more step out. And that'll tell us all the things that are two steps out from the starting point. So v six doesn't have anything adjacent to it. So there's nothing to update there. But v one is adjacent to V four, and v two. So v four, we now know we can get to in this path length of two, as well as v two.

29:40

So now we do the same exercise for the things that are two steps away. Well, v two is adjacent to V four, but that one's already visited. So we already have a faster way to get

there. So there's no thing to update. But v2 is also adjacent to the V six. And so now sorry, not v six v five. So now we know we can get to V five and 123 steps. Now let's do v four, V four could get me back to V three, but that's already got a lower number assigned to it that's already been visited. So nothing there, v4 could get me to V six, but that already has a lower number. So nothing to worry about there, v4 could get me to V five. Now, that would get me there in a length of three. So I've actually found another path of the same length as the other one. But we're good. We don't need to update it, we already updated it. And so finally, before it can get to V seven, and that one hasn't been updated yet. So now we know we can get to V seven at a path length of three. And now we've gone through the adjacency lists of everything of path length two, then we go through the adjacency lists of everything of path length three. But we realized very quickly that since all the other vertices have been visited, nothing else is going to get updated. And we've got our shortest path lengths. Great. What didn't I actually do in this whole process that we were supposed to do?

31:24

actually record the paths, right? So we need to keep track, not just the length of those paths, but of the paths themselves. And so we're going to do this with a chart. Alright, so I'm going to maybe do this on another piece of paper, but I think I'll try to keep them both in frame. I'm gonna go ahead and you there. Okay, good. Um, I feel like we were getting a Las Vegas airport for some reason that feels like it was a Las Vegas airport sound.

32:02

All right.

DS Lecture 20 Page 7 of 19 Transcribed by <https://otter.ai>

32:05

I don't know why I felt Las Vegas. Anyway. So we're gonna need a chart here. Um, and, well, we have the vertices, I need more space. Oh, man, we're gonna have to go back and forth between these two pieces of paper. So we're going to need the vertices. We're going to need to know whether or not they are no. All right. Right, whether we visited them left. You know, you're right. I haven't actually been in an airport in Vegas in in years. But the last time I went, there were a lot of a lot of slot machines. Alright. We're going to need to have known whether or not a vertex has is he cool, good guest. I'm known whether or not it is visited or not, we're going to have what's we're going to refer to as the DV. And that's just the distance of the shortest path we've seen so far. These little numbers that I've written next to them, those are going to be the DVDs. And now comes the most important thing, because we're trying to keep track of the paths, we're going to keep track of the PVS. And that's the vertex immediately before us on the shortest path. All right, because if we keep track of just the thing immediately before us, I can work backwards and reconstruct the shortest path. And we'll see that in a moment. Oh, it looks like I missed a question. Omar asked, so max magnitude is three and Max magnitude V is 4 million. What do we mean mag into e? In terms of what? I'm not quite sure what you're talking about? You mean in terms of this graph? Oh, so in terms of the graph itself, there are 12345678. I can't even count it 3678 910 there are 12 edges, right? And there are seven vertices. But it so happens that the maximum path length that we ever had was three edges, right? Because when count edges when we talk about path length, is that what you were asking? Yeah, I just wanted to like to say like, should we look at it like high in depth, like we look in the tree to see what's the max like the shortest longest too, so they are paths, they are definitely past just like their paths in the tree. Right? That much is true. Okay. And you can think of v3 is the root. Good. Sorry, we're closed. All right, moving on. Um, so we have the vertices, which are v one, v two, V three, V four, V five, V six, and V seven. And I'm going to go and just make sure. Now if we're actually going to represent this in code, the known the distance the DV and the previous, the PV would actually just be fields in the vertex objects, and this would be the array of vertex objects. We'll see that in a minute. Now, an initial set

of known means that we've processed its adjacency list. And so initially, they're all false, they haven't been processed yet. Now we're starting in d3. So we know that the distance for v3 is infinity, if there is zero. And for all the rest, before we know a distance, we're going to mark their distances as infinity.

36:24

Okay, now, the PVS are going to stay empty until I actually find paths to these things. But I know that the PV for v3 is just empty, right there, there's nothing before the starting vertex. And then I go through and I do my, my breadth first search that I did before. First I

DS Lecture 20 Page 8 of 19 Transcribed by <https://otter.ai>

start by going through the adjacency lists of everything associated with the things that a distance of zero, so I marked v3 as true. And then I update v ones distance to one. And I got there with V three just before it. And I update v six is distance, sorry, that should stay false. For the moment, V six is distance to one, and I got there through v three. Then I go through and update everything at a distance of one. So I'll start with v one will mark that true. Go through its adjacency list v1 was adjacent to be four in v2. So v2 now has a known distance of two, that we got there via v1, as does v4. Also, from v1, we've still have more vertices. Actually, so somebody asked, Does it matter that I went to one or three first? And the answer is no, you just go through the adjacency list in whatever order it shows up, and it doesn't really matter. And similarly, it doesn't matter that I took care of v one first, before v six, you just go through all the things that have the same distance, just in the order you encounter them, it doesn't really matter. All right. So now we're going to do v six, V six isn't adjacent to anything. So we mark it as true but there's nothing to update. Now we finished the things of distance one, and then we do the things of distance two. So we'll start with v two, v two, we can now market is known because we're about to go through its adjacency list, v two was adjacent to V four and V five. So we go to V four v four already has something so there's nothing to update. me five was at infinity. So v five, now I found a path of length three through v two.

38:59

Okay.

39:02

And that's it for a v two. And so finally, on the to, I settled v four. And then I go through everything adjacent to V four, V four was adjacent to V three and V six, but those don't get updated at all. v4 is adjacent to V five v five already has something in it too. And so the only thing that matters here is that V four is adjacent to V seven, which previously was infinity, and now is three coming from v4. And now we have to settle the rest of them. We finished everything of distance two. Now we finished things of distance three, so we can settle the five. Doesn't really matter what v fives adjacency list is because everything else already has something and then we settle v seven and we're Done. All right. And so now, we have our chart has all the distances we did previously. But it has these PVS, which allows us to calculate the shortest path back from an N vertex to the starting vertex, which was v3. So let's say I wanted to get the shortest path from the starting vertex V three to V seven. So what I do is I work backwards, I know I want to get to V seven. So I write that down at the end of my list. Then I looked up v seven in the chart, and I see what was before it. And I realized that I just drew on my table, hold on. Hold on. Spit is a

DS Lecture 20 Page 9 of 19 Transcribed by <https://otter.ai>

remarkably good solvent. I'm sorry, I said that out loud. Yes, but I don't have one of those here because I'm in my living room. Okay, so v seven, we got there through v four. So we know what's right before it is V four. So we write V four before it. Don't have any Jan in the house. So that doesn't help me either. Do that vodka though. All right. So I have a bottle of vodka for which I have literally never drank a single thing from I use it to make like a vodka sauce for my pasta. Alright, so I know v four comes directly before v seven. So now I can look up v4 in the chart and see what becomes before that. And it's v1. So I write the v1

before that. And then I look up the v1 and I see v3 comes before that. Then I look up v3. And I see that was the starting point. And so now I have my path built backwards from v3 to V seven. And I got that from the chart. Is everybody good with that? Excellent. All right. Now, what I'm going to do want to show you some pseudocode. for implementing this, I'm going to show it to you from the textbook. But I'd like to leave the chart or not the chart but the graph up on the screen. So what I'm going to do is I'm going to change this so you don't get to see my smiling happy face. And I'll leave it there. Now if anybody's curious, you can actually set side by side view, which means if you click that would normally be my face, but it's now this will look as big as the stuff that I'm sharing. And then I'm going to go ahead and I'm going to switch the share. Actually, let me cue up the code first. Switch the share. Oh, can you use the stack to do the backtrack, I think you can just do this with a while loop where you just adding on at the end of the list. If you so you keep adding sorry. You start by adding at the front of the list. So you use a link. But you know what? The way I'm using a list is exactly a stack. So the person who said can you use this back to backtrack? Yeah, essentially, that's what I'm doing. Okay. All right. So let me share this. Alright, so now you're seeing a page from the textbook, which has an implementation of the unweighted shortest paths.

43:51

So we're going to assume that we have some sort of array of vertices that we can iterate through, we're going to assume that each vertex object has three fields that correspond to sorry, that correspond to the things in my chart. So it'll have a known field, instead of a dv and it'll have a distance field, you can see the V dot distance, what I've shared, and instead of a PV will have a path field. Okay. So the first thing we're going to do is we're gonna go through and make sure the chart is initialized properly, we set all the vertices to have a distance of infinity. Then we set all the vertices to be false in terms of their being known or not, because we haven't processed them yet. We've isolated one of those vertices s in a separate variable s. And we'll go ahead and set its distance to be zero. Now, what we're going to do is exactly What I described on paper, all right, um, we're going to iterate through settling all the vertices at a distance of zero first, then all the vertices at a distance of one, and so on, until we've settled all the vertices up to one less than the total number of vertices, because in the worst case scenario, the longest path you get, sorry,

DS Lecture 20 Page 10 of 19 Transcribed by <https://otter.ai>

longest path you can have is the number of vertices minus one. So we're going to go through and settle all of those in turn. So now what we're going to do, and this is a really inefficient way of doing this, and we're gonna find a more efficient way to do this in a minute, we're going to iterate through each vertex. So every iteration of the outer loop, we're going to do a full iteration through each vertex. So this outer loop is happening magnitude v times. And this inner loop is also happening magnitude v times. So at the minimum, this is going to cost magnitude v squared. Do we agree? Just from those two loops. Now there's more stuff going on here, there's another loop that we're gonna have to figure out how to deal with that in a minute. All right. So for each vertex, if it is not yet known, and its distance is equal to the current distance. So in the first iteration of the outer loop, basically, this is looking for the thing with a distance of zero. So if it's not known, and its distance is the current one we're settling, we make it known. And then we do exactly what we did on paper, we iterate through that vertex V's adjacency list, we have to update everything that's adjacent to it. And so for each vertex W, that is adjacent to V, if that W's distance is infinity, we now know a better way. So we set that w distance to be the current distance plus one, because it takes just one more step from where I currently am. And then we set the path equal to V, because we've got that distance by going through v. And that's the whole algorithm. If you let this go, and it's until it finishes, it will simply update the chart to exactly what we drew on paper. And then if you want to find shortest paths, from the starting vertex to anywhere, you just look things up on the

chart. Any questions on this?

47:43

Everybody clear with how this worked? Okay, now we should talk about what the cost is for. Right? So we should talk about the cost for a second. No, no, it's a good point. We know

48:00

that this part happens magnitude. So this thing here that I've highlighted sort of, is it's gonna happen magnitude v square times, we think. So what is the cost of that? Well, here's the trick. This test happens magnitude v squared times do we agree? Like every single iteration of these two loops, this test has to happen. But this test is not true that frequently. In fact, this test is true only once per vertex. All right, so this inner for loop will actually only happen once per vertex. Do we agree so far? So now, what's the cost of this, the cost of this is the number of edges that are adjacent to that vertex V . Well, in the worst case, there could be as many as V vertices adjacent to it. But we can do better. Because really, if you look at this across all vertices, the stuff that's inside this adjacency list

DS Lecture 20 Page 11 of 19 Transcribed by <https://otter.ai>

happens once for each item in each adjacency list. And we already spoke about how many items there are across all the adjacency lists, how many items are there across all the adjacency lists? Anybody $2E$ or magnitude even number of edges. So this inner part, you can actually factor it out. From the dual loops outside, because we know by analyzing the conditions more carefully, that this will only happen once for each edge. And so the overall cost becomes magnitude v squared plus magnitude $2E$.

50:21

All right.

50:23

Which actually, well, doesn't matter if the graph is sparse or dense, right? Because if the graph is sparse, this magnitude becomes magnitude V , but that gets dominated by this term. If the graph is dense, this term is v squared. But so is this. So they just combined, so no matter what happens, the cost of this algorithm is quadratic. In the number of vertices, regardless of whether it's sparse or dense. Does everybody is where I found another one. You want to see it? Not really? Sure. Yeah, we really got to move here. Alright, fine. Sorry. All right. Is everybody okay with that analysis? Alright, so now we got to do better. This should be very reminiscent of something that you did on homework three. On homework three, you had a problem called pretty print? Oh, wait, no, no, no, no, you didn't have a problem called pretty print. I changed. It wasn't pretty print. It was called. What was it? It was it was called level order traversal. Right. Remember, the level order traversal. The reason why I mix the two up is that in previous versions of the homework, I used to have a pretty print where you actually had to print the tree, but that just use the level order traversal. But it caused so much confusion. I just explicitly made it a level order traversal. Okay. I I'm gesturing wildly as I say this, and I realize you can just barely see the shadow of my hand gesturing, sorry. All right, yes. Now it's important. It wasn't important beforehand. So now it's important. And this is this breadth first search is really a level order traversal. If you don't think about cycles, right? If you ignore the cycles, right, you're going through all the things that are one step out all the things that are two steps out, how did we implement level order traversal. What data structure did we add to this whole thing to make it work, it was a queue that was interestingly spelled, alright. So it was a queue. And so we can actually, possibly improve this by using a queue. By using a queue, we no longer need to mark things as known. All right, because the items get added to the queue once, and they get removed from the queue once and that's it. And so that's what we're gonna do, we're gonna create a queue of vertices, a queue of vertex objects, right here, we're gonna initialize each vertex is distance to infinity, but we don't need to bother with the

DS Lecture 20 Page 12 of 19 Transcribed by <https://otter.ai>

known fields. We again need to start the starting vertex SS distance of zero. And then we enqueue. So the idea is, once we know what the distance is to a vertex, we enqueue it. And then D queuing, it is the moment when we go through its adjacency list. So right now, the only thing whose distance we know is the starting vertex. So that's the only thing on the queue. But as we go through all the things that are adjacent to us, we will suddenly know those things distances. So we'll tack those onto the queue. So the idea is we're just going to keep going while the queue is not empty. Once the queue is empty, we've processed all of them. So the first thing we do is we extract the first vertex and this is now constant time the dq. And that's going to be initially the thing at a distance of zero. When we go through, you know, it must be tired because I realize I'm actually lecturing this at practically the full volume that I use when I'm in a large lecture all like God, my neighbor must hate me if you didn't already hate me. Because I'm, I'm shouting this at the shared wall. Sorry, this amuses the hell out of me. I just realized I was using full Volume, I should quiet down.

55:04

No, not quieting down. Alright, moving on. Um, so we go through the adjacency list of that vertex. All right. If that adjacent vertex is infinity, we updated distance to $b[v].dis + 1$ just like we did before, we set its path to be V , just like we did before. And then we enqueue w , because now that w has a distance known for it, and it's only going to happen once that will get the distance for it, we tack it onto the queue. So originally, the queue just had the thing with a distance of zero. Now that we're done with that, it comes back around, and the queue has all the things of distance of one. So now, the first thing that gets pulled off is the first thing of distance one, the rest of the things with distance one will still be on the queue, we go through and we process what's adjacent to those things of distance one. And those are going to create things of distance to which we tack on to the end of the queue, which are after the things of distance one. So we're guaranteed to go through all the things of distance one, before we get to the things of distance two, and so on. And ultimately, this will generate the same chart that we had before in terms of the distances and the paths. Does anybody have any questions on this algorithm? We're good with it. Okay, so now, sorry, go ahead. How are we guaranteed that we're going to go through all the other distance ones before we get to distance to I missed that part? Because it's a queue. And what you're doing is when you dq initially, your D queuing a thing of distance zero, so all the things that are of distance one are adjacent to that thing that's of distance zero. And so you're adding them on to the queue here. Okay, and then they're already on the queue before you pluck off the first thing of distance one, which will then start tacking on things of distance to at the end of the queue. So it's because the queue is first in first out that we get that. Does that make sense? Some? Excellent. All right. So Sally, I'm about to answer this question. Um, so now, we need to see what the big O cost of this is. And see

DS Lecture 20 Page 13 of 19 Transcribed by <https://otter.ai>

if we did any better than the big O of v^2 that we had before. So the outer while loop happens exactly magnitude v times because we only dq a vertex once over its lifetime. And this algorithm agreed. And the dq operation is order one. So these lines of code that I've just highlighted here, cost magnitude v . So far, so good. Yes. Okay. Now, we have this thing. But the analysis for this inner loop is exactly the same as the analysis of that innermost loop in the previous time, we can actually factor it out. Because the guts of this will happen precisely once for each edge. So that means we can factor it out. So this is magnitude V , plus magnitude E . Okay. So now that we have that, we see what the improvement is. If the graph is sparse, we can rewrite e as just magnitude v . so this becomes big O of magnitude V , it becomes linear in the number of vertices. If we have a sparse graph. That's a massive improvement. Because in the previous example, if it were a sparse graph, we'd still have the square. But if we have a dense graph, magnitude is still magnitude v^2 . So in a dense graph, it's big O of magnitude v^2 . So if the

graph is dense, have we improved anything? Now, but if the graph is sparse, have we improved anything? Oh, yeah. So if the graph is sparse, we're in great shape. If the graph is dense, we're no worse off than we were before. So this is really the general formula for the cost. its magnitude v plus magnitude $To eat, but the moment we know whether the graph is sparse or dense, that's when things start to get a lot better. Okay, so this is no worse than the other one in the worst case. But if you have a sparse case, it's way better. So you always do this, because it's never going to be any worse than what we had in the previous one that sometimes it's better. Any questions on the unweighted single source shortest paths? Okay, now, we got to start talking about the weighted version of this, the weighted version of this is a little trickier. So let me switch my shares around a little bit. You can you can look at my face again, I apologize for my face. I'm moving on. And let's have the main view be the camera. Okay.$

1:01:02

So now we're going to do the single source shortest paths in a weighted version, so we're gonna have a weighted graph. And the weighted version of this algorithm has a specific name, it's called based algorithm.

1:01:42

Now Dykstra is algorithm is an example of what's known as a greedy algorithm. And I'm forgetting did we talk about greedy algorithms before? In this class? Yes, no. So a greedy algorithm, let me digress on that for a moment. A greedy algorithm is an algorithm that a greedy algorithm is an algorithm that does what appears to be best at any given step without looking at the overall circumstances. Now, sometimes greedy algorithms can still

DS Lecture 20 Page 14 of 19 Transcribed by <https://otter.ai>

give you an optimal result, sometimes they don't. So let me give you an example of something that sometimes works with a greedy algorithm. And sometimes it doesn't. So the example here, not when I'm drawing, the example that I'm describing is making change. Let's say you were told you need to give somebody change, like a certain amount of sense. And you want to do it with the smallest number of coins, right? So they want to give somebody 85 cents, the algorithm that one would typically use if you were making change, and I guess if any of you have worked cashier jobs, you probably have been taught this algorithm. You go to the largest coin first. Greedy, you take the biggest thing you could, and then you take as many of the largest coin that will fit into the amount of change you need to give as you possibly can without going over. So if I'm trying to give 85 cents change. Oh, I'll be obnoxious. So you know, we do have half dollars in American coin systems, right? If any of you actually ever gotten a half dollar back has changed for something. I mean, I'm guessing the the slot machines probably did. You once. You know, it's funny. How old are you like 20? I'm, I am at least twice your age. I can only think of one time a human being has ever gave Oh, well. Okay. I can think of only one time in the 40 some odd years that I've been alive in my entire life when a human being has handed me a half dollar. And it was at a movie theater in Rockland County when I was 17. And I don't know why I remember that. But there you go. But let's pretend we're at that movie theater in Rockland County that I think has long since been torn down. And if I were going to give you change for that 85 cents, the first thing I would do is I would go to the 50 cent coin because that's the biggest one that would fit in and I would take that 50 cent coin and then I would subtract it from the 85 cents and I'd be left with 35 cents. And so then I would take the biggest coin that would fit into the 35 cents I take a quarter and then that would be good. And then I'd have 10 cents left and I take the biggest clinic I take a dime. And there we go three coins, a 50 cent coin, a quarter coin and a dime and you have the smallest unit Individual number of coins to make change. And that algorithm works with the American coin system. And that's a greedy algorithm, you're doing the biggest thing you, you can the best thing you can at each step. But that only works in certain circumstances. Um, I can break it. I can break it by adding another coin to the American

system. And so I'm going to break it by adding a 13 cent piece to the American system. All right, what do you want to put on this 13 cent coin that breaks everything? You know what? Don't answer that. Seriously, don't answer that. We have this 13 cent coin, James Buchanan. Alright, fine, we have the Buchanan and we have this 13 cent coin. Now imagine I asked you to give me 15 cents change. In the old coin system, the largest coin you would take would be the 10 cent piece the dime. And then you would take the nickel and two coins would be optimal. In our new system, two coins would still be optimal. But the 13 cent breaks our algorithm, because the largest coin that goes into 15 is the Buchanan with the 13 cents. And then you got two cents left. So you got to give two pennies back. And so now our greedy algorithm fails because of this new coin. And we ended up with three coins instead of the optimal two. So sometimes.

DS Lecture 20 Page 15 of 19 Transcribed by <https://otter.ai>

1:06:39

Okay, sometimes greedy works for optimal sometimes it doesn't. Dijkstra is algorithm is one of those times where greedy does work. We're not going to prove that textbook does go into that a little bit. But let's talk about what this algorithm is doing. So the first thing I'm going to do is I'm going to draw in some directed edges, but I'm going to make them weighted now.

1:07:15

Give me one sec. Okay, we now have a weighted graph. Now, the first thing you should note is that the algorithm we did for unweighted no longer works, because one step might have a lot more costs associated with it than multiple steps. So for example, if I wanted to get from v_2 to v_5 , a single edge to get from v_2 to v_5 costs me 10. But going v_2 to v_4 , and then v_4 to v_5 costs me five. So a pure breadth first search no longer works. So how is this greedy? The way we're going to do this is as we go through our algorithm, we're going to keep track in terms of like those same little DVDs and PVS that we did previously, we're going to keep track of what is the shortest path we've seen to each vertex so far. But we don't settle that vertex. And in this case, settling a vertex means saying that we know that this is truly the shortest path to it, we only know that it's the shortest path. So far, until of all the shortest paths so far, it is in fact, the smallest. So if I have unsettled vertices, v_2 , v_3 and v_4 , and the shortest known path to v_2 is one, the shortest known path to v_3 is two, and the shortest known path to v_4 is three, I'm going to settle the one that has the shortest of those, because I know that I'm never going to find anything shorter to it, because it would have to pass through pass through one of the other paths that I have and all of them are already longer. So I settle the shortest thing of all the shortest paths so far each time. And so the idea in this algorithm is each step of the algorithm we settle one vertex, the thing that we have the shortest way to get to. And so once we fix one vertex, then we update the other vertices shortest paths based on its adjacency list until we've settled all of them. Okay, that's Dijkstra is our diagram at the second. But I should note that Dijkstra, his algorithm really only works if you don't have what are known as negative edge cost cycles. So remember, last time around and kick, I was making fun of Buffalo, when we were talking about edges, that could have a cost of zero a weight of zero or a negative weight. This algorithm will work if there are negative weights, but if there is a cycle, where the total cost of going through that cycle is negative, so if you have something that's like,

1:11:05

DS Lecture 20 Page 16 of 19 Transcribed by <https://otter.ai>

one, two, then minus five, going through this cycle would give you a total cost of minus two. And so the problem is, with a greedy algorithm, you'd always want to just keep going around that over and over and over again, decreasing the cost more and more and more. So your algorithm wouldn't terminate. So you can't have negative cost cycles. Now, we're going to bypass that problem for this example, by just not having any negative cost

edges. And if we don't have a negative cost edge, we definitely don't have a negative cost cycle. So now, let's build our chart for Dijkstra's algorithm. So it's going to start out looking the same as we had before. We have the vertex. See, yeah, just make sure. That's the room I have, we have the vertices, we have whether or not it is known. We have the DV, which is the distance in the PV, which is the vertex before. So that's our chart, and we have vertices v one through v seven.

1:12:37

Now, initially, none of them are known, so they're all false. Now, we need to pick a starting vertex, right? Because this is finding the shortest path from one vertex to all of the others. So let's start with vertex one, just for the sake of argument. So we know that the DV for V was the one that the DV for v one is zero. And there's nothing before it. Everything else

1:13:17

has an initial distance of infinity. Okay. And so now we go through and do our algorithm, our algorithm says, do the greedy step, pick the vertex that hasn't yet been settled with the smallest current dv, and then settle that one. Everything else has a dv of infinity, except for the starting vertex. So I can settle the starting vertex, the starting vertex is now settled, I know I will never find a shorter path to it than zero. And by the way, if I had a negative was that if I had a negative cost cycle, that would break that assumption, but we don't have that. So now I go through v ones, adjacency lists, and I update the potential shortest paths to everything adjacent to v one v one is adjacent to v two and V four. Now, the question is, what is the cost of getting two v two through v one, it's the incremental costs of going between the two vertices plus whatever the DV is for v one, right? Because we have to get to v one first, and then take that edge. Now it just so happens that the DV for v one is zero. So in this first iteration, the cost the total cost to get to v2 through v one is the length of that edge plus the zero of v one. So now I have a way to get to v2. At a total cost of two, and I did it by going through v one. And I go keep going through, I can get from v one to V for incremental cost of one total cost also of one. So v four, I also now have a way to get to this time with a cost of one. And I also do it by going through v1. And that's it. I finished iteration one of this algorithm. And so now I have to do the second iteration. So I got to settle the next one. So we do our greedy step. And we settle the

DS Lecture 20 Page 17 of 19 Transcribed by <https://otter.ai>

smallest vertex of the currently unknown vertices. So who do we settle next. So v four has the shortest, dv, so we settle v four. And now we have to go through v fours adjacency list. Now remember, the cost to get to V four in the first place is one. So when I look at the total costs of getting to one of the adjacent vertices to V four, it's the sum of the cost to get from A to V four plus that incremental distance. Now I have a path to v3 at a total cost of what what's the new path that I found a v3 here costs? Maybe it's three, it's the sum of the incremental distance to plus the distance to v4. So I can get to v3 at a distance of two, which is better than the infinity that I had. So I updated. And I did it by going through v4. No, you really want to do the short ones first. Alright, what else is adjacent to actually it doesn't really matter here, you're just going through the list. The cost to get to v3? Oh, yes. That that was supposed to be a three, which is why I made a big deal about and then I wrote the wrong thing. We good cost three. Excellent. All right, we keep going through the adjacency list. v4. Bow, I said that wrong, it doesn't actually matter what order you go through them. So I just happened to go through it in terms of the the lowest numbered vertex. So Alright, let's do v six. Next, the cost to get to V six through v four is the total of nine. Better than infinity, the cost to get to V seven through v four is a total of five better than infinity. And the cost to get to V five through v four is a total of three, better than infinity. Now we've gone through the fours adjacency list, go do we settle Next, we settle the thing with the next smallest distance, that's going to be v2 with a distance of two.

1:18:16

So we go through v two's adjacency list. And the only thing about we find a path to v4, but v4 is already settled. And we find a path to V five that cost a total of 12. But we already have something that only costs three, so there's nothing to update, we're done with that, then we update it, we settle the next smallest vertex, which could be either v3 or V five, they're tied. So it doesn't matter who you pick. Alright, I am out of time. But you just keep going. I'll finish this chart up for you on Monday. And then on Monday, we'll go through the pseudocode. For this now, your homework is the written is posted, the programming will be this algorithm. And so what I want you to do is I want you to look at the pseudocode. And you're actually translating that pseudocode into code that you put into that program that I'm asking you to write. That's all I'm asking you to do, really. And so when I post that later tonight, you can start working on that. And then we'll go through that pseudocode on Monday. I did skip over a topological wait a minute is another programming assignment now. Yes, there are five. Um, so they're going to be um, what was I gonna say? I'm gonna post a separate video for topological sorting. And, yeah, that's about it. Have a good one.

DS Lecture 20 Page 18 of 19 Transcribed by <https://otter.ai>

1:20:04

Can I ask a quick question on the quicker algorithm for the unweighted? Yes? traversal? Is there any, when we is there any weird edge case? extra cost for? Um, in queueing a vertex without checking if it's distances infinity? Or does that does that not matter? So that again, so that one more time, the so when you when you loop through the adjacency matrix, you check off each vertex is infinity, and then do that. Is that, is there any weird edge case we're doing that would increase the time cost? Rather than Thank you,

1:20:56

oh, that's baked into the time cost that I gave you. Because that test is a constant time for each iteration of that loop, which we already baked that in with the V squared call or the V, whichever when we're talking about cost. Okay. Okay. Thanks. Martha, what's up?

1:21:14

Hey, Professor, Adam, quick question about the same algorithm. So each algorithm, the unweighted, shortest path one, right? So when we look at each vertex w adjusting to V, I don't understand why we set W dot path to V. Why aren't we adding to the path? Like I don't get I don't get what's just okay, what is the path basically,

1:21:48

path is the vertex directly before that vertex along the shortest path to it. So let's say we're at vertex V five, and its path field says v four. That means that the shortest path from the starting point to V five goes through v four. So we first take the shortest path to V four. Then we take one additional step from V four to V five. So the previous path just tells me what vertex is directly before me. Again, the shortest path to me.

1:22:30

Oh, so that then when we're reconstructing it, when we do it from backwards, then we get the shortest path. Okay. Thank you. Okay. All right. Anything else? Alright folks, have a good night.

DS Lecture 20 Page 19 of 19 Transcribed by <https://otter.ai>

DS Lecture 21

Wed, 8/11 2:02PM 1:18:44

SUMMARY KEYWORDS

vertex, priority queue, magnitude, graph, algorithm, dykstra, edges, homework, questions, degree, adjacency list, queue, update, shortest path, calc, cost, topological, written, log, squared

00:47

reluctantly crouched at the starting line, engines pumping and thumping in time the green light flashes the flags go up, churning and burning the urine for the cup they definitely maneuver and muscle for rank burning fast on an empty tank or reckless and while they pour through the turns their prowess is potent and secretly stern as they speed through the finish. The flags go down, the bands get up and they get out of town. The arena is empty except for one man still driving and striving as fast as he can. The sun has gone down and the moon has come up and long ago somebody left with the cup but he's driving and driving and hugging the turns thinking of someone still burns.

01:37

She's

01:42

because he's

01:43

racing

01:47

DS Lecture 21 Page 1 of 22 Transcribed by <https://otter.ai>

and riding on his knees going.

02:04

trophy no flowers no flashbulbs, no line, he's haunted by something he cannot define bowel shaking earthquakes of doubt and remorse assail him and bail him with muscle truck force in his mind. He's still driving still making the grade. She's hoping in time that her memories will fade because he's racing and pacing and haunting the course. He's fighting and fighting and riding on his horse. The sun has gone down and the moon has come up. And long ago, somebody left with the cup, but he's driving and driving and hogging the turns and thinking of someone he still burns. Because he's going he's going,

02:43

geez, Allah.

02:48

Because he's amazing.

02:51

He's finding and finding and writing on his own. He's raising and pacing and learning

02:57

and writing on his own. He's going

03:02

he's going,

03:48

Hey, she should get started. As you see on your screen, we have Dykstra his algorithm. I do want to make a few comments about our plan for the remainder of the week. Um, so let me just check on one thing. No, no, no need for that. I know the answer. Um, okay. So here

DS Lecture 21 Page 2 of 22 Transcribed by <https://otter.ai>

is here is the plan. Today, we're going to wrap up Dexter's algorithm that we really went through it in its entirety, but I want to run through the pseudocode because essentially, this is what you're being asked to do in the programming assign. Yeah, what does it do, by the way, Dr. Blair, it says on the assignment to Thursday, but it's a very, very short thing. That's the programming or the writing one. That would be both. You ate them both do on Thursday classes. The only way I could work Um, so That this assignment, I would say is because of the programming is so much simpler than the previous ones, I would say this assignment probably will take about half the time as the other assignments. Honestly. There is a little bit of weirdness since question seven point 20 and seven point 21. On the

written but in all honesty, once you come down to it and realize we want the absolute worst case possible, um, the answers are very straightforward. After we finish up Dykstra his algorithm, I'm going to back up a little bit, I was going to originally give this as a separate video, but I feel like it would be better if I just explained it myself. I had you read Excuse me, I had you read the topological sorting, which was reflected on the homework, but I would like to run through that today. And then the last thing I'm going to do is I'm going to introduce something called minimum spanning trees. Okay. We will wrap minimum spanning trees on Thursday, sorry, not Thursday, on Wednesday. And we will then do the section in chapter nine on P versus NP. Okay. And that will be it for the material. And that shouldn't take more than halfway through Wednesday's lecture. And then my plan is some point tomorrow, to release the review sheet for the final, the first half of it will be identical to the review sheets for the midterm because well, it's a cumulative exam. And I'd like to do a quick run through of what to expect for the exam on Thursday, sorry, on Wednesday at the second half of the lecture, and that'll be it for the class. Now, the one thing I would like to say, it's probably worthwhile for us to do a review session, which would have me going through that list, again, in more depth, maybe make some mention of the kinds of problems which will be on the exam, which, you know, I will mention also, on Wednesday, I'll be it in a more brief form, and then spend a bit of time entertaining questions. So, I don't think I'm gonna be able to do that during regular lecture. So we should schedule review. Um, I think the best I'm going to be able to do is Friday. Um, now, you guys know that I record those sessions. It's the standard class link. For the captioner, I will reach out to disability services for that. And my feeling is that five o'clock on Friday is the best I'm going to be able to do because I have my I have my individual meetings that go from 230 to 430, on Fridays, and a lot of those are already starting to get booked. So I need to half an hour to save my voice and then start again. So we'll do it at five o'clock on Friday. Okay, and don't worry, if you can't make it, I will release the video or the video will be automatically released right after. Okay, so is everybody good with that? Now I am sorry for the short duration on homework five. But I had warned you it was going to be a week. And I do drop the lowest assignment so if you were happy without homeworks once or three went in or pretty confident on for your most of the way done grading for by the way,

DS Lecture 21 Page 3 of 22 Transcribed by <https://otter.ai>

08:48

you can go ahead and skip five. Okay. Um, I think running through the written as part of your studying would be helpful. The programming is a good exercise, but the way dice does algorithm will be tested on the exam and it will be tested on the exam. We'll be filling out a chart more akin to what we did on pen and paper last week. That actually will be the question I dropped. Sorry, the questions in the chat. I'll get back to all of them in a second. I just want to finish my thought here. So I think I think my thought is mostly done. I think that gives us the rest of our schedule. Um, I should mention that the exam is actually the dates have been set since pretty much the beginning of the semester. If you scroll down on the front page, of course works and look at where I have the exams. I indicate that I will give you a 72 hour window to do the to do the final and it's the same kind of setup as the midterm, half a quiz multiple choice, fill in the blank all auto graded Half more written through great scope, each half will be an hour and a half. Um, I'll give you slightly longer on the, on the part two just to account for you having to sort of photograph and upload your your results if you hand wrote it. Um, and yeah, so you will have from wins. That's wrong. I said I was gonna make it available from the 20th through the 22nd. I believe the 22nd is the Thursday. Correct. So you have Tuesday, Wednesday and Thursday, starting at 12am. I didn't realize you could do that whoever annotated Don't let me see clear old drugs. But yes, I'm glad you're happy. But don't do that again. So you have from that 12am on Tuesday in New York time through 11:59pm. To finish it on the

Thursday, again, New York time, so you have a huge window. Once you start a part, the timer starts and that's it, you got to finish it in that hour and a half or longer if you have extended time. And then you can do the other part at some other point during that window. It just it must all be done by 11:59pm. All right. So and trying to give you as wide a window as possible. I do not remember exactly which day the registrar's scheduled our final exam slot, but it's one of those three days. So I basically set it up to surround when the exam was scheduled for so if you want to take it during the scheduled exam slot, or you be my guest. But I'm hoping that this will take care of any conflicts or anything like that. So I'm good. Alright, so let me go back and look through the questions.

12:10

Uh,

12:13

okay.

DS Lecture 21 Page 4 of 22 Transcribed by <https://otter.ai>

12:16

Do you know the range of days the exam will be Yes, Yes, I do. we just discussed that. So good timing. I'm using it on Wednesday the 21st. Well, that's one of those three days, but okay, you can if you want. Do you drop the whole piece at grade or just the written or just the programming? I've indicated this week, something just landed on the porch. Michelle? Michelle, I just heard some banging on the door. Sorry, I'm just disconcertingly loud. So I dropped the entire problem set because the writtens of the programmings are weighted differently. So I don't pick and choose. So all five, all four, all three, all of two or all in one. No way we can adjust that. You can use late hours on homework five. So you can use any late hours you have left on homework five, I'm not cutting it off. Your final you cannot use late hours on I don't know what exactly why that came up. But it did. I didn't say when homework four will be graded by I just said they're getting pretty close. I can't say exactly. But we're getting close. Written programming dropped together. Sure. Also, since we were sequentially addressing these questions from chat coming to us like so I got ahead of myself. No, you can't use it on the final exam, but you can use it on homework five. Um, the exam is approximately 60% to 65% longer than the midterm. So it's twice the time but not quite twice the content. Okay. Do you think written homework is good prep for the final homework five? That is I think homework five written helps. I think it does. Okay. As usual, I think my exam questions are easier than my homework questions. Okay. But it's a good start. Right, anything else? Okay, folks, let's do this. We're almost done. We are, as the song said, going the distance, although I think some people already finished just like the song said, are moving. So thanks to the algorithm. As you recall, this is a greedy algorithm. We keep our chart but our chart is baked into our vertex objects. So each vertex object has a field indicating whether it is yet settled. That's the known field. Draw False, it has the distance field, which is the currently known shortest path to that vertex. Only when the known is marked as true is that truly the shortest. Um, and it has a path field which corresponds to the PVS that I had on my chart. And that path field tells me who's directly before me along that shortest path, which allows me to work backwards and reconstruct the path, which is something else that you'll actually have to do as one of the parts on your homework. Okay? Alright, the first thing we do is we're going to loop through, and we're going to initialize all the distances to infinity, and all the knowns to false. And then we're going to set S dot distance equal to zero S is our starting vertex. Remember, this is single source shortest paths, meaning we calculate all the shortest path paths from the starting point, S is that starting point, so we know so far, we have a path to it, that length is zero, we have an outer while loop, while there is still an unknown distance vertex. So while any of the vertices are still labeled as unknown, there are a number of ways you can work that out, it depends on how this is implemented. Now, there are two types of implementations. For Dykstra rhythm, the simplistic implementation, which is

what we're going to discuss here, and what I was actually asking you to do on the

DS Lecture 21 Page 5 of 22 Transcribed by <https://otter.ai>

homework, just as you're going to just keep scrolling through the list, looking to see whether or not there are any more unknown vertices, what some of you who have already started doing, the programming portion has noticed is that you can make a separate list of the vertices. And then as you select them, you can just remove them. So that sort of guarantees that only unknown vertices are on that list, and that's acceptable.

16:54

Okay, so while there's still vertices to settle, we're going to settle one. So we go through that list of unknown vertices, and we look at their path field, sorry, their distance fields, we find the smallest one, and we set v equal to that. Now in the simplistic implementation, we're just iterating through that list. So that's a find min operation. So that's sort of a magnitude v cost operation. And further, this outer loop happens magnitude v times because you want to settle each vertex once. So the combination of these two lines tells me I've got a cost of at least magnitude v squared. So far, so good. We've now selected the smallest unknown vertex, that's our ready step, we mark that suckers true, because now we know that's the shortest we're ever going to get. So he's true. And then we've got to update its adjacency list. And so somebody asked, Can we use two lists, one to keep track of visited vertices and one to keep track of vertices to visit? Uh, I think the way this algorithm works, you don't actually need to keep track of the ones that are visited, I think you only need to keep track, the ones that are left to be visited. But if it if it makes you happy, that's okay. Just be careful, you're not sort of nesting your loops through them. Does that make sense? anonymous questioner Cool. All right. So now that we've selected our vertex V , we have to update its adjacency list. So we go through each vertex W , that is adjacent to V . And if W is not yet known, we are potentially going to update the cost to that vertex. So $CV W$ is the cost to get a no a priority Q is not necessarily easier. We're going to talk about priority queues in a minute, I do not want you to use the priority queue on your homework, but it is an alternative way to end this. Alright. So $CV W$ is the cost to get from that settled vertex V to its adjacent vertex W . This is that computes all Euclidean distances in your homework. That's where that distance cost comes from. So that's the incremental cost from V to W . And then you are checking to see if the sum remember this update rule is the total cost of getting to V and then that next step two v two from V to W . So if that total cost of V dot does plus t , $v w$ is less than the currently known distance of the shortest path to W , we have found a faster way. Okay? Now, at this point, you may be wondering why on earth doesn't say decrease W dot dist to V dot dist plus $v w$ gonna change my camera for a second? Um, so what you should do if you haven't already? Well, I don't know why you would have before. But if you go to side by side view, you'll get a larger view of my sort of face camera, which is now the overhead. So I'm going to jot down, really, why didn't I just say that as w dist equal to V dot dist plus $c v w$? And the answer is, as far as your homework is concerned, that is precisely what you're going to do.

DS Lecture 21 Page 6 of 22 Transcribed by <https://otter.ai>

The reason why the code sort of lists it as this sort of nebulous decrease method is because this operation becomes a little bit trickier if you're using a priority queue. But right now, we're not bothering with a priority queue. Once we've updated W dot dist, we then changed W dot path to indicate that we got there through v . And that's the end of this. Now the loop goes back around picks the next smallest, and we are good. Okay, this is what you're implementing. Does anybody have any questions on this?

21:24

All right, so let's talk costs. We already have a magnitude v squared costs as the combined cost of this outer loop and the greedy step, which for the moment, we're assuming we're doing by just iterating through a list, okay. Now, this is the standard analysis that we had been doing in the previous unweighted shortest paths, where we're

going through here each edge that emanates from V . And each v each vertex gets settled once. So the W is representing it is the W is like the variable and an enhanced for loop. The w this for each is iterating through these adjacency list. Does that make sense? anonymous questioner Awesome. All right. So now this happens, this happens once per vertex settled, and then goes through all the edges adjacent to that vertex. So the total number of times that this piece that I've highlighted here happens is exactly magnitude e times across all iterations. Okay? And so that's additive. So we've got V of magnitude v squared, plus magnitude e using the simple implementation.

23:14

And as you know, when you have something like this, if e is if the graph is sparse, e is magnitude V and it gets drowned out, it's just v squared. And if the graph is dense, e is magnitude v squared, which is the same term since this is just a V squared algorithm. Right? Now, we could potentially improve this by using not a queue. Because we're not just doing a level order traversal. Here, we need to find the minimum each time and finding the minimum each time sort of screams priority queue is somebody had suggested earlier in the chat. So we could do this with a priority queue. Now let's go look at the code for a minute and see what we would have to update. So the way we would do this is that we would put all of these vertices into a priority queue. Now, initially, all of these vertices have a distance of infinity. And we're going to prioritize them by their distance. Because while I mean, that's what we need to do, because we want the smallest distance each time. Now I should note, we really ought to set this to zero first. And so then we're going to build the heap. Out of all of these vertices, all of them are infinity except for the start, which is going to be zero, which means the zero will be the root of the tree that gets created in that priority queue. And everything else will be in some arbitrary order after

DS Lecture 21 Page 7 of 22 Transcribed by <https://otter.ai>

because they're all the same. They're all infinity. Now, since they're prioritized by the distance, the Delete min operation finding the smallest unknown Distance vertex is a matter of deleting min, which gives us a better cost for this line of code. Now, building the priority queue doesn't really change this cost because we just use a built in. So that's just magnitude v . so that that goes away. But this highlighted line here now changes from being magnitude v to being $\log$ of magnitude V , because our delete min is a $\log$ cost. Does everybody good with that? So this outer loop happens magnitude v times. But now, this contributes a $\log V$ cost. So instead of being v squared, here, we've got $v \log V$. Which so far looks like it's a slight improvement. But there is a penalty to this. And the second change happens here. All right. So previously, we were just setting $W \cdot \text{dest}$ equal to $V \cdot \text{disk} + c \cdot VW$. And that was the end of the game. Why isn't this isn't it as simple as that? If I'm using a priority queue? Anybody? Anybody?

26:27

Go the top half, I guess, right? is simpler. Get the top half is sort of the simpler for.

26:37

Somebody asked, What is CB_w ? That's simply the cost to travel from V to W directly. So the distance between just those two vertices, anonymous questioner, does that answer your question? Cool. Well not update the position of the node, um, the position of the node and what you're on the right track? Ah, we cannot change the priority of the node. Yeah. So there is this issue, that the priority queue order is now wrong, because the priority queue was originally structured based on the priorities of infinity. And now we've changed something. But we talked about this briefly, at the end of our discussion of priority queues, there was a decrease key operation in an increased key operation, which means if we can jump to the location directly in the priority queue, where that thing is being stored, you can make that work with a hash table. If you were really being clever, then what would I need to do to get that node in the right position in the priority queue? Once I've decreased? its key? Which direction? Does it go in the tree? Anybody?

27:58

If you decrease its priority, it has to move towards the what has to move towards the root, which means you're percolating it up. Does that make sense? Yeah, somewhat less friendly. percolating it up. What's the worst case cost of a percolate up? No, it's not linear. It's the log of the size of the priority queue. Because the priority queue, if you're using a binary heap is going to be height log event. And the problem is you're gonna have to

DS Lecture 21 Page 8 of 22 Transcribed by <https://otter.ai>

move all the way up the height. So it's not actually log of n , because our priority queue has magnitude v options. So the decrease, sorry, magnitude v entry. So the decrease operation that I've highlighted here, actually costs the log of magnitude v . Agreed. All right. And so here we go. Previously, we said this little highlighted piece of code happens exactly magnitude eight times. And that still happens magnitude at times. But man, oh, no, it's not the mantissa. In the smaller. In the simpler version, all of these operations were constant time. This is just an assignment. In this, this middle operation is no longer constant time. It's it's log V . So that cost that factors out is no longer just magnitude D , its magnitude e times the log of magnitude V , that we had before. So it's actually $e \log V$. So we end up with a total cost of magnitude $v \log V$, plus magnitude $e \log V$, is everybody following where I got that from? All right. So let's see. But we don't need to put all the vertices in the queue in the beginning, right? We're getting QB adjacency after for one vertex is marked is known.

30:18

Sure, but that doesn't change the fact that they might, they might have their priorities changed later. So it does not change the analysis. There's no way you're going to make this better than this. Liliana says, Can you repeat why he is multiplied by log V ? Because this highlighted chunk of code, you see the highlighted piece? That's what we're saying happens magnitude 11 times whatever the cost of this highlighted thing is, happens magnitude each time. In the simplistic implementation, this part? In the simplistic implementation, this part costs constant time. So it's e times one. But now this decreased cost log V . So it's $e \log V$. lilyana. Does that make sense? Cool. Now, somebody messaged me about sparse. And so here's the kicker. If the graph is sparse,

31:29

this he just becomes magnitude V , right? So you got $v \log V$ plus $V \log V$. So this is big O of magnitude V , log magnitude v . So in a sparse graph, $V \log V$ is my cost, which is better than the V squared I had before. So using the priority queue in a sparse graph actually does give me a better cost. But in a dense graph, magnitude he is what? They right? magnitude v squared. And so that is going to be magnitude v squared log V , which will drown out the magnitude v cost, and so much in the dense graph, this becomes magnitude v squared log V . Which, oh, that's actually worse than the simple implementation. So this one is not as straightforward as the unweighted one where if you use the queue in the unweighted one, it's at least the same cost as the simple version possibly better. If you use a priority queue and Dykstra, you've actually made things worse if you have a dense graph, and it's only better if you have a sparse graph. Okay. All right.

DS Lecture 21 Page 9 of 22 Transcribed by <https://otter.ai>

I'm asking you to do the magnitude v squared implementation on the homework. Alright, any questions on Dykstra or can I move on to the next bit?

33:14

Let me switch the camera. Does this mean it's not a good algorithm for pathfinding?

33:26

It's fine, it's fine for pathfinding it's actually quite good for pathfinding. But it depends on what the network you're trying to find your paths on. is gonna look like I'm so the runtime for dense Hold on, let me actually get this set up the way I want. I actually asked Siri that question. She couldn't figure it out. Hold on. Alright. So does that answer your question?

34:09

All right. So um,

34:15

yeah, that's actually usually what I do is that I just go with the simple version, unless I'm absolutely certain when I've got a sparse. That's that's my approach. Alright. So that wraps us up for Dykstra salvos. I want to move back now and I want to talk about topological sorting. Which you should have read and work with a little bit on your homework already.

34:53

I'm going to draw a graph here. And you can think about what maybe it represents

35:42

IP addresses No, I guess I haven't thought about it that way. These are classes, right? This is this is the course number we're currently in. This is the course number you probably took just beforehand. This is the course number, the course you were probably supposed to be taking at the same time as this. But the cap on it was probably lower. So you're probably waiting till next semester to take. This is the advanced programming class you're all going

DS Lecture 21 Page 10 of 22 Transcribed by <https://otter.ai>

to take next semester and some of you are unnecessarily terrified for but it's very straightforward. That's the, but you just need to set the time aside for it. That's the CS theory course 1007. I'm simplifying things. That is our programming languages and translators course, which you really want to have both CS theory and advanced programming for and. And that's what is that that's operating systems, which is sort of the spiritual successor to 3157. And maybe we'll add a little more to this. I don't remember the course numbers in the math department. So I'm just going to call this guy calc one. Which actually we would prefer you took before discrete math. And then calc two is a prerequisite for sorry, calc one is a prerequisite for calc two. And depending on who you ask calc two is a prerequisite for calc three. Math departments weird. They say you have to take out two before calc three unless you're an econ major, because somehow econ majors automatically know calc two. Don't understand. Anyway. So what we're gonna do here is we're gonna discuss this graph. And we already understand that this is a graph of courses, and what these arrows are actually representing our prerequisites. Right? So the idea is that this is like a dependency graph. Um, the inbound arrows are things that you depend on, right? Things that you must take, before you take that. So topological sorting is actually it's not a path through the graph at all. It's not some sort of a traversal through the graph, it is simply an ordering of the vertices. And it's done in such a way that if you're a vertex V , you're guaranteed that all the things that you depend on, will appear in that topological ordering before you. So in terms of this course, metaphor we're going with here, the way to think about it is that imagine you can only take one class a semester, which might actually be the reality for some of you, depending on whether or not you're working. But imagine you can only take one class a semester, a topological ordering of this graph would give you a legal order in which to take the classes one at a time, such that you always satisfy the prerequisites. That's what this is doing. Okay? gives you a legal order, such that you always satisfy the prerequisites. And there are clearly multiple legal orders. Doesn't matter if you take panels for first or calc one first. They don't depend on each other. So you can shuffle that order up quite a bit. Does that make sense? All right. So what we're going to do is we're going to add another property to these vertices, which we're going to call the in degree as in, in incoming degree, incoming degree, like you graduate, and then from a very long distance, they curl the diploma actually, which actually sort of sounds like exactly what they're doing this year. I'm sorry, that was a terrible joke. Ah, No, actually, it might have been a way to on the money joke. I'm sorry, I'm a little punchy today. So forgive me if I say weird things. If you came to my office hours today, you probably experienced that a little more directly than what you're seeing here.

Alright, so, um, our in degree is going to be a count of the number of inbound edges to a vertex. Or in other words, in terms of these dependencies, it's how many prerequisites do you have? So what's the integrys? Use me for 1004?

DS Lecture 21 Page 11 of 22 Transcribed by <https://otter.ai>

40:37

Zero. And so is the degree for calc one. What's the end degree for data structures? This 3134 over here? One, what's the end degree for 3203? That's discrete math.

40:53

Two, because it depends on two courses. What's the end degree for this 30 to 61 course.

41:05

It's just one, there's only one inbound edge. But you might notice that there are actually three courses that will ultimately need to be taken before it. Because to actually legally take it's one inbound dependency, you have to satisfy that things dependencies first to, alright, and I can fill this out, calc one is an integral of one, I should probably put that closer to the arrow, three has an end degree of 14115 has an end degree of 24118 has an integral of one, that's one alright, we fill the in degrees out. This is an a cyclic directed graph. I should say that another way this is a directed a cyclic graph a DAG what would happen if this dependency graph were sick? There would be no end. All right. Now it also feels a little bit on the nose, right? You wouldn't be able to come up with an ordering. Because there would be a cycle. So there would be no way of satisfying the prerequisites. So our topological sorting algorithms will fail. But as it turns out, that's actually an interesting way to detect the cycle. So you can actually use your topological sorting algorithm to see whether or not there's a cycle because if it fails, there has to be a cycle. Okay. All right. So let's, let's sort of discuss just on paper how we would do this. So we need to pick a starting vertex, what is the quality of a starting vertex wise, what kind of vertices Can I write down is my next value in my topological ordering? What vertices anything with an end degree of zero. So I can start with either 1004, calc one, let's say I start with 1004. So I'm going to write it down. When we actually look at the pseudocode. For this, what we're going to discover is that they just sort of assign a number to them, like that'll get number one, the starting number is zero. And the next thing you take will get number one, and so on. Now, now that I've taken this class, I need to update my graph. What should I do to update my graph to reflect what I can now take? Anybody. So you, we don't necessarily have to delete the vertices. But the idea of getting rid of the edges makes a lot of sense, a couple of you got it in there. The idea is that the energy degrees of the things that dependent on 1004 can be decremented, because now there is one fewer prerequisite to satisfy for them. Now in terms of code, we're going to have an adjacency list for this graph, just like we do with all our other graphs. So we iterate through our adjacency list list and decrement, the end degrees of everything that's adjacent to us. So

DS Lecture 21 Page 12 of 22 Transcribed by <https://otter.ai>

we'll have this one tune in degree of zero because we failed. It's one prerequisite and this one will have it's in degree updated to one. Okay, next step. What do we do? Where do we go next?

44:50

Good. Again, we look for things within degrees of zero. Now there are new things potentially of in degree zero. So I could take each 3134 or calc one doesn't matter. Let's say I take 3134, I mark that one is visited, so I don't come back to him again. And then I update its adjacency list by decrement in the in degree of those adjacent to it. And then I, again, pick a currently unvisited node that has an integral of zero. So I could pick either this one or this one, I pick this one. And then maybe I'll pick, oh, I have to update his adjacency lists, that one goes to zero, that one goes from two to one. Right? So the topological ordering is definitely not going to be unique. Well, okay. In this case, it's definitely not unique. Because there are a lot of ways I could have done that. Now, you

could construct the graph, like if something's very linear, right? First you take this, then you take this, then you take this, you couldn't construct the graph, or there won't ever be multiple vertices on visited in degree zero at the same time. And in that case, it's unique. But for a lot of graphs, this one very much included. It is clearly not. Okay. All right. So maybe now I'll take 4118 just keep on going down the line. But now I'm going to have to backtrack. And the after taking 4118 and going through its adjacency list, which is empty. I'm down, only one thing left. I've been putting calculus off for too long. Okay, so I have to come back to that one. Putting calculus off for too long I used to advise the seniors man was always sort of disturbing when you had seniors who are trying to finish calc three right before they graduate. Don't don't don't do that if you can avoid it. Anyway. Yes, at any point, I could have gone to calc one I just happened to have chosen not. Does that make sense? Cool. All right. So we took calc one, we updated adjacency lists. Now we've unlocked here and we've unlocked here. So let's take calc two. But we could have taken that one. We update its adjacency list. Let's take out three, nothing to update there. And now I've sort of closed out my options. This is the only thing left was zero. Updated adjacency lists only one thing left with zero. Update, its adjacency list. And now this one is the only one left at all. And it thankfully has an integral of zero. And we have assigned a position in our ordering to all vertices. Good. And that's a top a lot that is a valid topological sort of these vertices based on the dependency edges. But as you can see, there were many valid ones. Any questions on that? Okay, let's do a quick run through of pseudocode. for calculating this, this will look very familiar. There's a very standard pattern for this. So this will look kind of similar to the pattern we did for the unweighted shortest paths. So let me switch my camera and my share around, I'm going to leave this open. So if you want to do side by side view, you can refer back to that if you want.

DS Lecture 21 Page 13 of 22 Transcribed by <https://otter.ai>

48:58

And I want this I want to move this closer, I can see it a little bit bigger. Alright. So now

49:13

we're looking at a simplistic implementation of this topological sorting. So let's discuss what we've got. First of all, we're going to have it throwing this cycle found exception. So basically, if it ever reaches a point where it hasn't satisfied all the vertices, but there are no vertices left in degree zero, we know there's a cycle. So we can just throw the cycle found exception and then it's done. Alright, if that's not the case, well, first of all, let's talk about our vertex objects. Our vertex objects are rather similar to the vertex objects that we had in the previous examples. One thing, they're augmented with an in degree field, and we're assuming that the end degrees are set up correctly in the graph for the year. Before we start, so we assume the graph is all set up, all the links are made, all the in degrees are set. Now, in addition to the integration, we have a field called top num, which is going to just be the position in the output sorted array where this vertex appears. So whatever I would write first would get set equal to top num, zero, whatever, what I would write second, would get set equal to top number one, and so on. So that's our initial setup, our outer loop is going to do a count, because we're going to have to visit every vertex exactly once, right, and so we have a counter that goes from zero, while it's less than the number of vertices, which is our magnitude v. So that outer loop happens magnitude v times. And then we're going to have this this is very pseudocode, we're going to have this vertex V equals find new vertex of in degree zero. So we're going to be looping through our list of vertices, looking for a vertex that has not been visited yet. Looking for a vertex that has not been visited yet, that has an in degree of zero. Now, we're going to assume that this is a very simple thing, we're going to assume it's just going to iterate through the list each time. And so if it's going to iterate through the list, each time looking for a new vertex of in degree zero, what's the worst case cost of that highlighted line of code?

51:54

this magnitude, sorry, not and what is in its magnitude v . Right? So we now have kind of a beginning of our cost, right? This outer loop happens magnitude v times and then this highlighted line plus magnitude v . So right there, that's magnitude v squared cost. Now, if it's unable to find a new vertex of in degree zero, we're gonna go ahead and throw a new cycle found exception, all right, because that just means there's a cycle because we haven't gotten all of them yet. Whatever one we selected, we give it the next number. So that's our counter, that's its position in the output. And then we have to go through each

DS Lecture 21 Page 14 of 22 Transcribed by <https://otter.ai>

vertex w adjacent to V , we have to update its adjacency list, decorating each of them weight with their degrees by one. Now, this loop, it's the same analysis that we had in all the others, right, you're going to do this once for each vertex, you're going to go through the entire adjacency list of every vertex, which means the guts of this loop happen exactly magnitude e times same analysis is always so in the simple implementation that we have here. It's going to be big O of magnitude v squared, plus magnitude E , which is really just big O of magnitude v squared, whether the thing is sparse or dense. Does everybody good with this? Okay. Now, just like we did with the unweighted shortest paths, we can improve on this by using a queue. So let's look at that version here. In this variety, what we're going to do is we're going to say that the queue will always hold the unvisited vertices of in degrees zero. So the moment of vertex is in degree goes to zero, we add it to the queue. And then we know that if we dq we're going to get something of in degree zero. All right. So we create a queue. This is not a priority queue. This is a basic queue. Oh, sorry, somebody asked why the other one was squared. And that was just because the way we found a new vertex of degree zero was simply looping through the entire list looking for one, so it's just like a nested loop. Does that answer your question? Cool. All right, so this is just a plain old Q . The counter starts at zero. We're going to go through each vertex and look for whichever ones that at the get go have an end degree of zero, and we add them To the queue. And now our loop changes a little bit, what our loop will do is while the queue is not empty, all right, the moment the queue is empty, it means we've got no more vertices in degree zero. And so we stop. So the first thing we have to do in our loop, is get a new vertex in degree zero, which in this case is just dq, what is our big O cost of our dq operation?

55:32

Basic queue implementation, what's the big O cost? It's constant. So this is a constant operation, which is a big improvement. We then set the topological number for V , this is a typo in the textbook, this should be counter plus plus, because if you looked at the previous example, you may recall set the first one to zero, not one. So this should be counter plus plus, so we assign it its top number. And then we have to go through each vertex that is adjacent to V , we're going to decrement B in degrees of each vertex adjacent to V . And if the results of that decrement is equal to zero, we're going to add it to the queue. And that's it, then we go around, and we dq again, to see what our next vertex of degree zero is, and we proceed until the queue is empty. Now when the queue is empty, it means one of two things that has happened, it means either we've assigned a top number to all of them, and we're done. Or we've got a cycle, meaning we've run out of things to assign top numbers to, even though we haven't actually assigned them all yet. And so if the counter hasn't yet reached the number of vertices, then we throw a new cycle found exception. If it has reached the number of vertices, then we're done. So hold

DS Lecture 21 Page 15 of 22 Transcribed by <https://otter.ai>

up a second, does that mean a queue adds whatever vertex it sees first? Yes, it adds them as it finds them. But it doesn't really matter. It only adds them when they reach zero. So as soon as something reaches zero, it gets tacked on to the queue. omer Omer says, should we add adding the vertices to me? So slow down? I can't keep up? Shouldn't we be adding vertices we dq to a list that keeps track of their order? The order is kept tracking the top

number, you can reconstruct that later. Okay, the next one, why would use the previous code of using a queue has a better cost? Just like with single source shortest paths, we actually probably wouldn't use the previous one. We probably only use this one. We'll analyze it in a second. And we'll see why. And the last one is why do we assign $v \cdot \text{top num}$ to counter plus plus. So remember, top number is keeping track of its position in the output ordering. So this is the output ordering. top number corresponds to its index in it. So that has top number zero, that has top number one that has top number two. And so that's how we're keeping track of that. So setting $v \cdot \text{top numbers}$ will encounter test sets it equal to the one we're currently at. It's a choice. Either way works, okay. All right. So the last thing we need to do here is analyze this. So if we're going to use a queue,

58:46

assuming all numbers get assigned a top number, this outer loop will happen magnitude v times every operation we have here are all constant time operations. So it's no longer v squared. And then this is the same analysis, we've always been doing each vertex, we go through its adjacency list once. So this part, which happens to cost constant time, happens exactly magnitude e times across all pieces. So this ends up being magnitude V , plus magnitude E . And so now we've improved from magnitude v squared d , plus d to magnitude v plus E . And so at this point, we have to sort of differentiate between sparse and dense in a sparse graph. magnitude, he is just magnitude v . So in a sparse graph, this is actually linear in the number of vertices. This is fantastic in a sparse graph. In a dense graph, magnitude he is just v squared. So we If not actually improved anything with a dense graph over the simple implementation, but we haven't done worse either. So we usually choose to do this, because it's going to be either the same or better. Unlike deicers algorithm, it's not going to get worse. Okay. All right. Any questions on topological sort? in particular? Does anybody have any questions on topological sort as it applies to the homework assignment? I decided to spend time in this here because I think some people were getting a little tripped up in the homework on that.

1:00:38

We're good.

DS Lecture 21 Page 16 of 22 Transcribed by <https://otter.ai>

1:00:41

Okay. Let me switch the cameras and the chairs around a little bit.

1:00:58

Okay.

1:01:01

Let me just get one note for this open.

1:01:09

Alright.

1:01:15

You can't assume anything, you you'd have that that's what the question is asking is what would you do? Um, alright, so the second to last topic for the entire course. And the last topic for today is going to be what are known as minimum spanning trees. So a minimum spanning tree is a property that we would take on a weighted undirected graph. So let me jot down an example of a weighted undirected graph that will work with

1:02:25

and so I need this to be weighted. Alright, so we have a graph. The tree is really just an a cyclic graph. Okay. So a spanning tree, just the for those last two pieces. a spanning tree is a subset of this graph. Meaning we're only selecting vertices and edges that appear in this graph. And it is a tree meaning the subset of vertices and edges we have must be a cyclical and it is spanning, meaning that it must reach all the vertices. So the spanning tree of this graph is going to be all of the vertices and a subset of edges that constitute a

tree. And so if I want a tree that consists of seven vertices, how many edges Am I going to have to include into that tree? Anybody there's only one way, one number of edges you can include, to keep it a tree to keep they keep it a cyclical edges. So six, right. If you think about a tree, you've got a route, which has no edge in bound, and then all the other nodes have exactly one inbound edge on them. So if you have a tree, your tree will consist of magnitude V , that's the number of nodes minus one edges. So I need the magnitude v

DS Lecture 21 Page 17 of 22 Transcribed by <https://otter.ai>

minus one edges subset of this set of edges that gives me a tree. And then the last component of it is minimum. There are many spanning trees of this thing. But we want the smallest spanning tree, the one that has the smallest total sum of the edge weights that we include. Now, a minimum spanning tree is not necessarily unique. There could be multiple spanning trees, that happened to have the same, excuse me, that happened to have the same cost. And that cost happens to be minimum. So let me just show you what the spanning tree of this one is, we'll talk about how we got that, maybe that'll be probably the first third of Wednesday's lecture. But here's what the spanning tree for this would actually look like. We're going to highlight the edges that are going to be included. All right. So all the vertices are connected to the resulting tree. Right, so it is spanning, there are no cycles in the sort of highlighted edges that I selected. And it turns out, this is the smallest sum of weights, you can get to do this. Alright. Now before we move on, we should talk a little bit about physically what's the point of a minimum spanning tree. And the example that I like to give is, I like to give an example of Unfortunately, the example I like to give is maybe not the most relevant anymore, because we have Wi Fi. But let's imagine for a moment that you wanted to have a hard wired network connection in every room in your home. Right? Now, imagine you wanted to do that. Maybe also imagine you live in a home that has more than one room in it. But look, before I moved into the house that I'm in, my wife and I lived in a studio apartment in Greenwich Village that had two rooms, the room, and then the bathroom, which until we had to take it off the need the bathroom at least had a door on it.

1:07:30

There's a side amusing story about that. But I'll tell you after the lecture is over, but I'm not right, the second. But let's say you moved to the suburbs like I did, and you got multiple rooms. All right. And let's say we want to wire them up for network. Now, here's the thing, we don't need every room connected to every other room. We don't even care if the path of wires that flows through to get from the downstairs bathroom to the upstairs bedroom is optimal. All we care about is that all the rooms are connected somehow into this network. So a spanning tree makes sense, right? There. Why would we want to have multiple connections to each room, we just have one connection out that ties it into the whole thing. Right? Maybe we don't need redundancy for this. Now, running cables, frankly, is expensive, right? Either you have to run it through the walls, or you have to lay some kind of conduit down either on the outside or on the inside. The cables themselves if you want fiber can be expensive. So you really want to minimize the costs of laying these cables out. So a minimum spanning tree actually makes sense for something to calculate, right? You can imagine each room as a vertex. And each edge in the graph is a potential path to run your cable from one room to another room. And ultimately, you want a spanning tree, one that visits every room, you don't want cycles, because you don't need

DS Lecture 21 Page 18 of 22 Transcribed by <https://otter.ai>

the cost for the redundancy. And you want to minimize the cost. So you want to sort of to be the shortest way possible. And this is assuming that the distance traveled is directly proportional to the cost. That might not always be true, but we can assume that. Does everybody get what I'm talking about in terms of a minimum spanning tree being appropriate for that? So this is a physical example of where you might want to do something like that. And okay, fine. If we don't nobody wires their computers anymore.

Everybody uses Wi Fi. Imagine electrical power to your lights, or something like that. It's the same idea and we really don't have good ways to wirelessly transmit electricity sort of safely within the home. So you have to physically run wires. Does everybody okay with the concept here? Yes. All right. Now the question is, how are we going to calculate these minimum spanning trees? And the answer is we're going to do it in a greedy manner, just like we did with Dykstra, we're going to do what appears to be best at each step. All right. Now, there are two different ways of viewing the greedy approach to this. The first way to view it is, you want to find the cheapest way to incorporate each vertex. So you start at some vertex, and that can be the root of your tree, it doesn't matter what vertex you start out, if you have an acyclic graph, you can kind of treat any vertex as the root of it. All right. And so you can start at some vertex, that's the root of your tree. And now your goal is to incorporate other vertices in as cheaply as possible. And the way one does this, if you're going to incorporate vertices in is through something called prims algorithm. I don't remember if it's one M or two, let me check. It's one prims algorithm. And so prims algorithm is identical to tasters algorithm with one change. We don't care about shortest paths, we only care about the cost of incorporating the vertex. So only we only care about the incremental cost, just the CW , not the V dot display CV . So if you Dykstra his algorithm, and you replace the update rule, dated based not on the total distance traveled, but only on the incremental distance. I think my connection may be a little bit unstable. So forgive me if I'm jumping around here. All right. Am I coming through okay for everybody?

1:12:33

Okay, cool. So if you change that update rule, you get prims algorithm. And so the path field just gives you the edge, you can read the edges off, it's identical to Dykstra just with the change in the update rule. So that's prims algorithm. The other way that you can think about creating this in sort of a greedy fashion, is what's called cross skulls algorithm. And in cross Google's algorithm, what you're doing is you ignore the vertices for the most part, but instead, you incorporate edges in in a greedy fashion. And so you know, that if there are magnitude v vertices, there must be exactly magnitude v minus one edges. And so you go through and you find the smallest edge. And as long as it doesn't create a cycle, you incorporate it in. Then you go through all the edges that are left, find the smallest, and as long as it doesn't create a cycle and incorporates it in, and you keep doing that, until you incorporate in magnitude v minus one edges without creating a cycle. And that's crystals algorithm. Now, the trick with that, somebody asked how you spelled it, asking me for spelling is a dicey proposition. But it's, you can tell I'm scrolling down to make sure KRUSK a I preschools algorithm. Now to do this, we're gonna need a way to detect whether or not we're creating a cycle that involves something known as disjoint sets, which is actually chapter eight, which you only need to read like the first three pages of and so I'll talk about how we're going to do crystals algorithm for this and the first bit of Wednesday's lecture. Then we're going to talk about P versus NP and the traveling salesperson problem, and then we're going to do a brief review for the final. The real review for the final will be at 5pm on Friday. Okay. All right, folks, we're almost there. Have a good one and I will see you on Wednesday.

1:14:56

Dr. Blair, do you have time for a quick question? No, it has to be publicly recorded. But go ahead. Okay, if this is too close to the homework, that's okay.

1:15:09

If you don't want to answer it, but I just wanted to check my understanding of quicksort in general. So the whole point is that if you do quicksort, and you do the median of three, and then you sorted, you get that ideal case scenario. Is that not quite? If you don't quite follow, so you have quicksort, and then you sort

1:15:36

of quicksort, and you do median of three, that picks the pivot on which you partition the entire array. But that gives you the perfect pivot, right? No, no, it doesn't. The perfect pivot is the median of the entire list. But you can't get the perfect pivot without first sorting it. So that's off the table. Although the median of three sort of avoids the worst case scenarios as much as it can.

1:16:09

Okay, so it makes you median of three,

1:16:12

DS Lecture 21 Page 20 of 22 Transcribed by <https://otter.ai>

you can never get the worst case scenario, we're close to it. It just makes the worst case scenario very, very unlikely. Which is why the average case stays and login.

1:16:25

And if you're talking average case, that tells you nothing about best. Worst case, we know the bigger worst cases a different situation entirely. Okay. All right. So even doing median of three, you still don't know if that median is going to be give you the ideal case. And you need the ideal case for that lower bound to be analog. Yeah.

1:16:51

Right. Now, if I tell you something about the ordering of the array before doing the median of three, then it's possible you might that's sort of the heart of the questions that I know you're getting at in terms of the homework.

1:17:06

So is it just possible or is it then guaranteed? If that's what the question is asking? You? Depends on the employer range. Okay. Okay. I understand. Thank you so much. This was very helpful. All right,

1:17:18

take it easy. Thank you. Anything else? For the pickup six question. If at some point, we would need to do a topological sort, should we rewrite the method in the book since we're using pseudocode, or

1:17:35

whatever algorithm you ultimately decide to use to solve this problem? If you're not changing it, if you're just running it on a graph, you don't have to rewrite it. And in reality, whatever algorithm you choose will need to be rewritten. The work comes from figuring out how to build the graph to run that algorithm on. So what you need to be describing really is how do you build the graph? And then which algorithm to run on the graph. But you don't have to provide the code or the pseudocode for the algorithm that you're actually going to run on that graph? Because it's just a vanilla run. Let the algorithm All right. Thank you. All right. Anything else? I've got another minute and then I'm going to

DS Lecture 21 Page 21 of 22 Transcribed by <https://otter.ai>

have to bulk because I got a meeting coming up.

1:18:32

We're good. Thank you.

1:18:35

Thank you. Take care off.

DS Lecture 21 Page 22 of 22 Transcribed by <https://otter.ai>

DS Lecture 22

Wed, 8/11 2:06PM 1:15:39

SUMMARY KEYWORDS

problems, vertices, algorithm, np complete problem, polynomial time, edge, exam, solve, np, chapter, tree, priority queue, set, cycle, question, find, identifier, minimum spanning tree, point, incorporate

00:03

Where I check for sizes they didn't have one for you I was gonna you know I usually wait until grades are really so before I come here and get getting but I did find a pineapple. I got it

00:46

I brought free on a Saturday morning. I put them to bed at North on Mills Avenue and listen to the engine.

01:14

My brother and house behind me and good things ahead. A girl named Kathy wants a little of my time. Six cylinders underneath the hood cracking and picking through the engine wine. I am going to make it

01:38

heaven kills me. I played video games and I probably become Hayes I was 17 years young. Bear in mind punching machines. The taste of scotch match on my

02:25

DS Lecture 22 Page 1 of 19 Transcribed by <https://otter.ai>

family we hung out trading swigs from a bottle clean loggia guy holding hands twin high maintenance machines I'm gonna make it through this year kills me I am gonna make it through this year. If it kills me man the California dogs I can feel the alcohol inside of me home pictures on my father's face. Ready all the bad things that my downshifted as I pull into the driveway. The motor screaming out stuck in second gear the scene ends badly as you might imagine. a cab okay through this year kills me kills me.

04:14

Alright, let's get started. Um Welcome to our last official class. We are almost done.

04:27

Almost. You have a homework five, which programming wise anyway is quite short. That is due tomorrow at 11:59pm. You are allowed to use any late hours you may have remaining on that if you wish. We are hopefully going so you're not going to be able to get all of homework four back before homework five is do homework for programming was due close enough that they're not going to be able to get that done. homework for written I'm sort of hoping we'll get done might not be till Friday. And my plan or my hope is that homework for grading will be done. Prior to the exams, homework five grading, you will get your homework five scores back the weekend after the exams, because we're going to need to pause for a moment for my ta to take their exams, your final exams, we are grading on Monday, April 26. grades are at least for seniors, due on the 28th of April. And I'm hoping to be able to just get all my grades in on the 28th. That's the plan. If history is any guide, I will miss that wildly. But I'm trying seniors, though, are definitely getting their grades. It's probably on the 28th, possibly on the 27th. But no later than that. And there are a few seniors. Um, okay, office hours, we are running office hours for th through Friday, there was a little bit of shifting around of the Friday hours to accommodate one of my ta is somehow managed to have a final exam on Friday. Not sure how that worked, but I have a feeling he's the only one who has a final exam on Friday either. Not my fault. How many of you have final exams on Friday? Why? Why? Why? Well, tomorrow, I am guessing that's an in class final, right? Yeah, that homework five solutions will be made live right after the late deadline. But yes, they will be available. Um, okay, so let's plan. So you're gonna have a review session at 5pm. On Friday, it will be recorded, it is the same link as the class link. There is a review sheet, the first half of which is very similar to the review

sheet that you got before your, your midterm. That review sheet is in courseworks. Now, it is called final dash review dot PDF, and I'll send out an email about it. Um, I don't think we're going through that review sheet today. I think we're going to go through it in the review session, formally, but we'll see how today goes. Today, I need to wrap up minimum

DS Lecture 22 Page 2 of 19 Transcribed by <https://otter.ai>

spanning trees. We talked about what they were we mentioned prims algorithm for solving them. And now I want to talk about Chronicle's algorithm, and very briefly, the concept of disjoint sets, which we're going to use to solve Chronicles algorithm. Then, we're going to do the section in chapter nine on P versus NP problems. And in relation to that, we're going to talk about the traveling salesperson problem, which you might remember from early instead of work. And that's our plan for today. Any questions before I dive into it? We're good. All right. Let's do this. Um, so let's look at our graph for the minimum spanning tree that we're going to do. Now what you should recall is that our algorithms are greedy algorithms, right? You do what appears best at each step. Now, the greedy algorithm prims algorithm, which was just a modification of dices algorithm that we spoke about last time, was an algorithm that in the greedy step, we incorporated each vertex in one at a time. In crystals algorithm, we're going to incorporate each edge in one at a time, and the vertices will come with it. Somebody has their hands up, what's up?

09:18

Okay, I have one question. Can you start at V four? Or do you have to begin from the view? Like, is it like entering a cage? Or can you started before you need?

09:31

So for the kruskal Xh algorithm? More importantly, sorry? Yeah, I get I get the question. It's a good question for crystals algorithm and prims algorithm for calculating the minimum spanning tree. It doesn't actually matter what vertex you start at all.

09:51

That's all I know. Just from a visual standpoint, it kind of looks like it's kind of, you know, not approachable from the outside. That's why

10:01

Yeah, but this algorithm will work if you start anywhere. So you started v4 three dimensional, I don't see your, you know, it's not. Um, so in terms of, I guess in terms of starting points for prims algorithm where you're incorporating one vertex at a time starting point doesn't matter. But this is cross skulls algorithm. And in cross Google's algorithm, our greedy step is to incorporate the shortest vertex sorry, not the shortest vertex, the shortest edge that we can each time. Alright, now remember, we have

DS Lecture 22 Page 3 of 19 Transcribed by <https://otter.ai>

supposed to be a tree. So if there are magnitude v vertices, there are magnitude v minus one edges in the tree. So our goal is to accept magnitude v minus one edges. And our greedy step is we will attempt to incorporate the shortest edge first, then we will attempt to incorporate the second shortest edge and so on, until we have accepted the exact number of vertices in this particular graph, it's six vertices that we need. So we're doing this edge by edge, not vertex by vertex. All right. Now, because we want the smallest one each time, and because the edges already have weights associated with them, what data structure seems most appropriate to get me in an efficient manner, the smallest one each time. Good a priority queue. So what we can do is we can take all of these edge objects weight, the more prioritize them by their cost their weight, and build heap on them get a nice priority queue, that'll be linear time to build a priority queue. Now that priority queue is on the order of the number of edges, its magnitude E in size. And so now, the algorithm proceeds by deleting min looking at that edge and seeing if we can incorporate it into the graph into the well the tree without causing a cycle. Because our tree very specifically, what makes it a tree is it is a sick look. And so when I do my initial delete min, on this priority queue, I've just built one of two edges could legitimately come

off initially, in the chat, what are the two possible edges, and you can name them by their endpoints that might come off first. So $v_6 v_7$, which is a weight of one, and $v_1 v_4$, which has a weight of one.

13:08

You know how the priority queue ultimately ordered them is kind of going to sorry, hold on one second, my internet is apparently unstable. are folks hearing me clearly at this point? Okay, so yeah, it's so it's $v_1 v_4$, or $v_6 v_7$ are the first ones that come can come off. Now, I'm just gonna assume $v_1 v_4$ comes off first. Either way, is fine. And so I look at that edge, this edge here, the $v_1 v_4$ edge. And so far, I have no edges in my tree. So incorporating that edge is definitely not going to cause a cycle, right? No cycle yet, just one edge. And then I just keep deleting men checking to make sure it doesn't cause the cycle, incorporating it if it doesn't, until I have the correct number of edges. So the next thing that comes off will be the other one of weight one which was the $v_6 v_7$ edge, and I can safely incorporate that edge in without causing any cycles either nothing's really connected yet. When I delete min next similar deal, two possible edges might come off next, what are my two possibilities? $v_1 v_2$ and $v_3 v_4$, both having weights of two. Let's assume $v_1 v_2$ comes off first. Incorporating that in doesn't seem to create any sort of cycle so we're good there. we delete Min again. So we'll get the $v_3 v_4$ this time and incorporating that one in also will not create a cycle At this point, we've gotten four of our six edges in already. Right? So we're doing really well so far. So I delete min again, which edge comes off next? Anybody? So $v_2 v_4$ with a weight of three comes off next, can I incorporate that edge in without creating a cycle? New, this one's going to

DS Lecture 22 Page 4 of 19 Transcribed by <https://otter.ai>

definitely cause a cycle. So I have to reject that edge. And so as you can tell, I'm not just going to pluck the six smallest edges off, some of them may not work for me. So I've rejected that edge, I still only have four edges, even though I've considered five of them. And then I delete min again, two things might pop off next, what are my two options?

16:03

A then cat, if I included this edge, wouldn't we have a cycle $v_1 v_2 v_4$? Yes, no. Cool. I didn't say how we knew that. This is one of those things, we look at it. And we as humans are great pattern matchers instantly see that. But the computer is going to have to figure out how that happens. And that's that's a, that's a little more complicated. We're going to talk about how it actually is going to detect that algorithmically afterwards. Alright, so it was suggested that $v_1 v_3$ is an option, and $v_4 v_7$ is an option. So let's say $v_1 v_3$ comes off next. And as somebody said, but it's bad, because it's going to create a cycle with the edges $v_1 v_4 v_3 v_4$, so I'm going to reject that edge also. Then I'll try the $v_4 v_7$, that's the next smallest one, that one will not cause a cycle. Okay, which edge comes next, I've got five edges Incorporated. So I only need one more, what edge would come off the pq next, like $v_3 v_6$. So $v_5 v_7$ has the weight of six $v_3 v_6$ has a weight of five, this one is smaller, that comes off the priority queue next, but can I accept it new that will create a cycle also, it's a longer cycle, not as immediately obvious to us because it involves going through an extra edge before it cycles back. But it still would create a cycle then the $v_7 v_5 v_3 v_6 v_7$ however you want to call edge would get delete mins. Next, that one would not cause a cycle.

18:21

You're popping the smallest ones offer so it's a delete min. Um, and this is the six that I've incorporated. And so we know that a tree of seven vertices must have six edges. So I stop. And I don't even need to consider these last two edges. Are we okay with that? All right. So this is great. I like this algorithm a lot. I think it's a very intuitive algorithm. The problem is that we need to have a way to detect whether or not incorporating an edge actually causes a cycle or not. And that requires another data structure. So let me switch sheets

here for a second. So this is actually coming from the first few pages of chapter eight disjoint sets. I think I mentioned in class. But I wouldn't mind it if you read the first three ish pages of disjoint sets to get a sense of what they do. But I'm going to tell you all you need to know about them for the purposes of this discussion. In general, our that would be a lovely induction proof that I could give you on an exam.

DS Lecture 22 Page 5 of 19 Transcribed by <https://otter.ai>

19:55

So you stop at six edges because you know that it tree Something in a cyclic graph that fully incorporates seven vertices can only have six edges. So you think of each edge as inbound to a vertex, and the route doesn't have an inbound edge. And so if there are seven vertices, one of them's the root, six of them would have inbound edges. Melissa, does that make sense?

20:25

All right. I don't think we did anything from the divide and conquer chapter. Alright, so disjoint sets, you start out with Well, a bunch of items. And each item is in their own set. And they're disjoint. So ones in a set by itself, twos in a set by itself, threes in a set by itself for us, and set by itself fives and step by itself. And now there are two operations that a disjoint set has. The first operation is what's known as the find operation. And so the find operation tells me, the set identifier for an item doesn't tell me what other items are in the set. There is, in fact, no easy way to get what either other items are in the set, it just tells me the unique identifier of that set of that item set. So if two items happen to be in the same set, and at the moment, none of them are. But if two items happen to be in the same set, they would both tell me the same set identifier. So if I want to see whether or not two items are in the same set, I would do a find on both, and then check to see if that find return the same thing. Are we good with that? Cool. And then the second operation is what's known as the union method. Sometimes Actually, it's called union find, rather than finding those union find. And so the union operation, it doesn't take two specific items. But it takes two set identifiers and merges the items in those two sets. So if one two were in separate sets, and I merge a union, the sets that contain one and two, then one and two would be in the same set. So you always union on a set identifier, not on an individual values, you have to first find on an object to see what it says is, and then union. So if I wanted to Union, the sets that contain the one in the two, which at this point, would just be making one two part of the same set, I would do it the following way, I would invoke the union method on the Find of one and the find of two. So that we'll find the set that one's part of buying the set that's two part of and merge them together into a single set. And so I will bother you with the details of how we might represent that. But it ends up looking a bit like a tree doesn't matter. You can choose which identifier, I happen to have chosen ones identifier by moving to down like this, as long as all the items of the two sets now return the same set identifier, we don't care what it is. So in this case, I kept the set identifier for one. And now if I do a find on two, it will give me ones set identifier, and then I do a find on one, it'll give me that same set identifier. So now, after this union finds on one and two, give me the same identifier. On Are we good with that? Cool. All right, let's union another pair of them. So let's do a union on the Find of three and the fine to five. Now it so

DS Lecture 22 Page 6 of 19 Transcribed by <https://otter.ai>

happens these two. Yeah, in the way I've written it, here's a yes, that's what it will end up doing. But I could have just I could have done it the other way around, and it would have been equally valid. Good. Cool. All right. So I'm going to Union three and five So now, three and five had the same set identifier, whatever threes was previously. Now I can't just union an individual item I have to do the find on it. So let's imagine then that I do the following. I say union the find on two and the find on five. All right?

25:40

I could if I wanted to, but Well, no, don't think about it that way. Think about it purely as

set identifiers. So even though one and three appear to be the top level of these two, you would still want to do the find on one in the find on to Hana. Is that okay? Sorry, the find on one and find on three. Noreen sorry, the person who asked privately who I just named My bad. Sorry about that is the first is the first parameter passed away is going to be the one the arrow is pointing towards. I mean, in my head, that's how I'm doing it. But it doesn't have to be that way. It's however you decide to code it. Good.

26:38

So don't think of this as a graph. Because in reality, I'm building a tree here, where the root of that tree is the set identifier. And so when I Union, the fine of two and the Find the five, I really unioning, these two whole sets. And so what I would end up doing might be something like this. Julia, do you have a question?

27:13

So you see, I've kind of built this tree up, where if you follow if you start at any given node, you can follow it up to the root of the tree, and that gives you the set identifier. So this is sort of the inverse of how we've stored every other tree in our lives. There's no child nodes, there are only parent nodes. And so in fact, the best kind of tree to have for these sets is a really flat tree, where they all point directly to the root. Because that way, you don't have to traverse too many to find your set identifier. But after I've done this, did my union on the Find of the two and the find of the five? Would union on find one and find five, if given the same result? Yes, because we're joining these two sets together, meaning the only thing that means now is that 213, and five will all return the same set identifier, if I call find on the that's the only thing. So if I give you the item five, I cannot tell you what items are also in the same set as it. I can only tell you what it said identifiers. But if I pairwise, give you two items, and you independently do a find on each of them. That will tell you whether or not they happen to have been from the same set. Are we okay with this. So

DS Lecture 22 Page 7 of 19 Transcribed by <https://otter.ai>

this structure is really clever. And there are some neat tricks that you can do with them that will flatten these trees out, the more operations you perform. So over time, the find operations and the union operations end up averaging out to about constant time. All right? Yes, four is in a set all by its lonesome. Okay. So now that we have this structure, this is actually a super convenient structure. If you're curious about reading the chapter on this, which you do not need to do beyond the first three pages. One of the other applications of this is actually for a maze finder, like amazed solver. So this you can do some really neat stuff with this. But we're we're going to use it here is we're going to use it to indicate whether or not vertices have a path between them. So what we're going to do, in addition to having our priority queue of all the edges, is we're going to create a disjoint set. were initially before I've incorporated any edges. Each item each vertex will be in its own disjoint set by itself. Now when we incorporate an edge, you look at the endpoints of those vertices. And now, those vertices have a path between them. But if there were other paths from other vertices to each of those endpoints, you can now get from things that were connected to one endpoint to things that were connected to the other endpoint via this new edge. So whenever you incorporate an edge, you do a find on the endpoints, and then you union those two sets, because now you have a path between all of them. And this also tells you how to reject an edge. If you try to incorporate an edge, where the set that contains the endpoints of the edges is the same, it means there already exists a path between them, and therefore we be adding a new path between them, and would be creating a cycle. Alright. Yep, that's exactly how we're going to avoid our cycles. Does anybody have any questions about how we're doing our cycle detection using our disjoint sets, and here,

31:22

these are really cool. Um, now, I'm not going to show you the pseudocode for this.

31:31

Very similar to a lot of things we've already seen. But we can sort of briefly come up with the cost for it, right? If we're assuming that on average, our unions and are fine. They're basically constant time, we can just kind of wave a wave the cost of our cycle detection. But the main thing is the priority queue. And all of the edges are in the priority queue initially. So each delete min instead of costing $\log$ of, instead of costing $\log$ of V is logged to V . Now the question is, how many delete mins might we end up having to perform? And the answer is we might actually have to delete all of them might be the last six edges that we accept. And so we'd have to do potentially, as many as magnitude 11 , delete mins. So my cost is actually magnitude $e \log e$. Now, just a quick note, in a dense graph, $\log$ a V is

DS Lecture 22 Page 8 of 19 Transcribed by <https://otter.ai>

$\log$ of v squared, right? But $\log$ of v squared is just two v . So this is really an $E \log V$ algorithm is the same cost as using a priority queue and Dijkstra is algorithm. And since Dijkstra is algorithm and prims. algorithm are the same, it has the same cost as doing prims algorithm if you're using a priority queue and both. Alright, any questions on this? Or can I move on to our next topic? Okay. So now we're going to do something a little bit more abstract, this will kind of close out the material for today, or for today, close out the new material for the semester really. Um, so I want to talk about the class of problems known as the problems P versus the class of problems known as NP . So these are problems that can be solved in polynomial time, that's an N . So something that can be solved in constant time $\log$ of n time n squared and cubed n to the fourth, $N \log N$, what have you, as long as it's not exponential, or factorial. These are problems that can be solved in polynomial time. And basically, all of the algorithms we did well, for the most part, except for maybe, like towers of Hanoi, um, for the most part, everything we've done has been stuff that can be solved in power. monomial time, so those problems are part of the set of problems P . Okay. But there's another set of problems known as NP . And these are problems that can be solved in what's known as non deterministic polynomial time. So non deterministic polynomial time, there are kind of two ways we can describe this. So non deterministic polynomial time says that the problem can be solved in polynomial time, if there is an Oracle, who tells you the right thing to do at each step. So, hold on a second. All right. So, if we have some all knowing Oracle that can tell you what to do at each step. And that allows you to solve the problem in polynomial time. That's non deterministic polynomial time. Now, Oracle, what the hell is that? So, maybe, let's phrase this in a slightly different way. Let's imagine that an Oracle gives you what it says is a solution to the problem.

36:32

But we wish to at the very least verify that the solution we've been given is correct. And so, what problems that are an NP fall into these are problems that not necessarily can be solved in polynomial time, but can be verified in polynomial time. So, if I give you a solution to it, can you check it in something that is an exponential or factorial or something horrible like that. Now, the problems p , which can just be solved from scratch in polynomial time, are definitely also part of NP , right? Because if an Oracle gives you a solution, you can verify the solution just by going through and solving it. And then comparing your answer to the answer you were given. Right. So P is at the very least a subset of NP . But there's a question. And the truth is there is no answer to this question yet. is p equal to NP ? Alright, what does that mean? What would it mean, if p is equal to NP , that would mean that P and NP are the same set. Which would mean that if we said this was true, that any algorithm that can be verified in polynomial time can also be solved in polynomial time. Oh, that's a pretty, pretty big stretch. But it hasn't been proven

DS Lecture 22 Page 9 of 19 Transcribed by <https://otter.ai>

one way or another. It is possible still hasn't been disproven, that problems that can be verified in polynomial time can also be solved in polynomial time. In all cases, it is possible. And if it turns out that that's true, that's actually kind of dangerous. Because

there are a lot of sort of hard problems like problems that should be an NP only. And we'll talk a little more about that in a second, like prime factorization, or something like that. But things like encryption depend on. But if suddenly, we discover there is a way to solve those problems in polynomial time. Also, we might have a little bit of a problem. We might also end up being extremely wealthy because there are other Well, not just from robbing people, because now you can get into their bank accounts. getting money myself ideas? No, not really. Um, man, that sounded terrible. But because there are a lot of really complex problems and things like logistics, and stuff like that. supervillain origin story now, I'm at best sidekick material, man. All right, moving on. So um, They're like seriously useful problems in like logistics and stuff like that, that if we could solve them very efficiently, that would be very useful. So then CAD, you can verify it by like saying, If I give you an answer, can you check to see if it's right, that's what we're saying. Right. So there's another set of problems, which are, we think, a subset of NP called NP complete problems.

40:45

I'm going to tell you off the bat, I am not going to teach you how to prove that something is NP complete, take algorithms at the 4000 level for that. All right. Um, but I will tell you roughly what these NP complete problems are. The NP complete problems are sort of call them the hardest. I'm spelling everything wrong here. hardest problems in NP. And so NP complete problems are problems. That it turns out that if you can find a solution or an algorithm to solve an NP complete problem, you are guaranteed to be able to convert the solution to solving the NP complete problem to a solution for any problem in NP. And that conversion would happen in polynomial time. Now that was proven long story beyond the scope of this 1015 minute lecture, right. But if you solve any of these problems in polynomial time, you have suddenly solved all of these problems in polynomial time. Not every problem in NP is necessarily NP complete. All right, but some of them this is the hardest ones are. Now, one of the ways we show that new problem is NP.

42:28

Oh, that transformable an existing problem? That is NP complete. So we keep for such problems. So if you ever happen to find a solution to an NP complete problem in polynomial time? Where did I Where did you lose me? I don't know what my last sentence was. You gotta tell me words that came out of my mouth just before I froze. So the last

DS Lecture 22 Page 10 of 19 Transcribed by <https://otter.ai>

minute is not useful. Time means nothing to me anymore. I've always sitting in the same seat. The lights don't change. It's, it is a very, very, it could be four in the morning for all I know, man. All right. Okay, how do we tell if a new problem is NP complete? Thank you. So you can find conversions between New NP complete problems and old NP complete problems to show that there one can be converted into the other? Does that make sense? So we keep adding new NP complete problems. Now, if we happen to find a solution to any of those NP complete problems in polynomial time, we've unlocked the world. We've proven then that P is equal to NP if we find such a thing. And that's potentially very valuable, but potentially very dangerous. Now, there are a lot of people a lot smarter than me that have a lot of opinions on this. And some people think that P is not equal to NP. And that's eventually what will be frozen, proven. Some people think that they are equal to each other. But in all likelihood, the conversion. You're a bit lost on what it means to convert an NP complete problem into another problem, basically showing that solving the two problems is equivalent. So that solve like the approach to solving one problem would also give you an approach to solve the other problems. Does that make sense? This time? I remembered anonymous poster. Yes, no. I think so. I don't think they're equal. But I'm not. I'm not one of the really smart people when it comes to these kinds of problems. All right. So some people think that they are equal but the conversion For going from a solution to an NP complete problem, to a solution to everything else will be so expensive, like n to the

50th, or something like that, which is polynomial time, but such a big order of polynomial that might not, I'm going to give an example of an NP complete problem in a minute. The problem is, I'm not going to prove exactly why it's NP complete. But we're going to give an example. Um, some people think that the conversion is going to be so expensive as to not be used, such that it won't actually matter. Now, let's talk about an example of a problem that is NP complete. And so a problem that's going to be NP complete is going to be a problem that as far as we know, does not have a way of being solved in polynomial time. To the best of our knowledge, the best we could do is either exponential or factorial. All right, something really bad. But because it's in NP is NP complete is part of NP, we should be able to if given a solution to it, at least validate that that solution is correct. Okay. So an example of an NP complete problem is the traveling salesperson problem.

46:37

Now I'm working under the assumption that essentially all of you have seen this problem before, right? In your 1004. Class, the TSP are traveling so no. Your guy wants to visit like a whole bunch of cities in the shortest amount of time, they probably said something like that there may have been a homework question about that on, like, maybe homework one or homework three of 1004. I, yeah, I've taught that class too. And I'm planning on putting that on my first problems that next semester, awesome. Okay, GPS tells you where you are, it doesn't tell you how to get to the next spot. All right. I have a person who asked

DS Lecture 22 Page 11 of 19 Transcribed by <https://otter.ai>

me a direct message question. The answer is, I don't have the slightest clue. I'm so sorry. Y'all don't even need to know what was asked. Um, alright, so the traveling salesperson problem, I'm going to phrase it a little differently from the way you heard it. Because the way you heard it was actually not, the NP complete version of the problem is a little different. So let's, let's try it out. So first of all, the traveling salesperson problem assumes you have a set of vertices, we usually call them cities, because you know, you're traveling from city to city to try this out sell stuff. But you have a set of vertices in a complete graph. So you have a complete graph. So what is a complete graph mean? A complete graph is a graph where every vertex is connected to every other vertex. So basically, if you're going from city to city, imagine you have a helicopter and you can fly direct from city to city. Okay. Alright. So, you have this complete graph, you can go directly from every city to every other city from every vertex, every other vertex, and there is a weight associated with each edge in that complete graph. Now, what you're being asked to find is, you're being asked to find a simple cycle

48:57

that visits all vertices. Now remember, a simple cycle is a cycle, where you have a path through vertices that does not repeat, you do not visit a vertex more than once, with the exception of the connection back to the starting point. So a simple path is a path where every vertex along that path is unique. A simple cycle is like a simple path, but it connects back to the starting point. So you want to find a in order in which you can visit all of the vertices without ever backtracking to a previous vertex. Does that make sense? Now, there could be many ways to do that. In fact, because it's a complete graph, there are n factorial ways to do that. All the different permutations of the vertices. Now you're being asked to find such a simple cycle, whose length and here's where it's going to differ is less than some arbitrary K . So I'm going to give you as part of the problem, some max length of the overall path. Now it's possible that such a simple cycle doesn't even exist. But if it does exist, you need to give me a simple cycle, whose total length adding up all the weights of the edges you traverse is less than k . Now, I'm not going to prove that this is an NP complete by showing a connection to a different NP complete problem. But I will just point out that there's really no easy way to do this, other than just going through all of the possible permutations, adding up the length the weights of all of the edges in each of those permutations, and then finding the cycle, just go through it linearly until you find

one that has a length less than k . Which means to solve this problem using this brute force technique, which is really all we know how to do, would cost order n factorial time. But the important thing is, this is supposed to be an NP complete problem, which means it's supposed to be a problem in NP. And a problem in NP is verifiable in polynomial time.

Right? If I give you a solution to this problem, I give you such a simple cycle that visits all vertices. And I say, Yep, there's your answer. Can you check that its length is less than k in

DS Lecture 22 Page 12 of 19 Transcribed by <https://otter.ai>

polynomial time? Yeah, trivially, you just go through and add up the lengths of all the edges. And you can move easily in linear time verify that the solution you've been given is correct. So solving this takes factorial time, or there's actually a slight optimization that allows it to take exponential time. But solving it is not polynomial to the best of our knowledge, but verifying it is. So it's part of the class NP and it's NP complete. Okay.

What was the version of this problem? That you actually learned? What was what's the difference between what I taught you here? And what you actually learned?

52:50

Maybe, right, the vert, the other version, is the optimization version. So you're finding the smallest. Now, in terms of solving it to the best of our knowledge,

53:24

the cost of solving this version to find the shortest path is the same as the cost of this one, right? You still have to go through and generate all n factorial permutations. But instead of searching through it for one that has a length less than k , you search through it for the finding the smallest one, right. So solving this one and solving this one is the same. Do we agree? Why is this one different, though, in terms of the class of problems that it's probably in? Can I verify that something is the smallest, easily? Well, okay, to be fair, there might be a polynomial solution to this, we haven't proven at one way or another. But if as I think there isn't a polynomial solution to this with the best case for the first algorithm be different. Not really, Well, okay, if you get lucky and the first few paths that you generate happened to be less than k , then I guess you're right. Best case cost, but average or worst case is the same. Okay? But this one, to actually verify that it's the smallest you have to calculate what the smallest is. So unless P is equal to NP, you probably can't verify this. As a result, this version of the problem may not even be in NP, meaning we can't really call it NP complete. Because NP complete implies it's part of NP. So I actually call this version even harder than an NP complete problem, because not only can it not be solved necessarily, in polynomial time, it can even be verified, I would call this NP hard. And so an NP hard problem would be a problem, where you could use a solution to it to solve an NP complete problem, but itself can't even be verified in polynomial time. So it's an even harder class of problems that may actually be entirely outside of NP. So there you go. When you got that on your homework, not only did he even give you an NP complete problem, he gave you something even harder, but yet, is it? Um, so induction problems on NP problems, induction proofs on NP problems. I mean, it depends on what you're trying to prove that is a little bit outside of the scope of what we're talking about here. Um, so what I do want to say, though, is that there actually are problems that are even harder

DS Lecture 22 Page 13 of 19 Transcribed by <https://otter.ai>

than NP hard. In fact, there are some problems that fall into a category known as undecidable meaning you can't even solve them at all, you know, one of the undecidable problems is actually something known as the halting problem.

56:48

So the halting problem is, like, Can you write a program that determines whether or not another program will terminate? It turns out, you can't? Because it, how would you check it on itself. So these are problems that aren't even decidable. Alright, so there are progressively harder and harder classes of problems. All right. Now. So just to put this in a little bit of context, there are a lot of problems that end up being NP complete. And you

would imagine that finding sort of simple, efficient tours whose cost is less than some K , or even better, optimal, would be extremely useful for someone like the post office, or FedEx or pretty much any shipping company, like solving these things, are things that we kind of need to be able to do, but we can't. And so what we end up having to do is we end up having to use approximation algorithms, where we don't necessarily get the optimal answer. But we get something hopefully, okay and usable. And so traveling salesperson is an example of a problem that you can, you can come up with a really like intuitive, greedy algorithm to get an approximation, but it's definitely not optimal. And a greedy approximation of the TSP problem is something known as nearest neighbor, where you pick a starting point, and then you just look out at all the vertices you haven't visited yet, and go to the one that's closest to you. Then from there, you look out at all the vertices that haven't been visited yet and go to all the ones closest to that, and so on. And so it's not optimal, because you could get sort of bad configurations where you kind of zigzag across the script the scene a little bit. But that's one way of attempting to solve the problem. And there are better approximations than that. One approximation might be, well, minimum spanning trees actually provide an absolute lower bound on how short tsp cycle could be. But remember, you're not allowed duplicates, and a minimum spanning tree would require backtracking. So you can actually construct a simple cycle out of a minimum spanning tree, and that one's actually bounded. If you did something like that. It is at most two times as bad as the optimal. So there are ways of bounding things. So it was said, but I thought we can solve it just it would take a lot of time. And you're right. Um, if I asked, so if we had had time for homework six, I actually would have had you implement the brute force solution to traveling salesman where I would have you just generate all permutations of the cities and you wouldn't be able to specify the number of cities so really, that's a that's a problem for no Dykstra is just shortest paths, from one point to another point, no guarantee what you get would pass the role of It's a good thought though. So really, that's an exercise in recursively generating permutations. All right. So it's a good exercise, but we don't have time. That's fine. I could, you could do that for seven cities, you could do that for eight cities. If you're waiting, sitting, willing to sit and wait for DS Lecture 22 Page 14 of 19 Transcribed by <https://otter.ai>

1015 minutes, you could do it for 1015 cities. Um, uh, you know what, putting that together is something that I do not have time to do until after the exam is over. Because I have to write an exam for you. That's like next week, which you may think you're spending all nighters getting ready for these exams, guess who's not going to be sleeping this weekend. So don't ask me for extra until after the exams ready to go. And I've had a chance to take a nap after releasing those things. Um, alright, so I completely lost my train of thought there. What was I saying? Right? So you could calculate it. But if you get up to like 100 cities, even really fast computers, it would take millennia to calculate. So it really isn't that useful. Okay, hold on one second.

1:01:24

Sorry, that someone important or sorry, the quacking was my wife's cell phone. That is, in fact, a duck. Um, so the way we have things working is that on my wife's cell phone, all her ring tones, but mine are a duck. On my cell phone, all my all my ring tones are normal, except for ers, which is on my phone, a duck. So if it quacks it's my wife, that's the way I remember it.

1:01:57

Any professor, would you like me to share my ringtone with you? Um,

1:02:02

I have a feeling I don't that that sounds potentially terrifying. Why? I don't know what your ringtone is, obviously. All right, let me let me finish the thought here. We don't have that much time. Um, and no, we do not have any pets, especially not a duck. All right. So I think that wraps up the new material that I want to cover. And so let's just briefly in the time, we

have talked about what to expect for the final just to make sure everybody's heard it, and then I'll go into a lot more depth on Friday during the review. Okay, so you have this review sheet. Here, I'll actually switch the share to it.

1:02:59

You have this review sheet that's posted in the courseworks file section. And so I listed out the portions of the text that you need to be concerned with, we did chapter one. But you know, we really only need to worry about proofs and recursion. The Java stuff in there wasn't super useful. We just learned it as we went. And the chapter one stuff on Java was a

DS Lecture 22 Page 15 of 19 Transcribed by <https://otter.ai>

little too in depth chapter two, which was algorithm analysis, although we didn't necessarily do all the examples, you really should just read the whole chapter, chapter three, which was lists, stacks, and queues. Again, even though we didn't do all the examples, you're still pretty much responsible for the whole chapter. Obviously, if we didn't talk about an example in class, that's not going to be on the exam. But it might be good practice for something else I can concoct. CHAPTER FOUR was the trees chapter, we did sections 4.1 through 4.4, section 4.4, or the avielle. Tree is. And then there was also section 4.6, which was just a separate little section on tree traversals, we spent a lot of time talking about tree traversal. So if you want a little more reading on it, that's more point six. If you care at all about maps and sets, because we've kind of been using them as we go. You can read more about them in 4.8. But honestly, I'm not going to be testing that on the exams, I wouldn't worry about it. Chapter Five was our hash tables. We went 5.1 through 5.5, which is up through the rehashing section. And what I might note on that also, is that there was a little sub section on double hashing, which I don't actually remember whether or not I talked about double hashing in the lecture or not, it was such a throwaway topic, but it will not be covered on the exam. We're just going to worry about separate chaining linear probing and quadratic probing. Okay, um, 5.6 talks about hash tables in the Java libraries, which is used, but probably you don't need much in the way review chapter six, where the binary heaps the priority keys we were doing, the portions that we were working on went through from 6.1 through 6.3. In chapter seven, this is the sorting chapter. You know, we did 7.1 7.2, doesn't hurt to skim 7.3. But there are some proofs in there, you don't really have to worry about, we skipped shell sort. So no, 7.4 7.5, I believe, was heapsort, which we talked about in the context of the previous chapter. No, I didn't say that. Hold on a second. Um, I'm very confused. Hold on a second. And then we had merge sort and quicksort chapter eight. That's the disjoint sets that I spent like eight minutes on today. We don't have to worry about that too much. Just other than the context of crossbills algorithm, it would not spend more than like five minutes skimming the first three pages of that chapter. And then chapter nine. What did we do from nine we did 9.1, which was topological sorting. nine point sorry, 9.1, which were the preliminaries. 9.2 was topological sorting. 9.3 were the shortest paths focus on these two sections. That was the unweighted and the Dykstra is 9.5 was the minimum spanning trees, and 9.7 was the P versus NP stuff that we did today. Okay. Now, I have noted here that in terms of material from outside the textbook, if I left something off here, I'll mention it briefly. You we did do the Tower of Hanoi recursion, right? That's not going to appear directly on the exam. But it's useful to look over. Um, that wasn't in the textbook. But the code for that is in the few lectures that I actually did post code for, because everything else came out of the book. The other thing that we did in lecture that was not directly in the textbook was selection sort, where you repeatedly find men. Insertion Sort was the comparison based sort that was actually in the textbook. Okay. And so what was this about me saying 90% of the exam would be new stuff. I didn't say that. Yeah, the exam is,

DS Lecture 22 Page 16 of 19 Transcribed by <https://otter.ai>

1:07:47

oh, the exam is cumulative. There'll be more of an emphasis on the second half. But if you

think about it, a lot of the stuff we did in the second half, relies on the stuff we did in the first half, we've been repeatedly using things like lists, and queues, and stacks, and all of these things. So it all builds, so it's definitely going to be use me, it's definitely going to be cumulative. What you might remember, somebody asked me how much longer the final was than the midterm, and I'd say it was probably about 60 or so percent bigger, rather than exactly double. Okay? Now, over the course of the term, so what's the structure of the exam? Alright, so, logistics, you have the 20th, the 21st, and the 22nd. To take the exam, you can do it at any point during that window. But once you start a section of the exam, you have to finish it. The exam, like the midterm is broken up into two parts. One part will be a courseworks quiz, which will consist of multiple choice and like simple fill in the blank. By the way, in terms of the fill in the blank, you're just writing the number or the letter, you are not writing any explanation that will break the system if you write anything more than what I asked, okay, and so that's the auto graded part of the exam. And then the second half will be in great scope. You can do them independently of each other. That second half will include some coding problems, some sketched out stuff, that's more of a written problem, just like we did on the the midterm. You'll have an hour and a half for the courseworks quiz portion. I'm going to give you an hour and 45 minutes for the grade scope portion. And the reason why it's an hour and 45 minutes is to, you know account for the fact that you have to stop and then upload the file before the time runs out. The timer that you get through grade scope includes those 15 minutes. So once that timer runs out, you can't even upload it anymore. So you need to stop at the hour and a half mark and spend those last 15 minutes getting it up loaded. Okay. All right. So, Amelia, I'm not actually 100%? Sure, I think there will definitely be, there were a couple of coding problems on the midterm, right? There'll be three, I believe coding problems on the final. And so probably typing may do better for those, there might be one problem that involves some diagrams, but I haven't actually fully written that portion yet. So we'll kind of see how that goes. I might have more to say on that on Friday. Um, a few things that I've told you over time will be on the exam. Um, I think it will be problem one on the grade scope portion will be some kind of induction proof, it will be a simple one, very similar to what you would have seen on the homework, but that'll probably be problem one. on that, um, you had to do some kind of a recursion on some type of a tree on the midterm, you're going to have to do something like that again, on the final. Okay, I've said that before. Um, there will be questions very much, they'll be structured very similarly to the quiz on the original course works, where you had to specify bingo costs for things. And so that's going to be there again, except this time, we're going to be including all of the algorithms we did in the second half as well. So one of the first things you probably want to do is you want to go back and just look through all the algorithms we did and just familiarize

DS Lecture 22 Page 17 of 19 Transcribed by <https://otter.ai>

yourself with the big O costs for all of them. So you kind of have that at your fingertips when the time comes. The last thing that I know I told you is going to be on the exam so far, is that I'm going to have you crank through Dijkstra algorithm on the exam, I'm going to give you a graph and give you a starting vertex. And you're going to have to fill out a chart going through that algorithm. That'll be on the courseworks quizzes version, and you'll be filling in the fields one number at a time, as I indicate on the exam. Okay, so I'm guaranteeing you that's there. And I think

1:12:40

that's really all I have to say about the exam. For now, that gives you a pretty good sense of what most of the exam is. It's close book and close notes. Just like before, you know what the time limits are, you understand the parameters, you can do each part separately, but once you start a section, you've got to finish it, the chart will not be something you have to draw for Dijkstra, as the chart will be baked into the courseworks quiz with little fields to type in and you will simply type in the numbers. Okay. All right. So,

oh, one last thing that I did say you would have to do remember the avielle trees inserts problem you had, you're going to have to insert things into an abl tree on paper. Now, I don't yet know whether I'm going to ask you to draw that. Or I might ask you just to do it on scratch paper, and then I'll ask you questions about it in a multiple choice format. Either way, is a possibility. But you should just be able to take a list of numbers and insert them properly into an abl tree. Okay, so these, these are the things you're going to be asked to do. And there'll be more and I'll talk more about them on Friday. And I'll go into the details, because there's a lot of details in the rest of this document on Friday. But other than that, thank you guys very much for listening. And that's it for our regular lectures. Good luck with your finals. Thank you.

1:14:12

Take care of Dr. Blair. Wait till Friday five o'clock. Yep. And it won't be recorded. And how do you hear about my pineapple beans that I got pretend to see what the my pineapple

1:14:30

is mine. Yeah. I thought originally, like your site. And then I looked at it and I was like,

1:14:39

DS Lecture 22 Page 18 of 19 Transcribed by <https://otter.ai>

Alright, do you know this right? I usually show up with something. But you don't even know I'm gonna be in my office. So there's there's no way to find me right now. I'm afraid. I don't know why. I mean, Dr. Rubinstein is there now I have my own risks factors that I got to deal with.

1:15:07

I'm just joking. Stay safe and healthy. You're not allowed yesterday.

1:15:12

Yeah, already have times anyway. Alright folks, anything else you want to ask about before I get by everybody? Maybe. Alright folks, if there's nothing I'm gonna end the meeting. Thank you. Take care. Thank you.

DS Lecture 22 Page 19 of 19 Transcribed by <https://otter.ai>

DS Lecture 23

Wed, 8/11 2:08PM 53:24

SUMMARY KEYWORDS

dykstra, algorithm, questions, exam, midterm, trees, graph, familiar, nodes, chapter, problem, vertex, binary search tree, hash tables, shortest path, recursion, binary trees, remember, terms, asked

02:12

And maybe we'll welcome Alright folks, let's get started. Um, I would like to go through the final review. Um, and I guess this will be posted, the video will be posted in a little while. Bo, what's up?

03:59

I was just gonna ask if anybody's in the vicinity of Dr. Blair's house, you could bring him something to supplement his coke so that our final is much easier because if he keeps drinking Diet Coke, that finals is going to get longer and longer and longer. And now I'm going to just stop but just in case just bring him something nice. Do something. Um,

04:24

I

04:26

I think I will say the following. If I don't have a sufficient amount of caffeine, your final will

be worse, much, much worse. So you should be happy when you see me drinking the caffeine. Alright, let's not talk about me anymore. Um, let's go through this. So we'll start with going through the logistics of the exam. I know we've spoken about this before, but have it all in one place. Two parts A Multiple Choice fill in the blank part, very much like DS Lecture 23 Page 1 of 17 Transcribed by <https://otter.ai> what we had on the midterm and structure that's an hour and a half. Second part more written programming style stuff, for the most part, they'll be approved on it as well, we'll get into the specifics of that in a little bit.

05:23

We'll get into the specifics of that in a little bit. And that's going to be on great scope. The timer on that will be set for an hour and 45 minutes with those last 15 minutes really meant to help you have time to upload your submission. That's not separate, that's all rolled into one may start either. First, it doesn't matter. They can be done at different times. Once you start one. That's it. You have to finish that one. Okay. So that's the logistics, they will become live at midnight on Tuesday, the 20th. And you must have completed both by 11:59pm. On Thursday, the 22nd. All right. It's closed book, closed notes, no calculators, no computers, beyond what you're using the computer to actually fill in the answers. You're not compiling things, you're definitely not looking anything up. And then I need to trust you on that. Thankfully, um, I'd say our midterm average was actually very much in very much consistent with prior years. So it seems like most people are really following the rules well on that, so I'm pretty happy. Keep it up. Okay. So that's our logistics, if you have any accommodations on the exam, they were sent to me via Disability Services and the extended time you're entitled to, will be baked in to courseworks in grade scope. So if you're entitled to more time when you start it, you'll have it right there. Okay. piata, I will leave open until Monday. And then I'm going to close down to only private questions. Um, so that's more for logistics and stuff like that on the exam, I probably will not be answering too many detailed questions while you're taking the exam. Just be aware, I have to make sure everybody has the same information going in. But you're welcome to Ask clarifying questions. Or if you run into a problem, feel free, we will try to answer them in as timely a fashion as we can. I don't think it makes much of a difference. But if you really, really want to take the exam when I am more likely to see questions. Your best bet is to take it in the sort of the mid afternoon to evenings of those days with the exception of the 20th, where after I get the exams to you, I have a lot of stuff all day that day, and will probably be away from my computer. Okay. So that's really what I want to say in terms of logistics. Um, any questions on logistics before I move forward?

08:30

We're good.

08:32

DS Lecture 23 Page 2 of 17 Transcribed by <https://otter.ai>

Cool. This is the same as thing we did with the midterm just basically twice as long. So that should be pretty straightforward. Okay, so now I posted the sheet, I guess on the 14th. So this is sort of my review document for the exam. You've had it for a few days. I'm going to do I'm going to run through this. And I'm going to make commentary about the various things and probably make references to what kinds of questions I'm going to ask you on the exam as I'm going, um, don't interrupt me while I'm doing that first pass. I'm not going to take questions while I'm doing the first pass. You should be aware though, that the first thing I'm going to ask you after I'm done doing my first pass through is going to be what Didn't you hear that you expected to hear? So if there are things you expected to hear, but you're not hearing the saying jot them down, and then bring them up when I say to at the end of that. So it's not so much that you'll remind me of something, you know, I'll suddenly put it on the exam. The exam already mostly exists. And it's more of remind me to mention it because it might already be there. Okay, Does that sound good? All right. And

then after that, You can ask clarifying questions. And if there's stuff you want me to go over if there's time, I will. But be aware that if you ask me to go over something, I might say no or kind of push it off. And you should read something into that. Because I'm going to try to spend time on things that are relevant to the exams of you asked me for something that I don't think I'm going to put on the exam. And I sort of brush it off that you can take that as a hint. Okay. All right. So, in the checks textbook, you're responsible for Well, sort of chapter one. proofs are definitely something we did, there will be an induction proof on the exam, it'll probably be question one on the grade scope portion. So you should review the structures of those proofs. My expectation is that you show the base case, you state your inductive hypothesis explicitly, and then prove the next step, all three parts have got to be there, or you will not get full credit on that. Okay. And so chapter one will help to review for that, although Honestly, it will likely be either on a tree property like the ones we did, we're on a very similar graph property, which would take the same structure as the tree ones that we did in lecture. Also, from chapter one, some discussion of recursion. recursion is important in this class, we use the recursion a lot, just like we did on the midterm, I believe I will have a programming problem involving some recursion on some kind of tree. So recursion is good to review the rest of chapter one, you can kind of just skim the Java review, and it is kind of useless, just focus on the aspects of Java that we've been using throughout the course. Okay. Chapter Two, basically, the full chapter. There were some examples we didn't do. But it couldn't hurt to read over them. No, I'm not going to test you on. This was the big go analysis stuff. There will be

12:09

many bego like questions, much like you saw on the first part of the exam. On the midterm, they'll be very similar style stuff, probably focusing a bit more on algorithms we did in the second half, but the same styles of questions. Okay. CHAPTER THREE was lists

DS Lecture 23 Page 3 of 17 Transcribed by <https://otter.ai>

and the lists, abstract data type stacks, queues, we did iterators, on lists, those sorts of things. You should be responsible for all of that, um, will I have you implement something out of that directly, like I did on the midterm? No, probably not. I've got other things I need you to implement. But as building blocks for everything else, we're going to do, very important. Chapter Four, section 4.1, through 4.4, plus maybe 4.6. So 4.1, and 4.2. I think 4.1 was General Purpose trees, sort of trees that could have an arbitrary number, I should really wait for the later half of this discussion for that. But you know, general purpose trees, binary trees, related applications, like expression trees, then we did binary search trees and abl trees, were all in there. 4.6 was on tree traversals, we did a lot of traversals. Probably useful to read. If you want to skim 4.8, there's some conversation of maps and sets, we use sets now. And then we use maps now and then more in terms of hashes than anything. So probably not worth spending a lot of time on chapter five. This is the hashing chapter 5.1 through 5.5, that's through the section on rehashing you can kind of skip the stuff on double hashing but the stuff on separate chaining, linear probing, quadratic probing, probing all very relevant 5.6. Again, talks about hash tables in the Java libraries might be worth a skim, not super important as far as the exam is concerned. Chapter Six was our chapter on priority queues in binary heaps, we really only did through section 6.3. There. Chapter Seven was our sorting chapter 7.1 7.2 7.3 talks about the lower bound of sorting that was the big omega of sorting as $N \log N$, you're not going to have to prove anything about that. You can kind of just skim 7.3 we skipped 7.4. That was shell sort don't worry about that. 7.5 was heapsort, which we kind of more did as a house. Should I say this? we kind of did that more as an example of heaps in the previous chapter, but we did have to use a Mac seat for it. So it might be worth looking over. 7.6 was merge sort. And 7.7 was quicksort. Okay. All right. That was it for chapter seven, chapter eight. That was disjoint sets on which I spent approximately four minutes to five minutes in lecture discussing disjoint sets in the context of Chronicles algorithm. If you spend more than four

or five minutes studying disjoint sets and chapter eight, you have spent too much time. Just think you look over the first three pages of it, and just remember roughly how it was used in the context of cross Google's algorithm. Okay, so don't spend any time on that really. And then chapter nine, our Final Chapter or Chapter on graphs, we did section 9.1. Point two on topological sorting. 9.3 was the shortest path stuff, focusing on 9.3 point one and 9.3 point two, which was the unweighted single source shortest paths. And the Dijkstra is algorithm and I will have you run through Dijkstra is algorithm on paper in some form on the exam and buy in some form. I know exactly what form and we'll talk about that more a little bit later on.

16:27

Then we did 9.5. On minimum spanning trees, we briefly mentioned Prim's algorithm, which was a slight play on Dijkstra algorithm, the main takeaway of Prim's algorithm is the DS Lecture 23 Page 4 of 17 Transcribed by <https://otter.ai>

that you don't want to mix it up with Dijkstra algorithm, alright, just it's a different update rule and don't inadvertently do that algorithm and solve for a minimum spanning tree when I'm asking you for shortest paths. Okay. The more important one in there was Chronical's algorithm, which was, again, a greedy algorithm, sort of selecting shortest edges. That's where disjoint sets came in. And then the last bit we did in there was chapter 9.7, which was the P versus NP stuff, and the traveling salesperson problem, and there'll be a short answer question on that. Nothing really tricky. Just remember the basic definitions, you'll be good. Okay. There was a private question, which normally, I said, hold your questions, but to the person who asked privately. I don't know what you're talking about. Though, apparently, a whole bunch of other people seem to know what you're talking about. Okay. So that's the readings. The one thing the one thing from the that was not well, actually, I left something off here, there were two things that we did in lecture that we're not directly from the textbook, the first thing that we did, and the one that I actually have written here was the selection sort algorithm. Another one of those comparison based algorithms is a real messy, the real not so good one, because the worst case was still n^2 . And that selection sort is something that you should be familiar with, in the context of sorting in addition to what's in the textbook. Okay. The other thing that wasn't in the textbook that I didn't actually mention here is that when we were doing recursion, you may remember we had done the Tower of Hanoi problem is just another example of recursion. Okay. Well, I talked about that directly on the exam, probably not. Is it a good example of recursion? Just in general for review? Yes, it is. That's why we did it. Okay. All right. So that's the material that you're studying with. You knew all that already. And now let's maybe do a quick, quick, well, 30 minute, 20 minute, 20 minutes, let's say 20 minute, dive into the topics, they hopefully a little less than 20 minutes. So general concepts, abstract data types versus a data structure, right, we use those terms a lot, you should be okay with them. You should be very comfortable with recursion, and the rules of recursion and how to write recursive methods at this point. And you should be able to deal with a basic proof by induction. All right, I think on. I don't remember which one I had to do. I had to do a couple of inductions on the homework. Maybe one was, you know, you have n full nodes then show that there are $n + 1$ leaves or something along those lines. proofs of that structure will show up. I could tweak it for a property on a graph. I haven't decided which version of that question I'll ask. But overall it's the same basic structure. You won't be asked to do something like that Fibonacci proof that I did early on, there'll be much more like the proofs we did in terms of the trees. Okay. And that is question one on the part two of the exam will be that induction. Okay? Just I'm just telling you this.

20:20

DS Lecture 23 Page 5 of 17 Transcribed by <https://otter.ai>

Where can you repeat that again? Dr. Blair, I'll probably get it wrong anyway. But what

was it?

20:26

an induction proof will be part question one on part two, probably on some property of a tree. Okay, possibly on a property of a graph, but it will look very similar. Alright, next. Java concepts. You know, at this point, I expect a certain level of fluency in Java. So if I'm going to do basic object oriented programming, you should know the difference between a class and an object should know what methods are, the term that I used here is fields, but I just mean instance variables, private versus public, that sort of thing, I'm not going to ask you necessarily to construct some kind of complicated generic definition of a class. But if I give you a class that involves generics on the exam, you should be ok with that and be able to use it without any problems. Um, if I use a nested class, like a static nested class or an inner class, you should be able to handle that. On the exam. I did that actually on the midterm, right? Remember, there was the node class for the, I guess it was a stat we were building. So in that vein, if they show up, you should be able to deal with them. You should know what interfaces are particularly comparable, since we use it for practically everything. And, you know, if I asked you to deal with an iterator or ask you costs associated with an iterator, you should understand how iterators work. Okay. In terms of Big O analysis, look, you should remember the definitions for big O big feta and big omega. Okay? Remember, that was like the upper bound for bingo, the lower bound for big omega, and then the tight bound for big feta, where it's both bingo and big omega at the same time. But in terms of the analysis that I might ask you to do. We're going to we're going to focus on big on costs, if I don't specify otherwise, I'm asking for that cost in the worst case, but I could ask for best case, and I guess, if I'm being particularly mean, or didn't have my coffee, coffee, because I don't have a coffeemaker at home, didn't have my diet coke that morning, I might ask you for something about the average case, but mostly worst case, but just keep an eye out. If I happen to say best case somewhere, don't let that trip you up. You should remember what the typical growth rates for these costs are and how they're ordered. And you should be prepared to state what the costs are for all of the algorithms that we've done over the semester, including the graph algorithms in terms of magnitude V and magnitude II. The differences between the versions that use the cue or don't use the cue or use the priority queue and don't use the priority queue, whether it's dense or sparse, might have an effect. And I will ask many questions in the multiple choice the way I did ask many questions in the multiple choice on the midterm for stuff about that, spanning all of the algorithms we've seen so far. All right. Um, okay, so lists, I mean, you should know roughly what the list abstract data type looks like. I'm sorry, hold on one second. There we go. You should be able to know what the list abstract data type is, what the basic list operations are, you should be familiar with array lists, the basic DS Lecture 23 Page 6 of 17 Transcribed by <https://otter.ai>

costs of their methods, how they're implemented, and so on. You should be familiar with link lists, both singly and doubly linked lists. Their basic method is what the running times are for them. And you should be able to deal with the fact that sometimes we use Sentinel nodes, which you will either see referred to as the begin marker and marker or head or tail, much like you saw on the midterm. Okay. Uh, you should be able to implement stuff on this. This by the way that you see written here, this is exactly what I had written on the midterm review, because it was the same skills that you needed to be able to perform. Okay.

24:51

stacks and queues should know what the stack abstract data type is, what the queue abstract data type is, you should be able to get all your operations on the Things to work in order one time. We talked about a variety of stack applications, including symbol balancing, detecting palindromes, reordering sequences, evaluating postorder expressions and building expression trees, all of which are potentially a fair game, as is the

queue implementations and using a queue. And actually using queues showed up a lot more in the second half of the class. As you may have noticed, as we were getting into graphs and things like that, it also showed up in the context of those level order traversals we did on the binary search trees. So you should be able to implement these things. If I asked you to, as I already did on a midterm. Well, I know I'm going to focus on other stuff, but those skills will come in handy. Trees, make sure you have the terminology down Parent Child root leaf path depth versus height. Be familiar with the various implementations of trees. You saw, we did a recursion that involve first child and next sibling on the midterm, something like that fair game, you should be familiar with binary trees, you should make sure you remember the definitions of full versus complete versus perfect binary trees, I will use those definitions in various questions, I will not define them for you. So being familiar with what they are and make sure you don't mix them up, be able to perform in order, preorder and postorder traversals on trees, be familiar with our expression tree construction algorithms. And how we use the stack to do that be familiar between the relationship of number of nodes and the height of the binary tree depending on whether or not it's balanced. And as we said, Our induction proofs will likely be on trees or a similar type of structure. You should be able to do these kinds of things on paper, you should be able to show these implementations, binary search trees, you should remember the binary search tree property, you should remember the various methods that a binary search tree would have, including contains five min find max insert, remove roughly how not roughly you should be able to implement those should remember the costs on a binary search tree are usually big O of n . Not $\log$ of n because we don't know whether or not it's balanced. In the worst case, it's O of n unless I say something about it being

DS Lecture 23 Page 7 of 17 Transcribed by <https://otter.ai>

balanced. On the average, those operations could be $\log$ of n . But in the worst case, not unless they're balanced. we'd spent some time on lazy deletion on the homework. Can you hold your questions, please? We spend some time on lazy deletion on your homework. I'll do lazy deletion, more likely in terms of hash tables. But it's worth a review here as well. You should be able to perform these kinds of things on paper as well. avielle trees, these are the balanced binary search trees where you are maintaining the avielle property. All right. And there are a variety of cases that would cause imbalances, and then he's learned the rotations to balance them. All right, you're going to have to insert a bunch of nodes into an initially empty avielle tree as part of this exam. I'm just telling you that now whether I'm going to have you draw the whole tree out on part two, or whether I'm going to keep the problem on part one, and then just ask you what the heights are various nodes after you correctly do the inserts. I haven't decided yet which version of that problem I'm going to give. But as long as you know, all your other definitions and know how to do inserts on paper to an avielle tree, you should be able to do that. So is everybody clear that something you are going to need to do? Okay.

29:06

Ash tables, all right, you should be familiar with what a hash function is. And the various things we might use them for. What properties might make for a good two hash function is what in the particular circumstances you're trying to use them. The main thing we did with hash functions was use them to build hash tables. And that's where my questions will really focus on. And the main thing when it came to hash tables, were the collision resolution strategies. All right, what do we do when two things hash to the same location in the array? All right, so we had separate chaining, we had linear probing, we had quadratic probing, and they all had various, you know, constraints on the load factor, right. For separate chaining, you wanted to make sure the load factor was less than one for linear probing and quadratic pro You wanted to make sure it was less than a half for quadratic probing, it had to be less than a half. And the table size had to be prime. Okay, otherwise, inserts would not necessarily work. All right? I'll probably ask you to do some

inserts into a hash table, you'll probably have to mod by 11. Just saying, Okay, now you're definitely gonna have to mod by 11. All right. And that's going to be on part one of the exam with one of these fill in the blank type things. Okay. There'll be some other questions on hash tables, too. That's not it. But that that's one of the things you should know what rehashing is when we do it, how we do it. And I guess it couldn't hurt to remember what hash sets and hash maps are, in case I happen to use them as a sort of in passing on other questions. But the main thing is the stuff I've mentioned before, are from there, we're in chapter six onto priority queues. Alright, so you should be familiar with the difference between the min heap and a max heap, you should be familiar with binary heaps, and the tree representation as well as the array representation. All right, you should remember

DS Lecture 23 Page 8 of 17 Transcribed by <https://otter.ai>

what the property of a binary heap is that it's a complete tree, and what the heap order property is, and you should know how to do the inserts, the Delete mins, or delete maxes, depending on whether it's a min or Maxine, you should know how the percolate ups in the percolate downs work. Very importantly, in the array based notation, you should know how to calculate the parent and child nodes of the index of a given node. And you should be familiar with the build heap operation, that linear operation to build an initially unordered array. Now, you don't have to do any of the proof stuff on that. I'm going to have a coding problem associated with this, you're going to have to code something on part two, associated with a heap implementation. I haven't decided which version of that problems put there, I've got a few options, but you're going to have to do something. Okay. Good. Moving on to sorting. We talked about Insertion Sort. We talked about selection sort, which wasn't in the book, but straightforward enough, then heap sort more in the context of heaps in previous section, but you should certainly know what it is. And its costs, merge sorts, you should remember the merge operation. And you know, we didn't code the merge operation, but I don't know, a merge operation might be a cool thing for me to ask you to code or some play on it might be. Then we did quicksort. We talked about the various partitioning strategy, sorry, the various pivot selection strategies, and how we do the partition. And you know that partitioning might also be a fun thing for me to ask you how to code. So there's going to be a third coding question on that. Yeah, I'm going somewhere with this, there's gonna be a third coding question on that. Second half. I don't know if it's gonna be like a merge sort thing or quicksort thing, but it'll be one of them. Okay.

33:14

All right. Now we get to graphs. All right, we're moving pretty quickly here. You should be familiar with all the terminology associated with graphs should be familiar with vertices and edges, you should be familiar with the fact that we're using set notation there, V is the set of vertices, E is the set of edges, the magnitude of V , that's that absolute value sign on V , that's the number of vertices and our big O costs are often in terms of that, and then the magnitude is the number of edges. And likewise, with that, you should remember that a complete graph is one where every vertex has a link to every other vertex, you should remember the difference between directed versus undirected graphs. weighted versus unweighted graphs, you should remember what a DAG is, you should remember what the difference is between being connected, or disconnected in a undirected graph of the difference between being strongly or weakly connected on a directed graph. You should remember the distinction between dense and sparse graphs, you should remember how we represent these graphs using either adjacency matrices or adjacency lists. And from there we go into the graph algorithms you should be familiar with topological sorting the costs of the various ways to do it. The single source unweighted shortest paths algorithm

DS Lecture 23 Page 9 of 17 Transcribed by <https://otter.ai>

and the associated costs of the various ways to do it. That was basically a breadth first search. Similarly for Dykstra is algorithm if we're doing single source, shortest path for

weighted graphs, now Dijkstra is algorithm. I've told you this before and I'll tell it to you again. I'm going to give you a graph on the exam, and I'm going to give Starting vertex, and then you're going to have to run through Dijkstra algorithm on paper. Now I'm going to put it on part one of the exam, that's the fill in the blank one. And the way I'm going to do it is I'm actually going to give you a slightly different chart than you've seen, where each row corresponds to an iteration of the algorithm. And all I'm going to want you to jot down are the DVDs for each vertex at each iteration of the algorithm. So on your scratch paper, you can keep track of the DVDs and the PBS, but what you're filling in are just the distances as you run through the algorithm. Okay, so you will be asked to do that on paper. And then fill in the blanks. And that's a part one question. minimum spanning trees, we've got, again, hold questions, minimum spanning trees, we talked about prims algorithm, you should know what it is don't mix it up with Dijkstra algorithm, you should be familiar with Chronicles algorithm, I could ask you to run through criticals algorithm on paper without like, actually dealing with the disjoint sets directly. That's very straightforward. So just be familiar with it. And then you should be familiar with a set of problems P , those are the ones that can be solved in polynomial time, versus the set of problems NP , which are the ones that can be verified in polynomial time, you know that P is a subset of NP , because if you can solve it in polynomial time, you can certainly verify it in polynomial time. But we don't know whether or not P is equal to NP . Now, there's a set of problems that are the hardest problems, and then P , the ones called NP complete, which are hard, so hard, and they have the special property that if you can figure out how to solve one of them, you can use that solution to solve all the problems in NP . So there would be some interesting consequences. If you actually figured out a polynomial way of solving an NP complete problem. We gave an example of an NP complete problem, this was the traveling salesperson problem. But where we bounded it, where we were finding a simple cycle that visited all vertices

37:21

that had a length less than some fixed K , that was NP complete, because you could verify its result in polynomial time. But as far as we can tell, you can't solve it in any better really than exponential or really in factorial time. And then there was the NP hard version, which was the one where I asked you for the shortest, simple cycle that visited all vertices, which could be used to solve the NP complete version. But it's harder, because you can't actually verify it, as far as we know, in polynomial time at all. And that sort of covers everything, I think, is fair game for the exam. And I believe I told you what about half the exam was maybe two thirds of the exam as we went through that. And so now comes the question I

DS Lecture 23 Page 10 of 17 Transcribed by <https://otter.ai>

have for you, which I warned you about. Was there anything you expected me to mention there that I didn't? Anybody? Yes. What?

38:28

Is your extra credit? No, I'm joking. I'm digress. I'm like, I don't know how to pronounce and Dijkstra. Okay. Yeah. Can you? Alright, so, the last one that you drew, and then you change it to the one after that, which I don't even want to name it is crucibles, crystals, or skulls? Right. Have you started off with all being moved over to a crystal? So with Drake? Hold on,

38:55

hold on. Let's just just make sure. I know. But I just want to say one thing. First. Dijkstra is algorithm is solving single source weighted shortest paths.

39:09

I understand those hours in the solving for the minimum spanning ESU evaluate every single possible outlet before selecting said shortest note, correct. That's where dextrin is. character flaw, mostly but Dijkstra is algorithm right when you say Dijkstra go from here to here, from node A to node B, he has to evaluate every single possible route before

determining which one to use.

39:44

Right? Not necessarily. Not it doesn't necessarily shake out that way. What he does do is he finds the shortest path from The starting vertex to every single vertex, right, not just to a particular endpoint. Now, there's a greedy step in there, where you say, Okay, I'm certain, I have the shortest path to a particular vertex,

40:19

right? That's three that in the in the programming, by the way, just to double check it, and whatever, I just left it in there, but can take sure also, like, can you go? You can only jump one node, right? Like meaning like, you know that you can only go to the adjacent nodes, if you call? No, when you do the update step, you are only at the moment, looking at the adjacent nodes. What I'm saying is, do you want that thing to go from coast to coast? Or

DS Lecture 23 Page 11 of 17 Transcribed by <https://otter.ai>

is it fine the way it is? Being Dykstra?

40:58

I do not understand the question. No, we're gonna put it in that because that's not what I learned. Hold on this seriously. Stop for a second, please. I asked for here. We're first before we dive into the specifics of the algorithm. We're things topics that we didn't discuss yet, so I need to talk about that first, and then we can go into the weeds with Dykstra later. Okay. So were there topics specifically, not specific things about Dykstra is that we didn't hear about? Any anybody? Yes. Do have to worry about like reconstructing trees based on different commercial types. I mean, I had that on the midterm, right? Ah, I mean, yes, but I probably won't put that question again. Exactly. in that form. Okay. So just like, two minutes of review? Yeah. Andre, I think you were next, then Catherine.

41:57

Hello. You have not mentioned anything about the Quick Select algorithm?

42:03

No, we're not going to have that on the exam. I don't think clearly doing that in class either.

42:10

Okay, so the concepts of lazy deletion and adjacency matrix, are those important.

42:19

so lazy deletion is critical in the concept of, of deletions in quadratic in any sort of probing hash table. And that's where it might come up. That's a very good point. What was the other one? adjacency matrix? Um, I mean, you should have a sense of what an adjacency matrix is. But in terms of the algorithms themselves, we presumably are going to be mostly using adjacency lists, I think where it could come up primarily as in the concept of dense versus sparse in general.

DS Lecture 23 Page 12 of 17 Transcribed by <https://otter.ai>

42:57

Okay.

43:00

Oh, are you had your hands up this? Do you make solmar?

43:08

No, I just wanted to ask a question about the NP complete and NP hard. But

43:13

so maybe if that's not like a topic question, let's push that a little further down. Okay. Um, so let me just look at the chat for a minute. Um,

43:23

knowing knowing what NP unknown problems like the halting problem are nothing

outside of the basics that I spoke about in lecture is relevant. What about load factors? For hash tables? Yes, you should be familiar with them, and what your target is for them, depending on whether you're using separate chaining or one of the probing ones. I think I did mention that, but if I didn't mention it firmly enough, that is something you should know. Yes. Should we familiar with BSD methods like those from better BSD such as hyper mirror? Specifically, I don't know if I would ask about those. But you know, I need to come up with some kind of recursion for you to do for that recursion problem. So it's completely possible that I might ask you something similar to one of those. It's good practice for that. But those specifically, I'm not completely sure. In general, in terms of studying, can I ask you, I presume you would repeat a question that was asked on the midterm. I mean, there'll be enough that are very similar, that I'd be hesitant to say, No, I won't, like definitely won't repeat stuff, but they won't be the same. But they could be similar, right? Like the recursion on a tree already is pretty similar, and won't necessarily be the exact same thing will topological sorting beyond it. Potentially. Yes, I think I mentioned that. Any other topics, but I didn't. I spent more time I think talking about Dykstra, and Paul, School and things like that. So in terms of actually sitting down and cranking through it, less likely, but knowing what it is and where to use it, that's more possible. Okay. Other things that you didn't hear that you expected to hear. All right. Alright. So now, I think we're good there. So let's go back to more specific questions. Bo, we can pick up items still don't think

DS Lecture 23 Page 13 of 17 Transcribed by <https://otter.ai>

I fully understood what you were asking. So

45:31

let's try, I can really I can simplify this for you, right? If you got to go from New York to California, and there's four stops in between. Yeah. And on the if there's a menu, and one of the menu options is New York, and the other menu option is California, and you select them, and there's a bunch of nodes in between. and it's Dykstra algorithm. Dykstra is not going to know how to get there unless you, like, add something to the list of directives that are pre can. Correct. Because otherwise, it would be an infinite frontier, really. And this is, in part having to do with the homework, because,

46:19

oh, it really doesn't matter. No, doesn't matter, I'm just gonna say very specifically, nothing has to be added to the information that you're given to do that, you are given a graph with connections in the graph. So the only possible ways to get between New York and California could be things on the graph you're given. And that's what I express algorithm is working with. So there's nothing more that needs to be added than that graph. That was it. Okay. But but I'm saying it would be possible to add additional

46:54

adjacencies for like a better word. Hades uses intelligence. Why not do that for them? No, No, I didn't. But I'm saying but you could, but you could in theory, so that that's not really, I mean, then you're changing the input graph? I do not. I did not

47:10

know, I'm saying what you're suggesting isn't really related to deicers algorithms specifically, it's just saying you're changing the input graph that you then give the algorithm once the algorithm starts, it uses the graph as its if you wish to make changes to that graph. running it, that's a separate matter.

47:36

No, I mean, that's what I was a little confused about, because I was like, Dykstra knows how to draw this map. How come he doesn't know how to get from here to here.

DS Lecture 23 Page 14 of 17 Transcribed by <https://otter.ai>

47:51

We run the algorithm with any of the vertices is the starting point. And it will get you from

you. Once you run the algorithm once we can get you from New York to anywhere. just so happened. You picked California. Okay.

48:08

Okay, I just did it by the book. I did whatever it is said don't change this. And then I noticed that a little piece that said that it has been changed. in case somebody sneakily tried to go in there and I don't anyways, I don't want to I don't want to get into but so with, with Dexter, though, he basically only knows what you feed it. Yes, that's correct. Okay. So it is, it is possible to traverse? Yeah. Okay, that's all I want to know. Or if he was just an idiot, he only knows how to go to one place at a time. Andre, go ahead.

48:45

So for example, if we have insertions in an ArrayList, and you asks us for the cost for the cost, we say the amortized cost of one or the worst case in which we also have to resize, which is O of n ,

49:00

five, don't say amortized, then you don't get to tell me the amortized cost. So for such cases, we say O of n , for example, yes. Okay. And would you explain a bit more what you mean with that? By SRA iteration on the exam, like you said, we have to update the distances for each step. What a step exactly, because for what we discussed in class, we also updated distances for nodes that were not marked as known yet, all the vertices could potentially have their distances updated. When I talk about a step, what I mean is one step of the outer loop, meaning you settle a vertex and then you update its adjacency list. So for each step, which vertex to just settle and what all the distances end up being, after you've updated them,

49:53

okay, so all the distances regardless of whether they're known or not,

DS Lecture 23 Page 15 of 17 Transcribed by <https://otter.ai>

49:57

right, that's what I'm talking about. So man And I'm asking you for what are all the DVDs across all the vertices after one iteration of the outer loop, then after two iterations of the outer loop, and so on. Okay, thank you. Cool. Omar, go ahead.

50:15

Just had a quick question because I got confused, right in the traveling salesperson problem, we have the NP complete version and the NP hard version. Could you just remind us really quickly remind me what the difference between them?

50:29

So the NP complete version is where you're being asked to find a simple cycle that visits all vertices, meaning it's a simple cycles visiting the mall once, right? And you're trying to find one whose total cost is less than some K . So you're given a value k , find me such a simple cycle that is less than that K . And so that would be verifiable in the sense that if I gave you a solution, you could check by just adding up all the weights and seeing if those weights are less than k . Does that make sense? Yeah, I remember now. Thank you. So the NP hard version, let's just I mean, just stated. So the NP hard version doesn't ask you to find such a path that is less than k . It asks you to find the actual smallest one. And unfortunately, if I can't easily verify that, because I don't know what the smallest is in the first place. So I'd have to calculate that before I can even verify. That's the difference.

51:37

Good. Yep. Thank you, professor. Other questions? Are we good? No, we're not Ezra, what's up?

51:49

Hey, just a quick follow up on that. NP complete question for a traveling salesman. So the verification step there is that just like, just walk the path? And then if that's less than k ,

you're good. Is that it? Yeah. Okay. I could hear myself. slightly scary. Yeah. Sorry. My Computer echoes if I don't have headphones. Oh, okay. Other things. Are we good? We're all set for this exam. Okay. Well, thank you very much for coming. And great. Is everybody

Contents

No table of contents entries found.

else gone now? No, I think all 47 people are still here. Take care.

DS Lecture 23 Page 16 of 17 Transcribed by <https://otter.ai>

52:47

Thank you. Oh, yeah, that's what I was gonna ask you. The Happy Face question. Answer. Which question is that one again?

52:54

I don't know what you're talking about. I don't know what you're talking about either. But never there's like three bubbles? No, no clue. Not a no, no idea.

53:06

I'm denoting that but whatever it I'll put no claim to it. But if I do all the other ones and forget it. Now the homework thing that I submitted, that thing lit up. But I was like technically, I can

DS Lecture 23 Page 17 of 17 Transcribed by <https://otter.ai>

Addendum--Stack Algorithm for Expression Tree Creation

Wed, 8/11 12:47PM 13:33

SUMMARY KEYWORDS

operand, stack, expression, operator, tree, node, algorithm, push, token, pop, root node, build, hanging, postfix, evaluate, input, encapsulate, binary operators, construct, children

00:00

Here's an algorithm to construct an expression tree, which would likely be in the constructor of your expression tree class. And we got to build that out of something. So usually, the easiest thing to construct an expression tree out of is to build it out of a postfix expression. So this is going to be how to construct an expression tree out of a postfix expression. Now, that might not give me a unique expression tree for that postfix expression, but it'll certainly give me a valid one. Okay. And so we're going to start with some postfix expression.

00:56

Alright, so I'm probably going to regret having such a long one in about a second. But that's going to be the one we're going to work on. Now, think back to the stack based algorithm for evaluating an expression tree. And in the stack based algorithm for evaluating an expression tree, we asked you to basically do the following. You go through your input expression, one token at a time, where a token is either going to be an operator or an operand, and practically speaking will likely just allow you to differentiate between them by just separating them the whitespace. Now in that algorithm whose job it was, was to evaluate the expression rather than build an expression tree, the algorithm was in this review for the exam. The algorithm was when you encounter an operand, push it onto the stack. When you encounter an operator, pop the stack twice, apply that operator to those operands and then push the result back onto the stack. And then you

just keep doing that until you run out of characters in or sorry, run out of tokens in your input string. At that point, you then pop the stack one more time. And that's your answer.

Addendum--Stack Algorithm for Expression P Targeee 1C oref a4tion Transcribed by <https://otter.ai>

So it turns out with a very small modification, we can use that algorithm to build an expression tree rather than evaluate the expression. So we're going to need a stack. All right. And we're gonna go through the items in our input one character, sorry, one token at a time. Now, here's the thing. In this case, what am I trying to get, I'm trying to build an expression tree. But this is really going on inside the constructor of an expression tree class. And so the constructor's job is really to assign the instance variables of the of the class that's being built. And so an expression tree has a single instance variable, can anyone remind me what that instance variable is? It's the root node of the tree. So what we're really trying to accomplish here is get the root node of the tree. And so that means this stack is a stack of node objects. All right. And so what we're going to do is we're going to be pushing and popping nodes. Other than that the algorithm is pretty much the same. So we go through token by token, the first token we get is the three. So this time, before we push on to the stack, we instantiate a new node inside that encapsulates the three. So that's going to be an expression node, that corresponds to an operand, which is the three and then we push that node on to the stack of nodes. Then we advanced to the next token. Well, we're still on an operand. So we encapsulate it in a node just like we did before. And we push that node on to the stack of nodes. The stack currently has a size of two, we move on. We get to the plus. Now, if you recall the idea here Once you pop the stack twice to get the two operands of the Plus, you apply the operand and push the result back on. While it's the same thing you do here. But the difference is what it means to apply the operand. So the first thing we do is we take that plus, and we encapsulate it in an expression node. But this is now an operator. And what do we know about the children of

05:25

operator nodes and expression trees, anybody? Sorry, operators. So they're not necessarily operands, we just have to have children there that correspond to the things that are its operands. I say they're not necessarily operands, because they could be whole expression trees that evaluate to something. So we need to get its children first. And so we get it by popping the stack. So we pop the stack twice, just like we did before. The first thing that comes off is the right hand operand of this plus, and so instead of adding them, we hang it off. And so we set this new nodes, right child equal to the thing that popped off. But we've got to fill both children, right, we're trying to build the tree, we got to fill both children. So we're popping the stack twice. And that gives me the left hand operand. So right now before I do anything with this node that I just created with two children, what is the size of this stack, right now? It's zero, I popped everything off. And then if you recall, the algorithm said, then push the result back on, so you're pushing the plus back on to the stack. So the plus node goes back on to the stack. But that plus node happens to be dragging the three and five along for the ride. Now there's a reason why I'm making those

Addendum--Stack Algorithm for Expression P Targeee 2C oref a4tion Transcribed by <https://otter.ai>

arrows kind of dangle outside of the stack. Because the three in the five aren't part of the stack. They're just things referenced by the plus node that happens to be in the stack, at this point, what is the size of the stack, it's just one. And now we continue on with our algorithm, we get the two, well, we just create a note around that and push it on. The stack now has two items on it, we move on, we get to the one, we make a note out of it, we push that note on to the stack, the stack now has three items on it. I'm gonna make this a little bit bigger.

08:06

We move on.

08:09

Now we get the minus, we have to pop the stack twice to get its children. So we pop the

stack, the one comes off, that becomes the right operand. Pop the stack again the two comes off, that becomes the left operand. And then we push the result back on to the stack. So the minus node comes back onto the stack. And it just so happens that the two and the water hanging off of it. But the only node that actually just got pushed on is the minus. So what's the size of the stack now to let me redraw the stack just for clarity sake. The stack has two nodes, the plus and the minus on it in that order. Right you push the operator Yep, that's exactly right. And they have the other parts of the expression tree that had been built so far hanging off of them. We keep going. We get to the times. All right, well, that's an operator. So we're going to pop the stack twice, the first pop that I get is the minus node that will become the right child of the times. And that brings along the two and the one for the ride. I need the left operand for the time. So I popped the stack again. The plus comes off bringing along the three and the five for the ride. The stack is now empty, and we got to push this times back onto that all thing. So that's the only note on the stack. But it happens to have a bunch of stuff hanging off of it to the size of the stack is one button Everything else is hanging off of it. Oops. We keep going. We run out of tokens. And just like we had for the other version of this algorithm, you pop the stack one more time. The Times comes off, that becomes the root of our expression tree. And we've built the tree.

10:26

All right.

Addendum--Stack Algorithm for Expression P Targeee 3C oref a4tion Transcribed by <https://otter.ai>

10:29

Do we have any questions on this? How do you tell between operators and operands? Um, well, I guess we know what characters are meant to correspond to operators, like the plus minus times and divide. And so as you're scanning through your list of tokens, you look at the character that comes off. And if it's a plus, minus times divide, you know, you have an operator, you just compare against the character. And it No, it actually does have to be full. Because we only have binary operands binary operators, and full binary trees are trees whose nodes either have two children or zero children. Okay. All right. So someone said, so that stuff hanging off of it is embedded in the operators, essentially, yes. They're being referenced by the operators. I agree with that statement. So the rule for pushing only operands doesn't apply here. In some sense, no, you can, you are pushing nodes that contain operators, but only operators that have all of their children set, and then will become once the whole thing's evaluated and operand for another operator. May Is that okay? Um, but how do we make use of this tree? Well, once it's been constructed, then you can invoke methods like a vow, or postfix, or prefix or infix, the ones that we've actually, we've been diagramming on pre existing trees. That's how we deal with that. And since you have the root node, you can traverse through it. Yes, that's exactly right. All right. Is there a way to check the memory use for each operator in the stack? Not really, you just have a memory reference to where that operator node is. That's really all you need. Whatever is read off first is always on the left of the tree on the top and then looking at the stack at the right. So whatever pops off first goes to the right. But since you're reversing the order, whatever shows up first in the input string, would be the left hand operand, but that gets reversed in the order in the stack. it ever gets so big, it can't handle it. Yes, but you'd need a pretty big expression for it to reach that point. Okay, so let's see what time it is. It's 635 do we set using Yes, absolutely. That's what you're going to be doing.

13:31

Okay.

Addendum--Stack Algorithm for Expression P Targeee 4C oref a4tion Transcribed by <https://otter.ai>

Addendum--AVL Tree Code

Wed, 8/11 12:41PM 44:31

SUMMARY KEYWORDS

height, rotation, node, insert, left, balance, return, subtree, imbalance, binary search tree, dot, root, tree, questions, method, route, code, case, children, set

00:00

We're looking at it. And this is why it's code. So this is based very much on the same code that we had gone through both in Monday's pre record and the lecture from last Wednesday on the binary search tree. So a lot of things are pretty much the same. And then we're augmenting it to add in these rotations. Okay. So we start out pretty much the same as we had before, we have our list of methods that we're going to need. Our constructor is just initializing the route to be no, we're generic in exactly the same way we were generic before. That is we have a generic that extends comparable on either itself or one of its super classes. So that's the same as what we had done for binary search trees. And then what we're going to do now is we're going to just take a quick look at our nodes for this. So we might as well take a quick look at our sorry, let's take a quick look at our instance variables, we have a route, just like we did before. And then we have our abl node static nested class. And this one is a little different, in that we augment it with the height. So we have the element just like we normally did. We have our left and right children, and we have our height integer. Now our constructor we have our just our simple constructor that takes in the element with no left and right children, which then calls the three arg version that takes the elements in the two children initializes element left and right, and sets the height equal to zero. Now this constructor actually is one of those places, and there are a few places where I have a little quibbles with the weisse code. Can anybody suggest a mild issue with one of the design choices here in our in our avielle? node constructor? There's something about this that annoys me anybody get annoyed by something in it to anything you can see that might be a little off. So doesn't set height to zero? Well, the top constructor invokes the three our constructor, which does set height to zero. So the fact that the height is zero, is actually the thing that gets me. Because if you pass in, no, no, actually no. What note what type of node has a height of negative one?

Addendum--AVL Tree Code Page 1 of 13 Transcribed by <https://otter.ai>

03:09

day right?

03:11

Unknown node. So once you're instantiating, one of these, it's no longer height negative one, it actually exists. Right? If it has children, it's not height one. If it has children, its height would be defined by its children's height, right? Because you're going down to the deepest leaf. So in fact, setting the height to be zero by default, when you are allowing something non notice children is kind of broken. So this only makes sense. This only works if the left and right children are no, because if the left and right children are no, it's height should in fact be zero. Is everybody okay with that one little quibble I have. And in fact, that ends up not being a problem, because we only ever instantiate nodes initially is routes. Honestly, I would not actually bother having a constructor that takes in left and right children. Because in our binary search tree construction, we only ever create nodes that are starting out as leaves. So there doesn't seem to be any reason why we would need to be able to set these ltn arties through the constructor. And so if you didn't have that as an option, and just always had those go to know that Yeah, always have those arcs go to No, then starting the height at zero actually makes sense. Are we good with that? Cool. All right, but it ends up not being a problem because we only ever call this with those four left and right. So our height is baked into this whole thing. Now

05:04

So

05:05

there are a couple of different permission levels for variables, there's public, private and protected. And if you don't specify anything in front of them, it's something called the default level, which for our purposes is very similar to public. So you end up not really meeting it by reverting to the default does that okay?

05:35

All right, cool.

Addendum--AVL Tree Code Page 2 of 13 Transcribed by <https://otter.ai>

05:37

So,

05:37

we have our setup. Sorry, I just need to flip my phone over, because someone is texting me madly, and I need to stop seeing the messages come up. Um, okay, so we, we have that. And so now, one thing I want to note, as we go through our code, we're going to want to be able to query the heights of a nodes, children, right. So sometimes I will want to see what the heights of my children are to make sure that the avielle condition is the L. And so your first impulse is probably going to be all okay, if I have a node, and I want to see what the height of its left subtree is, I'll say that node is x, I'd say just x dot left dot height. And that would give me the height. Can anyone see a potential problem with extracting the height of a subtree? By saying something like x dot left dot height? Where could that go horribly wrong? Anybody?

06:54

Well, actually, we might want to be able to change the height values, because we're still sort of internal to the structure, that's okay. The outside world isn't ever gonna actually have access to this stuff anyway, since we never return node objects to them. So the fact that it's allowed to be changed is not a problem. In fact, that's a feature, not a bug. But what specifically is dangerous about saying something like x dot left dot height? So it's not so much that the subtree is imbalanced? We want to assume we assume that it's not because we sort of check these things from the bottom up. But the next comment is if it's a null node, and if we haven't, won't have instantiated the height. And in fact, it's slightly worse than that. If I give you a no reference, if x is no, and I say x dot height, I'll get no pointer exception, which is a runtime exception, and the whole thing bottoms out. And so if x is not, no, but it's left subtree is no which is a valid scenario, I really want to see the height of the left subtree is negative one, but x dot left would be no. So if I say x dot left dot height, I get a null pointer exception, because I'd be trying to access a field of a null reference. So in reality, we're not going to be directly accessing the height variable. Instead, we're going to have a helper method called height. Let me find it.

08:31

Find it.

Addendum--AVL Tree Code Page 3 of 13 Transcribed by <https://otter.ai>

08:35

Eventually, I'll find it. There it is. So we're going to have a height function, which is going to take in as a parameter a node. So whenever we want to know what a nodes height is, and staying setting, instead of saying node dot height, we'll call height method on that node. And by calling height the method on that node that allows us to do something like this. Now, it's leveraging something called the ternary operator, which I discussed in the pre recorded lectures. So if you didn't watch that yet, well watch it. But as you know, or should know, the ternary operator basically is an baked in if else, it's question mark colon. So if t is equal to No, then the whole expression evaluates to the thing here between the question mark and the colon. So if t is no, this whole expression evaluates to minus one,

and it returns minus one, which is what we want as the height of a null subtree. If t is not, no, then the other condition the thing that comes after the colon is what it evaluates to, which is $T \cdot \text{height}$, which is safe once we know that t is not null. So we have this little extra helper function helper method that takes in the node and then if it is null returns minus one if it isn't, it returns a type two field. So it actually works in all those cases. So it's probably worth it to mention that now, because we're gonna have to be using that as we go. So do we have any questions on that? All right. So let's make our way back up to the beginning of this thing. And, you know, just like we did in the binary search tree, we are going to have public and private insert contains whatever methods because the public ones just invoke the private ones with the appropriate parameters, because the recursive ones need to take in nodes. Someone said for implementation. Why is minus one more useful to know given they both mean the node has no children? Because remember that the avielle condition is asking what is the difference between the left and right subtree. So it's just generally easier if we could just subtract the two and see what the result is. And so to do that, it requires an end like you said, and so having minus one be the height of something that's no works in place, it works as the end, and it works out in terms of those calculations. We're good. All right. And so we have our public and private versions our contains is completely the same, we don't need to even bother looking at it. And the same thing for find min and find max. Because the only time the rotations matter is the only time we have to worry about maintaining the balance conditions, is when we've actually done something to change the tree. It's only when we change the tree could the tree become unbalanced. And only when it become unbalanced? Do we need to worry about the rotations. So the only methods that end up having any modifications at all to them are the insert and remove methods, because those are the methods that actually modify the truth. So for the purposes of this discussion, we're really just going to worry about the insert method, because it's fairly clean. When we actually see how the balance is implemented, it'll be immediately obvious how we can kind of inject that into the Remove method, but we're going to focus in on the Insert. Now just like normal, we start with our public insert method, our public insert method, then invokes the private insert method

Addendum--AVL Tree Code Page 4 of 13 Transcribed by <https://otter.ai>

that takes in both the value x that we're trying to insert, and the root of the subtree that we are currently trying to insert it into. And initially, we're currently trying to insert it into the root of the entire tree. Now, as was discussed extensively in the pre recorded lecture, being certain method needs to return the new

13:11

root

13:12

of the subtree that results from the insert. When I insert x into the original sub tree, the root of that sub tree might have changed. Now in a plain old binary search tree, the only time the route changes is if that sub tree was originally No, in which case it gets replaced by the new node. In this case, the root of the sub tree could change in a number of scenarios. One scenario is if the root of the sub tree was in fact null, in which case we're just replacing it with a new node. But the other scenario is that maybe we caused an imbalance at the root. And therefore there was a rotation that occurred which changes the root to make the thing balanced. So insert returns the new root. So we have to set roots equal to the result of inserting x comma root. Now if the root isn't supposed to change, then insert will just return back the original route, but it has to return it so that we can get this assignment set up. Okay. All right. So that's our public method. Now we go down to our private one. So our private insert is here. And as we said, it takes as parameters the value we're inserting, and the root of the subtree that we are currently inserting into and it returns the new roots of that subtree. What is it going to be replaced with which might be itself it might be something new. Now as we discussed inserts over

Start out, like contains checks. This is a set that we are representing. So each node needs to be unique. Okay. And so if each node is unique, then if we find it, there's nothing to do. However, if we find where it should be in that spot is no, we replace it with a new node. So if `t` is no, it means we have reached a no sub tree, and we need to place `x`

15:31

at that spot.

15:33

So we instantiate a new avielle node with the value `x`, no left and right subtrees. We know

Addendum--AVL Tree Code Page 5 of 13 Transcribed by <https://otter.ai>

that constructor defaults to having a height of zero, which is correct for a leaf. And so that constructs the new node around `x` and then returns it. As you saw in the public version of this, we set the root of the whole thing equal to whatever this thing returns. So if the thing was no, then we return the new node, and that overwrites the root, and now it's part of the tree. So that's the simple case. Now, if `t` is not No, we continue our compare. Alright, we continue our search. So we capture the results of comparing `X` to `T` dot element. And so compare result will be less than zero if `x` is smaller zero if they're the same great negative compare result will be positive if `x` is larger than `T` dot element. And so here we go. If compare results is less than zero, that means it needs to be inserted into the left subtree. So we set `T` dot left equal to the result of inserting `x` into `T` dot left. So we make our recursive call, it returns the new root of `T` dot left, and we set `T` dot left into that, okay, else, if compare result is greater than zero, that means we need to put it in `T` dot, right. So we set `T` dot right equal to insert `X` on `T` dot, right? same logic, we're just doing it on the right subtree instead of the left subtree. The else case happens if compare result is zero, meaning if `x` is equal to `T` dot element, and in that case, it means that we found it in the tree already. And because we're set, we're just going to leave it there, alright, there's nothing, there's nothing to do, it's already there, the tree doesn't need to change. So at this point, after this, if else if else block of code executes, `T` dot left or `T` dot right have been updated with the correctly inserted values. That means that unless we've discovered that there was a duplicate and the whole thing, `T` dot left, or `T` dot rights heights may have changed. And if `T` dot left or `T` dot, right, its heights may have changed, `t` itself may be imbalanced. Now, if we weren't caring about balance, the last line of this code would have just been to return `T`, because we have to return the root of this tree that we just inserted into, which is `t`, which may have had some stuff below it modified. But since we're not on this case, it's the original `t`. So the original insert before we convert it to abls, just return `T` and that was it. But now we're at a scenario where team may have an imbalance in it. And so it's easier to sort of functionally decompose all of these balance checks, and ultimately the rotations that we might need to perform in two separate methods. So we have a separate balanced method that takes in a node, it'll check the heights of the left and right sub trees of that node. And if the heights of those sub trees differ by more than one, it'll perform whatever rotation is appropriate being a single or double rotation will pick one of the four cases and do it. And then it will return the balance method itself will return the new route that occurs after performing the rotation about `T`. And so calling balance on `T` is a constant time thing that does one of those four cases of rotations, returns the new root and then we return whatever balance returns back to the previous caller. And so at every level of this recursion before we return to to the previous level, we invoke this balance method, which will check to make sure the balance is okay. And if not, we'll locally fix it. But since balance is a constant time operation, it doesn't add any big O cost to this whole life. operation.

Addendum--AVL Tree Code Page 6 of 13 Transcribed by <https://otter.ai>

20:02

So before I dig into how balanced itself is implemented, what kind of questions do we have? Is the return calling balance `T`, even if it did nothing? Is that a waste? So no, it's not a

waste, because what balance does is it returns the root of the subtree T, once I've confirmed that it's balanced, so balance won't return nothing. If it's already bounced, it will just re return T. But that's not a waste, because before returning tea, it actually had to do the check to confirm that it was balanced. So the purpose of calling this thing is to at the very least confirm that it's balanced, before passing it back to the previous level. Does that make sense? Okay, other questions on this, before I dig into balance? Is there any possible case in which a balance cannot happen? No, this will always be balanced. As long as we're maintaining the balance, as we go, as long as every single insert from nothing on up, always checks the balance, you'll always be able to balance it.

21:23

Okay.

21:28

Other questions? All right.

21:35

So, let's go to balance. And by the way, remove, same thing. If you remember the Remove code from the pre recorded lecture, the very last line of remove is to return t return the root of the tree after the Remove is occurred. And so as long as we call balance on T before we return it, same thing. So removing insert end up being modified in the same way, just before we do that final return, we trigger balance on it. So here's the balance. Okay. So first things first, the goal of this is to return the roots of the subtree that's initially returned, initially routed at T, return the root of that sub tree after the balance has been fixed. So if there's no need to rotate anything, we just return t right back to the user. Now as a quick note, we have this public static final int allowed imbalance set equal to one, that's just a hard coded constant that tells us that Yeah, the difference between the left and right subtrees can be no more than one, it's generally considered bad practice to have magic numbers. So it's better to sort of set it as a constant and then just use a loud unbalance rather than one in our comparisons. Are we ever going to change that from one? No, but this is just good practice. Alright, so first things first, if key itself is no, meaning we're trying to balance an empty subtree, there's nothing to be done. So we just

Addendum--AVL Tree Code Page 7 of 13 Transcribed by <https://otter.ai>

returned to the back, if the original route was normal, the new bounce route is not. And now we have to check to see which of if any of the cases holds are we case one, case two, case three, or case four.

23:41

So

23:43

we have this if statement here. And what this if statement does, it calls that height function that we already discussed? It calls that height function on T dot left. And then it calls the height function on T dot right and it takes the difference of the two. So we're doing height of T dot left minus the height of T dot, right? If that's greater than allowed imbalance. What does that mean? What does that tell us? It actually tells us two things. Anybody?

24:24

If that's true,

24:25

what two things does it tell us? Okay, so it tells us that the left is too big, and that the left has a height that's greater than the right. So the two things are a, it tells us that there is some kind of imbalance and that that particular imbalance. So it's not Zig Zig. It's just the first thing. It tells us that yes, it's imbalanced, but it's imbalanced to the left. So That means that if this is true, we know that we've narrowed it down to either case one or case two, we know that a, it's imbalanced and B, we took a step to the left. And that's because

we're not doing the absolute value here. If there was an imbalance, but T dot write was much bigger, this just would have been very negative, and this case would not have caught. So we know that if this is true, there is an imbalance. And the imbalance takes the first step to the left. So we're in one of the zig cases, but it doesn't yet tell us whether or not we've gone left twice, or we've gone left, then we've gone right. Okay, so that's our first check. And now inside of there, this if else tells me whether or not the second step was Zig or zag. So we know that the left is deeper than the right. So now what we do is we take the take a look at the height of two steps to the left, and see if that's greater than the height of left than right. So we look one step down to the left, and then compare the left

Addendum--AVL Tree Code Page 8 of 13 Transcribed by <https://otter.ai>

and right children there. If the left side, one step down, is in fact, deeper than the right side, then we know that we actually have a zigzag case, and therefore we do a single rotation with the left child, that's the case one rotation. If that's not true, then that means it's actually the right side. That's deeper. So that means our first step is to the left. But now, our second step is to the right. At that point, we trigger the double rotation with the left child. And that's the zigzag case, too. So do we have any questions on what this chunk of code is doing? So it's triggering methods that cause the rotations, but it doesn't actually do the rotations just yet. It's trying to figure out which case we're in. Questions on that.

27:21

Okay, now note, when we call rotate with left child, what's actually happening is we're setting t equal to the result of calling rotate with left child on t, which actually starts to look very much like the pseudocode I had jotted down for you before it returns rotate with left returns the new roof after performing that rotation, and changes t to be that same thing for the double rotation. So if the left side isn't the side calling the imbalance, then it does the reverse subtraction. It says, All right, height T dot writes minus height T dot left, is that greater than allowed imbalance? And if that's true, it means the first step is zag. And then we compare Right, right, with right left if right, right is larger, that means we went to the write twice. And we do a single rotation to our right, so the highlighted one here is case four. And if not, then that means right left is deeper. So it's the case three Zagazig alright. So, the outer if else determines whether or not the first step is a zig or zag, then the inner if else checks to see if the second step is Zig or zag, triggering the correct thing. If neither this if else or this if triggers, it means there is no imbalance, it means the heights of the left subtree and the height of the right subtree. Although they may have changed, because, you know, one of them might have gotten deeper than than the other one we did the insert, they didn't change enough to warrant a rotation. So it means if neither of these execute, it means we are still balanced, all right. So whether or not this happens, once we are outside of this if else block. We still have to update T's height because either a rotation happened and something new is the route or T itself might not have had an imbalance. But one of its sub trees may have grown a little bit larger if to originally before the insert had no left and right children. And then I inserted something to its left. It's not imbalanced but Height did change. So no matter what we have to update T's height, and we update T's height, by invoking the max function on the height of T dot left and the height of T dot, right, we see which branch which child is deeper. That's the branch that's going to bring us down to the furthest leaf. And then we add one to it for the step to get to that guy. So we update, we do the rotation if necessary. We update T's height, and then we return T. And that completes our balance. So we still have to look at the actual rotation code. But assuming those rotations work, does anybody have any questions on this balance code?

Addendum--AVL Tree Code Page 9 of 13 Transcribed by <https://otter.ai>

30:49

We're good.

30:53

Everybody has suddenly gotten very sound. Are we all just very tired or eating dinner at this point? Oh, dead silence. Okay. Oh, I'll deal with it and move on.

31:08

Alright.

31:12

So it took a while to like, get to that point. So all that's left for us to do with the rotations. Now we know that the sorry, we're just completely wrapped. Wrapped is not, you know, honestly, I spent four hours in various zoom meetings in the time leading up to this class. So I kind of know where you are in terms of all of this. And my meetings did not have musical interludes in them. So anyway, hold on a second. Now I'm suddenly zoned out. That's no good. Alright, let's go to the rotations. Okay, so we're not going to bother with the case three. And case four rotations, I believe that as an exercise for you to read through them. What we're going to look at now is the case one rotation rotate with left child. Alright. And so we had already kind of sketched out what that looked like. And what we're going to realize is that this ends up looking pretty much the same, all right, you pass in No. So in this notation, they've actually swapped k one and K two from what I drew on the paper, but rotate with left child takes in the root node of the sub tree that needs a left rotation, which happens to be called k two, and then returns the new root. The first step, we set k one equal to k two dot left, all that's doing is it's giving k two dot left the name. All right. And then these two steps are exactly the same two steps that I drew out for you on the paper. Hate to dot left, all right, is going to be the new route. So we're breaking its rights link. All right. We're breaking k two is left blank. And we're setting it equal to k one dot right. And then, oh, God, I said this all wrong. Forget I said any of that. Roll back, roll back, roll back. Okay. k two is the root of the tree. k one is its left child. So we're rotating k two and K one. So k one is ultimately going to become the root of the whole thing. And that's confirmed by the fact that we're returning k one at the end. So because k one, which is to the left of K two, because k one is coming up, k two dot left is that link from the top that got broken. So we'd set K to dot left equal to k one dot rights, that's the thing that's going to get orphaned. And then we set k one dot right equal to k two. And that's it. Those Addendum--AVL Tree Code Page 10 of 13 Transcribed by <https://otter.ai> are the two steps. Then we just have to update k one and K twos heights because those have swapped around a little bit. And this is exactly the same calculation that we had before where we just look at the left and right subtrees take the max and add one Then k one becomes the root of the whole thing. We have any questions on this?

35:12

All right.

35:15

The Case for is just a mirror of that. So we don't have to worry about that. And then case two, which is where we want to go first left then right? Well, in that case, it's just a pair of rotations. If the root of the whole thing is case three, we take a step down first to the left. And that thing to its left, we have to rotate with that things right child. So k three dot left gets rotated with its right child. And that converts it into case one. And then we just call rotate with left and solve the case one. So this is really trivial. When I drew it out on paper, you saw I did two single rotations, one single rotation a level down, which converted it to K one and one rotation at the root, which solved case one. And so since those are already existing methods, we just need to invoke them. We don't need to do anything else. Okay. And so that's it. Does anybody have any questions on the rotation?

36:40

Okay, so. So the second rotation is seen in the line with the return? Yes. This does the second rotation. And whatever the route is, after that rotation is done, is what the double rotation returns. Good. The insert code Oh, for sure.

37:14

Go ahead.

37:16

Yeah, so my question was When, when, when does balance get triggered? You don't get more than once. So balance gets triggered here at the return point. Okay. So if I'm so if the insert method, once it gets down to a no spot and creates the new node balance doesn't get triggered their rights, but when you create a new node at the very bottom, Addendum--AVL Tree Code Page 11 of 13 Transcribed by <https://otter.ai> that's already balanced by definition, because it has to know children, right? Yeah. Is that okay? Yeah. Right. And then, yeah, go ahead. The thing to consider is that remember, there are multiple calls to insert open at a given time. Hold on a second, I want to actually switch the camera. I want to get this so that you can see both my paper and the screen at the same time. So give me 30 seconds there.

38:26

Can people see the paper up in the corner now? Okay, so imagine we have the following tree to start with.

38:41

Okay, and so now imagine. Now, imagine we were inserting a four. Okay. Yep. So in the initial call to insert for x would be four, and C would be the root of the tree here. Agreed? Yes. Okay. So now, while T is not null, so it didn't hit this case, we do a comparison between the four that we're trying to insert in the data at t so we compare four and three. And so that means compare results will be a positive number because four is larger than three. So this case, will trigger a grade. I'm with you. Alright, so now what it's going to do is it's going to say T dot right? Equals insert X on T dot right. So then a new call to insert opens up. Where this is the new T. Agree. Yeah. And that's so that's, this is The new t so we're inserting four into this T. T is not No, we compare the four to the element at t, which tells me I should go left a grid, a grid. And then we do this compare results. And we realize, okay, we're in this case. So I'm setting T dot left equal to the result of inserting X on T dot left, Agreed? Agreed. So now a new call to insert will trigger. where t is no, because triggers the first f statement. So it triggers this statement, and it just returns the new node. So now the tree looks like this. Agreed? Yes. Okay. So that returns. And it brings me back to the immediately preceding insert, where I was inserting on five. So now I'm back to the call to insert where T was five. And I'm on this line. Agreed? Yes. So now this line finishes, and then it moves down to its return statement. So now balance is called on T, where this is T. So it checks to see whether or not this node is balanced. Agreed? Yes, that node is balanced. So in that balanced code, all it does is it updates the height of this node, because the height of the snow did change, correct? Yep. And then returns this node back. So this node then gets returned back to the original call to insert where T was item three. Which brings me to this line because I was inserting to the right, yeah, so now I just finished this line, where t is item three, and then it jumps to the balance code. Right? Right. At this point, balance determines, oh, crap, there's an imbalance. So it does the, the check realizes it's a zigzag and invokes the double rotation method on this tee. I lost you there somehow. No, I'm with you. Okay, good. So now the call to balance on t where t was Addendum--AVL Tree Code Page 12 of 13 Transcribed by <https://otter.ai> originally three will do the rotation. The net result of that will be it will return this node after doing the rotations. So this call to balance the original call to balance from the original insert will now actually return a new node four, which will then get returned back to the public one and route will get reset to the four. Thank you that really helped to go through it again. Okay, so we're good with that. We're good. Oh, so if the tree was already balanced, and nothing changed, it would return the three. Talia Yes, that's absolutely right. Okay. All right. I feel like I missed a question. ballon, can you please draw how the tree would originally look like with K notation and the double rotation, and K three is the

original root.

43:39

So this situation would look something like that. Is that okay? Well, not you I'm talking to someone in the class. Nope, wasn't saying Michelle. Sorry about that. Yes, no. I'm not sure if I answered that question or not. I need a little feedback. Oh, yeah. Sorry. I see it. Cool. All right. Other questions?

44:09

Okay, so I'm

44:13

done. That's really the only thing in that code. That's important in the sense of everything else in that code is the same effectively as in the binary search tree case. So I think we're good in that respect.

Addendum--AVL Tree Code Page 13 of 13 Transcribed by <https://otter.ai>